The C Programming Handbook

From the Language Itself to What Industry Actually Expects

Part 1 — The C Language (Modules 1–18)

Part 2 — What Industry Actually Expects (Modules 19–31)

Expanded edition

Contents

Preface & Roadmap 3
PART 1 — The C Language 12
Module 1 — Foundations 13
Module 2 — Types & Values 32
Module 3 — Variables, Scope, Lifetime 57
Module 4 — Operators & Expressions 75
Module 5 — Control Flow 95
Module 6 — Functions 112
Module 7 — Arrays 131
Module 8 — Strings 146
Module 9 — Pointers (the Core Skill) 163
Module 10 — Memory Management 179
Module 11 — Structs, Unions, Enums 196
Module 12 — The Preprocessor 214
Module 13 — Input / Output 231
Module 14 — The Standard Library Tour 248
Module 15 — Error Handling Strategies 265
Module 16 — Undefined, Unspecified, and Implementation-Defined Behavior 279
Module 17 — Concurrency in Standard C 293
Module 18 — C23 Highlights 306
PART 2 — What Industry Actually Expects (Beyond the Language) 323
Module 19 — Toolchain & Compilation 324
Module 20 — Build Systems & Libraries 340
Module 21 — Debugging & Analysis 356
Module 22 — Testing & Quality 373
Module 23 — Secure Coding 388
Module 24 — Systems / POSIX Programming 405
Module 25 — Performance Engineering 428
Module 26 — Portability 444
Module 27 — Embedded / Bare-Metal C 458
Module 28 — Software Design in C 479
Module 29 — Data Structures & Algorithms in C 495
Module 30 — Interoperability 511
Module 31 — Professional Practice 525

Preface & Roadmap

Preface

This book is organised around a single conviction: knowing the syntax of C is the easy part. The hard part — the part that separates someone who has “done some C” from someone a team can trust with production firmware, a database engine, or a network daemon — is understanding *why* the language behaves the way it does, what the compiler and linker are really doing on your behalf, and where the sharp edges are hidden.

The material is divided into two parts. **Part 1** covers the C language proper: its types, its operators, its memory model, its standard library, and the uncomfortable but essential topic of undefined behaviour. **Part 2** covers everything that surrounds the language in real professional work: toolchains, build systems, debuggers, sanitizers, testing, secure coding, systems programming, performance, portability, embedded development, software design, interoperability, and day-to-day engineering practice.

Every module ends with practice exercises. Do them. C is learned at the keyboard, by compiling, running, breaking, and fixing programs — not by reading about them. Where this book shows program output, that output came from real compilation and execution rather than from memory, so you can reproduce it and compare.

The Roadmap

What follows is the full map of the territory this book covers. Treat it as both a table of contents and a checklist: by the time you can explain every line of it with confidence, you are working at a professional level.

PART 1 — The C Language

1. Foundations

- History, K&R C, and why C exists
- Standards: C89/C90, C95, C99, C11, C17, C23 — and knowing which one your compiler uses
- Translation phases: preprocess → compile → assemble → link
- Translation units, object files, executables
- main() signatures, argc/argv, exit codes, envp

2. Types & Values

- Integer types, char (signedness is implementation-defined), short/int/long/long long

- Floating point: float, double, long double, IEEE-754 basics, NaN/Inf, epsilon comparison
- _Bool/bool, void
- Fixed-width types: stdint.h (int32_t, uint8_t, intptr_t, uintmax_t)
- size_t, ptrdiff_t, ssize_t, char16_t/char32_t, wchar_t
- limits.h, float.h, inttypes.h format macros (PRIu64)
- Integer promotion, usual arithmetic conversions, implicit conversion traps
- Signed overflow (UB) vs unsigned wraparound (defined)
- Type qualifiers: const, volatile, restrict, _Atomic
- Casting, type punning, strict aliasing rules

3. Variables, Scope, Lifetime

- Declaration vs definition
- Scope: block, file, function, function-prototype
- Linkage: external, internal (static), none
- Storage duration: automatic, static, thread (_Thread_local), allocated
- extern, static, register, auto
- Initialization rules, zero-initialization of statics, indeterminate values

4. Operators & Expressions

- Arithmetic, relational, logical, bitwise, assignment, compound assignment
- Ternary, comma operator
- sizeof, _Alignof, _Alignas
- Precedence and associativity
- Short-circuit evaluation
- Sequence points / evaluation order, and the UB it causes (i = i++ + 1)
- Bit manipulation: masks, shifts, set/clear/toggle/test, bit tricks, bitfields vs manual bits

5. Control Flow

- if/else, switch (fallthrough, jump tables, Duff’s device)
- for, while, do-while, nested loops
- break, continue, goto (and its legitimate use: cleanup chains)
- Labels, unreachable code

6. Functions

- Prototypes, definitions, forward declarations
- Pass-by-value semantics (C has no pass-by-reference)
- Recursion, tail calls, stack depth limits
- Variadic functions: stdarg.h, va_list, va_start/arg/end/copy
- inline, static inline, _Noreturn
- Function pointers, callbacks, dispatch tables
- Calling conventions (cdecl, stdcall, fastcall), ABI basics

7. Arrays

- 1D, 2D, multidimensional, row-major layout
- Array-to-pointer decay (and where it doesn't happen)
- Array parameters in functions (they're pointers)
- Variable Length Arrays (VLA) — and why many shops ban them
- Designated initializers, compound literals
- Flexible array members (struct { int n; char data[]; })

8. Strings

- Null-terminated convention, '\0'
- string.h: strlen, strcpy, strncpy, strcat, strcmp, strchr, strstr, strtok, memcpy, memmove, memset, memcmp
- Why strcpy/strcat/sprintf/gets are dangerous; snprintf, strlcpy/strlcat, strnlen
- memcpy vs memmove (overlap)
- Conversions: atoi (bad) vs strtol/strtod (good, with errno + endptr)
- Wide strings, multibyte, UTF-8 handling in C
- Building a string/buffer abstraction

9. Pointers (the core skill)

- Address-of, dereference, pointer declaration syntax
- Pointer arithmetic and its rules
- void* and generic programming
- NULL, null checks, nullptr (C23)
- Pointer to pointer, arrays of pointers, pointer to array
- const correctness: const char * vs char * const vs const char * const
- Function pointers, pointer to function returning pointer, typedef for readability
- Dangling pointers, wild pointers, uninitialized pointers
- Alignment, uintptr_t, pointer-integer conversion
- Reading complex declarations (spiral rule / cdecl)

10. Memory Management

- Memory layout: text, rodata, data, BSS, heap, stack
- Stack frames, stack overflow, recursion limits
- malloc, calloc, realloc, free, aligned_alloc
- Checking allocation failure; realloc failure pattern
- Leaks, double free, use-after-free, buffer overflow, heap corruption
- Ownership conventions and who-frees-what discipline
- Custom allocators: arena/bump, pool, freelist, slab, stack allocator
- alloca (and why to avoid it)
- Zeroing sensitive memory (memset_s, explicit_bzero)

11. Structs, Unions, Enums

- Struct declaration, initialization, nesting, assignment/copy semantics
- Passing structs by value vs pointer

- Padding, alignment, offsetof, struct packing, cache-friendly layout
- Bitfields and their portability caveats
- Unions, type punning via union, tagged unions
- Enums, enum scoping, C23 fixed underlying type
- typedef — good and bad practice
- Opaque structs (typedef struct Foo Foo;) for encapsulation
- Self-referential structs, intrusive linked structures

12. The Preprocessor

- #include (“” vs <>), include guards, #pragma once
- Object-like and function-like macros
- Macro pitfalls: multiple evaluation, missing parens, semicolons; do { } while(0)
- Stringification #, token pasting ##, variadic macros **VA_ARGS**
- Conditional compilation: #if/#ifdef/#elif/#else/#endif, #error, #warning
- Predefined macros: **FILE**, **LINE**, **func**, **DATE**, **STDC_VERSION**
- X-Macros, code generation patterns
- _Generic (type-generic dispatch, C11)
- #embed (C23)
- When not to use macros

13. Input / Output

- stdio.h: printf/scanf family, full format specifier knowledge, width/precision/flags
- Why scanf is a trap; fgets + strtol pattern
- FILE*, fopen modes, text vs binary mode
- fread, fwrite, fseek, ftell, rewind, fflush
- Buffering: full/line/none, setvbuf
- EOF, feof misuse, ferror, clearerr
- stdin/stdout/stderr, redirection, pipes
- Low-level POSIX I/O: open/read/write/close/lseek, file descriptors, mmap

14. Standard Library Tour

stdio · stdlib · string · math · ctype · time · assert · errno · signal · setjmp · locale · stdbool · stddef · stdint · inttypes · limits · float · stdarg · stdalign · stdnoreturn · complex · tgmath · threads.h · stdatomic.h · uchar.h

- Sorting/searching: qsort, bsearch (and writing correct comparators)
- Randomness: rand limitations, better PRNGs
- Time: time_t, struct tm, clock, monotonic clocks

15. Error Handling Strategies

- Return codes, sentinel values, out-parameters
- errno, perror, strerror, thread-safety of errno
- assert vs runtime checks; NDEBUG
- The goto cleanup / single-exit pattern
- setjmp/longjmp (and its dangers)

- Designing an error enum + error-string API

16. Undefined, Unspecified, Implementation-Defined Behavior

- The distinction between the three
- Common UB list: signed overflow, OOB access, null deref, uninitialized reads, strict aliasing violations, data races, invalid free, shifting by ≥ width, modifying string literals, misaligned access
- How optimizers exploit UB (why “it worked on my machine” fails)

17. Concurrency in Standard C

- C11 threads.h, mtx_t, cnd_t
- stdatomic.h, atomic types, memory_order_*
- The C memory model, data races, happens-before
- volatile ≠ thread safety (very common misunderstanding)

18. C23 Highlights

nullptr, typeof, constexpr, auto, bool as keyword, attributes ([[nodiscard]], [[deprecated]], [[fallthrough]]), binary literals, digit separators, #embed, stdbit.h, enhanced enums, memset_explicit, empty initializer {}

PART 2 — What Industry Actually Expects (beyond the language)

This is the part that separates “knows C” from “professional C developer.”

19. Toolchain & Compilation

- GCC, Clang, MSVC, ICC — differences and extensions
- Warning flags you should always run: -Wall -Wextra -Wpedantic -Werror -Wshadow -Wconversion -Wstrict-prototypes
- Optimization levels -O0..-O3 -Os -Ofast, -g, -march/-mtune
- Preprocessor/assembly output (-E, -S), reading generated assembly (Godbolt)
- Link-Time Optimization, -ffunction-sections -Wl,–gc-sections
- Cross-compilation, sysroots, toolchain files
- objdump, nm, readelf, strip, size, ldd, addr2line

20. Build Systems & Libraries

- Make (real Makefiles, pattern rules, dependency generation)
- CMake (the industry default), Meson, Ninja, Bazel; autotools legacy
- Static libraries .a vs shared .so/.dll/.dylib
- Position-independent code, symbol visibility, -fvisibility=hidden
- Symbol resolution, link order, undefined/duplicate symbols
- ABI vs API, soname, versioning, backward compatibility
- pkg-config, dependency management (Conan, vcpkg)

- Header design: what goes in .h vs .c, minimizing includes, forward declarations

21. Debugging & Analysis

- GDB/LLDB: breakpoints, watchpoints, backtraces, inspecting memory, TUI, scripting
- Core dumps and post-mortem debugging
- Valgrind (memcheck, callgrind, helgrind, massif)
- Sanitizers: ASan, UBSan, TSan, MSan, LeakSanitizer — non-negotiable in modern C
- strace, ltrace, perf, ftrace, bpftrace
- rr (record & replay debugging)
- Static analysis: clang-tidy, cppcheck, scan-build, Coverity, PVS-Studio
- Formal-ish tools: Frama-C, CBMC

22. Testing & Quality

- Unit test frameworks: Unity, CMocka, Check, Criterion, greatest, µnit
- Test doubles/mocks in C (link-time seams, function pointers)
- Code coverage: gcov/lcov, branch coverage
- Fuzzing: libFuzzer, AFL++, honggfuzz — huge in real C work
- Property-based and differential testing
- Regression suites, golden tests
- CI/CD: GitHub Actions, GitLab CI, building on multiple compilers/platforms

23. Secure Coding

- Buffer overflows (stack & heap), off-by-one, strcpy family abuse
- Format string vulnerabilities
- Integer overflow/underflow, signed/unsigned confusion, truncation
- Use-after-free, double free, type confusion
- TOCTOU races, unsafe temp files, system()/command injection
- Input validation, canonicalization
- CERT C, MISRA C:2012/2023, CWE Top 25
- Hardening: -fstack-protector-strong, _FORTIFY_SOURCE=3, PIE, RELRO, -D_GLIBCXX_ASSERTIONS, CFI
- Constant-time code for crypto, avoiding compiler-removed memset
- Threat modeling, secure API design (bounds-carrying interfaces)

24. Systems / POSIX Programming

- Syscalls vs library calls
- Processes: fork, exec*, wait, zombies, orphans, process groups
- Signals: handlers, async-signal-safety, sigaction, masks, signalfd
- IPC: pipes, FIFOs, UNIX sockets, shared memory, semaphores, message queues, mmap
- File system: paths, permissions, stat, directories, symlinks, inotify
- Networking: TCP/UDP sockets, bind/listen/accept/connect, byte order, non-blocking I/O
- Multiplexing: select, poll, epoll, kqueue, io_uring

- pthreads: create/join/detach, mutexes, condvars, rwlocks, barriers, TLS, thread pools
- Lock-free programming, ABA problem, memory barriers
- Daemons, resource limits, getopt, environment variables
- Dynamic loading: dlopen/dlsym, plugin architectures
- Windows counterparts if you target it: Win32 API, HANDLE model, IOCP

25. Performance Engineering

- Big-O in practice vs constants and memory
- CPU caches, cache lines, locality, prefetching, false sharing
- Branch prediction, branchless techniques
- Data-oriented design, SoA vs AoS
- Profiling: perf, gprof, callgrind, VTune, flamegraphs
- Benchmarking methodology (warmup, variance, avoiding compiler elision)
- SIMD: SSE/AVX/NEON intrinsics, auto-vectorization
- Inline assembly, compiler barriers
- restrict, likely/unlikely, alignment, memory pools
- Understanding what the optimizer does and doesn’t do

26. Portability

- Endianness, word size, LP64 vs LLP64
- Alignment and unaligned access on strict architectures
- Feature test macros (_POSIX_C_SOURCE, _GNU_SOURCE, _DEFAULT_SOURCE)
- Platform detection macros, compiler-specific extensions
- Freestanding vs hosted implementations
- Toolchain differences and abstraction layers

27. Embedded / Bare-Metal C (huge C job market)

- MCU architecture, memory maps, registers
- Memory-mapped I/O and correct volatile usage
- Interrupts, ISRs, vector tables, latency, reentrancy
- Linker scripts, startup code, .data/.bss init, stack setup
- No dynamic allocation / static allocation strategies
- Fixed-point arithmetic, no-FPU environments
- Peripherals & protocols: GPIO, UART, SPI, I2C, CAN, ADC, PWM, DMA, timers, watchdog
- RTOS: FreeRTOS, Zephyr, ThreadX — tasks, queues, priorities, priority inversion
- Low-power modes, boot loaders, firmware update / OTA
- Debug hardware: JTAG/SWD, OpenOCD, logic analyzers, oscilloscopes
- MISRA C compliance, safety standards (ISO 26262, DO-178C, IEC 61508)
- CMSIS, HAL vs register-level programming

28. Software Design in C

- Modularity: one module = header + implementation, clear boundaries
- Opaque handle APIs (typedef struct Db Db; Db *db_open(…))

- Polymorphism via function-pointer vtables
- Callbacks with context/user-data pointers
- Object lifecycle: init/deinit, create/destroy, ownership rules
- State machines (table-driven, switch-based)
- Plugin systems, dependency injection
- Generic containers: void*, macro templates, intrusive lists
- Layering, avoiding global state, testability
- API design: naming, versioning, deprecation, error reporting, thread-safety documentation

29. Data Structures & Algorithms in C

Dynamic array/vector · singly & doubly linked lists · intrusive lists · stacks/queues · ring buffers · hash tables (open addressing, chaining) · binary trees, BST, AVL/red-black · tries · heaps/priority queues · graphs · bitsets · string builders · memory-efficient variants · sorting/searching implementations

30. Interoperability

- extern “C” and C++ interop
- FFI: Python (ctypes, cffi, C extensions), Rust (bindgen, cbindgen), Go cgo, Java JNI, Lua, Node N-API
- WebAssembly via Emscripten/WASI
- Designing C APIs meant to be bound from other languages

31. Professional Practice

- Git: branching, rebasing, bisect (git bisect is gold for C bugs), code review
- Coding standards: Linux kernel style, GNU, Google, MISRA; clang-format, .editorconfig
- Documentation: Doxygen, README, man pages, architecture docs
- Semantic versioning, changelogs, release engineering
- Licensing: GPL/LGPL/MIT/BSD/Apache and the implications for C libraries
- Reading and contributing to large C codebases
- Debugging production issues, logging, telemetry, crash reporting

32. Study These Real Codebases

SQLite (testing discipline) · redis (clean C, data structures) · curl (portability) · musl libc (minimal, readable stdlib) · Lua (small, elegant, embeddable) · nginx (event-driven architecture) · Linux kernel (drivers/subsystems) · FFmpeg (performance) · zstd/lz4 (optimization) · stb single-header libs (API design)

Suggested Priority Order

Not every topic carries equal weight at every stage of a career. If you are preparing for your first C role, some modules are non-negotiable; others will make you stand out; still others matter mainly within a particular specialisation.

Must have before your first job: Modules 1–16, 19, 20 (Make and CMake), 21 (GDB, sanitizers, and Valgrind), the basics of 22, and the basics of 23.

Makes you strong: Modules 17, 24, 25, 26, 28, 29, and 31.

Specialise by domain:

Domain	Modules to go deep on
Embedded / firmware	27
Backend / systems	24 + 25
Library / tooling	30
Safety-critical (automotive, medical, aerospace)	23 + MISRA

> **The single biggest differentiator** in interviews and in real work is not syntax. It is *memory ownership discipline*, *undefined behaviour awareness*, and *comfort with GDB and sanitizers*. Keep those three in mind throughout the book; almost every module feeds into one of them.

PART 1 — The C Language

Module 1 — Foundations

In this module:

- History, K&R C, and why C exists
- Standards: C89/C90, C95, C99, C11, C17, C23 — and knowing which one your compiler uses
- Translation phases: preprocess → compile → assemble → link
- Translation units, object files, executables
- `main()` signatures, `argc`/`argv`, exit codes, `envp`

Every program in this module was actually compiled and run with GCC 13.3 on x86-64 Linux, so the outputs you see are real rather than reconstructed from memory. You will get far more out of this module if you open a terminal and follow along, typing each program yourself and comparing your results. Reading about compilation is useful; watching your own compiler do it is transformative.

Why start with history and toolchain mechanics rather than jumping straight into variables and loops? Because nearly every “strange” rule you will meet later — why arrays decay into pointers, why there is no bounds checking, why headers exist at all, why the same program can behave differently on two compilers — traces directly back to the material in this module. Once you understand what C was built for and how a C source file becomes a running program, the rest of the language stops looking arbitrary and starts looking inevitable.

1.1 Why C Exists (the History That Actually Matters)

Programming languages are not handed down fully formed; they are tools built by particular people to solve particular problems. C is no exception, and the problem it was designed to solve still shapes every line of C you will ever write.

The lineage

C did not appear from nowhere. It is the end point of a short chain of languages, each one a simplification or adaptation of the one before:

```
CPL  (1963, Cambridge)
  → BCPL (1967, Martin Richards)
  → B    (1969, Ken Thompson)
  → C    (1972, Dennis Ritchie)
```

CPL was an ambitious, large language that proved difficult to implement. Martin Richards distilled it into BCPL, a much smaller language that was easy to compile. Ken Thompson, working at Bell Labs, pared BCPL down even further into B, so that it would fit on the tiny machines he had available. Dennis Ritchie then evolved B into C. Each step traded generality for practicality — a theme that runs through C’s entire design.

The problem C was invented to solve

In the late 1960s at Bell Labs, Ken Thompson wrote the first version of Unix in PDP-7 assembly language. It worked, but it came with a serious cost: Unix was welded to one specific machine. Assembly language is simply a human-readable form of a particular processor's instruction set, so an operating system written in it can run only on that processor. If you wanted Unix on new hardware, you had to rewrite the entire operating system by hand, instruction by instruction.

Thompson created B to escape assembly. B let him write system software at a higher level, but it had one fatal flaw: it was *typeless*. In B, every value was simply one machine word. There was no distinction between an integer, a character, or an address — they were all "a word." On the word-addressed PDP-7, that was acceptable, because the machine itself thought in words.

Then Bell Labs acquired a PDP-11, and the PDP-11 was *byte-addressed*. Memory was organised in individual bytes, and efficient systems code needed to manipulate those bytes directly — characters in a text buffer, fields in a device register, flags in a status byte. B had no way to say "this is a single byte" as opposed to "this is a whole word." You could not write efficient byte-manipulating systems code in it.

So Dennis Ritchie added *types*: first `char` and `int`, later `float` and `struct`. Crucially, he also designed *pointer arithmetic that respects the size of the pointed-to type*: adding 1 to a pointer to `int` moves forward by the size of one `int`, while adding 1 to a pointer to `char` moves forward by a single byte. That combination — a small set of types that map directly onto what the hardware manipulates, plus pointers that understand those types — is what made C.

The payoff moment: 1973

In 1973, Unix was rewritten in C. Roughly 90% of the kernel became portable source code; only a small remainder had to stay in assembly for the truly machine-specific parts. This is arguably the single most important event in the history of systems programming.

Consider what changed. Before, porting an operating system meant rewriting everything. Afterwards, porting meant writing a C compiler back end for the new machine plus a small amount of assembly glue. The rest of the system simply recompiled. This is why Unix spread across so many different computers, and why C spread with it.

> **Say this in an interview:** C was created to make an operating system portable without giving up the ability to manipulate hardware directly. Every design decision follows from that.

Keep that sentence in mind. When you later wonder why C lacks some safety feature, the answer is usually that the feature would have cost either portability or direct hardware control — and C refused to give up either.

The design philosophy

The committee that later standardised C wrote a *Rationale* document explaining the "spirit of C." Its principles are worth knowing, because each one has a consequence you will feel every day you write C:

Principle	Consequence you'll feel daily
Trust the programmer	No bounds checking. Buffer overflows are your problem.
Don't prevent what needs doing	You can cast anything to anything.
Keep the language small	No strings, no lists, no garbage collector — those are the library's job or yours.
Make it fast	Constructs map roughly 1:1 onto machine instructions.
Portability matters, but so does the machine	Hence "implementation-defined" behaviour everywhere.

Look at the right-hand column carefully, because it is essentially a list of the responsibilities C hands to *you*. The language will not stop you from writing past the end of an array, because checking would cost time on every access. It will not stop you from reinterpreting a block of memory as a different type, because operating systems and device drivers need to do exactly that. It leaves many details — such as the size of an `int` — up to each implementation, so that C can map efficiently onto very different hardware.

C is often called "portable assembly." That is not an insult — it is, quite literally, the specification. C gives you roughly the control of assembly language, expressed in a form that can be recompiled for a different machine.

1.2 K&R C

In 1978, Brian Kernighan and Dennis Ritchie published *The C Programming Language*. For roughly eleven years, that book *was* the standard. There was no committee and no ISO document; if you wanted to know what C meant, you consulted "K&R." The term **K&R C** therefore refers to the dialect described in that first edition.

You are unlikely to write K&R C yourself, but you must be able to *recognise* it, because it still lurks in old codebases, legacy libraries, and code copied from decades-old sources. The most visible difference is in how functions are defined.

```
/* K&R style — parameter types declared separately, no prototype */
int add(a, b)
int a;
int b;
{
    return a + b;
}

/* Modern (ANSI) style */
int add(int a, int b)
{
```

```
    return a + b;
}
```

In the K&R form, the parameter *names* appear in the parentheses, and their *types* are declared on separate lines between the closing parenthesis and the opening brace. In the modern form, each parameter's type and name appear together inside the parentheses.

Why K&R style is dangerous

The difference is not merely cosmetic. A K&R-style definition does not provide a *prototype*, which means the compiler has no information about how many arguments the function takes or what their types are when it sees a call. A call such as `add(1.5, "x")` compiles silently — passing a `double` and a pointer where two `int`s were expected — and the result is corrupted values or a crash at run time, with no warning at compile time.

ANSI C added function prototypes, an idea borrowed from C++, specifically to fix this problem. With a prototype in scope, the compiler checks every call against the declared parameter list, converts arguments where a conversion is well defined, and reports an error where it is not. K&R-style function definitions were finally removed from the language entirely in C23.

The empty-parentheses trap

There is a related legacy that catches many programmers out even today. In K&R C, a declaration such as `int f()` meant "a function taking *unknown, unchecked* parameters" — not "a function taking no parameters." The empty parentheses simply meant "I'm not telling you." That meaning survived through every standard up to C17.

```
void f();          /* old meaning: takes unspecified args — no checking! */
void f(void);      /* correct: takes NOTHING */
```

With the first declaration, `f(1, 2, 3)` compiles without complaint in C17 and earlier, even if `f` really takes no arguments. With the second, the compiler rejects it.

> **Rule:** Always write `(void)`, never empty parentheses. C23 finally made `()` mean the same as `(void)`, but write `(void)` anyway — your code will then mean the same thing under every standard and every compiler your colleagues might use.

1.3 The Standards

After K&R, C was standardised formally, first by ANSI and then by ISO. Since then it has been revised several times. Knowing which features belong to which standard matters in practice, because the compiler you are handed — especially in embedded work — may not support the newest one, and because code that compiles for you may fail for someone using a stricter setting.

Standard	Year	Also called	Key additions
C89 / C90	1989/90	ANSI C, ISO C90	Prototypes, `void*`, `const`/`volatile`, a defined standard library. C89 and C90 are the same language — ANSI ratified it in 1989, ISO in 1990.
C95	1995	NA1 (amendment)	Wide characters (`wchar.h`, `wctype.h`), digraphs
C99	1999	—	`//` comments, declarations anywhere in a block, `long long`, `_Bool`, `inline`, `stdint.h`, VLAs, designated initializers, compound literals, `snprintf`, flexible array members, `restrict`
C11	2011	—	Threads (`threads.h`), atomics (`stdatomic.h`), `_Generic`, `_Static_assert`, anonymous structs/unions, `aligned_alloc`, Annex K `_s` functions; VLAs made optional
C17 / C18	2018	—	Bug-fix release only. Zero new features — just resolutions of defect reports against C11.
C23	2024	—	`nullptr`; `true`/`false`/`bool` as keywords; `typeof`; `constexpr`; `auto`; attributes such as `[[nodiscard]]`; binary literals `0b1010`; digit separators `1'000'000`; `#embed`; `stdbit.h`; removal of K&R definitions

A few observations help make sense of this table. C99 was the last really large expansion of everyday syntax; much of what feels “normal” in modern C — `//` comments, declaring a loop variable inside `for`, fixed-width integer types — arrived there. C11 focused on concurrency and generic programming. C17 changed nothing you would notice; it simply tidied up C11. C23 is the most significant modernisation in decades and brings in several ideas familiar from C++.

How to check which standard your compiler is using

The standard defines a predefined macro, `__STDC_VERSION__`, whose value identifies the version of the language in effect. GCC and Clang additionally define `__STRICT_ANSI__` when you request a strict ISO mode rather than a GNU dialect. The following program reports both:

```c
#include <stdio.h>

int main(void)
{
#ifdef __STDC_VERSION__
    printf("__STDC_VERSION__ = %ldL\n", __STDC_VERSION__);
#else
    printf("__STDC_VERSION__ not defined -> C89/C90\n");
#endif

#ifdef __STRICT_ANSI__
    printf("__STRICT_ANSI__ = defined (strict -std=cXX mode)\n");
#else
    printf("__STRICT_ANSI__ = NOT defined (GNU extensions enabled)\n");
#endif
    return 0;
}
```

Compiling this program once per `-std=` setting produced the following real output:

```
=== -std=c89 ===  __STDC_VERSION__ not defined -> C89/C90
=== -std=c99 ===  __STDC_VERSION__ = 199901L
=== -std=c11 ===  __STDC_VERSION__ = 201112L
=== -std=c17 ===  __STDC_VERSION__ = 201710L
=== -std=c2x ===  __STDC_VERSION__ = 202000L
=== default    === __STDC_VERSION__ = 201710L
                   __STRICT_ANSI__ = NOT defined (GNU extensions enabled)
```

There are three lessons to extract from that output:

1. **C89 does not define `__STDC_VERSION__` at all** — and that absence is precisely how code detects that it is being compiled as C89/C90.
2. **GCC 13's default is `gnu17`, not `c17`.** A plain `gcc file.c` gives you C17 *plus* GNU extensions. Your code may compile happily for you and then fail on a colleague's compiler, or on a different vendor's toolchain, because you unknowingly relied on an extension. Always pass `-std=` explicitly.
3. **On GCC 13, C23 is spelled `-std=c2x`;** the `-std=c23` spelling only arrived in GCC 14. The provisional value `202000L` reflects that the standard was not yet final. Check your compiler version before relying on C23 features.

Making the standard actually enforced

Selecting a standard with `-std=` is not quite the same as *enforcing* it. GCC will still quietly accept some non-standard constructs unless you ask it to be pedantic:

```
gcc -std=c99 file.c                  # allows GNU extras silently in some cases
gcc -std=c99 -pedantic file.c        # warns on non-standard usage
gcc -std=c99 -pedantic-errors file.c # → errors. Use this when portability matters.
```

`-pedantic` turns non-standard usage into warnings; `-pedantic-errors` turns it into hard errors that stop the build. For libraries intended to be portable, and for code that must build on unfamiliar toolchains, `-pedantic-errors` is the honest setting: it tells you today about problems someone else would otherwise discover later.

Live proof of why the standard you pick matters

Here is a small program that uses three features introduced in C99: a `//` comment, a variable declared inside a `for` statement, and a designated initializer.

```c
#include <stdio.h>
int main(void)
{
    printf("start\n");                  // C99+
    for (int i = 0; i < 2; i++)         // C99+
        printf("i=%d\n", i);
    int arr[] = { [2] = 99 };           // C99+
    printf("%d\n", arr[2]);
    return 0;
}
```

Compiled strictly as C89 and then as C99, the results are:

```
### -std=c89 -pedantic-errors ###
error: C++ style comments are not allowed in ISO C90
error: 'for' loop initial declarations are only allowed in C99 or C11 mode

### -std=c99 -pedantic-errors ###
start
i=0
i=1
99
```

The same source file is rejected under one standard and runs perfectly under another. Nothing about the code changed — only the rules it was judged by.

Which standard should you use professionally?

- **C99** is the safe floor. It is universally supported, including by old embedded toolchains that never caught up with later revisions.
- **C11 / C17** is the current default for most industry work. Pick this unless you are told otherwise.
- **C23** is at the early-adoption stage. It is worth learning, but verify that every toolchain your project targets actually supports the features you use.
- **C89** is only for legacy code, very old MSVC targets, or libraries aiming for maximum portability to obscure platforms.

1.4 Translation Phases

When you type `gcc hello.c -o hello`, it feels like a single step: source code goes in, a program comes out. In reality, several distinct programs run one after another, each transforming the output of the previous one. Understanding this pipeline is one of the most

practically useful things you can learn about C, because every error message you ever see comes from one specific stage — and knowing which stage tells you immediately what kind of mistake you made.

The C standard formally defines **eight translation phases**: trigraph replacement, line splicing, tokenization, macro expansion, escape-sequence conversion, string-literal concatenation, translation, and linking. You should know that they exist and roughly what they are. Day to day, however, you work with **four practical stages**:

```
hello.c  --[1 preprocess]-->  hello.i   (still C, expanded)
         --[2 compile   ]-->  hello.s   (assembly)
         --[3 assemble  ]-->  hello.o   (machine code + symbol table)
         --[4 link      ]-->  hello     (executable)
```

GCC lets you stop after any of these stages with a flag: `-E` stops after preprocessing, `-S` after compiling to assembly, `-c` after assembling into an object file, and supplying no stage flag at all runs all the way through linking.

To see each stage concretely, we will follow one small file through the whole pipeline:

```c
#include <stdio.h>

#define SQUARE(x) ((x) * (x))
#define GREETING "hello"

int helper(int n);

int main(void)
{
    int r = helper(SQUARE(4));
    printf("%s %d\n", GREETING, r);
    return 0;
}
```

Note that `helper` is *declared* here but not *defined* anywhere in this file. That is deliberate; it will matter when we reach the linker.

Stage 1 — Preprocess (`gcc -E demo.c -o demo.i`)

The preprocessor performs pure text substitution. It handles lines beginning with `#` — `#include`, `#define`, `#if`, and so on — and it expands macros. It knows nothing about C semantics: it does not know what a type is, does not check that functions exist, and will cheerfully produce nonsense if you ask it to.

Counting the lines before and after preprocessing is revealing:

```
lines in demo.c : 13
lines in demo.i : 826
```

Thirteen lines became 826. The reason is `#include <stdio.h>`: the preprocessor located the header and literally pasted its entire contents — along with every header *it* includes — into the output. This is what “include” means in C: textual insertion, nothing more sophisticated.

The tail end of `demo.i` shows what became of our own code:

```
# 6 "demo.c"
int helper(int n);

int main(void)
{
    int r = helper(((4) * (4)));        // ← SQUARE(4) expanded
    printf("%s %d\n", "hello", r);      // ← GREETING expanded
    return 0;
}
```

Notice several things. The `#include` and `#define` directives are gone, having done their work. The comments are gone too. `SQUARE(4)` has been replaced by `((4) * (4))` — exactly the text the macro specified, with `x` substituted — and `GREETING` has become the string literal `"hello"`. The lines of the form `# 6 "demo.c"` are *line markers*: they record where each piece of text originally came from, so that when the compiler later reports an error it can point at the correct line of your original file rather than line 820 of the expanded one.

> **Debugging superpower:** When a macro misbehaves, run `gcc -E` and read what it actually expanded to. This solves around 90% of macro bugs instantly, because you stop guessing what the preprocessor did and simply look.

Stage 2 — Compile (`gcc -S demo.c -o demo.s`)

Now the real C compiler goes to work. This is the stage that parses C syntax, checks types, reports most of the errors you think of as “compile errors,” and performs optimisations. Its output is assembly language for the target processor:

```
.section .rodata
.LC0: .string "hello"
.LC1: .string "%s %d\n"

main:
    push   rbp
    mov    rbp, rsp
    sub    rsp, 16
    mov    edi, 16          ; ← !! 4*4 already folded to 16 at -O0
    call   helper@PLT
    ...
```

Two observations here are worth internalising.

First, your **string literals live in `.rodata`**, the read-only data section. Both `"hello"` and the format string were placed there. This is the underlying reason why code such as `char *s = "hi"; s[0] = 'H';` crashes on most modern systems: `s` points into memory that the operating system has marked read-only, and writing to it triggers a fault. (Formally, modifying a string literal is undefined behaviour; the crash is simply how it usually manifests.)

Second, look at `mov edi, 16`. The compiler computed `4 * 4` *at compile time*, even with optimisation turned off, and passed the constant 16 directly to `helper`. This is called **constant folding**. The popular mental model that “C is a thin layer over assembly” is broadly true — you can see a clear correspondence between the C and the instructions —

but the compiler is considerably smarter than a naive translator, and it will rearrange and simplify your code whenever the result is guaranteed to be the same. That fact becomes very important when we study undefined behaviour in Module 16.

Stage 3 — Assemble (`gcc -c demo.c -o demo.o`)

The assembler converts the textual assembly into binary machine code and packages it as an **object file**. Asking the `file` utility what it is gives:

```
demo.o: ELF 64-bit LSB relocatable, x86-64, not stripped
```

The key word is **relocatable**. The machine code is real, but the addresses in it are not final yet. The object file does not know where in memory it will eventually be loaded, nor where the functions it calls live. Those questions are left for the linker.

You can inspect an object file's symbol table with `nm`:

```
                 U helper
0000000000000000 T main
                 U printf
```

Each line gives an address (where one is known), a single-letter type, and a symbol name. The letters you will see most often are:

Letter	Meaning
`T`	Text section — defined here, code, globally visible
`t`	The same, but `static` (internal linkage, visible only in this file)
`U`	Undefined — "I use this; linker, go find it"
`D` / `B`	Initialized data / BSS (zero-initialized data)

So this object file defines `main` and knows that it *needs* `helper` and `printf` — but it does not contain either of them. It is an incomplete piece of a program, carrying a list of what it provides and what it still requires.

Stage 4 — Link (`gcc demo.o -o app`)

The linker (`ld`, invoked for you by `gcc`) takes one or more object files plus libraries and stitches them into a finished executable. It resolves every `U` symbol by finding a matching definition somewhere else, assigns final addresses, and patches every reference accordingly.

Let us try to link our object file on its own:

```
$ gcc demo.o -o app
/usr/bin/ld: demo.o: in function `main':
demo.c:(.text+0x12): undefined reference to `helper'
collect2: error: ld returned 1 exit status
```

This is the single most common beginner error in C, and now you know exactly what it means. The symbol `helper` was *declared* — so the compiler, in stage 2, was satisfied that calling it was legitimate — but it was never *defined* anywhere the linker could see. The compiler checks promises; the linker checks that the promises were kept. Note also that the message comes from `ld`, not from the compiler — a clear sign of which stage failed.

Now provide a definition in a separate file and link both object files together:

```
echo 'int helper(int n) { return n + 1; }' > helper.c
gcc -c helper.c -o helper.o
gcc demo.o helper.o -o app && ./app
```

```
hello 17
```

`SQUARE(4)` gave 16, `helper` added one, and the program printed `hello 17`. Running `nm` on the finished executable shows what the linker did:

```
000000000000118c T helper                ← resolved, has an address
0000000000001149 T main                  ← resolved
                 U printf@GLIBC_2.2.5    ← STILL undefined!
```

Both `helper` and `main` now have real addresses. But `printf` is *still* marked `U`. That is not an error: the C library is *dynamically linked*, so `printf` is resolved by the dynamic loader when the program starts, not at build time. The `@GLIBC_2.2.5` suffix records which version of the library's `printf` symbol the program expects. Running `ldd app` lists the shared libraries the program will need at start-up.

> **Diagnosing errors by stage — memorise this:**
>
> - `error: 'X' undeclared` → **compiler**. A missing `#include` or a typo.
> - `warning: implicit declaration of function 'X'` → **compiler**. A missing prototype.
> - `undefined reference to 'X'` → **linker**. A missing `.c` file in the build, or a missing `-lfoo` library.
> - `multiple definition of 'X'` → **linker**. You defined a variable in a header, or forgot `static`.

Whenever you meet an error, your first question should be "which stage produced this?" The answer narrows the cause dramatically.

1.5 Translation Units, Object Files, Executables

The previous section introduced object files in passing. This section makes the underlying idea precise, because it explains more about the shape of real C projects than any other single concept.

A **translation unit** (TU) is one `.c` file *after preprocessing* — that is, your source file plus the full text of every header it pulled in. The translation unit is the compiler's unit of work. The compiler processes one TU at a time, from top to bottom, and it knows *nothing whatsoever* about any other TU. When it compiles `main.c`, it has no idea that `math.c` exists.

That single fact explains most of C's structure:

```
main.c ┐
math.c ┼─ each preprocessed independently → separate TU → separate .o
  io.c ┘
                      ↓ linker joins them
                  executable
```

Each source file becomes its own translation unit and its own object file; only the linker ever sees them all together.

Consequence 1 — why headers exist

Because `main.c` cannot see into `math.c`, it needs some other way to learn that a function called `add` exists and what its signature is. That is the job of a header file. `math.h` carries the *declarations* — promises — and every `.c` file that needs them includes it. The compiler trusts the promise when compiling each file; the linker later verifies that the promise was kept.

```c
/* math.h — DECLARATION: "this exists somewhere" */
int add(int a, int b);

/* math.c — DEFINITION: "here is the actual code" */
int add(int a, int b) { return a + b; }
```

Declaration versus definition

The distinction between these two words is fundamental:

- A **declaration** introduces a name and its type: "there is a function called `add` that takes two `int`s and returns an `int`."
- A **definition** is a declaration that also creates the thing: it supplies the function body, or reserves the storage for a variable.

> **The rule:** You may *declare* a thing as many times as you like. You may *define* it exactly once across the whole program. (This is the One Definition Rule.)

Consequence 2 — headers must contain declarations, not definitions

Because a header is pasted textually into every file that includes it, anything *defined* in a header is defined once per including file. Put `int counter = 0;` in a header that is included by three `.c` files and you have three definitions of `counter` — and the linker will report `multiple definition of 'counter'`.

The correct pattern is to declare the variable in the header with `extern`, and define it in exactly one `.c` file:

```c
/* config.h */
extern int counter;         /* declaration */

/* config.c */
int counter = 0;            /* the one definition */
```

Every file that includes `config.h` now knows that `counter` exists and is an `int`; only `config.c` actually creates it.

Consequence 3 — `static` at file scope means “private to this TU”

When you write `static` in front of a function or variable at file scope, you give it *internal linkage*: the name is visible only within that translation unit and is never exported to the linker. This is C’s only real encapsulation mechanism. There are no classes, no `private` keyword, no modules — just `static`.

Use it aggressively. Every helper function that is not part of a file’s public interface should be `static`. Doing so prevents name collisions between files, documents intent, and gives the optimiser more freedom, because it knows no other file can call the function.

What’s actually inside the files

Object files and executables share a format on most Unix-like systems (ELF), but they differ in important ways:

	Object file (`.o`)	**Executable**
Addresses	relocatable / placeholder	final
Undefined symbols	allowed	only those provided by dynamic libraries
Runnable	no	yes
Entry point	none	`_start` → `main`

The last row deserves a note. Execution does not actually begin at `main`. It begins at a start-up routine, conventionally called `_start`, supplied by the C runtime. That routine sets up the environment — the stack, the arguments, the standard library — and then calls `main`. When `main` returns, control goes back to the runtime, which performs clean-up and exits the process.

The standard ELF sections you should recognise are:

- **`.text`** — machine code (readable and executable)
- **`.rodata`** — string literals and `const` data (read-only)
- **`.data`** — initialized globals, such as `int x = 5;`
- **`.bss`** — zero-initialized globals, such as `int x;`. This section takes no space in the file itself; the file simply records how large it must be, and the loader provides zeroed memory at start-up.
- **`.symtab`** / **`.rel*`** — the symbol table and relocation entries the linker uses

To explore these for yourself, use `nm`, `objdump -d`, `readelf -S`, `size`, `strings`, `ldd`, and `file`. Spend ten minutes running each of them on a small program; they will reappear throughout this book, especially in Module 19.

1.6 `main()` — Signatures, `argc`/`argv`, Exit Codes, `envp`

Every hosted C program begins at `main`. It is the boundary between your code and the operating system: arguments come in through its parameters, and a success-or-failure status goes back out through its return value. Because it sits at that boundary, it has a few rules that apply to no other function.

The two standard-sanctioned signatures

```
int main(void)                      /* no arguments used */
int main(int argc, char *argv[])    /* command-line arguments */
```

That is the complete list. These two forms are guaranteed by the C standard to work everywhere. Any other form is implementation-defined — a particular compiler may accept it, but you have no guarantee.

Never write `void main()`. It is not standard C. Some compilers accept it, and some textbooks have even printed it, but it is still wrong. `main` returns a status to the operating system; declaring it `void` means the operating system receives a garbage exit status, and scripts or build systems that check whether your program succeeded will get unpredictable answers.

In a parameter list, `char *argv[]` and `char **argv` are identical. Array parameters are silently converted ("decay") into pointers — a rule we will study properly in Module 7. Use whichever spelling you find clearer; both are extremely common.

`argc` and `argv` in practice

`argc` ("argument count") tells you how many strings you received; `argv` ("argument vector") is an array of pointers to those strings. The following program prints everything it was given:

```
#include <stdio.h>

int main(int argc, char *argv[], char *envp[])
{
    printf("argc = %d\n", argc);
    for (int i = 0; i < argc; i++)
        printf("  argv[%d] = \"%s\"\n", i, argv[i]);
    printf("  argv[%d] = %s   <-- always NULL\n",
           argc, argv[argc] ? "not null" : "NULL");
    return 0;
}
```

Running it with a handful of arguments produces:

```
$ ./args --verbose -n 5 "two words"
argc = 5
```

```
argv[0] = "./args"
argv[1] = "--verbose"
argv[2] = "-n"
argv[3] = "5"
argv[4] = "two words"
argv[5] = NULL   <-- always NULL
```

Notice that the number `5` arrives as the *string* `"5"`. Everything in `argv` is text; converting it to a number is your job, and Module 8 shows the safe way to do it with `strtol`.

The standard gives you the following guarantees:

1. **`argc >= 0`.** In practice it is almost always at least 1, but a program can technically be launched with no arguments at all.
2. **`argv[0]` is the program name** (or an empty string if the name is unavailable). It is *not* the first user argument. User arguments start at index 1 — a very common off-by-one mistake for newcomers.
3. **`argv[argc]` is always `NULL`.** You can therefore iterate either by count (`i < argc`) or by walking until you reach the null pointer. Both are correct.
4. **The strings are modifiable and persist for the whole program.** You may keep pointers to them, and even alter their contents, for as long as the program runs.
5. **Quoted shell arguments arrive as a single element**, as `"two words"` did above. Splitting on spaces and handling quotation marks is done by the shell before your program ever starts; it is not C's concern.

A classic bug is to read an argument without first checking that it exists:

```c
if (argc < 2) { /* handle missing arg */ }
printf("%s\n", argv[1]);   /* ← reading argv[1] without this check = crash */
```

If the user runs the program with no arguments, `argv[1]` is the terminating `NULL`, and passing a null pointer to `%s` is undefined behaviour — typically a crash. Always check `argc` before indexing into `argv`, and make sure the "missing argument" branch actually leaves or returns.

Exit codes

When your program finishes, it reports a small integer to whatever launched it — a shell, a script, a build system, a CI pipeline. There are three common ways to end a program with status `n`:

```c
return n;     /* from main only */
exit(n);      /* from anywhere; flushes stdio, runs atexit handlers */
_exit(n);     /* POSIX: immediate, NO flushing — use after fork() */
```

`return n` from `main` is equivalent to calling `exit(n)`. `exit` can be called from anywhere in the program; it flushes and closes the standard I/O streams and runs any clean-up functions registered with `atexit`. `_exit` is a POSIX function that terminates the process *immediately*, without any of that clean-up. That sounds strictly worse, but it has an important use: in a child process created with `fork()` (Module 24), calling `exit` would flush a copy of the parent's buffered output a second time, so `_exit` is the correct choice there.

For portability, use the macros from `<stdlib.h>` rather than raw numbers:

```
#include <stdlib.h>
return EXIT_SUCCESS;   /* == 0 everywhere */
return EXIT_FAILURE;   /* 1 on Linux; whatever the platform means by "failed" */
```

The convention — enforced by every Unix shell and every CI system — is **0 means success, non-zero means failure**. Notice that this is the *opposite* of C's own notion of truth, in which 0 is false and non-zero is true. It is a natural place to slip up, especially when a function's return value is passed straight through as an exit status.

Here is a complete, well-behaved example:

```
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[])
{
    if (argc < 2) {
        fprintf(stderr, "usage: %s <file>\n", argv[0]);   /* errors → stderr */
        return EXIT_FAILURE;
    }
    printf("ok: %s\n", argv[1]);
    return EXIT_SUCCESS;
}
```

```
$ ./exitc
usage: ./exitc <file>          ← went to stderr
  exit status = 1

$ ./exitc data.txt
ok: data.txt
  exit status = 0
```

Two good habits are on display. The usage message goes to `stderr`, not `stdout`, so that it is still visible when normal output is redirected to a file or piped into another program. And the message uses `argv[0]`, so it shows the program's name however it was invoked.

The gotcha: exit status is only 8 bits

```
int main(void) { return 256; }
```

```
return 256 -> shell sees 0
```

The status reported to the parent process is truncated to its low 8 bits — effectively `status & 0xFF`. So 256 becomes 0, and a program that "failed" with status 256 appears to have succeeded. Keep your exit codes in the range 0–125. The values 126 and 127 are reserved by the shell for "found but could not execute" and "command not found," and values of 128 and above are used to report "killed by signal N" (as 128 + N).

Two more details

- **Falling off the end of `main`.** In C99 and later, reaching the closing brace of `main` without a `return` statement implicitly returns 0. This is a special exception that applies *only* to `main`. Any other function with a non-`void` return type that falls off the end, when the caller uses the result, gives you undefined behaviour. Write `return 0;` in `main` anyway — it is clearer and costs nothing.
- **`atexit()`.** This function registers clean-up functions that run when the program calls `exit()` or returns normally from `main`. They run in last-in, first-out (LIFO) order — the most recently registered runs first. They do *not* run on `_exit()` or `abort()`.

`envp` — the third parameter

Many systems allow a third parameter to `main`:

```
int main(int argc, char *argv[], char *envp[])
```

Printing its first few entries on the test machine gave:

```
envp[0] = PIP_ROOT_USER_ACTION=ignore
envp[1] = NODE_EXTRA_CA_CERTS=/etc/ssl/certs/ca-certificates.crt
envp[2] = RUST_BACKTRACE=1
```

`envp` is a `NULL`-terminated array of strings, each of the form `"KEY=VALUE"` — the program's environment variables.

But `envp` is not standard C. It is a widely supported extension; POSIX documents it and Windows provides it too, but the C standard does not guarantee it. The portable way to read an environment variable is `getenv`:

```
#include <stdlib.h>
char *path = getenv("PATH");   /* NULL if not set */
if (path != NULL) printf("%s\n", path);
```

Note the null check: `getenv` returns `NULL` when the variable is not set, and you must handle that case.

Use `getenv()` in production code. Reach for `envp` only when you genuinely need to enumerate the *entire* environment — and even then, POSIX's `extern char **environ;` is the more common idiom.

> ⚠ **Security note for later:** Never trust environment variables in privileged programs. `getenv` returns data that an attacker can control completely. Module 23 returns to this in detail.

Practice — Do These Before Module 2

These exercises are designed to make the ideas in this module concrete. Each one should take between ten minutes and an hour. Resist the temptation to skip them: seeing the compiler, the linker, and the shell respond to your own code is what turns this material from facts you have read into knowledge you own.

1. **Stage tracing.** Write any small program. Produce the `.i`, `.s`, and `.o` files and the final executable. Run `wc -l` on the `.i` file, read the `.s` file, and run `nm` on the `.o` file. Explain each symbol letter you see.

2. **Break the linker on purpose.** Declare `int mystery(void);`, call it, and never define it. Read the error carefully. Now define it in a second `.c` file and link the two together. Then mark the definition `static` in that second file and observe the linker fail again — and explain why.

3. **Standards archaeology.** Write a program that uses a VLA (`int n = 5; int a[n];`). Compile it with `-std=c89 -pedantic-errors`, then with `-std=c99`, then with `-std=c11 -pedantic-errors`. Explain all three results.

4. **A real CLI tool.** Write `wordcount.c`, which takes filenames as `argv`, prints a usage message to `stderr` and returns `EXIT_FAILURE` if given none, and returns `EXIT_SUCCESS` otherwise. Verify the exit status with `echo $?` after each run.

5. **Header discipline.** Split a program into `main.c`, `util.c`, and `util.h`, with an include guard in the header. Deliberately put `int global = 0;` in `util.h`, include it from both `.c` files, and read the linker error. Then fix it with `extern`.

6. **Reading question** (answer without compiling): What does `void f();` mean in C89 compared with C23, and why does `void f(void);` exist?

Build everything with:

```
gcc -std=c17 -Wall -Wextra -Wpedantic -g file.c -o prog
```

This command selects a specific standard, turns on a broad set of warnings, enforces standard conformance, and includes debugging information so that you can step through the program in GDB. Make it a habit now; later modules add more flags, but this is the baseline.

Module 1 in Summary

C was created to make Unix portable without surrendering direct control of the hardware, and that origin explains its trust in the programmer, its minimal runtime, and its closeness to the machine. The language has evolved through a series of standards — C89, C99, C11, C17, and C23 — and you should always choose one explicitly with `-std=` rather than accepting your compiler’s default dialect.

A C source file becomes a program through four practical stages: the preprocessor pastes and substitutes text, the compiler turns C into assembly, the assembler produces a relocatable object file, and the linker resolves symbols across object files and libraries to produce an executable. Each translation unit is compiled in isolation, which is why headers carry declarations, why each thing must be defined exactly once, and why `static` is your tool for keeping names private.

Finally, `main` is your program's interface with the operating system: arguments arrive in `argc` and `argv`, environment variables are best read with `getenv`, and a status of zero (success) or non-zero (failure) goes back to whoever ran you.

In Module 2, we turn to the building blocks every C program manipulates: types and values.

Module 2 — Types & Values

In this module:

- Integer types, `char` (signedness is implementation-defined), `short`/`int`/`long`/`long long`
- Floating point: `float`, `double`, `long double`, IEEE-754 basics, NaN/Inf, epsilon comparison
- `_Bool`/`bool`, `void`
- Fixed-width types: `stdint.h` (`int32_t`, `uint8_t`, `intptr_t`, `uintmax_t`)
- `size_t`, `ptrdiff_t`, `ssize_t`, `char16_t`/`char32_t`, `wchar_t`
- `limits.h`, `float.h`, `inttypes.h` format macros (`PRIu64`)
- Integer promotion, usual arithmetic conversions, implicit conversion traps
- Signed overflow (UB) vs unsigned wraparound (defined)
- Type qualifiers: `const`, `volatile`, `restrict`, `_Atomic`
- Casting, type punning, strict aliasing rules

As in Module 1, every output shown here comes from real compilation with GCC 13.3 on x86-64 Linux.

This module contains more career-saving material than any other in the language section of the book. That may sound like an exaggeration for a chapter about “types,” but consider what a type actually is in C: it is the compiler’s only information about how many bytes a value occupies, how those bytes should be interpreted, and which operations are legal on them. When your understanding of types differs from the compiler’s, the compiler wins — silently. Integer promotion and strict aliasing alone account for a huge share of the real-world C bugs that reach production, and both are covered here in depth.

Read the sections marked as critical (§2.7 and §2.11) slowly, and run the examples yourself. They describe rules that are applied invisibly on almost every line of C you will ever write.

2.1 Integer Types

What the standard actually guarantees

Newcomers to C are often surprised to learn that the language does not fix the size of `int`. Unlike Java, where `int` is always 32 bits, C deliberately leaves the exact sizes to each implementation. This is the “portability matters, but so does the machine” principle from Module 1 in action: C wanted `int` to be whatever size the hardware handles most naturally, whether that was 16 bits on an early minicomputer or 32 bits on a modern desktop.

What the standard *does* guarantee is an ordering and a set of minimum ranges. The ordering is:

```
sizeof(char) == 1  ≤  sizeof(short)  ≤  sizeof(int)  ≤  sizeof(long)  ≤  sizeof(long long)
```

`sizeof(char)` is 1 by definition — `sizeof` measures in units of `char` — and each type after it must be at least as large as the one before. The minimum ranges are:

Type	Minimum guaranteed range	Minimum bits
`char`	8 bits	8
`short`	−32,767 … 32,767	16
`int`	−32,767 … 32,767	16
`long`	−2,147,483,647 … 2,147,483,647	32
`long long`	$-(2^{63}-1)$ … $2^{63}-1$	64

Notice that `int` is only guaranteed 16 bits. Code that assumes `int` can hold values above 32,767 is, strictly speaking, not portable — and that assumption genuinely fails on some small microcontrollers still in use today.

Also note that the minimums are −32,767, not −32,768. For most of C's history the standard allowed machines that represented negative numbers in one's-complement or sign-magnitude form, where the range is symmetric. C23 finally mandated two's complement, which is what virtually every machine has used for decades anyway.

What you actually get on x86-64 Linux

The following program reports the size and range of each type on the machine that compiles it:

```c
#include <stdio.h>
#include <limits.h>

int main(void)
{
    printf("%-30s %5s   %s\n", "TYPE", "SIZE", "RANGE");
    printf("%-30s %5zu   %d .. %d\n",   "char",  sizeof(char),  CHAR_MIN, CHAR_MAX);
    printf("%-30s %5zu   %d .. %d\n",   "short", sizeof(short), SHRT_MIN, SHRT_MAX);
    printf("%-30s %5zu   %d .. %d\n",   "int",   sizeof(int),   INT_MIN,  INT_MAX);
    printf("%-30s %5zu   0 .. %u\n",    "unsigned int", sizeof(unsigned int), UINT_MAX);
    printf("%-30s %5zu   %ld .. %ld\n", "long",  sizeof(long),  LONG_MIN, LONG_MAX);
    printf("%-30s %5zu   %lld .. %lld\n", "long long", sizeof(long long), LLONG_MIN, LLONG_MAX);
    printf("%-30s %5zu\n", "void*", sizeof(void *));
    printf("CHAR_BIT = %d\n", CHAR_BIT);
    return 0;
}
```

```
TYPE                           SIZE   RANGE
char                              1   -128 .. 127
short                             2   -32768 .. 32767
int                               4   -2147483648 .. 2147483647
unsigned int                      4   0 .. 4294967295
long                              8   -9223372036854775808 .. 9223372036854775807
long long                         8   -9223372036854775808 .. 9223372036854775807
void*                             8
CHAR_BIT = 8
```

`CHAR_BIT` is the number of bits in a byte. It is 8 on every mainstream platform, but the standard only requires *at least* 8, and some digital signal processors use 16 or even 32. Save this program as `sizes.c`; you will reuse it in the practice exercises.

The data-model trap

The combination of sizes a platform chooses for `int`, `long`, and pointers is called its **data model**. Three models dominate:

Model	`int`	`long`	`void*`	Where
ILP32	32	32	32	32-bit Linux / Windows
LP64	32	64	64	Linux, macOS, BSD (64-bit)
LLP64	32	32	64	Windows 64-bit

The names describe which types are 64 bits wide: in LP64, *L*ong and *P*ointer; in LLP64, only *L*ong *L*ong and *P*ointer.

> **This is the #1 portability bug in real code:** `long` is 64 bits on 64-bit Linux but 32 bits on 64-bit Windows. Code that stores a pointer in a `long` works on Linux and silently truncates on Windows — chopping off the upper half of the address.
>
> **Rule:** Never assume `sizeof(long) == sizeof(void*)`. Use `intptr_t` (§2.4).

`char` — three distinct types

This is the part people get wrong. C has not one but three character types:

```
char            /* implementation-defined: signed OR unsigned */
signed char     /* definitely signed */
unsigned char   /* definitely unsigned */
```

Plain `char` is a *separate type* from both of the others. Even on platforms where `char` happens to be signed, `char` and `signed char` are still not the same type — a distinction that matters for `_Generic` selection (Module 12) and for pointer compatibility (a `char *` is not a `signed char *`).

Whether plain `char` is signed or unsigned is chosen by each platform. Here is what that means in practice — the same source file, compiled with two different flags:

```
char c = 200;
printf("%d\n", c);
```

```
### default (x86-64 Linux: char is SIGNED) ###
char c = 200 -> prints as %d: -56
three types: char=-56 signed char=-56 unsigned char=200

### with -funsigned-char (this is ARM's default!) ###
char c = 200 -> prints as %d: 200
three types: char=200 signed char=-56 unsigned char=200
```

Same code, different answers. When `char` is signed, 200 does not fit in the range −128…127, and the stored bit pattern (`0xC8`) is interpreted as −56. When `char` is unsigned, the same bits mean 200.

ARM, PowerPC, and s390 default to unsigned `char`; x86 and most others default to signed. This bites people porting code to a Raspberry Pi or to embedded ARM boards constantly, because code that “always worked” on a developer’s laptop suddenly behaves differently on the target.

The professional rule is to pick the type that states your intent:

Use	Type
Text / characters	`char`
Raw bytes / binary data	`unsigned char` (or `uint8_t`)
Small integer arithmetic	`signed char` or `int8_t` — be explicit

`unsigned char` also has a special superpower: it is the only type you are allowed to use to inspect the raw bytes of *any* object. It is exempt from the strict aliasing rule (see §2.11), and it has no padding bits or trap representations, so every possible bit pattern is a valid value.

The classic `getchar()` bug

One of the oldest bugs in C looks perfectly reasonable:

```
char c;
while ((c = getchar()) != EOF) { ... }   /*   BROKEN */
```

`getchar()` returns an `int`, not a `char`, and it does so deliberately. It needs to be able to return 257 distinct values: every possible byte, 0 through 255, *plus* a separate value, `EOF` (typically −1), meaning “there is no more input.” A `char` can hold only 256 distinct values, so storing the result in one inevitably merges two of those 257 possibilities. There are two failure modes, depending on the platform:

- **If `char` is unsigned:** `c` can never equal −1, so the comparison with `EOF` is never true, and the loop never ends — an infinite loop at end of input.
- **If `char` is signed:** a perfectly valid input byte `0xFF` is stored as −1 and becomes indistinguishable from `EOF`, so the loop ends early whenever it meets that byte in binary data.

The fix is simply to use the type the function actually returns:

```
int c;                                  /*   correct */
while ((c = getchar()) != EOF) { ... }
```

2.2 Floating Point

The three types

```
sizeof: float=4  double=8  long double=16
```

- **`float`** — IEEE-754 binary32: 1 sign bit, 8 exponent bits, and 23 mantissa bits, giving roughly 7 decimal digits of precision.
- **`double`** — IEEE-754 binary64: 1 + 11 + 52 bits, giving roughly 15–16 decimal digits.
- **`long double`** — *not* standardised. On x86-64 Linux it is the 80-bit x87 extended format padded out to 16 bytes. On MSVC it is simply the same as `double`. On ARM64 Linux it is a true 128-bit quadruple-precision type. Don't rely on it in portable code.

`double` is the default floating type in C. The literal `3.14` is a `double`; `3.14f` is a `float`. The math library follows the same convention: `sin` operates on `double`, with `f` and `l` variants (`sinf`, `sinl`) for `float` and `long double`. Unless you have a specific reason — memory pressure, SIMD throughput, or a hardware FPU that only handles single precision — prefer `double`.

Bit layout, decoded live

A `float` is not stored like an integer. It is stored in a form of binary scientific notation:

```
value = (−1)^sign × 1.mantissa × 2^(exponent − 127)
```

The sign bit selects positive or negative. The exponent is stored with a *bias* of 127, so a stored exponent of 127 means 2^0. The mantissa holds the fractional bits after an implicit leading 1. Decoding some real values makes this concrete:

```
VALUE  BITS        DECODE
1      0x3F800000  s=0 exp=127 (unbiased  0) mantissa=0x000000
2      0x40000000  s=0 exp=128 (unbiased  1) mantissa=0x000000
-2     0xC0000000  s=1 exp=128 (unbiased  1) mantissa=0x000000
0.5    0x3F000000  s=0 exp=126 (unbiased -1) mantissa=0x000000
0.1    0x3DCCCCCD  s=0 exp=123 (unbiased -4) mantissa=0x4CCCD
0      0x00000000  s=0 exp=  0               mantissa=0x000000
inf    0x7F800000  s=0 exp=255               mantissa=0x000000
-nan   0xFFC00000  s=1 exp=255               mantissa=0x400000
```

Read that table carefully, because it explains most floating-point surprises:

- **1.0 is exact, and 0.5 is exact.** Both are powers of two, so they need no mantissa bits at all.
- **0.1 is `0xCCCCCD…`** — a repeating bit pattern that never terminates and has to be rounded off. Binary floating point can represent exactly only sums of powers of two. 0.1 in binary is just like 1/3 in decimal: an infinitely repeating fraction that must be cut short somewhere.
- **An exponent of all ones (255) is a special marker.** With a zero mantissa it means Infinity; with a non-zero mantissa it means NaN ("not a number").
- **An exponent of all zeros** means either zero or a *subnormal* number — a very small value stored with reduced precision.

Why `0.1 + 0.2 != 0.3`

```
0.1 + 0.2 == 0.3 ? FALSE
0.1 + 0.2 = 0.30000000000000004441
0.3       = 0.29999999999999998890
```

Neither 0.1 nor 0.2 is exactly representable, so each is stored as the nearest representable `double`. Their sum is then rounded again, and it lands on a *different* nearest-representable value than the one chosen for the literal `0.3`. Both results are extremely close to 0.3, but they are not the same bit pattern, so `==` reports false.

> **Never use `==` on floating point.** The exceptions are comparing against exact values that you know are representable, such as `0.0`, or checking for a sentinel value that you yourself assigned earlier.

NaN and Infinity

```
 1.0/0.0 =  inf   isinf=1
-1.0/0.0 = -inf   isinf=-1
 0.0/0.0 = -nan   isnan=1
nan == nan ? FALSE   <-- NaN is not equal to itself!
nan != nan ? true
```

IEEE-754 division by zero does not crash: dividing a non-zero value by zero gives positive or negative infinity, and 0.0/0.0 gives NaN. (Integer division by zero is a completely different matter — it is undefined behaviour, as §2.8 explains.)

NaN is not equal to itself. This is IEEE-754's defining oddity, and it is genuinely useful: it is the one way to detect NaN without any library support.

```
if (x != x) { /* x is NaN */ }
```

In real code, however, use the classification functions from `<math.h>`: `isnan()`, `isinf()`, `isfinite()`, `isnormal()`, and `fpclassify()`. They say what they mean, and they survive aggressive optimisation settings that can break the `x != x` trick. Remember to link with `-lm`.

NaN's semantics have consequences that will bite you:

- **All comparisons with NaN are false** — including `<`, `>`, `<=`, and `>=`. Therefore `!(a < b)` is *not* the same as `a >= b` when NaN is possible.
- **A single NaN propagates** through every subsequent computation it touches. One bad input can poison a whole data pipeline, and the NaN may surface far from where it was created.
- **Sorting an array containing NaN with `qsort` gives undefined results,** because your comparator violates the consistent ordering that `qsort` requires.

Signed zero is another IEEE-754 subtlety:

```
0.0 == -0.0 ? true   but bit patterns differ
bits(+0.0)=0x0000000000000000   bits(-0.0)=0x8000000000000000
```

Positive and negative zero compare equal, but they are not interchangeable: `1.0/0.0` is `+inf` while `1.0/-0.0` is `-inf`. Use `signbit()` if you need to tell them apart.

Epsilon comparison — done properly

If `==` is off limits, how should floating-point values be compared? The usual answer is "check whether they are close enough," but "close enough" is harder to define than it looks.

`FLT_EPSILON` and `DBL_EPSILON` are frequently misused. Each is the gap between 1.0 and the *next representable value above 1.0* — not a universal tolerance:

```
FLT_EPSILON = 1.19209e-07
DBL_EPSILON = 2.22045e-16
```

Because the spacing between representable numbers grows with their magnitude, the naive test `fabs(a - b) < DBL_EPSILON` is wrong for anything not close to 1.0. Near 10^{16}, adjacent doubles are 2 apart, so no two distinct values can ever be within `DBL_EPSILON`; near 10^{-20}, *every* pair of values is within it. Measured:

```
big1=10000000000000000.0 big2=10000000000000002.0
near_abs(1e-9) : 0   <-- absolute tolerance FAILS at large magnitude
near_rel(1e-9) : 1   <-- relative works

tiny 1e-20 vs 2e-20 (100% different):
near_rel(1e-9) : 0   <-- correctly says different
```

The two large numbers are adjacent doubles — as equal as floating point can make them — yet an absolute tolerance calls them different. A *relative* tolerance, which scales with the size of the numbers, gets both cases right. Near zero, though, relative error becomes meaningless, so the robust approach is a hybrid of the two:

```c
#include <math.h>
#include <float.h>
#include <stdbool.h>

bool nearly_equal(double a, double b)
{
    double diff = fabs(a - b);

    /* absolute check: handles values near zero, where relative error blows up */
    if (diff <= DBL_EPSILON * 8) return true;

    /* relative check: scales with magnitude */
    return diff <= fmax(fabs(a), fabs(b)) * 1e-9;
}
```

Tune the tolerances to your domain; there is no universally correct value. A physics simulation, a graphics engine, and a statistics package all have different ideas of "equal."

For money, don't use floating point at all. Use integer cents (`int64_t`) or a fixed-point or decimal type. Rounding errors that are harmless in a physics simulation are unacceptable on an invoice.

Practical rules

- **Never accumulate floats in a long loop without thinking.** Rounding error compounds with every addition. If precision matters, look up Kahan summation.
- **`printf("%f", x)` takes a `double`.** `float` arguments to variadic functions are automatically promoted to `double` by the *default argument promotions*. That is why `printf` needs no separate specifier for `float`. (In `scanf`, however, `%f` means `float *` and `%lf` means `double *` — different rules, because `scanf` receives pointers, which are not promoted.)
- **Avoid `-ffast-math` unless you fully understand it.** It lets the compiler assume that NaN and Infinity never occur and lets it reassociate operations, breaking IEEE-754 semantics in exchange for speed.

2.3 `_Bool` / `bool`, and `void`

`_Bool`

C had no Boolean type at all until C99, which added `_Bool`. The awkward name was chosen because identifiers beginning with an underscore and a capital letter were already reserved, so it could not clash with any existing program's own `bool`. The header `<stdbool.h>` provides the friendlier macros `bool`, `true`, and `false`. In C23, `bool`, `true`, and `false` are real keywords, and the header is no longer needed.

`_Bool` is not just a small integer. It has a unique conversion rule: *any non-zero value becomes exactly 1*.

```
sizeof(_Bool) = 1
bool b = 5   -> 1
bool b = 0.5 -> 1
bool b = 256 -> 1   <-- NOT truncated to 0 like char would be
char c = 256 -> 0   <-- truncated
```

That difference matters. Converting 256 to `char` keeps only the low 8 bits, which are all zero, so a "true" value silently becomes false. `char flag = some_int;` can therefore turn a truthy 256 into 0. `bool flag = some_int;` cannot — it asks "is this non-zero?" rather than truncating.

Style note: in C, testing pointers and integers directly — `if (ptr)`, `if (count)` — is idiomatic and perfectly fine. But never write `if (x == true)`. A function that returns `int` might return 2 to indicate success; 2 is truthy, but `2 == true` compares it with 1 and is false.

`void`

`void` means three different things depending on where it appears:

1. **No return value:** `void f(void);`
2. **No parameters:** `int f(void);` — as covered in Module 1, and essential.
3. **`void *` — the generic pointer.** Any object pointer converts to and from `void *` without a cast.

```
void *p = malloc(100);   /* no cast needed in C — casting malloc is a C++ habit */
int *ip = p;             /* implicit conversion is legal and idiomatic */
```

You cannot dereference a `void *`, because the compiler does not know what type of object is there, and you cannot do arithmetic on it, because it does not know how large a step is. GCC does allow `void *` arithmetic as an extension, treating the pointed-to size as 1, but `-pedantic` rejects it. `sizeof(void)` is likewise a GNU extension rather than standard C.

Formally, `void` is an *incomplete type that can never be completed*. You cannot declare `void x;`, and you cannot create an array of `void`.

2.4 Fixed-Width Types — `<stdint.h>`

Section 2.1 showed that C deliberately leaves the size of `int` open. That is fine for ordinary arithmetic, but some code genuinely needs to know exactly how many bits it is working with: file formats, network protocols, hardware registers, and cryptographic algorithms all specify widths precisely. `<stdint.h>`, added in C99, is C's answer to the question "how big is `int`, really?" — it lets you stop asking and simply state the width you need.

Measured on x86-64 Linux:

```
exact-width : int8=1 int16=2 int32=4 int64=8
fast        : int_fast8=1 int_fast16=8 int_fast32=8
least       : int_least8=1 int_least16=2
pointer     : intptr_t=8 uintptr_t=8
max         : intmax_t=8 uintmax_t=8
```

The four families

Family	Meaning	Guaranteed to exist?
`int32_t`, `uint8_t`, …	Exactly *N* bits, no padding, two's complement	No — only if the platform has such a type
`int_least16_t`, …	Smallest type with at least *N* bits	Yes (8/16/32/64)
`int_fast16_t`, …	Fastest type with at least *N* bits	Yes (8/16/32/64)
`intmax_t`, `uintmax_t`	Largest supported integer type	Yes
`intptr_t`, `uintptr_t`	Integer that can hold a `void *` round-trip	Optional, but universal in practice

Each signed type has an unsigned counterpart with a `u` prefix. The exact-width types are "optional" because a machine with, say, 9-bit bytes simply cannot provide an exact 8-bit type; on every mainstream platform they exist.

Look at the measured output again: `int_fast16_t` is 8 bytes on x86-64. The "fast" types trade memory for speed, on the reasoning that the CPU prefers operating on full-width registers. That may be a good trade for a local variable, but it is a poor one for a field in a struct or an element of a large array, where it quadruples the memory used. Don't use the fast types in structures whose size you care about.

When to use which

```c
/* Binary formats, protocols, hardware registers, files on disk → EXACT width */
struct packet_header {
    uint16_t magic;
    uint8_t  version;
    uint32_t length;
};

/* Storing a pointer as an integer → intptr_t */
uintptr_t addr = (uintptr_t)&x;
if (addr % 16 == 0) { /* alignment check */ }

/* Ordinary loop counters and general arithmetic → just use int */
for (int i = 0; i < n; i++) { }
```

Don't reflexively write `uint32_t` everywhere. `int` is the natural word size for the platform and is often the fastest choice, and over-using unsigned types invites the wraparound traps described in §2.7 and §2.8. Use exact-width types where the width is part of the *contract* — where some external format, protocol, or piece of hardware dictates it.

`<stdint.h>` also provides limit macros such as `INT32_MAX`, `UINT8_MAX`, and `SIZE_MAX`, and constant macros such as `UINT64_C(1234)`, which give an integer literal the correct type for the corresponding width.

2.5 `size_t`, `ptrdiff_t`, and Friends

A handful of other types appear constantly in C code and in the standard library. Their sizes on x86-64 Linux:

```
size_t=8  ptrdiff_t=8  wchar_t=4  char16_t=2  char32_t=4
```

Type	Header	Meaning
`size_t`	`<stddef.h>`	Unsigned. The result of `sizeof`. Big enough for the size of any object.
`ptrdiff_t`	`<stddef.h>`	Signed. The result of subtracting two pointers.
`ssize_t`	`<sys/types.h>`	POSIX, not ISO C. A signed size — used for "a count, or −1 on error" (`read`, `write`).
`wchar_t`	`<wchar.h>`	Wide character. 4 bytes on Linux (UTF-32), 2 on Windows (UTF-16). Avoid for portable code.
`char16_t` / `char32_t`	`<uchar.h>`	C11. Explicitly UTF-16 / UTF-32 code units.

Use `size_t` for sizes, array indices, loop counters over arrays, and anything returned by `sizeof` or `strlen` or passed to `malloc`. It is guaranteed to be large enough for any object's size, which neither `int` nor `unsigned int` is on 64-bit systems.

Print it with `%zu` (C99). Not `%d`, and not `%lu` — `%zu` is correct on every platform, whatever the underlying width of `size_t`.

The `size_t` reverse-loop trap

```
for (size_t i = n - 1; i >= 0; i--) { }   /*   INFINITE LOOP */
```

This loop is meant to count down from `n - 1` to 0. But `size_t` is unsigned, so `i >= 0` is *always* true — an unsigned value cannot be negative. When `i` reaches 0 and is decremented, it wraps around to `SIZE_MAX`, and the loop continues forever (usually reading far out of bounds and crashing along the way).

Correct patterns:

```
for (size_t i = n; i-- > 0; ) { /* i goes n-1 ... 0 */ }

size_t i = n;
while (i > 0) { i--; /* use i */ }
```

In the first form, the test `i-- > 0` checks the old value and then decrements, so the body sees `n - 1` down to 0 and the loop stops cleanly. The second form makes the same logic explicit.

Note also that `n - 1` when `n == 0` gives `SIZE_MAX`, not −1. Check for an empty range first.

Because `wchar_t` is 2 bytes on Windows and 4 on Linux, professional C code that handles text almost always uses UTF-8 stored in plain `char` arrays instead. Module 8 covers this properly.

2.6 `<limits.h>`, `<float.h>`, `<inttypes.h>`

These three headers describe the numeric properties of your platform, so that your code can ask rather than assume.

`<limits.h>` — integer limits: `CHAR_BIT`, `CHAR_MIN`/`CHAR_MAX`, `SCHAR_*`, `UCHAR_MAX`, `SHRT_*`, `INT_*`, `UINT_MAX`, `LONG_*`, `ULONG_MAX`, `LLONG_*`, and `ULLONG_MAX`.

`<float.h>` — floating-point characteristics: `FLT_DIG` (6) and `DBL_DIG` (15), the number of decimal digits that survive a round trip; `FLT_EPSILON` and `DBL_EPSILON`; `FLT_MAX`/`FLT_MIN` and `DBL_MAX`/`DBL_MIN`; `FLT_RADIX`; and `DBL_MANT_DIG`.

`<inttypes.h>` includes `<stdint.h>` and adds the *format macros*, which solve a real and irritating problem: how do you `printf` a `uint64_t` portably, when it is `unsigned long` on Linux (so needs `%lu`) and `unsigned long long` on Windows (so needs `%llu`)?

```
#include <inttypes.h>

uint64_t big = 18446744073709551615ULL;
printf("%" PRIu64 "\n", big);    /* NOT %llu, NOT %lu */

printf("%" PRId32 "\n", i32);
```

```
printf("0x%" PRIX32 "\n", hex);
printf("%" PRIuPTR "\n", (uintptr_t)ptr);
```

```
PRIu64 : 18446744073709551615
PRId64 : -9223372036854775808
PRIX32 : 0xDEADBEEF
PRIuPTR: 140725905758232
```

The macros expand to string literals such as `"lu"` or `"llu"`, chosen correctly for the platform. Because adjacent string literals are concatenated by the compiler, `"%" PRIu64 "\n"` becomes a single format string such as `"%lu\n"`. That is why the syntax looks odd at first sight.

The naming scheme is systematic: `PRI` (for print) or `SCN` (for scan), followed by a conversion letter — `d`, `i`, `u`, `o`, `x`, or `X` — followed by a width or kind: `8`, `16`, `32`, `64`, `MAX`, `PTR`, `FAST32`, `LEAST16`, and so on.

> **Getting a `printf` format wrong is undefined behaviour, not a cosmetic issue.** `printf` reads its arguments according to the format string; if they disagree, it reads the wrong number of bytes from the wrong places. Always compile with `-Wformat` (included in `-Wall`), which checks format strings against their arguments.

2.7 Integer Promotion & Usual Arithmetic Conversions ⚠

This is the most important section in this module. The rules here are applied silently and constantly — in almost every arithmetic expression you write — and they produce bugs that look impossible until you know the rules. Then they look obvious.

Rule 1 — Integer promotions

> Any type narrower than `int` — `char`, `signed char`, `unsigned char`, `short`, `unsigned short`, `_Bool`, bit-fields, and enums — is promoted to `int` whenever it is used in an expression, provided `int` can hold all of its values. Otherwise it is promoted to `unsigned int`.

The practical upshot is striking: *there is essentially no arithmetic on `char` or `short` in C.* They are storage types only. The moment you use one in an expression, it becomes an `int`.

```
sizeof('A')=4      ← a character constant is an int in C (not char, unlike C++)
sizeof(c)=1        ← the variable itself
sizeof(c + 0)=4    ← promoted
sizeof(+c)=4       ← even unary + promotes
```

Even adding zero, or applying a unary plus, is enough to turn a one-byte `char` into a four-byte `int`. And a character constant like `'A'` is an `int` from the start.

```
TRAP5: (200 + 100) as int = 300, stored back in uchar = 44
```

Here two `unsigned char` values, 200 and 100, are added. Because both are promoted to `int` first, the addition happens in `int`, and 300 is the true intermediate value. It truncates (300 − 256 = 44) only when the result is stored back into an `unsigned char`. This is why `unsigned char` "overflow" is not really overflow at all: the arithmetic never happens in 8 bits.

```
TRAP7: (signed char)-1 >> 4 = -1, but (unsigned char)-1 >> 4 = 15
```

Both operands are promoted to `int` before the shift, but they promote to very different values. A `signed char` holding −1 promotes to the `int` −1 (`0xFFFFFFFF`), and right-shifting a negative value in practice performs an arithmetic shift that preserves the sign, so the result is still −1. An `unsigned char` holding 255 promotes to the `int` 255 (`0x000000FF`), and shifting that right by 4 gives 15. Sign extension happens during promotion — *before* the shift ever runs.

Rule 2 — Usual arithmetic conversions

When a binary operator (`+`, `-`, `*`, `<`, `==`, and so on) has operands of different types, C converts them to a common type. After the integer promotions have been applied, the rules are:

1. If either operand is `long double` → both become `long double`.
2. Otherwise, if either is `double` → both become `double`.
3. Otherwise, if either is `float` → both become `float`.
4. Otherwise (both are integers):
    - 1. Same signedness → convert to the type of higher rank.
    - 1. **The unsigned operand has rank ≥ the signed one → the signed operand is converted to unsigned.** ← this is the killer
    - 1. The signed operand has higher rank *and* can represent all values of the unsigned type → the unsigned operand is converted to signed.
    - 1. Otherwise → both are converted to the unsigned version of the signed type.

Step 4b is where careers go to die. When a signed value meets an unsigned value of equal or higher rank, the signed value is converted to unsigned — and negative numbers become enormous positive ones. Nothing warns you at run time; the comparison simply gives the "wrong" answer.

The traps, measured

```
TRAP1: -1 < sizeof(int) -> NOT taken (-1 became 4294967295!)
TRAP2: 3u - 5u = 4294967294 (as signed: -2)
TRAP3: strlen("abc") - 5 = 18446744073709551614
TRAP6: (-5 > 5u) ? TRUE (!!)
```

Let us walk through each one.

TRAP 1. `sizeof` returns a `size_t`, which is unsigned and 64 bits wide here. By rule 4b, the `int` −1 is converted to `size_t`, becoming 18,446,744,073,709,551,615. (The printed message shows the 32-bit equivalent, 4,294,967,295; the principle is identical.) The comparison is then "huge < 4," which is false. Mathematically −1 is obviously less than 4, yet the branch is not taken.

This matters most in bounds checks. A test that is meant to reject a negative index can silently do the wrong thing once the index is compared against an unsigned size — the mirror-image bug appears whenever you write something like `i > 0 && i < len` with an unsigned `len`. The canonical broken version looks like this:

```
int len = get_length();                    /* returns -1 on error */
if (len < BUFSIZE) memcpy(buf, src, len);    /* len promoted to size_t → huge copy */
```

If `BUFSIZE` is a plain `int` constant, the comparison `-1 < BUFSIZE` is true and the check passes. Then `memcpy`, whose length parameter is a `size_t`, receives −1 converted to an astronomically large number and copies far past the end of `buf`. This exact pattern is a real CVE category — it has produced security vulnerabilities in widely deployed software.

TRAP 2. `3u - 5u` is unsigned arithmetic, which wraps modulo 2^{32}, so the result is 4,294,967,294. Interpreted as a signed number, those bits would mean −2 — but the type is unsigned, so there is no −2 anywhere; there is only a very large positive number.

TRAP 3. `strlen()` returns a `size_t`. If the string is shorter than 5 characters, `strlen(s) - 5` wraps around to about 1.8×10^{19}. That becomes lethal in a loop condition:

```
for (size_t i = 0; i < strlen(s) - 1; i++)   /* empty string → 18 quintillion iterations */
```

For an empty string, `strlen(s) - 1` is `SIZE_MAX`, so the loop runs effectively forever — reading out of bounds long before it ends.

TRAP 6. `-5 > 5u` is true, because −5 is converted to `unsigned int` and becomes 4,294,967,291, which is certainly greater than 5.

How to defend yourself

1. Turn on the warnings. They catch most of these cases:

```
warning: comparison of integer expressions of different signedness:
         'int' and 'long unsigned int' [-Wsign-compare]
```

```
gcc -Wall -Wextra -Wsign-compare -Wsign-conversion -Wconversion
```

`-Wextra` already includes `-Wsign-compare`. `-Wconversion` is noisy on existing code, but it catches implicit narrowing conversions, and it is well worth enabling from the start on new code.

2. Don’t mix signedness in comparisons. Pick one signedness and convert explicitly at the boundary, with a range check:

```
if (len >= 0 && (size_t)len < bufsize)   /* check sign FIRST, then convert */
```

Because `&&` evaluates left to right and stops early, the conversion to `size_t` only ever happens once `len` is known to be non-negative, when it is guaranteed to be safe.

3. Restructure to avoid unsigned subtraction:

```
if (strlen(s) > 1) { ... }   /* instead of strlen(s) - 1 > 0 */
if (a > b) diff = a - b;     /* instead of if (a - b > 0) */
```

Comparing before subtracting means the subtraction can never wrap.

4. Default to signed `int` for arithmetic. Use unsigned types when you actually need modular arithmetic, bit manipulation, or the full positive range — not merely because "a count can't be negative." This is the guidance in Google's C++ style guide, and it applies equally to C: in practice, unsigned types cause more bugs than they prevent, because ordinary expressions like `a - b` stop meaning what they look like.

2.8 Signed Overflow (UB) vs Unsigned Wraparound (Defined)

C treats overflow in signed and unsigned integers in completely different ways, and the difference is one of the most consequential rules in the language.

Unsigned: fully defined

Unsigned arithmetic is defined to be *modulo* 2^N, where N is the number of bits in the type. It wraps around. This is guaranteed by the standard, portable to every platform, and safe to rely on — hash functions, checksums, and random-number generators do so routinely.

```
UINT_MAX + 1 = 0   (defined wraparound)
```

Signed: undefined behaviour

Signed overflow is **undefined behaviour** (UB). Not "implementation-defined," and not "wraps on most machines" — *undefined*. That means the standard places no requirements whatsoever on what happens, and — crucially — it means the compiler is allowed to *assume it never happens* and to optimise on that basis.

Watch this. Real code, real compiler:

```
int check(int x)
{
    if (x + 1 > x) return 1;
    return 0;
}
```

Mathematically, `check(INT_MAX)` should return 0: on a wrapping machine, `INT_MAX + 1` would be `INT_MIN`, which is not greater than `INT_MAX`. Here is what GCC generates at `-O2`:

```
check:
    endbr64
    mov   eax, 1
    ret
```

The entire comparison has been deleted. The function is now simply `return 1;`. The compiler's reasoning was entirely legitimate: "signed overflow is UB, so a correct program never makes `x + 1` wrap; therefore `x + 1 > x` is always true; therefore the `return 0` branch is dead code."

```
### -O0 ###   check(INT_MAX) = 1
### -O2 ###   check(INT_MAX) = 1
```

This is why "I tested it and it wrapped fine" is not a defence. An overflow check that relies on wrapping will be optimised away — usually the moment someone raises the optimisation level or upgrades the compiler, and usually without any warning.

Catching it: UBSan

The Undefined Behaviour Sanitizer instruments your program to detect UB at run time:

```
gcc -fsanitize=undefined prog.c -o prog
```

```
ub3.c:6:9: runtime error: signed integer overflow:
    2147483647 + 1 cannot be represented in type 'int'
```

It reports the exact file, line, and column, along with the values involved. Use `-fsanitize=undefined` in every debug build and every CI run. It is the single highest-value tool in C development, and it costs nothing but a slower debug binary.

Writing correct overflow checks

The key idea is to check *before* performing the operation, using arithmetic that cannot itself overflow.

Wrong (relies on UB):

```
if (a + b < a) { /* overflow */ }
```

Right — check before operating:

```
#include <limits.h>
if (b > 0 && a > INT_MAX - b) { /* would overflow */ }
if (b < 0 && a < INT_MIN - b) { /* would underflow */ }
```

If `b` is positive, `INT_MAX - b` cannot overflow, and `a + b` would exceed `INT_MAX` exactly when `a > INT_MAX - b`. The negative case mirrors it.

Better — use the checked-arithmetic builtins provided by GCC and Clang, now standardised in C23 as `<stdckdint.h>`:

```
int result;
if (__builtin_add_overflow(a, b, &result)) {
    /* overflow occurred, result is undefined */
}
/* also: __builtin_sub_overflow, __builtin_mul_overflow */

/* C23 standard version */
```

```c
#include <stdckdint.h>
if (ckd_add(&result, a, b)) { /* overflow */ }
```

These compile down to a single add followed by a check of the CPU's overflow flag — faster and clearer than hand-written range tests, and much harder to get wrong. Multiplication in particular is very awkward to check by hand, so the builtins are especially valuable there.

Other UB in this family to memorise

- `INT_MIN / -1` and `INT_MIN % -1` → UB. The true result, `INT_MAX + 1`, overflows — and on x86 it actually traps with `SIGFPE`.
- Division or modulo by zero → UB.
- `x << n` where `n` is greater than or equal to the width of `x`, or `n` is negative → UB.
- Shifting a negative value left → UB (still so in C23; C++20 defined it, C did not).
- `x >> n` on a negative `x` → implementation-defined (an arithmetic shift in practice).
- Converting a floating-point value to an integer type when the value is out of range → UB.

Compilers offer `-fwrapv`, which defines signed overflow as wrapping. It is a genuine escape hatch for legacy code that depends on wrapping, but it disables useful optimisations and makes your code's correctness depend on a compiler flag that other builds may not use. Fix the code instead.

2.9 Type Qualifiers

A *qualifier* modifies how an object of a given type may be accessed, without changing what kind of value it holds. C has four: `const`, `volatile`, `restrict`, and `_Atomic`. Each is widely misunderstood, often by people who have been writing C for years.

`const`

`const` means **"read-only through this name."** It does *not* mean "this object is immutable," and it does *not* mean "compile-time constant." Both of those misreadings cause real bugs.

When `const` appears in pointer declarations, it can apply either to the pointer or to the thing pointed at, and the position tells you which. The reliable technique is to **read right to left, starting from the identifier**:

```c
int *p1;              /* p1: pointer to int */
const int *p2;        /* p2: pointer to const int  (can't write *p2, CAN repoint) */
int const *p2b;       /* identical to p2 */
int *const p3;        /* p3: const pointer to int  (CAN write *p3, can't repoint) */
const int *const p4;  /* both locked */
```

Reading `int *const p3` from right to left gives "p3 is a const pointer to int": the pointer itself cannot change, but the `int` it points to can. Reading `const int *p2` gives "p2 is a pointer to an int that is const": the pointer can be moved, but it cannot be used to modify what it points to. Notice that `const int` and `int const` mean exactly the same thing.

Verified by the compiler:

```
p2: cannot write *p2, CAN repoint
p3: CAN write *p3, cannot repoint

error: assignment of read-only location '*p2'
error: assignment of read-only variable 'p3'
```

Casting away `const` and writing is undefined behaviour:

```c
const int c = 5;
int *p = (int*)&c;
*p = 99;
printf("c=%d *p=%d\n", c, *p);
```

```
c=5 *p=99
```

The same memory location, read two ways, gives two different answers. The compiler had folded `c` into the literal 5 wherever it was used, because it was entitled to assume that nobody would modify a `const` object. The write through `p` did change the memory, but the code reading `c` never looked at memory at all. (Had `c` been a global, it would have been placed in `.rodata`, and the write would have caused a segmentation fault instead.)

Where `const` earns its keep: function parameters. A declaration such as `void print(const char *s)` both documents and enforces that `print` will not modify your buffer. Callers can pass it read-only data with confidence, and the compiler will stop the function's author from accidentally writing through `s`. Const-correctness is expected in professional APIs, and reviewers will notice its absence.

`const` is not a compile-time constant in C (unlike in C++). You cannot use a `const int` as an array size in C89, as a `case` label, or in the initializer of a `static` object. For true constants, use `#define`, an `enum`, or C23's `constexpr`.

`volatile`

`volatile` means **"this object may change in ways the compiler cannot see — reload it every time, and never optimise the access away."**

Normally the compiler is free to keep a variable's value in a register and skip reloading it from memory if nothing in the visible code could have changed it. `volatile` forbids that. Consider two otherwise identical busy-wait loops:

```c
int flag = 0;
volatile int vflag = 0;

void wait_plain(void)    { while (flag == 0) { } }
void wait_volatile(void) { while (vflag == 0) { } }
```

At `-O2`, they compile very differently:

```
wait_plain:
    mov  eax, DWORD PTR flag
    test eax, eax
    jne  .L1
```

```
wait_volatile:
.L6:
    mov  eax, DWORD PTR vflag[rip]
    test eax, eax
```

```
    jmp  .L3   ← reads ONCE,            je   .L6   ← re-reads EVERY iteration
               then spins               ret
```

The non-`volatile` version hoisted the load out of the loop: it reads `flag` once, and if the value is zero it spins forever without ever looking again. It will hang even if another thread, a signal handler, or a hardware device later sets `flag`. The `volatile` version re-reads `vflag` from memory on every iteration.

Legitimate uses of `volatile`:

1. **Memory-mapped hardware registers**, such as `*(volatile uint32_t*)0x40021000`, whose contents are changed by the hardware itself.
2. **Variables modified by a signal handler.** These must be of type `volatile sig_atomic_t`.
3. **Local variables modified between a `setjmp` and the corresponding `longjmp`.**
4. **Preventing the compiler from optimising away benchmark code** whose results are otherwise unused.

> **`volatile` is NOT for thread safety.** This is the most common misconception in C. `volatile` guarantees that the access *happens*, but it provides no atomicity and no memory ordering between threads. For threads you need `_Atomic` / `<stdatomic.h>` or a mutex. (Java and C# gave `volatile` different, stronger semantics — don't carry that intuition into C.)

`restrict`

Added in C99, `restrict` is a **promise you make to the compiler**: "for the lifetime of this pointer, the object it points to is accessed only through this pointer." In other words, no other pointer *aliases* it.

Why does that help? Consider two versions of the same function:

```
void add_plain(int *a, int *b, int *c)                            { *a += *c; *b += *c; }
void add_restr(int *restrict a, int *restrict b, int *restrict c) { *a += *c; *b += *c; }
```

```
add_plain:                          add_restr:
    mov  eax, [rdx]                     mov  eax, [rdx]
    add  [rdi], eax                     add  [rdi], eax
    mov  eax, [rdx]   ← reloads *c      add  [rsi], eax   ← reuses the register
    add  [rsi], eax                     ret
    ret
```

Without `restrict`, the compiler must allow for the possibility that `a` and `c` point to the same `int`. In that case, writing to `*a` would change `*c`, so it must reload `*c` before the second addition. With `restrict`, you have promised that cannot happen, so the compiler loads `*c` once and reuses it. In a tight numeric loop, this kind of freedom is what enables vectorisation, and it can yield a multiple-times speedup.

`restrict` is a promise you must keep. Break it — by passing overlapping pointers — and you get undefined behaviour with no diagnostic at all. This is exactly why `memcpy`, which declares its parameters `restrict`, is UB on overlapping buffers, while `memmove`, which does not, is safe:

```
void *memcpy(void *restrict dest, const void *restrict src, size_t n);
void *memmove(void *dest, const void *src, size_t n);
```

`_Atomic`

Added in C11, `_Atomic` makes operations on an object atomic — indivisible from the point of view of other threads — and gives you memory-ordering guarantees.

```
#include <stdatomic.h>
_Atomic int counter = 0;
atomic_fetch_add(&counter, 1);   /* thread-safe */
counter++;                       /* also atomic, seq_cst by default */
```

The full treatment comes in Module 17. For now, remember the division of labour: **`_Atomic` is for threads; `volatile` is for hardware.** They solve different problems and are not interchangeable.

2.10 Casting

Implicit conversions happen automatically, following the rules in §2.7. An explicit *cast*, written `(type)expr`, forces a conversion that you choose.

```
double d = 3.99;
int i = (int)d;          /* 3 — truncates toward zero, does NOT round */
int j = (int)(d + 0.5);  /* 4 — poor man's rounding; use lround() properly */
```

Rules to internalise:

- **Floating point → integer truncates toward zero.** `(int)-3.7` is −3, not −4. When you want rounding or flooring, use `round()`, `floor()`, `ceil()`, or `lround()` from `<math.h>`. (The `+ 0.5` trick above also gives the wrong answer for negative numbers.)
- **Floating point → integer when the value is out of range is UB.** `(int)1e20` is undefined — it is *not* clamped to `INT_MAX`.
- **Narrowing integer conversions:** to an unsigned type, the result is well-defined modulo truncation. To a signed type, the result was implementation-defined before C23 (in practice, truncation); C23 mandates two's-complement truncation.
- **Widening conversions keep the value,** using sign extension for signed types and zero extension for unsigned types.

When to cast — and when not to:

```
/* GOOD: intentional, documented conversion at a boundary */
size_t n = (size_t)count;        /* after verifying count >= 0 */
double avg = (double)sum / n;    /* force float division, avoid integer division */

/* BAD: silencing a warning you don't understand */
```

```
foo((int)ptr);                  /* the warning was right */

/* UNNECESSARY: void* converts implicitly in C */
int *p = (int*)malloc(n);       /* the cast can hide a missing <stdlib.h> — omit it */
int *p = malloc(n);             /* idiomatic C */
```

The `avg` line is worth a closer look: without the cast, `sum / n` would be integer division, discarding the fractional part *before* the result was ever converted to `double`. Casting one operand forces the whole division into floating point.

The `malloc` case is subtle. In old C, calling a function with no prototype in scope made the compiler assume it returned `int`. If you forgot `#include <stdlib.h>`, a cast would silence the resulting type mismatch and hide a genuine bug — on a 64-bit platform, the returned pointer would be truncated to 32 bits.

> A cast is you overriding the type system. Every cast should have a reason that you could defend in code review. Excessive casting is a smell that the types are wrong upstream.

2.11 Type Punning & Strict Aliasing ⚠

Type punning means reading the same memory as a different type — for example, looking at the bits of a `float` as a `uint32_t`. It is extremely common in systems code: serialisation, hashing, hardware access, network protocols, and floating-point manipulation all do it. It is also a minefield.

The strict aliasing rule

> An object may only be accessed through an lvalue of:
>
> - its own type (or a qualified, or signed/unsigned, variant of it),
> - a struct or union containing that type, or
> - `char`, `signed char`, or `unsigned char`.

The compiler relies on this rule to assume that, for example, an `int *` and a `float *` can never point to the same memory. That assumption lets it skip reloads and reorder memory accesses — valuable optimisations. But if your code *does* make an `int *` and a `float *` point to the same place, the compiler's assumption is false, and the optimised code no longer does what you wrote.

Watch it break

```
int bad(float *f, int *i)
{
    *f = 1.0f;
    *i = 42;              /* compiler assumes this can't touch *f */
    return *f == 1.0f;
}
```

```
float x;
printf("%d\n", bad(&x, (int*)&x));   /* deliberately aliased */
```

Here both pointers point at the same `float`. Writing 42 through `i` overwrites it, so `*f` should no longer be 1.0 and the function should return 0. The same source, built three ways:

```
### -O0 ###                        bad() says *f is still 1.0f? 0
### -O2 (strict aliasing on) ###   bad() says *f is still 1.0f? 1   ← !!
### -O2 -fno-strict-aliasing ###   bad() says *f is still 1.0f? 0
```

At `-O2`, the compiler reasoned that writing through an `int *` cannot affect a `float`, so `*f` must still be 1.0, and it replaced the final comparison with a constant 1. GCC even warned about it:

```
warning: dereferencing type-punned pointer will break strict-aliasing rules
```

The bug appears only at `-O2`. This is the classic “works in debug, fails in release” nightmare: the debug build that you step through in a debugger behaves exactly as you expect, while the release build that ships to customers does not.

The safe ways to pun

1. `memcpy` — always correct, always portable, and it optimises to zero instructions:

```
#include <string.h>
#include <stdint.h>

float f = 1.0f;
uint32_t bits;
memcpy(&bits, &f, sizeof bits);   /* → 0x3F800000 */
```

```
safe memcpy punning: 1.0f -> 0x3F800000
```

Don’t worry about the apparent cost of the “copy.” Every modern compiler recognises a small fixed-size `memcpy` and turns it into a single register move. This is the idiomatic, standard-blessed method.

2. Union punning — allowed in C (this is a real difference from C++, where it is technically UB):

```
union { float f; uint32_t u; } pun;
pun.f = 1.0f;
printf("%08X\n", pun.u);   /* legal in C99 TC3 and later */
```

This is fine in C, but `memcpy` is more portable if your code may ever be compiled as C++ or shared across the C/C++ boundary.

3. `unsigned char *` — always permitted. Inspecting any object byte by byte through an `unsigned char *` is explicitly legal:

```
unsigned char *bytes = (unsigned char *)&some_struct;
for (size_t i = 0; i < sizeof some_struct; i++) printf("%02X ", bytes[i]);
```

4. `-fno-strict-aliasing`**.** The Linux kernel builds with this flag. It is a legitimate engineering decision for codebases full of legacy punning, but it costs performance, and it is a crutch, not a fix.

What you must never do

```
*(int*)&my_float              /* classic UB */
*(uint32_t*)byte_buffer       /* also an ALIGNMENT bug on ARM/SPARC */
struct foo *f = (struct foo*)buf;   /* UB + alignment */
```

The second and third are doubly dangerous. Besides violating strict aliasing, they may read a multi-byte value from an address that is not suitably aligned. On strict-alignment architectures, reading a `uint32_t` from an unaligned address either faults or silently returns garbage. Use `memcpy` instead, and you get both correct aliasing *and* correct alignment handling for free.

Quick Reference — The Rules That Will Save You

#	Rule
1	Use `unsigned char`/`uint8_t` for bytes; use `char` only for text
2	The result of `getchar()` goes in an `int`, never a `char`
3	Never compare floats with `==`; use a hybrid absolute + relative tolerance
4	Never store money in floating point
5	Print `size_t` with `%zu` and `uint64_t` with `PRIu64`
6	Never mix signed and unsigned in a comparison — compile with `-Wsign-compare`
7	Signed overflow is UB; check before the operation, or use `__builtin_*_overflow`
8	`volatile` is for hardware and signals; `_Atomic` is for threads
9	Type-pun with `memcpy`, never with a pointer cast
10	Run `-fsanitize=undefined,address` on every debug build

Your standing build command from here on:

```
gcc -std=c17 -Wall -Wextra -Wpedantic -Wconversion -Wsign-conversion \
    -Wshadow -g -fsanitize=address,undefined prog.c -o prog -lm
```

Compared with Module 1's command, this adds conversion and sign-conversion warnings (which catch most of §2.7), a warning for variables that shadow other variables, the AddressSanitizer and UndefinedBehaviorSanitizer, and the math library.

Practice

1. **Portability audit.** Run the `sizes.c` program from §2.1. Then recompile it with `-m32` (install `gcc-multilib` if needed) and `diff` the two outputs. Which types changed size?

2. **Reproduce the `char` bug.** Write a function that counts how many bytes in a buffer are ≥ 128, using `char`. Compile it normally, then with `-funsigned-char`. Explain the difference. Then fix it.

3. **Prove the optimiser removes your check.** Write `int will_overflow(int a, int b) { return a + b < a; }`. Compile it at `-O0` and `-O2`, dump the assembly with `-S -masm=intel`, and compare. Then rewrite it correctly and verify with UBSan.

4. **The infinite loop.** Write `for (size_t i = n - 1; i >= 0; i--)` over a 5-element array. Run it under ASan and read the report. Fix it in two different ways.

5. **Float forensics.** Write a program that prints the raw bits of a `double` (using `memcpy`) and use it to find: (a) the largest integer for which both x and $x + 1$ are exactly representable, (b) the bit pattern of `DBL_MAX`, and (c) a subnormal number.

6. **Aliasing bug hunt.** Write a `uint32_t hash(const void *data, size_t len)` that reads 4 bytes at a time. First write it the unsafe way (`*(const uint32_t*)p`), compile at `-O2 -Wall`, and note the warning. Then rewrite it with `memcpy` and confirm with `objdump -d` that the generated assembly is identical — proving that `memcpy` costs nothing.

7. **Reasoning question, no compiler:** given `unsigned short a = 65535, b = 1;` on a platform with 32-bit `int`, what are the type and the value of `a + b`? What if `unsigned short` were 32 bits and `int` were also 32 bits?

Module 2 in Summary

C's types are defined by guarantees rather than fixed sizes: `int` is only promised 16 bits, `long` differs between Linux and Windows, and plain `char` may be signed or unsigned depending on the platform. When width matters, `<stdint.h>` lets you state it exactly, and `<inttypes.h>` lets you print such values portably. `size_t` is the type for sizes and indices, and its unsignedness is the source of a whole family of loop bugs.

Floating-point numbers are binary approximations, so equality comparisons need tolerances, NaN is unequal to everything including itself, and money belongs in integers. Every narrow integer is promoted to `int` before arithmetic, and mixing signed with unsigned converts negative numbers into huge positive ones — the single most common source of "impossible" comparisons. Signed overflow is undefined behaviour that optimisers actively exploit, while unsigned overflow wraps predictably.

Finally, the qualifiers each have one specific job — `const` for read-only access, `volatile` for externally changing memory, `restrict` for no-alias promises, `_Atomic` for threads — and type punning must go through `memcpy`, a union, or `unsigned char`, never through a pointer cast.

In Module 3, we turn to Variables, Scope, Lifetime & Linkage — where `static` finally makes complete sense, and where you will understand exactly why `.bss` exists.

Module 3 — Variables, Scope, Lifetime

In this module:

- Declaration vs definition
- Scope: block, file, function, function-prototype
- Linkage: external, internal (`static`), none
- Storage duration: automatic, static, thread (`_Thread_local`), allocated
- `extern`, `static`, `register`, `auto`
- Initialization rules, zero-initialization of statics, indeterminate values

Every variable in C is governed by three independent properties. Beginners tend to blur them together into a vague sense of “where the variable lives”; professionals keep them strictly separate, because each one answers a different question and is decided at a different moment in the life of a program:

```
SCOPE     — WHERE the name is visible              (a compile-time, textual property)
LINKAGE   — WHETHER other TUs see the same object  (a link-time property)
DURATION  — WHEN the object exists in memory       (a run-time property)
```

Scope concerns names in source text: at a given line, can you refer to this identifier? *Linkage* concerns the linker from Module 1: when two translation units mention the same name, do they mean the same object? *Duration* concerns run time: when is the memory for this object created, and when is it destroyed?

These three properties are independent. A variable can be visible in only one block yet live for the whole program; another can be visible across an entire file yet be completely hidden from every other file. Keeping the axes separate is what lets you reason precisely about any declaration you meet.

The keyword `static` means a *different thing on each axis* — which is exactly why it confuses so many people. By the end of this module, that will be completely clear.

3.1 Declaration vs Definition

Module 1 introduced these two terms when explaining why headers exist. Here we make them precise.

> **Declaration:** introduces a name and its type. “This exists somewhere.”
>
> **Definition:** allocates storage or provides the body. “Here it is.”
>
> A definition is also a declaration. Not every declaration is a definition.

In code:

```c
/* DECLARATIONS — may repeat freely */
extern int g;
extern int g;
```

```c
extern int g;

int f(int);
int f(int);

/* DEFINITIONS — exactly one each, program-wide */
int g = 42;
int f(int x) { return x + 1; }
```

```
g=42 f(1)=2    ← compiles clean with -Wall -Wextra
```

Repeating a declaration is harmless: each one simply restates the same promise, and the compiler checks that they agree. Definitions are different. Each definition creates something — storage for a variable, or code for a function — and a program may contain only one of each. Break the rule and the linker catches it:

```
/usr/bin/ld: multiple definition of `g'; first defined here
```

How to tell them apart

Construct	Which?
`extern int x;`	declaration
`int x;` at file scope	tentative definition (see below)
`int x = 5;`	definition
`int f(void);`	declaration (prototype)
`int f(void) { ... }`	definition
`struct S;`	declaration (incomplete type)
`struct S { int a; };`	definition of the type
`typedef int myint;`	declaration — creates no object
`int x;` inside a function	definition (allocates automatic storage)

A few of these deserve comment. A `typedef` never creates an object; it only introduces a new name for an existing type. `struct S;` on its own declares that a structure type called `S` exists without saying what it contains — an *incomplete type*, which is the foundation of the opaque-handle pattern you will meet in Modules 11 and 28. And `int x;` means something quite different inside a function (a definition of a local variable) from what it means at file scope (a *tentative* definition), which brings us to one of C's odder corners.

Tentative definitions — a C-only quirk

At file scope, a declaration such as `int t;` with no initializer and no `extern` is called a **tentative definition**. C lets you write it more than once in the same translation unit:

```c
int t;
int t;          /* legal in C — ERROR in C++ */
int main(void){ printf("t=%d\n", t); }
```

```
t=0
```

At the end of the translation unit, if no real definition (one with an initializer) has appeared, the tentative definitions collapse into a single definition equivalent to `int t = 0;`. That is why the program prints 0.

Within a single file this is merely a curiosity. The trouble comes *across* translation units:

```
/* a.c */  int t;
/* b.c */  int t;     /* two "definitions" — but historically it linked! */
```

```
-- with -fcommon (old GCC default, pre-GCC 10) --
0                              ← linked fine, both refer to ONE object

-- with -fno-common (GCC 10+ default) --
/usr/bin/ld: multiple definition of `t'
```

Old linkers merged such uninitialized definitions into a single shared “common block,” a behaviour inherited from Fortran. Two files that each wrote `int t;` would silently share one variable. GCC 10 changed the default to `-fno-common`, which broke a great deal of ancient code — and correctly so, because the merging was never guaranteed by the standard, and it hid real bugs: two unrelated modules that happened to choose the same global name would silently corrupt each other’s data.

> **The rule that avoids all of this:** variables are *declared* in headers with `extern`, and *defined* in exactly one `.c` file.

```
/* config.h */
extern int verbose_level;    /* declaration */

/* config.c */
int verbose_level = 0;       /* the one definition */
```

3.2 Scope — Four Kinds

Scope is purely textual. It answers one question: *at this point in the source code, is this name visible?* The compiler determines scope by reading the source; nothing about it depends on run-time behaviour. C has four kinds of scope.

1. Block scope

A name declared inside a block — anything between `{` and `}` — is visible from the point of its declaration to the closing brace of the enclosing block. Blocks can nest, and a declaration in an inner block *shadows* (hides) a declaration of the same name in an outer one:

```
int x = 100;                /* file scope */

void demo(int x)            /* parameter — block scope of the body */
{
    printf("param x = %d\n", x);
    { int x = 2;
      printf("inner   = %d\n", x);
      { int x = 3;
        printf("deeper  = %d\n", x); }
```

```
      printf("back    = %d\n", x); }
    printf("param   = %d\n", x);
}
```

```
file-scope x      = 100
  param x         = 1
  inner block x   = 2
  deeper x        = 3
  back to inner   = 2
  back to param   = 1
file-scope x still = 100
```

At each level, the innermost visible declaration wins. When a block ends, its declarations disappear and the outer ones become visible again. The file-scope `x` is untouched throughout; inside `demo` it was hidden the whole time.

Function parameters have block scope covering the whole function body. That is why a parameter and a local variable declared at the top level of the body with the same name collide — they are in the same scope, not nested ones.

Shadowing is legal, but it is almost always a bug. GCC will tell you about it if you ask:

```
warning: declaration of 'x' shadows a global declaration [-Wshadow]
warning: declaration of 'x' shadows a parameter [-Wshadow]
```

> Put `-Wshadow` in your standard flags. Shadowing bugs are nasty: you “fix” a variable and nothing changes, because you modified the shadow instead of the variable you meant.

In C89, declarations had to appear at the top of a block, before any statements. C99 allows declarations anywhere. Take advantage of that: declare each variable at its point of first use, with the narrowest scope possible. A variable that exists only where it is needed cannot be misused anywhere else.

2. File scope

A name declared outside any function has file scope. It is visible from the point of declaration to the end of the translation unit. Note carefully: that means from the declaration *onwards*, not the whole file. A name declared on line 50 is not visible on line 10. This is why functions are usually declared (prototyped) near the top of a file, or in a header, before they are used.

3. Function scope

Only one kind of name has function scope: **labels**, the targets of `goto`. A label is visible throughout the entire function in which it appears — across all blocks, and even *before* the point where it is written:

```
void labels(void)
{
    goto deep_inside;
    { { deep_inside:
          printf("jumped into a nested block\n"); } }
}
```

```
jumped into a nested block via function-scope label
```

This is why label names must be unique within a function, and why `goto` can jump into a block — although not past the declaration of a variable-length array, since that would skip the code that allocates it. Module 5 covers the legitimate uses of `goto`.

4. Function-prototype scope

Parameter names in a function prototype that is not a definition exist only within that prototype's parentheses:

```
void takes(int n, int arr[n]);                /* n visible only here */
void takes(int count, int data[count]) { ... } /* different names — fine */
```

The `n` in the first line is visible just long enough to be used in `arr[n]`, and then it vanishes. The definition is free to use entirely different names. This is why parameter names in prototypes are optional, and why they are purely documentation from the compiler's point of view.

Write them anyway. `void copy(char *dst, const char *src)` in a header tells the reader which argument is which; `void copy(char *, const char *)` leaves them guessing.

3.3 Linkage — Who Else Can See This Object?

Linkage answers a different question: *do declarations of this name in different translation units refer to the same object?* Scope is about visibility within one file; linkage is about identity across files, and it is ultimately enforced by the linker.

Linkage	Meaning	How you get it
External	Same object across the whole program	File scope, non-`static` (the default)
Internal	Same object within one TU only	File scope + `static`
None	Each declaration is its own object	Block-scope locals, parameters, typedefs, enum constants

Seeing linkage in the symbol table

Linkage is not an abstract idea; you can see it directly in an object file. Consider this file:

```
int ext_var = 1;                  /* external */
static int int_var = 2;           /* internal */
int ext_func(void) { ... }        /* external */
static int int_func(void) { ... } /* internal */
```

```
$ nm link_a.o
0000000000000000 T ext_func   ← UPPERCASE = external linkage
0000000000000000 D ext_var    ← UPPERCASE
0000000000000010 t int_func   ← lowercase = internal linkage
```

```
0000000000000004 d int_var    ← lowercase
0000000000000020 T use
```

The capitalisation in `nm` output is exactly the linkage distinction. An uppercase letter means the symbol is exported for the linker to match against other files; a lowercase letter means it is private to this object file. The letter itself still tells you the section: `T/t` for code, `D/d` for initialized data.

Now try to reach the internal ones from another translation unit:

```
undefined reference to `int_func'
undefined reference to `int_var'
```

The objects certainly exist in memory — but their names were never published, so the linker cannot find them from outside.

`static` at file scope is C's encapsulation

This is the most important professional use of `static`. Anything that is not part of your module's public interface should be `static`:

```c
/* parser.c */
#include "parser.h"

static int   line_number;                    /* private state */
static Token next_token(Lexer *lx) { ... }   /* private helper */

Token *parse(const char *src) { ... }        /* PUBLIC — declared in parser.h */
```

You get several benefits for free:

- **No name collisions.** Two different files can each have their own `static int count;` without interfering.
- **Better optimisation.** The compiler knows that nothing outside this file can call `next_token`, so it can inline it aggressively, specialise it, or delete it entirely if it turns out to be unused.
- **A readable API.** Anyone can see the module's public surface simply by reading its header; everything else is visibly internal.
- **Smaller symbol tables and faster linking.**

> **Rule of thumb:** make everything `static` by default, then remove `static` only when a symbol genuinely must be shared.

`extern` — the "look elsewhere" keyword

```c
extern int g;        /* declaration only, no storage allocated */
extern int g = 5;    /* legal but pointless — the initializer makes it a definition */
```

`extern` tells the compiler "this object is defined somewhere else; just trust me about its type." Adding an initializer overrides that meaning and turns the line into an ordinary definition, which defeats the purpose (and GCC warns about it).

For functions, `extern` is the default and therefore redundant:

```
extern int f(int);   /* same as: */
int f(int);
```

Nobody writes `extern` on function declarations in modern C.

The global-variable header pattern, done right, looks like this:

```
/* globals.h */
#ifndef GLOBALS_H
#define GLOBALS_H
extern int   g_verbose;
extern char  g_progname[64];
extern FILE *g_logfile;
#endif

/* globals.c */
#include "globals.h"
int   g_verbose = 0;
char  g_progname[64] = "";
FILE *g_logfile = NULL;
```

Including `globals.h` in `globals.c` itself is a deliberate safeguard: the compiler then sees both the declaration and the definition in one translation unit, and it will report an error if they ever disagree.

That said — **minimise globals.** They defeat testing, because tests cannot easily isolate or reset them; they break thread safety, because every thread shares them; and they create hidden coupling between distant parts of a program. Prefer passing a context structure to the functions that need the data.

Linkage of block-scope declarations

Two special cases are worth knowing:

```
void f(void)
{
    extern int g;       /* refers to the FILE-SCOPE g — external linkage, block scope */
    static int s = 0;   /* NO linkage, but static duration */
    int local;          /* no linkage, automatic duration */
}
```

The `extern` declaration inside `f` is visible only within `f` (block scope), but it refers to the program-wide `g` (external linkage) — a neat illustration that scope and linkage are independent. It is legal but poor style: put such declarations in a header, where every user sees the same one.

3.4 Storage Duration — When Does the Object Exist?

Storage duration is the run-time axis. It determines when an object’s memory comes into existence and when it is released.

Duration	Lives from → to	Where	How you get it
Automatic	block entry → block exit	stack (usually)	plain local variables
Static	program start → program end	`.data` / `.bss`	file scope, or `static` anywhere
Thread	thread start → thread end	TLS block	`_Thread_local` / `thread_local`
Allocated	`malloc` → `free`	heap	dynamic allocation

All four, live

Printing the address of one object of each kind reveals where each lives in the process's memory:

```
ADDRESSES (grouped by region):
  code  &main      = 0x55cc242fe217   ← .text
  .data &g_init    = 0x55cc24301010   ← initialized globals
  .data &s_file    = 0x55cc24301014
  .bss  &g_zero    = 0x55cc2430101c   ← zero-initialized globals
  .bss  &g_uninit  = 0x55cc24301020
  TLS   &tls       = 0x7fb1aff9173c   ← thread-local block
  heap  heap       = 0x55cc2c6fe2a0   ← grows upward
  stack &automatic = 0x7ffef8595f6c   ← way up high, grows downward
```

Read the addresses. The code and static data cluster together near `0x5500…`, laid out in the order `.text`, `.data`, `.bss`. The heap sits a little above them and grows upward as more memory is allocated. The stack lives near `0x7fff…`, at the top of the user address space, and grows downward as functions are called. The thread-local block sits in between, in the region where shared libraries are also mapped. This arrangement is the *process memory layout*, and Module 10 examines it in detail. (Exact addresses change on every run because of address-space layout randomisation, but the relative arrangement does not.)

Automatic vs static, demonstrated

```c
void counter(void)
{
    static int calls = 0;   /* static duration, block scope, NO linkage */
    int local = 0;          /* automatic — recreated every call */
    calls++; local++;
    printf("call #%d : static calls=%d automatic local=%d\n", calls, calls, local);
}
```

```
call #1 : static calls=1  automatic local=1
call #2 : static calls=2  automatic local=1
call #3 : static calls=3  automatic local=1
```

`local` is created fresh, initialized to 0, and destroyed on every call, so it is always 1 when printed. `calls` is created once, before the program starts, and keeps its value between calls, so it counts upward.

Here `static` inside a function means **static duration**, not internal linkage. The variable is still invisible outside the function — that is its scope — and it has no linkage at all. Its initializer is applied once, before `main` runs, not each time the function is called.

> ⚠ **A function-local `static` makes a function non-reentrant and not thread-safe.** If two threads call the function at once, or if it is called recursively, they all share the same variable. This is why `strtok`, which keeps its position in a hidden static variable, is dangerous, and why `strtok_r` exists. Avoid function-local statics in library code.

Why `.bss` exists — measured

Module 1 mentioned that `.data` holds initialized globals and `.bss` holds zero-initialized ones. The distinction exists for a concrete reason: `.data` must be stored in the executable file, byte for byte, because its contents are specific values. `.bss` stores only a *size* — the loader simply provides that much zeroed memory when the program starts.

```c
int big_zero[1000000];          /* .bss */
int big_init[1000000] = {1};    /* .data */
```

```
### file sizes on disk ###
bss_prog     15808 bytes
data_prog  4015816 bytes        ← 254× larger!

### size command (text data bss dec hex) ###
bss_prog:  1229     544  4000032
data_prog: 1229 4000560        8
```

Both programs use the same 4 MB of memory at run time. But the `.data` version carries 4 MB of mostly zeros inside the binary, because setting even one element to 1 forces the entire array to be stored as initialized data.

This is why writing `= {0}` on a huge global array can be a mistake — it may push the array from `.bss` into `.data`. Good compilers detect all-zero initializers and keep them in `.bss` anyway, but don't rely on it, especially with embedded toolchains, where binary size and flash space are precious. A file-scope array with no initializer is already guaranteed to be zero.

Thread storage duration

```c
int               shared = 0;       /* ONE copy for all threads */
_Thread_local int per_thread = 0;   /* one copy PER THREAD */
```

An object with thread storage duration exists once *per thread*: each thread gets its own copy, created when the thread starts and destroyed when it ends. Here is a complete program in which four threads each increment both variables 100,000 times, using C11's `<threads.h>`:

```c
#include <stdio.h>
#include <threads.h>

#define NTHREADS 4
#define ITERS    100000

int               shared = 0;       /* ONE copy for all threads */
_Thread_local int per_thread = 0;   /* one copy PER THREAD */

static int worker(void *arg)
{
    int id = *(int *)arg;
```

```
    for (int i = 0; i < ITERS; i++) {
        shared++;                   /* data race! */
        per_thread++;               /* private to this thread */
    }
    printf("thread %d: per_thread=%d (%s)\n", id, per_thread,
           per_thread == ITERS ? "correct" : "WRONG");
    return 0;
}

int main(void)
{
    thrd_t t[NTHREADS];
    int ids[NTHREADS];
    for (int i = 0; i < NTHREADS; i++) {
        ids[i] = i;
        thrd_create(&t[i], worker, &ids[i]);
    }
    for (int i = 0; i < NTHREADS; i++)
        thrd_join(t[i], NULL);
    printf("shared = %d (expected %d)\n", shared, NTHREADS * ITERS);
    return 0;
}
```

The results:

```
thread 0: per_thread=100000 (correct)
thread 1: per_thread=100000 (correct)
thread 2: per_thread=100000 (correct)
thread 3: per_thread=100000 (correct)

run 1: shared = 300000   ← lost 100,000 increments to a data race
run 2: shared = 400000
run 3: shared = 400000
```

Look at run 1. `shared++` is not atomic: it is really three steps — load the value, add one, store it back. When two threads interleave those steps, one thread's store overwrites the other's, and increments are lost. `_Thread_local` sidesteps the race entirely by giving each thread private storage; every thread's `per_thread` reached exactly 100,000.

Notice also that runs 2 and 3 gave the correct answer. **A data race that produces the right output is still a bug.** Whether it shows up depends on timing, CPU load, and the number of cores — so it may pass a thousand times on your machine and fail in production. Never conclude "it works" from a passing run; use `-fsanitize=thread`, which detects the race whether or not it happens to corrupt the result.

`<threads.h>` provides `thread_local` as a macro for `_Thread_local`, and C23 makes `thread_local` a keyword. Common uses include a per-thread `errno` (which is how the C library makes `errno` safe to use in threaded programs), per-thread scratch buffers, and per-thread random-number-generator state.

Allocated duration

Objects created with `malloc`, `calloc`, or `realloc` exist until you pass them to `free`. Their lifetime is entirely under your control — not tied to any block, function, or thread — which is exactly why allocated memory is the source of leaks, double frees, and use-after-free bugs. Module 10 gives it the full treatment.

3.5 The Storage-Class Specifiers

C has a small set of keywords, called *storage-class specifiers*, that adjust linkage and duration: `extern`, `static`, `auto`, `register`, and `_Thread_local`. You may use at most one per declaration, except that `_Thread_local` may be combined with `extern` or `static`.

`static` — the four-in-one keyword

Where written	What it does
File-scope variable	Internal linkage (duration was already static)
File-scope function	Internal linkage
Block-scope variable	Static duration (linkage was already none)
Array parameter: `void f(int a[static 10])`	C99: "the caller guarantees ≥ 10 elements, non-NULL" — an optimisation hint, not checked

Look at the parenthetical notes in the first and third rows; they are the key to the whole puzzle. A file-scope variable *already* has static duration, so `static` can only change its linkage. A block-scope variable *already* has no linkage, so `static` can only change its duration. The keyword changes whichever property is not already fixed.

The fourth use is unrelated to both. Inside the brackets of an array parameter, `static` promises the compiler that the caller will always pass a non-null pointer to at least that many elements. The compiler may optimise on that basis, and some compilers warn when they can see a call that breaks the promise — but it is not checked at run time.

Same keyword, four unrelated meanings. Now you can explain it in an interview.

`extern`

"This is a declaration only — the definition is elsewhere." It is the default, and therefore redundant, for functions.

`auto`

`auto` is the default for block-scope variables. Nobody has written it in decades: `auto int x;` is identical to `int x;`.

In C23 it was repurposed to mean type inference, as in C++:

```
auto x = 3.14;   /* C23: x is double */
```

`register`

`register` is a hint that a variable is used heavily and would benefit from being kept in a CPU register. Modern compilers ignore the hint entirely — their register allocators are far better than human intuition.

It does have one real semantic effect that the compiler must honour:

```
register int x;
int *p = &x;   /* ERROR: cannot take the address of a register variable */
```

```
error: address of register variable 'x' requested
```

That is occasionally useful for enforcing “this variable must never be aliased,” but in practice: don’t use `register`. (It is deprecated in C++ and was removed in C++17, so it also hampers code that must compile as both languages.)

`_Thread_local`

```
_Thread_local int x;          /* C11 */
thread_local int x;           /* with <threads.h>, or C23 keyword */
static _Thread_local int y;   /* thread duration + internal linkage */
```

`_Thread_local` is valid only at file scope, or at block scope when combined with `static` or `extern`. (An ordinary local variable is already private to the thread executing the function, so a per-thread version of it would be meaningless.)

3.6 Initialization Rules

The single most important rule

> **Static and thread duration objects are guaranteed to be zero-initialized. Automatic duration objects are indeterminate — reading them before assigning a value is undefined behaviour.**

“Zero-initialized” is applied recursively: integers become 0, floating-point values become +0.0, pointers become `NULL`, and every member of a structure and every element of an array is zero-initialized in the same way.

```
STATIC storage is always zero-initialized:
  s_uninit = 0
  s_arr    = 0 0 0 0 0
  s_str[0] = 0 (empty string: "")

AUTOMATIC storage is indeterminate:
  uninitialized local  = -559038737 (0xDEADBEEF)
  uninitialized arr[0] = 0xDEADBEEF arr[1] = 0xDEADBEEF
```

That `0xDEADBEEF` is not random. A function called earlier in the program wrote that pattern into its own stack frame and then returned. The uninitialized locals of the next function landed on the very same bytes of stack and “inherited” the leftover values. Stack garbage is simply data left behind by previous calls — which is exactly why uninitialized reads can leak secrets such as passwords or keys. (The Heartbleed vulnerability, CVE-2014-0160, is the famous example of leftover memory contents leaking to an attacker; there the cause was an over-read of a buffer, but the underlying lesson — stale data in memory is sensitive — is the same.)

GCC catches many such cases, but only when optimisation is enabled:

```
warning: 'local' is used uninitialized [-Wuninitialized]
warning: 'arr' is used uninitialized [-Wuninitialized]
```

> `-Wuninitialized` depends on data-flow analysis, which only runs with optimisation, so it needs at least `-O1`. Compile your warning-check builds with `-O2 -Wall -Wextra` even if you ship `-O0` debug builds. For the cases GCC cannot see, use MemorySanitizer (Clang: `-fsanitize=memory`) or Valgrind.

The simplest defence is to **just initialize everything.** `int x = 0;` costs nothing after optimisation, because the compiler removes the initialization whenever it can prove the value is overwritten before use.

Aggregate initialization

Arrays and structures (collectively, *aggregates*) have their own initialization rules:

```c
int arr1[5] = {1, 2};              /* rest ZERO-filled */
int arr2[5] = {0};                 /* classic all-zeros idiom */
int arr3[5] = {};                  /* C23 empty initializer */
struct P p1 = {1};                 /* b and c zeroed */
struct P p2 = {.c = 9};            /* C99 designated */
int sparse[8] = {[0]=1, [7]=8};    /* C99 designated array */
```

```
arr1   = 1 2 0 0 0
arr2   = 0 0 0 0 0
arr3   = 0 0 0 0 0
p1     = 1 0 0
p2     = 0 0 9
sparse = 1 0 0 0 0 0 0 8
```

(`struct P` here has three `int` members, `a`, `b`, and `c`.)

The rule is simple and applies regardless of storage duration: **if you supply any initializer for an aggregate, every element you don't mention is zero-initialized.** `{0}` works as an "all zeros" idiom because you initialize the first element explicitly to 0 and all the rest by this rule. C23's `{}` says the same thing without mentioning any element at all.

> ⚠ **This applies to elements, not padding bytes.** `struct S s = {0};` does not guarantee that the padding between members is zeroed. So never `memcmp` two structs to compare them, and use `memset(&s, 0, sizeof s)` if you are about to write the struct to a file or a socket — otherwise stale bytes in the padding can leak information. Module 11 explains padding in detail.

Designated initializers (C99) are a major readability win, especially for configuration structures and lookup tables:

```c
static const struct Config defaults = {
    .timeout_ms  = 5000,
    .max_retries = 3,
```

```
    .verbose     = false,
};  /* everything else zeroed, order-independent, self-documenting */
```

Each value is labelled with the member it initializes, so the code survives the structure's members being reordered, and a reader never has to count commas to see which value goes where.

String initialization

Character arrays can be initialized from string literals, with one surprising special case:

```
char s1[6] = "hi";  /* size 6: 'h','i','\0','\0','\0','\0' */
char s2[]  = "hi";  /* size 3: compiler counts it */
char s3[2] = "hi";  /* size 2: 'h','i' — NO TERMINATOR. Legal C! */
```

```
sizeof s1=6 s2=3 s3=2
s1 bytes: 68 69 00 00 00 00
s3 bytes: 68 69     <- no 00 terminator
```

The first case zero-fills the unused space, following the aggregate rule. The second lets the compiler count the characters plus the terminator. The third is a special exemption in the standard: when the array is *exactly* as long as the string's visible characters, the terminating `'\0'` is silently dropped. The result is not a string at all, and passing `s3` to `strlen` reads off the end of the array. GCC 15 and later warn about this with `-Wunterminated-string-initialization`; until your toolchain has that, always write `char s[] = "..."` and let the compiler size the array.

What may appear in an initializer

- **Static and thread duration:** the initializer must be a *constant expression*, evaluated at compile time. The value has to be known before the program starts running.

  ```
  static int a = 5;       /* OK */
  static int b = 5 * 3;   /* OK */
  int n = get_n();
  static int c = n;       /* ERROR: not a constant expression */
  ```

- **Automatic duration:** any expression at all, evaluated at run time each time the block is entered.

  ```
  void f(int n) { int arr_size = n * 2; int x = compute(); }   /* fine */
  ```

Remember, too, that a `const int` is *not* a constant expression in C (unlike C++). Use `#define`, an `enum`, or C23's `constexpr` for compile-time constants.

3.7 Bugs That Come From This Material

The rules in this module are not academic. Each of the following bugs appears regularly in real code, and each follows directly from misunderstanding scope, linkage, or duration.

Returning a pointer to a local

```c
int *bad_return(void)
{
    int local = 42;
    return &local;        /* local dies at return */
}
const char *bad_array(void)
{
    char buf[] = "hello"; /* array on the stack */
    return buf;
}
const char *good_literal(void)
{
    return "hello";       /* string literal has STATIC duration — safe */
}
```

```
warning: function returns address of local variable [-Wreturn-local-addr]
warning: function returns address of local variable [-Wreturn-local-addr]

$ ./dangle
Segmentation fault
```

`local` and `buf` have automatic duration: they cease to exist the moment their function returns. The pointers returned to the caller therefore point at memory that no longer holds those objects — so-called *dangling pointers*.

Notice that the program didn't even print garbage. GCC recognised the undefined behaviour and made the function return `NULL` instead of the stale address, so the dereference crashed immediately. Under AddressSanitizer:

```
ERROR: AddressSanitizer: SEGV on unknown address 0x000000000000
Hint: address points to the zero page.
```

The key distinction is between the two functions that both mention `"hello"`. `char buf[] = "hello"` *copies* the string into a new array on the stack, and that array dies when the function returns. `return "hello"` returns a pointer to the string literal itself, which lives in `.rodata` with static duration — it lasts for the whole program, although it is read-only.

Safe ways to return data from a function:

1. **The caller supplies the buffer:** `void get_name(char *out, size_t n);` ← preferred. The caller controls the memory's lifetime, and the size parameter prevents overflow.
2. **Return heap memory, and document who frees it.**
3. **Return a pointer to static storage** — but then the function is non-reentrant, and each call overwrites the previous result.

The `static` in a header trap

```c
/* util.h */
static int counter = 0;   /*   WRONG */
```

This compiles without any error, but every `.c` file that includes the header gets its *own private copy* of `counter`, because `static` gives it internal linkage in each translation unit separately. Increment it in `a.c`, read it in `b.c`, and you will be baffled that it is still 0. (`-Wunused-variable` sometimes catches this, in files that include the header but never use the variable; often it doesn't.)

Headers should contain declarations, `#define`s, types, and `static inline` functions — never variable definitions.

Static initialization order

Within one translation unit, objects with static duration are initialized in declaration order. Across translation units, the order is unspecified. In C this is rarely a problem in practice, because static initializers must be constant expressions — but whenever setup needs to run *code* (opening a log file, reading configuration), do not build initialization dependencies between globals in different files. Use an explicit `init()` function called from `main`, so that the order is visible and under your control.

3.8 The Unified Reference Table

This table brings the three axes together for every common kind of declaration:

Declaration	Scope	Linkage	Duration
`int x;` (file)	file	external	static
`static int x;` (file)	file	internal	static
`extern int x;` (file)	file	external	static (defined elsewhere)
`const int x = 5;` (file)	file	external*	static
`int x;` (in function)	block	none	automatic
`static int x;` (in function)	block	none	static
`register int x;`	block	none	automatic (no address)
`int f(int x)` — the `x`	block	none	automatic
`_Thread_local int x;` (file)	file	external	thread
`static _Thread_local int x;`	file	internal	thread
`malloc(...)` result	—	—	allocated
`goto` label	function	none	—
prototype parameter name	prototype	none	—
string literal `"abc"`	—	—	static

Declaration	Scope	Linkage	Duration
compound literal `(int[]){1,2}` at block scope	block	none	automatic

* In C, a file-scope `const` still has external linkage by default (unlike C++, where it defaults to internal). Add `static` if you want it private.

If you can fill in this table from memory, and explain each entry, you understand this module.

Practice

1. **Linkage forensics.** Create `mod.c` with one external variable, one `static` variable, one external function, one `static` function, and one function-local `static`. Compile with `-c` and run `nm`. Predict every letter before you look. Then explain why the function-local static appears the way it does.

2. **Reproduce the header bug.** Put `static int counter = 0;` in a header, include it from `a.c` and `b.c`, increment it from `a.c`, and print it from `b.c`. Observe the wrong answer. Fix it with `extern` plus one definition, and confirm.

3. **`.bss` vs `.data`.** Build the two programs from §3.4 yourself and compare them with `ls -l` and `size`. Then try `int arr[1000000] = {0};` — does your GCC keep it in `.bss`? Check with `size`.

4. **Stack garbage archaeology.** Write a function `A` that fills a local array with a recognisable pattern, and a function `B` that declares an uninitialized local array and prints it. Call `A()` then `B()`. Explain what you see. Now run it under `valgrind --track-origins=yes`.

5. **Dangling pointer trio.** Write three functions that return: (a) the address of a local `int`, (b) a local `char buf[]`, and (c) a string literal. Compile with `-Wall` and note which ones warn. Run under ASan.

6. **Thread-local counter.** Take the `tls.c` program from §3.4 and run it 20 times in a loop, recording how often `shared` is wrong. Then fix the race in two ways: with `_Atomic int shared`, and with an `mtx_t`. Compare the run times.

7. **Reasoning, no compiler:** what are the scope, linkage, and duration of each name in this line?

```
static void f(int n) { static int n2 = 0; extern int n3; { int n = 1; } }
```

Module 3 in Summary

Every variable in C has three independent properties. *Scope* is textual and decides where a name can be used; there are four kinds — block, file, function (labels only), and function-prototype. *Linkage* decides whether the same name in different translation units refers to the same object: external by default at file scope, internal with `static`, and none for locals. *Storage duration* decides when the object exists: automatic for locals, static for globals and `static` locals, thread for `_Thread_local`, and allocated for `malloc`.

`static` changes whichever property is not already fixed — linkage at file scope, duration at block scope — and it is C's primary tool for encapsulation. Headers declare with `extern`; exactly one `.c` file defines. Static and thread objects start at zero, while automatic objects start as garbage, so initialize everything. Never return the address of a local, never define variables in headers, and never trust a data race because it happened to give the right answer.

In Module 4, we move on to Operators & Expressions — precedence, short-circuit evaluation, sequence points, and bit manipulation.

Module 4 — Operators & Expressions

In this module:

- Arithmetic, relational, logical, bitwise, assignment, compound assignment
- Ternary, comma operator
- `sizeof`, `_Alignof`, `_Alignas`
- Precedence and associativity
- Short-circuit evaluation
- Sequence points / evaluation order, and the UB it causes (`i = i++ + 1`)
- Bit manipulation: masks, shifts, set/clear/toggle/test, bit tricks, bitfields vs manual bits

Everything below is real GCC 13.3 output.

Operators look like the simplest part of any language. Everyone knows what `+` does. But C's operators carry a surprising amount of hidden machinery: conversions that happen before the operation, rules about which operand is evaluated first (or whether that is decided at all), and a precedence table with a few genuinely unfortunate entries. Two sections here — sequence points (§4.7) and the precedence traps (§4.6) — are where "code that looks right" silently isn't. The final sections cover bit manipulation, the daily toolkit of systems and embedded programming.

4.1 Arithmetic Operators

C's arithmetic operators are `+`, `-`, `*`, `/`, `%`, `++`, and `--`, plus unary `+` and `-`. Most behave exactly as you would expect. Division, remainder, and increment are where the surprises lie.

Integer division truncates toward zero

When both operands of `/` are integers, the result is an integer, and any fractional part is discarded by rounding *toward zero*:

```
 7 /  2 =  3    7 %  2 =  1
-7 /  2 = -3   -7 %  2 = -1
 7 / -2 = -3    7 % -2 =  1
-7 / -2 =  3   -7 % -2 = -1
```

Note that −7 / 2 is −3, not −4: the result moves toward zero rather than toward negative infinity. C99 standardised this truncation. C89 left the rounding direction implementation-defined when an operand was negative, so very old code — or code written for very old compilers — may assume otherwise.

The standard guarantees an invariant that ties division and remainder together:

```
(a / b) * b + (a % b) == a
```

Because division truncates toward zero, this invariant *forces* `%` to take the sign of the dividend (the left operand), not the divisor. You can check it against the table: for −7 / 2, the quotient is −3, and −3 × 2 = −6, so the remainder must be −1 to get back to −7.

`%` is a remainder, not a mathematical modulo

This distinction matters as soon as negative numbers appear:

```
-3 % 3 =  0
-2 % 3 = -2    ← mathematically, -2 mod 3 is 1
-1 % 3 = -1    ← mathematically, -1 mod 3 is 2
```

In mathematics, “a mod m” is always in the range 0 to m − 1. C’s `%` is not: it can return negative values. If you need a genuinely non-negative result — as hash tables, circular buffers, and ring indices all do — add the modulus back in:

```c
int mod(int a, int m) { return ((a % m) + m) % m; }   /* -1 mod 3 → 2 */
```

The inner `%` gives a value between −(m − 1) and m − 1; adding `m` makes it non-negative; the outer `%` brings positive results back into range.

Recall also from Module 2: `INT_MIN % -1` and `INT_MIN / -1` are undefined behaviour (and on x86 they raise `SIGFPE`), and division by zero is undefined behaviour.

The integer-division-in-an-expression trap

With `a = 3` and `b = 2`:

```
a/b           = 1      (integer division happens first!)
(double)a/b   = 1.50
a/(double)b   = 1.50
(double)(a/b) = 1.00   ← too late, already truncated
```

Casting the *result* does not help: by the time the cast is applied, the integer division has already thrown the fraction away. Cast an *operand* instead. That triggers the usual arithmetic conversions from Module 2, converting the other operand to `double` as well, so the whole division is carried out in floating point.

`++` and `--`

```c
int a = ++i;   /* PRE:  increment, then yield the NEW value */
int b = i++;   /* POST: yield the OLD value, then increment */
```

The prefix form increments first and gives you the new value; the postfix form gives you the old value and increments afterwards.

As a standalone statement (`i++;` versus `++i;`), the two are identical for ordinary scalar variables, and in C there is no performance difference between them. (In C++, the postfix form can be more expensive for heavyweight iterator objects, which is why C++ programmers often prefer the prefix form; that concern does not arise in C.) The real hazard is using them *inside larger expressions*, which is the subject of §4.7.

4.2 Relational, Equality, and Logical Operators

The relational operators `<`, `>`, `<=`, `>=` and the equality operators `==`, `!=` yield an `int` with the value 0 or 1 — not a `_Bool`. (In C++ they yield `bool`; another small difference between the languages.)

The logical operators `&&`, `||`, and `!` also yield an `int` 0 or 1, and they treat *any* non-zero operand as true.

Short-circuit evaluation — measured

The most important property of `&&` and `||` is that they may not evaluate their right-hand operand at all. Using a helper function `side()` that counts how often it is called:

```
&& : if left is 0, right is NEVER evaluated
  0 && side(1) -> side called 0 times
  1 && side(1) -> side called 1 times

|| : if left is nonzero, right is NEVER evaluated
  1 || side(2) -> side called 0 times
  0 || side(2) -> side called 1 times

&, | (bitwise) DO evaluate both sides:
  side(3) CALLED
  0 & side(3) -> side called 1 times   <-- !!
```

The logic is straightforward. If the left side of `&&` is false, the whole expression must be false no matter what the right side says, so C does not bother evaluating it. If the left side of `||` is true, the whole expression is already true. The bitwise operators `&` and `|`, by contrast, always evaluate both operands.

`&&` and `||` are among the very few operators in C with guaranteed left-to-right evaluation and a sequence point between their operands (§4.7). That guarantee is what makes the fundamental C safety idiom work:

```c
if (p != NULL && *p == 'x')            /* *p is only reached if p is non-NULL */
if (i < n && arr[i] > 0)               /* arr[i] is only reached if in bounds */
if (fp && (line = fgets(buf, n, fp)))  /* chained guards */
```

```
p was NULL, *p never dereferenced -> no crash
bounds check short-circuited, arr[5] never read
```

Each condition places a *guard* on the left that makes the right-hand test safe to evaluate.

> **Order matters, and `&&` is not commutative here.** `*p == 'x' && p != NULL` crashes when `p` is null, because the dereference happens before the check. Always put the guard first.

The bitwise operators `&` and `|` do not short-circuit and have no sequence point between their operands. Typing `&` when you meant `&&` still compiles — the result is usually even the same truth value — but it evaluates the right side unconditionally, potentially dereferencing a null pointer. Always use the correct operator.

`!` and the double-negation idiom

`!x` yields 1 if `x` is 0, and 0 otherwise. Applying it twice therefore normalises any value to exactly 0 or 1:

```
int is_set = !!(flags & MASK);   /* 0 or 1, not 0 or 0x40 */
```

This is useful when you need a canonical Boolean from a mask test — for example, to store in a 1-bit field, to use as an array index, or to compare against another Boolean.

4.3 Assignment and Compound Assignment

In C, `=` is an *operator*, not a statement. It yields the value that was assigned, which is why `a = b = c = 0` works: assignment is right-associative, so this parses as `a = (b = (c = 0))`, and each assignment passes its value leftward.

Compound assignment: `a op= b` is not the same as `a = a op b`

The compound assignment operators (`+=`, `-=`, `*=`, `/=`, `%=`, `&=`, `|=`, `^=`, `<<=`, `>>=`) look like simple shorthand, but there are two real differences.

Difference 1 — the left operand is evaluated only once:

```
arr[next()] += 10;                /* next() called ONCE */
arr[next()] = arr[next()] + 10;   /* next() called TWICE — different elements! */
```

```
next() called
arr = 10 0, idx=1   (next() called once)
```

In the long form, `next()` is called twice, and each call can return a different index — so the value is read from one element and stored into another. (Worse, the two calls are unsequenced relative to each other, so which one computes which subscript is not even specified.) This makes compound assignment strictly safer when the left-hand side contains a side effect.

Difference 2 — there is an implicit conversion back to the left operand's type:

```
char c = 100; c += 100;          /* → -56 */
unsigned char u = 200; u *= 2;   /* → 144 */
```

```
char c=100; c += 100; -> -56    (a = a + b would need a cast)
uchar u=200; u *= 2;  -> 144    (400 mod 256)
```

Formally, `a op= b` behaves like `a = (type of a)(a op b)`. The arithmetic still happens in `int`, because of the integer promotions from Module 2, and then the result is silently truncated back into the narrower type. Writing `c = c + 100` would produce a `-Wconversion` warning, because you are visibly assigning an `int` to a `char`; `c += 100` typically produces no warning at all. **Compound assignment hides narrowing.**

The `=` vs `==` classic

```
if (x = 5) { }    /* assigns 5, always true */
if (x == 5) { }   /* compares */
```

Because assignment is an expression, the first line is perfectly legal C: it assigns 5 to `x` and then tests the result, which is always true. GCC's `-Wparentheses` (part of `-Wall`) warns about assignments used as conditions, unless you wrap them in an extra pair of parentheses to signal that it is intentional:

```
while ((c = getchar()) != EOF) { }   /* intentional — extra parens */
if ((rc = do_work()) != 0) { }
```

Some codebases use "Yoda conditions" such as `if (5 == x)`, so that a mistyped `=` becomes a compile error (you cannot assign to a constant). Modern practice is to rely on the compiler warning instead; inverting every comparison makes code harder to read, and the warning catches the mistake anyway.

4.4 Ternary and Comma

Ternary `?:`

```
result = condition ? if_true : if_false;
```

The conditional operator evaluates `condition`, then evaluates *exactly one* of the other two operands. There is a sequence point after the condition, so `p ? *p : 0` is safe: `*p` is evaluated only when `p` is non-null.

The *type* of the result, however, is determined by applying the usual arithmetic conversions to *both* branches — regardless of which one actually runs:

```
sizeof(1 ? 'a' : 'b') = 4    (char promoted to int)
sizeof(1 ? 1 : 2.0)   = 8    (int converts to double)
1 ? 1 : 2.0 prints as 1.0    (it IS a double)
```

Even though the condition is always true and the `int` branch is always chosen, the result is a `double`, because the other branch is a `double`.

This brings back the signedness trap from Module 2. With `int s = -1` and `unsigned u`:

```
(1 ? s : u) as %d = -1
(1 ? s : u) as %u = 4294967295
```

```
warning: operand of '?:' changes signedness from 'int' to 'unsigned int'
         due to unsignedness of other operand [-Wsign-compare]
```

Even though only the `s` branch is taken, the type of the whole expression is `unsigned int`, because `u` is in the other arm. The value −1 has been converted to 4,294,967,295. Compare it with anything, and you inherit all the problems of §2.7.

Use the conditional operator for short value selections where an expression is required:

```c
int max = a > b ? a : b;
printf("%d item%s\n", n, n == 1 ? "" : "s");
static const int limit = DEBUG ? 10 : 10000;   /* if/else can't do this */
```

The last line shows something an `if` statement cannot do: choose between values inside an initializer (here, a constant expression, since `DEBUG` is a macro).

Don't use it for side effects, or for anything nested more than one level deep. `a ? b : c ? d : e` is legal — the operator is right-associative, so it means `a ? b : (c ? d : e)` — but write an `if` chain instead; readers should not have to work it out.

The comma operator `,`

The comma operator evaluates its left operand, discards the value, then evaluates its right operand and yields that value. There is a sequence point between the two.

```
left side runs
x = 42
sizeof((1,2.0)) = 8   (type of the RIGHT operand)
```

Its one genuinely idiomatic use is in `for` loops, to update more than one variable:

```c
for (int i = 0, j = 9; i < j; i++, j--) { ... }
```

```
i=0 j=9 / i=1 j=8 / i=2 j=7 / i=3 j=6 / i=4 j=5
```

(Note that the comma in `int i = 0, j = 9` is part of the declaration syntax, while the one in `i++, j--` is the comma operator.)

> ⚠ **Commas in function argument lists and declarations are separators, not the comma operator.** `f(a, b)` passes two arguments; `f((a, b))` passes one — the value of `b`. Same character, different grammar.

The comma operator is also used inside multi-statement macros, but `do { } while (0)` is clearer for that purpose (Module 12).

4.5 `sizeof`, `_Alignof`, `_Alignas`

`sizeof`

`sizeof` yields a value of type `size_t`, measured in bytes (that is, in units of `char`). It has two syntactic forms:

```
sizeof expression   /* no parens needed */
sizeof(type)        /* parens REQUIRED for a type name */
```

`sizeof` is an operator, not a function, which is why the parentheses are optional for expressions.

Crucially, `sizeof` **does not evaluate its operand**. It is a compile-time operation on the operand's type:

```c
int i = 5;
printf("sizeof(i++) = %zu, i is still %d\n", sizeof(i++), i);
```

```
sizeof(i++) = 4, i is still 5
```

The `i++` is never executed; the compiler only needed its type. (The one exception is a variable-length array: `sizeof` on a VLA is evaluated at run time, because the size is not known until then.)

The array-length idiom:

```c
int arr[10];
size_t n = sizeof arr / sizeof arr[0];   /* → 10 */
```

```
sizeof(arr) = 40 (whole array)
sizeof(p)   = 8 (a pointer)
arr length  = 10
```

Dividing the size of the whole array by the size of one element gives the number of elements, and it stays correct if you later change the element type or the array size.

> ⚠ **This only works on a real array that is in scope.** Pass the array to a function and it decays to a pointer — `sizeof` then gives you 8, the size of the pointer, silently. This is one of the most common bugs in C. Module 7 covers decay properly; for now: always pass the length alongside the array.

A safer macro, for GCC and Clang, refuses to compile when given a pointer:

```c
#define ARRAY_LEN(a) (sizeof(a) / sizeof((a)[0]) \
    + 0 * sizeof(struct { _Static_assert(!__builtin_types_compatible_p( \
          typeof(a), typeof(&(a)[0])), "ARRAY_LEN on a pointer"); int _; }))
```

It works by checking at compile time whether the type of `a` is the same as the type of a pointer to its first element. For a true array the types differ, the assertion passes, and the extra term contributes zero. For a pointer they are the same, and compilation stops with the message "ARRAY_LEN on a pointer."

Also, **prefer `sizeof *ptr` over `sizeof(Type)` in allocations** — it survives a change of type:

```
p = malloc(n * sizeof *p);                 /* good: no type repeated */
p = malloc(n * sizeof(struct Widget));     /* breaks silently if p's type changes */
```

If someone later changes `p` to point to a different, larger structure, the first line automatically allocates the right amount; the second silently allocates too little.

`_Alignof` / `alignof` (C11)

Every type has an *alignment requirement*: objects of that type must be placed at addresses that are a multiple of some number of bytes. `_Alignof` yields that requirement. `<stdalign.h>` provides the lowercase spelling `alignof`, and C23 makes `alignof` a keyword.

```
alignof(char)=1 alignof(int)=4 alignof(double)=8 alignof(max_align_t)=16
```

`max_align_t` is the type with the strictest alignment on the platform, and its alignment is what `malloc` guarantees for every block it returns — so any object can be stored in `malloc`ed memory.

`_Alignas` / `alignas` (C11)

`_Alignas` requests *stronger* alignment for an object or structure member than its type would normally need:

```
struct A64 { alignas(64) char buf[8]; };
```

```
alignas(64) struct: sizeof=64 alignof=64
address % 64 = 0
```

Notice that the size grew from 8 to 64. The size of a type must always be a multiple of its alignment; otherwise, in an array of these structures, the second element would not be correctly aligned.

Why would you want this? Common reasons are cache-line alignment to avoid *false sharing* between threads (Module 25), the alignment requirements of SIMD load instructions (16, 32, or 64 bytes), and DMA buffers on embedded targets, where hardware may require specific alignment.

Padding, previewed

Alignment has a visible consequence for structures:

```
struct S { char c; int i; char d; };
```

```
sizeof=12 alignof=4
offsetof c=0 i=4 d=8   <- padding inserted
packed version: sizeof=6
```

Three members totalling 6 bytes occupy 12, because the `int` needs 4-byte alignment. The compiler inserts 3 bytes of padding after `c` so that `i` starts at offset 4, and 3 more after `d` so that the whole structure's size is a multiple of 4. `__attribute__((packed))` removes the padding, but it creates misaligned members, which are slower to access on x86 and cause faults on some ARM configurations. Module 11 gives this the full treatment, including how to order members to minimise padding.

4.6 Precedence and Associativity

Precedence decides how an expression with several different operators is grouped: in `a + b * c`, multiplication has higher precedence, so the expression means `a + (b * c)`. *Associativity* decides how operators of the same precedence group: `a - b - c` means `(a - b) - c` because subtraction associates left to right, while `a = b = c` means `a = (b = c)` because assignment associates right to left.

Note that precedence and associativity only determine *grouping*. They say nothing about the *order in which operands are evaluated* — that is a separate question, answered in §4.7.

The full table, from highest precedence to lowest:

Prec	Operators	Assoc
1	`()` `[]` `.` `->` postfix `++` `--` compound literals	L→R
2	prefix `++` `--` unary `+` `-` `!` `~` `*` `&` `sizeof` `_Alignof` casts	R→L
3	`*` `/` `%`	L→R
4	`+` `-`	L→R
5	`<<` `>>`	L→R
6	`<` `<=` `>` `>=`	L→R
7	`==` `!=`	L→R
8	`&`	L→R
9	`^`	L→R
10	`|`	L→R
11	`&&`	L→R
12	`||`	L→R
13	`?:`	R→L
14	`=` `+=` `-=` `*=` …	R→L
15	`,`	L→R

The precedence bugs that actually happen

Trap 1 — bitwise operators bind more loosely than comparisons. This is the big one.

```
flags & MASK == 0   /* parses as: flags & (MASK == 0) */
```

```
flags & MASK == 0  parses as flags & (MASK==0) = 0
(flags & MASK) == 0                            = 0
warning: suggest parentheses around comparison in operand of '&'
```

Because `==` (level 7) binds more tightly than `&` (level 8), the expression compares `MASK` with 0 first — giving 0 or 1 — and then ANDs that with `flags`. The result has nothing to do with the intended test, even when (as in this run) it happens to produce the same value by coincidence.

This is a historical wart. In C's earliest form, `&` and `|` doubled as the logical operators, so giving them low precedence made sense; when `&&` and `||` were added, the old precedence was kept for compatibility. Ritchie later said he would fix it if he could, but by the time the problem was noticed, too much code depended on it. **Always parenthesise bitmask tests.**

Trap 2 — shifts bind more loosely than arithmetic:

```
1 << 2 + 3   parses as 1 << (2+3) = 32
(1 << 2) + 3                      = 7
warning: suggest parentheses around '+' inside '<<'
```

Programmers often think of `<<` as "multiply by a power of two" and expect it to bind like `*`. It doesn't; it binds more loosely than `+`.

Trap 3 — `?:` binds very loosely:

```
x = 1 ? 2 : 3;   /* parses as x = (1 ? 2 : 3) → 2, because = is even lower */
```

Here the result is what you expect, because assignment is lower still. But mix `?:` with other operators — for example `a + b ? c : d`, which means `(a + b) ? c : d` — and it surprises you. Parenthesise.

Trap 4 — unary operators bind more tightly than binary ones:

```
-3 % 2 = -1   (parses as (-3) % 2)
```

Trap 5 — pointer and increment combinations. Postfix `++` binds more tightly than unary `*`. With `int a[] = {10, 20, ...}` and `p` pointing at `a[0]`:

```
v = *p++   -> v=10, *p now = 20        /* deref old p, then advance p */
v = (*p)++ -> v=10, a[0] now = 11      /* increment the VALUE */
v = *++p   -> v=20                     /* advance p, then deref */
v = ++*p   -> v=11, a[0] now = 11      /* deref, then increment the value */
```

`*p++` means `*(p++)`: the pointer is incremented, but the dereference uses its old value. It is the canonical C string-copy idiom — `while ((*d++ = *s++)) ;` copies characters one at a time until it has copied the terminating `'\0'`. Learn to read it fluently. The other three forms are rare, and they deserve parentheses when you use them.

> **The professional rule:** memorise only three tiers — *unary* > *arithmetic* > *everything else* — and parenthesise the rest. Nobody in code review is impressed that you know `^` binds more tightly than `|`. Clarity wins. Compile with `-Wparentheses` (in `-Wall`) and fix every warning it gives.

4.7 Sequence Points & Evaluation Order ⚠

This is the single most misunderstood corner of C. It concerns not *how* an expression is grouped, but *when* each part of it is evaluated — and what happens when an expression both reads and modifies the same variable.

The rule

> Between two sequence points, an object may be modified at most once, and if it is modified, any read of it must be only for computing the value to be stored. Violate this and the behaviour is undefined.

A *sequence point* is a moment during execution at which all the side effects of earlier evaluations are guaranteed to be complete, and none of the later ones has begun. Between two sequence points, the compiler is free to perform reads and writes in whatever order it likes — and so an expression that modifies the same variable twice, or reads a variable while also modifying it for another purpose, has no well-defined meaning.

C11 replaced the “sequence point” terminology with a more precise model built on the relations *sequenced-before*, *unsequenced*, and *indeterminately sequenced*. The practical consequences are identical, and the older term remains the common way to talk about them.

Where sequence points exist

Guaranteed sequence point	Not a sequence point
`&&` between operands	`+` `-` `*` `/` `%` between operands
`||` between operands	`&` `|` `^` between operands
`?:` after the condition	`<` `>` `==` between operands
`,` operator between operands	`,` separator in an argument list
End of a full expression (`;`)	Assignment operands relative to each other
Before a function call executes (after all arguments are evaluated)	The order in which the arguments themselves are evaluated

Note the asymmetry: `&&`, `||`, `?:`, and the comma operator are the *only* operators with defined ordering. Everything else — including `+` and `=` — leaves the order unspecified.

The classics, with real output

```
i = i++ + 1   -> 1
i = ++i + i++ -> 3
i++ + i++     -> 1
a[i]=i++      -> a=0 0 0, i=1
f(i++, i++)   -> f(1, 0)
```

Every single line above is undefined behaviour. The outputs are simply what one compiler happened to produce at one optimisation level on one day. GCC warned about all of them:

```
warning: operation on 'i' may be undefined [-Wsequence-point]
```

Why each is UB:

- **`i = i++ + 1`** — `i` is modified twice, once by `++` and once by `=`, with no sequence point between the two.
- **`i++ + i++`** — `i` is modified twice.
- **`a[i] = i++`** — `i` is read to compute the subscript, which is not "computing the value to be stored" into `i`, while `i` is also being modified.
- **`f(i++, i++)`** — the argument expressions are unsequenced relative to each other, so `i` is modified twice without an intervening sequence point.

> **Important:** this is not a case where "the compiler picks an order." UB means the compiler may do *anything* — including producing different results for the same expression in different places in the same program, or optimising on the assumption that it never happens. Don't reason about "what my compiler does."

Unspecified order is a separate problem

Even where there is no undefined behaviour, the *order of evaluation* of function arguments, and of the operands of most operators, is **unspecified**. Using functions `g`, `h`, and `k` that each print their own name before returning a value:

```
int r = add3(g(), h(), k());   /* g,h,k each print their name */
int s = g() + h() * k();
```

```
### -O0, -O1, -O2, -Os (all identical) ###
argument evaluation order: k h g -> result 6
in an expression:          g h k -> result 7
```

GCC evaluated the function arguments right to left, but the operands of the arithmetic expression left to right. Note carefully that in `g() + h() * k()`, precedence says the multiplication is *grouped* before the addition, but that does not mean `h` and `k` are *called* before `g`: grouping and evaluation order are independent. MSVC also tends to evaluate arguments right to left; some ARM ABIs go left to right. There is no guarantee — and the order can even differ between two calls in the same program.

This is *unspecified* behaviour, not undefined behaviour. The program is valid, and it will do one of the permitted things; you simply cannot predict which. But if the functions have side effects that interact, your program's output depends on the compiler, which makes it non-portable:

```
printf("%d %d\n", get_next(), get_next());   /* which value comes first? */
buf[pos++] = read_byte(&pos);                /*   unsequenced, UB */
```

The first line is merely unpredictable. The second is genuinely undefined, because `pos` is modified by `pos++` while `read_byte` may also be modifying it through the pointer, with nothing to order the two.

How to stay safe — four rules

1. **One side effect per expression.** If a statement modifies something, that should be all it does.
2. **Never use `++`/`--` inside a larger expression**, unless it is the idiomatic `*p++` or `arr[i++]` form in which the variable appears exactly once.
3. **Split it up.** Sequence points are free — each `;` is one:

   ```
   /* BAD  */ result = f(i++, i++);
   /* GOOD */ int a = i++; int b = i++; result = f(a, b);
   ```

4. **Compile with `-Wsequence-point`** (included in `-Wall`) and treat those warnings as errors.

These are fine, because each variable is modified once and not otherwise read:

```
i++;
arr[i++] = x;
*dst++ = *src++;              /* two different objects */
while ((c = getchar()) != EOF)
sum += arr[i++];
```

In the `getchar` loop, `c` is modified once, and the read of `c` by `!=` uses the value just assigned — the value of an assignment expression is well defined — so there is no conflict.

These are UB:

```
i = i++;
a[i] = i++;
f(i++, i++);
x = i++ + i;
printf("%d %d", i++, i++);
i = ++i;
```

Every one of these either modifies `i` twice or reads it for some purpose other than computing its new value, with no sequence point in between. Treat all of them as bugs, and rewrite them in the split-up style shown above.

4.8 Bit Manipulation

Bit manipulation is the daily toolkit of systems and embedded work. Hardware registers, network protocol headers, file formats, permission flags, compression, hashing, and graphics all require you to set, clear, and test individual bits, or to pack several small values into one integer. C's bitwise operators map directly onto single machine instructions, which is one reason C remains the language of choice for this kind of work.

The operators

Op	Name	Note
`&`	AND	clear bits (with an inverted mask)
`|`	OR	set bits
`^`	XOR	toggle bits
`~`	NOT	invert all bits (unary)
`<<`	left shift	multiply by 2^n
`>>`	right shift	divide by 2^n (see below)

Each binary operator works on corresponding bits of its two operands independently. AND produces a 1 only where both bits are 1; OR produces a 1 where either is; XOR produces a 1 where they differ. Combined with a *mask* — a value with 1s in exactly the positions you care about — these give you precise control over individual bits.

The four core operations — memorise these

```c
#define BIT(n) (1u << (n))

flags |=  BIT(n);    /* SET    */
flags &= ~BIT(n);    /* CLEAR  */
flags ^=  BIT(n);    /* TOGGLE */
if (flags & BIT(n))  /* TEST   */
```

- **Set:** OR with the mask. Bits that are 1 in the mask become 1; all others are unchanged.
- **Clear:** AND with the *inverted* mask. `~BIT(n)` has a 0 only at position *n*, so that bit is forced to 0 while every other bit is ANDed with 1 and unchanged.
- **Toggle:** XOR with the mask. XOR with 1 flips a bit; XOR with 0 leaves it alone.
- **Test:** AND with the mask. The result is non-zero exactly when the bit is set.

Watching a byte change step by step:

```
start                 0x00  00000000
SET    f |= (1<<3)    0x08  00001000
SET    f |= (1<<5)    0x28  00101000
TOGGLE f ^= (1<<3)    0x20  00100000
SET again             0x28  00101000
CLEAR  f &= ~(1<<5)   0x08  00001000
TEST   (f >> 3) & 1   = 1
TEST   !!(f & (1<<3))  = 1
```

The last two lines show two equivalent ways to get a clean 0-or-1 answer: shift the bit down to position 0 and mask it, or test it in place and normalise with `!!` (§4.2).

Note the `1u` in the `BIT` macro. **Use unsigned literals for shifts.** `1 << 31` on a 32-bit signed `int` is undefined behaviour, because it shifts a 1 into the sign bit, whereas `1u << 31` is perfectly well defined. For 64-bit values, use `1ULL << 63` or `UINT64_C(1) << 63`; plain `1u << 40` is undefined when `unsigned int` is 32 bits wide.

Multi-bit fields

Often a group of adjacent bits forms a small number — a mode, a channel, a priority. Extracting and replacing such fields combines shifting with masking:

```
uint8_t x = 0xA5;
x & 0x0F                          /* extract low nibble  → 0x05 */
x >> 4                            /* extract high nibble → 0x0A */
(x & 0xF0) | 0x03                 /* replace low nibble with 3 → 0xA3 */
(x >> shift) & ((1u << width) - 1)  /* generic field extract */
```

The expression `(1u << width) - 1` builds a mask of `width` ones: for a width of 3, `1u << 3` is `0b1000`, and subtracting 1 gives `0b0111`. Shifting the value down and then ANDing with that mask isolates the field.

A generic insert combines both steps — clear the field's old contents, then OR in the new value, shifted into position and trimmed to the field's width:

```
#define FIELD_SET(reg, shift, width, val) \
    ((reg) = ((reg) & ~(((1u << (width)) - 1) << (shift))) \
           | (((val) & ((1u << (width)) - 1)) << (shift)))
```

Shift semantics — the UB list

```
1u << 0..7        : 1 2 4 8 16 32 64 128
-8 >> 1           = -4          (arithmetic shift: sign preserved)
(unsigned)-8 >> 1 = 2147483644  (logical shift)
```

- **Right shift of a negative signed value is implementation-defined.** Every mainstream compiler performs an *arithmetic* shift, copying the sign bit into the vacated positions, so −8 >> 1 is −4. **Right shift of an unsigned value is always *logical*** (zero-filling), which is why the same bit pattern treated as unsigned gives a huge positive result.
- **`x << n` or `x >> n` where `n` is greater than or equal to the width of `x` is UB.** `int x = 1; x << 32;` is undefined — and on x86 the hardware masks the shift count to 5 bits, so you actually get `x << 0`, which is 1, not the 0 you might expect.
- **A negative shift count is UB.**
- **Left-shifting a negative value is UB** — in C99, C11, and still in C23 (C++20 defined it; C did not).
- **Remember the integer promotions:** with `uint8_t v = 0xFF;`, the expression `v << 4` is computed in `int`, giving `0xFF0`, not `0xF0`. The high bits are only lost if you store the result back into a `uint8_t`.

Classic tricks worth knowing

```
is power of 2? n && !(n & (n-1))  for 40 -> 0, for 64 -> 1
clear lowest set bit n & (n-1)  : 40 -> 32
isolate lowest set bit n & -n   : 40 -> 8
round up to next pow2 of 40     : 64
popcount(0xA5)=4 clz(1)=31 ctz(40)=3
swap without temp: a=7 b=3
abs via branchless: 9
```

```c
n && !(n & (n - 1))             /* is n a power of two? */
n & (n - 1)                     /* clear lowest set bit (Brian Kernighan's popcount loop) */
n & -n                          /* isolate lowest set bit */
x & (align - 1)                 /* offset within a power-of-2 alignment */
(x + align - 1) & ~(align - 1)  /* round up to alignment */
x ^= y, y ^= x, x ^= y          /* XOR swap — a party trick, NOT faster, and broken if x and y alias
*/
```

The key to most of these is what happens when you subtract 1 from a binary number: the lowest set bit becomes 0, and every bit below it becomes 1. So `n & (n - 1)` removes the lowest set bit, and a power of two — which has exactly one set bit — becomes zero. (Take 40, binary `101000`: `n - 1` is `100111`, and ANDing gives `100000`, which is 32.) Repeating `n &= n - 1` until `n` is zero counts the set bits in as many steps as there are ones, which is Brian Kernighan's population-count method.

The alignment formulas are used constantly in allocators and I/O code; they work only when `align` is a power of two. The XOR swap is a well-known curiosity: it is no faster than using a temporary variable on modern hardware, and if `x` and `y` are the same object it zeroes it.

Prefer compiler builtins to hand-rolled loops — they compile to single instructions on hardware that supports them:

```c
__builtin_popcount(x)   /* count set bits       → POPCNT */
__builtin_clz(x)        /* count leading zeros  → LZCNT/BSR  (UB if x == 0!) */
__builtin_ctz(x)        /* count trailing zeros → TZCNT/BSF  (UB if x == 0!) */
__builtin_bswap32(x)    /* byte swap — endian conversion */
```

C23 standardises these operations in `<stdbit.h>`: `stdc_count_ones`, `stdc_leading_zeros`, `stdc_trailing_zeros`, `stdc_bit_ceil`, `stdc_has_single_bit`, and others. Use those when you can target C23; unlike the builtins, they are portable across compilers and are defined for a zero argument.

> ⚠ `__builtin_clz(0)` and `__builtin_ctz(0)` are undefined. Guard the zero case.

4.9 Bitfields vs Manual Bit Manipulation

C offers a second, more convenient-looking way to work with groups of bits: *bitfields*, which let you declare structure members with an explicit width in bits.

Bitfields

```
struct BF { unsigned a:1; unsigned b:3; unsigned c:4; unsigned d:24; };
```

```
sizeof(struct BF) = 4   (1+3+4+24 = 32 bits)
a=1 b=5 c=0xA d=0x123456 -> raw bytes = 0x123456AB
```

Four fields totalling 32 bits fit in a single 4-byte structure, and you access them with ordinary member syntax — `x.b = 5` — with the compiler generating all the shifting and masking for you. On this compiler, `a` occupied the lowest bit, followed by `b`, `c`, and `d` in turn, which is why the raw value reads `0x123456AB`.

The syntax is convenient — but the portability guarantees are almost nonexistent.

Everything the standard leaves unspecified

1. **Bit order within a storage unit.** Whether `a` occupies the lowest or the highest bit is implementation-defined. Your structure will not reliably match a hardware register or a wire format across compilers, or even across architectures with the same compiler.

2. **Whether a field may straddle a storage-unit boundary.**

3. **The alignment and total size of the containing structure.**

4. **The signedness of plain `int` bitfields:**

   ```
   struct { int s:1; unsigned u:1; } t;
   t.s = 1; t.u = 1;
   ```

   ```
   int a:1 set to 1 reads as -1 ; unsigned a:1 reads as 1
   ```

   You stored 1 and read back −1. A 1-bit *signed* field can hold only 0 and −1: its single bit is the sign bit. Whether a plain `int` bitfield is signed is up to the implementation, so **always write `unsigned` or `signed` explicitly — never plain `int` — in a bitfield.**

5. **Overflow silently truncates:**

   ```
   x.b = 9 -> reads back as 1   /* 9 doesn't fit in 3 bits */
   warning: unsigned conversion ... changes value from '9' to '1' [-Woverflow]
   ```

   9 is `1001` in binary; only the low three bits, `001`, fit.

6. **You cannot take the address of a bitfield** (`&x.a` is a compile error). That means no pointers to bitfields, no `sizeof` on them, and awkward interactions with generic code such as `memcpy` or functions that take output parameters.

7. **They are not atomic.** Writing one bitfield may read, modify, and write back the entire storage unit containing it, so two threads updating *different* bitfields in the same structure can race and overwrite each other's changes. (The C11 memory model treats a run of adjacent bitfields as a single memory location; only a zero-width field `: 0` between them separates them into different locations.)

The unnamed zero-width field `unsigned :0;` forces the next field to start in a new storage unit:

```
sizeof(struct BF2) = 8   (unnamed :0 forces alignment)
```

When to use which

Use bitfields for	Use manual masks for
Internal, single-compiler data compaction	Hardware registers
Readability where layout doesn't matter	Network protocols / file formats
Boolean flag clusters in a struct	Anything crossing a compiler or ABI boundary
	Code that must be portable
	When you need atomicity

The dividing line is whether the exact bit layout is part of a contract with something outside your program. If it is only your own program that reads and writes the data, bitfields are a pleasant convenience. If a device, another program, or a file on disk depends on where each bit sits, you need a layout you control.

The professional pattern for hardware and protocols uses explicit shifts and masks, whose behaviour is fully defined:

```c
#define STATUS_READY_SHIFT 0
#define STATUS_READY_MASK  (1u << STATUS_READY_SHIFT)
#define STATUS_MODE_SHIFT  1
#define STATUS_MODE_MASK   (0x7u << STATUS_MODE_SHIFT)

static inline unsigned status_mode(uint32_t reg)
{ return (reg & STATUS_MODE_MASK) >> STATUS_MODE_SHIFT; }

static inline uint32_t status_set_mode(uint32_t reg, unsigned mode)
{ return (reg & ~STATUS_MODE_MASK) | ((mode << STATUS_MODE_SHIFT) & STATUS_MODE_MASK); }
```

Each field gets a named shift and a named mask, and small `static inline` accessor functions hide the arithmetic from the rest of the code. It is more verbose than a bitfield, but it produces identical bits on every compiler and every architecture. That is the trade you want at an interface boundary. (Module 27 applies this pattern to real microcontroller registers.)

Module 4 Rules

#	Rule
1	Cast an operand, not the result, to get floating-point division
2	`%` follows the sign of the dividend — use `((a%m)+m)%m` for true modulo
3	Put the guard first in `&&` chains; never use `&` where you meant `&&`

#	Rule
4	Compound assignment silently narrows — watch `char`/`short` targets
5	Parenthesise every bitmask test: `(flags & MASK) == 0`
6	One side effect per expression; never use `i` twice in a statement that modifies it
7	Argument evaluation order is unspecified — GCC happens to go right to left
8	Shift with unsigned types; `n >= width` is UB; `1u << 31`, not `1 << 31`
9	`__builtin_clz(0)` / `__builtin_ctz(0)` are UB
10	Never use bitfields for hardware or wire formats — use explicit shifts and masks

Standing build command, updated:

```
gcc -std=c17 -Wall -Wextra -Wpedantic -Wshadow -Wconversion \
    -Wsequence-point -Wparentheses -g -fsanitize=address,undefined prog.c -o prog
```

(`-Wsequence-point` and `-Wparentheses` are already part of `-Wall`; listing them explicitly documents that you depend on them.)

Practice

1. **Modulo table.** Write a program that prints `i % 5` and `((i % 5) + 5) % 5` for `i` from −10 to 10. Explain where the two diverge, and why the second is what a hash table needs.

2. **Short-circuit proof.** Write `int noisy(const char *tag, int v)`, which prints its tag and returns `v`. Build expressions using `&&`, `||`, `&`, `|`, and `?:` that demonstrate exactly which operands are evaluated. Predict each result before running it.

3. **Sequence point hunt.** Take the six UB expressions from §4.7. Compile each with `-Wall` and record which ones GCC catches. Then rewrite each one to be well defined while preserving the intended meaning.

4. **Precedence quiz, no compiler.** Fully parenthesise each of these, then verify your answers:

```
a & b == c        *p++ = *q++          a << b + c
!x & y            a ? b : c ? d : e    x = y = z & w
~a & b | c        -a % b * c
```

5. **Bit toolkit.** Implement and test: `set_bit`, `clear_bit`, `toggle_bit`, `test_bit`, `extract_field(v, shift, width)`, `insert_field(v, shift, width, val)`, `count_bits` (both a naive loop and the `n & (n-1)` version), `is_power_of_2`, and `next_power_of_2`. Compare your `count_bits` against `__builtin_popcount` for all 65,536 16-bit values.

6. **Bitfield portability experiment.** Define a 32-bit bitfield structure, set known values, and dump the raw bytes with `memcpy`. Then compile with `-mbig-endian` if you have a suitable cross-compiler, or simply reason about what would change. Rewrite the same layout using explicit shifts and confirm that the manual version gives identical, predictable bytes.

7. **Real-world register model.** Write an API for a hypothetical UART control register in which bits 0–1 hold the parity (0 = none, 1 = even, 2 = odd), bit 2 holds the number of stop bits, bits 3–6 hold the baud divisor, and bit 7 is the enable flag. Provide `static inline` getters and setters using masks and shifts, plus `_Static_assert`s that verify your masks do not overlap.

Module 4 in Summary

Integer division truncates toward zero, and `%` takes the sign of the dividend, so true modular arithmetic needs an adjustment. `&&` and `||` short-circuit and evaluate left to right, which makes “guard first” conditions safe; the bitwise `&` and `|` do neither. Compound assignment evaluates its left side once but silently narrows the result. The conditional operator evaluates one branch, but its type comes from both.

`sizeof` never evaluates its operand (except for VLAs), and the array-length idiom works only on true arrays. Alignment determines where objects may live and causes padding inside structures.

Precedence governs grouping, not evaluation order — and the low precedence of `&`, `|`, and `<<` is a trap to be defeated with parentheses. Modifying a variable twice, or reading it while modifying it, without an intervening sequence point is undefined behaviour, and the order in which arguments are evaluated is unspecified. Keep one side effect per expression.

Finally, bit manipulation with unsigned masks and shifts is fully defined and portable, while bitfields are convenient only when no outside party depends on their layout.

In Module 5, we move to Control Flow — `if`, `switch` and its fall-through, loops, and the legitimate uses of `goto`.

Module 5 — Control Flow

In this module:

- `if`/`else`, `switch` (fallthrough, jump tables, Duff's device)
- `for`, `while`, `do-while`, nested loops
- `break`, `continue`, `goto` (and its legitimate use: cleanup chains)
- Labels, unreachable code

All outputs in this module are real GCC 13.3 output.

Control flow is the part of C that most closely resembles every other language you may know: `if`, `while`, and `for` look and behave almost identically in Java, JavaScript, and C++. That familiarity is precisely the danger. C's versions come with a handful of sharp edges — stray semicolons, fall-through in `switch`, a `continue` that behaves differently in different loops — that cause real, sometimes catastrophic, bugs.

This module also contains the `goto` cleanup pattern, which is the single most important idiom in professional C. It is used everywhere in the Linux kernel, OpenSSL, curl, and SQLite, and once you understand it you will see it in almost every serious C codebase you open.

5.1 `if` / `else`

```
if (expr) statement
if (expr) statement else statement
```

The condition can be any scalar expression — an integer, a floating-point value, or a pointer. Non-zero means true, and zero (or a null pointer) means false. There is no requirement for the condition to be a `bool`, which is why `if (ptr)` and `if (count)` are idiomatic C.

The three classic traps

```
TRAP 1: stray semicolon
  this ALWAYS runs, regardless of x

TRAP 2: missing braces + added line
  B <- runs even though y==0 (not part of the if!)

TRAP 3: dangling else binds to the NEAREST if
  (nothing printed — the else belonged to the inner if)
```

Trap 1 — `if (x > 0);`. The semicolon is a complete statement in its own right — an empty one. The `if` now controls that empty statement and nothing else, and the "body" on the following lines always executes, whatever the value of `x`. The indentation makes it look conditional; it isn't.

Trap 2 — missing braces. Without braces, an `if` controls exactly one statement. Someone later adds a second indented line beneath a braceless `if`, and that line silently escapes the conditional: it runs every time. This is exactly the Apple “goto fail” SSL bug (CVE-2014-1266). A duplicated `goto fail;` line after a braceless `if` caused the code to skip the final step of certificate verification unconditionally, leaving secure connections on Apple’s iOS and macOS devices open to interception.

Trap 3 — the dangling `else`. The grammar says that an `else` binds to the *nearest* unmatched `if`, regardless of indentation:

```
if (a)
    if (b) printf("both\n");
else printf("...\n");        /* belongs to `if (b)`, NOT `if (a)` */
```

The indentation suggests that the `else` handles the case where `a` is false. In reality it handles the case where `a` is true and `b` is false; when `a` is false, nothing happens at all.

GCC catches all three

```
warning: suggest braces around empty body in an 'if' statement [-Wempty-body]
warning: this 'if' clause does not guard... [-Wmisleading-indentation]
warning: suggest explicit braces to avoid ambiguous 'else' [-Wdangling-else]
```

`-Wmisleading-indentation` is part of `-Wall`; it was added in GCC 6, in direct response to goto fail. It compares your indentation with the actual structure of the code and warns when they disagree.

> **The rule that eliminates all three:** always use braces, even for one-line bodies. This is mandatory in MISRA C, required by the Linux kernel style for multi-line bodies, and recommended by virtually every professional style guide. Braces cost one line; the bugs they prevent have cost companies millions.

`else if` is not a keyword

```
if (a) { } else if (b) { } else { }
```

This is simply an `else` whose single statement happens to be another `if`. C has no `elif` or `elseif`; the familiar chain is a series of nested `if` statements formatted to look flat. For more than about three branches that all test the same value, prefer a `switch` or a lookup table.

5.2 `switch`

```
switch (integer_expression) {
case constant1:
    statements
    break;
case constant2:
    statements
```

```
    break;
default:
    statements
}
```

The mental model that makes `switch` make sense

`switch` is a *computed goto*, not a chain of `if`s. It evaluates the controlling expression and then jumps directly to the matching `case` label — into the middle of a block. Everything unusual about `switch` follows from this one idea:

```
int n = 2;
switch (n) {
    printf("never printed\n");   /* no label — unreachable */
case 2:
    printf("case 2 reached\n");
}
```

```
warning: statement will never be executed [-Wswitch-unreachable]
case 2 reached
```

Code before the first `case` label is dead: the `switch` always jumps to a label, and nothing can ever jump to that code.

Requirements

- The controlling expression must have an integer type (including `char` and enums). No floats, no strings, no pointers.
- `case` labels must be *integer constant expressions* — no variables and no function calls — because the compiler must know all the values at compile time.
- Case labels must be unique within a `switch`; a duplicate is a compile error, not a warning.
- The integer promotions (Module 2) are applied to the controlling expression.
- `default` may appear anywhere, not only last, but there may be only one per `switch`.

Fallthrough — the defining feature

```
switch (c) {
case 1: case 2: case 3:
    printf("small ");
    /* fall through */
case 4:
    printf("<=4 ");
    break;
case 5:
    printf("five ");
    /* NO break */
default:
    printf("other");
    break;
}
```

```
1: small <=4
2: small <=4
3: small <=4
4: <=4
5: five other     ← fell through into default
6: other
```

Because a `case` label is only a jump target, it does not end the code before it. Without a `break`, execution simply continues into the next case's code. There are two distinct kinds of fall-through:

1. **Grouped labels** (`case 1: case 2: case 3:`) — several labels with no code between them. This is universally understood, perfectly clear, and never warned about.
2. **Genuine fall-through** — some code, then no `break`, so execution runs on into the next case. This is almost always a bug.

GCC's `-Wimplicit-fallthrough` (part of `-Wextra`) catches the second kind:

```
warning: this statement may fall through [-Wimplicit-fallthrough=]
```

Notice that it warned about `case 5` but not about the fall-through from `case 3` into `case 4`, which was marked with a `/* fall through */` comment. GCC recognises that exact comment form as a statement of intent. Better, more robust options exist:

```c
case 3:
    do_something();
    __attribute__((fallthrough));   /* GCC/Clang */
    /* or, in C23: */
    [[fallthrough]];
case 4:
```

These are part of the language (or a compiler extension) rather than a comment, so they survive tools that strip comments and cannot be misspelled without a diagnostic.

> **Style rule:** every case ends in `break`, `return`, `continue`, `goto`, or an explicit fall-through marker. No exceptions. Put a `break` after `default` too, so that adding a new case below it later does not create a fall-through bug.

`switch` on an enum — free exhaustiveness checking

```c
enum Color { RED, GREEN, BLUE };
switch (c) {
case RED:   return "red";
case GREEN: return "green";
/* BLUE missing */
}
```

```
warning: enumeration value 'BLUE' not handled in switch [-Wswitch]
```

This is a major professional technique. `-Wswitch` (part of `-Wall`) tells you about every `switch` in the program that needs updating when you add a value to an enum — but *only if you omit `default`*. Adding a `default:` label tells the compiler that every other value is handled, and the warning disappears.

```c
/* GOOD: compiler tracks exhaustiveness for you */
switch (state) {
case IDLE:    ...; break;
case RUNNING: ...; break;
case DONE:    ...; break;
```

```
}
/* handle invalid values after the switch if needed */
```

Combine it with `-Werror=switch` to make a missing case a build failure. In a large program with a state machine that is switched on in a dozen places, this turns “add a new state” from a hunt through the codebase into a list of compile errors.

Declarations inside `switch` — a real gotcha

```c
switch (k) {
case 1:
    ;           /* C17 needs a statement before a declaration after a label */
    int x = 1;
    printf("x=%d\n", x);
    break;
case 2:
    printf("x is IN SCOPE here but UNINITIALIZED\n");
    break;
}
```

```
x is IN SCOPE here but UNINITIALIZED: skipped its initializer
```

The entire body of a `switch` is a single block, so the scope of `x` extends from its declaration to the closing brace — including `case 2`. But when `k` is 2, the `switch` jumps directly to `case 2`, skipping the line that initializes `x`. The result is a variable that is in scope but indeterminate, and reading it is undefined behaviour.

(The lone `;` is there because, before C23, a label had to be followed by a *statement*, and a declaration is not a statement. The empty statement satisfies the rule.)

Fix: give each case its own block.

```c
case 1: {
    int x = 1;
    printf("x=%d\n", x);
    break;
}
```

Now `x`’s scope ends at the case’s closing brace, and no other case can see it. (C23 relaxed the “label must precede a statement” rule, so the `;` hack is no longer needed there — but the per-case block is still the right fix for the scope problem.)

Jump tables — what the compiler actually generates

How does a `switch` perform? It depends on the case values.

Dense case values → the compiler builds a *jump table*: an array of addresses indexed by the case value, so that selecting the case takes one indexed, indirect jump — O(1), regardless of how many cases there are.

```
dense:
    cmp   edi, 7
    ja    .L2                       ; range check → default
    lea   rdx, .L4[rip]             ; address of the table
    movsx rax, DWORD PTR [rdx+rdi*4]   ; table[x]
    add   rax, rdx
    jmp   rax                       ; ONE indirect jump
```

```
.L4: .long .L11-.L4                    ; the table itself, in .rodata
     .long .L10-.L4
     .long .L9-.L4
     ...
```

The code first checks that the value is within the table's range (a single unsigned comparison handles negative values too), jumps to `default` if not, and otherwise looks up the target address and jumps to it.

Sparse case values → a comparison chain or a binary search:

```
sparse:
    cmp edi, 1000
    je  .L5
    cmp edi, 999999
    je  .L5
    cmp edi, 1
    ...
```

A table covering 1 to 999,999 would be absurdly large, so the compiler falls back to comparisons.

Practical implications:

- **A dense `switch` over 50 cases is genuinely faster than an `if/else if` chain.** Prefer it for opcode dispatch in interpreters, for state machines, and for parsers.
- **Keep enum values contiguous, starting near 0,** so that the compiler can build a table.
- **Compilers are smart.** A first dense test had cases that returned 10, 20, 30, …; GCC recognised the arithmetic pattern and emitted `lea eax, 5[rdi+rdi*4]` — a single calculation, with no branch or table at all. Always check the actual output before trying to optimise by hand.

Duff's device

Duff's device is a famous 1983 loop-unrolling trick by Tom Duff. It exploits the fact that `case` labels can appear *inside a nested loop* within the `switch` body:

```
void duff_copy(char *to, const char *from, size_t count)
{
    size_t n = (count + 7) / 8;
    switch (count % 8) {
    case 0: do { *to++ = *from++;
    case 7:      *to++ = *from++;
    case 6:      *to++ = *from++;
    case 5:      *to++ = *from++;
    case 4:      *to++ = *from++;
    case 3:      *to++ = *from++;
    case 2:      *to++ = *from++;
    case 1:      *to++ = *from++;
            } while (--n > 0);
    }
}
```

```
copied 21 bytes: "ABCDEFGHIJKLMNOPQRSTU"
matches source? yes
```

It works. The loop body copies eight bytes per iteration. To handle a count that is not a multiple of eight, the `switch` jumps into the *middle* of the loop body on the first pass, copying just the leftover `count % 8` bytes; after that, the `do-while` runs normally, eight bytes at a time. For 21 bytes, the `switch` jumps to `case 5`, copies 5 bytes, and then the loop runs twice more for the remaining 16. It even compiles cleanly with `-Wall`, because every case falls through into the next one by design.

Do not use it. It is here so that you can recognise it in old code and explain it in an interview. Modern compilers unroll loops far better than you can by hand, and `memcpy` compiles to a vectorised instruction sequence. Today, Duff's device typically makes code *slower*, because its unusual control flow defeats the optimiser.

Its real historical significance is conceptual: it proved that `switch` and loops are independent grammatical constructs that can interleave arbitrarily — which is exactly what the "computed goto" mental model predicts.

5.3 Loops

The three forms

```
for (init; condition; increment) body
while (condition) body
do body while (condition);   /* note the semicolon */
```

A `for` loop is really a `while` loop with its setup and update gathered into the header: `init` runs once, `condition` is tested before every iteration, and `increment` runs after every iteration. Any of the three parts may be left empty.

`for (;;)` and `while (1)` are both infinite loops. `for (;;)` is slightly preferred by convention, because some older compilers and lint tools warned about the constant condition in `while (1)`.

A `do-while` loop runs its body *at least once*, because it tests its condition at the bottom rather than the top:

```
while(k<5) with k=100: body ran 0 times
do-while body ran once even though k=100
```

Its main real uses are menu and retry loops — "do this, then ask whether to do it again" — and the `do { } while (0)` macro idiom (Module 12). Don't forget the semicolon after its closing parenthesis.

Half-open ranges — write loops this way

```
for (size_t i = 0; i < n; i++)   /* [0, n) */
```

```
10 20 30 40 50
count = n - 0 = 5, no off-by-one possible
```

The notation `[0, n)` means "from 0 inclusive to `n` exclusive" — a *half-open* range. It is the correct default for almost every loop, for four reasons:

- **The count is exactly `end - start`.** There is no `+ 1` to forget.
- **An empty range is simply `start == end`,** rather than the awkward `end < start`.
- **Adjacent ranges join cleanly:** `[0, n)` followed by `[n, m)` covers everything once, with no gap and no overlap.
- **It matches how `strlen`, `memcpy`, and every other C API works** — sizes and lengths, not "last index."

Use `<`, never `<=`, with a count. `i <= n` is the single most common source of off-by-one errors: it runs one iteration too many and reads one element past the end of the array.

Reverse iteration with `size_t`

As Module 2 showed, `for (size_t i = n-1; i >= 0; i--)` is an infinite loop, because an unsigned `i` is always `>= 0`. There are three correct forms, all verified:

```c
for (size_t i = n; i-- > 0; )  printf("%d ", arr[i]);              /* A: idiomatic */
{ size_t i = n; while (i > 0) { i--; printf("%d ", arr[i]); } }    /* B: explicit */
for (size_t i = n; i > 0; i--) printf("%d ", arr[i-1]);            /* C: offset */
```

```
A: 50 40 30 20 10
B: 50 40 30 20 10
C: 50 40 30 20 10
```

Form A is the standard C idiom. Read `i-- > 0` as "i goes down to zero": the test uses the *old* value of `i` and then decrements it, so the body sees `n - 1` down to 0, and the loop stops cleanly when `i` reaches 0 — before it could wrap around. Form B spells out the same steps. Form C keeps `i` in the range `n` to 1 and subtracts one when indexing.

Loop traps

Stray semicolon:

```c
for (size_t i = 0; i < n; i++);
    sum += 1;
```

```
warning: this 'for' clause does not guard... [-Wmisleading-indentation]
sum = 1 (loop body was empty!)
```

Exactly as with `if`, the semicolon becomes the entire loop body. The loop runs `n` times doing nothing, and then `sum += 1` runs once.

Modifying the loop variable inside the body:

```c
for (int i = 0; i < 3; i++) { printf("i=%d ", i); if (i==1) i++; }
```

```
i=0 i=1     ← i=2 skipped entirely
```

This is legal, occasionally intentional, and usually a bug: a reader looking at the loop header expects `i` to take every value from 0 to 2. If you need to skip iterations, use `continue` and keep all changes to the loop variable in the header.

Floating-point loop counters — never do this:

```
for (double x = 0.0; x != 1.0; x += 0.1)                /* never terminates (Module 2) */
for (int i = 0; i < 10; i++) { double x = i * 0.1; }     /* correct */
```

Because 0.1 is not exactly representable, adding it ten times does not produce exactly 1.0, so `x != 1.0` stays true forever. Count with an integer and compute the floating-point value from it.

Recomputing an invariant in the condition:

```
for (size_t i = 0; i < strlen(s); i++)   /* O(n²) — strlen runs every iteration */

size_t len = strlen(s);
for (size_t i = 0; i < len; i++)         /* O(n) */
```

The loop condition is re-evaluated on every iteration, and `strlen` must scan the whole string each time to find its end. For a 100,000-character string, the first version performs around five billion character comparisons. (The optimiser can sometimes hoist the call out of the loop, but only when it can prove that `s` is not modified inside it. Don't rely on that.)

Declaration in the `for` header (C99+)

```
for (int i = 0; i < n; i++) { }   /* i's scope ends with the loop */
```

Always do this. It gives the loop variable the tightest possible scope, prevents accidental reuse of a stale value after the loop, and eliminates a whole class of bugs in which two loops share a counter by mistake.

5.4 `break` and `continue`

`break` exits only the innermost enclosing loop or `switch`

```
break only exits the INNERMOST loop:
  outer i=0 still runs
  outer i=1 still runs
  outer i=2 still runs
```

A `break` inside a nested loop leaves only the loop that directly contains it; the outer loop carries on. And inside a `switch` that is inside a loop, `break` exits the *switch*, not the loop:

```
for (int x=0; x<4; x++) { switch(x) { case 2: break; default: printf("%d ",x); } }
```

```
0 1 3 <- 2 skipped but loop continued
```

This trips people up constantly: the `break` in `case 2` was intended to stop the loop, but it only ended the `switch`. If you need to leave a loop from inside a `switch`, use `goto`, a flag, or a `return`.

`continue` — different in `for` and `while`

`continue` skips the rest of the current iteration and moves on to the next. In a `for` loop, that still runs the increment expression:

```
0 1 3 4     /* i==2 skipped, loop still advanced */
```

In a `while` loop, `continue` jumps straight back to the condition — skipping everything else in the body, *including your manual increment*:

```c
int i = 0;
while (i < 5) {
    if (i == 2) continue;   /* i never increments → INFINITE LOOP */
    printf("%d ", i);
    i++;
}
```

```
0 1 (would spin forever; bailing)
```

Once `i` reaches 2, the `continue` fires on every pass, `i++` is never reached, and the loop spins forever.

> **Rule:** if a `while` loop body contains `continue`, put the increment at the top of the body or convert the loop to a `for` loop, where the increment lives in the header and always runs.

Breaking out of nested loops

C has no labelled `break`, so leaving several loops at once takes one of three approaches.

Way 1 — a flag variable (verbose, error-prone, and it adds a condition to every loop header):

```c
int found = 0;
for (int i=0; i<3 && !found; i++)
    for (int j=0; j<4; j++)
        if (m[i][j]==target) { found=1; fi=i; fj=j; break; }
```

Way 2 — `goto` (idiomatic C, and what the Linux kernel does):

```c
for (int i=0; i<3; i++)
    for (int j=0; j<4; j++)
        if (m[i][j]==target) { fi=i; fj=j; goto found_it; }
printf("not found\n");
goto done;
found_it:
printf("found at [%d][%d]\n", fi, fj);
done:
```

```
found at [1][2] - clean single exit
```

Way 3 — extract the loops into a function and `return` from the middle. This is often the cleanest option of all, and it makes the search testable in isolation.

5.5 `goto` — and When It's Actually Correct

In 1968, Edsger Dijkstra published a letter titled "Go To Statement Considered Harmful." It argued, convincingly, that unrestricted jumps make programs hard to reason about, and it left a lasting taboo on `goto` in every language that has one. In C, that taboo is wrong in one specific and extremely important case.

The rules

- `goto` targets a *label*, and labels have function scope (Module 3): a label is visible anywhere in its function, even before the point where it is written.
- You cannot jump between functions. (That is what `setjmp`/`longjmp` do, with considerable risks; see Module 15.)
- You cannot jump into the scope of a variable-length array, because that would skip the code that allocates it.
- You *can* jump into a block — which is how Duff's device and the switch-into-loop trick work.

The legitimate use: cleanup chains

C has no destructors, no RAII, and no `finally`. When a function acquires several resources — opens a file, allocates buffers, locks a mutex — and any step can fail, you must release everything acquired so far, in reverse order, on every failure path.

Without `goto`, you get either deeply nested `if` statements (each success opening a new level of indentation) or duplicated cleanup code at every early `return`. Both are exactly where resource leaks hide: every duplicate is one more place to forget something when the function changes.

With `goto`, there is one cleanup chain at the end of the function, and each label unwinds one more level:

```c
int process(const char *path, int fail_at)
{
    FILE *fp = NULL; char *buf1 = NULL, *buf2 = NULL;
    int rc = -1;                    /* pessimistic default */

    fp = fopen(path, "r");
    if (!fp) goto out;              /* nothing acquired yet */
    if (fail_at == 1) goto out_file;

    buf1 = malloc(1024);
    if (!buf1) goto out_file;
    if (fail_at == 2) goto out_buf1;

    buf2 = malloc(4096);
    if (!buf2) goto out_buf1;
    if (fail_at == 3) goto out_buf2;
```

```
    /* ... real work ... */
    rc = 0;                          /* success */

out_buf2: free(buf2);
out_buf1: free(buf1);
out_file: fclose(fp);
out:      return rc;
}
```

Read the labels from the bottom up and you see the acquisitions in order; read them from the top down and you see the releases in reverse order. A failure after acquiring the file and the first buffer jumps to `out_buf1`, which frees `buf1`, then falls through to close the file, then returns. The success path simply falls through the entire chain, releasing everything. (The `fail_at` parameter injects artificial failures, so that every error path can be tested.)

Verified across all four paths:

```
--- fail_at=0 --- acquired: file, buf1, buf2 → released: buf2, buf1, file → rc=0
--- fail_at=1 --- acquired: file             → released: file             → rc=-1
--- fail_at=2 --- acquired: file, buf1       → released: buf1, file       → rc=-1
--- fail_at=3 --- acquired: file, buf1, buf2 → released: buf2, buf1, file → rc=-1
```

```
$ ASAN_OPTIONS=detect_leaks=1 ./cleanup
ASan/LSan: CLEAN - zero leaks on all 4 paths
```

An honest mistake worth showing you

The first version of this example had the `fail_at == 3` check jump to `out_buf1` instead of `out_buf2`. LeakSanitizer caught it immediately:

```
==643==ERROR: LeakSanitizer: detected memory leaks
Direct leak of 4096 byte(s) in 1 object(s)
Direct leak of 1024 byte(s) in 1 object(s)
SUMMARY: AddressSanitizer: 5120 byte(s) leaked in 2 allocation(s).
```

That is the pattern's one real hazard: jumping to the wrong label leaks the resource you just acquired. Two things protect you:

1. **Name labels after what they release,** not `err1`/`err2`. `out_buf2` obviously frees `buf2`, so `if (!buf2) goto out_buf2;` looks wrong at a glance — you cannot free something that failed to allocate.
2. **Run LeakSanitizer or Valgrind on your error paths, not just the happy path.** Fault injection, like the `fail_at` parameter, is a genuine testing technique: inject a failure at every allocation in turn and verify zero leaks each time.

Why this pattern is correct, not a compromise

- **One exit point.** It is easy to add logging, tracing, or a debugger breakpoint in one place that sees every return.
- **Cleanup is written once.** Adding a fifth resource means one new label, not five new `free` calls scattered across the function.
- **The order is visually enforced.** The labels appear in reverse order of acquisition.
- **It's what everyone does.** Searching the Linux kernel source for `goto out` returns tens of thousands of hits. OpenSSL, curl, SQLite, and glibc all use the same pattern.

Supporting conventions

```
FILE *fp = NULL;   /* initialize to a safe sentinel */
int rc = -1;       /* default to failure — success is explicit */

/* free(NULL) and fclose on NULL-checked pointers are safe, so uniform
   labels work even when some resources were never acquired */
out:
    free(buf);           /* free(NULL) is a guaranteed no-op */
    if (fp) fclose(fp);  /* fclose(NULL) is NOT safe — check it */
    return rc;
```

Initializing every resource variable to a “nothing acquired” value, and the return code to failure, means that any path that forgets to set `rc = 0` fails safely rather than reporting a false success.

A common simplification is a single cleanup label that frees everything, relying on `free(NULL)` being a no-op:

```
    int rc = -1;
    char *a = NULL, *b = NULL, *c = NULL;
    if (!(a = malloc(N))) goto out;
    if (!(b = malloc(N))) goto out;
    if (!(c = malloc(N))) goto out;
    rc = 0;
out:
    free(c); free(b); free(a);
    return rc;
}
```

This is simpler, but it only works when every cleanup operation is safe to perform on a resource that was never acquired. Use the chained version for operations like `fclose`, `close`, `pthread_mutex_unlock`, or `munmap`, which are not.

Where `goto` is not acceptable

- Jumping backwards to build a loop — use a loop.
- Jumping into the middle of an unrelated block.
- Skipping over initializations (as in the `switch` gotcha in §5.2).
- As a substitute for restructuring genuinely tangled logic.

> **The professional consensus:** `goto` forward, to a cleanup label at the end of the function, is correct C. Everything else needs a strong justification.

5.6 Labels and Unreachable Code

Labels

```
identifier: statement
```

C has three kinds of label: ordinary `goto` labels (which have function scope), `case` labels, and `default`. In C17, a label must be followed by a statement — hence the `;` hack before a declaration seen in §5.2. C23 removes this restriction, so a label directly before a closing brace (`label: }`) or a declaration is now legal.

Unreachable code

```
int f(int x)
{
    if (x > 0) return 1;
    else       return 0;
    printf("unreachable\n");   /* dead */
    return 2;
}
```

Every path through the `if` returns, so the final two lines can never run. Dead code like this is usually harmless in itself, but it is a signal that something is wrong: a misplaced `return`, an inverted condition, or a leftover from a refactor. Ask why the code is there before deleting it.

Note that GCC's `-Wunreachable-code` option is currently accepted but does nothing — the analysis was removed because the optimiser made it too unreliable. Clang implements the warning properly, and static analysers (clang-tidy, cppcheck, Coverity) catch unreachable code reliably.

Falling off the end of a non-`void` function

```
int g(int x)
{
    switch (x) { case 1: return 1; case 2: return 2; }
    /* no return */
}
```

```
warning: control reaches end of non-void function [-Wreturn-type]

g(99)=0   <- read a garbage register, UB
```

For any value other than 1 or 2, `g` reaches its closing brace without returning a value. If the caller uses the result, this is undefined behaviour. It "worked" here by accident: the register that holds return values, `eax`, happened to contain 0 at that moment. On another day, with another compiler or optimisation level, it will contain something else — or the optimiser will assume the path is impossible and generate code accordingly.

> **Make this an error:** `-Werror=return-type`. It costs nothing and eliminates a real class of undefined behaviour. (Remember that `main` is the one exception: in C99 and later, falling off the end of `main` returns 0.)

`_Noreturn` / `[[noreturn]]`

Marking a function as never returning tells the compiler that control cannot come back from a call to it. That enables better optimisation and suppresses spurious "control reaches end" warnings in the functions that call it:

```c
_Noreturn void die(const char *msg)   /* C11 */
{ fprintf(stderr, "fatal: %s\n", msg); exit(1); }

[[noreturn]] void die(const char *msg);   /* C23 */
```

Standard library examples include `exit`, `abort`, `_Exit`, `longjmp`, and `thrd_exit`. After a call to one of these, the following code really is unreachable, and the compiler knows it.

Also useful are `__builtin_unreachable()` (GCC/Clang) and C23's `unreachable()` from `<stddef.h>`. Each is a *promise* that a particular path is never taken, which lets the compiler delete the checks leading to it entirely. Reaching one is undefined behaviour, so use them only where you are certain — for example, after a `switch` that provably handles every possible value.

Module 5 Rules

#	Rule
1	Always brace `if`/`else`/loop bodies, even one-liners
2	Watch for a stray `;` after `if (...)`, `for (...)`, `while (...)`
3	Every case ends in `break`/`return`/an explicit fall-through marker — including `default`
4	Omit `default` when switching on an enum, so `-Wswitch` enforces exhaustiveness
5	Give each case its own `{ }` block if it declares variables
6	Use half-open ranges `[0, n)` and `<`, never `<=` with a count
7	`for (size_t i = n; i-- > 0; )` for reverse iteration
8	`continue` in a `while` loop skips your increment — convert to `for`
9	`break` inside a `switch` inside a loop exits the `switch`, not the loop
10	`goto` forward to a cleanup label is correct C — name labels after what they release
11	`-Werror=return-type` — falling off the end of a non-`void` function is UB

Standing build command, updated:

```
gcc -std=c17 -Wall -Wextra -Wpedantic -Wshadow -Wconversion \
    -Wimplicit-fallthrough -Wempty-body -Wdangling-else \
    -Werror=return-type -Werror=switch \
    -g -fsanitize=address,undefined prog.c -o prog
```

Practice

1. **Reproduce goto fail.** Write a function that performs three validation checks, then deliberately duplicate a line after a braceless `if` so that validation is skipped. Compile with `-Wall` and confirm that `-Wmisleading-indentation` catches it. Then fix it with braces.

2. **Jump table forensics.** Write two switches — one with cases 0–15, and one with cases 1, 100, and 10000. Dump both with `gcc -O2 -S -masm=intel`. Find the jump table in `.rodata` for the dense one. Then add a `case 16:` and see whether the table grows.

3. **Enum exhaustiveness.** Define an enum with 5 states and a `switch` handling all 5, with no `default`. Add a 6th enum value and confirm that you get a warning at every `switch` site. Now add `default:` and observe the warning disappear — this is the trade-off.

4. **The switch-declaration trap.** Reproduce the uninitialized-`x` bug from §5.2. Run it under Valgrind or MSan. Then fix it with per-case blocks.

5. **Reverse iteration, three ways.** Implement `reverse_print` on a `size_t`-indexed array using all three idioms from §5.3, plus the broken `i >= 0` version. Run the broken one under ASan and read the report.

6. **Build a real cleanup chain.** Write `int copy_file(const char *src, const char *dst)`, which opens `src`, opens `dst`, allocates a buffer, copies in chunks, and cleans up correctly on every failure. Add a `fail_at` parameter to inject failures at each step. Verify with `-fsanitize=address,leak` that no path leaks. Then deliberately point one `goto` at the wrong label and confirm that LSan catches it.

7. **Multi-level break, three ways.** Write a 3D array search that returns the coordinates of a target. Implement it with (a) flag variables, (b) `goto`, and (c) extraction into a function with `return`. Compare readability and count the lines.

8. **Reasoning, no compiler:** what does this print, and why?

```c
int i = 0;
switch (i) {
default: printf("D");
case 0:  printf("A");
case 1:  printf("B"); break;
case 2:  printf("C");
}
```

Module 5 in Summary

C’s control flow looks familiar, but its details punish carelessness. A stray semicolon or a missing brace silently changes what an `if` or a loop controls — the root cause of goto fail — so always use braces and let `-Wall` check your indentation. `switch` is a computed jump into a block: execution falls through from case to case unless you stop it, declarations inside it

need their own blocks, and switching on an enum without `default` gives you free exhaustiveness checking. Dense switches compile to jump tables; Duff’s device is a historical curiosity.

Write loops over half-open ranges with `<`, count down with `i-- > 0`, and never count with floating-point values. `break` leaves only the innermost loop or `switch`, and `continue` in a `while` loop skips your increment. Finally, forward `goto` to a chain of cleanup labels is the standard, correct way to release resources on every error path in C — provided each label is named for what it releases and the error paths are tested under a leak checker.

In Module 6, we turn to Functions — prototypes, pass-by-value, recursion, variadic functions, function pointers, and calling conventions.

Module 6 — Functions

In this module:

- Prototypes, definitions, forward declarations
- Pass-by-value semantics (C has no pass-by-reference)
- Recursion, tail calls, stack depth limits
- Variadic functions: `stdarg.h`, `va_list`, `va_start`/`va_arg`/`va_end`/`va_copy`
- `inline`, `static inline`, `_Noreturn`
- Function pointers, callbacks, dispatch tables
- Calling conventions (cdecl, stdcall, fastcall), ABI basics

All outputs in this module are real GCC 13.3 output.

Functions are C's only unit of abstraction. There are no classes, no methods, no closures, and no modules beyond the translation unit — just functions, and pointers to them. That makes it all the more important to understand exactly how they work: how the compiler checks calls, what happens to arguments on the way in, how much stack each call costs, and what the machine actually does when one function calls another.

Function pointers (§6.6) deserve special attention. They are the mechanism behind every polymorphic design in C — plugin systems, state machines, the Linux kernel's `file_operations`, and the standard library's own `qsort`. Learn that section cold.

6.1 Prototypes, Definitions, Forward Declarations

```c
int add(int a, int b);                  /* declaration (prototype) */
int add(int a, int b) { return a+b; }   /* definition */
```

A **prototype** is a function declaration that specifies the types of the parameters. It is what lets the compiler type-check each call, and — just as important — *convert* arguments to the types the function expects. Pass an `int` where a prototype says `double`, and the compiler quietly converts it; without a prototype, it cannot.

What happens without one — real corruption

```c
/* lib.c */ double add(double a, double b) { return a + b; }
/* use.c */ printf("add(2,3) = %d\n", add(2, 3));   /* no prototype */
```

```
### no prototype: compiles, links, CORRUPTS ###
add(2,3) = 0

### with the prototype present ###
add(2,3) = 5.000000
```

Zero. Here is what happened. With no prototype in scope, the caller in `use.c` had no idea that `add` takes `double`s, so it passed the two `int` arguments exactly as they were — in the general-purpose integer registers. The function in `lib.c` was compiled to expect two

`double`s, which arrive in completely different registers, the floating-point XMM registers. So it added whatever happened to be sitting in those registers. (The caller also assumed an `int` return value, and printed it with `%d`.) Nothing crashed, the linker saw nothing wrong — both files mention a symbol called `add` — and the answer was simply wrong. This class of bug is precisely why prototypes exist. (§6.7 shows the registers involved.)

The warning that would have caught it:

```
warning: implicit declaration of function 'add' [-Wimplicit-function-declaration]
```

The standards timeline

Standard	Calling an undeclared function
C89	Legal — implicitly declared as `int f()`
C99	Removed from the language. GCC warns by default.
C23	Hard error

> Make it an error today: `-Werror=implicit-function-declaration`. In practice, a missing prototype almost always means a missing `#include`.

`()` vs `(void)` — the K&R legacy

Module 1 introduced this; here is the compiler's view of it:

```c
void f();       /* pre-C23: UNSPECIFIED parameters — NO type checking */
void g(void);   /* takes NOTHING */
f(1, 2, 3);     /* legal in C17! */
```

```
-- C17 -- warning: function declaration isn't a prototype [-Wstrict-prototypes]
-- C23 -- error: too many arguments to function 'f'
```

`void f();` is a *non-prototype declaration* — a holdover from K&R C meaning "I'm not telling you the parameters." The compiler checks nothing about calls to it. C23 finally fixed this: `()` now means `(void)`. Until you can rely on C23 everywhere:

> Always write `(void)` for functions with no parameters, and compile with `-Wstrict-prototypes`.

Forward declarations

C compilers read a translation unit from top to bottom, and a function must be declared before it is used. Two situations call for a declaration that appears separately from, and before, the definition.

Mutual recursion, where two functions call each other, so neither can be defined first:

```c
static bool is_odd(unsigned n);   /* forward declaration */
static bool is_even(unsigned n) { return n == 0 ? true  : is_odd(n-1); }
static bool is_odd (unsigned n) { return n == 0 ? false : is_even(n-1); }
```

A public API in a header, with the definition in the `.c` file:

```c
/* parser.h */
Token *parse(const char *src);          /* declaration — the promise */

/* parser.c */
static Token next(Lexer*);              /* private forward declaration */
Token *parse(const char *src) { ... }
static Token next(Lexer *lx) { ... }
```

The convention is: prototypes in the header (or at the top of the `.c` file for `static` helpers), and definitions in whatever order reads best. Don't contort the order of your file just to avoid forward declarations — declare what you need at the top, then write the functions in a logical order, typically public entry points first and helpers after.

6.2 Pass-by-Value — C Has No Pass-by-Reference

> **Every argument in C is copied. There are no exceptions.**

This one sentence explains a huge amount of C's behaviour. When you call a function, each parameter is a brand-new local variable, initialized with a copy of the corresponding argument. Whatever the function does to its parameters, it does to those copies.

```
swap_broken: x=1 y=2 (unchanged)
swap_ok    : x=2 y=1
```

```c
void swap_broken(int a, int b) { int t=a; a=b; b=t; }        /* swaps the COPIES */
void swap_ok(int *a, int *b)   { int t=*a; *a=*b; *b=t; }
```

`swap_broken` swaps its own two local copies, which are then discarded when it returns; the caller's variables are untouched. The "workaround" in `swap_ok` is to pass a *pointer* by value. The address is copied, but the copy points at the same object as the original, so writing through it changes the caller's variable. C++ has real references; C simulates them with pointers, and the simulation is always visible — both in the `&x` at the call site and in the `*a` inside the function. Many C programmers regard that visibility as a feature: you can see at the call site which arguments might be modified.

Structs are copied wholesale

```c
struct Big { int data[256]; };   /* 1 KB */
void by_value(struct Big s) { s.data[0] = 99; }
void by_ptr(struct Big *s)  { s->data[0] = 99; }
```

```
by_value : b.data[0]=0    (copy modified, original untouched)
by_ptr   : b.data[0]=99
sizeof(struct Big) = 1024 bytes COPIED on every by-value call
```

Passing a structure by value copies every byte of it — here a full kilobyte on every call. The function then modifies its own copy.

Guideline: pass structures larger than about two pointer-widths (16 bytes on x86-64) by `const` pointer. Small structures by value are fine, and often faster, because they can travel in registers rather than through memory.

```
void draw(const struct Rect *r);   /* large: pointer, const documents read-only */
Point midpoint(Point a, Point b);  /* small (8-16 bytes): by value is idiomatic */
```

The `const` in the first declaration matters: it tells the caller the function will only read the structure, which is the guarantee pass-by-value would otherwise have given them.

Arrays look like an exception — they aren’t

```
void takes_array(int arr[100], size_t *sz) { *sz = sizeof arr; arr[0] = 77; }
```

```
warning: 'sizeof' on array function parameter 'arr' will return size of 'int *'

sizeof(arr) in main     = 400
sizeof(arr) in function = 8   <-- it is a POINTER there
arr[0] modified through the pointer = 77
```

An array parameter is silently rewritten by the compiler as a pointer parameter. These four declarations are identical:

```
void f(int arr[100]);
void f(int arr[10]);    /* the 100 and 10 are ignored entirely */
void f(int arr[]);
void f(int *arr);
```

So arrays *appear* to be passed by reference — the function can modify the caller’s elements — but only because the array decays to a pointer to its first element at the call site. That pointer is still copied, like every other argument. Module 7 gives array decay the full treatment.

The consequence: `sizeof` on an array parameter is always the size of a pointer. Always pass the length explicitly:

```
void process(int *arr, size_t n);            /* the only correct signature */
void process(int arr[static 10], size_t n);  /* C99: hints at least 10 elements */
```

To modify the pointer itself, pass a pointer to the pointer

```
reassign_ptr_broken: *p=1 (pointer itself was a copy)
reassign_ptr_ok    : *p=5
```

```
void reassign(int **p) { *p = &other; }   /* now the caller's pointer changes */
reassign(&p);
```

The same rule applies one level up. If a function receives a pointer and assigns a new address to it, it is changing its own copy of the pointer. To change *the caller's* pointer, the function needs the address of that pointer — an `int **`. This is why APIs that may need to move or replace a buffer take `char **buf`, and why helpers such as `free_and_null(void **p)` exist: they need to set the caller's pointer to `NULL`.

Return values are copied too

```
struct Point make(int x, int y) { struct Point p = {x,y}; return p; }   /* safe */
int *bad(void) { int local = 5; return &local; }                         /* dangling! */
```

Returning a structure by value is safe: the value is copied out to the caller before the local variable disappears. Returning a *pointer* to a local is not, because the pointer is copied but the object it points to dies with the function (Module 3).

6.3 Recursion, Tail Calls, Stack Depth

Every call consumes stack

Each function call reserves a *stack frame* for the return address, saved registers, and local variables. Recursion therefore uses stack in proportion to its depth:

```
stack limit = 8192 KB
  depth  50000, stack used 1562 KB
  depth 100000, stack used 3124 KB
  depth 150000, stack used 4687 KB
  depth 200000, stack used 6249 KB
  depth 250000, stack used 7812 KB
Segmentation fault
```

That is about 32 bytes per frame for a trivial function, and it died at roughly 262,000 frames with the default 8 MB stack. Add a few local arrays and the number collapses:

```
void f(void) { char buf[4096]; ...; f(); }   /* ~2000 frames before overflow */
```

Stack overflow is not a clean error

A blown stack gives you `SIGSEGV` — a segmentation fault — with no message at all. The process simply dies when it touches the guard page below the stack. Under AddressSanitizer you at least get a diagnosis:

```
ERROR: AddressSanitizer: stack-overflow on address 0x7ffdb1979ff8
```

Typical stack limits are 8 MB by default on Linux (see `ulimit -s`), 1 MB on Windows, 512 KB for non-main threads created with pthreads on some platforms, and often just a few kilobytes on embedded microcontrollers.

> **In production C, unbounded recursion is a denial-of-service vulnerability.** Recursive-descent parsers that process untrusted input must have a depth counter. Deeply nested input — `[[[[[[…` — that crashes a parser is a recurring CVE pattern in JSON, XML, and protobuf libraries.

Tail calls

A *tail call* is a call in the last position of a function: its result is returned directly, and nothing is left to do afterwards.

```c
unsigned long sum_tail(unsigned long n, unsigned long acc)
{ if (n == 0) return acc; return sum_tail(n-1, acc+n); }   /* tail position */

unsigned long fact(unsigned long n)
{ if (n <= 1) return 1; return n * fact(n-1); }            /* NOT tail — multiply pending */
```

In `sum_tail`, once the recursive call is made, the current function has nothing more to do: it will simply hand back whatever the call returns. In `fact`, by contrast, the multiplication by `n` still has to happen after the recursive call returns, so the current frame must be kept alive.

At a tail call the current frame is dead, so the compiler can reuse it: no stack growth, and a `jmp` instead of a `call`. This is *tail-call optimisation* (TCO), and it effectively turns tail recursion into a loop.

```
### -O0 (no TCO -> 10 million stack frames) ###
Segmentation fault exit=139

### -O2 (tail call turned into a loop) ###
sum_tail(10,000,000) = 50000005000000 exit=0
```

Same source. One crashes, one works.

Three things to internalise:

1. **The C standard does not require tail-call optimisation.** It is an optimisation, not a guarantee, unlike in languages such as Scheme.
2. **It requires optimisation to be enabled.** `-O0` gives you no TCO, and `-foptimize-sibling-calls` alone was not enough here — it needs the surrounding `-O1`-and-above machinery. So debug builds can crash where release builds work — the reverse of the usual situation, and very confusing to diagnose.
3. **Compilers are cleverer than the textbook rule suggests.** At `-O2`, GCC also converted the *non*-tail-recursive factorial into a loop, because multiplication is associative and it could restructure the accumulation itself.

> **Practical rule:** never rely on TCO in C. If recursion depth could exceed a few thousand, write the loop yourself, or use an explicit stack (an array plus an index). Recursion is for trees and genuinely recursive structures, where the depth is logarithmic.

```c
/* recursive — depth O(n), fine for a balanced tree, dangerous for a list */
size_t count(Node *n) { return n ? 1 + count(n->next) : 0; }

/* iterative — depth O(1), always safe */
size_t count(Node *n) { size_t c=0; for (; n; n = n->next) c++; return c; }
```

A linked list of a million nodes would crash the first version; the second handles any length.

6.4 Variadic Functions — `<stdarg.h>`

A *variadic* function accepts a variable number of arguments. `printf` is the most famous example. C provides the machinery in `<stdarg.h>`:

```c
#include <stdarg.h>

int sum(int n, ...)                  /* at least one named parameter is REQUIRED */
{
    va_list ap;
    va_start(ap, n);                 /* n = the LAST named parameter */
    int total = 0;
    for (int i = 0; i < n; i++)
        total += va_arg(ap, int);    /* you must state the type — nothing checks it */
    va_end(ap);                      /* mandatory */
    return total;
}
```

The `...` in the parameter list marks where the variable arguments begin. Inside the function, you walk through them one at a time with a cursor.

The macros

Macro	Purpose
`va_list ap;`	The cursor
`va_start(ap, last_named)`	Initialize the cursor. C23 makes the second argument optional.
`va_arg(ap, TYPE)`	Fetch the next argument as `TYPE`, and advance the cursor.
`va_end(ap)`	Clean up. Required — it may be a no-op on your platform, or it may not.
`va_copy(dst, src)`	C99. Duplicate a cursor for a second pass.

The fundamental problem: there is no type information

`va_arg(ap, int)` checks nothing. It simply reinterprets whatever is at the cursor as an `int` and advances by the size of an `int`. Name the wrong type and you get garbage — and a cursor that is now misaligned for every argument after it. This is undefined behaviour with no diagnostic.

The function has no way of discovering how many arguments were passed either, so a variadic function must be told, “out of band.” There are two common conventions:

```
sum_count(4, 1,2,3,4)       /* explicit count — first arg says how many */
sum_sentinel(1,2,3,4,-1)    /* sentinel value terminates the list */
```

```
sum_count(4, 1,2,3,4)    = 10
sum_sentinel(1,2,3,4,-1) = 10
```

`printf` uses a third convention: the format string encodes both the number and the types of the arguments that follow — which is exactly why a mismatched format string is so dangerous.

> ⚠ **Sentinel pitfall:** `execl(path, "a", "b", NULL)` should be written with `(char *)NULL`, not a bare `NULL`. On platforms where `NULL` is defined as a plain `0` — an `int`, typically 4 bytes — the function reads an 8-byte `char *` and picks up garbage in the other half. GCC's `__attribute__((sentinel))` warns when a function declared this way is not terminated by a proper null pointer.

Default argument promotions

Arguments in the variadic part of a call are automatically promoted before they are passed:

- `char`, `short`, and `_Bool` → `int`
- `float` → `double`
- Arrays and functions → pointers

Consequences:

```
va_arg(ap, char)    /* UNDEFINED — never a char, always promoted to int */
va_arg(ap, float)   /* UNDEFINED — always promoted to double */
va_arg(ap, int)     /* correct for char/short/bool arguments */
va_arg(ap, double)  /* correct for float arguments */
```

This is also why `printf("%f", some_float)` works, and why `printf` needs no separate specifier for `float` (Module 2): every `float` has already become a `double` by the time `printf` sees it.

`va_copy` — required for a second traversal

A `va_list` is a one-way cursor. To traverse the arguments twice, you need a copy made before the first traversal:

```
va_list ap, ap2;
va_start(ap, n);
va_copy(ap2, ap);   /* independent cursor */

... first pass with ap ...
va_end(ap);
... second pass with ap2 ...
va_end(ap2);
```

```
max_and_sum(5, 3,9,2,7,4):
  max=9 sum=25
```

Every `va_copy` needs its own `va_end`. And note that on x86-64, `va_list` is an *array* type, so passing it to a helper function passes a pointer to the same cursor: the helper may consume arguments that you then cannot reread. Always `va_copy` before delegating to another function if you need the arguments again afterwards.

Writing `printf`-style wrappers

Never re-implement formatting yourself. The standard library provides `v`-prefixed variants that take a `va_list` instead of `...`: `vprintf`, `vfprintf`, `vsprintf`, `vsnprintf`, and (as a GNU extension) `vasprintf`.

```
void logmsg(const char *level, const char *fmt, ...)
{
    va_list ap; va_start(ap, fmt);
    printf("[%s] ", level);
    vprintf(fmt, ap);   /* NOT printf(fmt, ap) — that passes the va_list itself */
    va_end(ap);
}
```

```
[INFO] user alice id 42
```

Calling `printf(fmt, ap)` would be a serious bug: it would pass the cursor itself as a single argument, and `printf` would try to format it according to `fmt`.

Get compile-time checking with `format`

GCC and Clang can type-check calls to your own `printf`-style functions:

```
__attribute__((format(printf, 2, 3)))
void logmsg(const char *level, const char *fmt, ...);
/*                             ^fmt is arg 2, varargs start at arg 3 */

logmsg("X", "%d %s\n", "not_an_int", 42);
```

```
warning: format '%d' expects argument of type 'int', but argument 3 has type 'char *'
warning: format '%s' expects argument of type 'char *', but argument 4 has type 'int'
```

The two numbers tell the compiler which parameter is the format string and where the variable arguments begin. Put this attribute on every logging function you write. It converts a whole class of run-time crashes into compile-time warnings — and format-string mismatches are both a crash source and a security vulnerability class.

> ⚠ **Never pass user input as the format string.** `printf(user_input)` is the classic *format string vulnerability*: an attacker who types `%s` makes `printf` read arbitrary pointers, and `%n` makes it *write* to memory. Always write `printf("%s", user_input)`.

6.5 `inline`, `static inline`, `_Noreturn`

The C99 `inline` model is genuinely confusing

`inline` is a hint, not a command: the compiler decides whether to actually inline a call. But the part that surprises people is not the optimisation — it is the *linkage* rules.

```c
inline int f(int x) { return x * 2; }   /* "inline definition" — NOT an external definition */
int main(void) { return f(21); }
```

```
-- -O0 -- undefined reference to `f'
-- -O2 -- (linked fine) 42
```

The same code links at `-O2` and fails at `-O0`. The reason: in C99, `inline` on its own provides only an *inline definition*. The compiler may use it to inline calls, but it emits no callable, out-of-line copy of the function — no symbol for the linker. At `-O2` every call was inlined, so no symbol was needed. At `-O0` nothing is inlined, so the call needs an external definition of `f` — and there isn't one.

The three forms, and which to use

Form	Meaning	Use it?
`static inline`	Internal linkage, own copy per TU, always works	✓ Default choice
`inline` (alone)	Inline definition only, no external symbol	⚠ Only with the pattern below
`extern inline`	Emits the external definition	Exactly once, in one `.c`

`static inline` always works, at every optimisation level:

```
-O0: 42
-O2: 42
```

If the function is not inlined, each translation unit simply gets its own private copy.

The correct C99 header pattern, if you need one shared out-of-line copy:

```c
/* hdr.h */
inline int dbl(int x) { return x * 2; }

/* hdr.c — exactly ONE translation unit */
#include "hdr.h"
extern inline int dbl(int x);   /* emits the external definition */
```

```
-O0: dbl(21)=42
-O2: dbl(21)=42
```

Every file that includes the header gets the inline definition for inlining; `hdr.c` additionally provides the one real, linkable copy for calls that are not inlined.

> **Practical advice:** use `static inline` in headers and stop thinking about it. The cost of duplicate copies is negligible — identical copies are often merged, and inlined calls emit no copy at all. Reach for `extern inline` only when you have a measured reason.
>
> **Note:** GNU89 `inline` semantics are the *opposite* of C99's. Code written for old GCC with `-std=gnu89` behaves differently under `-std=c99`. This bit the Linux kernel during its move to newer C standards.

Should you use `inline` at all?

Mostly, no — not as an optimisation. Modern compilers inline aggressively according to their own cost models, and ignore your hint when they disagree. With link-time optimisation (LTO), they inline across translation units too.

`inline` still matters for:

- **Functions defined in headers,** where `static inline` is the *mechanism* that makes it legal to define the function in many translation units — not an optimisation request.
- **`__attribute__((always_inline))`,** when you have a measured need to force inlining.
- **`__attribute__((noinline))`,** for debugging, or to keep a function visible in stack traces and profiles.

`_Noreturn` / `[[noreturn]]`

```c
_Noreturn void die(const char *msg)   /* C11 */
{ fprintf(stderr, "fatal: %s\n", msg); exit(1); }

[[noreturn]] void die(const char *msg);   /* C23 */
```

As introduced in Module 5, this tells the compiler that the function never returns. Its benefits: the compiler stops warning that "control reaches end of non-void function" after a call to it, can delete unreachable code that follows the call, and can skip saving registers that would only matter if the function came back. Standard library examples include `exit`, `abort`, `_Exit`, `longjmp`, and `thrd_exit`.

If a `_Noreturn` function ever does return, that is undefined behaviour.

6.6 Function Pointers ⚠ (the important one)

A function lives at an address in memory, just as data does, and C lets you store that address in a variable, pass it to other functions, and call through it. That single capability is what makes callbacks, plugin systems, dispatch tables, and object-oriented designs possible in C.

Syntax

```
int add(int, int);

int (*fp)(int, int) = add;   /* pointer to function taking (int,int), returning int */
```

The parentheses around `*fp` are mandatory, because the function-call parentheses `()` have higher precedence than `*`:

```
int (*p)(int);   /* p is a POINTER to a function returning int */
int *p(int);     /* p is a FUNCTION returning int* — completely different */
```

A function name decays to a pointer to the function in almost every context, so `&` and `*` are optional — and, curiously, stackable:

```
f(3,4)       = 7
(*f)(3,4)    = 7
(****f)(3,4) = 7   <- deref of a function ptr yields the function again
```

Dereferencing a function pointer gives a function, which immediately decays back into a pointer, so any number of `*`s leaves you where you started. `fp = add;` and `fp = &add;` are identical, and so are `fp(3,4)` and `(*fp)(3,4)`. Write the simple forms: `fp = add;` and `fp(3,4)`.

Always `typedef` it

```
typedef int (*BinOp)(int, int);   /* pointer-to-function alias */
typedef int BinOpFn(int, int);    /* function-type alias — sometimes cleaner */

BinOp f = add;
void install(BinOp op);
BinOp table[3] = { add, sub, mul };
```

With a `typedef`, a function-pointer type reads like any other type. Compare writing it raw: `int (*(*get_handler(int))(int,int))(int)` declares a function that takes an `int` and returns a pointer to a function that takes two `int`s and returns a pointer to a function taking an `int` and returning an `int`. Nobody should have to read that. (Module 9 shows how to decode such declarations when you meet them.)

Dispatch tables

```
static const struct { char sym; BinOp fn; const char *name; } ops[] = {
    {'+', add, "add"}, {'-', sub, "sub"}, {'*', mul, "mul"},
};

for (size_t i = 0; i < ARRAY_LEN(ops); i++)
    printf("6 %c 3 = %d (%s)\n", ops[i].sym, ops[i].fn(6,3), ops[i].name);
```

```
6 + 3 = 9  (add)
6 - 3 = 3  (sub)
6 * 3 = 18 (mul)
```

This is how you replace a long `if`/`else` chain with *data*. Adding an operation means adding a row to the table — no control flow changes, no new branches to test. Dispatch tables are used for interpreters and virtual machines (opcode dispatch), state machines, command parsers, protocol handlers, and plugin registries.

Callbacks — and the `void *` context idiom

A *callback* is a function you pass to another function, to be called back later:

```
void for_each(int *a, size_t n, void (*cb)(int, void *), void *ctx)
{ for (size_t i=0; i<n; i++) cb(a[i], ctx); }

static void accumulate(int v, void *ctx) { *(long*)ctx += v; }

long total = 0;
for_each(a, n, accumulate, &total);
```

```
sum via callback = 24
```

C has no closures: a function cannot capture variables from the place where it was created. The `void *` "user data" or "context" parameter is how you carry state into a callback instead. `for_each` does not know or care what `ctx` points to; it simply passes it along, and the callback casts it back to the type it expects.

Every callback API you design should take one. Omitting it is the single most common C API design mistake, because it leaves callers no way to attach context except through global variables.

Look at how the standard library does it — and note that `qsort` gets this wrong:

```
void qsort(void *base, size_t n, size_t size,
           int (*cmp)(const void *, const void *));   /* no context! */
```

Because the comparator receives no context, sorting by a key chosen at run time (say, "sort by the column the user clicked") requires a global variable, or the non-standard extensions `qsort_r` (GNU/BSD) or `qsort_s` (C11 Annex K / Microsoft), which add the missing parameter. Learn from it.

Writing a comparator correctly

```
static int cmp_int_asc(const void *a, const void *b)
{
    int x = *(const int *)a, y = *(const int *)b;
    return (x > y) - (x < y);   /* NOT x - y — that overflows */
}
```

```
asc : 1 2 5 7 9
desc: 9 7 5 2 1
```

A comparator must return a negative value, zero, or a positive value depending on whether the first element sorts before, equal to, or after the second. The tempting `return x - y;` is a real bug: with `x = INT_MIN` and `y = 1`, the subtraction overflows, which is undefined behaviour, and in practice produces a large positive number — the wrong sign. `(x > y) - (x < y)` is branchless, cannot overflow, and returns exactly −1, 0, or +1.

Rules and gotchas

- **A function pointer is not a `void *`.** Converting between them is not guaranteed by ISO C. (POSIX requires it to work, which is what makes `dlsym` possible — Module 24.) Use a union, or a POSIX-sanctioned cast, and know that it is a portability compromise.
- **You can't do pointer arithmetic on function pointers.**
- **Calling through a mismatched type is UB** — a common source of crashes in callback registries, where a function with a slightly different signature is cast to the registry's type. Keep signatures exact.
- **Calling a null function pointer is UB,** usually a segmentation fault at address 0.
- **Function pointers are the natural target for Control Flow Integrity hardening.** An attacker who can overwrite one can redirect execution. `-fcf-protection` is what produces the `endbr64` instructions at the start of every function in our assembly dumps: an indirect call is only allowed to land on such a marker.

The pattern this unlocks: polymorphism in C

```c
typedef struct Shape Shape;
struct ShapeVTable {
    double (*area)(const Shape *);
    void   (*draw)(const Shape *);
    void   (*destroy)(Shape *);
};
struct Shape { const struct ShapeVTable *vt; };

double area(const Shape *s) { return s->vt->area(s); }   /* virtual dispatch */
```

Each kind of shape — circle, rectangle, polygon — provides its own table of functions, and every shape object carries a pointer to its type's table. Calling `area(s)` looks up the right function at run time. That is exactly what C++ generates for virtual functions, and exactly what the Linux kernel's `struct file_operations` is: each driver fills in a table of read, write, and open functions, and the kernel calls through it without knowing which driver it is talking to. Module 28 gives this pattern the full treatment.

6.7 Calling Conventions & ABI Basics

An **ABI** (Application Binary Interface) specifies how compiled code interoperates at the machine level: how arguments are passed, where return values go, which registers a function must preserve, how the stack is aligned, how structures are laid out, and how names appear in object files. The **API** is what the compiler checks against your source code; the **ABI** is what actually happens at run time. Two pieces of code compiled separately — by different compilers, or even in different languages — can call each other only if they agree on the ABI.

System V AMD64 (Linux, macOS, BSD) — the one you'll debug

Integer and pointer arguments 1–6 are passed in the registers **RDI, RSI, RDX, RCX, R8, R9**, in that order:

```
six:                    ; long six(a,b,c,d,e,f)
    add rdi, rsi        ; a + b
    add rdi, rdx        ; + c
    add rdi, rcx        ; + d
    add rdi, r8         ; + e
    lea rax, [rdi+r9]   ; + f, result in RAX
    ret
```

Arguments 7 and beyond go on the stack, pushed right to left, so the 7th sits just above the return address:

```
eight:
    ...
    add rax, QWORD PTR 8[rsp]    ; 7th argument
    add rax, QWORD PTR 16[rsp]   ; 8th argument
```

Floating-point arguments use a separate register class, **XMM0–XMM7**:

```
dbl6:
    addsd xmm0, xmm1
    addsd xmm0, xmm2
    ...                 ; result in XMM0
```

This is exactly why the missing-prototype bug in §6.1 returned 0: the caller, believing the arguments were `int`s, loaded RDI and RSI, while the callee, expecting `double`s, read XMM0 and XMM1.

Other System V AMD64 rules:

Item	Rule
Integer return	RAX (and RDX for 128-bit values)
Float return	XMM0
Large struct return	Caller allocates the space; a hidden pointer to it is passed in RDI
Callee-saved	RBX, RBP, R12–R15 — the callee must restore these before returning
Caller-saved	RAX, RCX, RDX, RSI, RDI, R8–R11, all XMM registers
Stack alignment	16 bytes at every call
Red zone	128 bytes below RSP, usable without adjusting RSP
Variadic calls	AL holds the number of vector registers used

"Callee-saved" registers survive a function call, so the caller can keep long-lived values in them; "caller-saved" registers may be overwritten by any call. The *red zone* lets small leaf functions use a little stack space without the cost of adjusting the stack pointer. The rule

about AL explains a detail you may see in disassembly: before calling `printf`, the compiler sets AL to the number of floating-point arguments, so `printf` knows which XMM registers to save.

The historical x86 (32-bit) conventions

You will meet these in Windows code, old libraries, and interview questions:

Convention	Args	Who cleans the stack	Notes
cdecl	Stack, right to left	Caller	C default. Caller cleanup is what makes variadics possible.
stdcall	Stack, right to left	Callee	Win32 API. Smaller code, but no variadics.
fastcall	First 2 in ECX, EDX; rest on stack	Callee	Compiler-specific, not standardised.
thiscall	`this` in ECX	Varies	C++ member functions on MSVC.

Why cdecl allows variadic functions: only the caller knows how many arguments it pushed, so only the caller can clean them off the stack correctly. A stdcall function cleans up its own arguments with a fixed-size return instruction, so it cannot be variadic — which is why `wsprintf` is one of the few cdecl functions in the otherwise stdcall Win32 API.

You will see these conventions spelled `__cdecl`, `__stdcall`, `WINAPI`, and `CALLBACK` in Windows headers. On 64-bit Windows they are all ignored, because there is only one convention: Microsoft x64, which passes the first four arguments in RCX, RDX, R8, and R9 and requires the caller to reserve 32 bytes of “shadow space” on the stack. That is different from System V, which is why cross-platform C libraries need care at their boundaries — especially with assembly or FFI code.

Stack frames — why this matters for debugging

```
higher addresses
  +------------------+
  | caller's frame   |
  +------------------+
  | arguments 7+     |
  +------------------+
  | return address   |  ← pushed by `call`
  +------------------+
  | saved RBP        |  ← `push rbp; mov rbp, rsp`
  +------------------+  ← RBP
  | local variables  |
  | saved registers  |
  +------------------+  ← RSP
lower addresses (stack grows DOWN)
```

When a function is called, the `call` instruction pushes the return address; the function then typically saves the caller’s frame pointer (RBP) and sets up its own, and finally reserves space for its locals by moving RSP down. The chain of saved RBP values is what a debugger follows to produce a backtrace.

This layout is also why buffer overflows are exploitable. Local arrays sit *below* the saved RBP and the return address, and array indices grow *upward*. A local array that overflows therefore runs into the saved RBP and then the return address. Overwrite that, and you control where the function "returns" to.

Mitigations you will see in real builds: `-fstack-protector-strong` (a random *canary* value placed between the locals and the return address, checked before returning), ASLR (randomised addresses, so an attacker cannot know where to jump), a non-executable stack, and `-fcf-protection` (the `endbr64` instructions in every dump above). Module 23 covers them in depth.

Where ABI knowledge earns its keep

- Reading stack traces when symbols are missing.
- Debugging optimised code, where `-fomit-frame-pointer` breaks naive backtraces because RBP is used as an ordinary register.
- Writing inline assembly, or interfacing with hand-written assembly.
- FFI — calling C from Python, Rust, or Go — where you are matching the ABI by hand.
- Shared-library versioning: changing a structure's layout or a parameter's type breaks the ABI even when the API still compiles perfectly (Module 20).
- Understanding why `long` differs between Linux and Windows (Module 2) — that is an ABI decision.

Module 6 Rules

#	Rule
1	Prototypes for everything; `-Werror=implicit-function-declaration`
2	Write `(void)`, never `()`, for functions with no parameters; `-Wstrict-prototypes`
3	Everything is copied — to modify a caller's object, pass a pointer to it
4	`sizeof` on an array parameter gives the pointer size; always pass the length
5	Pass structs larger than ~16 bytes by `const` pointer
6	Never rely on tail-call optimisation; bound recursion depth on untrusted input
7	`va_arg(ap, char)` and `va_arg(ap, float)` are always wrong — promotions apply
8	`va_copy` before a second traversal; every copy needs its own `va_end`
9	Put `__attribute__((format(printf, m, n)))` on every logging function

#	Rule
10	Never pass user input as a format string
11	Use `static inline` in headers; plain `inline` breaks at `-O0`
12	`typedef` every function-pointer type
13	Every callback API takes a `void *userdata`
14	Comparators return `(x>y)-(x<y)`, never `x-y`

Practice

1. **Reproduce the prototype corruption.** Put `double scale(double, double)` in its own `.c` file, call it from another translation unit without a prototype, and observe the wrong answer. Then dump both sides with `-S` and identify exactly which registers disagree.

2. **Swap, four ways.** Implement a swap for `int`, for a large struct, for two pointers (`swap_ptr(int **a, int **b)`), and a generic `swap_bytes(void *a, void *b, size_t n)`. Test each one.

3. **Measure your stack.** Write a recursion that reports its depth and stack consumption every 10,000 frames. (Use unbuffered `stderr` — buffered `printf` output is lost when the process dies with `SIGSEGV`.) Run it with `ulimit -s 1024` and with `ulimit -s unlimited`. Then add `char pad[1024]` to the frame and measure again.

4. **TCO experiment.** Write a tail-recursive `sum(n, acc)` and call it with `n = 10,000,000`. Build at `-O0`, `-O1`, `-O2`, and `-Os`, and record which builds crash. Dump the `-O2` assembly and find the `jmp` that replaced the `call`.

5. **Build a safe logger.** Write `void log_at(const char *file, int line, const char *level, const char *fmt, ...)`, using `vsnprintf` into a fixed buffer, with the `format` attribute. Add a `#define LOG(...) log_at(__FILE__, __LINE__, "INFO", __VA_ARGS__)` wrapper. Deliberately pass a wrong type and confirm the compile-time warning.

6. **`va_copy` proof.** Write a variadic function that traverses its arguments twice without `va_copy`, and observe the garbage. Then add `va_copy` and confirm the result is correct. Run both versions under UBSan.

7. **The inline maze.** Reproduce all three linkage cases from §6.5: plain `inline` failing at `-O0`, `static inline` working everywhere, and the `extern inline` header pattern. Use `nm` to inspect which symbols each emits.

8. **Build a calculator with a dispatch table.** Support `+ - * / % ^` using a table of `{char, BinOp, const char *}`. Add division-by-zero handling. Then extend it with unary operations using a second table — and notice how much easier that is than editing a `switch`.

9. **Generic `for_each` with context.** Write `map(int *a, size_t n, int (*f)(int, void *), void *ctx)` and use it to implement scale-by-k, clamp-to-range, and running-sum — each carrying its parameters through `ctx`.

10. **ABI archaeology.** Write `long f(long, long, long, long, long, long, long, long)` and `double g(double, double, ...)`. Dump the assembly and identify the location of each argument. Then write a function that returns a large structure and find the hidden return pointer.

Module 6 in Summary

Prototypes let the compiler check and convert every argument; without them, a mismatch between caller and callee silently corrupts values, so make implicit declarations an error and always write `(void)`. Every argument is copied: to change a caller's object you pass a pointer to it, to change a caller's pointer you pass a pointer to the pointer, and arrays only appear to be passed by reference because they decay to pointers — which is why their length must always travel with them.

Every call costs stack, stack overflow is a silent crash, and tail-call optimisation is never guaranteed, so bound recursion or write loops. Variadic functions carry no type information: promotions apply, the count must be supplied separately, `va_copy` is needed for a second pass, and the `format` attribute turns format-string mistakes into compile-time warnings. In headers, use `static inline`.

Function pointers — always behind a `typedef` — give C callbacks, dispatch tables, and polymorphism, and every callback interface should carry a `void *` context. Finally, the ABI determines what actually happens during a call: which registers carry arguments, who cleans up, and how stack frames are laid out — knowledge that pays off in debugging, security, and interoperability.

In Module 7, we turn to Arrays — their layout in memory, decay to pointers, variable-length arrays, and flexible array members.

Module 7 — Arrays

In this module:

- 1D, 2D, multidimensional arrays, row-major layout
- Array-to-pointer decay (and where it doesn't happen)
- Array parameters in functions (they're pointers)
- Variable Length Arrays (VLA) — and why many shops ban them
- Designated initializers, compound literals
- Flexible array members (`struct { int n; char data[]; }`)

All outputs in this module are real GCC 13.3 output.

Arrays are where C's closeness to the machine becomes most visible. In most modern languages, an array is an object that knows its own length, checks every index, and can be passed around, copied, and compared as a single value. In C, an array is none of those things. It is simply a run of objects laid end to end in memory — and the language quietly converts it into a pointer at almost every opportunity.

That conversion, *array-to-pointer decay* (§7.3), is the concept that unlocks most of C. Once you understand decay completely, `sizeof` bugs, function signatures, two-dimensional arrays, and half of pointer syntax stop being mysterious.

7.1 One-Dimensional Arrays

```
int a[5] = {10, 20, 30, 40, 50};
```

An array is *N contiguous objects of the same type*. Nothing more: there is no hidden length field, no bounds checking, and no metadata of any kind. Printing the address of each element shows them packed back to back, each one `sizeof(int)` = 4 bytes after the last:

```
&a[0] = 0x7fffafbd7c80 (offset  0 bytes)
&a[1] = 0x7fffafbd7c84 (offset  4 bytes)
&a[2] = 0x7fffafbd7c88 (offset  8 bytes)
&a[3] = 0x7fffafbd7c8c (offset 12 bytes)
&a[4] = 0x7fffafbd7c90 (offset 16 bytes)
```

Because there is no length stored anywhere, the program has no way to discover at run time that `a` has five elements. Reading `a[5]` or `a[100]` simply reads whatever memory lies at that offset — undefined behaviour, and the root of the buffer overflow.

`a[i]` is defined as `*(a + i)`

This is not an analogy; it is the standard's actual definition of the subscript operator. The expression `a + i` computes the address of the element `i` positions after the start (pointer arithmetic automatically multiplies by the element size), and `*` fetches the element there. And since addition is commutative:

```
a[2]=30  *(a+2)=30  2[a]=30  *(2+a)=30
```

`2[a]` is legal C. Never write it — it exists only as a curiosity and in obfuscation contests — but understanding *why* it works means you understand subscripting.

The array name vs `&array` — a critical distinction

```
a     = 0x7fffafbd7c80 (decays: pointer to first ELEMENT, type int*)
&a[0] = 0x7fffafbd7c80 (same value, type int*)
&a    = 0x7fffafbd7c80 (same value, type int(*)[5] -- DIFFERENT TYPE)
a+1   = 0x7fffafbd7c84 (+4 bytes: one int)
&a+1  = 0x7fffafbd7c94 (+20 bytes: one WHOLE ARRAY)
```

All three of `a`, `&a[0]`, and `&a` print the same address — the array starts where its first element starts. But they do not have the same *type*. `a` decays to `int *`, a pointer to one `int`. `&a` is a pointer to the whole array of five `int`s, written `int (*)[5]`. Adding 1 to the first advances by the size of one `int`, 4 bytes; adding 1 to the second advances by the size of the whole array, 20 bytes, landing just past its end.

This is the single cleanest proof that pointer arithmetic is driven entirely by the pointed-to type. The address is the same; the type decides what "+1" means.

What arrays cannot do

```c
int a[5], b[5];
b = a;                                  /* ERROR: arrays are not assignable */
if (a == b) { }                         /* compares ADDRESSES, not contents (always false here) */
int f(void) { int x[5]; return x; }     /* ERROR: cannot return an array */
struct S { int a[5]; };
struct S s1, s2; s2 = s1;               /* OK! struct assignment DOES copy the array */
```

Arrays are not *first-class values*: you cannot assign them, compare their contents with `==`, pass them to functions, or return them from functions. The comparison is particularly deceptive, because it compiles: both operands decay to pointers, so `a == b` asks whether the two arrays are at the same address — which two distinct arrays never are. Compare contents with `memcmp` or a loop.

Wrapping an array in a structure makes it copyable, because structures *are* first-class values. That is a real, widely used technique:

```c
typedef struct { double v[3]; } Vec3;
Vec3 add(Vec3 a, Vec3 b);   /* by value, copies the array — works fine */
```

7.2 Multidimensional Arrays and Row-Major Layout

```c
int m[3][4];   /* read as: an array of 3 elements, each of which is int[4] */
```

C has no true two-dimensional arrays. `int m[3][4]` is an *array of arrays*: an array of three elements, each of which is itself an array of four `int`s. And because every array is contiguous, the whole thing is one contiguous block of twelve `int`s.

Filled with the values 1 to 12 in order:

```
linear memory: 1 2 3 4 5 6 7 8 9 10 11 12

address map (offset from &m[0][0]):
  row 0:  0  4  8 12
  row 1: 16 20 24 28
  row 2: 32 36 40 44
```

This is **row-major order**: the rows are stored one after another, so the *last* index varies fastest as you move through memory. (Fortran and MATLAB are column-major, storing columns contiguously instead. That matters when you interface with numerical libraries such as BLAS and LAPACK, which often expect column-major data.)

The equivalences follow directly:

```
m[i][j] == *(*(m+i)+j) == flat[i*4 + j]
```

To reach element `[i][j]`, skip `i` whole rows of 4 elements, then `j` more elements.

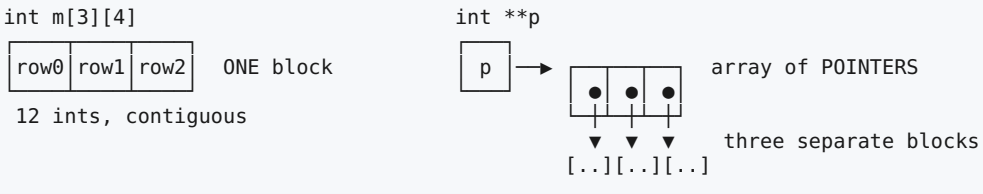
The types at each level

```
sizeof(m)       = 48  type int[3][4] (whole array)
sizeof(m[0])    = 16  type int[4]    (one row)
sizeof(m[0][0]) = 4   type int
m decays to     : int (*)[4] -- POINTER TO A ROW, not int**
m+1 advances    : 16 bytes (one whole row)
m[0]+1 advances : 4 bytes (one int)
```

Decay applies at each level. `m` is an array whose elements are rows, so it decays to a pointer to its first row: `int (*)[4]`. Adding 1 to that pointer moves forward one whole row, 16 bytes. `m[0]` is a row, an array of `int`, so it decays to `int *`, and adding 1 moves one `int`.

> ⚠ **`int m[3][4]` does NOT decay to `int **`.** It decays to `int (*)[4]` — a pointer to an array of 4 ints. Passing a 2D array where an `int **` is expected is a type error, and forcing it through with a cast produces garbage.

The difference is fundamental:



An `int **` points to an array of *pointers*, each of which points to a separately allocated row somewhere else. `m[1][2]` on a true 2D array is a single memory access at a computed offset. On an `int **` it is two dereferences: first fetch the row pointer, then fetch the element. Different code, different performance, and a different way of allocating and freeing the memory. If you cast `m` to `int **`, the called function will interpret the first `int`s of the matrix as if they were pointers — and crash.

Why row-major matters for performance

```c
/* FAST: sequential memory access, cache-friendly */
for (i = 0; i < rows; i++)
    for (j = 0; j < cols; j++)
        sum += m[i][j];

/* SLOW: strides across rows, cache-hostile */
for (j = 0; j < cols; j++)
    for (i = 0; i < rows; i++)
        sum += m[i][j];
```

The CPU fetches memory in *cache lines*, typically 64 bytes at a time. The first loop walks through memory in order, so every byte of every cache line it fetches gets used. The second loop jumps a whole row ahead on every step, touching one `int` per cache line before moving on, and on a large matrix those lines are evicted before it comes back for the rest. On a large matrix, the difference can be 5–10×, with identical results. Iterate in memory order. (Module 25 covers cache behaviour properly.)

Practical alternative: flatten it yourself

```c
size_t rows = R, cols = C;
int *m = malloc(rows * cols * sizeof *m);
#define M(i,j) m[(i)*cols + (j)]

M(1,2) = 7;
free(m);
```

This gives you one allocation, contiguous memory, dimensions chosen at run time, and no VLA. The indexing arithmetic is exactly what the compiler would generate for a real 2D array; you have just written it yourself. This is what most production code does.

7.3 Array-to-Pointer Decay ⚠

> **In almost every expression, an array is implicitly converted to a pointer to its first element.**

That is the whole rule. The array itself does not change; the *expression* that names it yields a pointer instead of an array value.

```
DECAY HAPPENS almost everywhere:
  assignment:    int *p = a;  OK
  function call: inside f: sizeof(arr) = 8 (it is int*)
  arithmetic:    a+1 = 0x7ffd1c756e24
  comparison:    (a == p) = 1
```

The five places decay does not happen

```
1. sizeof(a)          = 40 (whole array, not 8)
2. &a                 -> type int(*)[10], NOT int**
3. _Alignof(a)        = 4
```

```
4. char s[] = "hello" -> COPIES the literal, sizeof(s)=6
5. typeof(a) / compound literal keeps the array type
```

#	Context	Why it matters
1	Operand of `sizeof`	The array-length idiom depends on this
2	Operand of `&`	Gives `T (*)[N]`, enabling whole-array pointers
3	Operand of `_Alignof`	Alignment of the array type
4	String literal initializing a `char[]`	`char s[] = "hi"` copies; `char *s = "hi"` points
5	`typeof`, `_Generic` (C23)	Type queries see the real array type

Everywhere else — assignment, arithmetic, comparison, subscripting, and passing to a function — the array becomes a pointer.

Number 4 is worth dwelling on:

```c
char s1[] = "hello";   /* array of 6, on the stack, MODIFIABLE */
char *s2  = "hello";   /* pointer into .rodata, READ-ONLY */

s1[0] = 'H';           /* fine */
s2[0] = 'H';           /* UNDEFINED BEHAVIOR — usually segfault */
```

In the first line, the string literal is used as an initializer for an array, so its characters are *copied* into a new, writable array of 6 `char`s. In the second, the literal decays to a pointer to its first character, which lives in read-only memory (Module 1). Same-looking syntax, completely different objects. Always write `const char *s = "hello";` for the pointer form, so that the compiler enforces the read-only nature for you.

Decay is why `sizeof` fails on parameters

```c
void f(int arr[10]) { printf("%zu\n", sizeof arr); }   /* prints 8 */
```

```
warning: 'sizeof' on array function parameter 'arr' will return size of 'int *'
         [-Wsizeof-array-argument]
```

> **The most common C bug in this area:** `sizeof(arr)/sizeof(arr[0])` works in the function where the array is declared, and silently gives 8/4 = 2 in any function it is passed to.

7.4 Array Parameters Are Pointers

Module 6 introduced this; here is the full picture. These four declarations are identical:

```c
void f(int arr[100]);
void f(int arr[3]);    /* the number is IGNORED */
```

```
void f(int arr[]);
void f(int *arr);
```

GCC will even point out the mismatch if you declare the same function with different "sizes":

```
warning: argument 1 of type 'int *' declared as a pointer [-Warray-parameter=]
```

The dimension in `int arr[100]` is pure documentation. Nothing enforces it, and the compiler throws it away.

The correct signature

```
int sum_n(const int *arr, size_t n);   /* pointer + length. Always. */
```

`const` documents that the function will not modify the elements, and `size_t n` is the only thing that actually protects you from reading past the end.

`[static n]` — a C99 contract

```
int sum10(int arr[static 10]);   /* "caller guarantees ≥10 elements, and non-NULL" */
```

As Module 3 noted, `static` inside the brackets is a promise rather than a check — but it gives the compiler information it can use both to diagnose mistakes and to optimise:

```
int small[3] = {1,2,3};
sum10(small);
```

```
warning: 'sum10' accessing 40 bytes in a region of size 12 [-Wstringop-overflow=]

sum10(small) reads out of bounds -> 34 (UB)
```

And under ASan:

```
ERROR: AddressSanitizer: stack-buffer-overflow
  [32, 44) 'small' (line 32) <== Memory access at offset 44 overflows this variable
```

GCC caught it at compile time, and ASan caught it at run time. `[static n]` is underused: it costs nothing and adds real diagnostics for fixed-size buffers.

2D array parameters

For a 2D array parameter, only the *first* dimension may be omitted, because the compiler needs the row width to compute `i * cols + j`:

```
void f(int m[][4], size_t rows);    /* OK — width known */
void f(int m[3][4], size_t rows);   /* OK — 3 ignored */
void f(int (*m)[4], size_t rows);   /* OK — same thing, explicit */
void f(int m[][], size_t rows);     /* ERROR — width unknown */
void f(int **m, size_t rows);       /* Compiles, but WRONG TYPE for a 2D array */
```

The first three are all the same type, `int (*)[4]`, spelled differently — the same decay rule as for one-dimensional parameters, applied to an array of rows.

For dimensions known only at run time, you have three options:

```
/* 1. VLA parameter (C99) — dimensions must be declared first */
void f(size_t rows, size_t cols, int m[rows][cols]);

/* 2. Flat array + manual indexing — most portable */
void f(int *m, size_t rows, size_t cols) { m[i*cols + j] = x; }

/* 3. Array of pointers — needed for jagged data, costs an extra indirection */
void f(int **m, size_t rows, const size_t *cols);
```

The third form is the right choice only when the rows genuinely have different lengths — for example, a list of strings.

7.5 Variable Length Arrays — and Why Shops Ban Them

```
void f(size_t n) { int v[n]; }   /* size determined at run time */
```

C99 added *variable length arrays* (VLAs): arrays whose size is an expression evaluated at run time rather than a compile-time constant. C11 then made them optional — a conforming implementation may define the macro `__STDC_NO_VLA__` and reject them.

What they can do

```
n=3 sizeof(v)=12 (sizeof is EVALUATED at runtime for VLAs)
n=6 sizeof(v)=24
```

VLA function parameters give you genuinely runtime-dimensioned 2D arrays with natural `m[i][j]` syntax:

```
void print_matrix(size_t rows, size_t cols, int m[rows][cols]);
```

```
0 1 2
3 4 5
```

That is the one thing VLAs do that nothing else does cleanly.

`sizeof` on a VLA is evaluated

```
int k = 0;
int q[(k = 7)];
printf("sizeof q = %zu, k is now %d\n", sizeof q, k);
```

```
int q[(k=7)]; sizeof q = 28, and k is now 7
```

This is the one exception to the rule from Module 4 that "`sizeof` doesn't evaluate its operand." A VLA's size is a run-time value, so the size expression must actually be computed — side effects and all.

Why they get banned

1. There is no way to detect failure.

```
size_t n = strtoul(argv[1], 0, 10);
int v[n];   /* if this doesn't fit, you just die */
```

```
$ ./vb 1000
survived: v[0]=1 v[n-1]=2

$ ./vb 100000000
Segmentation fault exit=139
```

A request for 400 MB of stack produces a silent `SIGSEGV`. There is no return value to check, no `errno`, and no chance to recover. `malloc` returns `NULL` when it cannot satisfy a request; a VLA just leaves you with a dead process.

2. It's a security vulnerability. If `n` comes from untrusted input, you have the ingredients for a *stack clash* attack: a sufficiently large VLA can move the stack pointer right past the guard page that normally catches stack overflows, landing it in the middle of the heap or another thread's stack. Subsequent writes to the "stack" then corrupt that memory. This has produced real CVEs. The Linux kernel removed every VLA from its source in 2018 for exactly this reason, with Linus Torvalds arguing that VLAs were not only a security problem but also generated noticeably worse code.

3. Worse code generation. A fixed-size array lives at a constant offset from the stack pointer. A VLA requires the stack pointer to be adjusted at run time, which forces the function to keep a frame pointer and blocks some optimisations.

4. Optional since C11. MSVC has never supported VLAs, so portable code cannot use them.

5. They interact badly with `goto`, `longjmp`, and `switch` — you cannot jump into the scope of a VLA (Module 5).

How to ban them

```
gcc -Wvla -Werror=vla
```

```
error: ISO C90 forbids variable length array 'v' [-Werror=vla]
```

Add this to your build. It is what the Linux kernel, MISRA C, CERT C (rule ARR32-C, which requires size arguments of VLAs to be validated), and most embedded and safety-critical shops do.

What to use instead

```
/* Heap — checkable failure */
int *v = malloc(n * sizeof *v);
if (!v) return -1;
...
free(v);
```

```c
/* Fixed maximum with an explicit bound check */
#define MAX_N 1024
if (n > MAX_N) return -1;
int v[MAX_N];

/* Hybrid: stack for the common case, heap for large */
int stackbuf[64];
int *v = (n <= 64) ? stackbuf : malloc(n * sizeof *v);
if (!v) return -1;
...
if (v != stackbuf) free(v);
```

Each alternative gives you a failure you can detect and handle. The hybrid pattern is what high-performance C libraries actually do: small, common requests use a fixed stack buffer and avoid `malloc` in the hot path entirely, while rare large requests fall back to the heap — safely.

> VLA *types* in function parameters (`int m[rows][cols]`) are more defensible than VLA *objects*, since no stack allocation takes place — the parameter is still just a pointer. But `-Wvla` flags both, and most codebases simply use flat arrays with manual indexing.

7.6 Designated Initializers and Compound Literals

Designated initializers (C99)

Module 3 introduced these; they are especially valuable for arrays.

```c
int sparse[10] = {[0]=1, [5]=5, [9]=9};
struct P p = {.y = 9};
```

```
1 0 0 0 0 5 0 0 0 9
struct P {.y=9} -> (0,9)
```

Everything not mentioned is zero-initialized (Module 3). The benefits: designated initializers are order-independent, self-documenting, tolerant of gaps, and refactor-safe — reordering a structure's members will not silently scramble your initializers.

The killer use case is lookup tables keyed by an enum:

```c
static const char *const state_names[] = {
    [STATE_IDLE]    = "idle",
    [STATE_RUNNING] = "running",
    [STATE_DONE]    = "done",
};
/* Array size is inferred, entries stay correct if the enum is reordered,
   and _Static_assert can verify the count. */
_Static_assert(sizeof state_names / sizeof *state_names == STATE_COUNT, "table out of sync");
```

Each string is tied directly to its enum constant rather than to a position in a list, so the table stays correct when states are added or reordered. `STATE_COUNT` is the usual trick of adding a final enumerator whose value is the number of real states. The `_Static_assert` then catches the most common mistake — adding a new last state and forgetting its name — because the table stays one entry short of `STATE_COUNT`. It cannot catch everything: a state

inserted in the *middle* of the enum without a name still enlarges the table (the later entries move up), leaving a `NULL` hole. A quick loop in a unit test that checks every entry is non-null closes that gap.

> ⚠ **GNU range syntax** `[2 ... 5] = 7` is a GCC extension, not standard C. It works:
>
> ```
> GNU range [2...5]=7: 0 0 7 7 7 7 0 0 0 0
> ```
>
> …but `-pedantic` rejects it. Avoid it in portable code.

Compound literals (C99)

A compound literal is an *unnamed object created inside an expression*:

```
(type){ initializers }
```

It looks like a cast followed by an initializer list, and it creates a real object of the given type, which you can use anywhere an object of that type is allowed:

```c
show((struct P){3, 4});                /* pass a temporary struct */
sum((int[]){1,2,3,4,5}, 5);            /* pass a temporary array */
struct P *pp = &(struct P){7, 8};      /* take its address */
```

```
(3,4)
sum=15
&(struct P){7,8} -> (7,8)
```

Compound literals are modifiable (unlike string literals):

```c
int *m = (int[]){1,2,3};
m[0] = 99;   /* legal */
```

```
modified: 99 2 3
```

⚠ Compound literal lifetime

At block scope, a compound literal has **automatic storage duration** — it dies at the end of the enclosing block, exactly like a named local variable.

```c
const int *make_bad(void) { return (const int[]){1,2,3}; }            /* DANGLING */
const int *make_ok(void)  { static const int a[]={1,2,3}; return a; }
```

```
make_bad()[0] = 1   <-- UB, dangling
make_ok()[0]  = 1   <-- safe (static)
```

ASan confirms:

```
ERROR: AddressSanitizer: stack-use-after-return
```

It printed 1 — the memory had not been reused yet. That is the worst possible outcome: the code looks correct in testing and corrupts data in production, the moment some other function call happens to overwrite that stack space.

At file scope, a compound literal has static duration, and it is safe to return a pointer to it. In C23 you can also write `(static int[]){1,2,3}` at block scope to give a compound literal static duration explicitly.

Where compound literals shine

```
/* Passing config structs without a named temporary */
init_widget(&(WidgetConfig){ .width = 100, .height = 50, .title = "Hi" });

/* Returning a struct by value from a helper */
static Point pt(int x, int y) { return (Point){x, y}; }

/* Array arguments inline */
draw_polygon((Point[]){{0,0},{1,0},{1,1}}, 3);

/* Zero-initializing in an assignment */
mystruct = (MyStruct){0};   /* struct assignment, cleaner than memset */
```

The combination of compound literals and designated initializers gives C something close to named arguments: `init_widget` above receives a fully initialized configuration, with every unmentioned field zeroed, and the call site says exactly what each value means.

7.7 Flexible Array Members

```
struct Msg {
    size_t len;
    char   data[];   /* C99 flexible array member — no size */
};
```

A *flexible array member* (FAM) is an array with no stated size, declared as the last member of a structure. It lets a single allocation hold a fixed-size header immediately followed by a variable amount of data. Compare it with the two older alternatives — the pre-C99 “struct hack” (`char data[1]`) and a separately allocated buffer (`char *data`):

```
sizeof(struct Msg)  = 8   <- FAM contributes ZERO to sizeof
sizeof(struct Hack) = 16  <- the [1] costs a byte (+padding)
sizeof(struct Ptr)  = 16  <- pointer + separate allocation
```

The allocation pattern

```
struct Msg *msg_new(const char *s)
{
    size_t n = strlen(s);
    struct Msg *m = malloc(sizeof *m + n + 1);   /* header + payload, ONE alloc */
    if (!m) return NULL;
    m->len = n;
    memcpy(m->data, s, n + 1);
    return m;
}
```

```
len=11 data="hello world"
&m       = 0x55efd987b2b0
&m->data = 0x55efd987b2b8 (offset 8 -> immediately after the header)
ONE allocation, ONE free, ONE cache line
```

You allocate the size of the header plus however many bytes of data you need, in one call to `malloc`. The data begins right where the header ends.

Why this beats the alternatives

	Flexible array member	Pointer member
Allocations	1	2
`free` calls	1	2 (easy to leak)
Cache locality	header + data adjacent	two unrelated heap locations
Extra pointer	none	8 bytes
Failure paths	1	2

For network packets, variable-length records, string objects, and anything else you allocate millions of, this is a meaningful win. It is why Linux's `struct inotify_event` ends with a flexible `name[]` member, and why countless kernel structures use the same pattern. (The sockets API's `struct sockaddr`, with its fixed `sa_data[14]`, is an older design in the same spirit: a generic header that is really followed by address-family–specific data of varying length.)

The rules

- The flexible array member must be the last member.
- The structure must have at least one other member.
- A structure with a flexible array member cannot be a member of another structure or an element of an array.
- `sizeof` the structure excludes the flexible member — which is exactly why the `sizeof *m + n` idiom works.
- **Structure assignment does not copy the flexible part.** `*a = *b;` copies only the header. Use `memcpy` with the real total size.

The "struct hack" — its predecessor

Before C99, programmers wrote `char data[1];` and over-allocated. It was technically undefined behaviour (it accesses the array beyond its declared bound), it wastes a byte plus padding, and it makes the size arithmetic error-prone: you must write `sizeof *m + n - 1`, and forgetting the `- 1` is easy. Use the C99 flexible array member.

Get the size arithmetic wrong and it's a heap overflow

```c
struct Msg *m = malloc(sizeof *m);   /* forgot the payload! */
memcpy(m->data, "hello", 6);
```

```
ERROR: AddressSanitizer: heap-buffer-overflow
SUMMARY: heap-buffer-overflow in memcpy
```

The allocation covers only the 8-byte header, so the 6 bytes copied into `data` land past the end of the block.

When the length comes from untrusted input, always guard against integer overflow in the size computation:

```
if (n > SIZE_MAX - sizeof *m - 1) return NULL;   /* overflow check FIRST */
struct Msg *m = malloc(sizeof *m + n + 1);
```

If `n` is close to `SIZE_MAX`, the sum `sizeof *m + n + 1` wraps around to a small number (unsigned arithmetic, Module 2). `malloc` then succeeds with a tiny buffer, and the program proceeds to write a huge amount of data into it. That combination — a small allocation followed by a large write — is a classic remote-code-execution primitive.

Module 7 Rules

#	Rule
1	`a[i]` is `*(a+i)` — subscripting is pointer arithmetic
2	`a` and `&a` have the same value but different types and different arithmetic
3	Arrays decay everywhere except `sizeof`, `&`, `_Alignof`, `char s[]="..."`, and `typeof`
4	Array parameters are pointers — the dimension is documentation only
5	Always pass (pointer, length); consider `[static n]` for fixed minimums
6	`int[3][4]` decays to `int (*)[4]`, never `int **`
7	Iterate 2D arrays in row-major order for cache performance
8	Ban VLAs: `-Werror=vla`. Use `malloc` (checkable) or a bounded fixed array
9	Designated initializers for enum-keyed tables; add a `_Static_assert` on the count
10	Block-scope compound literals die with the block — never return pointers to them
11	Flexible array members for variable-length records: one allocation, one free
12	Check for `size_t` overflow before `malloc(sizeof *m + n)`

Standing build command, updated:

```
gcc -std=c17 -Wall -Wextra -Wpedantic -Wshadow -Wconversion \
    -Werror=vla -Warray-bounds=2 -Wstringop-overflow=4 \
    -Werror=return-type -Werror=implicit-function-declaration \
    -g -fsanitize=address,undefined prog.c -o prog
```

Practice

1. **Prove row-major.** Declare `int m[4][5]`, fill it with `i*10+j`, then print it through an `int *` walking linearly. Verify that the order matches your prediction. Then measure the run-time difference between row-major and column-major traversal of a 2000×2000 array.

2. **Decay checklist.** Write one program that demonstrates each of the five non-decay contexts, printing sizes and using `_Generic` (or `__builtin_types_compatible_p`) to prove that the types differ.

3. **The `sizeof` trap.** Write `void print_all(int arr[])` that tries `sizeof(arr)/sizeof(arr[0])`. Call it with a 10-element array. Then fix it. Then write an `ARRAY_LEN` macro that fails to compile when given a pointer.

4. **2D parameter forms.** Implement a matrix sum four ways: `int m[][4]`, `int (*m)[4]`, a VLA parameter `int m[r][c]`, and a flat `int *m` with manual indexing. Verify that all four give the same answer. Which of them compile under `-Werror=vla`?

5. **VLA danger demo.** Write a program that takes `n` from `argv` and declares `int v[n]`. Find the largest `n` that works and the smallest that crashes. Then rewrite it with `malloc` and confirm that you get a clean error instead of a segfault at the same size.

6. **Compound literal lifetime.** Reproduce the dangling `make_bad()` case. Run it under ASan. Then add three intervening function calls before printing and observe the value change — proving that the memory really was reclaimed.

7. **Build a FAM string type.** Implement:

   ```
   typedef struct { size_t len, cap; char data[]; } Str;
   Str *str_new(const char *s);
   Str *str_append(Str *s, const char *t);   /* may realloc */
   void str_free(Str *s);
   ```

   Include integer-overflow checks on every size computation. Verify with ASan and LSan that a workload of 10,000 appends is clean.

8. **Reasoning, no compiler.** Given `int a[3][4];`, what are the types and values of: `a`, `&a`, `a[0]`, `&a[0]`, `*a`, `a+1`, `&a+1`, `a[0]+1`, `sizeof a`, `sizeof a[0]`, and `sizeof *a`?

Module 7 in Summary

A C array is nothing but contiguous elements — no length, no bounds checks — and subscripting is defined as pointer arithmetic, `a[i] == *(a+i)`. In almost every expression an array decays to a pointer to its first element; the exceptions are `sizeof`, `&`, `_Alignof`, string-literal initialization of a `char` array, and type queries. `a` and `&a` share an address but not a type, which is why their arithmetic differs. Arrays cannot be assigned, compared, passed, or returned, but a structure containing one can.

Multidimensional arrays are arrays of arrays stored in row-major order; they decay to pointers to rows, never to `int **`, and should be traversed in memory order. Array parameters are pointers, so the length must always travel with them, and `[static n]` adds free diagnostics.

VLAs fail silently, invite stack-clash attacks, and are optional in the standard — ban them and use `malloc`, a bounded buffer, or the stack-then-heap hybrid. Designated initializers make enum-keyed tables robust, and compound literals create unnamed objects that die with their block. Finally, flexible array members put a header and its variable-length data in a single allocation — as long as the size arithmetic is checked for overflow.

In Module 8, we turn to Strings — the null-terminated convention, the `string.h` toolkit, which functions are dangerous, safe conversions, and UTF-8.

Module 8 — Strings

In this module:

- The null-terminated convention, `'\0'`
- `string.h`: `strlen`, `strcpy`, `strncpy`, `strcat`, `strcmp`, `strchr`, `strstr`, `strtok`, `memcpy`, `memmove`, `memset`, `memcmp`
- Why `strcpy`/`strcat`/`sprintf`/`gets` are dangerous; `snprintf`, `strlcpy`/`strlcat`, `strnlen`
- `memcpy` vs `memmove` (overlap)
- Conversions: `atoi` (bad) vs `strtol`/`strtod` (good, with `errno` + `endptr`)
- Wide strings, multibyte, UTF-8 handling in C
- Building a string/buffer abstraction

All output in this module is real, from GCC 13.3 and glibc 2.39.

C has no string type. It has a *convention* — a way of using `char` arrays that everyone agrees to follow — and a library of functions built around that convention. The convention is simple, compact, and fast, and it has a set of costs that every C programmer pays sooner or later. Practically every string-related bug of the last fifty years — from the Morris Worm to countless modern CVEs — traces back to those costs. This module explains the convention, shows exactly where its costs come from, and ends by building the kind of string abstraction that professional C code uses to escape them.

8.1 The Null-Terminated Convention

A C “string” is a `char` array whose end is marked by a `'\0'` byte — a byte with the value zero, called NUL. There is no length field anywhere. The only way to know where a string ends is to look for the NUL.

```
char s[10] = "hi"; bytes: 68 69 00 00 00 00 00 00 00 00
strlen(s)=2 sizeof(s)=10   <- LENGTH != CAPACITY
```

That example shows two different numbers, and you must always track them separately:

- **Length** — the number of bytes before the NUL, as reported by `strlen`.
- **Capacity** — the number of bytes the buffer can hold, from `sizeof` or from however much you allocated.

A string of length n needs a capacity of at least $n + 1$, because the NUL takes a byte too. Confusing length with capacity — or forgetting the + 1 — is the root of most buffer overflows.

`'\0'` vs `'0'` vs `"0"` vs `NULL`

```c
'\0'   /* the NUL character, value 0, type int (it's a char constant) */
'0'    /* the digit zero, value 48 */
```

```
"0"    /* a 2-byte array: '0', '\0' */
NULL   /* a null POINTER */
```

Four different things that are routinely mixed up. `'\0'` is a character that ends a string; `'0'` is the printable digit; `"0"` is a whole string containing that digit; and `NULL` is not a character at all but a pointer that points nowhere. `if (*s == '\0')` and `if (!*s)` are equivalent and idiomatic ways to ask “is this string empty?”. `if (s == NULL)` asks something entirely different: “is there a string at all?”

The three consequences

1. `strlen` is O(n). With no stored length, `strlen` has to walk the string until it finds the NUL.

```
for (size_t i = 0; i < strlen(s); i++)   /* O(n²) */

size_t n = strlen(s);
for (size_t i = 0; i < n; i++)           /* O(n) */
```

This is not hypothetical — the O(n²) version (Module 5) is a classic performance bug in string-processing loops, and it tends to go unnoticed until someone feeds the program a long input.

2. Strings cannot contain embedded NULs.

```
char e[] = "abc\0def";
```

```
sizeof=8 strlen=3
bytes: 61 62 63 00 64 65 66 00
```

The array holds all 8 bytes, but every `str*` function stops at index 3, because that is where it finds a NUL. You cannot store arbitrary binary data — images, compressed data, network packets — in a C string. Use the `mem*` functions, which take an explicit length, for binary data.

3. A missing terminator reads off the end.

```
char bad[4] = {'a','b','c','d'};   /* no NUL */
strlen(bad);
```

```
strlen(bad) = 7   <-- read past the array, UB

ASan: stack-buffer-overflow
  [48, 52) 'bad' <== Memory access at offset 52 overflows this variable
```

`strlen` kept going past the end of `bad` into whatever memory followed, until it happened to meet a zero byte three bytes later. Any function that expects a string will do the same.

Recall from Module 3 that `char s[3] = "abc"` silently drops the terminator. Always write `char s[] = "abc"` and let the compiler count.

8.2 The `<string.h>` Toolkit

Length and comparison

Function	Signature	Notes
`strlen`	`size_t strlen(const char *)`	O(n). UB if not terminated.
`strnlen`	`size_t strnlen(const char *, size_t n)`	POSIX; safer: stops after `n` bytes. Use for untrusted data.
`strcmp`	`int strcmp(const char *, const char *)`	<0, 0, >0. *Not* −1/0/1 — never assume.
`strncmp`	`int strncmp(const char *, const char *, size_t)`	Compares at most `n` bytes.
`memcmp`	`int memcmp(const void *, const void *, size_t)`	Byte-wise. Not constant time — see below.

⚠ `strcmp` returns "some negative, zero, or positive value." Testing `if (strcmp(a,b) == -1)` is a bug: glibc, for example, may return the difference between the first mismatching bytes. Always compare the result with 0: `== 0`, `< 0`, `> 0`.

⚠ Never `memcmp` two structures to compare them — their padding bytes are indeterminate (Module 3).

⚠ **Never use `memcmp` for secrets** — passwords, HMACs, authentication tokens. It stops at the first differing byte, so the time it takes reveals how many leading bytes of an attacker's guess were correct. By measuring response times, an attacker can recover a secret byte by byte. Use a constant-time comparison instead:

```c
static int ct_equal(const void *a, const void *b, size_t n)
{
    const unsigned char *x = a, *y = b;
    unsigned char diff = 0;
    for (size_t i = 0; i < n; i++) diff |= x[i] ^ y[i];
    return diff == 0;
}
```

This version always examines every byte, accumulating any differences with OR, so its running time depends only on `n` — never on where (or whether) the inputs differ.

Copying and concatenation

Function	Safety	Verdict
`strcpy`	none	✗ Banned
`strcat`	none	✗ Banned
`sprintf`	none	✗ Banned
`gets`	none	✗ Removed in C11

Function	Safety	Verdict
`strncpy`	partial, misleading	⚠ Not what you think
`strncat`	partial, confusing `n`	⚠ Error-prone
`snprintf`	good	✓ Use this
`strlcpy/strlcat`	good	✓ Where available
`memcpy/memmove`	you supply the length	✓ With correct arithmetic

§8.3 explains each verdict.

Searching

Function	Finds
`strchr(s, c)`	First occurrence of `c` (the NUL matches too — `strchr(s, 0)` finds the terminator)
`strrchr(s, c)`	Last occurrence
`strstr(hay, needle)`	First occurrence of a substring
`memchr(p, c, n)`	First matching byte within `n` bytes — works on binary data
`strspn` / `strcspn`	Length of the initial run of characters that are (not) in a given set
`strpbrk`	First occurrence of any character from a set

All of them return `NULL` when nothing is found. `strchr` and `strstr` return a pointer *into the original string*, not a copy: don't `free` it, and beware that it dangles if the underlying buffer is later `realloc`ed.

`strtok` — avoid it

```c
char *tok = strtok(buf, ",");
while (tok) { ...; tok = strtok(NULL, ","); }
```

`strtok` splits a string into tokens, and it has three fatal problems:

1. **It modifies its input in place,** writing NULs over the delimiters. It cannot be given a `const char *` or a string literal.
2. **It keeps hidden static state** to remember where it stopped, which is how `strtok(NULL, ...)` knows where to continue. So it is not reentrant, not thread-safe, and you cannot tokenize two strings at once — including indirectly, if a function called inside your loop also uses `strtok`.
3. **It collapses consecutive delimiters,** treating `"a,,b"` as two tokens, so it cannot parse CSV data containing empty fields.

Use `strtok_r` (POSIX) or `strtok_s` (C11 Annex K, and Windows), which take an explicit state pointer that you own:

```
char *save;
for (char *t = strtok_r(buf, ",", &save); t; t = strtok_r(NULL, ",", &save)) { ... }
```

Or write your own splitter using `strcspn`, which finds the length of the next field; it is usually clearer, and it handles empty fields correctly.

8.3 Why `strcpy` / `strcat` / `sprintf` / `gets` Are Banned

```
char dst[8];
strcpy(dst, "this is way too long");
```

At compile time (with `-O2`, thanks to `_FORTIFY_SOURCE`):

```
warning: '__builtin___memcpy_chk' forming offset [8, 20] is out of the bounds
         [0, 8] of object 'dst' with type 'char[8]' [-Warray-bounds=]
```

At run time with ASan:

```
ERROR: AddressSanitizer: stack-buffer-overflow
  [32, 40) 'dst' <== Memory access at offset 40 overflows this variable
```

At run time with `-D_FORTIFY_SOURCE=3`:

```
*** buffer overflow detected ***: terminated
Aborted
```

The problem is structural. These functions have no way to know the size of the destination: they receive only a pointer, and they write until they reach a NUL in the *source*. If the source is longer than the destination, they scribble over whatever comes next in memory — other local variables, saved registers, and eventually the return address, as Module 6's stack-frame diagram showed. Control the bytes written over the return address and you control where the function returns to. That is arbitrary code execution.

Here the overflow was caught three different ways, but only because the source was a constant the compiler could see, or because the program was built with extra checking. With input read at run time and an ordinary build, nothing would have stopped it.

`gets` was removed from the language

```
char b[8]; gets(b);
```

```
warning: implicit declaration of function 'gets'; did you mean 'fgets'?
warning: the `gets' function is dangerous and should not be used.
```

`gets` reads a line from standard input into a buffer — and it has no size parameter at all. There is no way to call it safely: any input line longer than the buffer overflows it. It was deprecated in C99 and removed entirely in C11, which is why the compiler above no longer even has a declaration for it. The Morris Worm of 1988, one of the first worms to spread across the internet, used a `gets` overflow in the `fingerd` daemon.

Use `fgets(buf, sizeof buf, stdin)` instead — but note that it keeps the newline:

```
if (fgets(buf, sizeof buf, stdin)) {
    buf[strcspn(buf, "\n")] = '\0';   /* strip newline, idiomatically */
}
```

`strcspn` returns the index of the first `'\n'`, or the length of the string if there is none, so this line works in both cases.

`strncpy` is not a safe `strcpy`

This is the single most misunderstood function in the C library. It was designed in 1979 for copying names into *fixed-width fields* in on-disk records — where filling every unused byte with NUL was the desired behaviour — not for strings.

Trap 1 — it does not terminate the result if the source fills the buffer:

```
char a[6]; strncpy(a, "abcdef", 6);
```

```
strncpy(a,"abcdef",6) 61 62 63 64 65 66
-> no 00 at the end. strlen(a) would run off.

warning: 'strncpy' output truncated before terminating nul [-Wstringop-truncation]
```

Trap 2 — it pads with NULs, always writing exactly `n` bytes:

```
char b[10]; strncpy(b, "hi", 10);
```

```
strncpy(b,"hi",10) 68 69 00 00 00 00 00 00 00 00
-> wrote 10 bytes to copy 2. strncpy is NOT 'safe strcpy'.
```

So `strncpy` into a 4 KB buffer writes 4 KB, regardless of how short the string is. Its cost is proportional to the buffer size, not the string length.

If you must use it:

```
strncpy(dst, src, sizeof dst - 1);
dst[sizeof dst - 1] = '\0';   /* mandatory manual terminate */
```

`strncat`'s `n` means something different

In `strncat(dst, src, n)`, `n` is the *maximum number of bytes to append* — not the size of the buffer. And `strncat` always adds a NUL afterwards, so it may write up to `n + 1` bytes beyond the existing content. The correct computation is:

```
strncat(dst, src, sizeof dst - strlen(dst) - 1);   /* the correct computation */
```

```
strncat(d,...,sizeof d - strlen(d) - 1) -> "abcdefghi" (len 9)
```

Get that arithmetic wrong by one and you overflow. It is also O(n) in `strlen(dst)` on every call, because it must first find the end of the existing string. Calling it repeatedly to build up a long string is “Schlemiel the Painter’s algorithm” — the painter who walks back to the paint can at the start of the road for every stroke — and it is $O(n^2)$ overall.

`snprintf` — the workhorse

```
int snprintf(char *dst, size_t size, const char *fmt, ...);
```

Its guarantees:

- It writes at most `size` bytes, *including* the NUL.
- It always NUL-terminates the output (when `size > 0`).
- It returns the number of characters that *would have been written* had the buffer been large enough, not counting the NUL.

That return value is the key:

```
char e[8];
int r = snprintf(e, sizeof e, "%s-%d", "value", 12345);
```

```
snprintf(e,8,...) -> ret=11, e="value-1" (len 7)
ret >= sizeof(e) means TRUNCATION occurred: YES
```

The full result, `"value-12345"`, needs 11 characters; only 7 fit, plus the terminator. The output is safe but incomplete, and the return value tells you so.

The truncation-check idiom:

```
int r = snprintf(buf, sizeof buf, fmt, ...);
if (r < 0 || (size_t)r >= sizeof buf) {
    /* error or truncated — handle it, don't ignore it */
}
```

A silently truncated file path or SQL query can be as dangerous as an overflow, so check.

The measure-then-allocate idiom:

```
int n = snprintf(NULL, 0, fmt, args);   /* size==0, dst may be NULL */
char *buf = malloc((size_t)n + 1);
snprintf(buf, (size_t)n + 1, fmt, args);
```

With a size of zero, `snprintf` writes nothing and simply reports how long the output would be, so you can allocate exactly the right amount. (Check the `malloc` result, as always.)

> ⚠ Never write `snprintf(buf, sizeof buf, buf)`, or otherwise let the source and destination overlap — that is undefined behaviour.

> ⚠ On old MSVC, `_snprintf` did *not* terminate on truncation, and returned −1. If you support ancient Windows toolchains, wrap it.

`strlcpy` / `strlcat` — OpenBSD's answer

```
size_t strlcpy(char *dst, const char *src, size_t dstsize);
size_t strlcat(char *dst, const char *src, size_t dstsize);
```

- They always NUL-terminate (if `dstsize > 0`).
- They never pad.
- They return `strlen(src)` (or `strlen(dst) + strlen(src)` for `strlcat`) — the length of the string they *tried* to create — so `ret >= dstsize` means truncation, exactly like `snprintf`.
- `strlcat` takes the *full* buffer size, not the remaining space, which makes it much harder to get wrong than `strncat`.

```
strlcpy(d, "toolongstring", 8) -> returns 13, d = "toolong"
```

Availability: BSD, macOS, and glibc 2.38 and later (the machine used for this book runs 2.39). They are not in ISO C, not in older glibc, and not in MSVC. If you need portability, ship your own ten-line implementation.

Annex K `_s` functions — don't bother

C11's optional Annex K added "bounds-checked" functions such as `strcpy_s` and `sprintf_s`. Only Microsoft implements them meaningfully (and its versions differ from the standard), glibc refused to adopt them, and the C committee has discussed removing the annex. Skip it.

8.4 `memcpy` vs `memmove`

The prototypes tell the whole story:

```
void *memcpy (void *restrict dst, const void *restrict src, size_t n);
void *memmove(void *dst,          const void *src,          size_t n);
```

`restrict` (Module 2) is a promise that the two regions do not overlap. Break that promise and the behaviour is undefined — the implementation is free to copy in any order, use wide vector loads and stores, or read from both ends at once.

`memmove` makes no such promise, and is guaranteed to be correct for overlapping regions — it behaves *as if* the source were first copied into a temporary buffer.

The honest test result

```
shift RIGHT by 2 (dst > src) — this is where memcpy breaks:
  memcpy(b+2,b,8)  "ABABCDEFGH"
  memmove(b+2,b,8) "ABABCDEFGH"
  expected: ABABCDEFGH
```

Both gave the right answer. Even at 4096 bytes, with the offset hidden from the optimiser, glibc's `memcpy` produced correct output.

That is exactly why the bug is dangerous. Overlapping `memcpy` works on your machine, with your glibc version, at your buffer size — until it doesn't. Around 2010, glibc changed its x86-64 `memcpy` to copy in a different order for performance, and Adobe Flash broke on Linux because it had been relying on overlapping `memcpy`. Linus Torvalds and glibc maintainer Ulrich Drepper had a famous public argument about whether glibc should accommodate the broken code. The underlying answer was never in doubt: the code had always been wrong; it had just been lucky.

What actually protects you is the compiler, which caught the problem statically:

```
warning: '__builtin_memcpy' accessing 8 bytes at offsets 0 and 2
         overlaps 6 bytes at offset 2 [-Wrestrict]
```

> **Rule:** if the regions might overlap, use `memmove`. The performance difference is negligible; on most platforms `memmove` just compares the two pointers once and then chooses a copy direction.

The other `mem*` functions

```c
memset(p, 0, n);   /* fill with a BYTE value — memset(a,1,n) is NOT all-ones ints */
memchr(p, c, n);   /* search binary data */
memcmp(a, b, n);   /* see the constant-time warning above */
```

`memset` sets every *byte* to the given value. `memset(a, 1, sizeof a)` on an `int` array does not give you ints equal to 1; it gives you ints whose four bytes are each 1, which is 16,843,009. Only 0 (and −1, all bits set) work as you might hope.

> ⚠ **A `memset` that zeroes secrets can be optimised away.** If the compiler sees you clear a buffer that is about to go out of scope and is never read again, it treats the stores as dead and deletes them — leaving the password or key in memory. Use `explicit_bzero` (glibc/BSD), `memset_explicit` (C23), or `SecureZeroMemory` (Windows), which are guaranteed not to be removed.

8.5 String → Number Conversion

Programs constantly need to turn text — command-line arguments, configuration values, fields in a file — into numbers. C offers two families of functions for it. One is simple and dangerous; the other is awkward and correct.

`atoi` is unusable in production

```
atoi("42")                    = 42
atoi("abc")                   = 0
atoi("0")                     = 0
atoi("99999999999999999999") = -1
```

`"abc"` and `"0"` both return 0, so you cannot tell an error from a valid zero. And when the value is out of range, the behaviour is undefined — here it happened to produce −1. `atoi` is specified as behaving like `(int)strtol(s, NULL, 10)`, but with all of the error reporting thrown away.

The same applies to `atol`, `atoll`, and `atof`. Never use them.

`strtol` done properly

```
long strtol(const char *nptr, char **endptr, int base);
```

`strtol` reports problems through three separate channels, and a correct caller must check all of them:

```c
static int parse_int(const char *s, long *out)
{
    char *end;
    errno = 0;                                      /* MUST clear first */
    long v = strtol(s, &end, 10);

    if (end == s)                   return -1;      /* no digits consumed */
    while (*end == ' ' || *end == '\t') end++;      /* allow trailing space */
    if (*end != '\0')               return -2;      /* trailing garbage */
    if (errno == ERANGE)            return -3;      /* out of long range */
    if (v < INT_MIN || v > INT_MAX) return -4;      /* out of int range */

    *out = v;
    return 0;
}
```

```
42                    -> OK value=42
-17                   -> OK value=-17
abc                   -> no digits
42abc                 -> trailing garbage
99999999999999999999  -> ERANGE (long overflow)
0x1F                  -> trailing garbage (base 10 stops at 'x')
(empty string)        -> no digits
2147483648            -> out of int range
```

Every kind of bad input produces its own distinct error, and only genuinely valid integers get through.

The four rules:

1. **Set `errno = 0` before the call.** `strtol` sets `errno` only when an error occurs; it never clears it. A stale `ERANGE` left over from some earlier call would otherwise look like a new error.
2. **`end == nptr` means no conversion happened.** Checking only `errno` misses input like `"abc"` entirely, because that is not a range error — it simply contains no number.
3. **Check `*end` for trailing garbage.** Otherwise `"42abc"` silently yields 42. `strtol` does skip *leading* whitespace for you; trailing whitespace is up to you, as above.
4. **Range-check separately** if your target type is narrower than `long`. `ERANGE` only fires at the limits of `long`, which on 64-bit Linux are far beyond those of `int`.

Bases

```
strtol("0x1F", &e, 0) = 31   (base 0 auto-detects)
strtol("017",  &e, 0) = 15   (leading 0 = octal!)
```

A base of 0 accepts C literal syntax: a `0x` prefix means hexadecimal, and a leading `0` means octal. That surprises users who type leading zeros — a user entering `017` means seventeen, not fifteen. Use an explicit base of 10 for user input, unless you specifically want C syntax.

The family

Function	Returns
`strtol` / `strtoll`	`long` / `long long`
`strtoul` / `strtoull`	unsigned — ⚠ accepts `"-1"` and wraps to `ULONG_MAX`
`strtod` / `strtof` / `strtold`	floating point
`strtoimax` / `strtoumax`	`intmax_t` / `uintmax_t` (`<inttypes.h>`)

```
strtod("3.14xyz") = 3.14, stopped at "xyz"
```

⚠ `strtoul("-1")` returns `ULONG_MAX`, not an error — the standard specifies that a minus sign negates the result in the unsigned type. If negative input is invalid, check for a `'-'` yourself.

⚠ `strtod` is locale-dependent. In a German (`de_DE`) locale, the decimal separator is `,`, so `"3.14"` stops at the `.`. For parsing configuration files and network protocols, either force the `"C"` locale or use `strtod_l` with an explicit locale.

`sscanf` — usable but leaky

`sscanf` cannot report overflow, and a `%s` conversion without a width is a buffer overflow waiting to happen. If you use it, always give `%s` a maximum width — one less than the buffer size — and check the return value, which is the number of fields successfully converted:

```
char name[32];
if (sscanf(line, "%31s %d", name, &num) != 2) { /* parse error */ }
```

8.6 Wide Strings, Multibyte, and UTF-8

A `char` is one byte, and one byte can distinguish only 256 values — nowhere near enough for the world's writing systems. C has two broad approaches to text beyond ASCII: wider character types, and multibyte encodings that use several `char`s per character.

The `wchar_t` mess

Platform	`sizeof(wchar_t)`	Encoding
Linux/macOS	4	UTF-32
Windows	2	UTF-16 (so it cannot hold every code point in one unit)

`wchar_t` was meant to hold "one character" per element, but its size and encoding vary by platform. On Windows, characters outside the Basic Multilingual Plane — including most emoji — need two `wchar_t` units, so even there it is not one character per element. `wchar_t` is not portable, and it does not solve Unicode. C11 added `char16_t` and `char32_t` in `<uchar.h>`, with explicit encodings, but almost nobody uses them.

The modern answer: UTF-8 in plain `char`

```
const char *s = "héllo→世界";
```

```
strlen() = 15   <- BYTES
u8_len() = 8    <- CODE POINTS

byte dump: 68 C3 A9 6C 6C 6F E2 86 92 E4 B8 96 E7 95 8C
```

The string has 8 characters (Unicode *code points*), but it occupies 15 bytes: `h`, `l`, `l`, `o` take one byte each, `é` takes two, and `→`, `世`, and `界` take three each.

UTF-8 was designed by Ken Thompson and Rob Pike in 1992 specifically so that it would work with existing C string functions. The properties that make it work:

Property	Consequence
ASCII (0x00–0x7F) is unchanged	All existing ASCII text and code keeps working
No byte of a multibyte sequence is ever 0x00	`strlen`, `strcpy`, `strchr` don't break
Lead bytes are `11xxxxxx`, continuation bytes are `10xxxxxx`	Self-synchronising — you can find a character boundary from anywhere
Multibyte sequences never contain an ASCII byte	`strchr(s, '/')` and `strstr` give no false positives
Byte order matches code-point order	`strcmp` sorts correctly by code point

The second and fourth properties are the crucial ones. Because every byte of a multibyte sequence has its high bit set, no part of `世` can ever be mistaken for a NUL terminator, a slash, a comma, or any other ASCII character. So all the ASCII-oriented code in the world — path handling, CSV splitting, HTTP parsing — works on UTF-8 text without modification.

The encoding

Code points	Bytes	Pattern
U+0000–U+007F	1	`0xxxxxxx`
U+0080–U+07FF	2	`110xxxxx 10xxxxxx`

Code points	Bytes	Pattern
U+0800–U+FFFF	3	`1110xxxx 10xxxxxx 10xxxxxx`
U+10000–U+10FFFF	4	`11110xxx 10xxxxxx 10xxxxxx 10xxxxxx`

The number of leading 1 bits in the first byte tells you how many bytes the sequence has; the `x` bits, concatenated, form the code point.

Counting code points is trivial — count the bytes that are *not* continuation bytes:

```
size_t u8_len(const char *s)
{
    size_t n = 0;
    for (; *s; s++) if ((*s & 0xC0) != 0x80) n++;
    return n;
}
```

`*s & 0xC0` keeps only the top two bits; a continuation byte is exactly the case where those bits are `10`.

Decoding each sequence back into a code point gives:

```
U+0068 (1 bytes) "h"
U+00E9 (2 bytes) "é"
U+2192 (3 bytes) "→"
U+4E16 (3 bytes) "世"
U+754C (3 bytes) "界"
```

The practical rules

1. **Store and process everything as UTF-8 in `char` arrays.** Use `wchar_t` only at the boundary with Windows APIs, converting on the way in and out.
2. **Most operations don't need decoding at all.** Concatenation, comparison, searching for ASCII delimiters, hashing, and copying all work byte by byte.
3. **Decode only when you need code points** — for cursor movement, counting characters, or case conversion.
4. **Never split a UTF-8 string at an arbitrary byte offset.** To find a safe split point, back up while `(b & 0xC0) == 0x80`, so that you land on the start of a character.
5. **Validate untrusted UTF-8 on input.** Reject overlong encodings (such as `C0 80` for NUL), surrogates (U+D800–U+DFFF), and values above U+10FFFF. Overlong encodings in particular have been used to bypass security filters: a filter that looks for the byte `/` can miss a `/` encoded in two bytes, which a lax decoder later turns back into a real slash — a classic path-traversal attack.
6. **Code points are not user-perceived characters.** "é" can be one code point, or two (`e` plus a combining accent), and many emoji are sequences of several code points joined by zero-width joiners. Real grapheme clustering needs a library such as ICU or utf8proc — don't roll your own.
7. **`printf("%s", utf8)` just works,** because it is byte-transparent. But `%-10s` pads by *bytes*, so column alignment will be wrong for non-ASCII text.

8.7 Building a String Abstraction

Serious C code stops passing raw `char *` around and builds a *length-carrying* string type. Here is a complete, verified one — a *string builder* that grows as you append to it:

```c
typedef struct { char *data; size_t len, cap; } SB;

static bool sb_reserve(SB *b, size_t need)   /* need = extra bytes, excl NUL */
{
    if (need > SIZE_MAX - 1 - b->len) return false;   /* overflow guard */
    size_t want = b->len + need + 1;
    if (want <= b->cap) return true;
    size_t cap = b->cap ? b->cap : 16;
    while (cap < want) { if (cap > SIZE_MAX/2) return false; cap *= 2; }
    char *p = realloc(b->data, cap);
    if (!p) return false;                              /* b->data still valid! */
    b->data = p; b->cap = cap;
    return true;
}

static bool sb_append(SB *b, const char *s, size_t n)
{
    if (!sb_reserve(b, n)) return false;
    memcpy(b->data + b->len, s, n);
    b->len += n;
    b->data[b->len] = '\0';
    return true;
}

__attribute__((format(printf, 2, 3)))
static bool sb_printf(SB *b, const char *fmt, ...)
{
    va_list ap, ap2;
    va_start(ap, fmt); va_copy(ap2, ap);
    int n = vsnprintf(NULL, 0, fmt, ap);   /* measure */
    va_end(ap);
    bool ok = false;
    if (n >= 0 && sb_reserve(b, (size_t)n)) {
        vsnprintf(b->data + b->len, b->cap - b->len, fmt, ap2);
        b->len += (size_t)n;
        ok = true;
    }
    va_end(ap2);
    return ok;
}

static void sb_free(SB *b) { free(b->data); *b = (SB){0}; }
```

```
result: "Hello, world! You are 42 years old. [raw]"
  len=41 cap=64

growth under 10000 appends:
  final len=48890 cap=65536 (amortized O(1) per append)

ASan/UBSan/LSan exit=0
```

An `SB` starts out as `(SB){0}` — no buffer at all — and allocates on the first append. `sb_reserve` is the heart of it: it makes sure there is room for `need` more bytes plus the terminator, growing the buffer if necessary. `sb_append` copies raw bytes (so it works for binary data too), and `sb_printf` appends formatted text using the measure-then-write idiom from §8.3.

Every design decision here matters

Decision	Why
Store `len` explicitly	O(1) length, allows embedded NULs, no repeated scanning
Keep a trailing NUL anyway	`b.data` works with every C API for free
Doubling growth	Amortised O(1) append. Linear growth gives $O(n^2)$.
`realloc` into a temporary first	On failure, `b->data` is still valid — `b->data = realloc(b->data, ...)` leaks on `NULL`
Overflow guard before arithmetic	`len + need + 1` can wrap; a wrapped size allocates tiny and then overflows the heap
`va_copy` before measuring	The measuring pass consumes `ap` (Module 6)
`format(printf, 2, 3)` attribute	Compile-time format checking
`*b = (SB){0}` on free	Prevents use-after-free and double-free

Two of these deserve a word more. *Doubling* means that when the buffer fills up, its capacity is doubled rather than increased by a fixed amount. Each byte is therefore copied only a small, constant number of times on average, no matter how many appends happen — that is what "amortised O(1)" means, and the 10,000-append test shows it: capacity went to 65,536 in a handful of reallocations. And *resetting the structure on free* means that a later accidental `sb_free` or `sb_append` on the same builder sees an empty, valid builder, rather than a dangling pointer.

> This is the pattern behind every real C string library: SDS (Redis), bstring, `struct Buffer` in curl, and `strbuf` in git. Once you have it, `strcpy`, `strcat`, and `sprintf` never appear in your code again.

Module 8 Rules

#	Rule
1	Track length and capacity as separate numbers, always
2	Hoist `strlen` out of loop conditions
3	`strcmp` returns a sign, not −1/0/1
4	Never `memcmp` structs (padding) or secrets (timing)
5	Banned: `strcpy`, `strcat`, `sprintf`, `gets`, `atoi`
6	`strncpy` neither terminates reliably nor is efficient — it is not "safe `strcpy`"
7	An `snprintf` return ≥ the buffer size means truncation — check it

#	Rule
8	`strlcpy`/`strlcat` where available; ship your own otherwise
9	Overlapping regions → `memmove`. "It worked" is not "it's defined."
10	`strtol` needs all four checks: `errno = 0`, `end == s`, `*end`, range
11	UTF-8 in plain `char`; decode only when you need code points
12	Validate untrusted UTF-8: reject overlong forms, surrogates, and values > U+10FFFF
13	Build a length-carrying string type; guard every size computation against overflow

Standing build command, updated:

```
gcc -std=c17 -Wall -Wextra -Wpedantic -Wshadow -Wconversion \
    -Wformat=2 -Wstringop-overflow=4 -Wstringop-truncation -Wrestrict \
    -Werror=vla -Werror=return-type -Werror=implicit-function-declaration \
    -D_FORTIFY_SOURCE=3 -O1 -g -fsanitize=address,undefined prog.c -o prog
```

(`_FORTIFY_SOURCE` requires at least `-O1` to work, which is why optimisation now appears in the debug build. `-Wformat=2` adds stricter format-string checks, including warnings for non-literal format strings.)

Practice

1. **Break `strcpy` deliberately.** Overflow a 16-byte stack buffer with a 100-byte string. Run it plain, under ASan, and with `-D_FORTIFY_SOURCE=3 -O2`. Compare the three failure modes. Then find where the return address sits relative to the buffer, using `&buf` and a backtrace.

2. **`strncpy` audit.** Write a program proving both traps (no terminator, NUL padding). Then implement your own `strlcpy` and `strlcat` from scratch and test them against the edge cases: `dstsize == 0`, a source longer than the destination, an exact fit, and an empty source.

3. **Truncation detection.** Write `bool build_path(char *out, size_t n, const char *dir, const char *file)` using `snprintf`, returning `false` on truncation. Test it with an 8-byte buffer.

4. **The overlap experiment.** Copy an array onto itself shifted right, using `memcpy`, at sizes 8, 64, 512, 4096, and 65536. Record where (if anywhere) glibc's `memcpy` diverges from `memmove`. Then compile with `-Wrestrict` and note that the compiler is your real defence.

5. **Robust integer parsing.** Write `int parse_u16(const char *s, uint16_t *out)` that handles: an empty string, leading and trailing whitespace, `+`/`-` signs, overflow, trailing garbage, and `NULL` input. Write a test table with 20 cases, including `"65535"`, `"65536"`, `"-0"`, `" 12 "`, and `"0x10"`.

6. **UTF-8 toolkit.** Implement `u8_len`, `u8_next` (decode and advance), `u8_prev` (back up one code point), `u8_validate` (rejecting overlong forms, surrogates, and values above U+10FFFF), and `u8_truncate(char *s, size_t max_bytes)`, which never splits a sequence. Test with a string mixing ASCII, Latin-1, CJK, and a 4-byte emoji.

7. **Finish the string builder.** Extend the `SB` from §8.7 with `sb_insert`, `sb_erase`, `sb_find`, `sb_split` (returning an array of views), and `sb_shrink_to_fit`. Add `_Static_assert`s, and run a 100,000-operation fuzz workload under ASan + UBSan + LSan. Then benchmark it against naive `strcat` in a loop and measure the O(n) vs O(n²) difference.

8. **Constant-time comparison.** Implement `ct_equal` and show that it is timing-independent by measuring 10 million comparisons of strings that differ at position 0 versus position 31. Compare against `memcmp`.

Module 8 in Summary

A C string is a `char` array terminated by a NUL byte, with no stored length. That convention makes `strlen` O(n), forbids embedded NULs, and turns a missing terminator into an out-of-bounds read — so always track length and capacity separately. Compare with `strcmp` against zero, never `memcmp` structures or secrets, and avoid `strtok`.

The classic copying functions — `strcpy`, `strcat`, `sprintf`, and the now-removed `gets` — cannot know the destination's size and are banned. `strncpy` and `strncat` are traps rather than fixes; `snprintf` (with its truncation check) and `strlcpy`/`strlcat` are the safe tools. Overlapping copies require `memmove`, however often `memcpy` appears to work, and zeroing secrets requires a function the optimiser cannot remove.

Convert text to numbers with `strtol` and its relatives, checking `errno`, the end pointer, trailing characters, and range — never with `atoi`. Represent text as UTF-8 in ordinary `char` arrays, which works with the existing string functions by design; decode only when you need code points, and validate untrusted input. Finally, real C programs wrap all of this in a length-carrying string builder that grows by doubling and checks every size computation for overflow.

In Module 9, we reach the core skill of C: Pointers.

Module 9 — Pointers (the Core Skill)

In this module:

- Address-of, dereference, pointer declaration syntax
- Pointer arithmetic and its rules
- `void *` and generic programming
- `NULL`, null checks, `nullptr` (C23)
- Pointer to pointer, arrays of pointers, pointer to array
- `const` correctness: `const char *` vs `char * const` vs `const char * const`
- Function pointers, pointer to function returning pointer, `typedef` for readability
- Dangling pointers, wild pointers, uninitialized pointers
- Alignment, `uintptr_t`, pointer-integer conversion
- Reading complex declarations (the right-left rule / cdecl)

All outputs in this module are real GCC 13.3 output.

This is the core skill of C. Everything in Module 7 (array decay) and Module 8 (strings) rests on it, and everything after this module assumes it. Pointers are what give C its power — direct access to memory, efficient data structures, zero-copy interfaces, hardware access — and they are also the source of most of its dangers. The goal of this module is to make pointers completely unremarkable to you: to replace any lingering sense of mystery with a precise model of what a pointer is, what you may do with one, and what you may not.

9.1 Address-of, Dereference, Declaration Syntax

```c
int x = 42;
int *p = &x;   /* & takes an address */
*p = 99;       /* * accesses the pointed-to object */
```

```
x = 42      &x = 0x7ffde852f9d0
p = 0x7ffde852f9d0   *p = 42
&p = 0x7ffde852f9d8  <- the pointer itself has an address

after *p = 99: x = 99 (same object)
```

The unary `&` operator yields the address of an object. The unary `*` operator, applied to a pointer, yields the object at that address — not a copy, but the object itself, so you can read it or assign to it. After `*p = 99`, `x` is 99, because `*p` and `x` are two names for the same object.

A pointer is just a variable whose value happens to be an address. It has its own address (here, eight bytes above `x` on the stack), its own size, and its own lifetime — which need not match the lifetime of the object it points to. That last point is the source of the dangling-pointer bugs in §9.8.

The `*` binds to the variable, not the type

```
int *a, b;    /* a is int*, b is int !! */
int *c, *d;   /* both pointers */
```

```
int *a, b; -> a is int*, b is int (sizeof b = 4)
```

In a declaration, the `*` is part of each individual *declarator*, not part of the type shared by all of them. This is why the C convention is to write `int *p`, not `int* p`. Writing `int* a, b;` reads as though both `a` and `b` were pointers — and they aren't. The Linux kernel, K&R, and virtually every C style guide use `int *p`.

> Better still: **one declaration per line.** It sidesteps the issue entirely and makes each declaration easy to find and comment.

Sizes

```
char*=8 int*=8 double*=8 void*=8 struct*=8
```

On mainstream platforms all object pointers are the same size, but the standard does not guarantee this. Historically, on word-addressed machines, a `char *` needed extra bits to say which byte within a word it referred to, and was therefore larger than an `int *`. Function pointers may also differ in size from object pointers, and they do on some Harvard-architecture microcontrollers, where code and data live in separate address spaces. Never assume; use `sizeof`.

`*` and `&` cancel

`*&x` is simply `x`: take the address, then follow it back. `&*p` is `p`. The standard makes the second case explicit: when `&` is applied to the result of `*`, neither operator is actually evaluated, and the result is just `p` — so `&*p` is well defined even when `p` is a null pointer. (The same rule makes `&a[i]` equivalent to `a + i`, without ever touching `a[i]`.)

9.2 Pointer Arithmetic and Its Rules

Arithmetic is scaled by the pointed-to type

```
int*    p+1 advances 4 bytes
double* q+1 advances 8 bytes
char*   advances 1 byte   <- that is why char* is used for byte math
```

`p + n` means "advance by *n objects*," not by *n* bytes. The compiler multiplies by `sizeof *p` for you. This is why `a[i] == *(a + i)` works for any element type: the arithmetic automatically steps over whole elements.

Pointer difference yields `ptrdiff_t`

```
&a[4] - &a[0] = 4 elements (not 16 bytes)
```

Subtracting two pointers gives the number of *elements* between them, not bytes, as a signed value of type `ptrdiff_t`. Print it with `%td`.

Both pointers must point into the same array object (or one past its end). Subtracting pointers to unrelated objects is undefined behaviour.

The one-past-the-end rule

```
a+5 = 0x7ffc8fe3e074 (legal address, dereferencing is UB)
loop idiom: for (int *it=a; it!=end; ++it) -> 10 20 30 40 50
a+6 or a-1 -> UNDEFINED even without dereferencing
```

For an array of five elements, you may *form* the pointer `a + 5`, which points one past the last element, and you may compare it with other pointers into the array. You may not dereference it. Anything further — `a + 6`, or `a - 1` — is undefined behaviour merely to *compute*, even if you never dereference it.

This is not pedantry. On segmented architectures, merely loading an out-of-range address into an address register could trap. And on modern compilers, the rule licenses optimisations: the compiler may assume that `p + n` stays within the array, and reason about loop bounds on that basis, producing code that breaks if you violate the assumption.

The rule is exactly what makes the standard iterator idiom legal:

```
int *end = arr + n;
for (int *it = arr; it != end; ++it) { ... }
```

`end` is one past the last element — a valid pointer to form and compare against, never dereferenced.

Comparison rules

Operation	Defined?
`p == q`, `p != q`	Always
`p < q`, `p > q`	Only within one array object (or one past its end)
`p - q`	Only within one array object
`p + n`	Only if the result stays within `[array, array + n]`

Comparing pointers to two unrelated objects with `<` or `>` is undefined behaviour in C. It usually “works” in practice, because mainstream machines have a flat address space and the compiler simply compares the numbers, but nothing requires a consistent answer. (If you genuinely need an ordering of unrelated pointers — say, to lock two mutexes in a consistent order — convert both to `uintptr_t` and compare the integers.)

Also UB

- **`NULL + 0` and `NULL - NULL`** are technically undefined in C. (C++ defines them, and a change adopted for the next C standard does too, but C23 and earlier do not.)
- **Arithmetic on a `void *`** is not standard C. GNU C treats `void` as having size 1, but `-Wpointer-arith` (or `-pedantic`) flags it:

  ```
  error: pointer of type 'void *' used in arithmetic [-Wpointer-arith]
  ```

- **Arithmetic on function pointers.**
- **Arithmetic on a pointer to an incomplete type** — the compiler does not know the size to scale by.

For byte-level arithmetic, cast to `unsigned char *`. It is the one type guaranteed to have size 1 and to be exempt from strict aliasing (Module 2).

9.3 `void *` and Generic Programming

`void *` is the generic object pointer. In C — unlike C++ — it converts implicitly to and from any other object pointer type:

```c
void *v = arr;
int *ip = v;            /* legal in C, no cast needed */
void *p = malloc(n);    /* don't cast malloc */
```

What you can't do

```c
*v;             /* ERROR: cannot dereference */
v + 1;          /* not standard C */
sizeof(void);   /* not standard C (GNU says 1) */
```

A `void *` carries an address and nothing else: the compiler does not know the size or type of what is there, so it cannot fetch it or step over it.

And critically: **`void *` does not cover function pointers.** ISO C makes no guarantee that a function pointer survives a round trip through `void *`. POSIX requires it to (which is what makes `dlsym` work), but that is a documented extension, not portable C.

The generic-programming pattern

C has no templates or generics, so generic code takes a `void *`, an element size, and, where needed, a comparison function:

```c
static void swap(void *a, void *b, size_t n)
{
    unsigned char *x = a, *y = b, t;
    while (n--) { t = *x; *x++ = *y; *y++ = t; }
}

static void *find(const void *base, size_t n, size_t sz, const void *key,
```

```
                  int (*cmp)(const void *, const void *))
{
    const unsigned char *p = base;
    for (size_t i = 0; i < n; i++, p += sz)
        if (cmp(p, key) == 0) return (void *)p;
    return NULL;
}
```

```
swap ints:    a=2 b=1
swap doubles: x=2.5 y=1.5
swap arrays:  xyz abc
find(9) -> index 2
```

`swap` exchanges any two objects of the same size, byte by byte. `find` walks an array of elements it knows only by their size, advancing an `unsigned char *` by `sz` bytes each time and delegating the comparison to the caller's function, which knows the real type.

This is exactly how `qsort` and `bsearch` work. The cost is type safety: pass the wrong `sz`, or a comparator for the wrong type, and you silently corrupt memory or get wrong answers, with no warning. Module 28 covers safer alternatives (macro-generated containers and `_Generic`).

9.4 `NULL`, Null Checks, `nullptr`

```
NULL prints as (nil), comparisons: (p==NULL)=1 (!p)=1
```

`NULL` is a *null pointer constant* — typically `((void*)0)` or plain `0`. A null pointer is guaranteed to compare unequal to a pointer to any real object or function, which is what makes it useful as "points to nothing."

Subtleties worth knowing

- **A null pointer is not guaranteed to be all-zero bits.** In practice it is address 0 on every platform you are likely to meet, but the standard only guarantees that a *constant expression* with the value 0, converted to a pointer type, produces a null pointer. The consequence: `memset(&ptr, 0, sizeof ptr)` and `calloc` are not formally guaranteed to produce null pointers. (They do on every real platform.)
- **Dereferencing `NULL` is undefined behaviour, not "a segfault."** The compiler may reason that a pointer you have already dereferenced must be non-null, and delete a null check that comes afterwards. This produced a real Linux kernel vulnerability, CVE-2009-1897: a function dereferenced a pointer, then checked it for `NULL`, and GCC removed the check.
- **Never pass a bare `NULL` as a variadic argument.** On platforms where `NULL` is a plain `0`, `execl(path, "a", NULL)` passes a 4-byte `int` where the function reads an 8-byte pointer (Module 6). Write `(char *)NULL`.

C23 `nullptr`

```
C23 nullptr available, sizeof(nullptr_t)=8, q==NULL: 1
```

C23 adds a real `nullptr` keyword with its own type, `nullptr_t`. Its benefits: it is unambiguously a pointer, which fixes the variadic problem, and it works cleanly with `_Generic` (Module 12), which can distinguish it from an integer.

Null-check discipline

```c
/* Check what can fail */
FILE *fp = fopen(path, "r");
if (!fp) return -1;

char *buf = malloc(n);
if (!buf) return -1;

/* Document what can't be NULL — and consider enforcing it */
void process(const char *s)
{
    assert(s != NULL);              /* debug builds */
    /* or: */ if (!s) return -1;    /* defensive, for public APIs */
}

/* C99: tell the compiler and the reader */
void process(const char s[static 1]);   /* "s is never NULL" */
```

The principle: check every result that can legitimately be null (anything that allocates, opens, or looks something up), and *document* every parameter that must not be. Whether to also check such parameters at run time is a design decision: an `assert` catches programming errors during development, while a public library API often returns an error instead, because it cannot trust its callers. `[static 1]` records the contract in the signature itself.

Style note: `if (!p)` and `if (p == NULL)` are both idiomatic; pick one per codebase. Never write `if (p == 0)` — it is legal, but it obscures the fact that `p` is a pointer.

9.5 Pointer to Pointer, Arrays of Pointers, Pointer to Array

The three shapes people confuse

```c
int *b[5];     /* ARRAY of 5 pointers to int — 40 bytes */
int (*c)[5];   /* POINTER to an array of 5 ints — 8 bytes */
int **d;       /* POINTER to a pointer to int — 8 bytes */
```

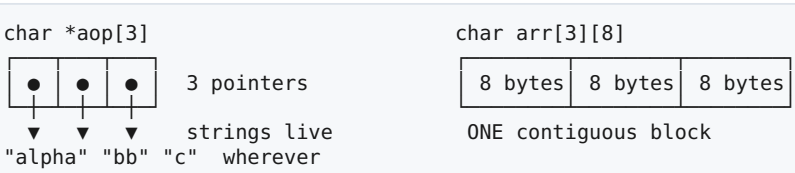
```
sizeof: int a[5]=20 int *b[5]=40 int(*c)[5]=8
```

`[]` binds more tightly than `*`, so `int *b[5]` is “an array of 5 pointers to `int`.” Parentheses are required to say “pointer to an array”: `int (*c)[5]`. And `int **d` is a pointer to a single pointer — which may, in practice, be the first element of an array of pointers.

Array of pointers vs 2D array — the memory difference

```
char *aop[3]   sizeof=24 (3 pointers)
char arr[3][8] sizeof=24 (24 contiguous bytes)
aop[0]=0x...80c0 aop[1]=0x...8100 -> scattered
arr[0]=0x...0080 arr[1]=0x...0088 -> exactly 8 apart
```

The same total size here, but a completely different structure:



Choose an array of pointers when the rows have different lengths (ragged data, such as a list of words), when you need to reorder rows cheaply (swap pointers, not data), or when the elements come from different allocations.

Choose a 2D array when you want one allocation, contiguous memory, good cache locality, and a fixed row width.

`argv` is the canonical array of pointers

```
argv    = 0x7fffa22da698 (char**)
argv[0] = 0x7fffa22dbd24 -> "./pp"
argv[1] = 0x7fffa22dbd29 -> "one"
argv[2] = 0x7fffa22dbd2d -> "two"
argv[3] = NULL
```

Note that the string addresses are only 5 and 4 bytes apart: the operating system packed the strings themselves contiguously, each followed by its NUL. `argv` itself is a separate array of pointers into that block, terminated by a null pointer (Module 1).

Pointer to pointer — modifying the caller's pointer

```c
char *buf = malloc(8);
char **pbuf = &buf;
*pbuf = realloc(buf, 64);   /* the caller's `buf` is updated */
```

```
before: buf=0x502000000010
after realloc via **: buf=0x506000000020 "hello"
```

Writing through `pbuf` changed `buf` itself: after the `realloc`, `buf` holds the new address. Any function that must change *where the caller's pointer points* takes a `T **` (Module 6):

```c
int grow(char **buf, size_t *cap);
void free_and_null(void **p) { free(*p); *p = NULL; }
```

(Calling `free_and_null` on anything other than a genuine `void *` variable requires a cast, and strictly speaking accessing, say, a `char *` through a `void **` is an aliasing violation. Many codebases therefore implement this as a macro instead.)

The two ways to allocate a 2D array

```c
/* A: flat — 1 malloc, contiguous, best cache behavior */
int *flat = malloc(R * C * sizeof *flat);
flat[i*C + j] = 7;
free(flat);

/* B: array of pointers — R+1 mallocs, ragged-capable, 2 dereferences */
int **rows = malloc(R * sizeof *rows);
```

```
for (size_t i = 0; i < R; i++) rows[i] = malloc(C * sizeof **rows);
rows[i][j] = 7;

for (size_t i = 0; i < R; i++) free(rows[i]);
free(rows);
```

```
A flat:   1 malloc, contiguous, flat[i*C+j]=7
B ragged: 4 mallocs, rows[i][j]=7, two derefs, needs 4 frees
```

Prefer A. It is faster, has a single failure path and a single `free`, and cannot leak partially. (In version B, if the third row allocation fails, you must free the first two rows and the pointer array — code that is easy to get wrong.) Use B only when rows genuinely differ in length.

There is also a hybrid worth knowing: allocate one flat block for the data, plus an array of row pointers that point into it. You get `m[i][j]` syntax with only two allocations and contiguous data.

9.6 `const` Correctness

Module 2 introduced `const` with pointers; here is the complete picture.

```
int *p1             = &x;   /* write OK, repoint OK */
const int *p2       = &x;   /* write NO, repoint OK */
int const *p2b      = &x;   /* IDENTICAL to p2 */
int *const p3       = &x;   /* write OK, repoint NO */
const int *const p4 = &x;   /* write NO, repoint NO */
```

```
p1: write OK, repoint OK
p2: write NO, repoint OK
p3: write OK, repoint NO
p4: write NO, repoint NO
```

“Write” means modifying the pointed-to value through the pointer (`*p = ...`); “repoint” means changing the pointer itself (`p = ...`).

The reading rule

> `const` applies to whatever is immediately to its **left**. If there is nothing to its left, it applies to what is on its **right**.

```
const int *p   → nothing left of const → applies right → const int.  Pointer to const int.
int const *p   → int is left of const → const int.  Same thing.
int * const p  → * is left of const → const pointer.  Const pointer to int.
```

Combined with reading declarations right to left from the identifier (Module 2):

- `const int *p` → “p is a pointer to an int that is const”
- `int *const p` → “p is a const pointer to an int”

The `int const *` spelling makes the rule uniform — `const` always modifies what is to its left — which is why some codebases mandate it. `const int *` is more common. Pick one and be consistent.

Conversions

```
char* -> const char*  : implicit, safe (adding const)
const char* -> char*  : requires a cast, and writing through it may be UB
```

Adding `const` is always safe and implicit: you are only promising to do less. Removing it requires a cast — and if the underlying object really is `const`, writing through the result is undefined behaviour. (Module 2 showed the compiler folding a `const` variable into a literal and giving two different answers for the same memory.)

Catch casts that discard `const` with:

```
warning: cast discards 'const' qualifier from pointer target type [-Wcast-qual]
```

Why `char **` → `const char **` is rejected

```c
void f(const char **p);
char *q;
f(&q);
```

```
warning: passing argument 1 of 'f' from incompatible pointer type
```

This surprises almost everyone, since `char *` → `const char *` is fine. But the rejection is correct. If it were allowed:

```c
const char *secret = "const data";
char *q;
f(&q);          /* if this were legal... */
/* inside f: *p = secret;   ← stores a const pointer into q */
q[0] = 'X';     /* ...you'd now be writing to const memory, legally-looking */
```

Inside `f`, `*p` has type `const char *`, so assigning `secret` to it is perfectly legal. But `*p` *is* `q`. So a pointer to read-only data has been smuggled into a plain `char *`, with no cast anywhere, and the next line writes through it.

The conversion `T *` → `const T *` is safe only at one level of indirection. To add `const` at a deeper level, every level above it must be `const` too: `char **` converts safely to `const char *const *`, because then `f` cannot store anything through the outer pointer.

Where `const` correctness earns its keep

```c
size_t strlen(const char *s);                          /* won't modify */
void *memcpy(void *dst, const void *src, size_t n);
int process(const Config *cfg, Output *out);           /* input vs output is obvious */
```

The benefits: self-documenting APIs, contracts enforced by the compiler, the ability for callers to pass genuinely `const` data (including string literals), and some optimisation opportunities. Apply `const` to every pointer parameter whose target you do not modify. It costs nothing, and it is what reviewers expect.

9.7 Function Pointers and Complex Declarations

This section extends Module 6, §6.6.

```c
int f(int);          /* function */
int *g(int);         /* function returning int* */
int (*d)(int);       /* POINTER to function */
int (*e[3])(int);    /* ARRAY of 3 function ptrs */
int (*h(int))(int);  /* function returning a function pointer */
```

```
sizeof: int(*d)(int)=8 int(*e[3])(int)=24

h(0)(41) = 42   <- h returns a function pointer, then we call it
e[1](10) = 11
```

The last declaration is the hardest to read. `h(int)` is a function taking an `int`; `(*h(int))` says it returns a pointer; and the surrounding `int (…)(int)` says that pointer is to a function taking an `int` and returning an `int`. So `h(0)` returns a function pointer, and `h(0)(41)` immediately calls it.

That shape is exactly the real prototype of the standard `signal()` function, which is famous for being unreadable. Always `typedef`:

```c
typedef void (*sighandler_t)(int);
sighandler_t signal(int sig, sighandler_t handler);   /* now readable */
```

With the `typedef`, the declaration says plainly that `signal` takes a signal number and a handler and returns a handler (the previous one).

Reading complex declarations — the right-left rule

> Start at the identifier. Go **right** as far as you can, then **left**. `()` and `[]` bind more tightly than `*`. Parentheses group.

Each time you hit a closing parenthesis while going right, or an opening one while going left, you switch direction and move outward to the next level. A worked example:

```c
char *(*(**foo[][8])())[]
```

```
foo           foo is
foo[]         an array of
foo[][8]      arrays of 8
**            pointers to pointers to
()            functions returning
(*    )[]     pointers to arrays of
char *        pointers to char
```

In words: `foo` is an array of arrays of 8 pointers to pointers to functions returning pointers to arrays of pointers to `char`.

The “spiral rule” you will see online is a memorable approximation, but it gives the wrong answer for some declarations. The right-left rule is the correct one. When in doubt, use `cdecl` — a program (and website) that translates C declarations into English and back — or, better, just add a `typedef`.

Practical advice

```c
/* Instead of this: */
void (*(*signal_table[16])(int, void (*)(int)))(int);

/* Write this: */
typedef void (*Handler)(int);
typedef Handler (*Installer)(int, Handler);
Installer signal_table[16];
```

Both declare the same thing: an array of 16 pointers to functions that, like `signal`, take a number and a handler and return a handler. Only one of them can be read at a glance.

> If you need the right-left rule to read your own code, you should have used a `typedef`.

9.8 Dangling, Wild, and Uninitialized Pointers

A pointer is only as valid as the object it points to — and nothing in C keeps the two in sync. The complete taxonomy of invalid pointers, each verified under ASan:

Bug	Cause	ASan diagnosis
Wild	Never initialized — holds stack garbage	often SEGV / random corruption
Dangling (stack)	Points to a dead local	stack-use-after-return / SEGV
Dangling (heap)	Used after `free`	heap-use-after-free
Double free	Freed twice	attempting double-free
Dangling after `realloc`	Old pointer kept after `realloc` moved the block	heap-use-after-free
NULL dereference	Dereferencing `NULL`	SEGV on address 0x0

```
=== dangling stack ===
ERROR: AddressSanitizer: SEGV on unknown address 0x000000000000

=== use after free ===
ERROR: AddressSanitizer: heap-use-after-free on address 0x502000000010

=== double free ===
ERROR: AddressSanitizer: attempting double-free on 0x502000000010

=== dangling after realloc ===
ERROR: AddressSanitizer: heap-use-after-free on address 0x502000000010
```

(The dangling-stack case shows up as a null-address SEGV for the reason seen in Module 3: GCC recognised the returned local address as undefined behaviour and returned `NULL` instead.)

Without a sanitizer, several of these bugs "work" much of the time — which is precisely what makes them dangerous. A use-after-free reads whatever now occupies that memory; if nothing has reused it yet, it reads the old, correct-looking value.

The `realloc` alias trap

```
char *a = malloc(8);
char *alias = a;          /* second pointer to the same block */
a = realloc(a, 1<<20);    /* may MOVE the block */
printf("%s", alias);      /* alias now dangles */
```

`realloc` may return a different address — when the block cannot be grown in place, it allocates a new one, copies the contents, and frees the old one. Every other pointer into that block then becomes invalid: pointers stored in structures, in other data structures, and pointers returned to callers earlier. This is why growable-array APIs typically hand out *indices* rather than pointers to their elements: an index survives a reallocation; a pointer does not.

Also: `p = realloc(p, n)` leaks the original block if `realloc` fails and returns `NULL`, because you have just overwritten your only pointer to it. Always:

```
char *tmp = realloc(p, n);
if (!tmp) { /* p is still valid — clean up properly */ return -1; }
p = tmp;
```

Defensive habits

```
int *p = NULL;        /* 1. Always initialize */

free(p); p = NULL;    /* 2. Null after free — makes double-free a no-op
                            (free(NULL) is a guaranteed no-op) */

/* 3. Have exactly one owner. Document it. */
/* 4. Never return a pointer to a local. -Wreturn-local-addr catches it. */
/* 5. Invalidate all aliases after realloc. */
/* 6. Run ASan on every debug build and in CI. */
```

Initializing to `NULL` converts a wild pointer — unpredictable garbage — into a predictable null pointer that crashes immediately and obviously when misused. Setting a pointer to `NULL` after freeing it does the same for later accidental uses through that variable.

> ⚠ `p = NULL` after `free` only protects *that* pointer. Every alias still dangles. There is no substitute for clear ownership rules — which are the subject of Module 10.

9.9 Alignment, `uintptr_t`, Pointer↔Integer Conversion

```
ALIGNMENT REQUIREMENTS:
  char=1 short=2 int=4 double=8 max_align_t=16
```

An object of type `T` must live at an address that is a multiple of `alignof(T)` (Module 4). Accessing an object through a misaligned pointer is undefined behaviour.

`uintptr_t` — the only portable pointer↔integer type

```
p=0x7ffe51fb57b4 -> uintptr_t=140730273847220 -> back=0x7ffe51fb57b4 same? 1
alignment check: (u % alignof(int)) == 0
round up to 64: 140730273847232
```

`uintptr_t` and `intptr_t`, from `<stdint.h>`, are guaranteed to hold any `void *` and convert back to an equal pointer. Never use `long` for this — it is 32 bits on 64-bit Windows (Module 2).

Legitimate uses:

```c
uintptr_t u = (uintptr_t)p;
if (u % 16 == 0) { /* aligned */ }
void *aligned = (void*)((u + 63) & ~(uintptr_t)63);     /* round up */
size_t bucket = ((u >> 4) * 2654435761u) % nbuckets;    /* pointer hashing */
uintptr_t tagged = u | 1;   /* tagged pointers (low bits are free when alignment > 1) */
```

These are alignment checks, rounding an address up to a boundary (the same formula as Module 4, applied to an address), hashing pointers into buckets — shifting away the low bits first, since they are always zero for aligned objects — and *tagged pointers*, which store a few bits of extra information in the low bits that alignment guarantees are zero. The tag must be masked off again before the pointer is used.

Misaligned access

```c
unsigned char buf[16];
uint32_t *p = (uint32_t*)(buf + 1);   /* misaligned */
*p = 0x11223344;
```

```
runtime error: store to misaligned address 0x56248fefa101 for type 'uint32_t',
               which requires 4 byte alignment
```

This is doubly undefined: it is both a misaligned access and a strict-aliasing violation (Module 2). On x86 it usually “works” but is slower (and faults for some SSE instructions). On some ARM and SPARC configurations it faults immediately, or silently returns rotated garbage.

The fix is always `memcpy`:

```c
uint32_t v;
memcpy(&v, buf + 1, sizeof v);   /* correct at any alignment, any aliasing */
```

```
memcpy round-trip: 0xDEADBEEF   <- always correct, any alignment
```

And it costs nothing: as Module 2 showed, a fixed-size `memcpy` compiles to a single move instruction on platforms that support unaligned loads, and to the correct byte-by-byte sequence on those that don't.

Aligned allocation

```
malloc -> 0x55e302aac2b0 (aligned to max_align_t = 16? yes)
aligned_alloc(64,128) -> 0x55e302aac300 (% 64 = 0)
```

`malloc` is guaranteed to return memory suitably aligned for any standard type — that is, to `alignof(max_align_t)`. For stronger alignment — SIMD buffers, cache-line alignment, DMA — use:

- **`aligned_alloc(alignment, size)`** — C11. In C11 the size had to be a multiple of the alignment; C17 relaxed this, but keeping it a multiple is the portable choice. Free with ordinary `free`.
- **`posix_memalign(&p, align, size)`** — POSIX.
- **`_aligned_malloc` / `_aligned_free`** — Windows. Memory from `_aligned_malloc` must be released with the matching `_aligned_free`, never plain `free`.

Module 9 Rules

#	Rule
1	Write `int *p`, one declaration per line
2	Pointer arithmetic is scaled by the element type; differences yield `ptrdiff_t` (`%td`)
3	One-past-the-end is legal to form; two past, and `a-1`, are UB even unread
4	`</>` between unrelated objects is UB; `==`/`!=` is always fine
5	Cast to `unsigned char *` for byte arithmetic — never `void *`
6	`void *` doesn't cover function pointers in ISO C
7	`(char*)NULL` in variadic calls; C23 gives you `nullptr`
8	`int *b[5]` vs `int (*c)[5]` vs `int **d` — three different things
9	Prefer a single flat allocation over an array of row pointers
10	`const` binds to its left; add `const` to every parameter you don't modify

#	Rule
11	`T **` → `const T **` is correctly rejected — it's not a compiler bug
12	`typedef` every function-pointer type; if you need the right-left rule, use a `typedef`
13	`realloc` invalidates every alias; never write `p = realloc(p, n)`
14	Initialize to `NULL`, null after `free`, one owner per allocation
15	Type-pun and read unaligned data with `memcpy`, never a pointer cast
16	`uintptr_t` for pointer↔integer conversion, never `long`

Standing build command:

```
gcc -std=c17 -Wall -Wextra -Wpedantic -Wshadow -Wconversion \
    -Wcast-qual -Wcast-align=strict -Wpointer-arith -Wwrite-strings \
    -Wnull-dereference -Wstrict-aliasing=2 \
    -Werror=vla -Werror=return-type -Werror=implicit-function-declaration \
    -O1 -g -D_FORTIFY_SOURCE=3 -fsanitize=address,undefined prog.c -o prog
```

`-Wwrite-strings` gives string literals the type `const char[]`, so that `char *s = "literal";` produces a warning — a genuinely useful addition that catches the writable-literal mistake from Module 7 at compile time. `-Wcast-align=strict` warns about casts that increase the required alignment, even on x86, where misaligned access happens to work.

Practice

1. **Arithmetic forensics.** For `int a[5]`, print the byte offsets of `a+i`, `&a[i]`, `(char*)a+i`, and `&a+1`. Then compute `&a[5] - &a[0]` and `(char*)&a[5] - (char*)&a[0]`, and explain both numbers.

2. **One past the end.** Write the iterator loop with `!=` against `arr+n`. Then deliberately compute `arr+n+1` and `arr-1`. Compile with `-fsanitize=address,undefined` and with `-O2`, and see whether anything complains. (Spoiler: mostly not — which is the lesson.)

3. **Generic library.** Implement `void *gmap(const void *src, size_t n, size_t sz, void *dst, void (*fn)(const void*, void*))` and use it to double an `int` array, uppercase a `char` array, and negate a `double` array. Then write `gsort`, wrapping `qsort`.

4. **`argv` map.** Print `argv`, each `argv[i]`, and each string's bytes, plus the gaps between consecutive strings. Then do the same for `envp`. Draw the memory layout.

5. **`const` matrix.** Write a table of all four `const` placements. For each, write the two lines that should fail to compile, and verify each error message.

6. **The `const char **` puzzle.** Reproduce the rejected conversion. Then write the version with `const char *const *` that is accepted, and explain why it is safe.

7. **Declaration gauntlet.** Without a compiler, describe each of these in English, then verify with a test program:

```
int *a[10];        int (*b)[10];        int (*c)(int);
int *(*d)(int);    int (*e[5])(void);   int (*(*f)(int))[10];
char *(*g[3])(const char *, int);
void (*h(int, void (*)(int)))(int);
```

8. **Bug taxonomy lab.** Write one program with six modes that reproduce a wild pointer, a dangling stack pointer, a use-after-free, a double free, a `realloc` alias, and a NULL dereference. Run each under ASan, under Valgrind, and plain. Record which tool catches what, and which cases “work” without a tool.

9. **Alignment tool.** Write `bool is_aligned(const void *p, size_t a)`, `void *align_up(void *p, size_t a)`, and a `read_u32_unaligned(const void *p)` using `memcpy`. Verify with UBSan that the naive cast version is flagged and yours is not. Then `objdump -d` both and compare the generated instructions.

10. **Tagged pointers.** Since `malloc` returns 16-byte-aligned memory, the low 4 bits of every pointer it returns are zero. Build a tagged-pointer type that stores a 4-bit tag in those bits, with `tag_get`, `tag_set`, and `ptr_get`. Verify with ASan that the untagged pointer is always usable.

Module 9 in Summary

A pointer is a variable holding an address; `&` produces one, `*` follows one, and in declarations the `*` belongs to each variable, not to the type. Pointer arithmetic counts in elements, not bytes; differences yield `ptrdiff_t`; and only addresses within an array or one past its end may even be formed. `void *` is the generic object pointer — the basis of C’s `qsort`-style generic code — but it cannot be dereferenced or stepped, and it does not cover function pointers. Null pointers must be checked wherever a result can fail, and dereferencing one is undefined behaviour that the optimiser may exploit.

Arrays of pointers, pointers to arrays, and pointers to pointers are three different shapes with different layouts; a single flat allocation is usually the best way to store a matrix. `const` binds to its left, is safe to add, dangerous to remove, and cannot be added two levels deep without adding it at every level. Complex declarations yield to the right-left rule, but a `typedef` is always better.

Finally, a pointer does not keep its object alive: wild, dangling, double-freed, `realloc`-invalidated, and null pointers are all undefined behaviour, so initialize pointers, null them after freeing, keep one owner per allocation, and run the sanitizers. For alignment and pointer-integer conversion, use `uintptr_t` and `memcpy`, never `long` or a misaligned cast.

In Module 10, we build on this with Memory Management — the memory layout of a process, the heap allocator, ownership discipline, and custom allocators.

Module 10 — Memory Management

In this module:

- Memory layout: text, rodata, data, BSS, heap, stack
- Stack frames, stack overflow, recursion limits
- `malloc`, `calloc`, `realloc`, `free`, `aligned_alloc`
- Checking allocation failure; the `realloc` failure pattern
- Leaks, double free, use-after-free, buffer overflow, heap corruption
- Ownership conventions and who-frees-what discipline
- Custom allocators: arena/bump, pool, freelist, slab, stack allocator
- `alloca` (and why to avoid it)
- Zeroing sensitive memory (`memset_s`, `explicit_bzero`)

All output in this module is real, from GCC 13.3 and glibc 2.39.

This is where C programs live or die in production. Languages with garbage collection decide for you when memory is released; C leaves every such decision to you. That gives you precise control over performance and resource use — and it means that every allocation is a small contract you must honour: check it, use it only while it is alive, free it exactly once. The previous modules introduced most of the individual pieces. This one assembles them into a complete working model of memory, and a set of disciplines that keep real programs correct.

10.1 Process Memory Layout

Printing the address of one object from each region of a running program gives:

```
REGION  ADDRESS          SYMBOL
text    0x56239ca9c1f4   main()
text    0x56239ca9c1e9   func()
rodata  0x56239ca9d008   rodata_str
rodata  0x56239ca9d087   string literal
data    0x56239ca9f010   data_var (=42)
bss     0x56239ca9f01c   bss_var
heap    0x5623db22a2a0   malloc(16) #1
heap    0x5623db22a2c0   malloc(16) #2 (+32)
mmap    0x7fbb934ff010   malloc(1MB) <- large allocs use mmap, far away
stack   0x7fffec3fb21c   local variable
```

And here is the kernel's own view of the same kind of process, from `/proc/self/maps`, which lists every mapped region with its permissions:

```
$ cat /proc/self/maps
5579a3c81000-5579a3c82000 r--p  /home/claude/c_mem/maps  ← rodata (read-only)
5579a3c82000-5579a3c83000 r-xp  /home/claude/c_mem/maps  ← text (execute, NOT writable)
5579a3c85000-5579a3c86000 rw-p  /home/claude/c_mem/maps  ← data + bss (read-write)
5579c1169000-5579c118a000 rw-p  [heap]
7f912ac00000-...          r--p  /usr/lib/.../libc.so.6
```

Put together, the layout of a typical Linux process looks like this:

```
HIGH ADDRESSES
   ┌──────────────────┐
   │ env / argv       │
   │ stack            │  grows DOWN ↓
   ├──────────────────┤
   │       ↓          │
   │   (unmapped)     │  ← guard pages here
   │       ↑          │
   ├──────────────────┤
   │ mmap region      │  shared libs, large malloc()
   ├──────────────────┤
   │ heap             │  grows UP ↑ (brk / sbrk)
   ├──────────────────┤
   │ .bss    rw-      │  zero-init globals — no file space
   │ .data   rw-      │  initialized globals
   │ .rodata r--      │  string literals, const data
   │ .text   r-x      │  code
   └──────────────────┘
LOW ADDRESSES (page 0 unmapped → NULL deref = SIGSEGV)
```

Each region has a distinct role. The sections from the executable file — `.text`, `.rodata`, `.data`, and `.bss` (Module 1) — are loaded at the bottom. The heap starts just above them and grows upward as the program allocates. Shared libraries and large allocations are mapped into a separate region higher up. The stack starts near the top of the user address space and grows downward, with unmapped *guard pages* below it so that running off its end causes a fault rather than silent corruption. And the very lowest page, containing address 0, is deliberately left unmapped, which is why dereferencing a null pointer usually crashes immediately.

The page permissions are enforced by hardware

```c
char *s = "literal";
s[0] = 'X';
```

```
about to write to .rodata...
Segmentation fault exit=139
```

`.rodata` is mapped read-only, and the processor's memory-management unit (MMU) enforces that on every access. This is the mechanism behind Module 7's distinction: `char *s = "hi"; s[0]='H';` crashes, but `char s[] = "hi"; s[0]='H';` doesn't, because the second form copies the literal into a writable array on the stack.

Similarly, `.text` is mapped `r-x` — readable and executable, but not writable — which is why self-modifying code or a JIT compiler has to request different permissions explicitly with `mprotect`. And the heap and stack are `rw-` but not `x`: data there cannot be executed. This *NX* ("no execute") bit is why the classic attack of injecting machine code onto the stack and jumping to it stopped working on mainstream systems in the mid-2000s, pushing attackers towards subtler techniques.

`malloc` uses two different mechanisms

Look again at the addresses: `malloc(16)` returned `0x5623db22a2a0`, inside `[heap]`, but `malloc(1MB)` returned `0x7fbb934ff010` — a completely different region.

glibc's allocator obtains memory from the operating system in two ways. For ordinary requests it carves blocks out of the heap, which it extends with the `brk` system call. Above a threshold — `M_MMAP_THRESHOLD`, 128 KB by default — it asks the kernel for a fresh, separate mapping with `mmap`. The consequences:

- **Large blocks are returned to the OS immediately on `free`;** small ones usually are not, but are kept for reuse by later allocations. (This is why a process's memory use often does not shrink after it frees many small objects.)
- **`mmap`ed pages arrive already zeroed by the kernel,** so `calloc` of a large block costs no extra work for zeroing.
- **You can tune the behaviour** with `mallopt(M_MMAP_THRESHOLD, ...)` — if you have a measured reason.

10.2 Stack Frames and Stack Overflow

This section builds on Module 6, §6.7.

The stack grows downward. Each function call pushes a frame containing the return address, the saved frame pointer, any callee-saved registers the function uses, and its local variables. From Module 6's measurement:

```
stack limit = 8192 KB
  depth 50000,  stack used 1562 KB (~32 bytes/frame)
  depth 250000, stack used 7812 KB
Segmentation fault
```

The key facts:

Fact	Implication
The default limit is 8 MB on Linux (`ulimit -s`), 1 MB on Windows, and often much less for secondary threads	You have far less stack than you think in threads
Overflow gives a bare `SIGSEGV`, with no message	Use ASan (which reports stack-overflow) or rely on a guard page
Stack memory is not zeroed	Uninitialized locals hold previous frames' data (Module 3's `0xDEADBEEF`)
Locals are adjacent to the return address	This is why stack buffer overflows are exploitable

(Thread stack sizes vary: macOS gives secondary threads 512 KB by default, while glibc typically uses the `ulimit -s` value; embedded RTOS tasks may have only a few kilobytes. Check your platform, and set the size explicitly when creating threads that need a lot.)

Never put large objects on the stack:

```c
void f(void) { char buf[1024*1024]; }                          /* 1 MB — blows a thread stack
instantly */
void f(void) { char *buf = malloc(1024*1024); ... free(buf); }     /* checkable */
```

And, as Module 7 explained: no VLAs. `int v[n]` with an attacker-controlled `n` is a stack-clash vulnerability.

Mitigations in modern builds:

- **`-fstack-protector-strong`** — places a random *canary* value between the locals and the return address and checks it before returning; a corrupted canary calls `__stack_chk_fail`, which aborts the program.
- **Guard pages** below the stack, so that running off its end faults.
- **ASLR**, which randomises the stack's base address on every run.
- **`-fstack-clash-protection`** — makes the compiler touch ("probe") each page of a large frame in turn, so that a huge frame cannot jump straight over the guard page.

10.3 The `malloc` Family

```
void *malloc(size_t size);                      /* uninitialized */
void *calloc(size_t n, size_t size);            /* zeroed, checks n*size overflow */
void *realloc(void *p, size_t size);            /* resize, may move */
void free(void *p);
void *aligned_alloc(size_t align, size_t size);   /* C11 */
```

`malloc` vs `calloc`

```
malloc: contents INDETERMINATE
  bytes: 00 00 00 00 00 00 00 00 ...   ← happened to be zero. Don't rely on it.

calloc: ZEROED, and checks n*size for overflow
  bytes: 00 00 00 00 00 00 00 00 ...
  calloc(SIZE_MAX/2, 4) -> (nil)       ← overflow detected, returns NULL
```

Memory from `malloc` may happen to contain zeros — fresh pages from the kernel always do — but after some allocations and frees it will contain leftovers from earlier use. Never rely on it.

The `calloc` overflow check is a real security feature. Compare:

```
p = malloc(n * size);   /* n*size can WRAP → tiny allocation → heap overflow */
p = calloc(n, size);    /* returns NULL on overflow */
```

An integer overflow in an allocation size is one of the most reliable remote-code-execution primitives: the multiplication wraps to a small number, `malloc` succeeds, and the program then writes `n` elements into a buffer that holds only a few. Use `calloc` for `n * size` computations, or check explicitly:

```
if (n > SIZE_MAX / size) return NULL;
p = malloc(n * size);
```

When the values are known at compile time, GCC even warns:

```
warning: product '9223372036854775807 * 4' of arguments 1 and 2 exceeds 'SIZE_MAX'
         [-Walloc-size-larger-than=]
```

Edge cases

```
malloc(0) -> 0x560d97e8c2d0 (may be NULL or a unique freeable ptr — both legal)
free(NULL) -> guaranteed no-op
realloc(NULL, n) == malloc(n): yes
```

- **`malloc(0)` is implementation-defined:** it returns either `NULL` or a unique pointer that you must still free. Never dereference it, and don't treat `!malloc(0)` as meaning anything in particular.
- **`free(NULL)` is a guaranteed no-op** — this is what makes uniform cleanup labels work (Module 5).
- **`realloc(NULL, n)` is exactly `malloc(n)`,** which lets growth code start from an empty `NULL` buffer.
- **`realloc(p, 0)`** was implementation-defined (some implementations freed `p`, others returned a new small block), and it is deprecated in C23. Don't use it; call `free` explicitly.

`realloc` semantics

```
before: 0x560d97e8c2d0 "hello"
after:  0x560d97e8c2d0 "hello" moved? no
```

`realloc` may grow the block in place, as it did here, or it may allocate a new block, copy the contents, and free the old one. **Assume it moves.** Everything from Module 9, §9.8 applies: every alias to the old pointer becomes invalid.

glibc rounds allocations up

```
malloc( 1) usable=24
malloc( 4) usable=24
malloc(16) usable=24
malloc(64) usable=72
```

Every allocation carries per-chunk metadata — its size and some flag bits — stored just before the returned pointer, and chunks are rounded to a 16-byte boundary. So a `malloc(1)` actually costs about 32 bytes of real memory. This is why allocating millions of tiny objects individually is wasteful — and why pool allocators (§10.7) exist.

> ⚠ `malloc_usable_size` is glibc-specific, and writing into the extra bytes is undefined behaviour even though they physically exist. ASan will catch you.

`aligned_alloc`

```
malloc                -> aligned to max_align_t (16)? yes
aligned_alloc(64,128) -> 0x55e302aac300 (% 64 = 0)
```

`malloc` guarantees alignment suitable for any standard type. For stronger alignment (SIMD, cache lines, DMA), as Module 9 described:

Function	Notes
`aligned_alloc(align, size)`	C11. The size should be a multiple of `align` (required in C11, relaxed in C17). Free with `free`.
`posix_memalign(&p, align, size)`	POSIX. Returns an error number rather than setting `errno`.
`_aligned_malloc` / `_aligned_free`	Windows — must use the matching free

10.4 Checking Allocation Failure

```c
char *p = malloc(n);
if (!p) {
    /* handle it */
    return -1;
}
```

Check every allocation. No exceptions. Dereferencing the `NULL` from a failed allocation is undefined behaviour, and in a function that writes at an offset — `p[n-1] = 0` with a large `n` — it can land on valid memory rather than crashing.

The `realloc` failure pattern

```c
/* WRONG — leaks the original block when realloc fails */
p = realloc(p, newsize);
if (!p) return -1;          /* original block is now unreachable */

/* CORRECT */
char *tmp = realloc(p, newsize);
if (!tmp) {
    /* p is still valid — free it, or return an error and let the caller decide */
    return -1;
}
p = tmp;
```

When `realloc` fails, it leaves the original block untouched and returns `NULL`. Assigning that `NULL` straight back to `p` destroys your only pointer to the still-allocated block. This is the most common `realloc` bug, and it is invisible without LeakSanitizer.

Does `malloc` ever actually fail on Linux?

Not usually, because of *overcommit*. By default, `malloc` returns a pointer to virtual address space, and the kernel assigns physical pages only when they are first touched. If the program then touches more memory than is actually available, the kernel's out-of-memory (OOM) killer terminates a process — possibly yours — and you never see a `NULL` at all.

Check anyway, because:

- Overcommit can be disabled (`vm.overcommit_memory=2`), and often is on servers and embedded systems.
- A 32-bit address space runs out long before physical RAM does.
- `RLIMIT_AS` and cgroup memory limits cause real allocation failures, including in containers.

- Windows, macOS, and most RTOSes behave differently.
- A huge allocation driven by an unvalidated length field *will* fail — and that is exactly the attacker-controlled path you most need to handle.

Failure strategy by context

```c
/* Library code: propagate. Never exit on someone else's behalf. */
int widget_create(Widget **out) { *out = malloc(...); return *out ? 0 : -ENOMEM; }

/* Application code: an xmalloc wrapper is legitimate and common */
static void *xmalloc(size_t n) {
    void *p = malloc(n);
    if (!p) { fputs("out of memory\n", stderr); abort(); }
    return p;
}

/* Embedded / no-heap: don't allocate dynamically at all (§10.7) */
```

A library cannot know whether its caller would prefer to retry, free a cache, degrade gracefully, or give up, so it must report the failure. An application, on the other hand, may reasonably decide that running out of memory is fatal, and an `xmalloc` wrapper that aborts with a clear message is far better than scattering unchecked allocations through the code.

10.5 The Heap Bug Taxonomy

Every one of these was verified live:

Bug	ASan	glibc alone
Heap buffer overflow	heap-buffer-overflow	often silent; sometimes `malloc(): corrupted top size`
Heap underflow (`p[-1]`)	heap-buffer-overflow	`double free or corruption (out)`
Use-after-free	heap-use-after-free	usually silent
Double free	attempting double-free	`free(): double free detected in tcache 2`
Free of a non-start pointer	invalid free	`free(): invalid pointer`
Free of stack memory	invalid free	crash or corruption
Leak	LeakSanitizer: detected memory leaks	nothing
Off-by-one read	heap-buffer-overflow	silent

```
=== heap overflow ===
ERROR: AddressSanitizer: heap-buffer-overflow on address 0x502000000018

=== double free ===
ERROR: AddressSanitizer: attempting double-free on 0x502000000010

=== use after free ===
ERROR: AddressSanitizer: heap-use-after-free /home/claude/c_mem/heapbugs.c:11

=== leak ===
ERROR: LeakSanitizer: detected memory leaks
SUMMARY: AddressSanitizer: 8 byte(s) leaked in 1 allocation(s).
```

```
=== off-by-one read ===
ERROR: AddressSanitizer: heap-buffer-overflow /home/claude/c_mem/heapbugs.c:15
```

Without sanitizers, glibc's own consistency checks catch some of them, but only at the moment of a later `free`:

```
case 2 (underflow):   double free or corruption (out)
case 3 (double free): free(): double free detected in tcache 2
case 5 (bad pointer): free(): invalid pointer
```

Now notice what glibc misses entirely: **leaks, use-after-free reads, and off-by-one reads.** Those are silent — and they are precisely the ones that corrupt data and leak secrets.

Why heap corruption is so dangerous

The allocator stores its bookkeeping — chunk sizes and free-list pointers — *inline*, right next to your data. Overflowing a buffer therefore overwrites the header of the next chunk. Then:

- A later `malloc` may return a pointer computed from corrupted metadata — potentially an address chosen by an attacker.
- The crash, when it comes, happens inside `malloc` or `free`, far from the actual bug, with a stack trace that points at innocent code.
- Attackers have well-known techniques for turning such corruption into control of the program, with names like the *unlink* attack, *tcache poisoning*, and the *House of Force*.

> This is why "it crashed in `malloc`" almost never means `malloc` is broken. It means something corrupted the heap earlier. Run ASan.

The discipline

```c
p = NULL;              /* 1. initialize */
if (!p) return -1;     /* 2. check every allocation */
free(p); p = NULL;     /* 3. null after free */
/* 4. exactly one owner per allocation */
/* 5. every early return goes through the cleanup chain (Module 5) */
/* 6. ASan + LSan on every debug build and in CI */
```

10.6 Ownership Conventions

C has no destructors, no RAII, and no garbage collector. *Ownership* — which part of the program is responsible for eventually freeing a given block — is therefore a convention that you enforce through discipline and documentation. Get this right, and most memory bugs simply disappear: leaks are allocations with no owner, double frees are allocations with two, and use-after-free is a borrower outliving its owner.

The four patterns

1. Caller allocates, caller frees (safest — no ambiguity)

```
/* The function never allocates. Caller controls everything. */
int format_name(char *out, size_t outsz, const Person *p);
size_t get_data(void *buf, size_t bufsz);   /* returns needed size if too small */
```

The caller provides the memory — on the stack, in a structure, or from any allocator it likes — and the function only fills it in. This is the pattern used by `snprintf`, `read`, and `strftime`. Prefer it whenever it fits.

2. Callee allocates, caller frees — must be documented

```
/**
 * Returns a newly allocated string. Caller must free() it.
 * Returns NULL on allocation failure.
 */
char *build_message(const Event *e);
```

Every such function needs that comment, because nothing in the signature says ownership is being handed over. Naming conventions help: suffixes such as `_new`, `_create`, `_dup`, and `_strdup` signal "you now own this."

3. Paired create/destroy (the standard object pattern)

```
Widget *widget_create(const Config *cfg);   /* allocates and initializes */
void    widget_destroy(Widget *w);          /* MUST accept NULL, like free() */
```

The object's internal structure — possibly several allocations — stays hidden behind a matching pair of functions. Making `widget_destroy(NULL)` a no-op makes cleanup chains trivial, exactly as with `free`.

4. Borrowed references — the returned pointer is not owned

```
const char *widget_name(const Widget *w);   /* points INTO w. Do not free.
                                               Invalid after widget_destroy(w). */
```

The caller may use the pointer, but it belongs to someone else, and its lifetime is tied to theirs. Document that lifetime explicitly. `strchr`, `strstr`, and `getenv` all work this way.

Making ownership visible

```
/* Naming */
Thing *thing_new(void);           /* transfers ownership */
Thing *thing_borrow(Pool *p);     /* does not */

/* Type-level: pass by ** when ownership transfers IN */
void take_ownership(char **s) { free(*s); *s = NULL; }

/* Container ownership: store a destructor with the container */
typedef struct { void **items; size_t n; void (*free_item)(void*); } List;
```

When a function takes ownership of something, taking a pointer to the caller's pointer lets it null the caller's copy, so the caller cannot accidentally use or free it again. And a container that owns its elements can store the function needed to destroy them, so that destroying the container frees everything correctly, whatever the element type.

Rules that scale

1. **Exactly one owner.** If two pointers both "own" a block, you will eventually double-free it.
2. **Free at the same layer that allocated,** whenever possible — the module that creates a thing should be the one that destroys it.
3. **Document ownership in the header,** where users of the API will see it — not only in the implementation.
4. **The destroy function accepts `NULL`.**
5. **Set the pointer to `NULL` after destroying** — including inside the destroy function, if it takes a `**`.
6. **On error paths, free everything you have acquired** (Module 5's `goto` chain).
7. **Use reference counting** when shared ownership is genuinely required — and then the count must be atomic if threads are involved (Module 17).

10.7 Custom Allocators

`malloc` is a *general-purpose* allocator: it is thread-safe, handles arbitrary sizes, and supports arbitrary lifetimes, with any block freed at any time in any order. You pay for all of that generality on every call, even when your program does not need it. When you know more about how your program uses memory, a specialised allocator can be dramatically faster.

Arena / bump allocator

An *arena* hands out memory by simply advancing ("bumping") an offset through a large block, and frees everything at once.

```
typedef struct { unsigned char *base; size_t cap, off; } Arena;

static void *arena_alloc(Arena *a, size_t n, size_t align)
{
    size_t p = (a->off + align - 1) & ~(align - 1);   /* round up */
    if (n > a->cap - p) return NULL;                   /* overflow-safe check */
    a->off = p + n;
    return a->base + p;
}

#define ARENA_NEW(a, T)    ((T*)arena_alloc((a), sizeof(T), _Alignof(T)))
#define ARENA_ARR(a, T, n) ((T*)arena_alloc((a), sizeof(T)*(n), _Alignof(T)))

static void arena_reset(Arena *a) { a->off = 0; }   /* free EVERYTHING, O(1) */

/* nested scopes */
static size_t arena_mark(Arena *a)               { return a->off; }
static void   arena_release(Arena *a, size_t m)  { a->off = m; }
```

```
int    at offset 0 (align 4 ok: 1)
double at offset 8 (align 8 ok: 1)   ← note the padding for alignment
```

```
values: 42 3.14 "arena string" used=48/65536
after temp alloc used=1048; after release=48
```

An allocation is two arithmetic operations and a comparison: round the current offset up to the required alignment (the formula from Module 4), check that the request fits, and advance. The `double` landed at offset 8 rather than 4 because it needs 8-byte alignment. `arena_mark` and `arena_release` let a function make temporary allocations and then discard them all in one step, as the last line shows.

Note the overflow-safe bounds check: `if (n > a->cap - p)` rather than `if (p + n > a->cap)`, because `p + n` could wrap around for a huge `n` and pass the test. (For complete robustness, also check that `p <= a->cap` before the subtraction — rounding up can push `p` past the end of an almost-full arena — and that `sizeof(T) * (n)` in `ARENA_ARR` cannot overflow.)

Perfect for: per-request or per-frame memory in servers and games, compiler passes, parse trees, and any workload with a natural "everything dies at once" boundary.

The cost: you cannot free individual objects. That is the trade.

Pool / freelist allocator

A *pool* serves objects of a single fixed size, with O(1) allocation and O(1) freeing. The free list is threaded through the free objects themselves, so there is no metadata overhead at all:

```c
typedef struct Node { struct Node *next; } Node;
typedef struct { unsigned char *mem; Node *free_list; size_t objsz, count; } Pool;

static void *pool_alloc(Pool *p)
{ Node *n = p->free_list; if (!n) return NULL; p->free_list = n->next; return n; }

static void pool_free(Pool *p, void *ptr)
{ Node *n = ptr; n->next = p->free_list; p->free_list = n; }
```

When an object is free, its own first bytes are reused to store the pointer to the next free object. Allocating pops the head of the list; freeing pushes the object back on. (The object size must therefore be at least `sizeof(Node)`, and suitably aligned for a pointer.)

Perfect for: many same-sized objects with individual lifetimes — network connections, AST nodes, particles, list nodes, task structures.

The measured difference

```
=== BENCHMARK: 2,000,000 allocations of 32 bytes (memory touched) ===
  malloc/free : 97.8 ms (1.00x)
  arena       :  7.0 ms (13.98x faster)
  pool        : 18.9 ms ( 5.17x faster)
```

Fourteen times and five times faster. That is why every high-performance C codebase has custom allocators.

Where the speedup comes from: no locking, no searching free lists, no size-class logic, no per-chunk metadata, and — often the biggest factor — *sequential* memory, which gives far better cache behaviour than blocks scattered across the heap.

The other allocators to know

Allocator	Idea	Used for
Stack allocator	An arena with LIFO freeing (mark/release)	Nested scratch buffers
Slab	Pools per object type, caching constructed objects	Linux kernel (`kmem_cache`)
Buddy	Split and merge power-of-two blocks	Kernel page allocator
Ring buffer	A fixed circular region	Streaming, logging, audio
Static pool	A compile-time array, zero heap	Embedded / safety-critical

When to write one

Do write a custom allocator when profiling shows allocation is hot, when you have a natural lifetime boundary, when you need deterministic timing (real-time systems), when fragmentation matters (long-running processes), or when the heap is banned outright (MISRA, DO-178C).

Don't write one as a default. `malloc` is well optimised, and a custom allocator is invisible to ASan — which sees only the one large block you got from `malloc` — so you lose your best bug-detection tool. If you write one, tell ASan about your allocations with manual poisoning:

```
#include <sanitizer/asan_interface.h>
ASAN_POISON_MEMORY_REGION(ptr, size);     /* on free */
ASAN_UNPOISON_MEMORY_REGION(ptr, size);   /* on alloc */
```

Poisoned memory is reported exactly like freed heap memory if the program touches it, so use-after-free bugs in objects from your pool or arena are caught again.

10.8 `alloca` — and Why to Avoid It

`alloca(n)` allocates `n` bytes in the current function's stack frame; the memory is released automatically when the function returns.

```
alloca(64) at 0x7ffc36594310
alloca(64) at 0x7ffc36594310   ← two calls, same frame reused across calls
```

It is tempting — as fast as a local array, with a run-time size — but it shares every problem of VLAs, and adds some of its own.

Why it's dangerous

1. Failure is a silent crash.

```
alloca(4294967296)...
Segmentation fault exit=139
```

No `NULL`, no `errno`, no chance to recover. With an attacker-controlled size, it is a stack-clash vulnerability.

2. In a loop, memory accumulates until the function returns.

```
iter 0: 0x7ffc36593f50 (offset    0)
iter 1: 0x7ffc36593b40 (offset 1040)
iter 2: 0x7ffc36593730 (offset 2080)
iter 3: 0x7ffc36593320 (offset 3120)
iter 4: 0x7ffc36592f10 (offset 4160)
```

Five calls to `alloca(1024)` consumed 5 KB, because nothing is released until the function returns — not at the end of each iteration. A loop with a large iteration count blows the stack.

3. It's not standard C. ISO C does not mention it, and MSVC provides a different `_alloca`.

4. It breaks if inlined into a loop — an `alloca` inside a small function that the compiler inlines into a caller's loop suddenly accumulates exactly as above — and it interacts badly with `longjmp` and other non-local control flow.

5. `alloca` inside a function argument list is UB on some platforms. `f(alloca(n), alloca(m))` can corrupt the area where the arguments themselves are being assembled on the stack.

```
warning: use of 'alloca' [-Walloca]
```

> Add `-Walloca -Werror=vla` to your build. Use `malloc` (checkable) or a fixed buffer with a bound check. The hybrid stack/heap pattern from Module 7 gives you the speed without the risk.

10.9 Zeroing Sensitive Memory

Passwords, keys, and tokens should not outlive their use. The obvious way to wipe one is `memset` — and the obvious way does not work.

```c
void handle_password(void)
{
    char pw[32];
    strcpy(pw, "hunter2-secret!");
    memset(pw, 0, sizeof pw);   /* wipe the secret */
}
```

Here is what GCC generates at `-O2`:

```
handle_password:
    endbr64
    ret
```

The entire function is gone. The compiler saw that `pw` is never read after the `memset`, so the `memset` is a *dead store* and can be deleted — and once it was gone, the `strcpy` was dead too. In this toy example nothing sensitive is left behind, because nothing was ever written. But in real code, where the secret *is* used before the wipe, the compiler keeps the use and deletes only the wipe — leaving the secret sitting on the stack, ready to be read by the next function's uninitialized locals (Module 3), leaked in a core dump, or swapped out to disk.

The correct version:

```
explicit_bzero(pw, sizeof pw);
```

```
handle_password_safe:
    sub     rsp, 56
    mov     edx, 32
    movdqa xmm0, XMMWORD PTR .LC0[rip]
    movaps XMMWORD PTR [rsp], xmm0
    call    __explicit_bzero_chk@PLT   ← the wipe actually happens
    ...
```

`explicit_bzero` is defined so that the compiler must assume its effect is observable, and the string, the copy, and the wipe are all preserved.

The portable options

Function	Availability
`explicit_bzero(p, n)`	glibc 2.25+, OpenBSD, FreeBSD
`memset_explicit(p, c, n)`	C23 — the standard answer
`memset_s(p, smax, c, n)`	C11 Annex K — optional, mostly MSVC
`SecureZeroMemory(p, n)`	Windows

A portable fallback:

```
static void secure_zero(void *p, size_t n)
{
    volatile unsigned char *v = p;   /* volatile forbids eliding the stores */
    while (n--) *v++ = 0;
}
```

Every write through a `volatile` pointer must actually be performed (Module 2), so the compiler cannot remove these stores. Alternatively, force a compiler barrier after an ordinary `memset`: `memset(p, 0, n); __asm__ __volatile__("" ::: "memory");`. The empty assembly statement claims to read and write arbitrary memory, so the compiler must assume the zeros are observed.

Beyond zeroing

Zeroing is necessary, but not sufficient. For real secret handling:

- **`mlock` the pages,** so that they are never swapped to disk.
- **Disable core dumps** (`setrlimit(RLIMIT_CORE, ...)` with a limit of 0) in processes that hold secrets.

- **Minimise copies.** Every `strcpy` of a key is one more location that must be wiped.
- **Remember that `realloc` may copy the data and leave the old block unwiped.** For secrets, allocate the new block yourself, copy, wipe the old block, then free it.
- **Use constant-time comparison** (Module 8) — timing leaks defeat all of the above.

Module 10 Rules

#	Rule
1	`.rodata` is read-only at the page level — `char *s = "x"; s[0]='y';` segfaults
2	Nothing large on the stack; thread stacks can be much smaller than 8 MB
3	`calloc(n, size)` for `n * size` — it checks for overflow
4	Check every allocation, including on platforms where it "never fails"
5	Never `p = realloc(p, n)` — use a temporary, or you leak on failure
6	`free(NULL)` is a no-op; `destroy(NULL)` should be too
7	Assume `realloc` moves; every alias is invalidated
8	Set pointers to `NULL` after `free` — and know that aliases still dangle
9	Exactly one owner per allocation, documented in the header
10	"Crashed in `malloc`" means something corrupted the heap earlier — run ASan
11	Custom allocators only where profiling justifies them; add ASan poisoning
12	Ban `alloca` and VLAs: `-Walloca -Werror=vla`
13	A `memset` of a dying secret is deleted — use `explicit_bzero` / `memset_explicit`
14	ASan + LSan + UBSan on every debug build and in CI. Non-negotiable.

Standing build commands:

```
# Debug / CI
gcc -std=c17 -Wall -Wextra -Wpedantic -Wshadow -Wconversion \
    -Wcast-qual -Wcast-align=strict -Wpointer-arith -Wwrite-strings \
    -Walloca -Werror=vla -Werror=return-type -Werror=implicit-function-declaration \
    -O1 -g -D_FORTIFY_SOURCE=3 -fsanitize=address,undefined -fno-omit-frame-pointer \
    prog.c -o prog

# Release
gcc -std=c17 -O2 -D_FORTIFY_SOURCE=3 -fstack-protector-strong \
```

```
    -fstack-clash-protection -fcf-protection -Wl,-z,relro,-z,now,-z,noexecstack \
    prog.c -o prog
```

Run debug builds with:

```
ASAN_OPTIONS=detect_leaks=1:strict_string_checks=1:detect_stack_use_after_return=1
```

This is the first time the book has shown two builds. The debug build maximises bug detection: sanitizers, and `-fno-omit-frame-pointer` for reliable stack traces. The release build adds the hardening features from this module and Module 6: stack canaries, stack-clash probing, control-flow protection, and linker options that make relocation tables read-only (`relro`, `now`) and mark the stack non-executable. Module 23 explains each hardening flag in depth.

Practice

1. **Map your own process.** Print addresses from every region as in §10.1, then dump `/proc/self/maps` and match each address to a mapping. Identify which regions are `r-x`, `rw-`, and `r--`, and explain why.

2. **Prove the page protections.** Write four programs, which respectively try to write to `.rodata`, execute code on the heap, write to `.text`, and read page 0. Record which signal each one produces.

3. **Overflow arithmetic bug.** Write `Buf *buf_new(size_t n, size_t elemsize)` using `malloc(n * elemsize)` with no check. Call it with `n = SIZE_MAX/2` and `elemsize = 4`, then write `n` elements. Watch ASan. Fix it three ways: an explicit check, `calloc`, and `__builtin_mul_overflow`.

4. **The `realloc` leak.** Write a growable array using `p = realloc(p, n)`. Force `realloc` to fail (with `RLIMIT_AS` via `ulimit -v`, or an `LD_PRELOAD` failure injector). Confirm that LSan reports the leak. Fix it with the temporary-pointer pattern.

5. **Full bug matrix.** Reproduce all eight heap bugs from §10.5. For each, record the result of a plain run, glibc's own checks, ASan, and Valgrind. Build a table of which tool catches what. This is the single most useful table you can have on your desk.

6. **Build an arena.** Implement the arena from §10.7 with alignment, mark/release, and a chained-block version that grows by allocating new blocks instead of failing. Add ASan poisoning. Benchmark it against `malloc` on a realistic workload (parsing 10,000 JSON-like records).

7. **Build a pool.** Implement a fixed-size pool with a free list, then extend it to grow by chaining new slabs. Add a debug mode that fills freed objects with `0xDD` and detects double frees by checking for that pattern.

8. **Ownership audit.** Take any program you have written during this course. For every pointer, write down who allocates it, who frees it, and what its lifetime is. Find at least one case where it is ambiguous, and change the API to make it unambiguous.

9. **The secret wipe.** Reproduce the `memset` elimination at `-O2` with `-S`. Then test `explicit_bzero`, `memset_explicit` (with `-std=c2x`), the `volatile` loop, and the memory-barrier trick. Verify in the assembly which of them survive.

10. **`alloca` in a loop.** Write a function that calls `alloca(4096)` in a loop and prints its stack usage on each iteration. Find the iteration count at which it crashes. Then rewrite it with `malloc` and confirm that you get a clean failure instead.

Module 10 in Summary

A process's memory is divided into regions with hardware-enforced permissions: read-only literals, execute-only code, read-write data, a heap growing up, an `mmap` area for libraries and large blocks, and a stack growing down above guard pages. The stack is small, unzeroed, and unforgiving — keep large objects off it and never size it from untrusted input.

`malloc` returns uninitialized memory, `calloc` zeroes it and checks the size multiplication, and `realloc` may move the block, so always use a temporary and assume every alias is invalidated. Check every allocation, even on systems that overcommit. The heap bugs — overflows, use-after-free, double frees, invalid frees, and leaks — are mostly silent without sanitizers, and heap corruption surfaces far from its cause, so ASan and LSan belong in every debug build.

Clear ownership prevents most of these bugs: exactly one owner, documented in the header, with destroy functions that accept `NULL`. Where profiling justifies it, arenas and pools are many times faster than `malloc`, provided they teach ASan about their allocations. Avoid `alloca`. And wipe secrets with a function the optimiser cannot remove — then keep them out of swap, core dumps, and extra copies.

In Module 11, we turn to Structs, Unions, and Enums — layout, padding, tagged unions, and the opaque types that make encapsulation possible in C.

Module 11 — Structs, Unions, Enums

In this module:

- Struct declaration, initialization, nesting, assignment/copy semantics
- Passing structs by value vs pointer
- Padding, alignment, `offsetof`, struct packing, cache-friendly layout
- Bitfields and their portability caveats
- Unions, type punning via union, tagged unions
- Enums, enum scoping, C23 fixed underlying type
- `typedef` — good and bad practice
- Opaque structs (`typedef struct Foo Foo;`) for encapsulation
- Self-referential structs, intrusive linked structures

All output in this module is real GCC 13.3 output, including the measured benchmarks.

Structures are C's only way to build new kinds of data. There are no classes, no records with methods, no tuples — just `struct`, `union`, and `enum`, together with the functions you write around them. Yet with those three tools, C programmers build everything from network protocol headers to interpreters to operating-system kernels. This module covers how they work, how they are laid out in memory, and the design patterns — tagged unions, opaque types, intrusive lists — that professional C code is built from.

Pay particular attention to §11.3. Padding and cache-friendly layout are where the biggest real-world performance wins in C hide, and they cost nothing but a moment's thought when you declare a structure.

11.1 Struct Declaration, Initialization, Copy Semantics

```c
struct Inner { int a, b; };
struct Outer { struct Inner in; char name[8]; int *ptr; };

struct Outer o1 = { .in = {1,2}, .name = "hello", .ptr = malloc(sizeof(int)) };
```

A structure groups named members, possibly of different types, into one object. Members can themselves be structures, arrays, or pointers. Access members with `.` on a structure object and with `->` on a pointer to one: `p->x` is exactly `(*p).x`, written more conveniently.

Struct assignment is a memberwise copy — including arrays

```c
struct Outer o2 = o1;
o2.in.a = 99; strcpy(o2.name, "world");
```

```
after modifying o2: o1.in.a=1 o1.name="hello" (independent)
```

Assigning one structure to another copies every member, and `o2` is a completely independent object afterwards. This is the one place C copies an array: a bare `int a[5]; b = a;` is an error (Module 7), but `struct S { int a[5]; }; s2 = s1;` copies all five elements. Wrapping an array in a structure turns it into a first-class value — a real technique for small vectors, matrices, fixed-size identifiers, and hash digests.

But it's a shallow copy

```
o1.ptr=0x5603c979f2a0 o2.ptr=0x5603c979f2a0 same? 1
*o1.ptr is now 555 <- modified through o2
-> double free if BOTH are freed
```

Structure assignment copies a pointer member's *value* — the address — not the thing it points at. After `o2 = o1`, both structures point at the same heap block, so a change through one is visible through the other, and freeing both frees the same block twice. If your structure owns heap memory, you need an explicit *deep-copy* function:

```c
int outer_clone(struct Outer *dst, const struct Outer *src)
{
    *dst = *src;                          /* shallow first */
    dst->ptr = malloc(sizeof *dst->ptr);  /* then fix up owned members */
    if (!dst->ptr) return -1;
    *dst->ptr = *src->ptr;
    return 0;
}
```

The shallow copy takes care of every plain member in one line; then each *owning* pointer is replaced with a freshly allocated copy of what it points to.

> **Rule:** if a structure contains an owning pointer, it needs a clone function and a destroy function (Module 10). Document that plain assignment is a shallow copy.

⚠ Never `memcmp` two structs

```c
struct P { char c; int i; };   /* 3 bytes of padding after c */
```

```
same member values: a.c=='x' a.i==42, b.c=='x' b.i==42
memcmp(&a,&b,sizeof) = 255   <-- NOT ZERO
raw bytes a: 78 FF FF FF 2A 00 00 00
raw bytes b: 78 00 00 00 2A 00 00 00
               ^^ the padding bytes differ
```

The compiler inserts three bytes of padding after `c` so that `i` is aligned (§11.3). Those padding bytes belong to no member, and their values are indeterminate — here they held leftovers from earlier use of the memory. Two structures with identical member values can therefore have different bytes. Compare member by member instead:

```c
static bool p_equal(const struct P *a, const struct P *b)
{ return a->c == b->c && a->i == b->i; }
```

The same reasoning applies to **hashing** a structure byte by byte; to **writing** it to a file or a socket, where the padding leaks whatever stack or heap contents it happened to contain — a real class of information-disclosure CVEs; and to using `memcmp` on structures containing **floats**, where `-0.0 == 0.0` although their bits differ, and a NaN is unequal to itself although its bits match.

If you must compare or serialise raw bytes, `memset(&s, 0, sizeof s)` first. But note that C only guarantees the padding is zero *at the moment of the `memset`*; the standard allows later member assignments to disturb padding bytes, so this is a mitigation, not a guarantee. For anything that crosses a trust boundary, serialise each field explicitly.

Initialization

```c
struct P p1 = {1, 2};          /* positional */
struct P p2 = {.i = 42};       /* designated (C99) — c is zeroed */
struct P p3 = {0};             /* all zero */
struct P p4 = {};              /* C23 */
p = (struct P){0};             /* compound literal assignment (Module 7) */
memset(&p, 0, sizeof p);       /* zeroes padding too */
```

Prefer designated initializers. They are order-independent, self-documenting, and survive reordering of the members — which, as §11.3 shows, you may well want to do.

11.2 By Value vs by Pointer

Measured over 20 million calls, at `-O2`, with inlining disabled so that each call really happens:

```
sizeof Small=16 Large=512
  Small by value : 60.9 ms
  Small by ptr   : 60.3 ms   ← no difference
  Large by value : 386.7 ms
  Large by ptr   : 57.6 ms   ← 6.7x faster
```

Small structs by value are free. On System V AMD64, a 16-byte structure of integers travels in two registers (Module 6, §6.7) — no memory traffic at all. Types like `Point`, `Vec2`, `Complex`, and `Range` should be passed and returned by value; it is idiomatic, and often faster than a pointer, because there is no indirection and the compiler need not worry that the callee's pointer aliases anything else.

Large structs by value cost real copies: here, 512 bytes copied on every call, plus the stack space to hold them.

The guideline

Size	Pass by
≤ 16 bytes (2 registers)	value
17 to ~64 bytes	either; measure
> ~64 bytes	`const` pointer

Size	Pass by
Must modify the caller's object	pointer (always)
Contains an owning pointer	pointer + explicit clone semantics

```
Point  midpoint(Point a, Point b);                /* small: by value */
double area(const struct Polygon *p);             /* large: const pointer */
int    update(struct State *s, const Event *e);   /* modifies s, reads e */
```

Always `const` the pointer parameters whose targets you don't modify (Module 9). It documents intent, and it lets callers pass `const` objects.

Returning a large structure by value is usually fine. As Module 6 described, the ABI has the caller allocate space for the result and pass a hidden pointer to it, so the callee builds the result directly in its final location, and there is often no extra copy at all.

11.3 Padding, Alignment, and Cache-Friendly Layout ⚠

The three layout rules

1. **Each member sits at an offset that is a multiple of its alignment.** The compiler inserts padding before a member when necessary to achieve this.
2. **The structure's alignment is the maximum of its members' alignments.**
3. **`sizeof` is rounded up to a multiple of the structure's alignment,** adding trailing padding if necessary — so that in an array of the structure, every element stays aligned.

Members are always laid out in the order you declare them; C compilers are not allowed to reorder them. That puts the responsibility for a good layout squarely on you.

Member order changes everything

```
struct Bad  { char a; double b; char c; int d; char e; };
struct Good { double b; int d; char a, c, e; };   /* same members! */
```

```
struct Bad
  a offset= 0 size= 1 align=1
  b offset= 8 size= 8 align=8   ← 7 bytes padding before b
  c offset=16 size= 1 align=1
  d offset=20 size= 4 align=4   ← 3 bytes padding before d
  e offset=24 size= 1 align=1
  TOTAL sizeof=32 alignof=8 (payload = 15 bytes)
  WASTED: 17 bytes of padding (53%)

struct Good
  b offset= 0 size= 8
  d offset= 8 size= 4
  a offset=12 c offset=13 e offset=14
  TOTAL sizeof=16 alignof=8
  WASTED: 1 byte -> 50% smaller than Bad
```

In `struct Bad`, every small member is followed by a larger one that needs alignment, so padding is inserted again and again — and then 7 more bytes of trailing padding after `e` round the size up to 32. `struct Good` holds exactly the same members, but orders them so that each one naturally falls on a suitable boundary. Identical members, identical semantics, half the memory.

> **The rule:** declare members in decreasing order of alignment — `double`s and pointers, then `long`, `int`, `short`, `char`, and `bool`. This is nearly optimal and costs nothing.

For a structure you allocate a million of, that is 32 MB versus 16 MB — and, more importantly, half as many cache lines touched whenever you walk through them.

Tools: `pahole` (from the `dwarves` package) analyses a compiled binary and reports the padding holes in every structure automatically. Run it on your hot structures.

`__attribute__((packed))` — usually wrong

```
__attribute__((packed)): sizeof=15 alignof=1
-> no padding, but members are MISALIGNED
```

Packing removes all padding. Here it saves just 1 byte compared with `struct Good`, and you pay for it:

- **Misaligned member access** — slower on x86, and a fault on some ARM and SPARC configurations (Module 9, §9.9).
- **Taking the address of a packed member yields a misaligned pointer,** which is undefined behaviour to dereference. GCC warns about it with `-Waddress-of-packed-member`.
- **It blocks vectorisation.**

Use `packed` only for parsing wire formats and on-disk structures whose layout you do not control — and even then, prefer explicit `memcpy`-based field extraction (Module 8), which is portable and makes byte order explicit.

Cache-friendly layout — the measured impact

```c
struct Fat  { int key; char blob[60]; };   /* 64 bytes: 1 per cache line */
struct Slim { int key; };                  /* 4 bytes: 16 per cache line */
```

Summing 4 million keys — identical arithmetic, different memory footprint:

```
AoS with 60-byte blob : 17.8 ms (244 MB touched)
keys only (SoA style) :  2.5 ms (15 MB touched)
speedup: 7.17x
```

Seven times faster, from layout alone. The CPU always loads a full 64-byte cache line from memory. Reading a 4-byte key from a 64-byte structure uses 4 bytes of every line fetched and wastes the other 60 — about 93% of the memory bandwidth. When the keys are stored densely, every byte fetched is used.

Array of Structs vs Struct of Arrays

```
/* AoS — natural, but wasteful when you touch one field */
struct Particle { float x,y,z, vx,vy,vz; float mass; int id; };
struct Particle particles[N];

/* SoA — every field is a dense contiguous array */
struct Particles {
    float x[N], y[N], z[N];
    float vx[N], vy[N], vz[N];
    float mass[N]; int id[N];
};
```

SoA wins when you process one or two fields across many objects — physics updates, filters, aggregations. Each field is perfectly contiguous, so the cache is used fully, and the compiler can easily auto-vectorise the loop with SIMD instructions.

AoS wins when you touch most of the fields of one object at a time — per-entity logic, tree nodes — because everything about that object is in one or two cache lines.

This is the core idea of *data-oriented design* (Module 25): design data layout around how the data is accessed. Layout is not an afterthought.

Two more layout techniques

Hot/cold splitting — keep frequently accessed fields together in the main structure, and move rarely used ones behind a pointer:

```
struct Connection {
    int fd; uint32_t state; uint64_t last_active;   /* hot: fits in one cache line */
    struct ConnDetails *cold;                       /* logs, config, names */
};
```

A server that scans thousands of connections checking `state` and `last_active` now touches one small line per connection instead of dragging logs and configuration strings through the cache.

Cache-line alignment to avoid false sharing (Module 4, §4.5). When two threads write to different variables that happen to share a cache line, the line bounces between their cores on every write — *false sharing*. Giving each variable its own line avoids it:

```
struct Counter { alignas(64) _Atomic long value; };   /* one per cache line */
struct Counter counters[NTHREADS];                    /* threads no longer fight over lines */
```

(In C, `alignas` is applied to a member or an object; the member's alignment makes the whole structure 64-byte aligned and 64 bytes in size. The C++ spelling `struct alignas(64) Counter` is not valid C.)

`offsetof(type, member)`, from `<stddef.h>`, gives a member's byte offset within its structure — essential for layout analysis, for serialisation, and for `container_of` (§11.9).

11.4 Bitfields (Recap + the Struct Angle)

Bitfields were covered in Module 4, §4.9. The essentials:

```
struct Flags { unsigned ready:1; unsigned mode:3; unsigned :0; unsigned other:4; };
```

Portability caveats — all implementation-defined:

- The bit order within a storage unit.
- Whether a field may straddle a storage-unit boundary.
- The total size and alignment of the structure.
- The signedness of plain `int x:1` — Module 4 showed `int s:1` storing 1 and reading back −1.

Also: you cannot take a bitfield's address, overflow silently truncates, and concurrent writes to *different* bitfields in the same storage unit are a data race. (In the C11 memory model, a run of adjacent bitfields counts as a single memory location; only a zero-width `:0` field between them splits them into separate locations.)

> Bitfields are fine for compacting internal flags. Never use them for hardware registers or wire formats — use explicit shifts and masks.

11.5 Unions

```
union U { uint32_t u; float f; unsigned char b[4]; };
```

```
sizeof(union U) = 4 (largest member), alignof = 4
wrote f=1.0f -> u.u = 0x3F800000 bytes: 00 00 80 3F
```

A union is like a structure in which every member starts at offset 0: all members share the same storage, and at any moment it holds just one of them. `sizeof` a union is the size of its largest member (rounded up for alignment), and its alignment is the strictest of its members'. Writing `1.0f` into `f` and then reading `u` and `b` shows the same four bytes seen three different ways: as a float, as the integer `0x3F800000` (Module 2's bit pattern), and as individual bytes in little-endian order.

Type punning via union

Reading a member other than the one most recently written is legal in C: this was clarified in C99's Technical Corrigendum 3 and is stated explicitly in C11 (§6.5.2.3). The stored bytes are simply reinterpreted as the type of the member you read.

> ⚠ **It is undefined behaviour in C++.** For code that must compile as both languages, or for maximum portability, use `memcpy` (Module 2, §2.11) — it compiles to identical instructions.

> ⚠ Reading a member whose type has trap representations, or a member larger than the one that was written, reads bytes whose values are indeterminate.

Tagged unions — the safe pattern

A bare union is unsafe: nothing records which member is currently active, so nothing stops you reading a `char *` from a union that holds a `double`. Pair the union with a *discriminant* — a tag saying which member is valid:

```c
typedef enum { V_INT, V_FLOAT, V_STR } Kind;

typedef struct {
    Kind kind;
    union { long i; double f; char *s; } as;
} Value;

static void print_value(const Value *v)
{
    switch (v->kind) {   /* omit default → -Wswitch enforces coverage */
    case V_INT:   printf("int %ld\n", v->as.i);       break;
    case V_FLOAT: printf("float %g\n", v->as.f);      break;
    case V_STR:   printf("str \"%s\"\n", v->as.s);    break;
    }
}
```

```
int 42
float 3.14
str "hello"
sizeof(Value) = 16 (enum 4 + union 8 + padding)
```

That is 16 bytes. Storing all three payloads side by side as separate members would need 4 + 8 + 8 + 8 = 28 bytes, rounded up to 32 for alignment — twice the size. It is how interpreters represent values, how compilers represent syntax-tree nodes, and how protocol code represents the different kinds of message. And because the `switch` has no `default`, adding a new `Kind` makes `-Wswitch` point at every function that needs updating (Module 5).

The discipline: always set the tag and the payload together, ideally through a constructor, so that they can never disagree:

```c
static Value v_int(long x)     { return (Value){ .kind = V_INT,   .as.i = x }; }
static Value v_float(double x) { return (Value){ .kind = V_FLOAT, .as.f = x }; }
```

Anonymous unions and structs (C11)

```c
struct { int tag; union { int i; float f; }; } an = { .tag = 1, .i = 7 };
printf("%d\n", an.i);   /* no ".u." needed */
```

```
an.i = 7 (no member name needed)
```

An unnamed union (or structure) inside another structure makes its members accessible as if they were members of the outer structure. This gives cleaner syntax for tagged unions and for protocol headers with overlapping fields.

Other legitimate uses

```c
/* Guaranteeing alignment/size */
union Aligned { max_align_t _a; char bytes[256]; };

/* Compact variant storage in a container */

/* Endian inspection */
union { uint32_t u; unsigned char b[4]; } e = { .u = 1 };
bool little_endian = (e.b[0] == 1);
```

The first forces a byte buffer to have the strictest alignment on the platform, so that any object can be stored in it. The last stores the integer 1 and checks which byte it ended up in: on a little-endian machine the least significant byte comes first.

11.6 Enums

```c
enum Color { RED, GREEN, BLUE };                /* 0, 1, 2 */
enum Mixed { M_X=10, M_Y, M_Z=20, M_W };        /* 10, 11, 20, 21 */
```

```
RED=0 GREEN=1 BLUE=2
M_X=10 M_Y=11 M_Z=20 M_W=21 (auto-increment from the last value)
```

An enumeration defines a set of named integer constants. Unless you say otherwise, the first is 0 and each subsequent constant is one more than the one before; an explicit value restarts the counting from there.

The three things people get wrong

1. There is no scoping. Enum constants live in the ordinary identifier namespace, at file (or block) scope, alongside variables and functions:

```c
enum A { OK }; enum B { OK };   /* ERROR: redefinition */
```

That is why prefixes are mandatory in real code: `COLOR_RED`, `STATE_IDLE`, `TOKEN_PLUS`. Two enums in different headers that both define `OK` or `ERROR` will collide the moment one file includes both.

2. Enums are not type-safe.

```c
enum Color c = 42;   /* no error, no warning by default */
```

```
enum Color c = 42; -> c=42 (any int assigns)
```

An enum type is compatible with some integer type, and the constants themselves have type `int`; arithmetic on enums yields `int`. So what you get from an enum is documentation and `-Wswitch` exhaustiveness checking — not enforcement. (Clang's `-Wassign-enum` catches some out-of-range assignments; with GCC, `-Wswitch` is the main win.)

3. The underlying type is implementation-defined (before C23):

```
sizeof(enum Color)=4 sizeof(enum Big)=4
```

The compiler picks some integer type able to hold all the values — usually `int`. Never put a bare enum in a structure that must match a wire format or an ABI boundary; use `uint8_t` or `uint32_t` for the field, and convert to and from the enum.

C23 fixes the underlying type

```c
enum Small : unsigned char { S_A, S_B, S_C };
enum Wide  : long long     { W_A = 1LL << 40 };
```

```
enum Small : unsigned char -> sizeof=1
enum Wide : long long      -> sizeof=8, W_A=1099511627776
```

C23 lets you specify the underlying type explicitly, and also allows enumeration constants whose values exceed the range of `int`. This finally makes enums usable directly in compact structures and binary formats. (A trailing comma after the last enumerator, which you will often see, has been permitted since C99.)

The two idioms worth memorising

Flag enums:

```c
enum Flags { F_A = 1u<<0, F_B = 1u<<1, F_C = 1u<<2, F_D = 1u<<3 };
unsigned f = F_A | F_C;
if (f & F_C) { ... }
```

```
f = F_A|F_C = 5 has F_C? 1 has F_B? 0
```

Each constant is a single bit, so combinations can be built with `|` and tested with `&` (Module 4). Store a combination in an `unsigned`, not in the enum type — the combined value is not one of the enumerators, and storing it in an enum variable misleads readers and tools.

The COUNT idiom — keeps parallel tables in sync:

```c
enum State { STATE_IDLE, STATE_RUN, STATE_DONE, STATE_COUNT };

static const char *const state_names[] = {
    [STATE_IDLE] = "idle",
    [STATE_RUN]  = "run",
    [STATE_DONE] = "done",
};
_Static_assert(sizeof state_names / sizeof *state_names == STATE_COUNT,
               "state_names out of sync with enum State");
```

`STATE_COUNT` is not a real state; because it is declared last, its value is the number of real states. Add a new last state and forget its name, and the build fails. (As Module 7 noted, a state inserted in the *middle* without a name slips past the assertion and leaves a `NULL` hole in the table, so a unit test that checks every entry is a useful complement.) This pattern, plus `-Werror=switch`, gives you most of the safety of a real sum type.

Anonymous enums as compile-time constants:

```c
enum { MAX_USERS = 1024, BUFFER_SIZE = 4096 };
int buf[BUFFER_SIZE];   /* works — enum constants ARE constant expressions,
                           unlike `const int` in C */
```

Unlike a `const int` (Module 2), an enumeration constant is a true compile-time constant, usable as an array size, a `case` label, or a static initializer. It also has a proper name visible in the debugger, unlike a `#define`.

11.7 `typedef` — Good and Bad Practice

A `typedef` creates an *alias* for an existing type, not a new type. No additional type checking is added: a `typedef int Meters;` and a `typedef int Seconds;` can be freely mixed.

Good uses

```c
/* 1. Function pointers — always (Module 9) */
typedef int (*Comparator)(const void *, const void *);

/* 2. Opaque handles (§11.8) */
typedef struct Database Database;

/* 3. Platform abstraction */
typedef uint32_t pixel_t;
typedef int64_t  timestamp_ms_t;

/* 4. Genuinely awkward declarations */
typedef char (*ArrayOfTenCharPtrs[10]);
```

Each of these either makes something readable that otherwise would not be, hides something that ought to be hidden, or gives a meaningful name to a type whose exact representation might change. (A note on the `_t` suffix: POSIX reserves names ending in `_t` for itself, so large projects often choose a different convention for their own types.)

Bad uses

```c
typedef struct { int x, y; } *PointPtr;   /* HIDING A POINTER — the worst offender */
typedef unsigned int uint;                /* saves 4 characters, adds a lookup */
typedef struct Foo FooStruct;             /* redundant */
```

Hiding a pointer behind a `typedef` is the cardinal sin. A reader of `PointPtr p;` cannot tell that `p` is a pointer, that it may need freeing, or that copying it creates an alias. They cannot apply `const` correctly either: `const PointPtr` makes the *pointer* constant, not the point it refers to, and there is no way to spell "pointer to const point" through the `typedef` at all. And the indirection — the thing most likely to be null, to dangle, or to alias — is invisible at every use.

The struct-typedef debate

```c
/* Style A — Linux kernel: no typedef for visible structs */
struct point { int x, y; };
struct point p;

/* Style B — common in application code */
```

```
typedef struct { int x, y; } Point;
Point p;
```

The kernel style argues that writing the `struct` keyword tells you, at every use site, that you are dealing with a structure — something that may be large, that is copied member by member, and whose layout you can see. Application code tends to prefer brevity. Both positions are defensible; be consistent within a codebase. The one universal rule: use a `typedef` for opaque handles, where hiding the layout is the whole point, and never for pointers.

Note the self-referential form:

```
typedef struct Node Node;   /* declare the tag AND the alias */
struct Node { int v; Node *next; };
```

Declaring the `typedef` first, using the structure's tag, lets the structure's own members use the short name.

11.8 Opaque Structs — Encapsulation in C

The header *declares* the type without *defining* it. Only the `.c` file knows the layout.

```
/* counter.h */
typedef struct Counter Counter;   /* incomplete type */

Counter    *counter_create(const char *name);
void        counter_destroy(Counter *c);   /* accepts NULL */
void        counter_inc(Counter *c);
long        counter_value(const Counter *c);
const char *counter_name(const Counter *c);
```

```
/* counter.c */
struct Counter { char *name; long value; int magic; };   /* PRIVATE */
#define MAGIC 0xC0FFEE

Counter *counter_create(const char *name)
{
    Counter *c = calloc(1, sizeof *c);
    if (!c) return NULL;
    c->name = strdup(name ? name : "");
    if (!c->name) { free(c); return NULL; }
    c->magic = MAGIC;
    return c;
}
void counter_destroy(Counter *c)
{ if (!c) return; c->magic = 0; free(c->name); free(c); }
```

```
hits = 5
ASan exit=0
```

Everything outside `counter.c` sees `Counter` only as an *incomplete type* (Module 3): it is known to exist, but its size and members are not. Callers can hold and pass around `Counter *` pointers, but they can do nothing else with them except call the functions in the header.

The compiler enforces the encapsulation:

```
error: invalid application of 'sizeof' to incomplete type 'Counter'
error: invalid use of incomplete typedef 'Counter'
```

What callers cannot do

- Evaluate `sizeof(Counter)` — it is an incomplete type.
- Declare an object: `Counter x;`.
- Access a member: `c->value`.
- Copy one by assignment.
- Depend on the layout in any way.

What you get

Benefit	Why it matters
ABI stability	You can add, reorder, or remove fields without recompiling callers
Real invariants	State can only change through your functions
Smaller headers	No need to `#include` the types your fields use
Faster builds	Changing the layout doesn't trigger a rebuild of the world

Note the `magic` field — a cheap sanity check that catches use-after-free and type confusion in debug builds. Each API function can `assert(c->magic == MAGIC);` `counter_destroy` clears it, so a later call on a freed counter is likely to fail the check instead of silently corrupting memory. Real libraries use the same trick; glibc's `FILE` structure, for example, carries a magic number in its flags.

> This is the single most important API design pattern in C. `FILE *`, `sqlite3 *`, `SSL_CTX *`, `png_structp` — every serious C library works this way.

The trade-off: each object needs its own allocation (callers cannot put one on the stack or embed it in their own structures), and every field access is a function call. When that matters, some libraries expose the structure in the header but document that only the API may touch its members — a weaker, convention-only guarantee. That is why `pthread_mutex_t` is a visible structure (so it can be embedded and statically initialized) whose contents are nonetheless off-limits.

11.9 Self-Referential Structs and Intrusive Data Structures

Self-reference

```c
struct Node { int value; struct Node *next; };   /* pointer to incomplete self: OK */
struct Bad  { int value; struct Bad self; };     /* ERROR: infinite size */
```

A structure cannot contain itself — its size would be infinite — but it can contain a *pointer* to its own type, because inside its own definition the structure is an incomplete type, and pointers to incomplete types are allowed. This is how every linked data structure in C is built.

Mutually recursive types need a forward declaration:

```
struct B;
struct A { struct B *b; };
struct B { struct A *a; };
```

Intrusive lists — the Linux kernel pattern

In a conventional (“non-intrusive”) list, the list allocates its own nodes, and each node points to a data object. In an *intrusive* list, the list node lives *inside* the object:

```
struct list_head { struct list_head *next, *prev; };

#define container_of(ptr, type, member) \
    ((type *)((char *)(ptr) - offsetof(type, member)))

#define list_entry(ptr, type, member) container_of(ptr, type, member)

#define list_for_each(pos, head) \
    for (pos = (head)->next; pos != (head); pos = pos->next)
```

The list code only ever sees `struct list_head` nodes, linked into a circular doubly linked list. `container_of` is the trick that makes it work: given a pointer to an embedded member, subtract that member’s offset within the structure to recover a pointer to the enclosing object.

```
struct Task {
    char name[16];
    int prio;
    struct list_head all;     /* membership in "all tasks" */
    struct list_head ready;   /* membership in "ready queue" */
};
```

```
offsetof(Task, all)   = 24
offsetof(Task, ready) = 40

ALL tasks (via .all):     READY tasks (via .ready):
  disk prio=20              disk prio=20
  net  prio=10              init prio=0
  init prio=0             ← SAME objects, DIFFERENT list
```

Each `Task` embeds two list nodes, so it can be a member of two lists at once. Walking the “ready” list yields pointers to `ready` members; `container_of(pos, struct Task, ready)` subtracts 40 to get back to the `Task`. The whole program was verified clean under ASan and UBSan.

Intrusive vs non-intrusive

```
NON-INTRUSIVE (std::list style)      INTRUSIVE (kernel style)

┌──────┐   ┌──────┐                  ┌──────────┐   ┌──────────┐
│ node │──▶│ node │                  │ Task     │──▶│ Task     │
│  ●   │   │  ●   │                  │ name,prio│   │ name,prio│
└──┼───┘   └──┼───┘                  │ [next,prev]  │ [next,prev]
```

```
  ▼          ▼                    └──────────┘    └──────────┘
[Task]     [Task]                  ONE allocation each
2 allocations per element
```

	Intrusive	Non-intrusive
Allocations per element	1	2
Cache locality	node adjacent to data	two hops
Membership in N lists	trivial — N embedded nodes	needs N node allocations
Removal given only the object	O(1)	O(n) search
Type genericity	list code is fully generic	needs `void *` or macros
Coupling	the type must know about lists	none

The O(1) removal is worth highlighting: given a `Task`, you can unlink it from any list it is on immediately, because its `next` and `prev` pointers are right there inside it — no search needed.

This is why the Linux kernel uses intrusive structures everywhere — `struct list_head`, `struct hlist_node`, `struct rb_node` — and why they also appear in FreeBSD (`sys/queue.h`), Redis, and most high-performance C code.

> ⚠ **`container_of` caveats:** by a strict reading of the standard it is technically undefined behaviour (it performs pointer arithmetic outside the bounds of the member object), but it is universally supported and the kernel depends on it. Always use `offsetof`, never a hand-computed constant. And never apply `container_of` with the wrong type — nothing checks it for you.

A safer variant (GCC/Clang) type-checks the member pointer:

```c
#define container_of(ptr, type, member) __extension__({         \
    const __typeof__(((type *)0)->member) *__mptr = (ptr);      \
    (type *)((char *)__mptr - offsetof(type, member)); })
```

The first line declares a temporary of exactly the member's type and initializes it from `ptr`; if `ptr` has an incompatible type, the compiler warns. (`((type *)0)->member` appears only inside `__typeof__`, which does not evaluate it, so no null pointer is ever dereferenced.)

Module 11 Rules

#	Rule
1	Struct assignment copies arrays, but is shallow for pointers
2	Never `memcmp`, hash, or write structs byte-wise — padding is indeterminate
3	Declare members in decreasing alignment order — often halves the size

#	Rule
4	Avoid `packed`; use explicit `memcpy` field extraction for wire formats
5	Layout is performance: 7× measured from cache footprint alone
6	≤ 16 bytes by value, > 64 bytes by `const` pointer
7	Bitfields never for hardware or protocols; always spell out `unsigned/signed`
8	Unions need a tag; set tag and payload together via a constructor
9	Union punning is legal in C, UB in C++ — use `memcpy` if portability matters
10	Enums have no scoping — prefix every constant
11	Use the COUNT idiom + `_Static_assert` + `-Werror=switch`
12	Never `typedef` a pointer
13	Opaque structs for every non-trivial type you expose
14	Intrusive containers where allocation count and locality matter

Standing build command:

```
gcc -std=c17 -Wall -Wextra -Wpedantic -Wshadow -Wconversion \
    -Wpadded -Waddress-of-packed-member -Wcast-align=strict \
    -Werror=switch -Werror=vla -Werror=return-type \
    -O1 -g -D_FORTIFY_SOURCE=3 -fsanitize=address,undefined prog.c -o prog
```

`-Wpadded` reports every padding hole in every structure. It is noisy, so you will not want it on permanently — but run it once on your hot structures.

Practice

1. **Layout analysis.** Write a `FIELD()` macro using `offsetof` and `_Alignof`, and use it to dump the layout of five structures from a real codebase (or invent them). For each, compute the wasted bytes and reorder the members for minimum size. Verify with `sizeof`. Then install `pahole` and compare its output with yours.

2. **The `memcmp` trap.** Build a structure with padding, `memset` two instances to different byte values, set identical member values, and prove that `memcmp` disagrees. Then write a correct `equal()` and a correct `hash()`.

3. **Cache benchmark.** Reproduce the AoS/SoA test at 1M, 4M, and 16M elements. Plot the speedup against your L1/L2/L3 cache sizes (from `lscpu`). Find the array size at which the two approaches converge, and explain why.

4. **Hot/cold split.** Take a 200-byte structure of which only 3 fields are read in a hot loop. Split it into hot and cold parts, benchmark before and after, and report the speedup.

5. **Tagged-union interpreter.** Build a `Value` type supporting int, double, string, and array. Write constructors, `value_print`, `value_free` (recursive for arrays), and `value_clone`. Verify with ASan + LSan that a 10,000-node tree round-trips cleanly.

6. **Enum table discipline.** Define an 8-state enum with a `_COUNT` constant, a designated-initializer name table, and a `_Static_assert`. Then add a 9th state without updating the table and confirm that the build fails. Add a `switch` without `default` and confirm that `-Werror=switch` fires.

7. **Build an opaque type.** Write `stack.h`/`stack.c` implementing an opaque `Stack` of `long`, with create, destroy, push, pop, peek, and size, automatic growth, and a magic field. Prove from a test file that `sizeof(Stack)` and member access are compile errors. Verify with ASan.

8. **Intrusive list library.** Implement `list_head` with `list_add`, `list_add_tail`, `list_del`, `list_empty`, `list_for_each_safe` (safe against deletion during iteration), and `container_of`. Then build a task structure that lives in three lists simultaneously and write a test that moves tasks between them. Verify with ASan.

9. **Reasoning, no compiler.** Given `struct S { char a; short b; char c; long d; char e; };` on x86-64, give every member's offset, the total `sizeof`, the `alignof`, and the number of padding bytes. Then give the optimal reordering and its size.

Module 11 in Summary

Structure assignment copies every member, including arrays, but only shallowly: pointers are copied, not what they point to, so owning structures need clone and destroy functions. Padding bytes are indeterminate, so structures must never be compared, hashed, or serialised as raw bytes. Pass small structures by value and large ones by `const` pointer.

Members are laid out in declaration order with padding for alignment, so ordering them by decreasing alignment can halve a structure's size. Layout drives performance through the cache — a 7× difference was measured from footprint alone — which motivates SoA layouts, hot/cold splitting, and cache-line alignment against false sharing; `packed` is rarely the answer.

Unions overlay their members in the same storage; they are safe only as tagged unions whose tag and payload are always set together. Enums are unscoped, untyped integer constants, so prefix them, pair them with a COUNT constant and a static assertion, and let `-Wswitch` check coverage; C23 finally lets you fix their underlying type. `typedef` should clarify, never hide a pointer. Opaque structures give C real encapsulation and ABI stability, and intrusive lists with `container_of` give linked structures with one allocation per object and O(1) removal.

In Module 12, we turn to the Preprocessor — macros, their pitfalls, conditional compilation, X-macros, and `_Generic`.

Module 12 — The Preprocessor

In this module:

- `#include` (`""` vs `<>`), include guards, `#pragma once`
- Object-like and function-like macros
- Macro pitfalls: multiple evaluation, missing parens, semicolons; `do { } while (0)`
- Stringification `#`, token pasting `##`, variadic macros `__VA_ARGS__`
- Conditional compilation: `#if/#ifdef/#elif/#else/#endif`, `#error`, `#warning`
- Predefined macros: `__FILE__`, `__LINE__`, `__func__`, `__DATE__`, `__STDC_VERSION__`
- X-Macros, code generation patterns
- `_Generic` (type-generic dispatch, C11)
- `#embed` (C23)
- When not to use macros

All output in this module is real GCC 13.3 output.

The preprocessor runs before the compiler proper, and it knows nothing about C. It does not know what a type is, what a function is, or what a scope is. It is a pure token-manipulation engine: it pastes files together, substitutes text for names, and includes or excludes lines according to simple conditions. That is exactly why it is so powerful — it can generate code that C itself cannot express — and exactly why it produces the most baffling error messages in the language. Module 1 showed its output with `gcc -E`; this module shows how to use it well, and when not to use it at all.

12.1 `#include`

```c
#include <stdio.h>      /* system headers: search the standard paths */
#include "myheader.h"   /* project headers: search the current dir FIRST, then the rest */
```

The two forms differ only in where the preprocessor looks. The actual search order on the machine used for this book, as reported by `gcc -v`:

```
#include "..." search starts here:
#include <...> search starts here:
 /usr/lib/gcc/x86_64-linux-gnu/13/include
 /usr/local/include
 /usr/include/x86_64-linux-gnu
 /usr/include
End of search list.
```

The quoted form first searches the directory containing the current file (the “search starts here” list for it is otherwise empty on this system), then falls back to the same list as the angle-bracket form. The `-I dir` option adds a directory to the angle-bracket path, ahead of the system directories; `-iquote dir` adds one only to the quoted path.

Convention: use `<>` for system and third-party headers, and `""` for your own. Some large projects use `<>` for everything, with `-I` pointing at the project root, so that every include path is written relative to the root — explicit and independent of where the including file lives. That is a defensible choice too.

`#include` is literal text substitution

```
stdio.h   812 lines
stdlib.h 1298 lines
string.h  370 lines
```

Every `#include <stdio.h>` pastes those 812 lines (after preprocessing) into your translation unit, and the compiler must parse all of them — for every `.c` file that includes it. In large projects, build time is dominated by parsing headers, which is why "include what you use" matters:

- **Include the minimum in headers; push includes down into `.c` files.** A header's includes are inflicted on every file that includes it.
- **Use forward declarations in headers when you only need a pointer:** `struct Widget;` instead of `#include "widget.h"`. A pointer to an incomplete type is perfectly usable in a declaration.
- **Opaque types** (Module 11, §11.8) exist partly for this reason: callers need no knowledge of your fields, and therefore none of the headers your fields depend on.

Include guards

```c
#ifndef GUARD_H
#define GUARD_H
struct Thing { int x; };
#endif /* GUARD_H */
```

The first time the header is included, `GUARD_H` is undefined, so the body is processed and `GUARD_H` gets defined. Any later inclusion in the same translation unit finds `GUARD_H` already defined and skips everything. Without a guard, a double include is a hard error:

```
error: redefinition of 'struct Other'
```

Every header needs a guard. Not because you would deliberately include it twice, but because header A includes B and C, and both B and C include D.

Naming: derive the guard from the header's path — `PROJECT_MODULE_FILE_H`. Avoid leading underscores: names like `_MYHEADER_H` (underscore followed by a capital letter) are reserved for the implementation.

`#pragma once`

```c
#pragma once
struct P1 { int z; };
```

```
compiled clean
```

`#pragma once` asks the compiler to include the file at most once per translation unit, with no macro needed.

	`#ifndef` guards	`#pragma once`
Standard C	✓	✗ (but universally supported)
Name collisions	possible	impossible
Speed	fast (compilers optimise it)	fast
Symlinks / hard links / same file via two paths	works	may fail — the compiler dedupes by file identity

The “name collisions” row refers to two different headers accidentally using the same guard macro, which makes the second one silently disappear. The last row is the reverse risk: if the same header is reachable by two different paths — through a symbolic link, or a copy made by the build system — the compiler may not realise it is the same file.

Both approaches are fine in practice. `#ifndef` guards are safer for maximum portability and for build systems that copy headers around; `#pragma once` is cleaner. Some projects use both.

12.2 Object-like and Function-like Macros

```c
#define PI 3.14159            /* object-like */
#define SQ(x) ((x) * (x))     /* function-like */
```

An *object-like* macro replaces a name with a sequence of tokens. A *function-like* macro takes arguments: each use of a parameter in the body is replaced by the text of the corresponding argument.

> ⚠ **No space before the `(` in a function-like macro definition.** `#define F (x) x` defines `F` as an *object-like* macro whose replacement text is `(x) x`.

Multi-line macros use backslash continuation — with nothing after the backslash, not even a space. A trailing space after the backslash ends the macro silently, and the next line becomes ordinary code.

12.3 The Five Macro Pitfalls

Every one of these was measured live. All five follow from one fact: a macro substitutes *text*, not values.

1. Missing parentheses around parameters

```c
#define SQ_BAD(x) x * x
SQ_BAD(2+3)   /* expands to: 2+3 * 2+3 */
```

```
SQ_BAD(2+3) = 11 (expands to 2+3*2+3 = 11)
SQ_OK(2+3)  = 25
```

The argument `2+3` is pasted in as three tokens, and multiplication binds more tightly than addition, so the result is 2 + (3 × 2) + 3.

2. Missing parentheses around the whole expansion

```
100 / SQ_BAD(5)   /* expands to: 100 / 5 * 5 */
```

```
100/SQ_BAD(5) = 100
100/SQ_OK(5)  = 4
```

Even with a simple argument, the surrounding expression can split the macro's body apart: `/` and `*` associate left to right, so this is (100 / 5) × 5.

3. Precedence inside the body

```
#define MAX_BAD(a,b) a > b ? a : b
1 + MAX_BAD(2,3)   /* expands to: 1 + 2 > 3 ? 2 : 3 */
```

```
1 + MAX_BAD(2,3) = 3
1 + MAX_OK(2,3)  = 4
```

The `+` binds more tightly than `>`, so the condition becomes `(1 + 2) > 3`, which is false, and the result is 3.

> **Rule 1:** parenthesise every parameter *and* the entire body. `#define SQ(x) ((x) * (x))`

4. Multiple evaluation — the one parentheses can't fix

```
#define SQ_OK(x) ((x) * (x))
int r = SQ_OK(f());
```

```
SQ_OK(f()) = 9, f() called 2 times (!!)
MAX_OK(a++,b++) = 3, a=2 b=4 (b incremented twice)
```

The argument *text* is substituted, so its side effects happen once for every appearance of the parameter in the body. `SQ_OK(f())` becomes `((f()) * (f()))`, calling `f` twice. `MAX_OK(a++, b++)` evaluates the larger argument twice — once in the comparison and once as the result — so `b` was incremented twice. This is the pitfall that survives correct parenthesisation, and it produces genuinely mysterious bugs: off-by-one counters, doubled I/O, corrupted iterators.

Three defences:

```
/* a. Use a static inline function (best — see §12.10) */
static inline int max_i(int a, int b) { return a > b ? a : b; }

/* b. GNU statement expression — evaluates each argument once */
#define MAX_SAFE(a,b) __extension__({ __auto_type _a=(a); __auto_type _b=(b); _a>_b?_a:_b; })
```

```
/* c. Name it so callers know */
#define MAX_EVAL_TWICE(a,b) ((a) > (b) ? (a) : (b))
```

```
MAX_MACRO(f(),3) = 5, f() called 2 times
max_fn(f(),3)    = 5, f() called 1 time
MAX_SAFE(f(),3)  = 5, f() called 1 time (GNU stmt-expr)
```

The GNU *statement expression* `({ ... })` is a block that yields a value; inside it, each argument is evaluated exactly once into a local variable. `__auto_type` gives those variables the type of the argument, whatever it is. It works in GCC and Clang, but it is not standard C.

5. Statement macros and the semicolon

```
#define SWAP_BAD(a,b) int t=a; a=b; b=t;
if (x<y) SWAP_BAD(x,y); else printf("no\n");
```

```
error: expected expression before 'int'
error: 't' undeclared
```

The macro expands to several statements, but an `if` without braces controls only one. The rest escape the `if`, and the `else` is left orphaned with no `if` to belong to — the same class of bug as Module 5's missing braces and dangling `else`.

The fix: `do { ... } while (0)`

```
#define SWAP_OK(a,b) do { int t_ = (a); (a) = (b); (b) = t_; } while (0)

if (x < y) SWAP_OK(x, y); else printf("no swap\n");   /* works */
```

```
do{}while(0) version works with if/else: x=2 y=1
```

Why it works: `do { ... } while (0)` is a *single statement* that requires a trailing semicolon, so `MACRO(a, b);` parses exactly like a function call — the body runs exactly once. A bare `{ ... }` block would not work: the `;` the caller writes after it becomes a separate empty statement, and that breaks the `else` just as before.

Note the name `t_`. Give macro-local variables a distinctive suffix so that `SWAP_OK(t, x)` does not end up declaring a `t` that shadows the caller's `t`. (If the caller's variable were called `t_`, it would still break — a limitation of macros. The widely supported `__COUNTER__` extension can generate unique names when that matters.)

12.4 Stringification, Token Pasting, Variadics

`#` — stringify

```
#define STR(x)  #x
#define XSTR(x) STR(x)   /* expand first, THEN stringify */
#define VERSION 42
```

```
STR(VERSION)  = "VERSION"   <- literal text, NOT expanded
XSTR(VERSION) = "42"        <- expanded first
STR(a + b)    = "a + b"
STR("quoted") = "quoted"    <- quotes/backslashes are escaped
```

The `#` operator turns a macro argument into a string literal containing its source text. (In the last line, `STR("quoted")` produces the literal `"\"quoted\""` — the quotation marks are escaped so the result is a valid literal — which is why printing it shows the quotes.)

The two-level idiom is essential. Normally, macro arguments are fully expanded before being substituted — but not when they are operands of `#` or `##`. So `STR(VERSION)` stringifies the name `VERSION` itself. `XSTR` adds a layer: its argument is expanded to `42` when `XSTR` substitutes it, and only then is it passed to `STR`. The same applies to `##`:

```
#define CAT(a,b)  a##b
#define XCAT(a,b) CAT(a,b)
```

`##` — token pasting

```
#define CAT(a,b) a##b
int id_var = 7;
CAT(id_, var)   /* → id_var */
```

```
CAT(id_, var) = 7 (built the identifier id_var)
```

`##` glues two tokens together into one. The result must be a valid single token: `CAT(1, +)` would produce `1+`, which is not a single token, and that is undefined behaviour.

Common uses: generating unique identifiers (`XCAT(tmp_, __LINE__)` — note the two-level form, so that `__LINE__` becomes a number first), building function names in code-generation macros, and X-Macros (§12.7).

Variadic macros

```
#define LOG(fmt, ...)     printf("[LOG] " fmt "\n", __VA_ARGS__)
#define LOG_GNU(fmt, ...) printf("[GNU] " fmt "\n", ##__VA_ARGS__)          /* GNU */
#define LOG_C23(fmt, ...) printf("[C23] " fmt "\n" __VA_OPT__(,) __VA_ARGS__)
```

```
[LOG] value = 42
[GNU] no args at all
[C23] also no args
```

A macro declared with `...` accepts any number of extra arguments, which appear in the body as `__VA_ARGS__`. Note how `"[LOG] " fmt "\n"` relies on the compiler concatenating adjacent string literals, so the prefix and newline are joined to the caller's format string.

The zero-argument problem: with plain `__VA_ARGS__`, calling the macro with no variadic arguments leaves a trailing comma — `printf("..." , )`:

```
warning: ISO C99 requires at least one argument for the "..." in a variadic macro
error: expected expression before ')' token
```

Three solutions:

- `, ##__VA_ARGS__` — a GNU extension that deletes the preceding comma when `__VA_ARGS__` is empty. Widely supported: GCC, Clang, and MSVC with `/Zc:preprocessor`.
- `__VA_OPT__(,)` — the C23 standard solution. It emits its contents (here, a comma) only when `__VA_ARGS__` is non-empty. Use this when you can.
- **Require at least one argument:** `#define LOG(...) printf("[LOG] " __VA_ARGS__)` — the format string itself goes into the variadic part, so there is always at least one argument. (The literal prefix still concatenates, because the first variadic argument is expected to be a string literal.)

Counting arguments:

```
#define COUNT_ARGS(...) (sizeof((int[]){__VA_ARGS__}) / sizeof(int))
```

```
COUNT_ARGS(1,2,3,4,5) = 5
```

This builds a compound-literal array (Module 7) from the arguments and divides its size by the size of one element. It works only for arguments convertible to `int`, and not for zero arguments.

> ⚠ **Commas inside macro arguments must be protected.** `FOO(struct {int a, b;})` passes *two* arguments to `FOO`, split at the comma, because the preprocessor only respects parentheses — not braces or brackets — when finding argument boundaries. Wrap the argument in parentheses, or use a `COMMA` macro.

12.5 Conditional Compilation

```
#if   defined(FEATURE_A) && FEATURE_A
#elif __STDC_VERSION__ >= 202000L
#else
#endif

#ifdef NAME    /* == #if defined(NAME) */
#ifndef NAME   /* == #if !defined(NAME) */
```

```
FEATURE_A: enabled FEATURE_B: disabled
standard : C11/C17
note     : undefined macros are 0 in #if
```

Conditional compilation includes or excludes blocks of source text before the compiler ever sees them. Code in an excluded block does not even need to be valid C (though it must still consist of valid preprocessing tokens).

The rules of `#if` expressions

- **Only integer constant expressions are allowed.** No `sizeof`, no floating point, no variables, no function calls — the preprocessor runs before any of those exist.

- **Undefined identifiers evaluate to 0.** So `#if FOO` is false when `FOO` is undefined, and `#if FOO == 0` is *true* when `FOO` is undefined. That asymmetry causes real bugs: a typo in a macro name silently selects the "0" branch. Prefer `#ifdef` or `defined()` when you mean "is it defined?" (GCC's `-Wundef` warns whenever an undefined identifier is used in `#if`.)
- **Arithmetic is done in `intmax_t`/`uintmax_t`.**
- **`defined(X)` and `defined X` both work.**

`#error` and `#warning`

```
#if __STDC_VERSION__ < 201112L
# error "this code requires C11 or later"
#endif
#warning "build note: experimental feature enabled"
```

```
-- as C99 -- error: #error "this code requires C11 or later"
-- as C17 -- warning: #warning "build note: experimental feature enabled" [-Wcpp]
```

`#error` is standard; `#warning` was a widely supported extension and was standardised in C23. Use `#error` to fail fast on unsupported configurations — a single clear message at the top of the build is far better than a mysterious failure 200 errors later.

Style

Indent nested directives *after* the `#`, not before it:

```
#if A
# if B
#  define X
# endif
#endif
```

(Some old preprocessors required the `#` to be in column 1, which is where this convention came from.) Comment each `#endif` when its block is long: `#endif /* FEATURE_A */`.

Prefer a run-time `if` to `#ifdef` when you can:

```
/* BAD: code inside #ifdef isn't compiled, so it rots */
#ifdef DEBUG
    validate(x);
#endif

/* BETTER: always compiled and type-checked, optimized away when DEBUG is 0 */
if (DEBUG) validate(x);
```

In the first version, the call to `validate` is compiled only in debug builds. If someone renames `validate` or changes its parameters, release builds still pass, and the debug build is broken the next time anyone tries it. In the second version (with `DEBUG` always defined as 0 or 1), the code is compiled and type-checked in every configuration, and the optimiser deletes it entirely when `DEBUG` is 0.

Excessive `#ifdef` is a well-known maintainability disaster: with n independent options you effectively have 2^n different programs — and you test one of them.

12.6 Predefined Macros

```
__FILE__          = cond.c
__LINE__          = 40
__func__          = main (NOT a macro — a static const char[])
__DATE__ __TIME__ = Jul 28 2026 06:03:10
__STDC__          = 1
__STDC_VERSION__  = 201710L
__GNUC__          = 13.3.0
__x86_64__        = defined
__COUNTER__       = 0 1 2 (GNU: increments each use)
```

Macro	Notes
`__FILE__`	Source path as given to the compiler
`__LINE__`	Current line number
`__func__`	C99. Not a macro — a `static const char[]` inside each function, so it is unavailable to the preprocessor (you cannot stringify or paste it)
`__DATE__`, `__TIME__`	Compilation timestamp — breaks reproducible builds; avoid
`__STDC__`, `__STDC_VERSION__`	Conformance and standard version (Module 1)
`__COUNTER__`	Extension (GCC, Clang, MSVC); a unique increasing integer per use — for generating unique identifiers
`__GNUC__`, `_MSC_VER`, `__clang__`	Compiler detection
`__x86_64__`, `__aarch64__`, `_WIN32`, `__linux__`	Platform detection
`__STDC_NO_VLA__`, `__STDC_NO_THREADS__`, `__STDC_NO_ATOMICS__`	Optional-feature detection (C11)

A note on `__DATE__` and `__TIME__`: a *reproducible build* is one in which building the same source twice produces byte-identical binaries, which lets anyone verify that a distributed binary really came from the published source. Embedding the build time defeats that. (Note also that Clang defines `__GNUC__` too, for compatibility, so check `__clang__` first if you need to tell them apart.)

The classic use — a debug/logging macro:

```c
#define LOG(fmt, ...) \
    fprintf(stderr, "[%s:%d %s] " fmt "\n", __FILE__, __LINE__, __func__ __VA_OPT__(,) __VA_ARGS__)

#define CHECK(cond) do { \
        if (!(cond)) { \
            fprintf(stderr, "%s:%d: CHECK failed: %s\n", __FILE__, __LINE__, #cond); \
            abort(); \
        } \
    } while (0)
```

Because these are macros, `__FILE__` and `__LINE__` expand *at the call site*, so every message reports where the macro was used — something no function could do. And `#cond` stringifies the source text of the condition: that is how `assert` manages to print `assert(x > 0)` when it fails. This is the single most useful thing `#` does.

12.7 X-Macros — Code Generation

The problem: an enum, a table of names, a table of values, and a parser must all stay in sync. Add one entry and you must edit four places. Miss one, and you get a silent bug.

The solution: define the data once, as a list of calls to a macro `X` that is not yet defined, and expand that list several times with different definitions of `X`.

```c
/* ---- THE SINGLE SOURCE OF TRUTH ---- */
#define COLOR_LIST                  \
    X(RED,   0xFF0000, "red")       \
    X(GREEN, 0x00FF00, "green")     \
    X(BLUE,  0x0000FF, "blue")      \
    X(WHITE, 0xFFFFFF, "white")

/* 1. the enum */
#define X(name, rgb, str) COLOR_##name,
typedef enum { COLOR_LIST COLOR_COUNT } Color;
#undef X

/* 2. the name table */
#define X(name, rgb, str) [COLOR_##name] = str,
static const char *const color_names[] = { COLOR_LIST };
#undef X

/* 3. the RGB table */
#define X(name, rgb, str) [COLOR_##name] = rgb,
static const unsigned color_rgb[] = { COLOR_LIST };
#undef X

/* 4. a parser */
#define X(name, rgb, str) if (strcmp(s, str) == 0) return COLOR_##name;
static int color_parse(const char *s) { COLOR_LIST return -1; }
#undef X

_Static_assert(sizeof color_names / sizeof *color_names == COLOR_COUNT, "out of sync");
```

```
generated from ONE list (4 colors):
  red   = #FF0000 (enum value 0)
  green = #00FF00 (enum value 1)
  blue  = #0000FF (enum value 2)
  white = #FFFFFF (enum value 3)

parse("blue")   -> 2 (blue)
parse("purple") -> -1
```

Each time `COLOR_LIST` is expanded, every `X(...)` in it is replaced according to the definition of `X` in force at that moment. The first definition produces enumerators, the second produces designated initializers for the name table, the third for the RGB table, and the fourth produces a chain of `if` statements.

What the preprocessor actually produced:

```c
typedef enum { COLOR_RED, COLOR_GREEN, COLOR_BLUE, COLOR_WHITE, COLOR_COUNT } Color;
static const char *const color_names[] = { [COLOR_RED] = "red", [COLOR_GREEN] = "green",
                                           [COLOR_BLUE] = "blue", [COLOR_WHITE] = "white", };
```

Add one `X(...)` line, and the enum, both tables, and the parser all update together. Combined with `_Static_assert` and `-Werror=switch`, it becomes impossible for them to drift apart.

Where X-Macros shine

- Enum ↔ string conversion (in both directions)
- Error-code tables
- Serialisation: generating pack/unpack code for every field of a structure
- Register maps in embedded code
- Opcode dispatch tables for virtual machines
- Test registries
- Command tables for CLIs

The costs

- **Debugging is painful** — errors point at the macro definition rather than at a readable line. Use `gcc -E` to see the expansion (Module 1).
- **The code looks unusual.** Comment the pattern where you use it.
- **IDE tooling struggles** with the generated names.

For large-scale generation, a real code generator — a Python script plus a build rule — is often clearer. But X-Macros need no build-system changes, and because the list lives in the source, it can never get out of sync with it.

12.8 `_Generic` — Type-Based Dispatch (C11)

```
_Generic(controlling_expression,
         type1: expr1,
         type2: expr2,
         default: expr_default)
```

`_Generic` selects one of several expressions according to the *type* of its controlling expression. The selection is made entirely at compile time.

```
#define type_name(x) _Generic((x),           \
    char: "char", int: "int", long: "long",  \
    float: "float", double: "double",        \
    char *: "char *", default: "unknown")
```

```
1     -> int
1.0   -> double
1.0f  -> float
'c'   -> int      <- character constants are int in C
"lit" -> char *
```

Note that `'c'` reports `int` — confirming Module 2's point that character constants have type `int` in C — and that the string literal reports `char *`, because the array decays to a pointer before selection.

Type-generic function dispatch

```
#define my_abs(x) _Generic((x), \
    int: abs, long: labs, float: fabsf, double: fabs, default: fabs)(x)
```

```
#define print(x) _Generic((x), \
    int: print_int, double: print_dbl, char*: print_str, const char*: print_str)(x)
```

```
int: 1
double: 2.5
string: hi
my_abs(-5)=5 my_abs(-5.5)=5.5
```

Here `_Generic` selects a *function*, and the `(x)` after it calls the selected function with the argument. This is exactly how `<tgmath.h>` works — its `sqrt(x)` chooses `sqrtf`, `sqrt`, or `sqrtl` according to the argument's type — and it is the closest C gets to function overloading.

The rules that trip people up

1. **The controlling expression is not evaluated** — only its type is used, as with `sizeof`.
2. **Non-selected branches must still be valid and type-check.** The first version of the `my_abs` example failed to compile because `abs` was not declared (its header had not been included), even though the `int` branch was not the one being selected:

   ```
   error: 'abs' undeclared (first use in this function)
   ```

3. **Arrays and functions decay before selection** — you cannot match `int[10]`, only `int *`.
4. **Top-level qualifiers are stripped** — you cannot distinguish `int` from `const int`. (But `const char *` and `char *` differ in what they point to, not in top-level qualification, so they are different types and can both appear.)
5. **Every listed type must be distinct;** duplicates are an error. Beware of typedefs: on a platform where `int64_t` is `long`, listing both `long` and `int64_t` is a duplicate.
6. **`_Generic` is an expression, not a statement.**

Real uses

```
/* Type-safe max without multiple-evaluation risk (GNU) */
#define max(a,b) __extension__({ __auto_type _a=(a); __auto_type _b=(b); _a>_b?_a:_b; })

/* Format-string selection */
#define FMT(x) _Generic((x), int:"%d", long:"%ld", double:"%g", char*:"%s")

/* Compile-time type assertions */
#define IS_POINTER(x) _Generic((x), default:0, int*:1, char*:1, void*:1)
```

```
IS_POINTER(i)=0 IS_POINTER(s)=1 IS_POINTER(&i)=1
```

`FMT` lets a generic printing macro choose the right `printf` conversion automatically — removing a whole class of format-string mismatches (Module 2). `IS_POINTER` works only for the pointer types it lists; it is a demonstration of the technique rather than a complete test.

12.9 `#embed` (C23)

`#embed` includes a file's *bytes* directly as a comma-separated initializer list — no `xxd -i`, no extra build step:

```
static const unsigned char logo[] = {
    #embed "logo.png"
};
static const char shader[] = {
    #embed "shader.glsl" suffix(, 0)   /* NUL-terminate */
};

#if __has_embed("config.bin")
    /* ... */
#endif
```

Optional parameters control the result: `limit(n)` embeds at most *n* bytes, `prefix(...)` and `suffix(...)` add tokens before and after the data (the `suffix(, 0)` above appends a terminating zero so the shader can be used as a string), and `if_empty(...)` supplies replacement tokens when the file is empty.

Availability check on this machine:

```
$ gcc --version
gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0

error: invalid preprocessing directive #embed
```

GCC gained `#embed` in version 15, and Clang in version 19. It is genuinely useful — for embedding assets, firmware blobs, default configurations, and test fixtures — and compiles dramatically faster than the giant-initializer-list workarounds generated by tools like `xxd -i`. But you will need a recent toolchain. Guard its use with `#if defined(__has_embed)` (to detect support) and `__has_embed(...)` (to check that a particular file can be embedded).

12.10 When Not to Use Macros

`static inline` beats a macro almost every time

```
MAX_MACRO(f(),3) = 5, f() called 2 times   ← bug
max_fn(f(),3)    = 5, f() called 1 time    ← correct
```

And the generated code is identical at `-O2`:

```
use_macro:              use_inline:
    cmp    esi, edi         cmp    edi, esi
    mov    eax, edi         mov    eax, esi
    cmovge eax, esi         cmovge eax, edi
    ret                     ret
```

The same instructions. One version is correct; the other has a latent bug waiting for someone to pass an argument with a side effect.

	Macro	`static inline`
Type checking	✗ none	✓ full
Multiple evaluation	✗ risk	✓ impossible
Debugger visibility	✗ invisible	✓ steppable, breakpointable
Error messages	✗ point at the expansion	✓ point at the call
Scope	✗ everything after `#define`	✓ normal C scoping
Address-of	✗	✓
Recursion	✗	✓
Generated code at `-O2`	same	same

Use a macro only when you genuinely can't use a function

Legitimate	Why
Needs `__FILE__`/`__LINE__` at the call site	`assert`, logging, `CHECK`
Needs the source text (`#cond`)	assertions, test frameworks
Conditional compilation	`#ifdef` blocks
Type-generic code	X-Macros, `_Generic` wrappers, container templates
Compile-time constants	`#define MAX_USERS 1024` (or better: `enum`)
Token construction	building identifiers with `##`

Constants: prefer `enum` or `static const`

```c
#define MAX 1024                /* no type, no scope, no debugger symbol */
enum { MAX = 1024 };            /* typed int, scoped, constant expression */
static const int MAX = 1024;    /* typed, scoped — but NOT a constant expression in C */
constexpr int MAX = 1024;       /* C23: typed, scoped, AND a constant expression */
```

In C (unlike C++), a `const int` is not a constant expression — it cannot size a (non-VLA) array or serve as a `case` label (Module 2). An enumeration constant can. C23's `constexpr` finally gives you all of it in one declaration.

Macro hygiene checklist

```c
#define MACRO_NAME(a, b) ((a) > (b) ? (a) : (b))   /* 1. UPPER_CASE names
                                                      2. parens on every parameter and the whole body
*/
#define STMT(x) do { ... } while (0)               /* 3. do-while for statement macros */
                                                   /* 4. name locals with a trailing _ */
                                                   /* 5. document multiple-evaluation risk */
#undef TEMP_MACRO                                  /* 6. #undef helper macros */
                                                   /* 7. never define macros in public headers
                                                         with generic names (min, max, DEBUG) */
```

Upper-case names act as a warning to readers: "this is a macro, and the usual rules may not apply."

That last rule matters: `#define max(a,b) ...` in a public header silently breaks any code that includes it and has a variable, a structure member, or a function called `max`, because the preprocessor replaces every `max(` it sees. The Windows headers' `min` and `max` macros are a legendary source of this kind of pain (and the reason `NOMINMAX` exists). Prefix the macros in your public headers: `MYLIB_MAX`.

Module 12 Rules

#	Rule
1	Every header gets a guard; `#pragma once` is fine but non-standard
2	Minimise includes in headers; forward-declare where possible
3	Parenthesise every parameter and the whole macro body
4	Assume every argument is evaluated multiple times — document or avoid
5	`do { } while (0)` for every statement macro
6	Two-level `STR`/`CAT` when the argument is itself a macro
7	`__VA_OPT__(,)` (C23) or `, ##__VA_ARGS__` (GNU) for zero-argument variadics
8	Undefined identifiers are 0 in `#if` — use `#ifdef` when you mean "defined"
9	Prefer run-time `if (DEBUG)` to `#ifdef DEBUG` so all configurations stay compiled
10	X-Macros + `_Static_assert` + `-Werror=switch` for tables that must not drift
11	`_Generic`'s unselected branches must still compile
12	`static inline` unless you need `__FILE__`, `#x`, `##`, or conditional compilation
13	`enum` (or C23 `constexpr`) for constants, not `#define`
14	Prefix macros in public headers
15	`gcc -E` is your debugger for macro problems

Practice

1. **Header hygiene.** Take a three-file project and remove every unnecessary `#include` from the headers, replacing them with forward declarations. Measure the build time before and after with `time make -B`. Then run `gcc -E file.c | wc -l` before and after.

2. **Reproduce all five pitfalls.** Write one program demonstrating each, predict each output before running it, then fix each one. For pitfall 4, find a fix that works without GNU extensions.

3. **Build a logging framework.** Implement `LOG_ERROR`, `LOG_WARN`, `LOG_INFO`, and `LOG_DEBUG` with `__FILE__`, `__LINE__`, `__func__`, a compile-time level filter, and `__VA_OPT__`. Add `__attribute__((format(printf, ...)))` to the underlying function (Module 6). Verify that a format mismatch is caught at compile time.

4. **`CHECK` and `assert`.** Implement `CHECK(cond)`, which prints the source text of the condition on failure. Then implement `CHECK_EQ(a, b)`, which prints both expressions and both values — making sure each argument is evaluated exactly once.

5. **X-Macro suite.** Define an error-code list with `X(name, number, message)` and generate an enum, a `const char *err_string(int)`, an `int err_from_string(const char *)`, and a `_Static_assert` on the count. Then add one entry and confirm that everything updates.

6. **X-Macro serialisation.** Define a structure's fields as an X list and generate the structure definition, a `to_json`, a `from_json` skeleton, and a `print`. Verify round-tripping.

7. **`_Generic` toolkit.** Build `type_name(x)`, `FMT(x)` (format-string selection), and a `PRINT(x)` that works for `int`, `long`, `double`, `char *`, and pointers. Then discover a limitation: try to distinguish `int[10]` from `int *`, and explain why you can't.

8. **Macro vs inline codegen.** Write five common macros (`MIN`, `MAX`, `ABS`, `CLAMP`, `SWAP`) and their `static inline` equivalents. Diff the `-O2` assembly for each. Then find a case where the macro is genuinely necessary.

9. **Expansion archaeology.** Take the most complex macro you can find in a real header (`assert.h`, `container_of`, `offsetof`, a kernel macro) and run `gcc -E` on a file that uses it. Explain the expansion line by line.

10. **`#embed`, if you can.** If you have GCC 15+ or Clang 19+, embed a binary file and compare compile time against the `xxd -i` approach for a 1 MB file.

Module 12 in Summary

The preprocessor is a text engine that runs before the compiler and knows nothing about C. `#include` pastes whole files into each translation unit — so keep headers lean, forward-declare, and guard every header against repeated inclusion.

Macros substitute text, not values: parenthesise every parameter and the whole body, wrap statement macros in `do { } while (0)`, and remember that no amount of parentheses prevents an argument from being evaluated once per use. `#` turns arguments into strings, `##` pastes tokens, and both need a second macro layer to expand their arguments first; `__VA_OPT__` handles empty variadic lists. In `#if`, undefined names silently become 0, and every `#ifdef` doubles the number of programs you ship — so prefer run-time `if` on a constant where you can.

The preprocessor earns its place with the things only it can do: call-site `__FILE__` and `__LINE__`, stringified conditions, X-Macros that generate enums, tables, and parsers from a single list, and `_Generic` for type-based dispatch. C23's `#embed` brings binary data straight into the source. For everything else — small functions and constants — `static inline`, `enum`, and `constexpr` do the same job with type checking, scoping, and debugger support.

In Module 13, we turn to Input and Output — streams, buffering, formatted and binary I/O, and error handling.

Module 13 — Input / Output

In this module:

- `stdio.h`: the `printf`/`scanf` family, full format-specifier knowledge, width/precision/flags
- Why `scanf` is a trap; the `fgets` + `strtol` pattern
- `FILE *`, `fopen` modes, text vs binary mode
- `fread`, `fwrite`, `fseek`, `ftell`, `rewind`, `fflush`
- Buffering: full/line/none, `setvbuf`
- `EOF`, `feof` misuse, `ferror`, `clearerr`
- `stdin`/`stdout`/`stderr`, redirection, pipes
- Low-level POSIX I/O: `open`/`read`/`write`/`close`/`lseek`, file descriptors, `mmap`

All output in this module is real, from GCC 13.3 and glibc 2.39.

Almost every program reads input and produces output, and C gives you two layers for doing so. The upper layer is ISO C's `<stdio.h>`: portable, buffered, and formatted, built around the `FILE *` stream. The lower layer is the operating system's own interface — on Unix, POSIX file descriptors and system calls such as `read` and `write`. This module covers both, and how they relate.

Two topics cause more confusion than the rest of stdio combined: buffering (§13.6), which explains why output appears out of order or vanishes entirely, and `scanf` (§13.2), which looks like the natural way to read input and is a trap in almost every respect.

13.1 The `printf` Family

```
int printf  (const char *fmt, ...);                    /* to stdout */
int fprintf (FILE *fp, const char *fmt, ...);          /* to a stream */
int sprintf (char *buf, const char *fmt, ...);         /*   BANNED — no bound */
int snprintf(char *buf, size_t n, const char *fmt, ...); /*   use this */
int vprintf / vfprintf / vsnprintf(..., va_list ap);     /* for wrappers (Module 6) */
```

The return value is the number of characters written (not counting the terminating NUL), or a negative value on error. For `snprintf`, it is the number that *would* have been written — the basis of Module 8's truncation check.

The specifier anatomy

```
%[flags][width][.precision][length]conversion
```

Every conversion specification starts with `%` and ends with a conversion letter; everything in between is optional. The *flags* adjust formatting, the *width* sets a minimum field width, the *precision* has a meaning that depends on the conversion (digits after the decimal point, maximum characters of a string, minimum digits of an integer), and the *length modifier* states the exact type of the argument.

Conversions

Conversion	Meaning	Conversion	Meaning
`%d` `%i`	signed decimal	`%f` `%F`	fixed-point
`%u`	unsigned decimal	`%e` `%E`	scientific
`%o`	octal	`%g` `%G`	shorter of `%e`/`%f`
`%x` `%X`	hex	`%a` `%A`	hex float (exact)
`%c`	character	`%s`	string
`%p`	pointer (cast to `void *`!)	`%%`	a literal `%`
`%n`	writes a count — security hazard, never use		

`%p` expects a `void *` specifically, so other pointer types should be cast: `printf("%p", (void *)ptr)`.

Length modifiers — get these wrong and it's UB

Modifier	Type
`hh` / `h`	`char` / `short` (promoted to `int`, so the modifier tells `printf` to convert back before printing)
`l` / `ll`	`long` / `long long`
`z`	`size_t`
`t`	`ptrdiff_t`
`j`	`intmax_t`
`L`	`long double`

```
%d   -42   %i   -42
%u   42    %o   52    %x ff   %X FF
%hhd -1    %hd  -1    %ld -1  %lld -1
%zu  4     %td  -5    %jd -9
PRId64 -1  PRIx32 abcd
```

Because `printf` is variadic, it has no way to know the real types of its arguments: it trusts the format string completely. A mismatch makes it read the wrong number of bytes, or read from the wrong register (Module 6), which is undefined behaviour.

- **Use `%zu` for `size_t`, `%td` for `ptrdiff_t`, and the `<inttypes.h>` macros for fixed-width types** (Module 2). `printf("%d", sizeof x)` is undefined behaviour on 64-bit platforms, where `size_t` is 8 bytes and `int` is 4.
- **There is no need for `%lf` in `printf`.** A `float` argument is promoted to `double` by the default argument promotions (Module 6), so `%f` handles both. (C99 made `%lf` legal in `printf` as a synonym, but it adds nothing.) In `scanf`, it is different: `%f` means `float *` and `%lf` means `double *`, because pointers are not promoted.

Floats

```
%f 3.141590   %e 3.141590e+04
%g 31415.9    %g 3.1415e-05   (shortest of %e/%f)
%a 0x1.999999999999ap-4       (hex float — exact, round-trips)
%.17g 0.10000000000000001     (enough digits to round-trip a double)
```

`%f` prints a fixed six digits after the point by default, and `%e` uses scientific notation. `%g` drops trailing zeros and picks whichever representation is shorter — usually what you want for output meant for humans.

The last two lines matter for debugging. `%.17g` prints enough decimal digits that reading the text back reproduces exactly the same `double` — and it reveals that 0.1 is really 0.10000000000000001 (Module 2). `%a` prints the value in hexadecimal floating point, which is exact and compact, because it shows the actual binary mantissa and exponent. Use `%a` when chasing floating-point bugs: it shows you the true value, with nothing rounded away.

Flags, width, precision

```
|%10d|    |        42|
|%-10d|   |42        |      left-justify
|%010d|   |0000000042|      zero-pad
|%+d|     |+42| |-42|       always show sign
|% d|     | 42|              space for positive
|%#x|     |0xff| |%#o| |010|   alternate form
|%.3f|    |3.142|
|%.3s|    |abc|             truncate string
|%10.3f|  |     3.142|
|%*d|     |      42|        width from an argument
|%-*.*s|  |abcd      |      both from arguments
```

A `*` in place of the width or precision takes the value from an `int` argument, placed before the value being printed — useful for table columns whose widths are computed at run time.

`%.*s` is the safest way to print a buffer that is not NUL-terminated:

```
printf("%.*s", (int)len, ptr);   /* prints exactly len bytes */
```

With a precision, `%s` stops after that many characters and never looks further — so it works on a slice of a larger buffer, or on data received from a network with an explicit length.

Security

> ⚠ **Never pass user input as a format string.**

```
printf(user_input);          /* FORMAT STRING VULNERABILITY */
printf("%s", user_input);    /* correct */
```

If the input contains conversion specifications, `printf` obeys them. `%s` with no matching argument makes `printf` fetch a “pointer” from wherever the next argument would have been and read memory through it; `%x` leaks stack contents; and `%n` *writes* the number of

characters printed so far through a pointer argument — with a crafted format string, that becomes an arbitrary memory write. This is a classic remote-code-execution class: CVE-2000-0573 (in the wu-ftpd FTP server) and hundreds since.

Compile with `-Wformat=2`, which includes `-Wformat-security` and `-Wformat-nonliteral`, and put `__attribute__((format(printf, m, n)))` on every wrapper function (Module 6).

13.2 Why `scanf` Is a Trap

`scanf` reads formatted input — the mirror image of `printf` — and at first sight it seems the natural way to read numbers from the user. It has four separate traps.

Trap 1 — the newline stays in the buffer

```
scanf("%d", &n);
fgets(name, sizeof name, stdin);
```

```
input "42\nhello\n":
  got n=42
  fgets got: "\n"   <- the leftover newline!
```

`%d` skips any leading whitespace, reads digits, and stops at the first non-digit — which is the newline the user typed. That newline stays in the input buffer, so the next `fgets` (or `%c`) reads it immediately and returns what looks like an empty line. Beginners "fix" this with an extra `getchar()`, or `while (getchar() != '\n')`, which mostly works and occasionally doesn't.

Trap 2 — on failure, the input is not consumed

```
now reading %d from "abc":
  scanf returned 0 (0 = matched nothing)
  trying again returns 0 - SAME input still there -> infinite loop
```

When `scanf` cannot match the input, it leaves the offending characters in the buffer. This is the classic "my program hung" bug: a retry loop around `scanf` spins forever, because the letters `abc` never leave the buffer. You must drain them manually.

Trap 3 — `%s` has no bound

```
char b[8];
scanf("%s", b);
```

```
ERROR: AddressSanitizer: stack-buffer-overflow
  [32, 40) 'b' <== Memory access at offset 40 overflows this variable
```

`scanf("%s")` is as dangerous as `gets` (Module 8): an unbounded write into a fixed buffer. The bounded form works:

```
scanf("%7s", b);   /* 7 = sizeof b - 1; the NUL is added on top */
```

```
got: AAAAAAA
```

But the width must be written into the format string as a literal. Unlike `printf`, `scanf` has no way to take the width from an argument — in `scanf`, `*` means "read but discard" (see below) — so the format cannot track `sizeof buf` automatically. Every change to the buffer's size requires editing the format string too, and forgetting to is a buffer overflow.

Trap 4 — no error detail, no overflow detection

`scanf` returns the number of items successfully assigned, or `EOF`. It cannot tell you *why* something failed, and `%d` applied to `"99999999999999999999"` is undefined behaviour — the same problem as `atoi` (Module 8).

The other subtleties

- **Whitespace in the format matches any amount of whitespace in the input,** including none.
- **Other characters in the format must match the input literally,** or the scan stops.
- **`%c` does not skip leading whitespace,** unlike every other conversion. Use `" %c"`, with a leading space, to skip it.
- **`%[^\n]`** (a *scanset*) reads up to a newline — but fails and returns 0 on an empty line.
- **Suppression:** `%*d` reads an integer and discards it.

> **The verdict:** `scanf` on `stdin` is unusable for robust programs. `sscanf` on a string you have already read is defensible — the input is bounded and cannot get stuck — but even then you must bound every `%s` and check the return value.

13.3 The Correct Input Pattern: `fgets` + `strtol`

Read a whole line, then parse it. That separates the two concerns — getting the input and interpreting it — and makes each one checkable.

```c
static int read_line(char *buf, size_t n, FILE *fp)
{
    if (!fgets(buf, n, fp)) return -1;                          /* EOF or error */
    size_t len = strcspn(buf, "\n");
    if (buf[len] != '\n') {                                     /* line was truncated */
        int c;
        while ((c = fgetc(fp)) != '\n' && c != EOF) { }      /* drain the rest */
        return -2;
    }
    buf[len] = '\0';                                            /* strip the newline */
    return 0;
}
```

Combined with the `parse_long` from Module 8:

```
"42"    -> 42
"abc"   -> not a number
"99xyz" -> trailing garbage
```

```
[line too long, discarded]
"-7"    -> -7
```

Every failure mode is detected and reported distinctly — and a bad line never blocks the next one, because each line is fully consumed. That is exactly what you cannot get from `scanf`.

Key points:

- **`fgets` keeps the newline.** `buf[strcspn(buf, "\n")] = '\0'` is the idiomatic way to strip it.
- **A missing newline means the line was truncated** (or that the file's last line had no newline). You must drain the remainder, or the next read gets the tail end of the long line.
- **`fgets` returns `NULL` on end-of-file or error** — distinguish the two with `feof`/`ferror` (§13.7).
- **POSIX `getline()` allocates and grows the buffer for you,** handling lines of any length:

  ```
  char *line = NULL; size_t cap = 0;
  ssize_t len;
  while ((len = getline(&line, &cap, stdin)) != -1) { ... }
  free(line);
  ```

  It is not ISO C, but it is available on Linux, macOS, and the BSDs, and it is much nicer to use. ⚠ Note that `getline` also keeps the newline, and returns −1 at end of file.

13.4 `FILE *` and `fopen` Modes

```
FILE *fp = fopen(path, mode);
if (!fp) { perror(path); return -1; }   /* ALWAYS check */
```

`fopen` returns `NULL` on failure — missing file, no permission, too many open files — and sets `errno`. `perror` prints your message followed by a description of `errno`, such as `data.txt: No such file or directory`.

Mode	Read	Write	Truncate	Create	Position
`"r"`	✓				start
`"w"`		✓	✓	✓	start
`"a"`		✓		✓	end, always
`"r+"`	✓	✓			start
`"w+"`	✓	✓	✓	✓	start
`"a+"`	✓	✓		✓	reads anywhere; writes always append

Add `b` for binary (`"rb"`, `"wb"`), and `x` (C11) to make the open fail if the file already exists. `"wx"` is the atomic "create a new file" mode you want in order to avoid TOCTOU races — *time-of-check to time-of-use* bugs, in which a program checks that a file does not exist and then creates it, and an attacker slips in between the two steps (for example, with a symbolic link to a sensitive file).

Text vs binary mode

On Windows, text mode translates `\n` to `\r\n` on output and back on input, and historically treated Ctrl-Z as end of file. On Unix there is no difference between the two modes at all.

> **Always use `"rb"`/`"wb"` for binary data.** Forgetting the `b` is a Windows-only corruption bug that Unix developers never see: every byte `0x0A` in an image or archive gets a `0x0D` inserted before it. There is no downside to using `b` on Unix.

Also: with an update mode (`"r+"`, `"w+"`, `"a+"`), you must call `fflush`, `fseek`, `fsetpos`, or `rewind` when switching between reading and writing, or the behaviour is undefined. The stream has a single buffer, which cannot be holding read-ahead data and pending writes at the same time.

Cleanup

`fclose` flushes any buffered output and releases the `FILE *`. Check its return value — a write error (a full disk, a failed network filesystem) may only surface when the final buffer is flushed:

```c
if (fclose(fp) != 0) { perror("fclose"); /* data may not have been written */ }
```

For *durability* — surviving a crash or power failure — you need more than `fclose`. `fflush(fp)` pushes stdio's buffer to the operating system, and `fsync(fileno(fp))` then asks the OS to push its own buffers to the physical disk.

13.5 `fread`, `fwrite`, `fseek`, `ftell`, `rewind`

```c
size_t fread (void *ptr, size_t size, size_t nmemb, FILE *fp);
size_t fwrite(const void *ptr, size_t size, size_t nmemb, FILE *fp);
```

These transfer `nmemb` items of `size` bytes each. Both return the number of *items* (not bytes) transferred. A short count means end of file or an error — check which with `feof`/`ferror`.

```
fwrite wrote 3 of 3 records (24 bytes each)
file size = 72 bytes (3 * 24 = 72)

random access with fseek:
  record[1]: id=2 name=bob score=87.25 (ftell now 48)

fread read 3 records after rewind:
  1 alice 95.5
  2 bob 87.25
```

```
  3 carol 99
round-trip identical? 1
```

Because every record has the same size, record *i* begins at byte `i * 24`, so `fseek` can jump straight to any one of them — after reading record 1, the position is 48, the start of record 2.

Positioning

```c
fseek(fp, offset, SEEK_SET);   /* from the start */
fseek(fp, offset, SEEK_CUR);   /* from the current position */
fseek(fp, 0,      SEEK_END);   /* to the end */
long pos = ftell(fp);          /* current position */
rewind(fp);                    /* = fseek(fp,0,SEEK_SET) + clearerr() */
```

⚠ `ftell` returns a `long` — which is 32 bits on Windows, so it breaks for files larger than 2 GB. Use `fgetpos`/`fsetpos` (portable, with an opaque `fpos_t`) or POSIX `ftello`/`fseeko`, which use `off_t`, together with `-D_FILE_OFFSET_BITS=64` on 32-bit systems.

⚠ `fseek` on a text-mode stream is only well defined with an offset previously obtained from `ftell` (or 0 with `SEEK_SET`), because of the newline translation described above.

⚠ Using `fseek(fp, 0, SEEK_END)` followed by `ftell` to get a file's size is not portable for text streams, and it does not work at all on pipes. Use `stat`/`fstat`.

Writing raw structs is not portable

```
WARNING: writing raw structs is NOT portable:
  - padding bytes leak stack data (Module 11)
  - endianness differs between machines
  - sizeof/alignment differ between ABIs
  -> serialize field by field, in a fixed byte order
```

`fwrite(&mystruct, sizeof mystruct, 1, fp)` is fine for a scratch file that your own program reads back on the same machine. For any file that another program or another machine will read, serialise each field explicitly, in a defined byte order:

```c
static void put_u32le(FILE *fp, uint32_t v)
{ fputc(v & 0xFF, fp); fputc((v>>8)&0xFF, fp); fputc((v>>16)&0xFF, fp); fputc((v>>24)&0xFF, fp); }
```

This writes the four bytes of `v` least significant first ("little-endian"), producing identical bytes on every machine regardless of its own byte order, and with no padding.

The padding leak is a real information-disclosure CVE class: you write your structure to a network socket, and three bytes of uninitialized stack memory go along with it.

13.6 Buffering ⚠

A system call is expensive compared with an ordinary function call. So stdio collects output in a buffer in your process's memory and hands it to the operating system in large chunks. The buffering mode depends on what the stream is connected to — which is why the same program behaves differently in a terminal and in a pipe.

Stream	Default mode
`stdout` to a terminal	Line buffered — flushes on `\n`
`stdout` to a pipe or file	Fully buffered — flushes when the buffer fills
`stderr`	Unbuffered — always immediate

The reasoning: when a human is watching a terminal, each line should appear as soon as it is complete. When output goes to a file or another program, throughput matters more, so stdio waits until it has a full buffer (typically 4 KB or more). And error messages should never be delayed.

The ordering surprise

```
printf("this printf may sit in the buffer...\n");
fprintf(stderr, "this fprintf(stderr) appears IMMEDIATELY\n");
write(1, "this raw write(2) bypasses stdio entirely\n", 41);
printf("...and only now does the earlier printf flush\n");
```

Piped to `cat`:

```
this fprintf(stderr) appears IMMEDIATELY
this raw write(2) bypasses stdio entirelystdout is a pipe/file -> FULL buffered
stderr is ALWAYS unbuffered
this printf may sit in the buffer...
...and only now does the earlier printf flush
```

The output is completely out of source order. The `stderr` message and the raw `write` went out immediately. The `printf` calls — including the very first one — all sat in `stdout`'s buffer and appeared together at the end, when the buffer was flushed at exit. (Two further lines, from other diagnostic `printf`s in the test program, came out of the same buffer. And the `write` line runs into the next one because its message is exactly 41 characters long, so the count of 41 omitted the trailing `\n` — a small illustration of how unforgiving raw byte counts are.)

Two lessons:

1. **Interleaving `stdout` and `stderr` produces scrambled logs.** Pick one stream for a given kind of output, or make `stdout` unbuffered.
2. **Never mix `printf` and `write(2)` on the same file descriptor.**

The “my output disappeared” bug

```
printf("step 1 done\n");
printf("step 2 done\n");
abort();
```

```
-- output to a pipe (buffered -> LOST): --
  (nothing above = both printfs were lost)

-- with setvbuf(stdout, NULL, _IONBF, 0): --
step 1 done
step 2 done
```

`abort()`, `_exit()`, and a segmentation fault do **not** flush stdio buffers. The process dies with the output still sitting in its memory. Your last few lines of debug output — the ones right before the crash, which are the ones you needed — are exactly the ones you lose.

This is why “the program crashes, but the `printf` just before the crash didn’t print” is so common, and why beginners wrongly conclude that the crash must be on an earlier line. (In a terminal, line buffering usually hides the problem, which is why it tends to appear only when output is redirected — in CI logs, for instance.)

Fixes:

```
setvbuf(stdout, NULL, _IONBF, 0);   /* unbuffered — do this while debugging */
setvbuf(stdout, NULL, _IOLBF, 0);   /* line buffered even when piped */
fflush(stdout);                     /* explicit flush at critical points */
fprintf(stderr, ...);               /* stderr is always unbuffered */
```

```
stdbuf -o0 ./prog                   # from the shell, no recompile
```

`exit()` and returning from `main` *do* flush and close all open streams, as part of normal termination. `_exit()` and `abort()` don’t.

`setvbuf`

```
int setvbuf(FILE *fp, char *buf, int mode, size_t size);
/* mode: _IOFBF (full), _IOLBF (line), _IONBF (none) */
```

`setvbuf` must be called after the stream is opened and before any other operation on it. Pass `NULL` for `buf` to let stdio allocate the buffer itself (and `size` as a hint for its size).

Performance note: full buffering is dramatically faster for bulk output. Unbuffered `printf` in a tight loop makes one system call per call — which is why you should unbuffer `stdout` for debugging, not in production.

13.7 `EOF`, `feof`, `ferror`, `clearerr`

The `while (!feof(fp))` bug

```
while (!feof(f)) { c = fgetc(f); ... }
```

```
iter 1: c=97 'a'
iter 2: c=98 'b'
iter 3: c=99 'c'
iter 4: c=-1 '?'   ← one iteration too many
```

On a three-byte file, this loop runs four times. `feof` does not predict the future: it reports that a read has *already* failed by hitting the end. After the third character, no read has failed yet, so `feof` returns false and the loop runs again. The fourth `fgetc` fails and returns `EOF`, and the body processes that `EOF` as though it were a character.

The correct form tests the read operation's own return value:

```
while ((c = fgetc(f)) != EOF) { ... }
while (fgets(buf, sizeof buf, fp)) { ... }
while ((n = fread(buf, 1, sizeof buf, fp)) > 0) { ... }
```

```
iter 1: 'a' iter 2: 'b' iter 3: 'c'   ← exactly 3
```

`feof`/`ferror` are for after the loop

```
while ((c = fgetc(f)) != EOF) { ... }
if (ferror(f))    { perror("read error"); }
else if (feof(f)) { /* clean end of file */ }
```

```
feof(f)=1 ferror(f)=0 -> clean end of file
```

A return of `EOF` alone does not tell you *why* the reading stopped: the end of the file, or a genuine I/O error (a disk failure, a network filesystem that disappeared). The stream records which one in two flags. Always check them after a loop that processes real data — otherwise a read error looks exactly like a short file, and your program silently processes half of its input.

`clearerr(fp)` resets both flags. You need it to keep reading a stream after a transient error, or after reaching end-of-file on a terminal (where the user pressed Ctrl-D but may type more).

`EOF` must go in an `int`

As Module 2 explained, `fgetc` and `getchar` return `int` so that `EOF` (−1) can be distinguished from the valid byte `0xFF`. `char c = getchar()` is broken both ways: an infinite loop if `char` is unsigned, and a premature exit on binary data if it is signed.

13.8 `stdin`, `stdout`, `stderr`, Redirection

Three `FILE *` streams are already open when `main` starts, backed by file descriptors 0, 1, and 2:

```
STANDARD FILE DESCRIPTORS: stdin=0 stdout=1 stderr=2
```

The conventions that make Unix tools compose

Stream	Purpose
`stdout`	Program output — the data. Redirectable, pipeable.
`stderr`	Diagnostics — errors, progress, warnings. Stays on the terminal even when `stdout` is piped.
`stdin`	Input, possibly a pipe or file

```
./prog > out.txt      # stdout to a file
./prog 2> err.txt     # stderr to a file
./prog > out 2>&1     # both to the same file
./prog 2>/dev/null    # discard errors
./prog < in.txt       # stdin from a file
a | b | c             # a's stdout → b's stdin, b's stdout → c's stdin
```

The shell sets up these redirections before your program starts, by arranging which files or pipes descriptors 0, 1, and 2 refer to. Your program needs no special code: it simply reads `stdin` and writes `stdout`, and works equally well interactively, with files, and in the middle of a pipeline.

> **Rule:** errors, warnings, and usage messages go to `stderr`; data goes to `stdout`. A tool that prints errors to `stdout` corrupts every pipeline it is used in — the error text flows into the next program as if it were data. This is why the Module 1 example wrote its `usage:` message to `stderr`.

Also, check whether `stdout` is a terminal before emitting colours or progress bars:

```
#include <unistd.h>
if (isatty(STDOUT_FILENO)) { /* colors, spinners */ }
```

Colour escape codes and spinner animations are helpful on a terminal and garbage in a file.

13.9 POSIX Low-Level I/O

Beneath stdio lies the operating system's interface. On POSIX systems, an open file is represented by a small integer, the *file descriptor*, and I/O is done with system calls:

```
#include <fcntl.h>
#include <unistd.h>
```

```
int     open(const char *path, int flags, mode_t mode);
ssize_t read(int fd, void *buf, size_t count);
ssize_t write(int fd, const void *buf, size_t count);
int     close(int fd);
off_t   lseek(int fd, off_t offset, int whence);
```

```
open() returned fd=3 (lowest available)
write() returned 15 (may be SHORT - always loop!)
read() got 15 bytes: "hello posix io"
lseek to 6: 6, then read: "posix"
```

`open` always returns the lowest-numbered descriptor not currently in use — here 3, since 0, 1, and 2 are taken by the standard streams.

Flags

Choose exactly one of `O_RDONLY`, `O_WRONLY`, or `O_RDWR`, and combine it with any of `O_CREAT`, `O_TRUNC`, `O_APPEND`, `O_EXCL`, `O_NONBLOCK`, `O_CLOEXEC`, and `O_SYNC` using `|`. The `mode` argument — the permissions of a newly created file, such as `0644` — is used only with `O_CREAT`.

`O_CREAT | O_EXCL` is the atomic "create only if absent" operation — the system-call-level fix for the TOCTOU races mentioned in §13.4.

⚠ Short reads and writes are normal

`read` and `write` may transfer *fewer* bytes than you asked for, and that is not an error. It happens routinely on pipes, sockets, and terminals, and whenever a signal interrupts the call. You must loop:

```
static ssize_t write_all(int fd, const void *buf, size_t n)
{
    const char *p = buf;
    size_t left = n;
    while (left > 0) {
        ssize_t w = write(fd, p, left);
        if (w < 0) {
            if (errno == EINTR) continue;   /* interrupted — retry */
            return -1;
        }
        p += w; left -= (size_t)w;
    }
    return (ssize_t)n;
}
```

Each iteration writes as much as the system will accept and advances past it. If a signal interrupts the call before anything was written, `write` fails with `errno` set to `EINTR`, which simply means "try again."

Forgetting this is one of the most common bugs in networking code. It works in testing, with small messages that always fit in one call, and truncates in production, with large ones.

`mmap` — the file is memory

```
#include <sys/mman.h>
void *map = mmap(NULL, len, PROT_READ, MAP_PRIVATE, fd, 0);
```

```
mmap: file is now MEMORY at 0x7fc4681b1000, size 15
  map[0..4] = "hello" (no read() syscall, just a page fault)
```

`mmap` maps a file directly into your address space — the same mechanism the loader uses for executables and shared libraries (Module 10). Afterwards, the file's contents are simply memory: indexing into `map` reads the file. Pages are loaded on demand by the kernel the first time they are touched (a *page fault*), with no copying into a separate buffer and no system call per access.

`mmap` wins	**`read/write` wins**
Large files, random access	Sequential streaming
Multiple processes sharing a file	Small files (setup cost dominates)
Avoiding a copy into your buffer	Pipes, sockets (can't be mapped)
Treating a file as an array	Files that may be truncated (→ `SIGBUS`)

> ⚠ If the file shrinks while it is mapped, touching the pages that no longer exist raises the `SIGBUS` signal — not an error return you can check. And `mmap` does not work on non-seekable streams such as pipes.

Always `munmap` a mapping when you are done with it. Use `MAP_SHARED` plus `msync` when writes through the mapping must reach the file and be visible to other processes.

`FILE *` vs raw file descriptor

	`FILE *`	**file descriptor**
Buffering	yes (stdio)	none (each call is a system call)
Portability	ISO C, everywhere	POSIX
Formatted I/O	`printf/scanf`	none
Needed for	ordinary files	sockets, pipes, `select/poll/epoll`, `fcntl`, `mmap`, `ioctl`
Bridge	`fileno(fp)` → fd	`fdopen(fd, mode)` → `FILE *`

> ⚠ **Never mix buffered and unbuffered I/O on the same file.** `printf` followed by `write(1, ...)` gives you the scrambled output demonstrated in §13.6. If you must mix them, call `fflush` first.

Also worth knowing:

- **`pread/pwrite`** — positioned I/O that does not change the file offset, which is essential when several threads share a descriptor.
- **`readv/writev`** — scatter/gather I/O, transferring several buffers in one call.
- **`sendfile/splice`** — zero-copy transfers from a file to a socket, inside the kernel.
- **`O_CLOEXEC`** — closes the descriptor automatically when the process `exec`s another program, so descriptors do not leak into child programs (Module 24).

Module 13 Rules

#	Rule
1	`%zu` for `size_t`, `%td` for `ptrdiff_t`, `PRIu64` for `uint64_t`; cast to `void *` for `%p`
2	Never pass user input as a format string; compile with `-Wformat=2`
3	`%.17g` or `%a` to see a float's true value
4	Don't use `scanf` on `stdin` — use `fgets` + `strtol`
5	If you must use it, bound every `%s` (`%15s`) and check the return count
6	Always check `fopen`'s return — and `fclose`'s too
7	Always use `"rb"`/`"wb"` for binary — free on Unix, required on Windows
8	Never `fwrite` raw structs to a file another program reads
9	`while (!feof(fp))` is always wrong — test the read's return value
10	Check `ferror` after the loop to distinguish EOF from an I/O error
11	`abort()` and `_exit()` don't flush — `setvbuf(stdout, NULL, _IONBF, 0)` while debugging
12	Errors and diagnostics to `stderr`, data to `stdout`
13	`read`/`write` can return short — always loop, and handle `EINTR`
14	Never mix `FILE *` and raw fd I/O on the same file

Standing build command:

```
gcc -std=c17 -Wall -Wextra -Wpedantic -Wformat=2 -Wshadow -Wconversion \
    -Werror=vla -Werror=return-type -Werror=implicit-function-declaration \
    -D_FILE_OFFSET_BITS=64 -O1 -g -D_FORTIFY_SOURCE=3 \
    -fsanitize=address,undefined prog.c -o prog
```

`-D_FILE_OFFSET_BITS=64` makes `off_t` (and functions like `fseeko`, `ftello`, and `lseek`) 64-bit even on 32-bit systems, so the program handles files larger than 2 GB everywhere.

Practice

1. **Format-specifier drill.** Write a program that prints the same value with 15 different flag/width/precision combinations. Then deliberately mismatch types (`%d` with a `long`, `%s` with an `int`) and confirm that `-Wformat=2` catches each one. Run the mismatched version under UBSan.

2. **Break `scanf` four ways.** Reproduce all four traps from §13.2 in one program. For the infinite-loop trap, write a correct retry loop that drains the bad input.

3. **Build a robust reader.** Implement `read_line` and `parse_long` from §13.3, plus `parse_double` and `parse_bool`. Feed them a file with 20 pathological inputs (empty lines, whitespace-only lines, huge numbers, embedded NULs, a 10 KB line, no trailing newline) and verify that every case is reported correctly. Then rewrite using POSIX `getline` and compare.

4. **Buffering forensics.** Write a program that prints 5 lines to `stdout` and 5 to `stderr`, alternating. Run it (a) to a terminal, (b) piped to `cat`, (c) redirected to a file, and (d) under `stdbuf -o0`. Record the ordering in each case and explain it.

5. **Lost output.** Reproduce the `abort()` buffer-loss bug. Then fix it four ways: `setvbuf` unbuffered, `setvbuf` line-buffered, an explicit `fflush`, and switching to `stderr`. Which would you use in production, and why?

6. **`feof` off-by-one.** Write both loop forms over a 3-byte file and a 0-byte file. Then write a version that correctly distinguishes a clean EOF from a read error — simulate the error by reading from a directory (`fopen("/tmp", "r")`).

7. **Portable serialisation.** Write `rec_write(FILE *, const struct Rec *)` and `rec_read(FILE *, struct Rec *)` that serialise field by field in little-endian order with explicit sizes. Verify that the file is byte-identical regardless of structure padding. Then write the naive `fwrite(&rec, ...)` version and dump both files with `xxd` to compare.

8. **`write_all` / `read_all`.** Implement both, with `EINTR` handling and short-transfer looping. Test `write_all` by writing 10 MB to a pipe with a slow reader on the other end, and prove that a naive single `write` would have truncated.

9. **`mmap` vs `read`.** Write two versions of "count the newlines in a file" — one using `read` into a 64 KB buffer, one using `mmap`. Benchmark them on a 1 GB file. Then repeat on a 1 KB file and explain the reversal.

10. **A real tool.** Write a `wc`-alike: it reads from the files named in `argv`, or from `stdin` if there are none; counts lines, words, and bytes; writes the counts to `stdout` and errors to `stderr`; and returns `EXIT_FAILURE` if any file failed. Verify that it composes: `cat big.txt | ./mywc` and `./mywc *.c | sort -n`.

Module 13 in Summary

The `printf` family trusts its format string completely, so every conversion's length modifier must match its argument's type exactly — `%zu`, `%td`, and the `<inttypes.h>` macros exist for this — and a format string must never come from user input. `%.17g` and `%a` reveal a floating-point value's true contents, and `%.*s` prints a buffer of known length safely.

`scanf` leaves newlines and unmatched input behind, cannot bound strings at run time, and cannot report overflow, so robust programs read whole lines with `fgets` (or `getline`) and parse them with `strtol`. Open files with checked `fopen` calls, always use binary mode for binary data, check `fclose`, and serialise records field by field rather than writing raw structures.

stdio buffers output — line-buffered on a terminal, fully buffered in a pipe, unbuffered for `stderr` — which reorders mixed output and loses buffered text when a program aborts or crashes. Test each read's return value rather than calling `feof` first, then use `ferror` to tell an error from end-of-file. Keep data on `stdout` and diagnostics on `stderr`, so that tools compose.

Beneath stdio, POSIX file descriptors offer unbuffered system calls, where short reads and writes are normal and must be looped over, and `mmap`, which turns a file into memory. Never mix the two layers on the same file.

In Module 14, we take a tour of the rest of the standard library — the headers every C programmer should know.

Module 14 — The Standard Library Tour

Headers covered: `stdio` · `stdlib` · `string` · `math` · `ctype` · `time` · `assert` · `errno` · `signal` · `setjmp` · `locale` · `stdbool` · `stddef` · `stdint` · `inttypes` · `limits` · `float` · `stdarg` · `stdalign` · `stdnoreturn` · `complex` · `tgmath` · `threads.h` · `stdatomic.h` · `uchar.h`

In this module:

- Sorting and searching: `qsort`, `bsearch` (and writing correct comparators)
- Randomness: `rand` limitations, better PRNGs
- Time: `time_t`, `struct tm`, `clock`, monotonic clocks

All output in this module is real, from GCC 13.3 and glibc 2.39.

C's standard library is small compared with those of most modern languages — there are no containers, no networking, and no JSON — but it is everywhere: every hosted C implementation provides it, and much of it has been stable for over thirty years. It also carries the scars of that age. Many functions predate threads, some predate the idea that input might be hostile, and several have subtle contracts that are easy to violate. This module is a working map of every standard header: what is in it, what is safe to use, and what is a trap.

14.1 The Header Map

Core

Header	Contents	Notes
`<stdio.h>`	`FILE *`, `printf/scanf`, `fopen`, `fread`	Module 13. Banned: `gets`, `sprintf`, unbounded `scanf("%s")`
`<stdlib.h>`	`malloc/free`, `exit`, `atexit`, `qsort/bsearch`, `rand`, `strtol`, `abs`, `div`, `getenv`, `system`	Banned: the `atoi` family. ⚠ `system()` is a command-injection vector
`<string.h>`	`str*`, `mem*`	Module 8. Banned: `strcpy`, `strcat`, `strtok`
`<ctype.h>`	`isalpha`, `toupper`, …	⚠ See §14.2 — the argument must be an `unsigned char` value
`<math.h>`	Everything numeric	Link with `-lm`. See §14.6
`<time.h>`	`time_t`, `struct tm`, `clock`, `strftime`	§14.5
`<errno.h>`	`errno`, `E*` codes	Thread-local. §14.6
`<assert.h>`	`assert`, `static_assert`	§14.6

A word on `system()`: it passes its argument to the shell, so any untrusted text included in the command — a filename, a user name — can inject shell syntax such as `; rm -rf ~`. Use `fork`/`exec` or `posix_spawn` with an argument array instead (Module 24).

Types and limits (Module 2)

Header	Contents
`<stddef.h>`	`size_t`, `ptrdiff_t`, `NULL`, `offsetof`, `max_align_t`, C23 `nullptr_t`, `unreachable`
`<stdint.h>`	`int32_t`, `uintptr_t`, `intmax_t`, `SIZE_MAX`, `INT64_C`
`<inttypes.h>`	Includes `stdint.h` + `PRIu64`/`SCNd32` format macros + `strtoimax`
`<limits.h>`	`CHAR_BIT`, `INT_MAX`, `LONG_MIN`, …
`<float.h>`	`DBL_EPSILON`, `FLT_MAX`, `DBL_DIG`, …
`<stdbool.h>`	`bool`, `true`, `false` — obsolete in C23 (they're keywords)
`<stdalign.h>`	`alignas`, `alignof` — obsolete in C23
`<stdnoreturn.h>`	`noreturn` — deprecated in C23; use `[[noreturn]]`

Specialised

Header	Contents	Verdict
`<stdarg.h>`	`va_list`, `va_start`/`va_arg`/`va_end`/`va_copy`	Module 6
`<signal.h>`	`signal`, `raise`, `sig_atomic_t`	ISO C's version is nearly useless — use POSIX `sigaction` (§14.8)
`<setjmp.h>`	`setjmp`, `longjmp`	§14.7 — dangerous, rarely correct
`<locale.h>`	`setlocale`, `localeconv`	§14.6 — a hidden global that changes parsing
`<complex.h>`	`double complex`, `cabs`, `creal`	Optional since C11 (`__STDC_NO_COMPLEX__`). Rarely used
`<tgmath.h>`	Type-generic math via `_Generic`	Include instead of `<math.h>` to get `sqrt(x)` working for `float`/`double`/`long double`
`<threads.h>`	`thrd_t`, `mtx_t`, `cnd_t`	Optional (`__STDC_NO_THREADS__`). Module 17. Not in MSVC
`<stdatomic.h>`	`_Atomic`, `atomic_fetch_add`, memory orders	Optional (`__STDC_NO_ATOMICS__`). Module 17

Header	Contents	Verdict
`<uchar.h>`	`char16_t`, `char32_t`, `mbrtoc32`	Module 8 — UTF-8 in `char` is usually better
`<wchar.h>` / `<wctype.h>`	Wide-character everything	Avoid; `wchar_t` is 2 bytes on Windows, 4 on Unix
`<iso646.h>`	`and`, `or`, `not` macros	A historical curiosity for keyboards and character sets lacking `&&/||`
`<stdckdint.h>`	C23 `ckd_add/ckd_sub/ckd_mul`	Checked arithmetic — Module 2
`<stdbit.h>`	C23 `stdc_count_ones`, `stdc_bit_ceil`	Module 4

Optional-feature detection:

```
#ifdef __STDC_NO_THREADS__   /* also: _VLA, _ATOMICS, _COMPLEX */
```

If your code relies on an optional feature, test for its macro and fail early with `#error` (Module 12), rather than letting the build fail somewhere obscure.

14.2 `<ctype.h>` — the Argument Trap

> Every `ctype` function takes an `int` whose value must be representable as an `unsigned char`, or equal to `EOF`. Anything else is undefined behaviour.

```
char c = (char)0xE9;   /* 'é' in Latin-1 */
isalpha(c);            /* passes -23 → UB: indexes before the lookup table */
```

```
char c = 0xE9 -> as int: -23
  isalpha(c)                -> UB! index -23 into the table
  isalpha((unsigned char)c) -> 0 <- CORRECT
```

These functions are typically implemented as a lookup table indexed by the argument, which is why the range is restricted: the table has entries for −1 (`EOF`) and for 0 to 255. A negative argument other than −1 reads memory *before* the table — silently giving wrong answers, or crashing. And a `char` holding a byte above 127 is negative on platforms where `char` is signed (Module 2).

The idiom, every time:

```
for (const char *p = s; *p; p++)
    putchar(toupper((unsigned char)*p));
```

```
input : "Hello, World! 42"
upper : HELLO, WORLD! 42
```

The cast converts the byte to its value in the range 0–255 before it is widened to `int`. This bug bites specifically on non-ASCII input on platforms with a signed `char` — so the code passes every test with English text and breaks on the first accented character a real user types.

The full set: `isalpha`, `isdigit`, `isalnum`, `isspace`, `ispunct`, `isupper`, `islower`, `isxdigit`, `iscntrl`, `isgraph`, `isprint`, `isblank` (C99), `toupper`, `tolower`.

> ⚠ All of them are locale-dependent, and none of them work on UTF-8 multibyte sequences — calling `isalpha` on one byte of a multibyte character is meaningless.

14.3 `qsort` and `bsearch`

```
void  qsort  (void *base, size_t n, size_t size, int (*cmp)(const void*, const void*));
void *bsearch(const void *key, const void *base, size_t n, size_t size,
              int (*cmp)(const void*, const void*));
```

These are the standard library's generic algorithms, built on the `void *` + element size + comparison function pattern from Module 9. `qsort` sorts an array in place; `bsearch` finds an element in a sorted array by binary search.

The comparator contract

The comparator must return a negative value if `a < b`, zero if they are equal, and a positive value if `a > b`. And it must define a consistent ordering:

- **Consistent** — the same inputs give the same answer, every time.
- **Antisymmetric** — `cmp(a,b) > 0` if and only if `cmp(b,a) < 0`.
- **Transitive** — if `a < b` and `b < c`, then `a < c`; and likewise for equality.

Violating this is undefined behaviour — and the sorting algorithm relies on it. A sort that trusts a broken comparator can make inconsistent decisions, and some implementations will run off the end of the array and segfault.

⚠ `return a - b;` is a real bug

```
static int cmp_bad(const void *a, const void *b)
{ return *(const int*)a - *(const int*)b; }
```

```
cmp_bad(INT_MIN, INT_MAX) = 1    <- WRONG SIGN (overflowed)
cmp_int(INT_MIN, INT_MAX) = -1   <- correct
```

`INT_MIN - INT_MAX` is far outside the range of `int`. The subtraction overflows — undefined behaviour — and in practice wraps around to 1, a *positive* number, claiming that the smallest `int` is greater than the largest. And it genuinely fails to sort:

```
cmp_bad result: 0 5 2147483646 2147483647 -2147483648 -2147483647 -1000000000 1000000000
  sorted? NO
```

```
cmp_ok result:  -2147483648 -2147483647 -1000000000 0 5 1000000000 2147483646 2147483647
  sorted? yes
```

UBSan confirms the cause:

```
runtime error: signed integer overflow: 2147483647 - -2147483648 cannot be represented in type 'int'
```

The correct form — branchless, overflow-free, and returning exactly −1, 0, or 1:

```c
static int cmp_int(const void *a, const void *b)
{ int x = *(const int*)a, y = *(const int*)b; return (x > y) - (x < y); }
```

The subtraction version only works when the values are known to be small — and "known to be small" has a way of becoming false years later.

Comparators for other types

```c
/* array of strings — note the DOUBLE indirection */
static int cmp_str(const void *a, const void *b)
{ return strcmp(*(const char *const *)a, *(const char *const *)b); }

/* multi-key: sort by dept, then name */
static int cmp_emp(const void *a, const void *b)
{
    const struct Emp *x = a, *y = b;
    if (x->dept != y->dept) return (x->dept > y->dept) - (x->dept < y->dept);
    return strcmp(x->name, y->name);
}

/* doubles — must handle NaN or you break transitivity */
static int cmp_dbl(const void *a, const void *b)
{
    double x = *(const double*)a, y = *(const double*)b;
    if (isnan(x)) return isnan(y) ? 0 : 1;   /* NaNs sort last */
    if (isnan(y)) return -1;
    return (x > y) - (x < y);
}
```

```
sorted: 1 3 3 5 7 9
bsearch(7) -> index 4
bsearch(4) -> NULL (not present)
alpha bravo charlie delta
dept=1 amy / dept=1 cal / dept=2 bob / dept=2 zoe
```

The string comparator is a frequent source of confusion. The array being sorted is an array of `char *`, so the comparator receives pointers *to* those elements — pointers to `char *`, i.e. `char **` — and must dereference once before calling `strcmp`. Passing `a` and `b` straight to `strcmp` compares the bytes of the pointers themselves.

The `double` comparator shows why NaN needs explicit handling: every comparison involving NaN is false (Module 2), so a naive comparator reports a NaN as "equal" to everything, which breaks transitivity (1 "equals" NaN, and NaN "equals" 2, yet 1 < 2).

The gotchas

Gotcha	Detail
Not stable	Equal elements may be reordered. Add a tiebreaker key if order matters.

Gotcha	Detail
No worst-case guarantee	glibc's implementation is O(n log n) in practice, but the standard promises nothing about complexity.
No user-data parameter	You need a global (not thread-safe) or the non-standard `qsort_r`/`qsort_s`. A real API design flaw (Module 6).
`bsearch` needs a sorted array	Sorted by the same comparator. Otherwise it returns garbage or `NULL`.
`bsearch` returns any match	With duplicates, you don't know which one you get.
Comparator called on the same element	Some implementations call `cmp(x, x)` — it must return 0.
NaN breaks everything	See above.

14.4 Randomness

`rand()` limitations

```
RAND_MAX = 2147483647 (the standard only guarantees >= 32767)
```

That "only guarantees 32767" is important: on some platforms — including Microsoft's C runtime — `rand()` returns at most 15 bits. Code that assumes a large range, such as `rand() % 1000000`, breaks silently there.

Modulo bias, made visible with a small generator:

```
a generator returning 0..9 uniformly, mod 3
  values 0..9 mapped by %3 -> counts: 4 3 3
  0 and 1 get 4 slots, 2 gets 3 -> 33% more likely!

Same effect for rand()%n whenever (RAND_MAX+1) % n != 0:
  2147483648 % 3 = 2 -> bias present
  2147483648 % 4 = 0 -> no bias (power of two)
```

Ten equally likely values cannot be divided evenly into three groups: the values 0–8 split three ways, and the leftover 9 goes to group 0. In general, whenever the generator's number of possible outputs is not a multiple of `n`, the smaller results of `% n` are slightly more likely than the larger ones.

With a 31-bit `RAND_MAX`, the bias for small `n` is tiny (about one part in a billion), but it is real — and with a 15-bit `RAND_MAX` and a large modulus, it is substantial. The fix is *rejection sampling*:

```c
static int rand_below(int n)   /* uniform in [0, n) */
{
    int lim = RAND_MAX - (RAND_MAX % n);   /* largest usable multiple */
    int r;
    do { r = rand(); } while (r >= lim);
```

```
    return r % n;
}
```

`lim` is the largest multiple of `n` not exceeding `RAND_MAX`. Values below it divide evenly into `n` groups, so any result at or above it is simply discarded and redrawn. Since at most `n - 1` values out of more than two billion are rejected, the loop almost never repeats.

Other `rand()` problems:

- **Not thread-safe** — it keeps shared hidden state. (POSIX `rand_r` exists, but it is marked obsolescent and has a tiny state.)
- **Quality varies wildly** — glibc's is decent; some C libraries' implementations fail basic statistical tests badly.
- **Not cryptographically secure** — its internal state can be recovered from its output, after which every future value is predictable.
- **`srand(time(NULL))`** gives the same seed to every process started in the same second.

Reproducibility is a feature

```
srand(42): 66 40 81 41 12
srand(42): 66 40 81 41 12
```

The same seed always produces the same sequence. Fixed seeds make tests, simulations, and bug reports reproducible. Never seed from the clock in a test suite — a test that fails one run in a thousand, differently each time, is almost impossible to debug. (A common compromise: seed randomly, but print the seed, so any failure can be replayed.)

A better PRNG

For simulation, games, hashing, and procedural generation, use a modern small pseudo-random number generator. xoshiro256** (by Blackman and Vigna, public domain) is a good default:

```c
static uint64_t s[4];
static uint64_t rotl(uint64_t x, int k) { return (x << k) | (x >> (64 - k)); }

static uint64_t xoshiro(void) {
    uint64_t r = rotl(s[1] * 5, 7) * 9, t = s[1] << 17;
    s[2] ^= s[0]; s[3] ^= s[1]; s[1] ^= s[2]; s[0] ^= s[3];
    s[2] ^= t; s[3] = rotl(s[3], 45);
    return r;
}

/* seed with splitmix64 — never seed xoshiro with all zeros */
static uint64_t sm(uint64_t *x) {
    uint64_t z = (*x += 0x9E3779B97F4A7C15ULL);
    z = (z ^ (z >> 30)) * 0xBF58476D1CE4E5B9ULL;
    z = (z ^ (z >> 27)) * 0x94D049BB133111EBULL;
    return z ^ (z >> 31);
}
```

```
raw: E01D6FAFC557F1B9 BD627EBE4406B404 2C23132B578B57DB 2E8B319D4D1F276A
unbiased bounded(6) x12: 1 6 3 5 5 5 6 3 2 6 5 3
```

The generator's entire state is four 64-bit words, updated with shifts, XORs, and rotations — operations that compile to a handful of fast instructions. An all-zero state would stay zero forever, which is why it is seeded from SplitMix64, a tiny generator that turns any single 64-bit seed into well-mixed state.

Its advantages: explicit state (so each thread or subsystem can own an independent generator — thread-safe by construction), a period of $2^{256} - 1$, better speed than `rand()`, and it passes the demanding BigCrush statistical test suite.

Other good choices: PCG; SplitMix64 on its own (trivially small); and the Mersenne Twister (statistically fine, but with a large state, slower, and — like all of these — not cryptographic).

For anything security-related, never use a PRNG

Platform	Use
Linux/BSD	`getrandom(2)`, `getentropy(3)`
Any Unix	`/dev/urandom`
BSD/macOS/glibc 2.36+	`arc4random_buf()`
Windows	`BCryptGenRandom`

Session tokens, keys, nonces, salts, password-reset links — all of these must come from the operating system's cryptographically secure random number generator (CSPRNG). A statistically excellent generator like xoshiro is still predictable to anyone who sees enough of its output, and predictable-token vulnerabilities are a recurring CVE class.

14.5 Time

```
time(NULL) = 1785221286, sizeof(time_t)=8
32-bit time_t overflows 2038-01-19 -> the Y2038 problem
```

`time_t` counts seconds since the Unix *epoch*, 1 January 1970 00:00:00 UTC. On modern 64-bit systems it is a 64-bit integer. Where it is still a signed 32-bit integer, it runs out at 03:14:07 UTC on 19 January 2038 and wraps around to 1901 — the *Y2038 problem*, which still threatens long-lived embedded systems.

The three time concepts

```
clock()          = 0.1409 s CPU time (CLOCKS_PER_SEC=1000000)
CLOCK_MONOTONIC  = 0.1436 s elapsed <- USE THIS for durations
CLOCK_REALTIME   = wall-clock date/time; can JUMP (NTP, DST, admin)
```

Clock	Measures	Use for
`time()` / `CLOCK_REALTIME`	Wall-clock date and time	Timestamps, dates, "when did this happen"

Clock	Measures	Use for
`CLOCK_MONOTONIC`	Elapsed time since some fixed point	Durations, timeouts, benchmarks
`clock()`	CPU time consumed by this process	Profiling actual computation

> ⚠ **Never measure elapsed time with `time()` or `CLOCK_REALTIME`.** NTP can step the clock backwards, an administrator can set it, and daylight-saving changes alter local time. You get negative durations, timeouts that never fire (or fire immediately), and "milliseconds" that last an hour. Every production timeout bug in this category traces back to using the wrong clock.

```c
#include <time.h>
struct timespec t0, t1;
clock_gettime(CLOCK_MONOTONIC, &t0);
/* ... work ... */
clock_gettime(CLOCK_MONOTONIC, &t1);
double secs = (t1.tv_sec - t0.tv_sec) + (t1.tv_nsec - t0.tv_nsec) / 1e9;
```

A monotonic clock only ever moves forward at a steady rate; its absolute value is meaningless (often the time since boot), but the difference between two readings is a true duration. (`clock_gettime` is POSIX. C11's `timespec_get(&ts, TIME_UTC)` gives only wall-clock time; C23 adds an optional `TIME_MONOTONIC` base, but support is not yet universal.)

Note in the measurement above that CPU time (0.1409 s) is slightly less than wall time (0.1436 s): the difference is time the process spent waiting to be scheduled. In a multithreaded program, `clock()` can *exceed* wall time, because it adds up the CPU time of every thread.

`struct tm` — three different bases

```
gmtime_r -> 2026-07-28 06:48:06 UTC
NOTE: tm_year is years since 1900, tm_mon is 0-11, tm_mday is 1-31
      (three different bases in one struct — a classic bug source)
```

`struct tm` holds a *broken-down* time — year, month, day, and so on — as produced by `gmtime` (UTC) or `localtime` (local time zone).

Field	Range
`tm_year`	years since 1900
`tm_mon`	0–11
`tm_mday`	1–31
`tm_hour`	0–23
`tm_wday`	0–6 (Sunday = 0)
`tm_yday`	0–365
`tm_isdst`	> 0 DST in effect, 0 not, < 0 unknown — let `mktime` work it out

Printing `tm_year` directly shows 126 for 2026, and printing `tm_mon` directly shows July as 6. Always add 1900 and 1 respectively when displaying.

⚠ The static buffer trap

```c
struct tm *a = gmtime(&t1);
struct tm *b = gmtime(&t2);   /* OVERWRITES what a points to */
```

```
a->tm_year+1900 = 2001   <- should be 1970!
b->tm_year+1900 = 2001
a == b ? 1 (same static buffer)
```

`gmtime`, `localtime`, `ctime`, and `asctime` all return a pointer to a single internal static buffer, which every call overwrites. So they are not thread-safe — and they are broken even in single-threaded code whenever you hold two results at once, as here.

Use the reentrant variants, which write into a buffer you supply: `gmtime_r`, `localtime_r`, `ctime_r`, and `asctime_r` (POSIX), or `gmtime_s` and `localtime_s` (C11 Annex K and Windows — note that the two use different argument orders).

Formatting and parsing

```c
char buf[64];
strftime(buf, sizeof buf, "%Y-%m-%d %H:%M:%S %Z", &tm);   /* → 2026-07-28 06:48:06 GMT */
strftime(buf, sizeof buf, "%A, %d %B %Y", &tm);           /* → Tuesday, 28 July 2026 */
```

`strftime` is the `printf` of dates, with its own conversion letters. It returns 0 if the result does not fit in the buffer — check it. `%A` and `%B` (day and month names) are locale-dependent.

For parsing, there is `strptime` (POSIX, not ISO C). And `mktime` converts a `struct tm` back into a `time_t`, interpreting it as local time — and *normalises* out-of-range fields along the way, which makes it a handy date calculator:

```c
tm.tm_mday += 45;              /* "45 days from now" */
time_t future = mktime(&tm);   /* normalizes month/year automatically */
```

Day 73 of July becomes the correct day of September, with the month (and year, if needed) adjusted automatically. (Set `tm_isdst = -1` before calling `mktime` if the new date may fall on the other side of a daylight-saving change.)

Other traps

- **`difftime(t2, t1)` is the portable way to subtract `time_t` values.** The standard does not guarantee that `time_t` is a plain count of seconds — it is everywhere in practice, but `difftime` costs nothing.
- **Leap seconds do not exist in `time_t`.** POSIX time pretends every day has exactly 86,400 seconds.
- **`localtime` reads the `TZ` environment variable and may call `tzset()`** — a hidden global dependency, and a performance trap when called in a loop.

- **Y2038:** a 32-bit `time_t` overflows at 2038-01-19 03:14:07 UTC. On 32-bit Linux systems with glibc, build with `-D_TIME_BITS=64 -D_FILE_OFFSET_BITS=64` to get a 64-bit `time_t`.

14.6 `assert`, `errno`, Math, Locale

`assert` and `NDEBUG`

```
assert(f() == 1);
```

```
### normal build ###   assert(f()==1): f called 1 time(s)
### -DNDEBUG build ### assert(f()==1): f called 0 time(s)
```

`assert(expr)` checks that `expr` is true and, if not, prints the expression, file, and line (Module 12's `#cond` trick) and calls `abort`. But when the macro `NDEBUG` is defined — as it is in many release builds — `assert(expr)` expands to nothing at all, and the expression is *not evaluated*.

> **Rule:** never put a side effect inside an `assert`.

```
assert(do_the_work() == 0);                 /* the work vanishes in release! */
int rc = do_the_work(); assert(rc == 0);    /* correct */
```

`assert` is for *programmer* errors — broken invariants and impossible states, which indicate a bug. It is not for run-time conditions such as a failed `malloc`, bad user input, or a missing file. Those can happen in a correct program, and need real error handling that survives `NDEBUG` (Module 15).

`_Static_assert(cond, "msg")` (C11; spelled `static_assert` via `<assert.h>`, and a keyword in C23) checks a condition at *compile* time, and costs nothing at run time:

```
_Static_assert(sizeof(int) >= 4, "int must be at least 32 bits");
_Static_assert(sizeof table / sizeof *table == COUNT, "table out of sync");
```

Use it liberally — it is the cheapest correctness tool in C.

`errno`

```
fopen failed: errno=2 strerror="No such file or directory"
perror form : fopen: No such file or directory
```

Many library functions report *why* they failed by storing an error code in `errno`, and `strerror` turns the code into a message.

Rule	Why
`errno` is thread-local	Safe to use per thread; in practice it is a macro expanding to a function call that returns the current thread's copy
Clear it before the call (`errno = 0`) when the function has no other failure indication	Functions only set it; they never clear it
Only check it after a failure indication	A *successful* call may set `errno` to anything
`strerror` is not thread-safe	Use `strerror_r` (POSIX) or `strerror_s`
`perror(msg)` prints `msg: description` to `stderr`	Convenient, but no formatting

The second and third rules work together. Most functions indicate failure through their return value (`NULL` from `fopen`, −1 from `read`), and only then is `errno` meaningful. A few, like `strtol`, can return any value on success, so the only way to detect an error is to set `errno = 0` beforehand and check it afterwards (Module 8).

> ⚠ **`strerror_r` has two incompatible versions.** The XSI version returns `int` and fills your buffer; the GNU version returns `char *` and may not use your buffer at all. The demo for this section hit exactly this — it printed the wrong message because it used the return value incorrectly. Check which one you are getting (`_POSIX_C_SOURCE` versus `_GNU_SOURCE`).

Only functions documented to set `errno` do so. `strtol` does; `printf` mostly doesn't.

`<math.h>`

```
sqrt(-1) = -nan, errno=33 (Numerical argument out of domain)   EDOM
log(0)   = -inf, errno=34 (Numerical result out of range)      ERANGE
math_errhandling = 3 (1=errno, 2=fp exceptions)
```

Link with `-lm` on Linux (it is not needed on macOS or Windows).

Error reporting is split. `math_errhandling` tells you whether the implementation reports errors through `errno` (`MATH_ERRNO`), through floating-point exception flags (`MATH_ERREXCEPT`, examined with `<fenv.h>`), or both — here, both. Check results with `isnan()`/`isinf()` rather than relying on `errno` alone.

Useful pieces beyond the obvious: `fabs`, `fmod`, `remainder`, `round`/`lround`/`floor`/`ceil`/`trunc`, `fma` (fused multiply-add, with a single rounding), `hypot` (an overflow-safe $\sqrt{(x^2 + y^2)}$), `copysign`, `nextafter`, `fmin`/`fmax`, `signbit`, `fpclassify`, and `isfinite`.

`<tgmath.h>` gives you type-generic versions via `_Generic` (Module 12): `sqrt(x)` dispatches to `sqrtf`, `sqrt`, or `sqrtl` automatically.

`<locale.h>` — a hidden global that changes parsing

```
default locale: C
strtod("3.14") = 3.14
```

Under a German locale, the decimal separator is a comma:

```
setlocale(LC_NUMERIC, "de_DE.UTF-8");
strtod("3.14", NULL);   /* → 3.0, stops at the dot! */
strtod("3,14", NULL);   /* → 3.14 */
```

This silently corrupts configuration-file parsing, network protocols, and CSV handling on non-English systems. The program works perfectly in testing and produces wrong numbers for European users.

```
/* For anything machine-readable, force the C locale */
setlocale(LC_ALL, "");          /* user's locale for messages/dates... */
setlocale(LC_NUMERIC, "C");     /* ...but C for number parsing */
```

Alternatively, use `strtod_l` with `newlocale` (POSIX) for per-call locale control. The locale also affects `printf("%f")` output, `isalpha`, `strcoll`, and `strftime` — and because it is process-wide, changing it in one thread affects all of them.

A program that never calls `setlocale` runs in the `"C"` locale — which is why this bug appears only after someone adds internationalisation support.

14.7 `<setjmp.h>` — Non-Local Jumps

```
static jmp_buf env;

int rc = setjmp(env);          /* returns 0 first time, the longjmp value later */
if (rc == 0) { deep(5); }      /* deep() calls longjmp(env, 42) */
else         { /* rc == 42 */ }
```

```
setjmp returned 0 (first pass)
  reached depth 0, jumping out...
setjmp returned 42 (came back from longjmp)
```

`setjmp` saves the current execution context (registers, stack pointer, program counter) in a `jmp_buf` and returns 0. Later, `longjmp(env, 42)` — called from anywhere deeper in the call stack — restores that context, so execution resumes as if `setjmp` had just returned again, this time with the value 42. It is a *non-local goto*: it unwinds an arbitrary number of stack frames in one step.

⚠ The `volatile` requirement

```
int plain = 1;
volatile int vol = 1;
if (setjmp(e) == 0) { plain = 2; vol = 2; longjmp(e, 1); }
printf("plain=%d vol=%d\n", plain, vol);
```

```
after longjmp: plain=1 (INDETERMINATE) vol=2 (guaranteed 2)
```

Local, non-`volatile` automatic variables that are modified between `setjmp` and `longjmp` have *indeterminate* values afterwards. The compiler may have kept such a variable in a register, and `longjmp` restored the registers to their state at the time of `setjmp` — so `plain` reverted to 1. `volatile` forces the variable to live in memory, which `longjmp` does not touch.

GCC's `-Wclobbered` catches some such cases, but not reliably. Mark every local that must survive a `longjmp` as `volatile`.

The other dangers

- **It does not unwind.** No `free`, no `fclose`, no mutex unlock. Every resource acquired between `setjmp` and `longjmp` leaks.
- **Calling `longjmp` after the function that called `setjmp` has returned is undefined behaviour** — and a common bug, because the saved context points into a stack frame that no longer exists.
- **`longjmp(env, 0)` is silently converted to 1,** so you cannot use 0 as a signal value.
- **Not safe across threads.** Each thread needs its own `jmp_buf`, and you cannot jump into another thread.
- **From a signal handler, use `sigsetjmp/siglongjmp`,** which also save and restore the signal mask.

When it's legitimate

Fatal-error escapes in interpreters and parsers, and libraries that adopted it before better options existed: libpng, libjpeg, and Lua's error handling all use it. If you use it, pair it with an arena allocator (Module 10), so that cleaning up after the jump is a single `arena_reset`.

> Otherwise: return an error code and use the `goto` cleanup chain from Module 5. It is explicit, it releases resources, and it survives code review.

14.8 `<signal.h>` — Why ISO C's Version Is Nearly Useless

```
void (*signal(int sig, void (*handler)(int)))(int);
```

A *signal* is an asynchronous notification delivered to a process — for example, `SIGINT` when the user presses Ctrl-C. `signal` installs a function to be called when one arrives. (The prototype is the unreadable one from Module 9: a function taking a signal number and a handler, and returning the previous handler.)

ISO C guarantees only six signals — `SIGABRT`, `SIGFPE`, `SIGILL`, `SIGINT`, `SIGSEGV`, and `SIGTERM` — and leaves almost everything else unspecified: whether the handler is reset to the default after it fires, whether interrupted system calls are restarted, and what you may safely do inside a handler.

A signal can arrive at any instant — including in the middle of a `malloc` that is halfway through updating the heap's internal lists. So inside a signal handler, ISO C lets you do essentially two things:

- Assign to a variable of type `volatile sig_atomic_t`.
- Call `abort`, `_Exit`, `quick_exit`, or `signal`.

No `printf`, no `malloc`, no `free` — those functions are not *async-signal-safe*, and calling them from a handler can deadlock or corrupt the heap.

The standard pattern:

```c
static volatile sig_atomic_t g_stop = 0;
static void on_sigint(int sig) { (void)sig; g_stop = 1; }   /* that's ALL */

/* main loop */
while (!g_stop) { do_work(); }
```

The handler only records that the signal happened; the main loop notices the flag at a safe point and does the real work — saving state, closing files, exiting cleanly. `volatile` ensures that the loop re-reads the flag every time (Module 2).

For real work, use POSIX `sigaction`: it has defined reset behaviour, blocks other signals while the handler runs, supports `SA_RESTART`, and provides extra information in `siginfo_t`. Alternatively, `signalfd` or `pselect` turn signals into ordinary file-descriptor events that can be handled in a normal event loop. Module 24 gives signals the full treatment.

Module 14 Rules

#	Rule
1	Always cast to `(unsigned char)` before calling any `ctype` function
2	Comparators: `(x>y)-(x<y)`, never `x-y`
3	`qsort` isn't stable and takes no user data; `bsearch` needs the same comparator
4	Handle NaN explicitly in floating-point comparators
5	`rand() % n` is biased — use rejection sampling, or a real PRNG
6	Never use `rand()` for anything security-related — use the OS CSPRNG
7	`CLOCK_MONOTONIC` for durations; `CLOCK_REALTIME` only for dates
8	`gmtime/localtime/ctime` share one static buffer — use the `_r` variants
9	`tm_year` is years since 1900; `tm_mon` is 0-based

#	Rule
10	Never put side effects in `assert`; use `_Static_assert` liberally
11	`errno = 0` before; check only after a documented failure
12	Force `LC_NUMERIC` to `"C"` for machine-readable number parsing
13	`setjmp` locals need `volatile`; `longjmp` leaks every resource
14	Signal handlers: set a `volatile sig_atomic_t` flag and nothing else
15	Link `-lm` for `<math.h>`; check `isnan`/`isinf`, not just `errno`

Practice

1. **`ctype` audit.** Write a word-counting function that uses `isspace` without the cast. Feed it a UTF-8 file with accented characters and run it under ASan with a hand-built lookup table, so that the out-of-bounds index is detectable. Then fix it.

2. **Comparator suite.** Write correct comparators for `int`, `unsigned`, `double` (NaN-safe), `char *`, a multi-key structure, and case-insensitive strings. Test each against a shuffled 10,000-element array and verify the result with an `is_sorted` check. Then run the `a - b` version under UBSan.

3. **Stability test.** Sort an array of `{key, original_index}` pairs by key alone with `qsort`, and check whether the original order is preserved among equal keys. Then make it stable by adding `original_index` as a tiebreaker.

4. **Bias measurement.** Write a generator returning 0–255 uniformly and measure the distribution of `x % 100` over 10 million samples. Tabulate the counts. Then implement rejection sampling and measure again.

5. **PRNG comparison.** Implement `rand()`-based, xoshiro256**, and SplitMix64 generators. Benchmark 100 million calls of each. Then run a simple statistical check (chi-squared on byte frequencies, or a count of runs) and compare.

6. **Time forensics.** Write a benchmark harness using `clock()`, `CLOCK_REALTIME`, and `CLOCK_MONOTONIC`. Run it while manually changing the system clock (`date -s`, inside a container) and observe which measurements break.

7. **The `gmtime` trap.** Reproduce the static-buffer aliasing bug. Then write a `format_range(time_t start, time_t end)` that needs both broken-down times at once — first incorrectly, then with `gmtime_r`.

8. **Date arithmetic.** Using `mktime` normalisation, write `time_t add_days(time_t t, int days)` and `int days_between(time_t a, time_t b)`. Test across a daylight-saving boundary and across a leap day.

9. **`assert` and `NDEBUG`.** Build a program with an `assert` that contains a side effect. Compile it with and without `-DNDEBUG` and show the difference in behaviour. Then convert every run-time check to proper error handling, and every invariant that can be checked at compile time to `_Static_assert`.

10. **Locale bug.** Write a configuration parser that reads `"threshold = 3.14"` with `strtod`. Run it under `LC_ALL=de_DE.UTF-8` (install the locale with `locale-gen`) and observe the corruption. Fix it three ways: `setlocale(LC_NUMERIC, "C")`, `strtod_l`, and a hand-written parser.

Module 14 in Summary

The standard library is small, universal, and old, and every header has its traps. `ctype` functions need their argument cast to `unsigned char`. `qsort` and `bsearch` depend on a comparator that defines a consistent ordering: never subtract, handle NaN explicitly, remember the double indirection for arrays of strings, and remember that `qsort` is not stable.

`rand()` has a range that may be only 15 bits, is biased when reduced with `%`, keeps hidden global state, and is predictable. Use rejection sampling for bounded values, a modern generator such as xoshiro256** for simulation, fixed seeds for tests, and the operating system's CSPRNG for anything security-related.

Measure durations with a monotonic clock, never with wall-clock time. `struct tm` mixes three different bases, the classic time functions share one static buffer, and 32-bit `time_t` ends in 2038. `assert` disappears under `NDEBUG`, so it must never contain side effects; `_Static_assert` is free. `errno` is meaningful only after a reported failure, math errors are best detected with `isnan`/`isinf`, and the locale can silently change how numbers are parsed. Finally, `setjmp`/`longjmp` skip all cleanup and clobber non-`volatile` locals, and ISO C signal handlers should do nothing but set a flag.

In Module 15, we turn to Error Handling Strategies — return codes, `errno`, error objects, cleanup patterns, and how to design an API's error reporting.

Module 15 — Error Handling Strategies

In this module:

- Return codes, sentinel values, out-parameters
- `errno`, `perror`, `strerror`, thread-safety of `errno`
- `assert` vs runtime checks; `NDEBUG`
- The `goto` cleanup / single-exit pattern
- `setjmp`/`longjmp` (and its dangers)
- Designing an error enum + error-string API

All output in this module is real GCC 13.3 output.

C has no exceptions. There is no `try`, no `catch`, no stack unwinding that frees resources automatically when something goes wrong. Every function that can fail must *report* its failure through some ordinary channel — a return value, an output parameter, a global variable — and every caller must *check* for it and decide what to do.

That makes error handling a design decision rather than a language feature: you make it once per codebase, and then you enforce it everywhere. Getting it right is what separates a library that people can build on from one they cannot. This module surveys the conventions, shows how to use `errno` and `assert` correctly, and then builds a complete, reusable error-reporting API.

15.1 The Four Conventions

Style 1 — Error code returned, value via out-parameter

```
enum { E_OK = 0, E_NOMEM = -1, E_RANGE = -2, E_INVAL = -3 };

static int parse(const char *s, int *out)
{
    if (!s || !out) return E_INVAL;
    ...
    *out = (int)v;
    return E_OK;
}
```

```
"42"                  rc= 0 value ok
"abc"                 rc=-3 invalid
"99999999999999999999" rc=-2 range
```

The function's return value is reserved for status; the actual result goes through a pointer parameter (an *out-parameter*, Module 6).

Its advantages: every failure mode is distinguishable; the return type is uniform across the whole API, so functions compose easily (a caller can simply pass a callee's code upward); and it works for any result type, including those with no spare value to use as a sentinel.

This is the right default for library code.

Style 2 — Sentinel value

```
static int find(const int *a, size_t n, int key)
{ for (size_t i=0;i<n;i++) if (a[i]==key) return (int)i; return -1; }
```

```
find(20) = 1 find(99) = -1 (-1 means not found)
PROBLEM: what if -1 is a VALID result? Sentinels don't compose.
```

A *sentinel* is a special value of the result type that means "failure." This is only safe when that value genuinely cannot be a real result. `malloc`, `strchr`, and `fopen` returning `NULL` work well, because a null pointer is never a valid result.

−1 from a function returning an index works, because indices are never negative. −1 from a function returning an arbitrary number supplied by the user does not. And a sentinel carries no information about *why*: you learn "it failed," never "it failed because…".

Style 3 — `errno` (the libc style)

```
static long dbl(long v)
{ if (v > LONG_MAX/2) { errno = ERANGE; return LONG_MAX; } return v*2; }
```

```
double(100)=200 errno=0
double(LONG_MAX)=9223372036854775807 errno=34 (Numerical result out of range)
```

The function returns some value and records the reason for any failure in `errno`. The caller must clear `errno` before the call and check it afterwards — the discipline from Module 14.

Use this style only for compatibility with existing libc conventions. For new code it is error-prone: forgetting `errno = 0` gives you a stale error from some earlier call, and the error is easily lost when another function call intervenes before you check it.

Style 4 — `bool` + out-parameter

```
static bool divide(int a, int b, int *q)
{ if (b == 0) return false; *q = a/b; return true; }
```

This is clean and readable when there are exactly two outcomes and the caller does not need to know why an operation failed. Escalate to Style 1 the moment a second failure mode appears.

⚠ The out-parameter evaluation-order trap

```
printf("%d q=%d\n", divide(10, 2, &q), q);
```

```
-> 1 q=-999   <- q read BEFORE divide ran (unspecified order, Module 4)
```

The order in which function arguments are evaluated is unspecified (Module 4, §4.7). GCC happens to evaluate them right to left here, so `q` was read — still holding its initial −999 — before `divide` wrote to it.

```
bool ok = divide(10, 2, &q);   /* sequence the call first */
printf("%d q=%d\n", ok, q);    /* -> 1 q=5 */
```

> **Rule:** never read an out-parameter in the same expression as the call that fills it.

Choosing

Situation	Convention
Library with multiple failure modes	Error code + out-parameter
A truly impossible value exists	Sentinel (`NULL`, −1, `(size_t)-1`)
Two outcomes, reason irrelevant	`bool` + out-parameter
Matching POSIX/libc	−1 + `errno`
Application `main` level	Whatever the layers below use, plus a top-level report

Pick one convention and use it everywhere. A codebase where some functions return 0 on success and others return 1 on success is where bugs live.

Note also the polarity clash. Returning 0 means *success* in Unix exit codes and in most C error conventions — but 0 means *false* in C's truth values. Never write `if (parse(s, &v)) { /* success */ }` against a 0-on-success API: that branch runs exactly when parsing *failed*. Write `if (parse(s, &v) == E_OK)`, or name the error variable and test it: `if (rc != 0) { /* error */ }`.

15.2 `errno` in Depth

```
errno is THREAD-LOCAL:
  thread 3: errno=103 (unchanged - thread-local!)
  thread 2: errno=102 (unchanged - thread-local!)
  thread 1: errno=101 (unchanged - thread-local!)
  thread 0: errno=100 (unchanged - thread-local!)
```

Each of the four threads set `errno` to a different value, and each still saw its own value afterwards. That is possible only because `errno` is not really a global variable. It is a macro. On glibc:

```
#define errno (*__errno_location())
```

It expands to a call to a function that returns a pointer to the current thread's own copy, stored in thread-local storage (Module 3). The consequences:

- ✓ **It is thread-safe** — each thread has its own.
- ✗ **You cannot write `extern int errno;`** — that is pre-C89 code, and it will not link (or will silently refer to the wrong object).
- ⚠ **Taking `&errno` is fine, but the address is per-thread.**

The four rules

```c
errno = 0;                          /* 1. clear before */
long v = strtol(s, &end, 10);
if (end == s) { ... }               /* 2. check the primary failure indication first */
if (errno == ERANGE) { ... }        /* 3. only then consult errno */
```

1. **Only functions documented to set `errno` do so — and a successful call may set it to anything.** For instance, the first `printf` to `stdout` typically checks whether `stdout` is a terminal, using a system call that fails harmlessly when output is redirected — leaving `ENOTTY` in `errno` even though `printf` itself succeeded. That is why rule 2 matters: `errno` means something only after the function has told you, through its return value, that it failed.

Reporting

```
fopen failed: errno=2 strerror="No such file or directory"
perror form : fopen: No such file or directory
```

Function	Notes
`perror(msg)`	Prints `msg: description` to `stderr`. Convenient, but no formatting.
`strerror(e)`	Not thread-safe — may return a pointer to a shared static buffer.
`strerror_r(e, buf, n)`	⚠ Two incompatible versions. The XSI version returns `int` and fills your buffer; the GNU version returns `char *` and may ignore your buffer entirely. Which you get depends on `_GNU_SOURCE`.
`strerror_l(e, locale)`	POSIX 2008; thread-safe, with no ambiguity — prefer it.

The demo in Module 14 hit the `strerror_r` ambiguity live and printed the wrong message. Check which variant your feature-test macros select.

Preserving `errno` across cleanup

Cleanup code can clobber `errno` before your caller sees it:

```c
int saved = errno;   /* save it FIRST */
close(fd);           /* may set errno */
free(buf);
errno = saved;       /* restore before returning */
return -1;
```

The caller wants to know why the *original* operation failed, not whether a `close` during cleanup happened to fail too.

15.3 `assert` vs Runtime Checks

The decision rule

> **Can this happen in a correct program, given valid input?**
>
> - **No** → `assert()`. It documents an invariant, and costs nothing in release builds.
> - **Yes** → a runtime check that survives `-DNDEBUG`.

```c
/* ASSERT: a caller bug if violated */
static int array_get(const int *a, size_t n, size_t i)
{
    assert(a != NULL);
    assert(i < n);
    return a[i];
}

/* RUNTIME CHECK: user input can be anything */
static int parse_age(const char *s, int *out)
{
    if (!s || !out) return -1;
    char *end; long v = strtol(s, &end, 10);
    if (end == s || *end || v < 0 || v > 150) return -1;
    *out = (int)v; return 0;
}
```

```
parse_age("30")  -> ok
parse_age("-5")  -> rejected (runtime check, survives NDEBUG)
parse_age("abc") -> rejected
parse_age("200") -> rejected
```

`array_get` is an internal helper: if it is ever called with an out-of-range index, that is a bug in the calling code, and the right response is to stop the program loudly during development. `parse_age` processes user input, where every one of these "bad" cases is an entirely expected event that the program must handle gracefully.

Use `assert` for	Use a runtime check for
Internal invariants	`malloc` / `fopen` / `read` failure
Unreachable branches	User input, config files, CLI arguments
Preconditions of `static` functions	Preconditions of public API functions
"This can't be NULL here" after you just checked	Network data, file contents
Post-conditions during development	Anything an attacker controls

What `NDEBUG` does

```
### debug build ###
af: af.c:2: main: Assertion `i<n' failed.
Aborted

### -DNDEBUG build ###
(silently gone, exit=0)
```

Under `-DNDEBUG`, `assert(expr)` expands to nothing, and the expression is not evaluated at all (Module 14 measured `f()` being called once versus zero times). Never put a side effect inside an `assert`.

The debate about shipping with assertions on

Arguments for `-DNDEBUG` in release builds: performance; and for user-facing software, the view that an abrupt crash is worse than degraded behaviour.

Arguments against: a failed assertion means the program is *already* in a state its author believed impossible; continuing from there is how a bug becomes a security vulnerability. Many large projects and much of the security community therefore keep important assertions enabled in production.

A middle path used by many production systems is a separate check macro that is always on:

```
#define CHECK(cond) do {                                        \
        if (!(cond)) {                                          \
            fprintf(stderr, "%s:%d: CHECK failed: %s\n",        \
                    __FILE__, __LINE__, #cond);                 \
            abort();                                            \
        }                                                       \
    } while (0)
```

Use `assert` for cheap development-time invariants, and `CHECK` for the ones that must never be violated in production. (Note the `#cond` — Module 12's stringification — which prints the failed condition's source text.)

`_Static_assert` costs nothing

```
_Static_assert(sizeof err_msgs / sizeof *err_msgs == ERR_COUNT, "err table out of sync");
_Static_assert(sizeof(void*) == 8, "64-bit only");
```

Every invariant that *can* be checked at compile time *should* be. It is the cheapest correctness tool in C: it can never fire in production, because a violation stops the build.

15.4 Designing an Error API

Here is a complete, working design. It is about 60 lines of code, and it scales to large codebases.

The header

```
/* ---- SINGLE SOURCE OF TRUTH (X-Macro, Module 12) ---- */
#define ERR_LIST                                \
    X(OK,        0, "success")                  \
    X(NOMEM,     1, "out of memory")            \
    X(INVAL,     2, "invalid argument")         \
    X(RANGE,     3, "value out of range")       \
    X(NOTFOUND,  4, "not found")                \
    X(IO,        5, "I/O error")                \
    X(PARSE,     6, "parse error")              \
```

```
    X(PERM,     7, "permission denied")         \
    X(AGAIN,    8, "temporarily unavailable")

#define X(name, val, msg) ERR_##name = val,
typedef enum { ERR_LIST ERR_COUNT } err_t;
#undef X

const char *err_str(err_t e);        /* stateless code → message */
const char *err_detail(void);        /* rich per-thread context */
void        err_clear(void);

err_t err_set(err_t code, const char *file, int line, const char *fmt, ...)
    __attribute__((format(printf, 4, 5)));

#define ERR(code, ...) err_set((code), __FILE__, __LINE__, __VA_ARGS__)

#define TRY(expr)        do { err_t e_ = (expr); if (e_ != ERR_OK) return e_; } while (0)
#define TRY_GOTO(expr,l) do { rc = (expr); if (rc != ERR_OK) goto l; } while (0)
```

The design has two layers. An `err_t` *code* is a small integer that callers can branch on. Alongside it, a per-thread *detail* string records a human-readable description of the most recent error — which file and line raised it, and the specific values involved. `ERR(...)` records both and returns the code, so a failing function can report and return in one statement. `TRY` propagates any error from a call straight back to the caller; `TRY_GOTO` does the same but jumps to a cleanup label (§15.5) instead.

The implementation

```
#define X(name, val, msg) [ERR_##name] = msg,
static const char *const err_msgs[] = { ERR_LIST };
#undef X
_Static_assert(sizeof err_msgs / sizeof *err_msgs == ERR_COUNT, "err table out of sync");

const char *err_str(err_t e)
{ return (e >= 0 && e < ERR_COUNT) ? err_msgs[e] : "unknown error"; }

static _Thread_local char g_detail[256];   /* per-thread, like errno */

const char *err_detail(void) { return g_detail[0] ? g_detail : "(no detail)"; }
void        err_clear(void)  { g_detail[0] = '\0'; }

err_t err_set(err_t code, const char *file, int line, const char *fmt, ...)
{
    if (code == ERR_OK) { err_clear(); return code; }
    int n = snprintf(g_detail, sizeof g_detail, "%s:%d: %s: ", file, line, err_str(code));
    if (n < 0 || (size_t)n >= sizeof g_detail) return code;
    va_list ap; va_start(ap, fmt);
    vsnprintf(g_detail + n, sizeof g_detail - (size_t)n, fmt, ap);
    va_end(ap);
    return code;
}
```

`err_set` writes the location and the generic message first, then appends the caller's formatted detail after it using the `va_list` technique from Module 6 — carefully checking that the prefix fitted before appending (Module 8's truncation check). `err_str` guards against out-of-range codes, so a corrupted value cannot index outside the table.

Using it

```
static err_t parse_port(const char *s, int *out)
{
    if (!s || !out) return ERR(ERR_INVAL, "null argument");
    char *end; errno = 0;
    long v = strtol(s, &end, 10);
    if (end == s)           return ERR(ERR_PARSE, "no digits in \"%s\"", s);
    if (*end)               return ERR(ERR_PARSE, "trailing garbage \"%s\"", end);
```

```
    if (errno == ERANGE)    return ERR(ERR_RANGE, "\"%s\" overflows long", s);
    if (v < 1 || v > 65535) return ERR(ERR_RANGE, "port %ld not in 1..65535", v);
    *out = (int)v;
    return ERR_OK;
}

static err_t configure(const char *port_s, const char *backlog_s, int *port, int *backlog)
{
    TRY(parse_port(port_s, port));        /* propagates, preserving the detail */
    TRY(parse_port(backlog_s, backlog));
    return ERR_OK;
}
```

```
configure(8080 ,128  ) -> success     port=8080 backlog=128
configure(abc  ,128  ) -> parse error | emain.c:13: parse error: no digits in "abc"
configure(8080 ,99999) -> range       | emain.c:16: range: port 99999 not in 1..65535
configure(70000,1    ) -> range       | emain.c:16: range: port 70000 not in 1..65535
configure(80x  ,1    ) -> parse error | emain.c:14: parse error: trailing garbage "x"

ASan/UBSan exit=0
```

Each failure is reported with its precise cause and the exact source line that detected it, yet `configure` itself contains no error-handling code beyond two `TRY`s. (In the recorded run, the RANGE message printed as simply “range” rather than “value out of range”, so the demo program’s table evidently used a shorter string for that entry; the mechanism is identical.)

Why each piece is there

Decision	Reason
X-Macro table	Codes, names, and messages can never drift apart (Module 12)
`_Static_assert` on the count	Adding a code without a message fails the build
`ERR_OK == 0`	`if (rc)` reads naturally as “if error”; matches libc conventions
Both a code and a detail string	The code is for branching; the string is for humans
`_Thread_local` detail	Thread-safe by construction, exactly like `errno`
`__FILE__`/`__LINE__` in the macro	Free provenance, with no manual bookkeeping
`format(printf, 4, 5)`	Compile-time checking of every detail message (Module 6)
`TRY` macro	Propagation without four lines of boilerplate per call
Fixed-size detail buffer	No allocation — so it works when the error is `ERR_NOMEM`

That last point is easy to miss: **an error-reporting path must not allocate memory**, because the most important error to report is often “out of memory.” A reporting function that calls `malloc` to build its message fails at exactly the moment it is needed.

Alternative designs and their trade-offs

Design	Pros	Cons
This one (code + thread-local detail)	Rich, no allocation, thread-safe	Detail is "global-ish"; easy to forget `err_clear`
Error-object out-parameter (`err_t f(..., Error **e)`)	Explicit lifetime, chainable causes	Verbose; needs allocation or a pool
Bare integer codes	Simplest, zero overhead	No context — "parse error" without saying where
`errno`-style negative `errno` values	Interoperates directly with POSIX	Limited to the standard code set
Result struct (`struct { err_t e; T v; }`)	Value and error travel together	Only practical for small `T`; verbose in C

The "negative `errno` values" design is the Linux kernel's: functions return 0 on success or `-EINVAL`, `-ENOMEM`, and so on.

For *error chaining* — where a low-level `EIO` becomes a high-level "config load failed" — extend the detail buffer to append context rather than overwrite it, or move to the error-object design with a pointer to a "cause."

15.5 The `goto` Cleanup Chain with Error Propagation

Module 5 established the cleanup-chain pattern. Here it is carrying real error information:

```c
static err_t process(const char *inpath, const char *outpath, size_t bufsz, int fail_at)
{
    FILE *in = NULL, *out = NULL;
    char *buf = NULL;
    err_t rc = ERR_OK;

    in = fopen(inpath, "rb");
    if (!in) { rc = ERR(ERR_IO, "cannot open input \"%s\"", inpath); goto out; }

    out = fopen(outpath, "wb");
    if (!out) { rc = ERR(ERR_IO, "cannot open output \"%s\"", outpath); goto out_in; }

    buf = malloc(bufsz);
    if (!buf) { rc = ERR(ERR_NOMEM, "buffer of %zu bytes", bufsz); goto out_out; }

    size_t n;
    while ((n = fread(buf, 1, bufsz, in)) > 0)
        if (fwrite(buf, 1, n, out) != n) { rc = ERR(ERR_IO, "short write"); goto out_buf; }
    if (ferror(in)) { rc = ERR(ERR_IO, "read error"); goto out_buf; }

out_buf: free(buf);
out_out: if (fclose(out) != 0 && rc == ERR_OK) rc = ERR(ERR_IO, "fclose failed");
out_in:  fclose(in);
out:     return rc;
}
```

```
happy path        -> success
input open fails  -> I/O error   cleanup.c:14: I/O error: cannot open input "missing"
output open fails -> I/O error   cleanup.c:17: I/O error: cannot open output "/no/dir/x"
mid-work failure  -> parse error cleanup.c:22: parse error: simulated mid-work failure
```

```
ASan/LSan exit=0 (no leaks on ANY error path)
```

(The "mid-work failure" case comes from a fault-injection check driven by `fail_at`, placed inside the work section of the test program and omitted from the listing above for brevity.)

The three refinements over Module 5

1. **`rc` is set before the `goto`, not at the label.** Each failure site records its own specific cause, so the cleanup labels do nothing but release resources.
2. **`fclose` errors are checked — and only overwrite `rc` if nothing has failed yet.** Don't clobber the first error with a cleanup error; the original cause is the useful one. (Recall from Module 13 that a write error may only surface when `fclose` flushes the final buffer, so on the success path this check is essential.)
3. **Every label unwinds exactly one more resource than the one below it.** Name labels after what they release.

Testing error paths

Notice the `fail_at` parameter. *Fault injection* is a real technique, not a toy:

- Add a test-only parameter, or a global, that forces failure at a chosen step.
- `LD_PRELOAD` a `malloc` that fails after *N* calls.
- Use `ulimit -v` to force real allocation failures.
- Point paths at `/dev/full` (which fails every write with "no space left on device"), unreadable files, or full filesystems.

Then run the whole matrix under LeakSanitizer. The happy path is easy; the error paths are where the leaks are — and they are exactly the paths that never get exercised in normal testing.

15.6 `setjmp` / `longjmp`

These were covered in Module 14, §14.7. The summary, for error-handling purposes:

```
after longjmp: plain=1 (INDETERMINATE) vol=2 (guaranteed 2)
```

Danger	Consequence
Non-`volatile` locals are indeterminate after `longjmp`	Silent wrong values; `-Wclobbered` catches only some
No unwinding	Every `malloc`, `fopen`, and locked mutex between `setjmp` and `longjmp` leaks
`longjmp` into a function that has returned	Undefined behaviour, usually a crash
`longjmp(env, 0)` becomes 1	You can't use 0 as a signal value
Sharing a `jmp_buf` across threads	Not safe — each thread needs its own
Signal handlers need `sigsetjmp`/`siglongjmp`	Otherwise the signal mask isn't restored

It is tempting to see `setjmp`/`longjmp` as C's exceptions. They are not: exceptions in other languages run destructors or `finally` blocks as the stack unwinds, and `longjmp` runs nothing at all.

When it is defensible: you have an arena allocator (Module 10), so that all cleanup is a single `arena_reset`, and the error is genuinely fatal to the current operation. That is how Lua and some parsers use it.

Otherwise: error codes plus the `goto` chain. It is explicit, it releases resources, and it survives code review.

15.7 Putting It Together — a Checklist

At the API boundary

```
/**
 * Load configuration from `path`.
 *
 * @param path Config file path. Must not be NULL.
 * @param out  On success, receives a newly allocated Config.
 *             Caller must free with config_free(). Untouched on failure.
 * @return ERR_OK on success.
 *         ERR_INVAL if path or out is NULL.
 *         ERR_IO    if the file cannot be read.
 *         ERR_PARSE if the contents are malformed.
 *         ERR_NOMEM on allocation failure.
 *         On error, err_detail() describes the specific problem.
 */
err_t config_load(const char *path, Config **out);
```

This comment answers every question a caller needs answered: what may be passed, who owns the result (Module 10), what happens to the output on failure, and every error the function can return.

> **Document every error code a function can return.** An undocumented failure mode is a failure mode nobody handles.

The rules

1. **One convention per codebase.** Enforce it in review.
2. **Never ignore a return value.** Mark functions `__attribute__((warn_unused_result))` (or C23 `[[nodiscard]]`) where ignoring the result is always a bug; the compiler then warns at every call that discards it.
3. **Fail fast at the boundary; propagate inside.** Validate untrusted input once, at the edge of the system; internal functions can then `assert` their preconditions.
4. **Leave out-parameters untouched on failure.** The caller should never have to guess whether a partial result was written.
5. **Don't lose the original error.** Cleanup failures must not overwrite the real cause.
6. **The error path must not allocate.**
7. **Errors and diagnostics go to `stderr`** (Module 13), and the process exits with a non-zero status.

8. **Test the error paths.** Fault injection plus LSan, on every failure branch.
9. **`assert` for bugs, checks for conditions.** Ask: could this happen with valid input?
10. **Never call `exit()` from a library.** Return the error and let the application decide.

Module 15 Rules

#	Rule
1	Error-code return + out-parameter is the default for library code
2	Sentinels only when a value is genuinely impossible
3	Never read an out-parameter in the same expression as the call that fills it
4	`errno` is thread-local and is a macro — never `extern int errno`
5	`errno = 0` before; check the primary failure indication first
6	Save and restore `errno` around cleanup
7	`assert` for “impossible with valid input”; runtime checks for everything else
8	No side effects in `assert`; `_Static_assert` everything you can
9	X-Macro your error table + `_Static_assert` the count
10	Carry both a machine code and a human detail string
11	The error path must not allocate
12	Cleanup errors must not overwrite the original cause
13	Document every error code each function can return
14	Test error paths with fault injection under LSan
15	Libraries return errors; only applications call `exit`

Practice

1. **Convention comparison.** Implement the same `parse_ipv4(const char *, uint32_t *)` four times — sentinel, `bool` + out, error code, and `errno` style. Write a caller for each. Which reads best? Which catches the most bugs?

2. **Evaluation-order bug.** Reproduce the `printf("%d %d", f(&x), x)` trap. Then find one in real code by searching for out-parameters passed inside a larger expression.

3. **`errno` discipline.** Write a function that calls three libc functions in sequence and correctly reports which one failed and why — including saving `errno` across cleanup. Test it by making each of the three fail in turn.

4. **`strerror_r` archaeology.** Write a program using `strerror_r` under `-D_GNU_SOURCE` and under `-D_POSIX_C_SOURCE=200809L`. Observe the two different signatures and the two different behaviours. Then write a wrapper that works correctly under both.

5. **Build the error library.** Implement `err.h`/`err.c` from §15.4 in full. Add `err_code()` returning the last code, error chaining (appending a cause rather than overwriting), and an `err_print()` that writes the full chain to `stderr`. Verify with ASan under 4 concurrent threads that the thread-local storage behaves correctly.

6. **`TRY` macro variants.** Implement `TRY`, `TRY_GOTO`, and `TRY_CLEANUP(expr, label)`. Then write a six-step function using them, and verify with LSan that none of the six failure points leaks anything.

7. **Fault-injection harness.** Add an `#ifdef FAULT_INJECT` global counter to an allocating function so that the *N*th allocation fails. Write a loop that runs the function with $N = 1 \ldots 20$ and asserts that LSan reports zero leaks each time. This is how real projects test out-of-memory paths.

8. **`assert` audit.** Take any program you have written during this course. For every `assert`, decide whether it should be an `assert` or a runtime check, using the rule in §15.3. Convert the ones that are wrong. Then build with `-DNDEBUG` and confirm that nothing important disappeared.

9. **Error-path leak hunt.** Write a function that acquires 5 resources with 5 possible failure points. Deliberately point one `goto` at the wrong label, and confirm that LSan finds it. Then write a small script that runs all 6 paths automatically.

10. **API documentation.** Take the `config_load` doc comment from §15.7 and write the full implementation, matching it exactly — including "`out` is untouched on failure." Then write a test that verifies each documented error code is actually returned in the documented situation.

Module 15 in Summary

C reports errors through ordinary values, so error handling is a convention you choose and enforce. Return a status code and pass results through out-parameters by default; use sentinels only when a value is truly impossible, `bool` when the reason does not matter, and `errno` only to match libc. Never read an out-parameter in the same expression as the call that fills it, and beware the clash between "0 means success" and "0 means false."

`errno` is a per-thread macro, meaningful only after a function reports failure, and it must be preserved across cleanup. `assert` is for conditions that are impossible in a correct program and disappears under `NDEBUG`; everything that valid input or the environment can cause needs a real check, and everything checkable at compile time belongs in `_Static_assert`.

A good error API pairs a machine-readable code with a human-readable, per-thread detail string, generated from an X-Macro table, recorded with file and line, and propagated with `TRY` — without ever allocating on the error path. Combined with the `goto` cleanup chain, which records the first failure and never lets a cleanup error overwrite it, and with fault injection under LeakSanitizer to prove that every error path releases everything, this gives C programs robust, debuggable failure handling. `setjmp`/`longjmp` is not a substitute: it unwinds nothing.

In Module 16, we confront the subject that has been in the background of every module so far: Undefined, Unspecified, and Implementation-Defined Behaviour.

Module 16 — Undefined, Unspecified, and Implementation-Defined Behavior

In this module:

- The distinction between the three
- Common UB: signed overflow, out-of-bounds access, null dereference, uninitialized reads, strict aliasing violations, data races, invalid `free`, shifting by ≥ width, modifying string literals, misaligned access
- How optimizers exploit UB (why "it worked on my machine" fails)

All output in this module is real GCC 13.3 output.

This module changes how you read C. The phrase "undefined behaviour" has appeared in almost every module so far — in signed overflow, in sequence points, in pointer arithmetic, in `realloc`, in `printf` formats. Here, all of it comes together into a single framework: what the standard means by leaving something undefined, what compilers are entitled to do as a result, what they actually do, and how professional C programmers protect themselves.

16.1 The Three Categories (Four, Actually)

The standard defines four levels of "the language doesn't fully pin this down." They are not interchangeable, and the differences between them matter enormously.

Category	Definition	Documented?	Consistent?	Portable?
Implementation-defined	Multiple behaviours allowed; the implementation must choose one *and document it*	✓ yes	✓ yes	⚠ per platform
Unspecified	Multiple behaviours allowed; the implementation need not choose consistently or document its choice	✗ no	✗ no	✗ no
Undefined	No requirements whatsoever. The standard imposes nothing.	—	—	✗ the program is invalid
Locale-specific	Behaviour depends on the current locale	✓	✓	⚠ per locale

Implementation-defined — measured

```
sizeof(int)          = 4
char is signed?      = yes
-7 >> 1              = -4              (arithmetic vs logical shift)
(int)3000000000u     = -1294967296     (out-of-range conversion)
&main prints as      = 0x56530819f169  (pointer→integer representation)
```

Your compiler's manual documents each of these choices. A program that depends on them is valid and well defined — it simply gives a different answer on a different platform. You can write portable code by not depending on the choice at all, or by testing which choice you have (`#if CHAR_MIN < 0`, for instance).

Other examples: the number of bits in a byte (`CHAR_BIT`), whether bitfields may straddle storage units, the underlying type of an enum (before C23), and whether `>>` on a negative value is an arithmetic shift.

Unspecified — measured

```c
int i = 0;
printf("%d %d\n", i++, i++);
```

```
argument eval order: 1 0
^ which i++ ran first? unspecified. GCC does right-to-left.
```

For unspecified behaviour, the compiler picks one of the allowed possibilities, need not tell you which, and may pick differently for two calls in the same program. Your program is still valid — you just cannot predict which of the permitted outcomes you will get.

(Note: this particular example is actually *undefined*, not merely unspecified, because `i` is modified twice without an intervening sequence point — Module 4, §4.7. The *order* of the argument evaluations is unspecified; the *double modification* is UB. It is shown here because it makes the evaluation order visible.)

Other examples of unspecified behaviour: the order of evaluation in `f() + g()`, the values of padding bytes, whether two identical string literals share storage, and the order in which the initializer expressions of a structure are evaluated.

Undefined — the important one

> **Undefined behavior:** behavior … for which this document imposes no requirements.

A note in the standard adds that possible undefined behaviour ranges from ignoring the situation completely with unpredictable results, through behaving in some documented manner characteristic of the environment (with or without a diagnostic message), to terminating translation or execution (with a diagnostic).

The crucial reframing: **UB is not "the result is unpredictable." It is "the standard does not define what this program means."** A program that contains undefined behaviour on an executed path has no defined behaviour at all — not just at that point, but for the entire execution, including what happened before it.

That is why the optimiser is allowed to do what §16.3 shows.

16.2 The Undefined Behaviour Catalogue

The C standard's Annex J.2 lists more than 200 cases of undefined behaviour. Here are the ones that actually happen in practice, organised by the module where you met them.

Arithmetic

UB	Module	Note
Signed integer overflow	2	Unsigned arithmetic wraps and is defined; signed is not
`INT_MIN / -1`, `INT_MIN % -1`	2	Also raises `SIGFPE` on x86
Division or modulo by zero	2	Integer only; floating-point division by zero gives ±Inf
`x << n` or `x >> n` with `n >= width(x)` or `n < 0`	4	x86 masks a 32-bit shift count to 5 bits, so you get `x << 0`
Left-shifting a negative value	4	Still UB in C23 (C++20 defined it)
Float→int conversion when out of range	2	`(int)1e20`

Memory

UB	Module	Note
Out-of-bounds array access	7	Read or write
Null pointer dereference	9	Not "a segfault" — see §16.3
Reading an uninitialized automatic variable	3	Statics are guaranteed zeroed
Use after free, double free, freeing a non-`malloc` pointer	10	
Dangling pointer to a dead local	3, 9	
Computing `arr - 1` or `arr + n + 1`	9	Even without dereferencing
Comparing `</>` between unrelated objects	9	Works on flat-memory machines, but not defined
Misaligned access	9	Faults on some ARM/SPARC
Modifying a string literal	8, 10	`char *s = "hi"; s[0]='H';`

UB	Module	Note
Casting away `const` and writing	2, 9	
Overlapping `memcpy` (a `restrict` violation)	8	

Types

UB	Module	Note
Strict aliasing violation	2	Accessing an object through an incompatible type
Calling a function through a mismatched function-pointer type	6	
`va_arg` with the wrong type	6	Including `char` or `float`
Reading a union member with a trap representation	11	Plain union punning is legal in C

Sequencing and control flow

UB	Module	Note
Two modifications between sequence points (`i = i++`)	4	
Falling off the end of a non-`void` function and using the value	5	
`longjmp` to a function that has already returned	14	
Reading non-`volatile` locals modified after `setjmp`	14	Formally indeterminate values
An infinite loop with no side effects	16	Formally “may be assumed to terminate” — see §16.3

Library and concurrency

UB	Module	Note
Wrong `printf`/`scanf` format specifier	13	
`ctype` function with a negative argument	14	
`str*` on a non-NUL-terminated buffer	8	
Inconsistent `qsort` comparator	14	Can crash
Data race (unsynchronised concurrent access, at least one a write)	17	

UB	Module	Note
Non-async-signal-safe function in a signal handler	14, 24	

The ones that surprise people

- **`NULL + 0` and `NULL - NULL` are UB in C.** C++ defines them, and a change adopted for the standard after C23 makes them defined in C too — but not in C23 or earlier.
- **Declaring `main` as `void main()`.** It is outside the two forms the standard guarantees; unless your implementation documents it, the behaviour is undefined.
- **Using an identifier from the reserved namespace** — names beginning with an underscore at file scope, names beginning with a double underscore or an underscore and a capital letter anywhere, and names that future library versions reserve, such as those starting with `str`, `is`, or `E` followed by a digit or capital letter.
- **`sizeof` on a VLA evaluates its operand** (Module 7) — so side effects in it happen, unlike every other use of `sizeof`. (`_Generic`, by contrast, never evaluates its controlling expression.)
- **Recursive `main`** is perfectly legal in C — and not allowed at all in C++.

16.3 How Optimizers Exploit UB ⚠

The compiler's reasoning is always the same:

> "This program contains an operation that would be undefined if condition X held. A correct program never invokes undefined behaviour. Therefore I may assume X is false — everywhere — and optimise accordingly."

This is not compiler writers being hostile. It is the only way to make `a[i]` fast without a bounds check, `a + b` fast without an overflow check, and pointer arithmetic fast without validity checks. Every one of those checks would cost time on every operation in every program; instead, the language makes the checks the programmer's responsibility and lets the compiler assume they have been done.

Case 1 — A null check deleted after a dereference

This is the pattern behind CVE-2009-1897, a real Linux kernel privilege-escalation bug.

```c
int use_after_deref(struct S *p)
{
    int v = p->x;          /* deref: compiler now ASSUMES p != NULL */
    if (p == NULL) {       /* ...so this branch is unreachable */
        log_msg("null pointer!");
        return -1;
    }
    return v;
}
```

At `-O0`, the check is there:

```
use_after_deref:
    mov  rax, QWORD PTR -24[rbp]
    mov  eax, DWORD PTR [rax]     ; p->x
    mov  DWORD PTR -4[rbp], eax
    cmp  QWORD PTR -24[rbp], 0    ; the null check
    jne  .L2
    lea  rax, .LC0[rip]
    call log_msg@PLT              ; the error handler
    mov  eax, -1
```

At `-O2`, it is gone:

```
use_after_deref:
    endbr64
    mov eax, DWORD PTR [rdi]
    ret
```

The entire null check and its error path have vanished. The function is now a bare load. The compiler's reasoning: "`p->x` was evaluated; if `p` were null, that would already be undefined behaviour; so `p` is not null; so `p == NULL` is always false."

In the kernel case, the dereference sat in a variable's initializer at the top of a function, before the null check. On that system, page 0 could be mapped with `mmap`, so an attacker could arrange for the "impossible" null pointer to point at memory they controlled — and the check that would have stopped them had been optimised away. (The kernel responded by building with `-fno-delete-null-pointer-checks`, and by tightening the protections that stop user processes from mapping low addresses.)

> **Lesson:** check *before* you dereference, not after. A null check after a dereference isn't merely redundant — it is deleted.

Case 2 — A non-terminating loop turned into a return

C11 §6.8.5p6 says, in essence, that an iteration statement whose controlling expression is not a constant expression, and which performs no input/output, accesses no `volatile` objects, and performs no synchronisation or atomic operations in its body, may be assumed by the implementation to terminate.

```
static int find_never(int target)
{
    int i = 0;
    while (i != target) i += 2;   /* never terminates for odd target */
    return i;
}
int caller(int t) { return find_never(t); }
```

At `-O2`:

```
caller:
    endbr64
    mov eax, edi
    ret
```

`caller(3)` returns 3 immediately. The loop — which, mathematically, runs forever for an odd target — has been deleted entirely. The compiler reasoned: "this loop may be assumed to terminate; if it terminates, then `i == target`; so return `target`."

Add a side effect to the body, and the loop is preserved:

```
caller2:
    ...
    call side@PLT   ; the loop is real now
```

(A loop with a *constant* controlling expression, such as `while (1)` or `for (;;)`, is exempt from this rule — so an intentional infinite loop, such as a firmware main loop, is safe.)

Case 3 — Signed overflow (Module 2, recap)

```
int check(int x) { if (x + 1 > x) return 1; return 0; }
```

```
check:
    mov eax, 1   ; the entire comparison deleted
    ret
```

Case 4 — Strict aliasing (Module 2, recap)

The same source, three builds, three answers:

```
-O0                        → 0
-O2 (strict aliasing on)   → 1
-O2 -fno-strict-aliasing   → 0
```

Case 5 — Dead-store elimination of a secret wipe (Module 10, recap)

```
handle_password:
    endbr64
    ret   ; strcpy AND memset both deleted
```

(Strictly, this one is not UB exploitation — it is the ordinary "as-if" rule removing stores that no correct program could observe. It belongs here because the mechanism, and the surprise, are the same.)

"UB time travel"

Because UB makes the whole execution undefined, a compiler may eliminate or alter code that appears *before* the UB — including observable side effects such as I/O. If a path is certain to reach undefined behaviour, the compiler may treat the whole path as impossible.

The canonical demonstration is Clang's handling of a function pointer: if a `static` function pointer is initially null, and the only value ever assigned to it anywhere in the program is the address of `evil`, then calling it while it is still null would be UB — so Clang may assume that every call happens after the assignment, and emit a direct call to `evil`, running code that no reachable path ever actually assigned.

Honest reporting: GCC 13 did not perform this transformation in the test for this book, nor did it delete code placed before a divide-by-zero or an out-of-range array access:

```
call:
    jmp [QWORD PTR fp[rip]]   ; GCC kept the indirect call
```

Compilers differ in how aggressively they exploit UB, and they grow more aggressive over time. “GCC 13 doesn’t do this” is not a guarantee about GCC 14, or Clang, or whatever compiler your code is built with in three years’ time. The standard permits it; that is the only thing that matters.

Why “it worked on my machine” fails

```
Your build:  -O0, glibc 2.39, x86-64, this GCC, small inputs
Their build: -O3 + LTO, musl, ARM64, Clang 19, adversarial inputs
```

Every one of those axes can change whether a program with UB behaves as you intended. The specific ways it breaks:

1. **Optimisation level.** Nearly every demonstration in this module behaves differently at `-O0` and `-O2`.
2. **Compiler and version.** Optimisers get smarter. Code that “worked” for a decade breaks on upgrade.
3. **Architecture.** Misaligned access faults on some ARM configurations and is merely slow on x86. The signedness of `char` flips. Unaligned loads that x86 tolerates trap elsewhere.
4. **Inlining.** UB inside a function may only become exploitable once the compiler can see the call site — so it can appear when an unrelated caller changes.
5. **LTO.** Link-time optimisation’s whole-program analysis exposes UB across translation-unit boundaries.
6. **Timing.** Data races and uninitialized reads depend on load and scheduling.
7. **Input.** The UB path may only be reachable with input an attacker supplies.

> **“It works” is not evidence of correctness in C.** It is evidence that *this* compilation, on *this* machine, with *these* inputs, produced acceptable output.

16.4 The Tooling Matrix

One program containing 12 distinct cases of undefined behaviour was run plain, under UBSan, and under ASan, with the sanitizers at `-O0`:

#	UB	Plain run	UBSan	ASan
1	Signed overflow	silent	✓	—
2	Stack OOB write	crash	✓	✓
3	Heap OOB write	silent	—	✓

#	UB	Plain run	UBSan	ASan
4	Null deref	crash	✓	✓
5	Uninitialized read	silent	—	—
6	Strict aliasing	silent	—	—
7	Double free	silent	—	✓
8	Shift ≥ width	silent	✓	—
9	Modify string literal	crash	—	✓
10	Misaligned access	silent	✓	—
11	Use after free	silent	—	✓
12	Division by zero	silent	✓	—

Three things to take from this table:

1. **8 of the 12 are completely silent without tooling.** They do not crash. They produce wrong answers — or right answers today and wrong answers after a recompile.
2. **UBSan and ASan are complementary, not redundant.** UBSan covers arithmetic, alignment, and type-related UB; ASan covers memory UB. You need both.
3. **Two are caught by neither.** Uninitialized reads need MemorySanitizer (Clang only) or Valgrind. Strict-aliasing violations are caught by no runtime tool at all — only by `-Wstrict-aliasing` at compile time, which misses plenty.

(Note that integer division by zero *usually* traps with `SIGFPE` on x86. It was silent in this test program, most likely because the result was never used and the division was optimised out, or because it was a floating-point division. Either way, UBSan reported it — which is exactly the point.)

⚠ Optimisation interacts with instrumentation

At `-O1`, ASan missed the heap out-of-bounds write and the double free: the optimiser had deleted the "dead" code — writes whose results were never read, allocations never otherwise used — before the sanitizer could instrument it. At `-O0`, both were caught.

> Run sanitizers at `-O0` or `-O1`. At higher levels, the optimiser may remove the very code you are testing. (Conversely, run *warnings* at `-O2` — `-Wuninitialized` and friends need the data-flow analysis that optimisation provides.)

The practical compromise used by most projects is `-O1` for sanitizer builds — fast enough to run a real test suite — combined with test code that actually uses its results, so the optimiser has nothing to delete.

The complete toolchain

Compile time:

```
-Wall -Wextra -Wpedantic -Wshadow -Wconversion -Wsign-conversion
-Wcast-qual -Wcast-align=strict -Wpointer-arith -Wwrite-strings
-Wstrict-aliasing=2 -Wnull-dereference -Warray-bounds=2
-Wstringop-overflow=4 -Wformat=2 -Wsequence-point -Walloca
-Werror=vla -Werror=return-type -Werror=implicit-function-declaration
```

Run time:

Tool	Catches	Cost
UBSan (`-fsanitize=undefined`)	Arithmetic, shifts, alignment, null, bounds, enum/bool range	~20%
ASan (`-fsanitize=address`)	Heap/stack/global overflow, use-after-free, double free, leaks	~2×
MSan (Clang, `-fsanitize=memory`)	Uninitialized reads	~3×
TSan (`-fsanitize=thread`)	Data races	~5–15×
Valgrind memcheck	Memory errors + uninitialized reads, with no recompile needed	~20–50×
`-D_FORTIFY_SOURCE=3`	Buffer overflows in libc calls	~free

> ⚠ ASan cannot be combined with TSan or MSan. Run separate builds.

Static analysis: clang-tidy, cppcheck, scan-build, Coverity, PVS-Studio. **Formal verification:** Frama-C, CBMC. **Fuzzing:** libFuzzer and AFL++ — the most effective way to reach UB paths that tests never exercise. Fuzzing combined with ASan and UBSan is the industry standard for finding memory-safety bugs: the fuzzer generates millions of unusual inputs, and the sanitizers turn any silent UB they trigger into an immediate, precise report.

The practical CI setup

```
# Build 1: maximum warnings, treated as errors
gcc -std=c17 -O2 $WARNINGS -Werror -c *.c

# Build 2: ASan + UBSan + LSan
gcc -std=c17 -O1 -g -fsanitize=address,undefined -fno-omit-frame-pointer ...
ASAN_OPTIONS=detect_leaks=1:detect_stack_use_after_return=1 \
UBSAN_OPTIONS=print_stacktrace=1:halt_on_error=1 ./tests

# Build 3: TSan (if threaded)
gcc -std=c17 -O1 -g -fsanitize=thread ... && ./tests

# Build 4: Valgrind (catches what ASan doesn't)
valgrind --error-exitcode=1 --leak-check=full --track-origins=yes ./tests

# Build 5: a second compiler
clang -std=c17 -O2 $WARNINGS -Werror -c *.c
```

`halt_on_error=1` matters: by default UBSan prints a report and carries on, which is easy to miss in a long CI log. Making it fatal turns every report into a failed build.

Building with two compilers is genuinely valuable — they exploit different UB and warn about different things.

16.5 How to Think About This

The mental shift

Before: “UB means the result is unpredictable, so I’ll check what my compiler does.”

After: “UB means my program is not a valid C program. The compiler is under no obligation to compile it into anything resembling what I wrote.”

There is no such thing as “my compiler’s UB.” There is only *this compilation* of an invalid program.

Practices that follow

1. **Never reason from observed behaviour.** “I tested it and it wraps” is not a fact about your program; it is a fact about one build.
2. **Never use `-fwrapv` or `-fno-strict-aliasing` as a fix.** They are legitimate escape hatches for legacy code you cannot audit, but they hide bugs and cost performance. Fix the code.
3. **Learn the list.** The catalogue in §16.2 is perhaps 30 items that you will actually meet. That is a weekend of study for a career of avoided bugs.
4. **Assume every warning is a real bug** until you have proven otherwise.
5. **Sanitizers are not optional.** They are the main reason C can be written safely at all today.
6. **Fuzz.** Sanitizers only catch UB on paths that actually execute. Fuzzing finds the paths.
7. **When you can’t avoid it, isolate it.** If you must do something the standard does not bless (`container_of`, union punning shared with C++, `dlsym` casts), put it in one documented, commented function.

The counterargument, fairly stated

Some experienced engineers — notably in the Linux kernel and embedded worlds — argue that aggressive exploitation of UB was a mistake: that compilers should have treated UB as “do the obvious thing on this hardware” rather than “assume it never happens,” and that the performance gains rarely justify the security cost.

There is real substance to this. Exploiting signed overflow, in particular, buys little on modern hardware and has contributed to genuine CVEs. That is why `-fwrapv` and `-fno-strict-aliasing` exist, and why the kernel builds with them. Newer standards have moved a little in the same direction too: C23 made two’s complement mandatory — though it left signed overflow undefined.

But the standard says what it says, and your compiler implements the standard. Whatever you think the rules *should* be, the rules you must program against are the ones in §16.1.

Module 16 Rules

#	Rule
1	Implementation-defined = documented and consistent; unspecified = neither; undefined = your program is invalid
2	UB makes the entire execution undefined, not just one operation
3	The compiler assumes UB never happens, and deletes code accordingly
4	Check before you dereference — a null check after a dereference is deleted
5	A loop with no side effects (and a non-constant condition) may be assumed to terminate
6	8 of 12 common UBs are completely silent without tooling
7	UBSan and ASan are complementary — run both
8	Uninitialized reads need MSan or Valgrind; strict aliasing has no runtime tool
9	Run sanitizers at `-O0`/`-O1`; run warnings at `-O2`
10	Build with two compilers; fuzz with sanitizers on
11	`-fwrapv`/`-fno-strict-aliasing` are escape hatches, not fixes
12	“It worked on my machine” is not evidence of correctness

Practice

1. **Category sorting.** Classify each of the following as UB, unspecified, implementation-defined, or well defined — then verify with the standard or a compiler:

   `i = i++;` · `printf("%d %d", f(), g());` · `-5 >> 1` · `(char)200` · `1u - 2u` · `1 - 2` (as `int`, where the 1 is `INT_MIN`) · `int a[5]; &a[5]` · `&a[6]` · `sizeof(void)` · `free(p); free(p);` · `struct S s1, s2; memcmp(&s1, &s2, sizeof s1)`

2. **Reproduce the null-check deletion.** Build the §16.3 example and dump the assembly at `-O0`, `-O1`, `-O2`, `-Os`, and `-O3`. Find the optimisation level at which the check disappears. Then move the check before the dereference and confirm that it survives.

3. **The vanishing loop.** Build `find_never` and call it with an odd argument at `-O0` (it hangs) and at `-O2` (it returns immediately). Then make `i` `volatile` and check again. Explain the difference using C11 §6.8.5p6.

4. **Build your own tool matrix.** Take the 12-case program from §16.4 and add: an `alloca` overflow, `qsort` with an inconsistent comparator, `ctype` with a negative `char`, a `printf` format mismatch, and an overlapping `memcpy`. Run it plain and under UBSan, ASan, MSan (if you have Clang), and Valgrind. Publish the table.

5. **The `-O1` blind spot.** Reproduce the finding that ASan missed the heap out-of-bounds write at `-O1` but caught it at `-O0`. Explain why, and decide what optimisation level your CI should use.

6. **Two-compiler diff.** Take any non-trivial program you have written during this course. Build it with `gcc -Wall -Wextra -Werror` and with `clang -Wall -Wextra -Weverything`. Collect the warnings each finds that the other doesn't.

7. **Fuzz something.** Write a small parser (the `parse_port` from Module 15 will do). Wrap it in a libFuzzer harness, build with `-fsanitize=fuzzer,address,undefined`, and run it for 60 seconds. Fix whatever it finds.

8. **UB archaeology.** Study a real bug: CVE-2009-1897 (null-check deletion), CVE-2012-2100 (a crafted ext4 filesystem triggering an arithmetic fault in the kernel), or the 2014 "goto fail" bug (Module 5). Read the patch. Explain in your own words what went wrong, whether undefined behaviour was involved — note that "goto fail" was a plain logic error, not UB — and which tool or warning would have caught it.

9. **Audit your own code.** Take the largest program you have written so far. Run it under every tool in §16.4. Fix everything. Then add the full warning set to your Makefile with `-Werror`, and fix that too.

10. **The counterargument, seriously.** Read Linus Torvalds's and John Regehr's positions on UB exploitation. Write a paragraph arguing each side. Then decide what your team's policy should be: `-fwrapv` or not? `-fno-strict-aliasing` or not? Be able to defend the decision.

Module 16 in Summary

C distinguishes behaviour that is implementation-defined (documented and consistent), unspecified (one of several outcomes, undocumented and possibly inconsistent), locale-specific, and undefined — where the standard imposes no requirements at all and the program has no meaning. Annex J.2 lists more than 200 undefined behaviours; the few dozen that matter in practice span arithmetic, memory, types, sequencing, and the library, and every one of them has appeared earlier in this book.

Optimisers assume undefined behaviour never happens, and reason backwards from that assumption: null checks after a dereference vanish, side-effect-free loops are assumed to terminate, overflow checks are folded to constants, aliasing assumptions change results, and in principle even earlier code on a doomed path may be removed. Different compilers, versions, optimisation levels, architectures, and inputs therefore make the same UB behave differently, so "it worked" proves nothing.

The defence is a toolchain: extensive warnings at `-O2`; UBSan and ASan (and TSan, MSan, or Valgrind as needed) at low optimisation; two compilers; and fuzzing with sanitizers enabled. Above all, treat UB as a bug in your program rather than a quirk of your compiler — and where you must step outside the standard, isolate and document it.

In Module 17, we turn to Concurrency in Standard C — threads, atomics, the memory model, and the data races that are the concurrent form of undefined behaviour.

Module 17 — Concurrency in Standard C

In this module:

- C11 `<threads.h>`, `mtx_t`, `cnd_t`
- `<stdatomic.h>`, atomic types, `memory_order_*`
- The C memory model, data races, happens-before
- `volatile` ≠ thread safety (a very common misunderstanding)

All output in this module is real, from GCC 13.3 and glibc 2.39, including verification with ThreadSanitizer.

This is the final module of the core language section. For most of C's history, the language said nothing at all about threads: a C program was, as far as the standard was concerned, a single sequence of operations, and multithreaded programs relied on libraries (POSIX threads, Win32 threads) and compiler-specific guarantees. C11 changed that. It added a formal *memory model* that defines what it means for threads to share memory, a portable thread library, and atomic types. This module covers all three — and the most misunderstood topic in C concurrency, §17.3: `volatile` is not thread safety.

17.1 `<threads.h>` — C11 Threads

```c
#include <threads.h>

static int worker(void *arg) { ... return exit_code; }

thrd_t th;
thrd_create(&th, worker, &task);
int result;
thrd_join(th, &result);
```

```
thread 0 finished, result=500000
joined thread 0, returned 0
...
```

`thrd_create` starts a new thread running `worker(&task)`. The new thread runs concurrently with the one that created it. `thrd_join` waits for a thread to finish and collects the `int` it returned. Every thread must eventually be either joined or *detached* (`thrd_detach`, which tells the system to clean it up automatically when it ends); otherwise its resources leak, much like an unfreed allocation.

The API surface

Category	Functions
Threads	`thrd_create`, `thrd_join`, `thrd_detach`, `thrd_exit`, `thrd_sleep`, `thrd_yield`, `thrd_current`, `thrd_equal`

Category	Functions
Mutexes	`mtx_init` (`mtx_plain` | `mtx_recursive` | `mtx_timed`), `mtx_lock`, `mtx_trylock`, `mtx_timedlock`, `mtx_unlock`, `mtx_destroy`
Condition variables	`cnd_init`, `cnd_wait`, `cnd_timedwait`, `cnd_signal`, `cnd_broadcast`, `cnd_destroy`
One-time init	`once_flag`, `call_once`
Thread-local with destructor	`tss_t`, `tss_create`, `tss_get`, `tss_set`, `tss_delete`
Return codes	`thrd_success`, `thrd_error`, `thrd_busy`, `thrd_nomem`, `thrd_timedout`

Plus the `_Thread_local` / `thread_local` storage duration from Module 3.

⚠ The portability reality

`<threads.h>` is optional (`__STDC_NO_THREADS__`), and for most of its history MSVC did not provide it (Microsoft added it only in Visual Studio 2022 version 17.8, in 2023). glibc only added it in version 2.28, in 2018. In practice, most real C code uses POSIX threads (pthreads) on Unix and Win32 threads on Windows, often behind a thin abstraction layer.

Note also that on Linux, `<threads.h>` is a thin wrapper around pthreads, and ThreadSanitizer's interception of it was not reliable in the setup used for this book: the `thrd_create` test crashed TSan outright, while the identical pthreads version worked perfectly. For TSan-verified code, use pthreads directly.

The concepts below are identical either way; the pthreads functions (`pthread_create`, `pthread_mutex_lock`, `pthread_cond_wait`, …) map almost one-to-one onto the C11 ones.

The mutex + condition variable pattern

A *mutex* ("mutual exclusion") ensures that only one thread at a time executes the code between `mtx_lock` and `mtx_unlock` — the *critical section*. A *condition variable* lets a thread sleep until some condition, protected by that mutex, might have become true. Together they implement the classic bounded producer–consumer queue:

```
static int   buf[CAP], head = 0, tail = 0, count = 0;
static bool  done = false;
static mtx_t m;
static cnd_t not_full, not_empty;

/* producer */
mtx_lock(&m);
while (count == CAP) cnd_wait(&not_full, &m);   /* WHILE, not if */
buf[tail] = v; tail = (tail+1) % CAP; count++;
cnd_signal(&not_empty);
mtx_unlock(&m);

/* consumer */
mtx_lock(&m);
while (count == 0 && !done) cnd_wait(&not_empty, &m);
if (count == 0 && done) { mtx_unlock(&m); break; }
int v = buf[head]; head = (head+1) % CAP; count--;
```

```
cnd_signal(&not_full);
mtx_unlock(&m);
```

```
produced 1 (count=1)
  consumer 1 got 1 (count=0)
produced 2 (count=1)
produced 3 (count=2)
  consumer 2 got 2 (count=3)
...
```

A producer waits while the buffer is full, adds an item, and wakes a consumer. A consumer waits while the buffer is empty (and production is not finished), removes an item, and wakes a producer. (The trace's `count` values come from `printf`s that run after the mutex has been released, so another thread may already have changed `count` by the time it is printed — as in the `count=3` line. Printing shared state outside its lock shows you a snapshot, not a consistent view.)

The four rules of condition variables

1. **Always wait in a `while` loop, never an `if`.** `cnd_wait` may return *spuriously* — without anyone having signalled (POSIX explicitly allows this) — and even without spurious wakeups, another thread may consume the item between the signal and your thread reacquiring the mutex. The loop re-checks the condition every time.
2. **`cnd_wait` atomically releases the mutex and sleeps, then reacquires the mutex before returning.** You must hold the mutex when calling it. The atomic release is what prevents a "lost wakeup," where the signal arrives in the gap between checking the condition and going to sleep.
3. **The predicate is protected by the mutex.** The condition variable carries no state of its own — it is purely a wakeup mechanism. The state (`count`, `done`) lives in ordinary variables that are only touched with the mutex held.
4. **`cnd_signal` wakes one waiter; `cnd_broadcast` wakes all of them.** Use broadcast when different waiters are waiting for different predicates, or on shutdown, when everyone must notice `done`.

Also: calling `mtx_destroy` on a locked mutex is undefined behaviour, locking an `mtx_plain` mutex twice from the same thread is undefined behaviour, and unlocking a mutex you do not own is undefined behaviour.

17.2 Data Races

> A **data race** occurs when two threads access the same memory location concurrently, at least one of the accesses is a write, and the accesses are not synchronised.
>
> **A data race is undefined behaviour.** Not "you get a torn value" — undefined, with everything Module 16 implies.

```
static int counter = 0;
void *worker(void *a) { for (int i=0;i<100000;i++) counter++; return NULL; }
```

`counter++` is three operations — load, add, store. If two threads both load the value 41, both add one, and both store 42, one increment has been lost. Module 3 showed exactly this happening.

TSan finds it immediately:

```
WARNING: ThreadSanitizer: data race (pid=698)
  Write of size 4 at 0x55c43103f014 by thread T2:
  Previous write of size 4 at 0x55c43103f014 by thread T3:
  Location is global 'counter' of size 4 at 0x55c43103f014
SUMMARY: ThreadSanitizer: data race racep.c:4 in worker
```

The report names the variable, the two conflicting accesses, and the line of code. TSan works by tracking which memory accesses are ordered by synchronisation (§17.5), so it reports the race whether or not the lost update actually happened on that particular run.

TSan is the only reliable way to find data races. Races are timing-dependent: they pass a thousand test runs and then fail in production under load.

```
gcc -fsanitize=thread -O1 -g prog.c -o prog -lpthread
```

Its cost is roughly a 5–15× slowdown and 5–10× more memory. It cannot be combined with ASan — run separate builds.

17.3 `volatile` Is NOT Thread Safety ⚠

This is the single most common concurrency misconception in C, and it can be proved three ways.

Experiment: 8 threads × 500,000 increments, three variables

```
static int          plain = 0;
static volatile int vol   = 0;
static _Atomic int  atom  = 0;
```

```
--- run 1 ---
expected          4000000
plain int++       4000000   lost 0
volatile int++    3358953   lost 641047    ← volatile LOST 641,047 updates
atomic_fetch_add  4000000   lost 0

--- run 2 ---
volatile int++    2673045   lost 1326955   ← lost a third of them

--- run 3 ---
volatile int++    4000000   lost 0         ← and sometimes it's fine!
```

The `volatile` counter lost anywhere from none to a third of its updates, varying from run to run — the signature of a race.

Why `volatile` performed worse than a plain `int` — the assembly

```
w_plain:
    add DWORD PTR plain[rip], 500000   ← the optimizer collapsed the whole loop
```

```
    ret

w_vol:
    mov eax, DWORD PTR vol[rip]        ← load
    add eax, 1                         ← modify      3 separate steps,
    mov DWORD PTR vol[rip], eax        ← store       500,000 times
    sub edx, 1
    jne .L4

w_atom:
    lock add DWORD PTR atom[rip], 1    ← the LOCK prefix. THIS is atomicity.
```

Three lessons in one dump:

1. **`plain` "won" by accident.** The optimiser turned 500,000 separate increments into a single `add [plain], 500000`. That is still a non-atomic read-modify-write, and still a data race — it just has a tiny window (one instruction per thread instead of half a million), so it rarely loses. A correct answer is not evidence of correctness.
2. **`volatile` made the race worse,** by forcing a real load, modify, and store on every iteration — a wide window, hit constantly. `volatile` prevented an optimisation that was accidentally hiding the bug.
3. **Only `_Atomic` emits `lock`.** That prefix is what makes a read-modify-write indivisible on x86: it gives the core exclusive ownership of the cache line for the duration of the instruction, so no other core can slip in between the read and the write.

What `volatile` actually guarantees

`volatile` gives you	`volatile` does NOT give you
Each access in the source produces an actual memory access	Atomicity — a `volatile` read-modify-write is still three operations
The compiler won't cache the value in a register	Ordering — no barriers are emitted relative to non-`volatile` accesses
The compiler won't elide or duplicate accesses	Any protection from CPU reordering or store buffers
	Cache coherence beyond what the hardware already provides

`volatile` is a directive to the *compiler* about observable accesses. `_Atomic` is a contract with the *memory model*, binding both the compiler and the hardware. They solve different problems.

Where the confusion comes from

Java and C# gave `volatile` acquire/release semantics, making it useful for thread communication. C did not. Programmers carrying that intuition into C write broken code. Likewise, pre-C11 code — including older parts of the Linux kernel — used `volatile` plus hand-written memory barriers, because the language had no atomics. That was a historical workaround, not a pattern to copy.

> **`volatile` is for memory-mapped hardware registers, `sig_atomic_t` flags set by signal handlers, and locals that must survive `longjmp`. `_Atomic` (or a mutex) is for threads.**

17.4 `<stdatomic.h>`

```c
#include <stdatomic.h>

_Atomic int          counter = 0;
atomic_int           counter2 = 0;          /* same thing, typedef'd */
_Atomic(struct Foo)  big;                   /* any type can be atomic */
atomic_flag          f = ATOMIC_FLAG_INIT;  /* guaranteed lock-free */
```

Any type can be made atomic. The header provides convenient `typedef`s (`atomic_int`, `atomic_long`, `atomic_bool`, …) for the common ones, and `atomic_flag` — a simple test-and-set flag — which is the one atomic type guaranteed to be lock-free everywhere.

Operations

```c
atomic_store(&x, v);   atomic_load(&x);
atomic_exchange(&x, v);
atomic_fetch_add(&x, n);   atomic_fetch_sub / _or / _xor / _and
atomic_compare_exchange_strong(&x, &expected, desired);
atomic_compare_exchange_weak(&x, &expected, desired);   /* may fail spuriously — for loops */
atomic_thread_fence(order);
```

The *compare-exchange* operations (often called CAS, “compare-and-swap”) are the most powerful: they atomically check whether `x` still equals `expected` and, only if so, replace it with `desired`. On failure they write the current value of `x` into `expected`, so a retry loop has fresh data. The `_weak` form may fail even when the values match, which lets it compile to faster instructions on some architectures; it is the right choice inside a loop.

Plain operators work too: `counter++` on an `_Atomic int` is an `atomic_fetch_add` with sequentially consistent ordering. (But `counter = counter + 1` is *not* atomic — it is an atomic load followed by a separate atomic store, with a window between them.)

Lock-freedom: `ATOMIC_INT_LOCK_FREE` and friends report at compile time whether a type’s atomic operations are lock-free; `atomic_is_lock_free(&x)` reports it at run time. An `_Atomic` type that is not lock-free is implemented with a hidden mutex — correct, but not fast, and not usable in a signal handler. Large structures are the usual culprits.

The memory orders

Every atomic operation can take an explicit *memory order* (using the `_explicit` variants, such as `atomic_load_explicit`), which controls how it orders *other* memory accesses around it:

Order	Guarantee
`memory_order_relaxed`	Atomic, but no ordering with respect to any other operation
`memory_order_consume`	Data-dependency ordering — unusable in practice; compilers promote it to acquire
`memory_order_acquire`	On a load: no memory access after it in program order may be moved before it
`memory_order_release`	On a store: no memory access before it in program order may be moved after it
`memory_order_acq_rel`	Both, for read-modify-write operations
`memory_order_seq_cst`	The default. All `seq_cst` operations also appear in a single global total order agreed on by every thread

How they map to x86-64

```
load_relaxed  : mov eax, DWORD PTR x[rip]
load_acquire  : mov eax, DWORD PTR x[rip]      ← identical
load_seqcst   : mov eax, DWORD PTR x[rip]      ← identical
store_relaxed : mov DWORD PTR x[rip], edi
store_release : mov DWORD PTR x[rip], edi      ← identical
store_seqcst  : xchg edi, DWORD PTR x[rip]     ← the ONLY one that costs extra
rmw_relaxed   : lock xadd DWORD PTR x[rip], eax
```

x86-64 has a *strong* memory model: ordinary loads already have acquire semantics and ordinary stores release semantics, for free. Only a sequentially consistent store needs anything extra — the `xchg`, which acts as a full barrier.

And the measured cost of `fetch_add`:

```
seq_cst : 46.3 ms
relaxed : 47.0 ms (0.98x)   ← no difference
```

— because on x86, every read-modify-write uses `lock` regardless of the order requested.

> ⚠ **This is exactly why you must not tune memory orders by benchmarking on x86.** On ARM64, POWER, and RISC-V — which have *weak* memory models — each order maps to genuinely different instructions with genuinely different costs. And code that "works" with `relaxed` on x86, because the hardware silently provides stronger ordering than you asked for, can break on ARM.
>
> Use the default `seq_cst` unless you have measured a problem on the actual target architecture *and* can prove that the weaker order is correct.

The one case where `relaxed` is obviously right and obviously safe: a counter whose value is read only at the end (for statistics, say), with no other data depending on it.

17.5 The C Memory Model and Happens-Before

The formal machinery

- **Sequenced-before** — ordering within a single thread (Module 4's sequence points).
- **Synchronizes-with** — a release operation *synchronizes with* an acquire operation that reads the value it wrote.
- **Happens-before** — the transitive closure of sequenced-before and synchronizes-with: if A is sequenced before B, and B synchronizes with C, and C is sequenced before D, then A happens-before D.

> If two conflicting accesses (to the same location, at least one a write) are not ordered by happens-before, the program has a data race, and its behaviour is undefined.

That is the entire rule. Everything else — mutexes, atomics, fences, thread creation — is machinery for establishing happens-before edges.

The canonical release/acquire pattern

```c
static int           data = 0;       /* plain, NON-atomic */
static _Atomic bool  ready = false;

/* producer */
data = 42;                                                  /* 1 */
atomic_store_explicit(&ready, true, memory_order_release);  /* 2 */

/* consumer */
while (!atomic_load_explicit(&ready, memory_order_acquire))   /* 3 */
    ;
printf("%d\n", data);                                       /* 4 — guaranteed 42 */
```

```
consumer saw data = 42
```

The chain:

```
(1) data = 42      sequenced-before   (2) store-release(ready)
(2) store-release  SYNCHRONIZES-WITH  (3) load-acquire(ready) that reads it
(3) load-acquire   sequenced-before   (4) read of data
⟹ (1) HAPPENS-BEFORE (4) ⟹ no data race, value guaranteed
```

Note that `data` is a plain `int`. It does not need to be atomic: the happens-before edge established through the atomic flag makes the non-atomic accesses safe. This is the entire point of release/acquire — to *publish* arbitrary data, of any size, with a single atomic operation. The release store says "everything I wrote before this is ready"; the acquire load that sees it says "and I will see all of it."

TSan proves it

```
### release/acquire version ###
  warnings: 0

### same code with memory_order_relaxed on both ###
```

```
WARNING: ThreadSanitizer: data race (pid=803)
  Location is global 'data' of size 4
```

Change two enumeration constants and a correct program becomes undefined behaviour. With `relaxed`, the flag itself is still accessed atomically, but it no longer creates a synchronizes-with edge, so nothing orders the write of `data` before its read. TSan understands the memory model and reports exactly this — even on x86, where the hardware would probably have produced 42 anyway.

What else establishes happens-before

Mechanism	Edge
`mtx_unlock` → `mtx_lock` (same mutex)	release → acquire
`thrd_create`	Everything before the call happens-before the thread body
`thrd_join`	Everything in the thread happens-before the join returns
`cnd_signal` → `cnd_wait` return	Via the mutex
`call_once`	The initialization happens-before every subsequent call returns
Release store → acquire load of the same atomic	As above
`atomic_thread_fence` pairs	Explicit barriers

This is why mutexes work. `mtx_unlock` is a release and `mtx_lock` is an acquire, so everything the first thread did before unlocking is visible to the second thread after it locks.

Two things the memory model does and doesn’t guarantee

1. Adjacent variables are separate memory locations. C11 defines *memory locations* so that distinct scalar objects (and bitfields separated by a `:0` field) can be updated concurrently by different threads without a race — even if they share a cache line. (Before C11, some compilers would implement a write to one small variable by reading and rewriting a whole word containing its neighbour, which could corrupt a concurrent write to the neighbour. That was a real problem.)

But *false sharing* remains a **performance** problem: two threads updating adjacent variables in the same cache line cause that line to bounce between their cores’ caches on every write. Fix it with `alignas(64)` (Module 11).

2. Nothing about fairness or progress. The model does not promise that a thread spinning on a relaxed atomic will observe another thread’s write within any bounded time — only that it is eventually visible, in practice. Real code should not depend on the timing of such visibility.

Lock-free programming — a warning

```
int expected = old;
while (!atomic_compare_exchange_weak(&x, &expected, compute(expected)))
    ;   /* expected is updated with the current value on failure */
```

This *CAS loop* reads a value, computes a new one from it, and installs the new one only if nobody changed `x` in the meantime — retrying otherwise. CAS loops are the foundation of lock-free data structures. They are also where the *ABA problem* lives: `x` changes from A to B and back to A between your read and your CAS, so the CAS succeeds — but your reasoning about the state in between was wrong. (In a lock-free stack, A might be a node that was popped, freed, and reallocated at the same address.) The solutions — tagged pointers, hazard pointers, epoch-based reclamation — are genuinely hard.

> **Practical advice: use a mutex.** An uncontended mutex lock/unlock costs on the order of 20 ns, and it is correct. Lock-free algorithms are for measured hot paths, written by people who have read the papers, and verified with TSan plus model checking. The number of published, peer-reviewed lock-free queues that later turned out to be wrong should be sobering.

17.6 Practical Guidance

The hierarchy — use the simplest thing that works

1. **Don’t share.** Thread-local state, message passing, immutable data. No sharing, no races.
2. **Mutex.** Simple, correct, and fast enough for almost everything.
3. **Atomics with `seq_cst`.** For counters and flags.
4. **Atomics with explicit weaker orders.** Only with a measured need, on the real target.
5. **Lock-free data structures.** Only if you can prove correctness.

The checklist

```
/* One-time initialization */
static once_flag init_flag = ONCE_FLAG_INIT;
call_once(&init_flag, do_init);   /* NOT double-checked locking by hand */

/* Always release what you lock — the goto chain from Module 5 applies */
mtx_lock(&m);
if (bad) { rc = -1; goto out_unlock; }
...
out_unlock: mtx_unlock(&m);

/* Consistent lock ordering prevents deadlock */
/* Always lock A before B, everywhere in the program */
```

Double-checked locking — checking a flag without the lock, then locking and checking again — is a classic pattern that is broken without correct atomics and memory orders; `call_once` does it correctly for you. And a *deadlock* happens when thread 1 holds lock A and waits for B while thread 2 holds B and waits for A; a single global order for acquiring locks makes that cycle impossible.

- **Deadlock:** enforce a global lock ordering, and never call user callbacks while holding a lock (the callback may try to take a lock of its own).
- **`errno` is thread-local** (Module 15) — safe.
- **Not thread-safe:** `strtok`, `gmtime`/`localtime`, `strerror`, `rand`, `asctime`. Use the `_r` variants (Module 14).
- **A function-local `static`** makes a function non-reentrant (Module 3).
- **Signal handlers in a threaded program** need `volatile sig_atomic_t` or lock-free atomics — a mutex in a handler can deadlock, if the signal interrupts the very thread that holds it.
- **Only lock-free atomics are safe in signal handlers.** Check with `atomic_is_lock_free`.

The CI setup

```
# TSan build (separate — cannot combine with ASan)
gcc -std=c17 -O1 -g -fsanitize=thread -fno-omit-frame-pointer prog.c -o prog -lpthread
TSAN_OPTIONS=halt_on_error=1:second_deadlock_stack=1 ./tests

# Run the test suite many times — races are timing-dependent
for i in $(seq 1 100); do ./tests || break; done
```

Run threaded tests repeatedly, and under load (for example, alongside a CPU-heavy process). A race that fires once in a thousand runs in testing will fire constantly in production.

Module 17 Rules

#	Rule
1	A data race is undefined behaviour, not “a torn value”
2	`volatile` provides no atomicity and no ordering — it is not thread safety
3	Only `_Atomic` emits the `lock` prefix; only that is atomic
4	A correct-looking result proves nothing — run TSan
5	`cnd_wait` always goes in a `while` loop
6	The mutex protects the predicate; the condition variable carries no state
7	Default to `memory_order_seq_cst`; weaken only with measurement on the real target

#	Rule
8	Never tune memory orders by benchmarking on x86 — it hides the cost
9	Release/acquire publishes arbitrary non-atomic data with one atomic operation
10	`mtx_unlock` → `mtx_lock`, `thrd_create`, and `thrd_join` all create happens-before edges
11	`alignas(64)` on per-thread counters to avoid false sharing
12	Prefer no sharing > mutex > atomics > lock-free, in that order
13	`<threads.h>` is optional and was long absent from MSVC; pthreads is the portable reality
14	TSan cannot be combined with ASan — separate builds, and run tests repeatedly

Practice

1. **Reproduce the three-counter experiment.** Build the `plain`/`volatile`/`_Atomic` program with 8 threads. Run it 20 times and tabulate the losses. Then dump the assembly for all three and explain each instruction sequence.

2. **Make `plain` lose.** The optimiser collapsed the `plain` loop. Defeat that (with an opaque function call in the loop, or `-O0`) and show that a plain `int++` loses updates too.

3. **TSan a real program.** Take any multithreaded program (or write a work queue) and run it under TSan. Fix every race. Then run the test suite 200 times and confirm it is clean.

4. **Producer–consumer.** Implement a bounded queue with an `mtx_t` and two `cnd_t`s, supporting multiple producers and consumers plus clean shutdown. Verify it with TSan. Then deliberately change a `while` to an `if`, and find the failure.

5. **Release/acquire proof.** Reproduce the §17.5 pattern. Verify that TSan reports 0 warnings. Change both orders to `relaxed` and confirm that TSan reports the race on `data`. Explain the missing happens-before edge.

6. **Memory-order codegen.** Dump the assembly for all six memory orders on a load, a store, and a read-modify-write. If you have access to an ARM64 machine (or `aarch64-linux-gnu-gcc`), do the same and compare. This is the clearest way to internalise why x86 hides the cost.

7. **False sharing.** Write 4 threads that each increment their own counter in a `long counters[4]`. Benchmark. Then change to `struct { alignas(64) long v; } counters[4]` and benchmark again. Measure the speedup.

8. **CAS loop.** Implement `atomic_max(_Atomic long *x, long v)` using `compare_exchange_weak`. Test it with 8 threads pushing random values, and verify that the result equals the true maximum. Explain why you use `weak` inside a loop and `strong` outside one.

9. **Lock-free stack, and its ABA problem.** Implement a Treiber stack with CAS. Then construct a scenario in which ABA corrupts it. Fix it with a tagged pointer (using the low bits of a 16-byte-aligned pointer, Module 9).

10. **Portability layer.** Write a `thread.h` that abstracts over `<threads.h>`, pthreads, and Win32 threads, selected with `#ifdef` (Module 12). Include thread creation, mutexes, condition variables, and once-initialization. Build the same test program against every available backend.

Module 17 in Summary

C11 gave C a memory model, a thread library, and atomics. Threads are created and joined with `thrd_create` and `thrd_join` (or, more portably in practice, pthreads). Mutexes protect shared state; condition variables let threads sleep until that state changes, always rechecking their predicate in a `while` loop while holding the mutex.

Two threads accessing the same memory without synchronisation, at least one writing, is a data race, and a data race is undefined behaviour — silent, timing-dependent, and reliably found only by ThreadSanitizer. `volatile` does nothing to prevent it: it forces memory accesses but provides neither atomicity nor ordering, and the experiment showed it losing a third of its updates. `_Atomic` types provide both, with memory orders from `relaxed` to the default `seq_cst`; on x86 most orders compile identically, so weaker orders must never be chosen by benchmarking there.

The model's single rule is that conflicting accesses must be ordered by happens-before — built from sequenced-before within a thread and synchronizes-with edges from release/acquire atomics, mutexes, thread creation and joining, and `call_once`. A release store and an acquire load can publish any amount of ordinary data safely. Prefer not sharing at all, then mutexes, then simple atomics; leave lock-free algorithms, with their ABA hazards, to measured hot paths; and run TSan repeatedly under load.

This completes the core C language section (Modules 1–17). You now have the language itself.

Next comes Module 18, C23 Highlights — a short module consolidating what is new in the latest standard and what it changes about the idioms you have learned. Then Part 2 begins: the industry toolchain, starting with Module 19 (compilers, warnings, and reading generated assembly), which is where "knows C" becomes "professional C developer."

Module 18 — C23 Highlights

In this module: `nullptr`, `typeof`, `constexpr`, `auto`, `bool` as a keyword, attributes (`[[nodiscard]]`, `[[deprecated]]`, `[[fallthrough]]`, ...), binary literals, digit separators, `#embed`, `<stdbit.h>`, enhanced enums, `memset_explicit`, the empty initializer `{}` — and what all of it changes about the idioms from Modules 1–17.

C23 (formally ISO/IEC 9899:2024) is the biggest update to the language since C99. Most of its changes are not new capabilities so much as *clean-ups*: things that previously needed a header, a macro, a compiler extension, or an awkward workaround now have a proper, standard spelling. Many of them have already appeared in this book, each in the module where it mattered; this module gathers them in one place.

Everything below was verified on GCC 13.3 with glibc 2.39, using `-std=c2x`, and the module notes explicitly wherever support is missing. A probe program run at the start reported:

```
__STDC_VERSION__ = 202000L   ← GCC 13's c2x mode (final C23 is 202311L)
attributes: nodiscard=202003 deprecated=201904 fallthrough=201910
            maybe_unused=202106 noreturn=202202
<stdbit.h>=1 <stdckdint.h>=0
```

Three things are worth noticing already. First, GCC 13 predates the final standard and reports a *provisional* version number (§18.6 shows why that matters). Second, `__has_c_attribute(name)` — itself new in C23 — reports each supported attribute as a date-like number identifying the version of its specification. Third, compiler and library support arrive separately: `<stdbit.h>` is present, `<stdckdint.h>` is not.

Turning C23 on

```
gcc -std=c2x ...    # GCC 9–13 spelling
gcc -std=c23 ...    # GCC 14 and later
clang -std=c23 ...  # Clang 18 and later (earlier versions: -std=c2x)
```

The `c2x` spelling dates from before the standard was finished; once it was published, compilers added `c23`. Newer compilers accept both.

18.1 Clean-ups — Things That Stop Being Awkward

`bool`, `true`, `false` are keywords

```
bool b = true;   /* no #include <stdbool.h> */
```

```
bool b = true -> 1, sizeof(bool)=1
```

C's Boolean type arrived in C99 under the reserved name `_Bool`, with `<stdbool.h>` supplying the friendly macros `bool`, `true`, and `false`. In C23 they are real keywords. `<stdbool.h>` still exists, but it is now nearly empty and deprecated. The same happened to `<stdalign.h>`

(`alignas` and `alignof` are now keywords) and `<stdnoreturn.h>` (replaced by the `[[noreturn]]` attribute). `static_assert` and `thread_local` became keywords too, and `static_assert`'s message is now optional:

C11/C17	C23
`_Bool` + `<stdbool.h>` for `bool`, `true`, `false`	`bool`, `true`, `false` are keywords
`_Alignas`, `_Alignof` + `<stdalign.h>`	`alignas`, `alignof` are keywords
`_Static_assert(cond, "msg")`	`static_assert(cond)` — keyword, message optional
`_Thread_local` + `<threads.h>` for `thread_local`	`thread_local` is a keyword
`_Noreturn` + `<stdnoreturn.h>`	`[[noreturn]]` (old forms deprecated)

```c
static_assert(sizeof(int) >= 4);
thread_local int tl = 3;
alignas(64) static char line[64];
```

```
thread_local keyword: 3, alignof(line) = 64
```

The same file compiled as C17 fails immediately, because without their headers none of those words is a keyword there:

```
t10.c:6:10: error: expected ';' before 'struct'
t10.c:8:13: error: expected ';' before 'int'
t10.c:9:9: error: expected declaration specifiers or '...' before numeric constant
```

One subtle change: `true` and `false` now have type `bool`, where in C17 they were simply the `int` constants 1 and 0:

```
true has type bool (C17 would say int)
```

That matters to `_Generic` (Module 12), which can now distinguish a Boolean argument from an integer one.

`nullptr` and `nullptr_t`

```c
int *p = nullptr;
```

```
int *p = nullptr; p==NULL: 1 sizeof(nullptr_t)=8
_Generic: nullptr -> nullptr_t, NULL -> void *, 0 -> int
```

Why it matters: `NULL` may be defined as `0` (an `int`) or as `((void*)0)`, which is why Module 6 warned you to write `(char *)NULL` in variadic calls. `nullptr` is a null pointer constant with its own type, `nullptr_t` (from `<stddef.h>`), and it is unambiguously a pointer. So:

```c
execl(path, "arg", nullptr);   /* correct — no cast needed */
```

It also works cleanly with `_Generic` (Module 12), where `NULL` was ambiguous: the `_Generic` line above shows `nullptr`, `NULL`, and `0` all selecting different branches.

Binary literals and digit separators

```c
int mask   = 0b1010'1100;
long big   = 1'000'000'000;
unsigned h = 0xDEAD'BEEF;
```

```
0b1010'1100   = 172 (0xAC)
1'000'000'000 = 1000000000
0xDEAD'BEEF   = 0xDEADBEEF
```

`0b` literals were a GCC/Clang extension for decades; now they are standard, and they make bit masks such as Module 4's register definitions self-explanatory. The `'` separator is purely visual — the compiler ignores it — and it works in any base, including floating-point constants:

```
3.141'592 = 3.14159
```

(The apostrophe was chosen, as in C++14, because the more obvious underscore would clash with existing syntax.)

And `printf` gained matching `%b` and `%B` conversions for printing in binary (glibc supports them):

```
%b test: 172 in binary = 10101100
```

`u8` character constants

```c
auto c = u8'A';
```

```
u8'A' = 65, sizeof=1
```

`u8'x'` is a UTF-8 character constant of type `unsigned char` — useful alongside Module 8's UTF-8-in-`char` approach. (C23 also changed `u8"..."` string literals to have type `char8_t[]`, where `char8_t` is an alias of `unsigned char`.)

Empty initializer `{}`

```c
struct S s = {};
int arr[5] = {};
```

```
struct S s = {}; -> a=0 b=0 buf[0]=0
int arr[5] = {}; -> 0 0 0 0 0
```

`= {}` zero-initializes any object — scalar, array, or structure — and replaces the `= {0}` idiom from Module 3. It is clearer (you are not "initializing the first member to 0"), it works for structures whose first member is not a scalar (where `{0}` provokes warnings about missing braces), it works for plain scalars, and it can even initialize variable-length arrays, which could not be initialized at all before.

> ⚠ Same caveat as before: `{}` zeroes the *members*, not necessarily the padding bytes (Module 11). For byte-exact zeroing, use `memset`.

`()` finally means `(void)`

```
void f();
f(1, 2, 3);
```

```
error: too many arguments to function 'f'
```

The K&R non-prototype declaration from Module 1, §1.2 is gone. `void f();` now means "takes nothing" — matching C++, and matching everyone's intuition for 45 years. (Writing `(void)` still works everywhere and remains the right choice for code that must also compile as C17.)

K&R-style function *definitions* are also removed from the standard. But removal from the standard does not force every compiler to reject them immediately. GCC 13 in C2x mode still accepts them, with only a warning:

```
int add(a, b) int a; int b; { return a + b; }
```

```
-std=c2x: warning: old-style function definition [-Wold-style-definition]
-std=c17: (no diagnostic)
```

Make it an error yourself — `-Werror=old-style-definition` — if you want the C23 rule enforced today.

Two's complement is mandatory

```
INT_MIN = -2147483648, INT_MAX = 2147483647, -INT_MAX-1 == INT_MIN: 1
no more one's-complement or sign-magnitude implementations
```

C23 finally requires signed integers to use two's-complement representation — which every mainstream machine already did. That retires a pile of Module 2 caveats: `INT_MIN` is now guaranteed to be `-INT_MAX - 1`, `~0` is −1, and the minimum range of `int` becomes −32,768 to 32,767.

What did **not** change matters just as much. **Signed overflow is still undefined behaviour** — mandating the representation says nothing about what happens when arithmetic overflows, and every optimisation in Module 16 still applies. And **left-shifting a negative value is still undefined** (C++20 defined it; C23 did not). UBSan in C2x mode confirms it:

```
sh.c:1:25: runtime error: left shift of negative value -1
```

Labels may precede declarations and end a block

```
    goto end;
    x = 2;

```

```
end:          /* legal now — no `;` hack needed */
}
```

```
    goto L;
L:
    int after_label = 7;   /* a declaration straight after a label */
```

```
declaration after label: 7
```

Before C23, a label had to be followed by a *statement* — and neither a declaration nor a closing brace is one. That is what forced the awkward `case 1: ;` workaround in Module 5, §5.2. Note that the *scope* problem from that section — a variable declared in one `case` being visible, uninitialized, in the next — is unchanged; per-case braces are still the right fix.

One-argument `va_start`

```
static int sum(int n, ...)
{
    va_list ap; va_start(ap);   /* C23: second argument optional */
    ...
}
```

```
va_start(ap) with one argument: sum=6
```

The second argument of `va_start` (Module 6) was always redundant — the compiler knows where the variable arguments begin. It is now optional, which also makes it possible to declare functions whose parameter list is *only* `...`.

18.2 New Language Features

`typeof` and `typeof_unqual`

```
const int ci = 7;
typeof(ci)        same = 8;   /* const int */
typeof_unqual(ci) mut  = 9;   /* int — qualifier stripped */
mut = 10;                     /* OK */
```

```
typeof(ci) keeps const; typeof_unqual(ci) strips it -> mut=10
```

`typeof` was a GNU extension for decades — it appeared in Module 4's `ARRAY_LEN` and Module 11's type-checked `container_of`. Now it is standard, and it makes safe macros possible without GNU statement expressions:

```
#define SWAP(a, b) do { typeof(a) tmp_ = (a); (a) = (b); (b) = tmp_; } while (0)
```

`typeof_unqual` is the one you usually want inside macros. With plain `typeof`, a `const` argument makes the macro's temporary `const` too — and then it cannot be assigned to.

`auto` — type inference

```
auto x = 3.14;   /* double */
auto y = 42;     /* int */
```

```
auto x = 3.14 -> sizeof=8 (double)
auto y = 42   -> sizeof=4 (int)
```

The old meaning of `auto` — the default storage class that nobody ever wrote (Module 3) — is repurposed: in a declaration with an initializer, `auto` now means "infer the type from the initializer." This is the most controversial C23 change. C has always been explicit about types, and in C the exact type matters a great deal — to integer promotions, signedness, and overflow (Module 2). `auto` hides it. `auto i = strlen(s);` makes `i` a `size_t`, which may not be what a later `i >= 0` loop expects.

Where it is genuinely useful is inside macros, where you need "the type of this expression" and do not know it in advance:

```
#define MAX(a, b) ({ auto a_ = (a); auto b_ = (b); a_ > b_ ? a_ : b_; })
```

(That example still uses a GNU statement expression `({ ... })` to yield a value; C23 standardised the declaration part but not statement expressions.) Where it is not useful is in ordinary variable declarations. Use it sparingly.

`constexpr`

```
constexpr int SIZE = 16;
int fixed[SIZE];   /* a real constant expression */
```

```
constexpr int SIZE = 16; int fixed[SIZE]; sizeof=64
```

This closes a long-standing C wart. As Modules 2, 3, and 12 explained, a `const int` is *not* a constant expression in C: you cannot use it for array sizes, `case` labels, bitfield widths, or static initializers. An anonymous `enum` worked, but only for `int` values; `#define` worked, but with no type and no scope.

```
constexpr double PI  = 3.14159265358979;   /* enum can't do this */
constexpr size_t CAP = 1024;               /* typed, scoped, usable as an array size */

struct Point { int x, y; };
constexpr struct Point ORIGIN = { 0, 0 };  /* structures work too */
```

```
constexpr struct: ORIGIN = (0,0)
```

A `constexpr` object is typed, scoped like any other variable, visible in the debugger, and usable wherever a constant expression is required.

The compiler enforces that the value really is known at compile time — and that it fits the type exactly:

```
constexpr int k = g();    /* g() is a function call */
constexpr int k = 3.5;    /* would silently truncate */
```

```
error: initializer element is not constant
error: 'constexpr' integer initializer is not an integer constant expression
```

The second error is a small but welcome safety feature: an ordinary `int k = 3.5;` silently truncates to 3, but a `constexpr` initializer must be exactly representable in the declared type.

C23 `constexpr` applies to *objects* only — not to functions, unlike C++.

`unreachable()`

```
#include <stddef.h>

static const char *name(enum State s)
{
    switch (s) { case S_A: return "A"; case S_B: return "B"; }
    unreachable();
}
```

A second, runnable example:

```
static int classify(int v)
{
    switch (v & 1) { case 0: return 0; case 1: return 1; }
    unreachable();
}
```

```
classify(7) = 1 (unreachable() after an exhaustive switch)
```

`unreachable()` standardises `__builtin_unreachable()` (Module 5). It suppresses the "control reaches end of non-void function" warning, lets the compiler delete the dead path, and documents your intent.

> ⚠ **Reaching it is undefined behaviour.** Use it only where you are certain — and, for enums, combine it with `-Werror=switch`, so that the compiler proves every case is handled for you (Module 5).

18.3 Attributes

C23 adopts C++'s `[[attribute]]` syntax, giving portable spellings to things that previously needed `__attribute__((...))`:

```
[[nodiscard]] int must_use(void);
[[nodiscard("the return value carries the error code")]] int must_use2(void);
[[deprecated("use new_api()")]] void old_api(void);
[[maybe_unused]] static int helper(void);
[[noreturn]] void die(void);
[[fallthrough]];
```

All firing correctly:

```
warning: ignoring return value of 'must_use', declared with attribute 'nodiscard'
warning: ignoring return value of 'must_use2', declared with attribute 'nodiscard':
         "the return value carries the error code"
warning: 'old_api' is deprecated: use new_api()
```

Attribute	Use
`[[nodiscard]]`	Put this on every fallible function — it is Module 15's rule "never ignore a return value," enforced by the compiler. An optional message explains why.
`[[deprecated("msg")]]`	Migrate APIs without breaking builds: every use warns, with your message
`[[maybe_unused]]`	Silence "unused" warnings on conditionally used things, such as a parameter used only in debug builds
`[[noreturn]]`	Replaces `_Noreturn` (Modules 5 and 6)
`[[fallthrough]]`	Replaces the `/* fall through */` comment (Module 5)
`[[reproducible]]`, `[[unsequenced]]`	Purity hints about a function's side effects, enabling common-subexpression elimination and reordering

`[[fallthrough]]` in action — the marked case is silent, and the unmarked one warns:

```
ft.c:6:13: warning: this statement may fall through [-Wimplicit-fallthrough=]
one two
three four
```

Detect availability with `__has_c_attribute(nodiscard)`, as the probe at the top of this module did.

> **`[[nodiscard]]` on your error-returning functions is the single highest-value C23 change for reliability.** Combined with Module 15's error API, it turns ignoring an error into a compile-time warning.

18.4 Enhanced Enums

```c
enum Small : unsigned char { S_A, S_B, S_C, };   /* fixed type + trailing comma */
enum Wide  : long long     { W_BIG = 1LL << 40 };
enum Flags : uint32_t      { F_A = 1u<<0, F_C = 1u<<31 };

enum Fwd : int;              /* forward declaration — impossible before C23 */
int use(enum Fwd f);
enum Fwd : int { FWD_X = 1, FWD_Y = 2 };
```

```
enum Small : unsigned char -> sizeof=1
enum Wide : long long      -> sizeof=8 W_BIG=1099511627776
```

```
enum Flags : uint32_t      -> sizeof=4 F_C=0x80000000
forward-declared enum: use(FWD_Y)=2
```

These fix Module 11, §11.6's complaints:

1. **A fixed underlying type** — enums are now usable in packed structures, wire formats, and ABI boundaries, because their size no longer depends on the compiler.
2. **Values beyond the range of `int` are allowed**, provided the underlying type can hold them (`W_BIG`, and `F_C`, which does not fit in a signed `int`).
3. **Forward declaration works** — because the size is known from the declaration alone — so an enum type can appear in a header's function prototypes without its full definition, just like an opaque structure (Module 11, §11.8).
4. The trailing comma after the last enumerator, handy for X-Macros and generated code, is also legal — as it has been since C99.

18.5 Library Additions

`<stdbit.h>` — standardised bit operations

```
x = 0xAC
  count_ones        = 4
  count_zeros       = 28
  leading_zeros     = 24
  trailing_zeros    = 2
  first_leading_one = 25
  bit_width         = 8
  bit_floor         = 128
  bit_ceil          = 256
  has_single_bit(64) = 1
  __STDC_ENDIAN_NATIVE__ == LITTLE? 1
```

Everything from Module 4, §4.8, now portable: `stdc_count_ones`, `stdc_count_zeros`, `stdc_leading_zeros`/`_ones`, `stdc_trailing_zeros`/`_ones`, `stdc_first_leading_one`, `stdc_bit_width`, `stdc_bit_floor`, `stdc_bit_ceil`, and `stdc_has_single_bit`, each with `_uc`/`_us`/`_ui`/`_ul`/`_ull` variants for specific types plus type-generic macros that pick the right one. (`first_leading_one` numbers bit positions from 1, counting from the most significant end, and returns 0 if no bit is set — which is why 0xAC, whose highest set bit is bit 7 of a 32-bit value, gives 25.)

Plus endianness macros: `__STDC_ENDIAN_NATIVE__`, `__STDC_ENDIAN_LITTLE__`, and `__STDC_ENDIAN_BIG__` — finally a standard way to test byte order at compile time, replacing the union trick from Module 11.

> `stdc_leading_zeros(0)` is *defined* — it returns the width of the type — unlike `__builtin_clz(0)`, which is undefined behaviour. A real improvement:
>
> ```
> leading_zeros(0u)=32
> ```

`<stdckdint.h>` — checked arithmetic

```
int r;
if (ckd_add(&r, a, b)) { /* overflow */ }
ckd_sub(&r, a, b);
ckd_mul(&r, a, b);
```

Each returns `true` on overflow. They standardise `__builtin_add_overflow` and friends (Module 2, §2.8): the operation is performed as if in infinite precision, and the result is then converted to the type of `*r` — so `r` always holds the wrapped value, and the return value tells you whether wrapping happened. The arguments may even be of different types.

This header was not available with GCC 13 and glibc 2.39:

```
<stdckdint.h> not available in this libc.
GCC builtin equivalent: __builtin_add_overflow(INT_MAX,1,&r) = 1
```

(`<stdckdint.h>` ships with the compiler rather than the C library; GCC provides it from version 14.) Until then, the builtins do the same job.

`memset_explicit`

```
memset_explicit(pw, 0, sizeof pw);
```

This is the standard answer to Module 10, §10.9: a `memset` that the compiler is forbidden to optimise away. It is not in glibc 2.39 yet:

```
warning: implicit declaration of function 'memset_explicit' [-Wimplicit-function-declaration]
```

Use `explicit_bzero` in the meantime.

Now-standard functions

Functions that were previously POSIX or GNU extensions are now ISO C:

`strdup`, `strndup`, `memccpy`, `gmtime_r`, `localtime_r`, `timespec_getres`, `memalignment`, `free_sized`, and `free_aligned_sized`.

```
strdup (C23 standard): "now standard in C23"
```

This is a real portability win. Module 14's advice to use `gmtime_r` no longer depends on POSIX or a feature-test macro. (C23 also marks the non-reentrant `asctime` and `ctime` as deprecated.) `free_sized` lets code that knows an allocation's size pass it to the allocator, which can then free it faster; `memalignment` reports the alignment of a pointer.

Other additions and changes

Item	Change	Module
`timespec_get` with `TIME_MONOTONIC`	An optional standard monotonic clock	14

Item	Change	Module
`realloc(p, 0)`	Now **undefined behaviour** (it was implementation-defined, then obsolescent in C17) — call `free`	10
`_BitInt(N)`	Exact-width integers of any size — not in GCC 13, which treats `_BitInt` as an undeclared function; supported from GCC 14	—

`#embed` (preprocessor)

```
static const unsigned char logo[] = {
    #embed "logo.png"
};
```

This was covered in Module 12, §12.9. It is not in GCC 13:

```
error: missing binary operator before token "("   ← __has_embed unsupported
error: invalid preprocessing directive #embed
```

It landed in GCC 15 and Clang 19.

`__VA_OPT__`

```
#define LOG(fmt, ...) printf("[log] " fmt "\n" __VA_OPT__(,) __VA_ARGS__)
```

```
[log] no args
[log] x=7
```

It works with `-pedantic` — the standard replacement for the GNU `, ##__VA_ARGS__` hack (Module 12, §12.4).

Also standardised in the preprocessor: `#warning`, `#elifdef`/`#elifndef`, and `__has_include`, `__has_c_attribute`, and `__has_embed`:

```
#ifdef NOPE
#  define WHICH "nope"
#elifdef FEATURE_X
#  define WHICH "elifdef FEATURE_X"
#endif
```

```
#elifdef -> elifdef FEATURE_X
```

18.6 Support and Adoption

Feature	GCC 13.3 (`-std=c2x`) + glibc 2.39	Notes
`bool`/`true`/`false`, `alignas`, `static_assert`, `thread_local` keywords	✓ verified	
`nullptr` / `nullptr_t`	✓ verified	
Binary literals, digit separators, `u8` char constants	✓ verified	
Empty initializer `{}`	✓ verified	
`typeof` / `typeof_unqual`	✓ verified	
`auto` type inference	✓ verified	
`constexpr`	✓ verified	
`unreachable()`	✓ verified	
All six attributes	✓ verified	
Enum fixed type + forward declaration	✓ verified	
`()` means `(void)`	✓ verified	
Labels before declarations / at end of block	✓ verified	
One-argument `va_start`	✓ verified	
`__VA_OPT__`, `#warning`, `#elifdef`, `__has_include`	✓ verified	
`%b`/`%B` in `printf`	✓ verified	
`<stdbit.h>`	✓ verified	
`<stdckdint.h>`	✗	GCC 14+; use `__builtin_*_overflow`
`memset_explicit`	✗ not in glibc 2.39	use `explicit_bzero`
`_BitInt(N)`	✗	GCC 14+
`#embed`	✗	GCC 15+, Clang 19+
K&R definitions rejected	⚠ warns only	add `-Werror=old-style-definition`

There are two moving parts: the **compiler** (GCC 13 and 14 have most of the language; GCC 15 adds `#embed`) and the **C library** (glibc 2.38 added `strlcpy`; 2.39 still lacks `memset_explicit`). A feature is usable only when *both* sides of your toolchain have it — on every platform you build for.

MSVC's C23 support is partial and lags well behind. For cross-platform code, plan for a subset.

Beware the provisional version number

The final C23 standard defines `__STDC_VERSION__` as `202311L`, but GCC 13 reports `202000L`. So:

```
#if __STDC_VERSION__ >= 202311L    /* false on GCC 13 in -std=c2x mode! */
#if __STDC_VERSION__ > 201710L     /* "newer than C17" — true on both */
```

A version check written for the final value silently takes the fallback path on transitional compilers. Where you can, test for the specific feature instead.

How to adopt safely

```
#if __STDC_VERSION__ >= 202311L
#  define MY_NODISCARD [[nodiscard]]
#elif defined(__GNUC__)
#  define MY_NODISCARD __attribute__((warn_unused_result))
#else
#  define MY_NODISCARD
#endif

#if __has_include(<stdckdint.h>)
#  include <stdckdint.h>
#elif defined(__GNUC__)
#  define ckd_add(r,a,b) __builtin_add_overflow((a),(b),(r))
#endif
```

Use `__has_include`, `__has_c_attribute`, and `__STDC_VERSION__` rather than compiler-version checks. (Given the provisional-version issue, the first block would be more robust as `#if defined(__has_c_attribute)` followed by `#if __has_c_attribute(nodiscard)`.)

A practical adoption order:

1. **Decide on a minimum toolchain first** — including CI images, distribution packages, and embedded cross-compilers, which typically lag several years behind.
2. **Adopt the zero-risk features early:** the keywords, `nullptr`, `{}`, attributes, and binary literals are widely supported.
3. **Guard the rest** with `__has_include` and `__has_c_attribute`, falling back to the builtins or extensions you used before.
4. **Remember what C23 did not fix.** Signed overflow, strict aliasing, data races, out-of-bounds access, and uninitialized reads are all still undefined behaviour. Everything in Module 16 still applies; the sanitizers are just as necessary.

18.7 What Changes About the Idioms You've Learned

Module	Old idiom	C23
1	`int f(void)` mandatory	`int f()` now means the same
2	`char` signedness, `~0` caveats	Two's complement mandatory (signed overflow still UB)
2	`#include <stdbool.h>`	`bool` is a keyword
3	`= {0}`	`= {}`

Module	Old idiom	C23
3	`#define MAX 1024` or `enum`	`constexpr size_t MAX = 1024;`
4	`__builtin_popcount`	`stdc_count_ones`
4	`__builtin_clz(0)` is UB	`stdc_leading_zeros(0)` is defined
4, 11	GNU `typeof`	Standard `typeof`, `typeof_unqual`
5	`/* fall through */`	`[[fallthrough]]`
5	`case 1: ;` before a declaration	A label may precede anything
6	`_Noreturn`	`[[noreturn]]`
6, 15	`__attribute__((warn_unused_result))`	`[[nodiscard]]`
6	`va_start(ap, last)`	`va_start(ap)`
8	`explicit_bzero`	`memset_explicit`
8	`strdup` needs `_POSIX_C_SOURCE`	Now ISO C
9	`(char *)NULL` in variadics	`nullptr`
10	`realloc(p, 0)` implementation-defined	Undefined — call `free`
11	Enum underlying type is implementation-defined	`enum E : uint8_t`
11	Union trick to detect endianness	`__STDC_ENDIAN_NATIVE__`
12	`, ##__VA_ARGS__` (GNU)	`__VA_OPT__(,)`
12	`xxd -i` for embedded assets	`#embed`
14	`gmtime_r` needs POSIX	Now ISO C
14, 15	`_Static_assert(c, "msg")`	`static_assert(c)`
14	`__builtin_add_overflow`	`ckd_add`

Module 18 Rules

#	Rule
1	Put `[[nodiscard]]` on every function whose return value must be checked
2	`constexpr` replaces `#define` and `enum` for typed compile-time constants
3	`typeof_unqual` (not `typeof`) inside macros, so `const` arguments still work
4	`nullptr` in variadic calls and `_Generic` — never a bare `NULL`
5	Use `auto` in macro internals, not in ordinary declarations

#	Rule
6	`enum E : uint8_t` makes enums safe in packed structs and at ABI boundaries
7	`unreachable()` only where `-Werror=switch` proves the case is impossible
8	Feature-test with `__has_include` / `__has_c_attribute`, not compiler versions
9	Compiler support ≠ library support — check both
10	For cross-platform code, assume MSVC has a smaller C23 subset
11	GCC 13 reports `__STDC_VERSION__` as `202000L` — don't test for `>= 202311L` alone
12	Two's complement is mandatory, but signed overflow and negative left shifts are still UB
13	`realloc(p, 0)` is now UB — call `free`
14	Standard removal ≠ compiler rejection: enforce with `-Werror=old-style-definition`

Practice

1. **Probe your toolchain.** Run the §18 probe program (version macro, `__has_c_attribute` values, and `__has_include` checks) on every compiler you have access to — different GCC and Clang versions, and a cross-compiler if you use one. Build a support table for the features you would actually want to use.

2. **`[[nodiscard]]` retrofit.** Take the error library from Module 15 and add `[[nodiscard]]` to every function returning `err_t`. Then deliberately ignore a return value and confirm the warning. Add `-Werror=unused-result`.

3. **Safe macros with `typeof_unqual`.** Rewrite `MIN`, `MAX`, `SWAP`, and `CLAMP` from Module 12 using `typeof_unqual`, so that each argument is evaluated exactly once. Verify with the side-effect counter that they call `f()` once. Then test them with `const` arguments.

4. **`constexpr` migration.** Take a file full of `#define` constants and convert them to `constexpr`. Note which ones now get type checking they did not have before. Find one where the `#define` was silently the wrong type. Then find five initializers that `const` accepts but `constexpr` rejects, and explain each error.

5. **`<stdbit.h>` vs builtins.** Reimplement the Module 4 bit toolkit using `<stdbit.h>`. Compare the generated assembly with the `__builtin_*` versions, and verify that both agree on all 65,536 16-bit values. Then test the zero case for `leading_zeros` and confirm that it is defined where `__builtin_clz` was UB.

6. **Enum ABI fix.** Take a structure with a bare enum field that is written to a file. Show its size on two different compilers (or with `-fshort-enums`). Then fix it with `enum E : uint8_t` and confirm that the size is now guaranteed.

7. **Compatibility shim header.** Write `c23compat.h`, providing `MY_NODISCARD`, `MY_FALLTHROUGH`, `MY_MAYBE_UNUSED`, `ckd_add`, and `secure_zero` so that they work on C23, on GCC/Clang before C23, and on MSVC. Test it by building the same source with `-std=c17` and `-std=c2x`.

8. **`nullptr` and variadics.** Write a sentinel-terminated variadic function (Module 6) and call it with `NULL`, `0`, `(char *)NULL`, and `nullptr`. On a platform where `NULL` is a plain `0` (or by `#define`-ing it that way in a test), show which calls are wrong.

9. **`auto` pitfalls.** Write three short examples in which `auto` deduces a type that surprises the reader — at least one involving `size_t`, and one involving character constants or integer promotion. Explain each.

10. **`#embed`, if you have GCC 15+ or Clang 19+.** Embed a 1 MB binary and compare compile time against `xxd -i`. If you don't have it, write the `xxd -i` build rule and note what `#embed` would replace.

Module 18 in Summary

C23 is mostly a modernisation: it turns long-standing macros, extensions, and workarounds into standard language features. `bool`, `true`, `false`, `alignas`, `alignof`, `static_assert`, and `thread_local` become keywords; `nullptr` gives C an unambiguous null pointer; binary literals, digit separators, `%b`, `= {}`, labels before declarations, and one-argument `va_start` smooth out rough edges; and `()` finally means "no parameters," with K&R definitions removed from the standard (though GCC 13 still only warns).

New language features include `typeof` and `typeof_unqual` for safe macros, `auto` for type inference (best confined to macros), `constexpr` for typed compile-time constants, `unreachable()`, and standard attributes — above all `[[nodiscard]]`, which turns an ignored error into a warning. Enums gain a fixed underlying type, larger values, and forward declarations. The library adds `<stdbit.h>` (with a defined zero case and endianness macros), `<stdckdint.h>`, `memset_explicit`, and several functions promoted from POSIX; `realloc(p, 0)` becomes undefined.

Two's complement is now mandatory — but signed overflow and negative left shifts remain undefined, and every hazard from Module 16 still applies. Support arrives piecemeal: GCC 13 with glibc 2.39 handles most of the language, but not `<stdckdint.h>`, `_BitInt`, `memset_explicit`, or `#embed`, and even reports a provisional version number. So adopt C23 feature by feature, testing for each one with `__has_include` and `__has_c_attribute`.

That completes Part 1 — the C language (Modules 1–18).

You now have the language itself: types, memory, pointers, the standard library, undefined behaviour, and concurrency, plus what C23 changes about all of it.

Part 2 begins with Module 19 — Toolchain & Compilation: GCC/Clang/MSVC differences, the complete warning set and why each flag exists, optimisation levels and what they actually do, reading generated assembly (you have been doing this all along — now systematically), LTO, cross-compilation, and the binutils suite (`objdump`, `nm`, `readelf`, `addr2line`) that you will live in when debugging production binaries.

PART 2 — What Industry Actually Expects (Beyond the Language)

This is the part that separates “knows C” from “professional C developer.”

You have the language. Everything from here is about the ecosystem around it: building, debugging, testing, securing, and shipping C. Part 1 asked *what does this code mean?* Part 2 asks *how do real teams turn C source into reliable software, and keep it that way for years?*

Module 19 — Toolchain & Compilation

In this module:

- GCC, Clang, MSVC, ICC — differences and extensions
- Warning flags you should always run: `-Wall -Wextra -Wpedantic -Werror -Wshadow -Wconversion -Wstrict-prototypes`
- Optimisation levels `-O0`…`-O3`, `-Os`, `-Ofast`, `-g`, `-march`/`-mtune`
- Preprocessor and assembly output (`-E`, `-S`); reading generated assembly (Godbolt)
- Link-Time Optimisation; `-ffunction-sections -Wl,--gc-sections`
- Cross-compilation, sysroots, toolchain files
- `objdump`, `nm`, `readelf`, `strip`, `size`, `ldd`, `addr2line`

All output in this module is real, from GCC 13.3 and GNU binutils.

Module 1 introduced the four stages that turn a `.c` file into an executable. This module is about controlling that process like a professional: choosing and combining compilers, configuring warnings and optimisation deliberately, verifying what the compiler actually produced, shrinking and splitting binaries, building for other machines, and inspecting any binary with the standard toolkit.

19.1 The Compilers

Compiler	Notes
GCC	The Linux and embedded default. The widest target coverage (60+ architectures). Excellent optimiser.
Clang/LLVM	The best diagnostics and sanitizers (MSan is Clang-only), and fast builds. Default on macOS and FreeBSD. Comes with clang-tidy, clang-format, and scan-build.
MSVC	Windows. Historically weak C support: C99 was only substantially complete around Visual Studio 2019, C11/C17 support arrived in 2020 without VLAs, `<threads.h>` only appeared in 2023, and C23 support is minimal. Its C++ compiler is excellent; its C compiler has long been an afterthought.
ICC / ICX	Intel. The classic ICC is retired; ICX is LLVM-based. Excellent auto-vectorisation for Intel CPUs and HPC workloads.
tcc	The Tiny C Compiler. Compiles roughly 10× faster than GCC, with no real optimiser. Good for quick scripts and experiments.

What to know practically

Build with two compilers. GCC and Clang warn about different things, exploit different undefined behaviour (Module 16), and catch different bugs. This is the single cheapest quality improvement available: a second CI job that only compiles.

Extensions to be aware of:

Extension	GCC	Clang	MSVC
`__attribute__((...))`	✓	✓	✗ (`__declspec`)
Statement expressions `({ ... })`	✓	✓	✗
`typeof` (pre-C23)	✓	✓	✗
`__builtin_*`	✓	mostly	✗ (different intrinsics)
Nested functions	✓	✗	✗
VLAs	✓	✓	✗
`#pragma once`	✓	✓	✓
Zero-length arrays	✓	✓	partial

Every ✗ in that table is a line of code that will not compile somewhere. Isolate extensions behind macros, and detect support with feature tests rather than version numbers:

```
#if defined(__GNUC__) || defined(__clang__)
#  define MY_UNUSED __attribute__((unused))
#elif defined(_MSC_VER)
#  define MY_UNUSED
#endif
```

Note that Clang defines `__GNUC__` as well as `__clang__` — it masquerades as GCC for compatibility. When you need to tell them apart, check `__clang__` first.

The compiler is a driver

```
$ gcc -v t.c -o t
 /usr/libexec/gcc/x86_64-linux-gnu/13/cc1 -quiet -v -imultiarch ... t.c ...
 /usr/libexec/gcc/x86_64-linux-gnu/13/collect2 -plugin .../liblto_plugin.so ...
```

`gcc` is not really the compiler. It is a *driver* that runs `cc1` (the actual C compiler, which produces assembly), `as` (the assembler), and `collect2/ld` (the linker), passing each the right options — exactly the pipeline from Module 1. `gcc -v` shows you precisely what it invoked, with every path and flag, which is invaluable when a build fails mysteriously.

```
-dumpmachine     : x86_64-linux-gnu
-print-multiarch : x86_64-linux-gnu
libc.so path     : /usr/lib/gcc/.../libc.so
```

19.2 Warning Flags

Measured on one deliberately bad file:

```
### -Wall only ###           warnings: 8
### -Wall -Wextra ###        warnings: 9
### full recommended set ### warnings: 16
```

The full set nearly doubles what `-Wall` finds. Despite its name, `-Wall` is not “all warnings” — it is a historical, deliberately conservative set that GCC’s maintainers keep stable so that enabling it never becomes too noisy. The extra flags caught:

```
[-Warray-bounds=]     [-Wfloat-conversion]  [-Wformat=]        [-Woverflow]
[-Wparentheses]       [-Wshadow]            [-Wsign-compare]   [-Wsign-conversion]
[-Wstrict-prototypes] [-Wunused-function]   [-Wunused-variable]
```

The recommended set, and why each flag exists

```
# The baseline — non-negotiable
-Wall -Wextra -Wpedantic

# Bugs from earlier modules
-Wshadow                 # M3: shadowing a variable you meant to modify
-Wconversion             # M2: implicit narrowing
-Wsign-conversion        # M2: the signed/unsigned trap
-Wstrict-prototypes      # M6: `void f()` non-prototypes
-Wold-style-definition   # M1: K&R definitions
-Wmissing-prototypes     # M3: a non-static function with no header declaration
-Wcast-qual              # M9: casting away const
-Wcast-align=strict      # M9: producing a misaligned pointer
-Wpointer-arith          # M9: void* arithmetic
-Wwrite-strings          # M8: makes string literals const char[]
-Wformat=2               # M13: format-string bugs and security
-Wnull-dereference       # M16: the CVE-2009-1897 pattern
-Wstrict-aliasing=2      # M2: aliasing violations
-Warray-bounds=2         # M7: out-of-bounds indexing
-Wstringop-overflow=4    # M8: strcpy/memcpy overflows
-Wimplicit-fallthrough   # M5: switch fallthrough
-Wsequence-point         # M4: i = i++ family
-Walloca                 # M10: alloca
-Wvla                    # M7: variable length arrays
-Wdouble-promotion       # M2: accidental float→double in embedded code
-Wfloat-equal            # M2: == on floats
-Wswitch-enum            # M11: unhandled enum values
-Wundef                  # M12: #if on an undefined macro
-Wredundant-decls        # duplicate declarations

# Make the dangerous ones fatal
-Werror=implicit-function-declaration   # M6: silent ABI corruption
-Werror=return-type                     # M5: UB
-Werror=incompatible-pointer-types
-Werror=int-conversion
-Werror=vla
```

Each flag corresponds to a class of bug from Part 1, which is the right way to think about warnings: not as style nagging, but as a cheap, automatic scan for bugs you already know to be real. `-Wmissing-prototypes` deserves a note: it warns about any non-`static` function defined without a previous prototype — which almost always means either the function should be `static` (Module 3) or its header was not included, so the compiler never checked the definition against the declaration callers use. (`-Wswitch-enum` is stricter than `-Wswitch`: it warns about unhandled enumerators even when there is a `default` label.)

`-Werror` — yes or no?

In CI: absolutely yes. A warning that nobody fixes is noise, and noise hides real bugs. Once a codebase has 200 warnings, nobody notices the 201st.

In local development: often no. A new compiler version adds new warnings, and suddenly nobody can build. The common compromise:

```
CFLAGS += $(WARNINGS)
ifdef CI
  CFLAGS += -Werror
endif
```

— plus `-Werror=` on the specific warnings that are *always* bugs (the list above), which should never be allowed to build anywhere.

Noise management

`-Wconversion` and `-Wsign-conversion` are genuinely noisy on existing code. Two strategies:

1. **New code only** — enable them per directory, or for new files.
2. **Fix incrementally** — enable them, count the warnings, and fix ten a day.

Suppress locally when you are sure a warning is a false positive:

```
#pragma GCC diagnostic push
#pragma GCC diagnostic ignored "-Wconversion"
/* ...deliberate narrowing, explained in a comment... */
#pragma GCC diagnostic pop
```

Never suppress a warning globally: that silences the next real bug of the same kind, too.

19.3 Optimisation Levels

Measured on a 400×400 matrix multiply:

Level	Time	Binary	`.text`
`-O0`	217.6 ms	16248	2798
`-O1`	48.3 ms	16224	2459
`-O2`	30.6 ms	16224	2848
`-O3`	26.1 ms	16192	2881
`-Os`	43.7 ms	16224	2368
`-Ofast`	25.1 ms	17736	2981

Going from `-O0` to `-O3` is **8.3× faster**. That is the single largest performance lever you have, and it costs nothing.

What each level means

Level	Purpose
`-O0`	No optimisation. Fast compiles and exact debugging — variables live in memory and statements execute in source order. **Never ship this.**
`-O1`	Cheap optimisations. Already 4.5× faster here.
`-O2`	The production default: everything that does not dramatically increase code size.
`-O3`	`-O2` plus aggressive inlining, loop unrolling, and vectorisation. Sometimes *slower*, because of code bloat and instruction-cache pressure. Measure.
`-Os`	`-O2` minus the size-increasing passes. The smallest `.text` here (2368 bytes versus 2848). The right default for embedded code and for instruction-cache-bound code.
`-Oz`	Even more aggressive size reduction (Clang; GCC 12+ also accepts it).
`-Ofast`	⚠ `-O3` plus `-ffast-math`, which breaks IEEE 754: it assumes there are no NaNs or infinities and allows floating-point reassociation. Not standards-conforming. Only for numerical code whose results you have verified.
`-Og`	Roughly `-O1`, but tuned for a good debugging experience. Better than `-O0` for debug builds of large programs.

Note that `-O3` bought only 15% over `-O2` here, and `-Os` was 43% slower than `-O2` for this compute-heavy loop while producing the smallest code. There is no universally best level; the defaults (`-O2` for general code, `-Os` for size-constrained code) are good starting points, and anything beyond them should be measured.

`-g` is orthogonal

`-g` does *not* disable optimisation. `-O2 -g` is a legitimate and common production configuration: full speed plus full debug information, at the cost of a larger binary — which you then split into a separate debug file (§19.7). Optimised code is harder to debug, because variables may live only in registers and statements may be reordered or merged, but it is far better than having no symbols at all when a production crash arrives.

Levels: `-g1` (line tables only), `-g` = `-g2`, and `-g3` (which adds macro definitions, so that GDB can expand your macros). `-gsplit-dwarf` puts debug information in separate `.dwo` files, for faster links in large projects.

`-march` and `-mtune`

```
-O3                             26.6 ms
-O3 -march=native               24.3 ms
```

```
-O3 -march=x86-64-v3              23.3 ms
-O3 -march=native -ffast-math     22.0 ms
```

`-march=native` enabled these instruction-set extensions on this machine:

```
-mavx2 [enabled] -mavx512f [enabled] -mfma [enabled] -msse4 [enabled]
```

And the vectoriser reported:

```
bench.c:10:22: optimized: loop vectorized using 32 byte vectors
```

Flag	Meaning
`-march=X`	Generate instructions for CPU X. The binary will *not run* on older CPUs.
`-mtune=X`	Schedule instructions for X, but stay compatible with everything. Free.
`-march=native`	The machine you are compiling on. Never for distributed binaries.
`-march=x86-64-v2/v3/v4`	Portable feature levels: v2 ≈ SSE4.2 (2009+), v3 ≈ AVX2 (2013+), v4 ≈ AVX-512.

> **For shipped software:** use `-mtune=generic` (the default) or a conservative `-march=x86-64-v2`, plus run-time dispatch (`__builtin_cpu_supports("avx2")`) for the hot paths that benefit from newer instructions. `-march=native` in a distributed binary produces `SIGILL` (illegal instruction) on customers' older machines.

Useful diagnostics: `-fopt-info-vec` (loops that were vectorised), `-fopt-info-vec-missed` (why a loop *wasn't*), and `-fopt-info-inline`.

19.4 Reading Generated Assembly

`-E`, `-S`, and `-c` were covered in Module 1. Here is the systematic view.

```
gcc -S -masm=intel -fno-asynchronous-unwind-tables -fverbose-asm -O2 f.c -o -
```

- **`-masm=intel`** — Intel syntax (`mov dst, src`). AT&T syntax is GCC's default (`mov src, dst` — operands reversed, with `%` and `$` sigils). Every listing in this book uses Intel syntax.
- **`-fno-asynchronous-unwind-tables`** — removes the `.cfi_*` directives (unwinding information) that otherwise clutter the output.
- **`-fverbose-asm`** — annotates instructions with the source variable names they involve.
- **`-o -`** — write to standard output instead of a file.

The same function at two levels

```
int sum_array(const int *a, int n)
{ int s = 0; for (int i = 0; i < n; i++) s += a[i]; return s; }
```

`-O0` — every variable lives in memory, one operation at a time:

```
sum_array:
    push rbp
    mov  rbp, rsp
    mov  QWORD PTR -24[rbp], rdi   ; spill the parameter
    mov  DWORD PTR -8[rbp], 0      ; s = 0
    mov  DWORD PTR -4[rbp], 0      ; i = 0
.L3: mov eax, DWORD PTR -4[rbp]   ; load i
    cdqe
    lea  rdx, 0[0+rax*4]           ; i*4
    mov  rax, QWORD PTR -24[rbp]   ; load a
    add  rax, rdx
    mov  eax, DWORD PTR [rax]      ; load a[i]
    add  DWORD PTR -8[rbp], eax    ; s += ...
    add  DWORD PTR -4[rbp], 1      ; i++
```

`-O3 -march=native` — vectorised:

```
    vpaddd       ymm1, ymm1, YMMWORD PTR [rax]   ; 8 ints added per instruction
    vextracti128 xmm1, ymm1, 0x1                 ; horizontal reduction
    vpaddd       xmm0, xmm0, xmm1
```

This is what `-O0` versus `-O3` looks like. The `-O0` version performs about nine memory operations per element; the `-O3` version does one 32-byte load and one `vpaddd` per *eight* elements, keeping eight partial sums in the 256-bit `ymm1` register and adding them together only at the end (the “horizontal reduction”).

Why read assembly

1. **To verify that an optimisation happened** — was that loop vectorised? Was that function inlined?
2. **To understand UB exploitation** — every demonstration in Module 16.
3. **To debug optimised code,** where the source does not match what executes.
4. **To confirm a security property** — did that `memset` survive? (Module 10)
5. **For micro-optimisation** — but only after profiling has shown where the time goes.

You do not need to be able to write assembly to benefit. Reading it — recognising loads, stores, arithmetic, calls, and branches, and matching them to your source — is enough, and it comes quickly with practice.

Godbolt Compiler Explorer

godbolt.org is the tool for this. Paste C code, pick from dozens of compilers and versions for many architectures, see colour-matched assembly side by side with your source, diff two compilers or two sets of flags, and share a permalink with a colleague.

Use it constantly. It turns “I think the compiler does X” into “I know the compiler does X.”

19.5 Link-Time Optimisation

Normally the compiler optimises one translation unit at a time (Module 1, §1.5): when it compiles `a.c`, it cannot see into `b.c`, so it cannot inline a function defined there, or notice that a function is never called. *Link-time optimisation* (LTO) removes that barrier. The compiler writes its intermediate representation into the object files, and the real code generation is deferred to link time, when the optimiser can see the whole program at once.

```
gcc -O2 -flto -c a.c -o a.o
gcc -O2 -flto -c b.c -o b.o
gcc -O2 -flto a.o b.o -o prog    # the SAME flags must appear at link time
```

Measured

Without LTO — a real cross-TU call:

```
1075: call 1190 <helper>
1091: call 1050 <__printf_chk@plt>
```

With LTO — `helper` is gone entirely; its loop was inlined into the caller and constant-folded:

```
107b: call 1050 <__printf_chk@plt>
```

```
no LTO   : 16128 bytes, text=1621
with LTO : 15936 bytes, text=1412

dead code (unused_a/b/c symbols):
  no LTO   : 3
  with LTO : 0
```

Practical notes

- ✓ **Benefits:** cross-TU inlining, whole-program dead-code elimination, better devirtualisation of function pointers, and smaller binaries.
- ⚠ **Much slower links and high memory use.** Use `-flto=auto` or `-flto=jobserver` to parallelise.
- ⚠ **It exposes latent UB** — the optimiser now sees across the boundaries where bugs were hiding (Module 16). A strict-aliasing violation between two files, harmless for years, can suddenly change behaviour.
- ⚠ **The same `-flto` and optimisation flags must be used for both compiling and linking.**
- ⚠ **Debugging is harder;** functions and symbols may vanish.
- **Thin LTO** (`-flto=thin`, Clang) is much faster, with most of the benefit — the standard choice for large projects.

Verdict: enable LTO for release builds, keep it off for development builds, and make sure your CI *tests* the LTO build — not just the debug build — because that is the binary you ship.

19.6 Section Garbage Collection

The linker's unit of inclusion is a *section*, and by default every function in an object file shares a single `.text` section. So if a program uses one function from an object file, the linker pulls in all of them.

```
gcc -ffunction-sections -fdata-sections -c *.c
gcc -Wl,--gc-sections *.o -o prog
```

These flags put each function in its own `.text.funcname` section and each data object in its own `.data.name` section, and `--gc-sections` ("garbage-collect sections") tells the linker to discard every section that nothing references. (`-Wl,` passes the following option through the compiler driver to the linker.)

Measured — 20 functions, one used

```
plain build:                        size=16616 text=2512 fn* symbols=20
-ffunction-sections --gc-sections:  size=15920 text=1461 fn* symbols=1
```

`.text` almost halved, and 19 of the 20 functions were removed.

```
/usr/bin/ld: removing unused section '.text.fn2' in file 'bigs.o'
/usr/bin/ld: removing unused section '.text.fn3' in file 'bigs.o'
...

plain .o               : 1 .text section
-ffunction-sections .o : 20 .text.* sections
```

This is essential for embedded work, where flash memory is measured in kilobytes, and for static linking against large libraries. Use `-Wl,--print-gc-sections` to see what is being removed (as above), and `-Wl,-Map=out.map` to produce a full link map showing where everything ended up and why it was included.

> ⚠ Links are slightly slower, and garbage collection can break code that relies on sections being present even though nothing references them by name — for example, tables placed by a linker script and discovered at run time (Module 27). Mark those `KEEP()` in the linker script, or `__attribute__((used))` in the source.

19.7 The binutils Suite

These tools inspect any compiled binary — your own, a library, or a crash-dump's executable. A single small program, `tools`, is used for every example.

```
$ file tools
ELF 64-bit LSB pie executable, x86-64, dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2
```

`file` identifies the format, architecture, and linking style — the quickest way to confirm that a cross-compiled binary really is for the target (§19.8).

`size` — section sizes

```
text data bss  dec  hex filename
1578  616   8 2202  89a tools
```

Module 3's `.bss` distinction, visible on any binary. This is the first thing to check when a binary is unexpectedly large.

`nm` — symbols

```
0000000000001149 T api      ← T = text, global
0000000000004024 B g_bss    ← B = bss, global
0000000000004010 D g_init   ← D = data, global
0000000000001168 T main
```

Module 1's letter table applies. `nm -C` demangles C++ names, `nm -D` shows dynamic (exported) symbols, and `nm -u` shows undefined ones. This is how you debug "undefined reference" and "multiple definition" errors: find out which object file defines — or fails to define — the symbol.

`readelf` — ELF structure

```
$ readelf -S tools       # sections
$ readelf -h tools       # header (type, machine, entry point)
$ readelf -d tools       # dynamic section (NEEDED libraries, RPATH, soname)
$ readelf -l tools       # program headers (what the loader maps)
$ readelf --relocs tools # relocations
```

`readelf -d` is how you diagnose "the wrong library version was loaded": it shows the `NEEDED` libraries, the library's `SONAME`, and any `RUNPATH` (Module 20).

`objdump` — disassembly

```
$ objdump -d --no-show-raw-insn -M intel tools

0000000000001149 <api>:
  1149: endbr64
  114d: mov eax,DWORD PTR [rip+0x2ebd]   # 4010 <g_init>
  1153: lea eax,[rax+rdi*2+0x7]
  1157: ret
```

`objdump -S` interleaves the source (it requires a `-g` build):

```
int api(int x){ return helper(x) + g_init + s_priv; }
  1149: endbr64
  114d: mov eax,DWORD PTR [rip+0x2ebd]
  1153: lea eax,[rax+rdi*2+0x7]
```

Note how `helper(x)` (a `static` function) and `s_priv` (a `static` variable) were both inlined and constant-folded into a single `lea` instruction: the entire function body became "load `g_init`, then add `x*2 + 7`." This is why debugging optimised code is confusing — there is no call to `helper` to set a breakpoint on.

Also: `objdump -t` (symbols), `-r` (relocations), `-x` (everything), and `-h` (section headers).

`addr2line` — crash address → source location

```
$ ./tools crash
Segmentation fault

$ addr2line -e tools -f -C -i 0x115c
crasher
/home/claude/c_tool/tools.c:10
```

This is the tool that turns a production crash log into a bug report. `-f` shows the function name, `-C` demangles, and `-i` expands inlined frames, so that a crash inside an inlined helper is attributed correctly.

Given a stack trace containing raw addresses — from `backtrace_symbols`, a core dump, or a crash reporter — `addr2line` maps each one back to a file and line, *provided* you kept the unstripped binary or its separate debug file.

> ⚠ With position-independent executables (PIE, the default on modern Linux), run-time addresses are randomised by ASLR. Subtract the load base (from `/proc/self/maps`, or the crash report) before calling `addr2line`.

`strip` — and the right way to do it

```
before strip: 18472 bytes, 35 symbols
after strip:  14488 bytes, 1 symbol
```

Stripping removes the symbol table and debug information, making the binary smaller and harder to reverse-engineer. But it also destroys your ability to debug crashes. The professional pattern is to *split* the debug information out:

```
objcopy --only-keep-debug prog prog.debug
strip --strip-debug --strip-unneeded prog
objcopy --add-gnu-debuglink=prog.debug prog
```

```
binary: 14592 bytes   debug file: 7088 bytes
```

Ship the small binary; archive `prog.debug` alongside the release. GDB finds it automatically through the “debuglink” recorded in the binary, and `addr2line -e prog.debug` works on production crash addresses. Every serious project does this (Linux distributions ship the debug files as separate `-dbg` or `-debuginfo` packages).

`ldd` and `strings`

```
$ ldd tools
    linux-vdso.so.1
    libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6
    /lib64/ld-linux-x86-64.so.2

$ strings tools | grep distinctive
a distinctive string constant
```

`ldd` lists the shared libraries a program will load, and where they will be found. `strings` extracts printable text from any file — handy for checking version strings, or for spotting an accidentally embedded secret.

> ⚠ `ldd` works by running the dynamic loader on the binary — so never run it on an untrusted binary, which could execute code. Use `objdump -p | grep NEEDED` or `readelf -d` instead.

The others

`ar` and `ranlib` (static libraries, Module 20), `objcopy` (format conversion and section surgery — `-O binary` produces the raw image used for flashing embedded devices), `c++filt` (demangling), `nm --size-sort` (find the biggest functions), and `bloaty` (not part of binutils, but the best binary-size analyser available).

19.8 Cross-Compilation

Cross-compilation means building on one kind of machine a program that runs on another — an ARM board, a router, a Windows machine, a microcontroller.

The toolchain naming convention

```
<arch>-<vendor>-<os>-<abi>

aarch64-linux-gnu-gcc     64-bit ARM, Linux, glibc
arm-none-eabi-gcc         32-bit ARM, bare metal (no OS)
riscv64-unknown-elf-gcc   RISC-V, bare metal
x86_64-w64-mingw32-gcc    Windows, from Linux
```

This “target triple” (sometimes with four parts) identifies what the toolchain produces. `none` or `unknown` in the OS field means *bare metal* — no operating system, which in C terms means a *freestanding* implementation with only a minimal library (Module 27).

Each cross toolchain has its own preprocessor, `cc1`, assembler, linker, C library, and headers. They are not interchangeable.

The sysroot

```
aarch64-linux-gnu-gcc --sysroot=/opt/arm-rootfs prog.c -o prog
```

The *sysroot* is a directory tree containing the target system’s `/usr/include` and `/usr/lib`. Without it, the compiler would find your *host’s* headers and libraries — and the result would compile, but against the wrong ABI, the wrong library versions, or both.

Related options: `-isysroot`, `-B` (where to find the tools), and `-L`/`-I` for extra paths.

Useful driver queries

```
gcc -dumpmachine              # x86_64-linux-gnu — the target triple
gcc -print-search-dirs        # where it looks for tools and libraries
gcc -print-file-name=libc.so  # resolve a library to a full path
gcc -print-multiarch
gcc -v                        # what the driver actually runs
```

`-m32` needs multilib

```
$ gcc -m32 t.c -o t32
/usr/bin/ld: cannot find Scrt1.o: No such file or directory
/usr/bin/ld: cannot find crti.o: No such file or directory
```

Even for the "same" architecture, a 32-bit build needs the 32-bit C runtime start-up files and libraries (`gcc-multilib`, `libc6-dev-i386`). `Scrt1.o` and `crti.o` are the start-up code that runs before `main` (Module 1's `_start`).

Build-system integration

A CMake *toolchain file* describes the target once:

```
set(CMAKE_SYSTEM_NAME Linux)
set(CMAKE_SYSTEM_PROCESSOR aarch64)
set(CMAKE_C_COMPILER aarch64-linux-gnu-gcc)
set(CMAKE_SYSROOT /opt/arm-rootfs)
set(CMAKE_FIND_ROOT_PATH_MODE_PROGRAM NEVER)
set(CMAKE_FIND_ROOT_PATH_MODE_LIBRARY ONLY)
set(CMAKE_FIND_ROOT_PATH_MODE_INCLUDE ONLY)
```

```
cmake -DCMAKE_TOOLCHAIN_FILE=arm.cmake ..
```

The last three lines tell CMake to run *host* programs during the build but to look for libraries and headers only inside the sysroot.

With Autotools: `./configure --host=aarch64-linux-gnu --build=x86_64-linux-gnu`.

Note the terminology: **build** is where you compile, **host** is where the resulting binary runs, and **target** is what the compiler you are building will itself generate code for (relevant only when building a compiler).

Where to get toolchains: distribution packages (`gcc-aarch64-linux-gnu`), Arm's official GNU toolchain, crosstool-NG, Buildroot, Yocto, Zig (`zig cc` is a remarkably good drop-in cross-compiler with bundled sysroots), and Docker images.

Testing: `qemu-aarch64` runs foreign binaries directly on your machine, and the kernel's `binfmt_misc` mechanism makes that transparent — so you can run a cross-compiled test suite as if it were native.

Module 19 Rules

#	Rule
1	Build with two compilers — GCC and Clang catch different bugs
2	`-Wall` is not all warnings; the full set nearly doubles coverage
3	`-Werror` in CI; `-Werror=` on always-a-bug warnings everywhere
4	`-O0` → `-O3` is ~8× — measured. Never ship `-O0`
5	`-O2` for production, `-Os` for embedded, `-O3` only if measured faster
6	`-Ofast` breaks IEEE 754 — verify your numerics before using it
7	`-g` is orthogonal to `-O`; `-O2 -g` is a valid production build
8	Never ship `-march=native`; use `-mtune=generic` or a v2/v3 baseline
9	Read the assembly. Godbolt makes it trivial
10	LTO for release builds; expect it to expose latent UB
11	`-ffunction-sections -fdata-sections -Wl,--gc-sections` for size
12	Split the debug info (`objcopy --only-keep-debug`); don't just strip
13	`addr2line` + the archived debug file turns crash logs into bug reports
14	Never `ldd` an untrusted binary — use `readelf -d`
15	Cross-compiling needs a sysroot, not just a differently named `gcc`

Practice

1. **Build your warning set.** Write a file that triggers 20 distinct warnings. Compile it with `-Wall`, then `-Wall -Wextra`, then your full set, and record which flag caught which problem. Then run it through Clang and compare.

2. **Optimisation benchmark.** Take a compute-heavy function of your own. Measure it at `-O0`, `-O1`, `-O2`, `-O3`, `-Os`, and `-Ofast`, recording both time and `.text` size. Find a case where `-O3` is slower than `-O2` (hint: something instruction-cache bound, or a hot loop that the unroller bloats).

3. `-march` **portability failure.** Build with `-march=native`. Then check `/proc/cpuinfo`, pick an instruction-set extension your CPU has, and confirm with `objdump -d | grep vpaddd` (or similar) that such instructions are in the binary. Explain why shipping it would break on older hardware.

4. **Vectorisation hunt.** Write three loops: one that vectorises, one that doesn't because of a data dependency, and one that doesn't because of possible aliasing. Use `-fopt-info-vec-missed` to see why. Fix the aliasing one with `restrict` (Module 2) and confirm that it now vectorises.

5. **Assembly-reading drill.** For each of `if`/`else`, `switch` (dense and sparse), a `for` loop, a function call, and a structure return, dump the `-O0` and `-O2` assembly and annotate every instruction. Then do the same on Godbolt for ARM64 and compare.

6. **LTO experiment.** Split a program into 5 `.c` files with cross-file calls. Measure binary size, `.text` size, symbol count, link time, and run time with and without `-flto`. Then introduce a strict-aliasing violation between two files and see whether LTO exposes it.

7. **Section GC.** Build a static library with 50 functions, and link a program that uses 3 of them. Measure with and without `-ffunction-sections -Wl,--gc-sections`. Use `-Wl,-Map=out.map` to see exactly what got included.

8. **Full binutils tour.** Take one binary and run every tool on it: `file`, `size`, `nm`, `readelf -h/-S/-d/-l`, `objdump -d/-S/-t`, `strings`, and `ldd`. Write one sentence explaining what each told you.

9. **Crash → source.** Write a program that segfaults in a nested function. Enable core dumps (`ulimit -c unlimited`), get the faulting address from the core file or from a signal handler using `backtrace()`, and use `addr2line` to find the exact line. Then repeat after splitting the debug info out, proving that the workflow survives stripping.

10. **Cross-compile something.** Install `gcc-aarch64-linux-gnu` and `qemu-user-static`. Build a hello-world for ARM64, verify with `file` that it is an ARM binary, and run it under QEMU. Then write a CMake toolchain file and cross-build a small project with it.

Module 19 in Summary

`gcc` is a driver that runs the preprocessor, compiler, assembler, and linker for you; GCC, Clang, and MSVC differ in extensions and C support, so isolate extensions behind feature-tested macros and build with at least two compilers. `-Wall` is only a starting point: a deliberate warning set, where each flag maps to a known class of bug, nearly doubles what the compiler catches — make it fatal in CI, and make always-a-bug warnings fatal everywhere.

Optimisation is the biggest free performance lever — `-O2` was 7× faster than `-O0` here — while `-Os` minimises size, `-O3` must be measured, `-Ofast` abandons IEEE 754, and `-march=native` must never ship. `-g` combines freely with optimisation. Reading the generated

assembly, locally or on Godbolt, is how you verify what the compiler really did. LTO optimises across translation units and section garbage collection discards unused code, both shrinking binaries.

The binutils — `file`, `size`, `nm`, `readelf`, `objdump`, `addr2line`, `strip`, `objcopy`, `strings`, `ldd` — let you inspect any binary; splitting debug information into a separate file lets you ship small binaries and still map production crashes back to source lines. Cross-compilation needs a matching toolchain *and* a sysroot, integrated through a CMake toolchain file or Autotools' `--host`, and can be tested under QEMU.

Next: Module 20 — Build Systems & Libraries, covering real Makefiles (pattern rules and automatic dependency generation), CMake as the industry default, static versus shared libraries, symbol visibility, the ABI-versus-API distinction and versioning, and header design — the practical skills for making a codebase that more than one person can build.

Module 20 — Build Systems & Libraries

In this module:

- Make (real Makefiles, pattern rules, dependency generation)
- CMake (the industry default), Meson, Ninja, Bazel; the Autotools legacy
- Static libraries (`.a`) vs shared libraries (`.so`/`.dll`/`.dylib`)
- Position-independent code, symbol visibility, `-fvisibility=hidden`
- Symbol resolution, link order, undefined and duplicate symbols
- ABI vs API, soname, versioning, backward compatibility
- `pkg-config`, dependency management (Conan, vcpkg)
- Header design: what goes in `.h` vs `.c`, minimising includes, forward declarations

All output in this module is real, from GNU Make 4.3, CMake 4.4, and GNU binutils.

Compiling a single file by hand is easy. A real codebase has dozens or thousands of files, several build configurations, libraries of its own, libraries from other people, and many developers who must all be able to build it identically — on their laptops, in CI, and on the release machine. This module is about making a codebase that more than one person can build: build systems that rebuild exactly what has changed, libraries that other programs can link against, and interfaces that can evolve without breaking the binaries already in the field.

20.1 Make

Make is the oldest and simplest build tool still in wide use. A *Makefile* declares *targets* (files to produce), their *prerequisites* (the files they are built from), and the *recipe* (shell commands) that builds one from the others. Make compares file timestamps and runs a recipe only when a target is missing or older than one of its prerequisites.

A production Makefile

```
CC       ?= gcc
CSTD     := -std=c17
WARN     := -Wall -Wextra -Wpedantic -Wshadow -Wconversion -Wstrict-prototypes \
            -Wcast-qual -Wpointer-arith -Wwrite-strings -Wformat=2 \
            -Werror=implicit-function-declaration -Werror=return-type
OPT      ?= -O2 -g
CPPFLAGS := -Iinclude -MMD -MP        # ← automatic dependency generation
CFLAGS   := $(CSTD) $(WARN) $(OPT)

BUILD  := build
SRCS   := $(wildcard src/*.c)
OBJS   := $(SRCS:src/%.c=$(BUILD)/%.o)
DEPS   := $(OBJS:.o=.d)
TARGET := $(BUILD)/app

.PHONY: all clean test debug release
all: $(TARGET)

$(TARGET): $(OBJS)
    @mkdir -p $(@D)
    $(CC) $(LDFLAGS) $^ $(LDLIBS) -o $@
```

```
$(BUILD)/%.o: src/%.c                 # PATTERN RULE — one rule for every .c
    @mkdir -p $(@D)
    $(CC) $(CPPFLAGS) $(CFLAGS) -c $< -o $@

debug:   OPT := -O0 -g3 -fsanitize=address,undefined
debug:   LDFLAGS += -fsanitize=address,undefined
debug:   clean all

release: OPT := -O3 -DNDEBUG -flto
release: LDFLAGS += -flto
release: clean all

test: $(TARGET); ./$(TARGET)
clean: ; $(RM) -r $(BUILD)

-include $(DEPS)                      # ← pull in the generated dependencies
```

Walking through it: the variables at the top collect the settings from Modules 19 and earlier. `SRCS` finds every `.c` file in `src/`, and the substitution `$(SRCS:src/%.c=$(BUILD)/%.o)` turns each `src/name.c` into `build/name.o`. The *pattern rule* `$(BUILD)/%.o: src/%.c` tells Make how to build *any* object file from the matching source file, so adding a new `.c` file needs no Makefile change at all. `debug` and `release` use *target-specific variables* to change the flags for that build only. (Each is written as `clean all` so that a switch between configurations rebuilds everything; the build directory does not record which flags produced it.)

Automatic dependency generation — the part most Makefiles get wrong

Make knows that `build/main.o` depends on `src/main.c`, because the pattern rule says so. But `main.c` also `#include`s headers, and if a header changes, `main.o` must be rebuilt too. Listing those header dependencies by hand is tedious, and the list is always out of date.

`-MMD -MP` makes the compiler do it: while compiling, it writes a `.d` file alongside each `.o`:

```
$ cat build/main.d
build/main.o: src/main.c include/mathlib.h include/strlib.h
include/mathlib.h:
include/strlib.h:
```

The first line is a genuine Make rule declaring the header dependencies. `-include $(DEPS)` pulls all the `.d` files into the Makefile (the leading `-` means “don’t complain if they don’t exist yet,” as on the very first build), and Make now knows the full dependency graph without you maintaining it.

Proof that it works:

```
-- touch a .c: only that file rebuilds --
   rebuilt: src/strlib.c

-- touch a HEADER: everything that includes it rebuilds --
   rebuilt: src/main.c
   rebuilt: src/mathlib.c

-- touch nothing --
make: Nothing to be done for 'all'.
```

`-MP` matters too. It emits those empty `include/mathlib.h:` targets. Without them, deleting or renaming a header makes the build fail with `No rule to make target 'include/foo.h'` — because the old `.d` file still claims the object depends on it — instead of simply rebuilding. Every developer has hit this; the fix is one flag.

Use `-MMD` (not `-MD`) to leave out system headers: they don't change between builds, and tracking them bloats the `.d` files.

Automatic variables and syntax

Syntax	Meaning
`$@`	the target
`$<`	the first prerequisite
`$^`	all prerequisites (deduplicated)
`$?`	prerequisites newer than the target
`$(@D)` / `$(@F)`	directory / file part of the target
`:=`	immediate assignment (evaluated now)
`=`	recursive assignment (evaluated at each use — beware)
`?=`	assign only if not already set — lets `CC=clang make` work
`+=`	append

The difference between `:=` and `=` matters: with `=`, the right-hand side is re-expanded every time the variable is used, so a `$(shell ...)` or `$(wildcard ...)` in it runs again and again, and a variable that refers to itself loops. Prefer `:=` unless you specifically need late expansion.

Rules that save pain

- **`.PHONY:` every target that is not a file** (`all`, `clean`, `test`, `install`). Otherwise a stray file named `clean` makes `make clean` do nothing, because the "target" is up to date.
- **Recipes need real tab characters, not spaces.** This is Make's most infamous wart: a recipe line indented with spaces produces `missing separator`.
- **Each recipe line runs in its own shell.** A `cd` on one line does not affect the next. Join lines with `\`, or use `.ONESHELL:`, for multi-line logic.
- **`$(shell ...)` runs at parse time; `$$` escapes a shell `$`** inside a recipe.
- **Avoid recursive Make** (`make -C subdir` from a parent Makefile). It splits the dependency graph into pieces that cannot see each other, which breaks both parallelism and correctness. Read *Recursive Make Considered Harmful* (Peter Miller, 1997); use a single non-recursive Makefile or a real build system.
- **`make -j$(nproc)`** builds in parallel; `make -n` does a dry run; `make -p` dumps Make's internal database; and `make --debug=b` shows *why* each target was rebuilt.

20.2 CMake and the Alternatives

CMake is the industry default — not because it is elegant, but because it is what everyone else uses, and every IDE, package manager, and CI system supports it. Strictly, CMake is a *build-system generator*: it reads a description of your project and generates the actual build files for Make, Ninja, Visual Studio, or Xcode.

The same project in CMake

```
cmake_minimum_required(VERSION 3.20)
project(mathapp VERSION 1.2.3 LANGUAGES C)

set(CMAKE_C_STANDARD 17)
set(CMAKE_C_STANDARD_REQUIRED ON)
set(CMAKE_C_EXTENSIONS OFF)               # -std=c17, not -std=gnu17
set(CMAKE_EXPORT_COMPILE_COMMANDS ON)     # compile_commands.json for clangd/clang-tidy

add_library(mathlib src/mathlib.c src/strlib.c)

target_include_directories(mathlib
    PUBLIC $<BUILD_INTERFACE:${CMAKE_CURRENT_SOURCE_DIR}/include>
           $<INSTALL_INTERFACE:include>)

target_compile_options(mathlib PRIVATE -Wall -Wextra -Wpedantic -Wshadow -Wconversion)

set_target_properties(mathlib PROPERTIES
    VERSION   ${PROJECT_VERSION}          # libmathlib.so.1.2.3
    SOVERSION ${PROJECT_VERSION_MAJOR}    # libmathlib.so.1
    C_VISIBILITY_PRESET hidden
    POSITION_INDEPENDENT_CODE ON)

add_executable(app src/main.c)
target_link_libraries(app PRIVATE mathlib)   # inherits includes and flags transitively

include(GNUInstallDirs)
install(TARGETS mathlib app EXPORT mathlibTargets
        LIBRARY DESTINATION ${CMAKE_INSTALL_LIBDIR}
        ARCHIVE DESTINATION ${CMAKE_INSTALL_LIBDIR}
        RUNTIME DESTINATION ${CMAKE_INSTALL_BINDIR})
install(DIRECTORY include/ DESTINATION ${CMAKE_INSTALL_INCLUDEDIR})

enable_testing()
add_test(NAME smoke COMMAND app)
```

```
$ cmake -S . -B build -G Ninja -DCMAKE_BUILD_TYPE=Release
-- Build files have been written to: .../build

$ cmake --build build
[3/5] Linking C static library libmathlib.a
[5/5] Linking C executable app

$ ctest --test-dir build --output-on-failure
100% tests passed out of 1
```

The first command *configures* the project (reads `CMakeLists.txt`, checks the compiler, and generates Ninja files in `build/`); the second builds it; the third runs the tests. The `$<BUILD_INTERFACE:...>` and `$<INSTALL_INTERFACE:...>` *generator expressions* give consumers the right include directory depending on whether they use the library from its build tree or after installation. (The library was built as static here because shared was not requested; setting `BUILD_SHARED_LIBS=ON` switches it, and the version and visibility properties then take effect.)

Modern (“target-based”) CMake — the only style worth learning

Everything is a property of a *target* (a library or executable), and every property has a visibility keyword:

Keyword	Applies to
`PRIVATE`	this target only
`INTERFACE`	consumers only
`PUBLIC`	both

That is why `target_link_libraries(app PRIVATE mathlib)` is enough: `app` automatically receives `mathlib`’s `PUBLIC` include directories and compile definitions, but not its `PRIVATE` warning flags. There is no manual `include_directories()` anywhere. Dependencies compose correctly through any number of layers.

> **Do not use the old commands:** `include_directories()`, `link_directories()`, `add_definitions()`, `set(CMAKE_C_FLAGS ...)`. They are global, and they do not compose. Every tutorial older than about 2018 shows them. Use the `target_*` family exclusively.

Other essentials:

- **Always build out of source** (`-B build`), so generated files never mix with your sources.
- **`CMAKE_BUILD_TYPE`** selects `Debug`, `Release`, `RelWithDebInfo` (`-O2 -g`, Module 19), or `MinSizeRel`.
- **Generator expressions** such as `$<$<CONFIG:Debug>:-fsanitize=address>` apply settings only to certain configurations.
- **`FetchContent`** downloads and builds dependencies at configure time; **`find_package()`** finds installed ones through their config files.
- **`CMAKE_EXPORT_COMPILE_COMMANDS`** writes `compile_commands.json`, which records the exact compile command for every file — and powers clangd, clang-tidy, and every modern editor’s code intelligence.

The landscape

System	Verdict
CMake	The default. An ugly language, but universal support. Learn it.
Ninja	Not a build system you write by hand — a fast back end. Its files are machine-generated. Use it as CMake’s generator.
Meson	Much nicer syntax, generates Ninja, excellent cross-compilation. Used by GNOME, systemd, and Mesa. Genuinely better than CMake if you control the whole project.

System	Verdict
Bazel	Google's. Hermetic, reproducible builds with remote caching. A huge win at monorepo scale, and heavy for anything smaller.
Autotools	`./configure && make && make install`. Legacy: slow, opaque, built on M4 macros. You will encounter it; don't start new projects with it.
Plain Make	Fine for small projects. Portability across platforms is the limit.
xmake, premake, SCons, Waf	Niche.

Practical advice: use CMake with the Ninja generator. Learn Meson if you get the choice. Know how to build an Autotools project, because half of Linux still ships that way.

20.3 Static vs Shared Libraries

A *library* is compiled code packaged for reuse by other programs. There are two kinds.

```
# static
gcc -c mathlib.c strlib.c
ar rcs libmath.a mathlib.o strlib.o

# shared
gcc -fPIC -c mathlib.c strlib.c
gcc -shared -Wl,-soname,libmath.so.1 mathlib.o strlib.o -o libmath.so.1.2.3
ln -sf libmath.so.1.2.3 libmath.so.1
ln -sf libmath.so.1 libmath.so
```

```
static binary : 16328 bytes
shared binary : 16176 bytes
libmath.so    : 15688 bytes
```

```
$ ar t libmath.a
mathlib.o
strlib.o   ← a .a is literally just an archive of .o files
```

A *static* library is an archive of object files; at link time, the linker copies the code the program needs into the executable. A *shared* library is a separate file loaded at run time by the dynamic loader, and shared between every program that uses it. (The symbolic links and the `-soname` are explained in §20.6.)

	Static `.a`	**Shared `.so` / `.dll` / `.dylib`**
Linked at	build time (copied in)	load time (referenced)
Binary size	larger	smaller
Deployment	a single self-contained file	must ship the library too
Security fixes	rebuild everything	replace one file
Memory across processes	one copy each	shared
Start-up	faster	dynamic-linker work

	Static `.a`	**Shared `.so` / `.dll` / `.dylib`**
Cross-TU optimisation	✓ (with LTO)	✗ across the boundary
Version conflicts	impossible	“DLL hell”

The “security fixes” row is often decisive. When a vulnerability is found in a widely used library such as zlib or OpenSSL, a system using shared libraries is fixed by updating one package; every statically linked program must be found, rebuilt, and redeployed.

A `.a` contributes only the `.o` files that are actually needed — whole object files at a time, which is the granularity problem that `-ffunction-sections` (Module 19) solves.

Platform notes: macOS uses `.dylib` files and an *install name* instead of a soname; Windows uses a `.dll` plus a `.lib` *import library* for linking, and requires explicit `__declspec(dllexport)`/`__declspec(dllimport)` markings.

20.4 PIC and Symbol Visibility

Position-independent code

A shared library can be loaded at a different address in every process that uses it, so all of its code must work at any address — it must be *position-independent*. That is what `-fPIC` produces. (`-fPIE` does the same for executables, and is the default on modern distributions, because it lets ASLR randomise the program’s own address as well as its libraries’.)

Position-independent code reaches global variables through the *GOT* (Global Offset Table) and calls exported functions through the *PLT* (Procedure Linkage Table), which costs an extra indirection per access. `-fno-plt` and `-Bsymbolic` reduce that cost for calls within the library.

`-fvisibility=hidden` — measured

```c
int public_a(int x);          /* intended API */
int public_b(int x);
int internal_helper1(int x);  /* implementation detail */
int internal_helper2(int x);
static int truly_private(int x);
```

```
default visibility            : 5 exported symbols
   internal_helper1 internal_helper2 public_a public_b use_all

-fvisibility=hidden           : 0 exported symbols

hidden + explicit API markers : 2 exported
   public_a public_b

sizes: default=15424 hidden+API=15256
```

By default, every non-`static` symbol in a shared library is exported. That means your internal helpers are part of your ABI whether you meant them to be or not; users can call them (and then complain when you change them); name collisions with other libraries become possible; the dynamic symbol table is bloated, slowing program start-up; and every call to them goes through the PLT.

The correct pattern:

```
/* mylib_export.h */
#if defined(_WIN32)
#  ifdef MYLIB_BUILDING
#    define MYLIB_API __declspec(dllexport)
#  else
#    define MYLIB_API __declspec(dllimport)
#  endif
#elif defined(__GNUC__)
#  define MYLIB_API __attribute__((visibility("default")))
#else
#  define MYLIB_API
#endif

MYLIB_API int public_a(int x);   /* in the header */
```

Build with `-fvisibility=hidden`, and mark only the API. On Windows the same macro expands to “export” while building the library (`MYLIB_BUILDING` defined) and “import” when a program uses it. CMake does all of this for you with `C_VISIBILITY_PRESET hidden` plus `generate_export_header()`.

Benefits: smaller binaries, faster load times, better optimisation (the compiler knows nothing outside the library can call a hidden function, so it can inline or specialise it freely), no accidental ABI surface, and no symbol collisions.

The alternative is a *version script* (`-Wl,--version-script=lib.map`), which lists the exported symbols explicitly and also enables symbol versioning (§20.6) — more powerful, and more work.

20.5 Symbol Resolution and Link Order

⚠ Static library link order matters

`libA` depends on `libB`:

```
-- WRONG order (-lB before -lA) --
la.c:(.text+0x9): undefined reference to `b_func'

-- RIGHT order (-lA then -lB) --
42
```

> The linker processes its arguments left to right, once each. When it reaches a static library, it takes only the members that resolve symbols *currently* undefined, then moves on. If `libA` then introduces a new undefined symbol and `libB` has already been processed, it is too late.
>
> **Rule:** a library must appear *after* everything that uses it.

In the wrong order, when the linker reached `-lB` nothing yet needed `b_func`, so it took nothing from `libB`. Then `-lA` supplied `a_func` — which calls `b_func` — and the linker never looked back.

For circular dependencies, where `libA` needs `libB` and `libB` also needs `libA`:

```
gcc main.o -Wl,--start-group -lA -lB -Wl,--end-group -o prog
```

```
--start-group works: 42
```

The linker rescans the libraries in the group repeatedly until no new symbols are resolved. (Or just list a library twice. `--start-group` is cleaner.)

Shared libraries do not have this problem: the dynamic linker resolves everything at load time, looking across all loaded libraries.

Duplicate symbols

```
/usr/bin/ld: multiple definition of `shared_thing'; first defined here
/usr/bin/ld: multiple definition of `dup'; first defined here
```

This is the One Definition Rule from Module 1, enforced by the linker. The usual causes: a variable *defined* in a header (Module 3), a missing `static` on a helper function, or the same source file compiled into the program twice.

The three errors you'll see, and what they mean

Error	Cause
`undefined reference to 'X'`	A missing `.c` or `-lfoo`, the wrong link order, or a typo — check with `nm -u`
`multiple definition of 'X'`	Defined twice — check `nm` on each `.o`
`cannot find -lfoo`	The library is not on the `-L` search path

`nm -u prog.o` lists what a file *needs*; `nm --defined-only lib.a` lists what a library *provides*. That pair solves almost every link error.

20.6 ABI vs API ⚠

> **API** — what the *source code* sees: function names, signatures, types, and macros. Break it, and code fails to compile.
>
> **ABI** — what the *machine code* sees: structure layout and size, calling conventions, symbol names, enum widths, alignment. Break it, and already-compiled code *silently corrupts*.

Module 6 introduced the ABI as the rules for calling functions. For a library, the ABI is everything a compiled program has baked into it about that library.

A real ABI break, measured

```
/* v1.h — what the application compiled against */
struct Config { int width; int height; };
void show(const struct Config *c);

/* v2.c — a new library version. A field was INSERTED at the front. */
struct Config { int depth; int width; int height; };
```

The header change looks harmless from the source point of view: code that sets `width` and `height` still compiles. But:

```
-- compiled against v1, running against v1 --
  lib sees: w=800 h=600

-- SAME BINARY, library replaced with v2 --
  lib sees: d=800 w=600 h=-2058889728
```

The application was never recompiled. Its idea of the structure's layout — `width` at offset 0, `height` at offset 4, total size 8 — is baked into its machine code. The new library reads `depth` at offset 0, `width` at 4, and `height` at 8, which is past the end of the 8-byte object the application passed in. So `width` became `depth`, `height` became `width`, and `height` is whatever happened to lie next in memory.

No error, no crash — just wrong data. That is potentially a security bug, and it is exactly the kind of problem that appears only after a routine library update on a customer's machine.

What breaks the ABI

Breaks ABI	Safe
Adding, removing, or reordering struct members	Adding a new function
Changing a member's type or size	Adding members to a struct only ever allocated by the library (opaque!)
Changing a function's parameters or return type	Changing the implementation
Changing an enum's underlying type	Adding a value to the end of an enum (usually)
Changing alignment or packing	
Removing a symbol, or changing its visibility	
Changing the calling convention	

This is the strongest argument for opaque structures (Module 11, §11.8). If callers cannot see the layout — because they only ever hold a pointer, and the library allocates the object — then the library can change the layout freely without breaking anyone.

Soname and versioning

```
libmath.so       -> libmath.so.1      (dev symlink, used at build time)
libmath.so.1     -> libmath.so.1.2.3  (soname symlink, used at run time)
libmath.so.1.2.3                      (the real file)
```

```
$ readelf -d libmath.so.1.2.3 | grep SONAME
 0x000000000000000e (SONAME) Library soname: [libmath.so.1]

$ readelf -d app_shared | grep NEEDED
 0x0000000000000001 (NEEDED) Shared library: [libmath.so.1]
```

The *soname* is the name embedded in the library that identifies its ABI version. When a program is linked (with `-lmath`, which finds `libmath.so` via the development symlink), the linker records the library's soname — `libmath.so.1` — not its filename. At run time, the dynamic loader looks for a file called `libmath.so.1`, which is a symlink to the newest compatible version. So:

- **Bump the soname** (`.so.1` → `.so.2`) on any ABI break. Both major versions can then be installed side by side, and old binaries keep working with the old one.
- **Bump only the minor or patch number** for compatible changes. The soname symlink is updated to point at the new file, and old binaries pick up the fix automatically.

Semantic versioning for C libraries: MAJOR = ABI break, MINOR = API addition (ABI compatible), PATCH = bug fix.

Tools: `abi-compliance-checker` and `abidiff` (from libabigail) compare two builds of a library and report ABI differences. Run them in CI if you ship a library.

Symbol versioning (`--version-script`) lets a single `.so` contain several ABI versions of the same function, so that old binaries get the old behaviour and new ones the new. That is how glibc has maintained backward compatibility since 1997 — and why Module 1's `printf` appeared as `printf@GLIBC_2.2.5`.

20.7 `pkg-config` and Dependency Management

A consumer of your library needs to know which include directories and libraries to pass to the compiler. Hard-coding paths breaks on every other machine. `pkg-config` solves this with a small `.pc` file installed alongside the library:

```
Name: mathlib
Version: 1.2.3
Libs: -L${libdir} -lmath
Libs.private: -lm
Cflags: -I${includedir}
```

```
--cflags              : -I/usr/local/include
--libs                : -L/usr/local/lib -lmath
--libs --static       : -L/usr/local/lib -lmath -lm
--modversion          : 1.2.3
--atleast-version=1.0 : yes
--atleast-version=2.0 : no
```

```
zlib here: -lz
```

```
gcc $(pkg-config --cflags --libs zlib) prog.c -o prog
```

Ship a `.pc` file with every library you publish. It is about ten lines, and it makes your library usable from Make, CMake, Meson, and Autotools without anyone hard-coding paths.

In CMake:

```
find_package(PkgConfig REQUIRED)
pkg_check_modules(ZLIB REQUIRED IMPORTED_TARGET zlib)
target_link_libraries(app PRIVATE PkgConfig::ZLIB)
```

`Libs.private` and `Requires.private` are emitted only with `--static`. That captures the distinction between "what my consumers need in order to link against me" and "what I need internally": a program linking the *shared* `libmath` does not need `-lm`, because the shared library already records its own dependency on it, but a program linking the *static* `libmath.a` does.

The package managers

Option	Notes
System packages (apt, dnf, brew)	Simplest, but you get whatever version the distribution ships
Conan	Decentralised, binary caching, good CMake integration. The most C/C++-native option.
vcpkg	Microsoft's; source-based, a huge catalogue, excellent CMake integration
CMake FetchContent	Zero infrastructure — clones and builds a dependency at configure time. Great for a small number of dependencies.
Git submodules / vendoring	Full control and reproducibility, but painful to update
Meson subprojects / wraps	Meson's built-in equivalent

C has no universal package manager, and that is a real weakness compared with Rust, Go, or Python. Most projects vendor their dependencies (copy them into the source tree) or rely on system packages.

20.8 Header Design

What goes where

Header (`.h`)	Implementation (`.c`)
Include guard	`#include` of its own header, first

Header (`.h`)	Implementation (`.c`)
Public type declarations, opaque `typedef`s	Private struct definitions
Function declarations	Function definitions
`#define` constants, public macros	`static` helpers
`static inline` for tiny hot functions	Global variable definitions
`extern` declarations of globals	

Never in a header: variable definitions (Module 3's "multiple definition"), `static` variables (each translation unit gets its own copy — the Module 3 trap), function definitions other than `static inline`, and unprefixed generic macros such as `min`, `max`, or `DEBUG` (Module 12).

The rules

1. Self-contained. A header must compile on its own:

```
/* widget.h */
#ifndef WIDGET_H
#define WIDGET_H
#include <stddef.h>   /* it uses size_t, so it includes stddef */
...
#endif
```

A header that only works if something else was included first is a trap for every future user. Enforce the rule with a test that compiles each header alone.

2. Include what you use, and no more. Module 12 measured `<stdio.h>` at 812 lines. Every unnecessary include is multiplied across every translation unit that includes your header.

3. Forward-declare instead of including. If you only need a pointer:

```
/* GOOD */
struct Database;   /* incomplete type is enough */
int query(struct Database *db, const char *sql);

/* BAD */
#include "database.h"   /* pulls in everything it includes */
```

4. Include your own header first in the `.c` file. If the header is missing an include it needs, this is the file where compilation fails — immediately, and in the right place — which proves the header is self-contained.

5. Prefix everything. C has no namespaces: `ml_add`, `ML_MAX_SIZE`, `MLConfig`. Module 11's rule for enum constants generalises to the whole API.

6. Use opaque types for anything non-trivial (Module 11, §11.8) — ABI stability, encapsulation, and fewer header dependencies all at once.

7. Add `extern "C"` guards if C++ code will consume your header:

```
#ifdef __cplusplus
extern "C" {
#endif
```

```
    /* ... declarations ... */
#ifdef __cplusplus
}
#endif
```

C++ compilers *mangle* function names to encode their parameter types; `extern "C"` tells a C++ compiler to use plain C names, so that it can link against your library (Module 30).

Include-order convention

```
/* widget.c */
#include "widget.h"   /* 1. own header — proves it's self-contained */

#include <stdio.h>    /* 2. system headers */
#include <stdlib.h>

#include "log.h"      /* 3. project headers */
#include "util.h"
```

Tools: include-what-you-use (IWYU) analyses a codebase and reports unnecessary or missing includes. It is worth running once on any codebase of more than about 50 files.

Module 20 Rules

#	Rule
1	`-MMD -MP` in `CPPFLAGS` + `-include $(DEPS)` — never hand-maintain header dependencies
2	Pattern rules, `.PHONY`, `?=` for overridable variables, tabs not spaces
3	Avoid recursive Make
4	CMake with Ninja; target-based only — never `include_directories()`
5	Always out-of-source builds; always `CMAKE_EXPORT_COMPILE_COMMANDS`
6	Static for self-contained deployment; shared for updatability and memory sharing
7	`-fPIC` for shared libraries; `-fvisibility=hidden` + explicit API markers
8	Static libraries link left to right — put them after their users
9	`--start-group`/`--end-group` for circular dependencies
10	API breaks fail to compile; ABI breaks silently corrupt
11	Bump the soname on any ABI break; opaque structs prevent most of them
12	Ship a `.pc` file with every library

#	Rule
13	Headers must be self-contained; forward-declare instead of including
14	Include your own header first in the `.c`
15	Prefix every public identifier — C has no namespaces

Practice

1. **Build the Makefile.** Write the §20.1 Makefile for a five-file project. Prove that dependency tracking works: touch a `.c`, touch a header, and delete a header. Then remove `-MP` and confirm that the deleted-header failure appears.

2. **Add targets.** Extend it with `install`, `uninstall`, `format` (clang-format), `tidy` (clang-tidy using `compile_commands.json`), `coverage`, and `analyze`. Make `make -j8` work correctly.

3. **CMake conversion.** Convert the same project to CMake, with a library target, an executable, install rules, and `ctest`. Build with both the Make and Ninja generators and compare build times. Then add a Debug build with sanitizers via a generator expression.

4. **Static vs shared, measured.** Build the same library both ways. Compare binary size, start-up time (`perf stat`, or a loop of 1,000 executions), and what `ldd` reports. Then change one function in the library and measure what has to be rebuilt in each case.

5. **Visibility audit.** Take a library with 20 functions, 5 of them public. Count the exported symbols with default visibility, with `-fvisibility=hidden`, and with hidden plus explicit markers. Measure the `.so` size and the dynamic symbol table size (`readelf -sD | wc -l`) each time.

6. **Link-order failure.** Build three static libraries with a chain of dependencies. Find an ordering that fails. Fix it with the correct ordering, then with `--start-group`. Then make the dependency circular and confirm that only `--start-group` works.

7. **Break an ABI on purpose.** Reproduce the §20.6 experiment. Then try four more breaks: reorder members, widen a member, change a parameter type, and change an enum’s underlying type. For each, record whether it crashes, corrupts silently, or happens to work.

8. **Fix it with opacity.** Rewrite the same library with an opaque `Config` type plus accessor functions. Repeat every ABI break from exercise 7, and confirm that none of them affects the old binary.

9. **Version a library properly.** Build `libfoo.so.1.0.0` with a soname and link a program against it. Then ship 1.1.0 (compatible) and 2.0.0 (incompatible). Verify that the old binary picks up 1.1.0 automatically and refuses 2.0.0. Install `abidiff` and confirm that it detects the break.

10. **Header hygiene pass.** Take a real project (yours or an open-source one). For each header, ask: is it self-contained? Compile each one alone and fix what fails. Then run include-what-you-use and measure the build-time improvement from removing unnecessary includes.

Module 20 in Summary

A good build system rebuilds exactly what changed. In Make, that means pattern rules, `.PHONY` targets, overridable `?=` variables, a single non-recursive Makefile, and compiler-generated dependency files (`-MMD -MP`) instead of hand-written header lists. CMake is the industry default: use it in the modern target-based style — properties with `PRIVATE`/`PUBLIC`/`INTERFACE` visibility that propagate through `target_link_libraries` — with the Ninja generator, out-of-source builds, and `compile_commands.json`. Meson is a cleaner alternative; Autotools is legacy you must still be able to build.

Static libraries are archives copied into each program; shared libraries are position-independent code loaded at run time, easier to update and shared in memory. Export only your real API with `-fvisibility=hidden` and explicit markers. Static libraries must follow their users on the link line, circular dependencies need `--start-group`, and `nm` diagnoses almost every link error.

An API break fails to compile, but an ABI break — a reordered structure, a changed type — silently corrupts programs that were never rebuilt, as the measured `Config` example showed. Opaque types prevent most breaks; the soname, bumped on every incompatible change, lets old and new versions coexist. Ship a `pkg-config` file with every library, and design headers to be self-contained, minimal, prefixed, and included first in their own `.c` files.

Next: Module 21 — Debugging & Analysis, the module that most changes your day-to-day effectiveness: GDB properly (not just `break` and `print`), core dumps and post-mortem debugging, Valgrind’s full tool suite, sanitizers in depth, `strace`/`ltrace`/`perf`, `rr` for reverse debugging, and the static analysers that find bugs before you run anything.

Module 21 — Debugging & Analysis

In this module:

- GDB/LLDB: breakpoints, watchpoints, backtraces, inspecting memory, TUI, scripting
- Core dumps and post-mortem debugging
- Valgrind (memcheck, callgrind, helgrind, massif)
- Sanitizers: ASan, UBSan, TSan, MSan, LeakSanitizer — non-negotiable in modern C
- `strace`, `ltrace`, `perf`, ftrace, bpftrace
- `rr` (record-and-replay debugging)
- Static analysis: clang-tidy, cppcheck, scan-build, Coverity, PVS-Studio
- Formal-ish tools: Frama-C, CBMC

All output in this module is real, from GDB 15.1, Valgrind 3.22, cppcheck 2.13, and clang-tidy (LLVM 22).

This is the module that most changes your day-to-day effectiveness. Most C programmers learn just enough of a debugger to set a breakpoint and print a variable, and fall back on adding `printf` calls and rebuilding for everything else. The tools in this module are far more powerful: they can tell you who wrote a bad value, reconstruct a crash from a dead process, trace a bug back to the allocation that caused it, replay an execution backwards, and find bugs in code that has never run. Learning them well is the difference between spending an afternoon on a bug and spending five minutes.

21.1 GDB

Build for debugging

```
gcc -Og -g3 -fno-omit-frame-pointer prog.c -o prog
```

- **`-g3`** includes macro definitions (so `print MY_MACRO` works), not just line tables.
- **`-Og`** optimises without destroying debuggability — better than `-O0` for large programs (Module 19).
- **`-fno-omit-frame-pointer`** keeps backtraces reliable, by keeping the chain of saved frame pointers (Module 6) intact.

> ⚠ At `-O2`, variables get “optimized out” and line numbers jump around, because the compiler has kept values only in registers, merged statements, or deleted code. Even at `-Og`, the first watchpoint attempt for this module failed with `value has been optimized out`. That is normal — and it is why you keep a dedicated debug build.

Batch mode — how you actually use GDB in scripts and CI

```
gdb -q -batch \
    -ex 'break average' -ex 'run' \
```

```
    -ex 'info args' -ex 'print *rs[0]' -ex 'bt' \
    ./buggy
```

```
Breakpoint 1, average (rs=0x5555555592a0, n=5) at buggy.c:21
21      for (size_t i = 0; i < n; i++) sum += rs[i]->score;
rs = 0x5555555592a0
n = 5
$2 = {id = 0, name = "alice\000\000\000\000\000\000\000\000\000\000", score = 0}
#0 average (rs=0x5555555592a0, n=5) at buggy.c:21
#1 0x0000555555555425 in main (argc=1) at buggy.c:41
```

Each `-ex` runs one GDB command, in order, and `-batch` exits when they are done. No interactive session is required. This form is scriptable, reproducible, and can be pasted straight into a bug report — anyone can rerun it and see the same thing.

The commands that matter

Category	Commands
Run	`run [args]`, `start` (break at `main`), `attach PID`, `run < in.txt`
Breakpoints	`break FILE:LINE`, `break FUNC`, `break FUNC if COND`, `tbreak` (temporary), `info break`, `delete`, `disable`
Stepping	`next/n` (step over), `step/s` (step into), `finish` (step out), `until`, `continue/c`, `advance LINE`
Inspect	`print EXPR`, `print/x`, `ptype`, `info locals`, `info args`, `display EXPR`
Memory	`x/16xb ptr`, `x/8dw arr`, `x/s str`, `x/4i $pc`
Stack	`bt`, `bt full`, `frame N`, `up`, `down`, `info frame`
Watchpoints	`watch EXPR`, `rwatch`, `awatch`
Threads	`info threads`, `thread N`, `thread apply all bt`
Data	`set var x = 5`, `call func(args)`, `return VALUE`

The `x` (“examine”) command reads raw memory: `x/16xb ptr` shows 16 bytes in hex, `x/8dw arr` shows eight 4-byte words as decimal, `x/s str` shows a string, and `x/4i $pc` disassembles the next four instructions. `ptype` shows an expression’s type — invaluable with complex declarations (Module 9). `call` runs a function in the debugged program, which lets you invoke your own debug helpers (a `dump_tree()`, say) from the prompt.

Conditional breakpoints — the biggest productivity win

```
gdb -q -batch -ex 'break make if id == 3' -ex run -ex 'info args' -ex finish -ex 'print *$' ./buggy0
```

```
Breakpoint 1, make (id=3, name=0x555555556014 "dave", score=7) at buggy.c:9
id = 3
name = 0x555555556014 "dave"
Value returned is $1 = (Rec *) 0x555555559360
$2 = {id = 3, name = "dave", '\000' <repeats 11 times>, score = 7}
```

The breakpoint fired only on the call with `id == 3`. `finish` then ran to the end of the function and showed its return value, and `print *$` dereferenced that value (`$` is the most recent value).

Stopping on iteration 10,000 by pressing `continue` 10,000 times is how beginners use GDB. `break f if i == 10000` is how professionals do it.

Related: `ignore N COUNT` skips the first `COUNT` hits of breakpoint `N`; `finish` runs to the end of the current function and shows the return value; and `$` refers to the last value printed.

Breakpoint command lists — `printf` debugging without recompiling

```
break make
commands
  silent
  printf "make(id=%d, name=%s, score=%.1f)\n", id, name, score
  continue
end
run
```

```
make(id=0, name=alice, score=0.0)
make(id=1, name=bob, score=6.0)
make(id=2, name=carol, score=1.0)
make(id=3, name=dave, score=7.0)
make(id=4, name=eve, score=2.0)
```

Every time the breakpoint is hit, GDB silently prints a formatted line and continues. This is instrumented tracing with zero source changes and zero rebuilds — enormously useful when a build takes minutes, or when the bug only reproduces in a deployed binary you cannot modify.

Watchpoints — “who changed my variable?”

```
break 35
run
watch rs[0]->score
continue
```

```
Hardware watchpoint 2: rs[0]->score
Old value = <unreadable>
New value = 0
0x00005555555554bb in main (...) at buggy.c:36
```

A *watchpoint* stops the program whenever an expression’s value changes, reporting the old value, the new one, and the exact line responsible. It is the single best tool for “this field has the wrong value, and I don’t know who wrote it.” (Here the watchpoint was set before the record was allocated, hence `<unreadable>` for the old value.)

x86 CPUs provide 4 *hardware* watchpoints, implemented by debug registers in the processor, which cost nothing while the program runs. Beyond that, GDB falls back to *software* watchpoints, which single-step the program and check the value after every instruction — thousands of times slower.

`rwatch` stops on a read, and `awatch` on a read or a write.

TUI and other front ends

```
gdb -tui ./prog   # or Ctrl-X A inside gdb
```

The text user interface splits the terminal into source and command panes. `layout src`, `layout asm`, `layout regs`, and `layout split` choose what is shown; `Ctrl-X 2` cycles through layouts.

Better options for daily work: gdb-dashboard (a single `.gdbinit` file), pwndbg or GEF (security-focused, with excellent memory views), your IDE's DAP integration (VS Code, CLion), or gdbgui in a browser.

`.gdbinit` — set it up once

```
set history save on
set history size 10000
set print pretty on
set print array on
set pagination off
set confirm off
set disassembly-flavor intel

define bta
  thread apply all backtrace full
end
```

This keeps command history across sessions, prints structures and arrays readably, stops GDB from pausing output or asking for confirmation, uses the Intel assembly syntax this book uses, and defines a `bta` command that dumps every thread's full backtrace — the first thing to run on a hung multithreaded program.

Project-local `.gdbinit` files (in the project directory) are ignored for security unless you allow them with `add-auto-load-safe-path` in your `~/.gdbinit`.

Python scripting

GDB embeds a Python interpreter. *Pretty-printers* are its highest-value use:

```
class RecPrinter:
    def __init__(self, val): self.val = val
    def to_string(self):
        return "Rec(id=%d, %s, %.1f)" % (self.val['id'], self.val['name'].string(), self.val['score'])

def lookup(val):
    if str(val.type.strip_typedefs()) == 'Rec': return RecPrinter(val)
    return None
gdb.pretty_printers.append(lookup)
```

Now `print *rec` shows `Rec(id=3, dave, 7.0)` instead of a raw structure dump. Ship pretty-printers with your data structures — a printer that walks a hash table or a list and shows its elements transforms the experience of debugging a custom container.

LLDB

LLDB is the LLVM debugger, and the default on macOS. The concepts are the same; the syntax differs:

GDB	LLDB
`break f`	`breakpoint set -n f` / `b f`
`info locals`	`frame variable`
`x/16xb p`	`memory read -c 16 -f x -s 1 p`
`bt`	`bt`

LLDB has better C++ support and a cleaner Python API; GDB has better C and embedded support. Learn whichever your platform defaults to — the skills transfer.

21.2 Core Dumps and Post-Mortem Debugging

A *core dump* is a snapshot of a process's memory and registers at the moment it crashed. Loading it into a debugger lets you inspect the crash as if you had been running under the debugger all along — after the fact, even on a different machine.

Enabling cores

```
ulimit -c unlimited
sudo sysctl -w kernel.core_pattern=/tmp/core.%e.%p
# systemd systems: coredumpctl list / coredumpctl gdb PID
```

Most systems disable core dumps by default (`ulimit -c 0`). The `core_pattern` controls where they are written; `%e` is the executable name and `%p` the process ID. On systemd-based systems, `systemd-coredump` usually captures them automatically, and `coredumpctl` lists and opens them.

Post-mortem GDB on a real core

```
gdb -q -batch -ex bt -ex 'frame 1' -ex 'info args' -ex 'info registers rip rsp' ./crash core.crash.608
```

```
Core was generated by `/home/claude/c_dbg/crash boom'.
Program terminated with signal SIGSEGV, Segmentation fault.
#0 0x000055740df1d179 in level3 (p=0x0) at crash.c:3
#1 0x000055740df1d199 in level2 (p=0x0) at crash.c:4
#2 0x000055740df1d1ba in level1 (p=0x0) at crash.c:5
#3 0x000055740df1d208 in main (argc=2) at crash.c:9

p = 0x0
rip 0x55740df1d199 <level2+28>
rsp 0x7ffcb1d67ea0
```

The full stack, every function's arguments, and the exact line — from a program that had already died. (`rip` shows `level2+28` because the registers are displayed for the selected frame, frame 1.) This is why core dumps matter in production: a crash that happens once a week on a customer's server can be diagnosed from a single core file.

Requirements: the exact same binary (or its split debug file, Module 19) and matching shared libraries. Archive both with every release.

Backtraces from a signal handler

When you cannot get a core dump — in containers, on embedded systems, or in restricted environments — a program can print its own backtrace when it crashes:

```c
#include <execinfo.h>
static void handler(int sig)
{
    void *frames[32];
    int n = backtrace(frames, 32);
    fprintf(stderr, "*** signal %d, %d frames ***\n", sig, n);
    backtrace_symbols_fd(frames, n, STDERR_FILENO);   /* avoids malloc */
    _exit(128 + sig);
}
```

```
*** caught signal 11, 9 frames ***
/home/claude/c_dbg/bt(+0x1221)[0x558170396221]
/lib/x86_64-linux-gnu/libc.so.6(+0x45330)[0x7fc2d9045330]
/home/claude/c_dbg/bt(main+0x26)[0x5581703962f5]
```

Build with `-rdynamic` so that function names appear, and feed the offsets to `addr2line -e prog -f -C -i` (Module 19) to get file and line numbers.

> ⚠ `backtrace_symbols` calls `malloc`, and must not be used in a signal handler (Module 14) — the crash may have happened inside `malloc`. `backtrace_symbols_fd` writes straight to a file descriptor without allocating. Strictly, neither `fprintf` nor `backtrace` is on POSIX's async-signal-safe list either (`backtrace` may load a library the first time it runs, so call it once at start-up to "warm it up"); this handler is a pragmatic last-gasp tool, not a model of signal safety. And with PIE, subtract the load base before using `addr2line`.

For production: libunwind (more robust unwinding), Google's Breakpad or Crashpad, or a crash-reporting service such as Sentry.

21.3 Valgrind

Valgrind runs your program on a synthetic CPU, translating and instrumenting every instruction as it executes. It needs no recompilation — that is its superpower, and also the source of its 20–50× slowdown.

memcheck — the default tool

```
valgrind -q --leak-check=full --show-leak-kinds=all --track-origins=yes ./prog
```

```
==627== Invalid read of size 4
==627==    at 0x109455: main (buggy.c:43)
==627==  Address 0x4a7c0b0 is 0 bytes inside a block of size 32 free'd
==627==    at free (vgpreload_memcheck-amd64-linux.so)
==627==    by 0x109450: main (buggy.c:43)
==627==  Block was alloc'd at
==627==    at malloc (vgpreload_memcheck-amd64-linux.so)
==627==    by 0x10929D: make (buggy.c:9)
==627==    by 0x109368: main (buggy.c:36)
```

Three stack traces: where the bad access happened, where the block was freed, and where it was allocated. That is the complete story of a use-after-free. (The `==627==` prefix is the process ID.)

`--track-origins=yes` — the killer feature

```
static int compute(void){ int x; return x * 2; }
```

```
==636== Conditional jump or move depends on uninitialised value(s)
==636==    at __printf_buffer (vfprintf-process-arg.c:58)
==636==    by 0x1091D9: main (uninit.c:4)
==636==  Uninitialised value was created by a stack allocation
==636==    at 0x109189: compute (uninit.c:3)   ← THE ORIGIN
```

Valgrind does not complain when you *copy* an uninitialized value — only when the program's behaviour depends on it (here, deep inside `printf`, when formatting the number). By then, the garbage may have passed through many functions. `--track-origins=yes` traces it back to the exact stack allocation that produced it.

Module 16's table showed that uninitialized reads are caught by neither ASan nor UBSan. This is the tool that catches them (along with Clang's MemorySanitizer). It costs roughly another 2× slowdown. It is worth it.

The tool suite

Tool	Purpose
memcheck (default)	Leaks, use-after-free, double free, uninitialized reads, invalid free, overlapping `memcpy`
helgrind	Data races and lock-order violations (deadlock potential)
drd	An alternative race detector, with lower memory use
callgrind	Instruction-level profiling plus a call graph
cachegrind	Cache and branch-prediction simulation
massif	A heap profiler over time — peak usage, and who allocated it
dhat	Heap access patterns: short-lived? read-only? over-sized?

callgrind:

```
Ir                  file:function
5,601,209 (97.74%)  prof.c:main
   24,256 ( 0.42%)  dl-lookup.c:do_lookup_x
```

`Ir` is "instructions read" — the number of instructions executed in each function. Because these are exact counts rather than statistical samples, callgrind's results are deterministic: the same run gives the same numbers, which makes them ideal for performance regression tests. Visualise them with KCachegrind.

massif:

```
    MB
1.391^ ::::::::::::::::::::::::::::::::::::::::#
     0 +------------------------------------------------------------------>Mi
       0                                                                   1.531
```

`ms_print ms.out` draws heap usage over time (here, against millions of instructions executed) and then shows a tree of which call sites were responsible for the memory at the peak. It is the tool for "why does my server use 4 GB?"

Practical notes

- `--error-exitcode=1` makes Valgrind usable in CI: the run fails if any error is found.
- `--gen-suppressions=all` writes suppression entries for known third-party noise, which you collect into a suppressions file.
- `--vgdb=yes --vgdb-error=1` lets GDB attach to the program at the first error, so you can inspect its state live.
- **Use `-O0` or `-O1` builds** — at higher optimisation levels, the line information in reports is less useful.

> ⚠ Do not combine Valgrind with sanitizers. They both take over memory management, and fight.

21.4 Sanitizers

Sanitizers work by *compile-time instrumentation*: the compiler inserts checks around memory accesses, arithmetic, and synchronisation, and links in a run-time library that tracks the state those checks need. They are much faster than Valgrind, and they catch some things Valgrind cannot — overflows of stack and global arrays, for instance, because Valgrind only knows about heap block boundaries.

The ASan report

```
==703==ERROR: AddressSanitizer: heap-use-after-free on address 0x503000000040
READ of size 4 at 0x503000000040 thread T0
    #0 in main buggy.c:43
freed by thread T0 here:
    #0 in free
    #1 in main buggy.c:43
previously allocated by thread T0 here:
    #0 in malloc
    #1 in make buggy.c:9
    #2 in main buggy.c:36
```

The same three-trace structure as Valgrind — access, free, allocation — in a fraction of the time.

Measured overhead

```
baseline (no sanitizer)        0.001s
-fsanitize=undefined           0.002s (~2×)
-fsanitize=address             0.014s (~14×)
-fsanitize=address,undefined   0.016s
-fsanitize=thread              0.025s (~25×)
valgrind memcheck              0.532s (~530×)
```

(This workload is tiny and allocation-heavy, so ASan's overhead is at the high end of its usual 2–3×, and Valgrind's start-up cost dominates its figure. The relative ordering is what matters: UBSan is nearly free, ASan is cheap, and Valgrind is expensive but needs no rebuild.)

The suite

Sanitizer	Catches	Overhead	Notes
ASan	Heap/stack/global overflow, use-after-free, double free, use-after-return	2–3×	Includes LSan by default on Linux
LSan	Leaks	~free	Standalone via `-fsanitize=leak`
UBSan	Signed overflow, shifts, alignment, null, bounds, enum/bool range	~1.2×	Enable everywhere
TSan	Data races, lock-order inversions	5–15×, + 5–10× memory	Module 17
MSan	Uninitialized reads	~3×	Clang only; needs every dependency instrumented
CFI	Control-flow integrity violations	small	Clang; needs LTO

MSan's "every dependency instrumented" requirement is what limits its use: any library built without MSan (including the C library) writes memory that MSan believes is uninitialized, producing false reports. That is why Valgrind's `--track-origins` remains the practical choice for most projects.

Incompatible combinations: ASan, TSan, and MSan are mutually exclusive — each needs its own build.

Run-time options

```
ASAN_OPTIONS=detect_leaks=1:detect_stack_use_after_return=1:strict_string_checks=1:abort_on_error=1
UBSAN_OPTIONS=print_stacktrace=1:halt_on_error=1
TSAN_OPTIONS=halt_on_error=1:second_deadlock_stack=1
LSAN_OPTIONS=suppressions=lsan.supp
```

`detect_stack_use_after_return=1` is off by default (in the GCC 13 runtime used here) and catches a real bug class — Module 9's dangling pointers to dead stack frames — so turn it on. `abort_on_error=1` makes ASan abort rather than exit, so that a core dump is produced; `print_stacktrace=1` gives UBSan reports a full stack.

Which to use when

- **Sanitizers for your own code:** they need a rebuild, but they are fast enough to run the whole test suite and give the best diagnostics for what they cover.
- **Valgrind for third-party binaries** you cannot rebuild, uninitialized-memory hunting with `--track-origins`, heap profiling with massif, and deterministic profiling with callgrind.
- **Both in CI, on different jobs.** They have different blind spots (Module 16's coverage table).

21.5 Tracing and Profiling

`strace` — system calls

```
strace -c -f ./prog                       # summary table
strace -e trace=openat,read,write ./prog  # filter
strace -f -o trace.log -p PID             # attach, follow forks
strace -T -tt ./prog                      # timestamps and durations
```

```
$ strace -c ./buggy
...
100.00 0.000000 0 34 1 total
```

`strace` shows every system call a program makes, with its arguments and result. It answers questions a debugger cannot: Which file did it actually open? Why did that `connect` fail? Is it spinning on a `futex`? Where is it blocked?

```
openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3
write(1, "sorted by score:\n alice 0.0\n "..., 98)
```

It is the first thing to run when a program fails mysteriously. The failing system call and its `errno` — `openat(..., "config.ini", ...) = -1 ENOENT (No such file or directory)` — are usually the whole answer.

`ltrace` does the same for library calls (`malloc`, `strcpy`, …), but it is less reliable with modern binaries hardened against PLT interception (Module 19's `-fno-plt`, full RELRO).

`perf` — the Linux profiler

```
perf stat ./prog                    # cycles, instructions, IPC, cache misses, branch misses
perf record -g ./prog               # sample with call graphs
perf report                         # interactive browser
perf top                            # live, system-wide
perf record -e cache-misses ./prog  # sample a specific hardware event
perf annotate                       # per-instruction attribution
```

`perf` reads the CPU's hardware performance counters and periodically samples where the program is executing, so it measures real code at nearly full speed.

Flame graphs are the standard visualisation of `perf` samples: each box is a function, its width is the fraction of samples in which it (or something it called) was running, and stacking shows who called whom — so the widest boxes at the top are where the time goes.

```
perf record -F 99 -g ./prog
perf script | stackcollapse-perf.pl | flamegraph.pl > out.svg
```

`perf stat` alone tells you an enormous amount. An IPC (instructions per cycle) below about 1.0 suggests the program is memory-bound — waiting for data rather than computing; a high branch-miss rate means unpredictable control flow. (Module 25 covers this properly.)

`perf` requires suitable `perf_event_paranoid` permissions, and is often unavailable in containers — as it was on the machine used for this book.

ftrace and bpftrace

ftrace is the kernel's built-in tracer, accessed through `/sys/kernel/debug/tracing`. It can record function graphs of kernel execution.

bpftrace uses eBPF — small, verified programs that run safely inside the kernel — through a scripting language designed for tracing production systems with near-zero overhead:

```
bpftrace -e 'uprobe:./prog:make { printf("make called with id=%d\n", arg0); }'
bpftrace -e 'tracepoint:syscalls:sys_enter_openat { @[comm] = count(); }'
bpftrace -e 'uprobe:libc:malloc { @bytes = hist(arg0); }'
```

The first prints every call to `make` in a running program; the second counts `openat` calls per process name across the whole system; the third builds a histogram of `malloc` sizes. This is the modern production-debugging tool: you attach to a running process with no restart, no instrumentation, and no measurable slowdown. Brendan Gregg's *BPF Performance Tools* is the reference.

`rr` — record and replay

```
rr record ./prog
rr replay   # a GDB session where you can go BACKWARDS
```

`rr` records an execution — every system call result, every signal, every source of non-determinism — and then replays it exactly, under GDB. Inside `rr replay` you get `reverse-continue`, `reverse-step`, `reverse-next`, and reverse watchpoints.

> This is a genuine superpower for hard bugs. You see corrupted memory; you set a watchpoint on it and `reverse-continue`; `rr` takes you directly to the write that corrupted it. No re-running, no guessing.

It also makes non-deterministic bugs deterministic: record once, and replay identically forever. That is ideal for race conditions and *heisenbugs* — bugs that vanish when you try to observe them.

Its overhead is only about 1.5–2×. It runs on Linux (x86-64, and more recently some ARM64 systems), and needs access to hardware performance counters, which are often unavailable in VMs and containers. When it works, it is the best debugging tool that exists.

21.6 Static Analysis

Static analysers find bugs without running the program — including on paths your tests never reach.

cppcheck

```
cppcheck --enable=all --inconclusive --std=c17 --quiet src/
```

```
analyze.c:9:11: style: Condition 'n=0' is always false [knownConditionTrueFalse]
    if (n = 0) return -1;
analyze.c:16:56: error: Memory leak: m [memleak]
```

cppcheck is standalone, needs no build integration, and has a low false-positive rate. Add `--suppress=missingIncludeSystem` to quiet the noise about system headers it does not parse, and `--xml` for CI integration.

clang-tidy

```
clang-tidy src/*.c --checks='clang-analyzer-*,bugprone-*,cert-*' -p build/
```

```
analyze.c:8:5: warning: Call to function 'strcpy' is insecure as it does not provide
    bounding of the memory buffer. CWE-119 [clang-analyzer-security.insecureAPI.strcpy]
analyze.c:9:11: warning: an assignment within an 'if' condition is bug-prone
    [bugprone-assignment-in-if-condition]
analyze.c:10:14: warning: Call to 'malloc' has an allocation size of 0 bytes
    [clang-analyzer-optin.portability.UnixAPI]
```

Note that last warning: the analyser followed `n = 0` through the (buggy) condition and deduced the allocation size on that path. That is *path-sensitive* analysis — reasoning about the values variables hold along each route through the code — not mere pattern matching.

clang-tidy uses `compile_commands.json` (Module 20's `CMAKE_EXPORT_COMPILE_COMMANDS`), via `-p build/`, so it sees your exact flags and include paths. Its check groups include `clang-analyzer-*` (deep, path-sensitive), `bugprone-*`, `cert-*` (the CERT C secure coding rules), `misc-*`, `performance-*`, and `readability-*`. Configure it with a `.clang-tidy` file in the project, and use `--fix` to apply many fixes automatically.

The rest

Tool	Notes
scan-build	Wraps your build with the Clang Static Analyzer and produces an HTML report with path visualisations
GCC `-fanalyzer`	Built into GCC 10 and later. Interprocedural; catches double free, use-after-free, leaks, and null dereferences. Free — just add the flag.
Coverity	Commercial, an industry standard, free for open source (scan.coverity.com). A very low false-positive rate.
PVS-Studio	Commercial; excellent at copy-paste bugs. Free for open source.
Infer (Meta)	Based on separation logic; good at memory and null issues
CodeQL	GitHub's. Query your code like a database; powers GitHub's security scanning

Use at least two. They have very different strengths, and static analysis is cheap.

Formal methods

Tool	What it does
Frama-C	ACSL contracts (pre-conditions, post-conditions, invariants) with deductive verification. Used in avionics and nuclear systems.
CBMC	A bounded model checker — exhaustively explores all paths up to a bound, and produces a concrete counterexample when a property fails
KLEE	Symbolic execution; generates test cases achieving high coverage
TIS-Interpreter	Interprets C while detecting undefined behaviour, including cases no run-time tool catches
Astrée	A commercial sound static analyser; proves the absence of run-time errors. Used for Airbus flight-control software.

These are for safety-critical code (DO-178C, ISO 26262) and for verifying small, critical functions — a parser, a cryptographic primitive, a state machine. They are not practical for a whole application, but transformative for the 200 lines that absolutely must be correct. A *sound* analyser like Astrée differs from the bug-finders above in one crucial way: if it reports no errors, there are provably none of the kinds it checks for.

21.7 The Workflow

A debugging methodology

1. **Reproduce it reliably.** A bug you cannot reproduce, you cannot fix — or know you have fixed. Minimise the failing input; tools such as `creduce` and delta-debugging automate this.
2. **Run the sanitizers.** ASan + UBSan + LSan first, then TSan, then Valgrind with `--track-origins`. Most memory bugs die here within minutes.
3. **Run the static analysers.** `-fanalyzer`, cppcheck, clang-tidy. They are free, and they find things tests do not.
4. **Read the error carefully.** The three-trace ASan or Valgrind report usually *is* the answer.
5. **Bisect.** `git bisect run ./test.sh` finds the commit that introduced the bug automatically, by binary search through history.
6. **Then reach for GDB** — with a conditional breakpoint or a watchpoint, not by stepping.
7. **Use `rr` for the truly hard ones.** Reverse-continue from the corruption to the code that corrupted it.
8. **Write a regression test before you fix it** (Module 22), so the bug can never silently return.

The CI matrix

```
# 1. Warnings as errors, two compilers
gcc   -std=c17 -O2 $WARNINGS -Werror -c src/*.c
clang -std=c17 -O2 $WARNINGS -Werror -c src/*.c

# 2. ASan + UBSan + LSan
cmake -B b-asan -DCMAKE_C_FLAGS="-O1 -g -fsanitize=address,undefined"
ASAN_OPTIONS=detect_leaks=1:detect_stack_use_after_return=1 ctest --test-dir b-asan

# 3. TSan (separate build)
cmake -B b-tsan -DCMAKE_C_FLAGS="-O1 -g -fsanitize=thread" && ctest --test-dir b-tsan

# 4. Valgrind (catches what sanitizers don't)
valgrind --error-exitcode=1 --leak-check=full --track-origins=yes ./tests

# 5. Static analysis
gcc -fanalyzer -c src/*.c
cppcheck --enable=all --error-exitcode=1 src/
clang-tidy src/*.c -p build/ --warnings-as-errors='*'

# 6. Fuzzing (Module 22)
```

Every one of these jobs tends to find real bugs the first time it is applied to an existing codebase.

Module 21 Rules

#	Rule
1	Build debug with `-Og -g3 -fno-omit-frame-pointer`; keep the debug build around

#	Rule
2	Conditional breakpoints and breakpoint command lists — not stepping 10,000 times
3	Watchpoints answer “who changed this variable?”
4	`gdb -batch -ex ...` is scriptable and pasteable into bug reports
5	Enable core dumps in production; archive the binary and its split debug file
6	`backtrace_symbols_fd` (not `backtrace_symbols`) in a signal handler; build with `-rdynamic`
7	`valgrind --track-origins=yes` for uninitialized reads — ASan and UBSan miss them
8	massif for “why is memory growing?”; callgrind for deterministic profiles
9	Measured on this workload: ASan ~14×, UBSan ~2×, TSan ~25×, Valgrind ~500×
10	ASan, TSan, and MSan are mutually exclusive — separate builds
11	`ASAN_OPTIONS=detect_stack_use_after_return=1` — off by default, catches real bugs
12	`strace` first when a program fails mysteriously
13	`rr` + `reverse-continue` for corruption bugs; it makes heisenbugs deterministic
14	`gcc -fanalyzer` is free — add it today
15	Run at least two static analysers; they find different things
16	`git bisect run` before you open a debugger

Practice

1. **GDB fluency drill.** Take the `buggy.c` from this module. Without stepping once: (a) break on the 4th call to `make`, (b) print the whole `rs` array, (c) watch `rs[2]->score` and find every write, (d) call `average(rs, 3)` from the prompt, and (e) change a score with `set var` and continue.

2. **Breakpoint tracing.** Write a `.gdb` script that traces every call to a function, with its arguments and return value, without modifying the source. Compare the output with what you would get from adding `printf`s — and compare the turnaround time.

3. **Pretty-printer.** Write a GDB Python pretty-printer for a linked list or hash table you have built. Make `print mylist` show the elements, not the raw pointers.

4. **Core-dump round trip.** Enable core dumps, crash a program five frames deep, and extract from the core: the full backtrace, all locals in each frame, the faulting instruction, and the register state. Then strip the binary, split out the debug info (Module 19), and prove that the workflow still works.

5. **Sanitizer vs Valgrind matrix.** Take the 12-bug program from Module 16, §16.4, and add Valgrind and MSan columns to the table. Record what each tool catches and misses, and how long each takes.

6. **`--track-origins` hunt.** Write a program in which an uninitialized value propagates through three function calls before being used. Confirm that ASan and UBSan miss it, and that Valgrind names the exact origin.

7. **massif investigation.** Write a program with a slow memory leak at one of five allocation sites. Use massif to identify which one, without reading the source.

8. **`strace` diagnosis.** Write a program that fails for a non-obvious reason (permission denied on a configuration file, a missing library, a full disk via `/dev/full`). Have a colleague run it and diagnose it using only `strace`.

9. **Static-analysis sweep.** Run `gcc -fanalyzer`, cppcheck, and clang-tidy on a real project. Count the unique findings per tool and the overlap. Fix everything actionable, then add all three to CI with `--error-exitcode`.

10. **`rr`, if you can install it.** Write a program that corrupts a structure field somewhere in a 1,000-iteration loop. Find the corrupting write with `rr`: set a watchpoint on the field after the corruption, then `reverse-continue`. Time yourself against doing the same with plain GDB.

Module 21 in Summary

Build a dedicated debug configuration (`-Og -g3 -fno-omit-frame-pointer`) and use GDB properly: conditional breakpoints instead of repeated stepping, breakpoint command lists for tracing without rebuilding, hardware watchpoints to find who wrote a bad value, batch mode for reproducible sessions, and Python pretty-printers for your own data structures. Enable core dumps in production and archive each release's binary and debug file, so that a dead process can be debugged after the fact; where cores are unavailable, print a backtrace from a signal handler with `backtrace_symbols_fd`.

Valgrind needs no rebuild and, with `--track-origins`, is the practical tool for uninitialized reads, while massif and callgrind profile memory and instructions. Sanitizers are far faster and catch stack and global overflows too; run ASan+UBSan and TSan as separate builds, with `detect_stack_use_after_return` enabled, and Valgrind in its own job. `strace` explains mysterious failures through their system calls, `perf` and flame graphs show where time goes, bpftrace traces production systems safely, and `rr` records an execution so you can run it backwards to the moment of corruption.

Static analysers — `-fanalyzer`, cppcheck, clang-tidy, and others — find bugs on paths no test executes, and formal tools can prove small critical functions correct. Put it all together as a methodology — reproduce, sanitize, analyse, read the report, bisect, then debug, then write a regression test — and as a CI matrix that runs every tool on every change.

Next: Module 22 — Testing & Quality, covering unit-test frameworks for C, test doubles and link-time seams, coverage measurement, and fuzzing — which finds more real bugs in C than every other technique combined.

Module 22 — Testing & Quality

In this module:

- Unit-test frameworks: Unity, CMocka, Check, Criterion, greatest, µnit
- Test doubles and mocks in C (link-time seams, function pointers)
- Code coverage: gcov/lcov, branch coverage
- Fuzzing: libFuzzer, AFL++, honggfuzz — huge in real C work
- Property-based and differential testing
- Regression suites, golden tests
- CI/CD: GitHub Actions, GitLab CI, building on multiple compilers and platforms

All output in this module is real, from GCC 13.3, gcov, and gcovr.

Module 21 was about finding bugs you know exist. This module is about finding the ones you don't — and making sure bugs you have fixed stay fixed. C gives testing an extra dimension: a test can pass while the code corrupts memory, because the corruption happens not to change the result. So in C, testing and the sanitizers of Modules 16 and 21 are inseparable, and one technique — fuzzing (§22.4) — finds more real bugs than every other technique combined. If you take one thing from this module, take that.

22.1 Unit-Test Frameworks

What a framework actually is

Here is a complete xUnit-style framework in about 40 lines. Understanding it demystifies every framework you will meet:

```
static int mt_run = 0, mt_fail = 0, mt_asserts = 0;

#define TEST(name) static void name(void)

#define RUN_TEST(fn) do {                                          \
        mt_run++; int before = mt_fail; fn();                      \
        printf(" %s %s\n", (mt_fail==before) ? "PASS" : "FAIL", #fn);   \
    } while (0)

#define CHECK_EQ_INT(a,b) do { mt_asserts++;                       \
        long a_=(long)(a), b_=(long)(b);                           \
        if (a_!=b_) { mt_fail++;                                   \
            printf("    %s:%d: %s == %s (%ld != %ld)\n",           \
                   __FILE__,__LINE__,#a,#b,a_,b_); return; } } while (0)

#define TEST_REPORT() (printf("\n%d tests, %d assertions, %d failures\n", \
                              mt_run, mt_asserts, mt_fail), mt_fail ? 1 : 0)
```

Every technique from Module 12 is in there: `do { } while (0)` for statement macros, `#x` stringification to print the source text of each expression, `__FILE__` and `__LINE__` to report the location, and hygienic local names (`a_`) so that each argument is evaluated exactly once. A test is just a `static void` function; a failed check prints its details and `return`s from the test; and `TEST_REPORT` evaluates to the process exit status.

```
running slist tests:
  PASS test_new_is_empty
  PASS test_push_pop
  PASS test_growth_preserves_data
  PASS test_null_safety
  PASS test_out_of_range

5 tests, 2025 assertions, 0 failures
exit=0
```

And a failure:

```
    t_fail.c:5: slist_len(l) == 3 (2 != 3)
  FAIL deliberately_broken
    t_fail.c:9: "hello" == "world" ("hello" != "world")
  FAIL str_and_float

2 tests, 2 assertions, 2 failures
exit=1   ← non-zero exit is what makes CI fail the build
```

Note that the failure messages print both the *expressions* and their *values*. That is the difference between a useful framework and a bare `assert(x == y)`, which tells you only that something was false.

The real frameworks

Framework	Style	Notes
Unity	3 files, no dependencies	The embedded standard. Runs on bare metal. Pairs with CMock and Ceedling.
greatest	Single header, ~1000 lines	Zero dependencies, easy to drop in. A great default.
µnit (munit)	Single file	Parameterised tests, fixtures, a seeded RNG for reproducibility
utest.h	Single header	Very minimal; auto-registers tests
CMocka	Library	The best mocking support — `will_return`/`expect_value` — plus its own leak checker
Check	Library	Runs each test in a forked process, so a segfault fails one test instead of the whole run
Criterion	Library	Auto-registration, parallel execution, theories, nice output
Unity/CMocka + CTest	—	CMake's `add_test` (Module 20) wraps any of them

Auto-registration means you do not need to list every test in a `main` function by hand; the framework discovers tests declared with its macros (typically using linker sections or constructor attributes).

How to choose:

- Embedded, or no OS → **Unity**
- Need mocking → **CMocka**
- Crashes must not abort the suite → **Check** (process isolation)
- Just want something now → **greatest** or **utest.h** (single header: copy and go)
- A modern developer experience → **Criterion**

Always run your tests under sanitizers:

```
gcc -std=c17 -Wall -Wextra -g -fsanitize=address,undefined slist.c test_slist.c -o t && ./t
```

Half of what tests catch in C is memory corruption, and the sanitizers catch it at the point of the bug, rather than three functions later — or never.

What to test

The suite above covers the categories that matter:

Test	Category
`test_new_is_empty`	Initial state / boundary
`test_push_pop`	Happy path
`test_growth_preserves_data`	The `realloc` path (1,000 elements forces many growths)
`test_null_safety`	`NULL` arguments to every public function
`test_out_of_range`	Index 0, index *n*, index `(size_t)-1`

In C specifically, always test: `NULL` arguments, zero-length inputs, the maximum size, allocation failure, and every error return documented in the header (Module 15). The `(size_t)-1` index is worth singling out: it is what a negative `int` becomes when passed to a `size_t` parameter (Module 2), and it is exactly the value an off-by-one or underflow bug produces.

22.2 Test Doubles and Mocking

A *test double* stands in for a real dependency — a network call, a clock, the file system, an allocator — so that a test can control its behaviour and observe how it was used. C has no run-time polymorphism, so mocking means either replacing a symbol or injecting a function pointer. There are three techniques, in increasing order of preference.

Technique 1 — the linker's `--wrap`

```
gcc app.c net.c test.c -Wl,--wrap=net_send -o t
```

With `--wrap=net_send`, the linker redirects every reference to `net_send` to `__wrap_net_send` instead; the original remains reachable as `__real_net_send`.

```
static int mock_rc = 0, mock_calls = 0;
static char mock_host[64]; static size_t mock_len;

int __wrap_net_send(const char *host, const void *buf, size_t n)
{
    mock_calls++; mock_len = n;
    snprintf(mock_host, sizeof mock_host, "%s", host);
    return mock_rc;
}
```

```
linker --wrap mock:
  called=1 host=example.com len=5 -> success path OK
  injected failure -> upload returned -2 as designed
  null host rejected without touching the network
```

The mock records how it was called, and returns whatever the test sets `mock_rc` to — which is how the test injects a failure. This technique requires no source changes. But it is GNU-linker specific (it also works with lld and gold, but not on macOS or with MSVC), it cannot intercept calls within the same translation unit (the compiler resolves those directly, before the linker is involved), and it applies to the whole binary, not per test.

Technique 2 — weak symbols

```
__attribute__((weak))
int net_send(const char *host, const void *buf, size_t n) { /* real */ }
```

A *weak* definition is used only if no *strong* (ordinary) definition exists. A strong definition in the test file therefore overrides it:

```
int net_send(const char *h, const void *b, size_t n) { calls++; return 0; }
```

```
strong symbol won: mock called 1 time(s)
(without the test file, the weak real implementation is used)
[REAL weak net_send to prod.example, 1 bytes]
```

This is simpler than `--wrap`, and it is the standard mechanism for overridable defaults in libraries — hooks in the C runtime, stub functions in embedded hardware-abstraction layers, and replaceable `malloc` implementations. Its limitations are otherwise the same, and the production code must be marked weak specifically to allow it.

Technique 3 — dependency injection ✓ preferred

```
typedef int (*SendFn)(const char *host, const void *buf, size_t n, void *ctx);
typedef struct { SendFn send; void *ctx; } Transport;

static int upload2(const Transport *t, const char *host, const char *msg)
{
    if (!t || !t->send || !host || !msg) return -1;
    return t->send(host, msg, strlen(msg), t->ctx) == 0 ? 0 : -2;
}
```

Production code passes the real implementation; tests pass a *spy* that records its calls:

```
typedef struct { int calls; int rc; char last_host[64]; size_t last_len; } Spy;
static int spy_send(const char *h, const void *b, size_t n, void *c)
{ Spy *s = c; s->calls++; s->last_len = n; snprintf(s->last_host,64,"%s",h); return s->rc; }
```

```
[REAL send to prod.example, 7 bytes]
spy: calls=1 host=test.local len=5
spy injected failure -> upload2 returned -2
no linker tricks, no rebuild, works with any test framework
```

This is Module 6's callback pattern, with a `void *` context, applied to testing. It is portable, it works per test, it works within a translation unit, it composes with any framework — and, critically, it makes the dependency *visible* in the function's signature, so readers can see that `upload2` talks to the network.

> Code that is hard to test is usually code with hidden dependencies. Injection isn't a testing trick; it's better design that happens to be testable.

CMocka's mocking API (`will_return`, `expect_value`, `check_expected`) builds on the `--wrap` approach with a nicer interface, and it automatically verifies that the expected calls actually happened.

22.3 Code Coverage

Coverage measures which parts of the code your tests actually executed.

```
gcc -O0 -g --coverage slist.c test_slist.c -o t_cov
./t_cov
gcov -b t_cov-slist.gcda
```

`--coverage` instruments the program to count how many times each line and branch runs; running it writes the counts to `.gcda` files; and `gcov` turns them into a report.

The measurement that matters

```
File 'slist.c'
Lines executed:100.00% of 20
Branches executed:100.00% of 24
Taken at least once:87.50% of 24
```

100% line coverage. 87.5% branch coverage. These are not the same thing, and the difference is where bugs hide. A line like `if (!p) return -1;` counts as "executed" as soon as the condition is evaluated — but unless `p` has *also* been null in some test, the `return -1` path has never run.

```
$ gcovr --branches
File     Branches  Taken  Cover  Missing
slist.c        24     21    87%  15,23,28
```

Line 15 is:

```
           if (!p) return -1;
branch  0 taken 0% (fallthrough)   ← the realloc-failure path NEVER ran
```

> The untested branches are the error paths — exactly the ones that are hardest to reach and most likely to be wrong. Module 15's fault-injection technique exists precisely to reach them.

The annotated source

```
function slist_push called 1012 returned 100% blocks executed 91%
     1012:   10:int slist_push(SList *l, int v){
     1012:   11:    if (!l) return -1;
branch  0 taken 1% (fallthrough)
     1011:   12:    if (l->len == l->cap) {
branch  0 taken 1% (fallthrough)
       13:   13:        size_t nc = l->cap ? l->cap*2 : 4;
      13*:   15:        if (!p) return -1;
branch  0 taken 0% (fallthrough)   ← ← ←
```

The left column is the execution count of each line; the branch lines show how often each outcome was taken. A `*` marks a line with an unexecuted branch. (Note that line 13 ran only 13 times across 1,012 pushes in the whole suite: because capacity doubles, growth is rare — the amortised O(1) behaviour from Module 8, visible in the counts.)

Tools and interpretation

Tool	Purpose
`gcov -b -c`	Branch and call coverage, per file
lcov + genhtml	HTML reports with line highlighting
gcovr	A modern alternative; text, HTML, XML, JSON, and Cobertura output for CI
llvm-cov + llvm-profdata	Clang's equivalent (`-fprofile-instr-generate -fcoverage-mapping`)
Codecov / Coveralls	CI dashboards with per-pull-request coverage changes

Coverage types, from weakest to strongest: **statement → branch → condition → MC/DC → path**. *Condition coverage* requires each individual Boolean sub-expression (in `a && b`, both `a` and `b`) to take both values; *MC/DC* ("modified condition/decision coverage", required by DO-178C Level A for flight software) additionally requires showing that each condition independently affects the outcome; *path coverage* — every route through the code — is combinatorially infeasible for real programs.

How to think about coverage

- **Coverage is a floor, not a goal.** 100% coverage proves that every line ran, not that it is correct. A test suite that executes everything and asserts nothing scores 100%.
- **Uncovered code is definitely untested.** That is the actionable signal.
- **Target branch coverage, not line coverage.**
- **Chase the uncovered error paths with fault injection.**
- **Enforce a floor in CI** (`gcovr --fail-under-branch 80`) and ratchet it upward, never down.

- ⚠ **Never ship a `--coverage` build.** It is slow, and it writes `.gcda` files when it exits.

22.4 Fuzzing ⚠

This is the highest-value testing technique in C. *Fuzzing* feeds a program enormous numbers of automatically generated inputs, looking for any that make it crash or trigger a sanitizer report. Google's OSS-Fuzz service has found tens of thousands of bugs in critical infrastructure — OpenSSL, SQLite, curl, FFmpeg, the Linux kernel — most of which already had extensive hand-written test suites.

Why is it so effective? Hand-written tests check the inputs a programmer thought of. Fuzzers generate the inputs nobody thought of — truncated headers, lengths of 65,535, nested structures 10,000 levels deep — which are exactly the inputs an attacker will try.

The harness — one function, every fuzzer

```c
#include <stdint.h>
#include <stddef.h>

int LLVMFuzzerTestOneInput(const uint8_t *data, size_t size)
{
    uint8_t out[64];
    parse_tlv(data, size, out, sizeof out);
    return 0;
}
```

That is the entire integration: a function that takes a buffer of bytes and passes it to the code under test. libFuzzer, AFL++, honggfuzz, and OSS-Fuzz all use this exact signature, so one harness works with all of them.

Finding real bugs

Consider a deliberately buggy parser for a simple TLV ("type–length–value") format — one byte of type, a two-byte big-endian length, then that many bytes of value — with three bugs: no check that the input is as long as the length claims, no check that the output buffer is large enough, and a resulting heap out-of-bounds write.

```
=== BUGGY parser under a naive RANDOM fuzzer + ASan ===
  crashed: ERROR: AddressSanitizer: stack-buffer-overflow
  (first iteration)

=== FIXED parser: 2,000,000 random inputs ===
  survived 2000000 random inputs
  exit=0 (clean under ASan + UBSan)
```

The fix:

```c
if (len > n - 3) return -3;     /* input bound */
if (len > outcap) return -4;    /* output bound */
```

(The first check is written as `len > n - 3` rather than `len + 3 > n` to avoid overflow — Module 2 — and relies on an earlier check that `n >= 3`.)

Even a purely random fuzzer found the bug on its very first input. Real fuzzers are *coverage-guided*: they instrument the program, keep any input that reaches code no previous input reached, and mutate those inputs further. That makes them orders of magnitude more effective at reaching deep states hidden behind checksums, magic numbers, and nested structures, which random bytes almost never satisfy.

The real fuzzers

```
# libFuzzer — in-process, fastest, part of Clang
clang -fsanitize=fuzzer,address,undefined parser.c harness.c -o fz
./fz corpus/ -max_total_time=300 -jobs=8

# AFL++ — out-of-process, best mutation engine, huge feature set
afl-clang-fast harness.c parser.c -o fz   # or afl-clang-lto for better coverage
afl-fuzz -i seeds/ -o findings/ -- ./fz @@

# honggfuzz — good hardware-counter-based feedback
hfuzz-clang harness.c parser.c -o fz
honggfuzz -i seeds/ -- ./fz

# OSS-Fuzz — Google runs the same harness continuously, for free, on open-source projects
```

In-process means libFuzzer calls your harness function millions of times inside a single process — very fast, but a crash ends the run. AFL++ runs the target as a separate process (with a fork server to keep that cheap), which is more robust.

Getting it right

1. **Always fuzz with sanitizers on** — ASan and UBSan. Without them, the fuzzer only finds crashes; with them, it finds every memory bug and every instance of undefined behaviour it triggers (Module 16), even ones that would not crash.
2. **Seed the corpus with real, valid inputs.** A fuzzer starting from an empty input may take hours to guess a file's magic number; a single valid seed file gives it that instantly.
3. **Use a dictionary** (`-dict=`) containing your format's keywords, magic bytes, and field names.
4. **Minimise crashers.** `./fz -minimize_crash=1 crash-file` reduces, say, a 4 KB crashing input to the 6 bytes that matter, which makes the bug far easier to understand.
5. **Keep the corpus in Git, and replay it as a regression test on every commit.** This is the single cheapest way to keep the value of fuzzing.
6. **Make the harness deterministic** — no time, no randomness, no network — so that every crash can be reproduced.
7. **Fuzz the parsing boundary:** file formats, network protocols, configuration parsers, decompressors, image, font, and media decoders — anything reachable from untrusted input.
8. **Use structure-aware fuzzing** (libprotobuf-mutator, FuzzTest) when raw bytes rarely produce valid inputs — for example, for a program that only accepts well-formed SQL.

> If your C project parses anything an attacker can supply, and it isn't fuzzed, it has bugs. That's not rhetoric — it's the empirical finding of the last decade.

22.5 Property-Based and Differential Testing

Property-based testing

Instead of example-based assertions ("push 1, check that the length is 1"), assert *invariants* over many random sequences of operations:

```
for (int trial=0; trial<2000; trial++) {
    SList *l = slist_new();
    size_t n = rnd() % 50;
    for (size_t i=0;i<n;i++) { ref[i]=rand16(); slist_push(l,ref[i]); }

    CHECK(slist_len(l) == n);
    for (size_t i=n; i-- > 0; ) {               /* PROPERTY: LIFO ordering */
        int v; slist_pop(l,&v);
        CHECK(v == ref[i]);
    }
    CHECK(slist_len(l)==0 && slist_pop(l,&d)==-1);   /* PROPERTY: emptied == new */
}
```

```
property-based (2000 random sequences): PASS
```

Each trial pushes a random number of random values and then checks two properties that must hold for *every* input: values come back in last-in-first-out order, and an emptied list behaves exactly like a new one. (Note the `i-- > 0` reverse loop from Module 5.)

Good properties in C: **round-trips** (`decode(encode(x)) == x`), **invariants** (a sorted array stays sorted; `len <= cap`), **idempotence** (doing something twice equals doing it once), **commutativity**, and "**never crashes, never leaks**" — which the sanitizers verify for free on every random input.

Libraries for C: *theft*, RapidCheck (C++, but usable), or simply hand-rolled loops like the one above.

Differential testing

Compare your implementation against a trivially correct *reference model*:

```
typedef struct { int a[300000]; size_t n; } Model;   /* obviously correct */
static void m_push(Model *m,int v){ if(m->n<300000) m->a[m->n++]=v; }
static int  m_pop (Model *m,int *o){ if(!m->n) return -1; *o=m->a[--m->n]; return 0; }

for (int op=0; op<200000; op++) {
    if (rnd()%3 < 2) { slist_push(l,v); m_push(&m,v); }
    else { r1=slist_pop(l,&v1); r2=m_pop(&m,&v2);
           CHECK(r1==r2 && (r1!=0 || v1==v2)); }
    CHECK(slist_len(l) == m.n);
}
```

```
differential vs model (200k ops): PASS
```

The model is too simple to be fast or general — a fixed array — but simple enough to be obviously correct. Driving both with the same random operations and checking that they always agree tests the real implementation against a specification you can read at a glance.

A useful thing happened while this example was being written. The first version of the model capped the list at 4,096 elements, while the real list grows without limit:

```
FAIL: len diverged at op 11962
```

The test found a divergence — and the bug was in the reference model, not the implementation. That is differential testing working exactly as intended: it does not tell you which side is wrong, only that they disagree.

> ⚠ A second lesson from that run: the `FAIL` message did not print at all until `setvbuf(stdout, NULL, _IONBF, 0)` was added. LeakSanitizer's abort discarded the buffered `stdout` — exactly the Module 13, §13.6 bug. **Always run test binaries unbuffered.**

Real-world differential testing: compare an optimised implementation against a naive one, a new version against the old one, your parser against a reference parser, or two independent libraries against each other. Differential testing with Csmith — a generator of random, UB-free C programs — compiled by different compilers is how many compiler bugs have been found.

22.6 Regression Suites and Golden Tests

Regression tests

> **Every bug fix gets a test that fails before the fix and passes after.**

This is the single most valuable testing discipline. It prevents the same bug from returning, documents the exact shape of the bug, and builds a test suite anchored in reality rather than imagination — because every test in it corresponds to a failure that actually happened.

Name them after the issue: `test_issue_142_overflow_on_empty_input`.

Fuzzer crashers are free regression tests: commit each minimised crashing input to the corpus, and replay it on every build.

Golden / snapshot tests

Compare a program's output against a stored, reviewed expected output:

```
./mytool --format json input.txt > actual.json
diff -u expected/output.json actual.json || exit 1
```

```
test-golden:
    @for t in tests/cases/*.in; do \
        ./mytool < $$t > /tmp/out; \
        diff -u $${t%.in}.expected /tmp/out || exit 1; \
    done
    @echo "all golden tests pass"
```

```
update-golden:
    @for t in tests/cases/*.in; do ./mytool < $$t > $${t%.in}.expected; done
```

For each input file, the program's output is compared with the matching `.expected` file; `update-golden` regenerates the expected files after a deliberate change. (`$$` passes a single `$` through to the shell, Module 20.)

Golden tests are excellent for compilers, formatters, serialisers, report generators, and CLI tools — anything with complex output, where hand-writing assertions is impractical.

> ⚠ **The failure mode:** blindly running `make update-golden` whenever a test fails. That converts every regression into an "accepted" change. Review the diff, every time.

Related tools: `expect`/`pexpect` for interactive programs, and CLI testing tools such as bats, cram, and lit.

22.7 CI/CD

The matrix that catches everything

```yaml
# .github/workflows/ci.yml
name: CI
on: [push, pull_request]

jobs:
  build-test:
    strategy:
      fail-fast: false
      matrix:
        os: [ubuntu-latest, macos-latest, windows-latest]
        compiler: [gcc, clang]
        std: [c17, c2x]
    runs-on: ${{ matrix.os }}
    steps:
      - uses: actions/checkout@v4
      - run: cmake -B build -DCMAKE_C_COMPILER=${{ matrix.compiler }} -DCMAKE_C_STANDARD=$
{{ matrix.std }} -DCMAKE_BUILD_TYPE=Release
      - run: cmake --build build
      - run: ctest --test-dir build --output-on-failure

  sanitizers:
    strategy:
      matrix:
        san: ["address,undefined", thread]
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: cmake -B b -DCMAKE_C_FLAGS="-O1 -g -fsanitize=${{ matrix.san }}"
      - run: cmake --build b
      - env:
          ASAN_OPTIONS: detect_leaks=1:detect_stack_use_after_return=1:abort_on_error=1
          UBSAN_OPTIONS: print_stacktrace=1:halt_on_error=1
        run: ctest --test-dir b --output-on-failure

  valgrind:
    runs-on: ubuntu-latest
    steps: [...]
      - run: valgrind --error-exitcode=1 --leak-check=full --track-origins=yes ./b/tests

  static-analysis:
    steps:
      - run: gcc -fanalyzer -c src/*.c
```

```
      - run: cppcheck --enable=all --error-exitcode=1 src/
      - run: clang-tidy src/*.c -p build/ --warnings-as-errors='*'

  coverage:
    steps:
      - run: cmake -B bc -DCMAKE_C_FLAGS="--coverage -O0 -g" && cmake --build bc
      - run: ctest --test-dir bc
      - run: gcovr --fail-under-branch 80 --xml-pretty -o coverage.xml
      - uses: codecov/codecov-action@v4

  fuzz:
    steps:
      - run: clang -fsanitize=fuzzer,address,undefined fuzz/*.c src/*.c -o fz
      - run: ./fz corpus/ -runs=100000   # regression: replay the committed corpus
```

This is a sketch of the full matrix rather than a complete, runnable file: the later jobs abbreviate their `runs-on` lines and checkout steps (shown in full in the first two jobs), and a real `windows-latest` job needs a MinGW or MSVC toolchain set up for the "gcc" entry. One detail matters: the sanitizer list is written as `["address,undefined", thread]` — without the quotes, YAML would split `address,undefined` into two separate matrix entries, and you would get an ASan-only job and a UBSan-only job instead of one combined job.

The principles

Principle	Why
Multiple compilers	GCC and Clang catch different bugs (Module 19)
Multiple platforms	Windows finds `long`-size and text-mode bugs; macOS finds different libc behaviour
Multiple standards	`-std=c17` and `-std=c2x`
`fail-fast: false`	See all the failures, not just the first
Sanitizers in separate jobs	ASan, TSan, and MSan are mutually exclusive (Module 21)
Fast feedback first	Compile and test in two minutes; fuzzing and Valgrind can take longer
Cache dependencies	`actions/cache` on the build directory
Coverage ratchet	A `--fail-under-branch` threshold that only ever goes up
Nightly deep runs	Long fuzzing sessions, full Valgrind runs, cross-compilation matrices

GitLab CI, Jenkins, Buildkite, and CircleCI all express the same matrix in their own syntax; the concepts transfer directly.

Also worth adding: `clang-format --dry-run --Werror` (formatting), a licence check, an ABI check with `abidiff` (Module 20) if you ship a library, and an alarm for build-time regressions.

Module 22 Rules

#	Rule
1	A test framework is ~40 lines — pick one and start; don't shop
2	Assertions must print both the expressions and the values
3	Always run tests under ASan+UBSan; run test binaries unbuffered
4	Test `NULL` args, zero length, maximum size, allocation failure, and every documented error return
5	Prefer dependency injection over `--wrap` and weak symbols
6	Hard-to-test code has hidden dependencies — fix the design, not the test
7	Branch coverage, not line coverage — 100% of lines was only 87.5% of branches here
8	The uncovered branches are the error paths; reach them with fault injection
9	Coverage is a floor, not a goal
10	Fuzz anything that parses untrusted input — always with sanitizers on
11	Seed the corpus, use a dictionary, minimise crashers, commit the corpus
12	Properties over examples; differential tests against a simple model
13	Every bug fix gets a regression test named after the issue
14	Review golden-test diffs; never blindly regenerate
15	CI matrix: 2+ compilers × 3 platforms × sanitizers × Valgrind × static analysis × fuzzing

Practice

1. **Build the framework.** Write `minitest.h` from §22.1 with `CHECK`, `CHECK_EQ_INT`, `CHECK_EQ_STR`, `CHECK_NEAR`, and a `SKIP` macro. Add setup/teardown support. Then write a 15-test suite for a data structure you built in an earlier module.

2. **Framework comparison.** Port that same suite to greatest and to Unity. Compare the lines of boilerplate, the quality of the failure output, and how each handles a segfault in the middle of the suite.

3. **All three mocking techniques.** Take a function that calls `time()`, `malloc()`, and a network function. Mock each with `--wrap`, with weak symbols, and with injection. Then measure which lets you write the most tests with the least setup.

4. **Fault injection.** Add an injectable allocator (`static void *(*alloc_fn)(size_t) = malloc;`) to a library. Write a test that makes the Nth allocation fail for $N = 1 \ldots 20$, and asserts that LSan reports zero leaks each time. Measure the branch-coverage improvement.

5. **Coverage hunt.** Build any project with `--coverage`, run the suite, and generate a branch-coverage report. Find the 5 least-covered functions. Write tests for their uncovered branches. Report the before-and-after numbers.

6. **Fuzz a real parser.** Write a parser for something (INI, CSV with quoting, a binary header, base64). Write the `LLVMFuzzerTestOneInput` harness. Run it under ASan+UBSan with a random fuzzer for 10 million iterations. Fix everything it finds.

7. **Fuzz with a real fuzzer.** Install Clang and libFuzzer (or AFL++). Fuzz the same parser for 10 minutes with a seed corpus and a dictionary. Compare the number of unique crashes and the coverage reached with those of your random fuzzer.

8. **Property tests.** For a hash table you write, test: an encode/decode round-trip; "inserting N distinct keys gives a count of N"; "lookup after insert always finds it"; "delete then lookup always misses"; and "resizing preserves all entries." Run 10,000 random operation sequences under ASan.

9. **Differential test.** Implement a sorted-array set and a linked-list set with the same API. Drive both with the same random operation sequence and assert identical observable behaviour at every step. Then deliberately introduce a bug in one, and confirm that the test catches it.

10. **Full CI pipeline.** Set up GitHub Actions for a project with the compiler/platform matrix, sanitizer jobs, Valgrind, three static analysers, coverage with a ratcheting threshold, and a fuzz-corpus regression run. Make one job fail on purpose, and verify that you get a useful report.

Module 22 in Summary

A C test framework is little more than a handful of macros that print the failing expression, its values, and its location, and return a non-zero exit status; choose Unity for embedded work, CMocka for mocking, Check for process isolation, or a single-header framework to start immediately — and always run tests under sanitizers, unbuffered. Test the C-specific edges: `NULL`, zero length, maximum size, allocation failure, and every documented error.

Replace dependencies with the linker's `--wrap`, weak symbols, or — best — dependency injection through function pointers, which also improves the design. Measure branch coverage rather than line coverage: here, 100% of lines hid a 12.5% gap of untested branches, all of them error paths reachable only through fault injection. Treat coverage as a floor that CI enforces and ratchets upward.

Fuzzing is the most productive technique of all: a one-function harness, run under ASan and UBSan by a coverage-guided fuzzer with a seed corpus and dictionary, finds the inputs nobody thought of — and the committed corpus becomes a permanent regression suite. Property-based tests check invariants over random inputs, differential tests compare against a simple model, every fixed bug earns a named regression test, and golden tests guard complex output provided their diffs are reviewed. A CI matrix across compilers, platforms, standards, sanitizers, Valgrind, static analysis, coverage, and fuzzing runs all of it on every change.

Next: Module 23 — Secure Coding, which pulls the vulnerability classes from Modules 8, 10, and 16 into a systematic security framework: the CWE Top 25 as they appear in C, CERT C and MISRA, exploit mechanics (so you understand why the mitigations exist), hardening flags, and secure API design.

Module 23 — Secure Coding

In this module:

- Buffer overflows (stack and heap), off-by-one errors, abuse of the `strcpy` family
- Format-string vulnerabilities
- Integer overflow and underflow, signed/unsigned confusion, truncation
- Use-after-free, double free, type confusion
- TOCTOU races, unsafe temporary files, `system()`/command injection
- Input validation and canonicalisation
- CERT C, MISRA C:2012/2023, CWE Top 25
- Hardening: `-fstack-protector-strong`, `_FORTIFY_SOURCE=3`, PIE, RELRO, CFI
- Constant-time code for cryptography, and avoiding compiler-removed `memset`
- Threat modelling and secure API design (bounds-carrying interfaces)

All output in this module is real, from GCC 13.3 and glibc 2.39.

This module explains how vulnerabilities work at the depth needed to write code that resists them, and demonstrates the defences working.

The framing that matters: **in C, most security bugs are simply the memory-safety and integer bugs from Modules 2, 8, 9, and 10 — reachable from input an attacker controls.** Security is not a separate discipline bolted on at the end. It is correctness on the paths an adversary chooses. An ordinary user never types a 10,000-character name or sends a packet whose length field says 65,535 when only 10 bytes follow; an attacker does exactly that, deliberately and repeatedly, until something breaks in a useful way.

23.1 Buffer Overflows

Why they matter — the stack frame

```
STACK FRAME LAYOUT:
  &buf[0]           = 0x7fffe4e0f280
  saved rbp         = 0x7fffe4e0f2a0 (+32 bytes from buf)
  return addr       = 0x7fffe4e0f2a8 (+40 bytes from buf)   <-- the target
  return addr value = 0x55e46fdf828b
```

A 16-byte local buffer sits 40 bytes below the function's return address — the layout from Module 6, §6.7, measured. Writing past the end of the buffer reaches the saved frame pointer, and then the return address. Controlling the return address controls where execution resumes when the function returns. That is the entire mechanism, and it is why *every* buffer overflow is treated as potentially exploitable. (Modern mitigations — non-executable stacks, ASLR, canaries — make exploitation harder, and attackers respond with techniques such as *return-oriented programming*, which chains together fragments of existing code instead of injecting new code. The overflow remains the entry point.)

Heap overflows work differently, but are equally serious: they corrupt the allocator's inline chunk metadata (Module 10, §10.5), so that a later `malloc` can be steered into returning an address the attacker chose — or they overwrite a neighbouring object's data, such as a function pointer.

The `strcpy` family — CWE-120/121/122

Module 8 banned these functions. Here is the security framing:

Function	Why it's unsafe
`gets`	No size parameter at all. Removed in C11. Used by the Morris Worm (1988).
`strcpy`, `strcat`	Write until they find a NUL in the *source*
`sprintf`, `vsprintf`	Unbounded formatted output
`scanf("%s")`	Unbounded read (Module 13)
`strncpy`	Doesn't reliably terminate; not a "safe `strcpy`"
`strncat`	Its `n` is the number of bytes appended, not the buffer size
`alloca`, VLAs	Unchecked stack allocation from a length (Modules 7 and 10)

(The CWE numbers are identifiers from MITRE's *Common Weakness Enumeration*, the standard catalogue of software weakness types — §23.7.)

Off-by-one — CWE-193

The most common overflow, and often exploitable:

```
char buf[N];
for (i = 0; i <= N; i++) buf[i] = ...;         /* writes buf[N] */
buf[strlen(s)] = '\0';                         /* if s is exactly N chars, writes buf[N] */

strncat(dst, src, sizeof dst - strlen(dst));   /* forgot the -1 for the NUL */
memcpy(dst, src, sizeof dst);                  /* if dst must also hold a NUL */
```

A single byte written past the end of a buffer — often a zero byte, from a string terminator — into the saved frame pointer or a heap chunk header has been the basis of numerous real exploits, known as "off-by-one-byte" or "poison NUL byte" attacks. One byte is enough.

The defences, measured

The same `strcpy` of a 64-byte string into a 16-byte buffer, built four ways:

Build	Result
`-fno-stack-protector -U_FORTIFY_SOURCE`	`rc=139 Segmentation fault` — silent corruption, then a crash
`-fstack-protector-strong`	`rc=134 *** stack smashing detected ***: terminated`

Build	Result
`-D_FORTIFY_SOURCE=3`	`rc=134 *** buffer overflow detected ***: terminated`
`-fsanitize=address`	`ERROR: AddressSanitizer: stack-buffer-overflow` (with the exact line)

All three defences turn “corrupt memory and continue” into “abort immediately.” That converts a potential code-execution vulnerability into a denial of service — a very large improvement. (Exit status 134 is 128 + 6, `SIGABRT`; 139 is 128 + 11, `SIGSEGV` — Module 1’s exit-code convention.)

Note *when* each fires. `_FORTIFY_SOURCE` replaces `strcpy` with a checked version that knows the destination’s size, so it catches the overflow *at the call*, before any memory is corrupted. The stack canary only detects the corruption *at function return*, after the damage has been done — and anything the function does with its corrupted locals in the meantime still happens. Use both.

23.2 Format-String Vulnerabilities — CWE-134

```
printf(user_input);          /* VULNERABLE */
printf("%s", user_input);    /* safe */
```

With the input `%p %p %p`:

```
VULNERABLE - printf(user_input):
  0x7ffc11f8d9a0 (nil) (nil)   ← leaked stack/register contents

SAFE - printf("%s", user_input):
  %p %p %p                     ← printed literally
```

`printf` trusts its format string (Module 13). Given `%p`, `%s`, or `%x` with no corresponding arguments, it fetches “arguments” from wherever they would have been — the argument registers and the stack — leaking their contents. That information leak can defeat ASLR (by revealing addresses) and can dump secrets or stack canaries.

And `%n` *writes*:

```
hello world
  %n stored 5 into the variable
```

`%n` stores the number of characters printed so far through a pointer argument. With a format string they control, an attacker chooses both the value written (by using field widths to print a chosen number of characters) and the target address (by selecting which stack slot is used as the pointer, with positional arguments such as `%7$n`). The result is an arbitrary memory-write primitive.

Defences

```
$ gcc -Wformat=2 -Wformat-security -c fmt.c
fmt.c:7:5: warning: format not a string literal and no format arguments [-Wformat-security]

$ gcc -O2 -D_FORTIFY_SOURCE=3 fmtdyn.c && ./fmtdyn
*** %n in writable segment detected ***
Aborted
```

glibc's `_FORTIFY_SOURCE` refuses to process `%n` when the format string lives in writable memory — which is exactly the attacker-controlled case, since string literals live in read-only `.rodata`.

Rules:

1. **Never pass non-literal data as a format string.** If you genuinely need a dynamic format, `-Wformat-nonliteral` will flag it, and you must justify it explicitly.
2. **Use `-Wformat=2 -Werror=format-security` in every build.**
3. **Put `__attribute__((format(printf, m, n)))` on every wrapper** (Module 6).
4. **This applies to `syslog`, `err`/`warn`, and every logging function too** — any function that takes a format string.

23.3 Integer Bugs — CWE-190/191/197/195

The classic chain: overflow → undersized allocation → heap overflow

```c
void *bad_alloc_array(size_t n, size_t elemsize)
{ return malloc(n * elemsize); }   /* n*elemsize can WRAP */
```

```
n = 4611686018427387905, elemsize = 4
  n * elemsize (wrapped) = 4 bytes   <-- TINY
  bad_alloc_array -> SUCCEEDED (a 4-byte buffer for 4611686018427387905 elements!)
  good_alloc_array -> NULL (rejected)
  calloc(n, es)    -> NULL (rejected)
```

`n` here is $2^{62} + 1$. Multiplied by 4, it is $2^{64} + 4$, which wraps around (unsigned arithmetic is modulo 2^{64}, Module 2) to just 4. The allocation succeeds with 4 bytes, and the subsequent loop writing `n` elements is an unbounded heap overflow. This exact pattern is behind a large share of serious CVEs in C code — typically, an attacker supplies the element count in a file or network message.

Fixes:

```c
if (elemsize && n > SIZE_MAX / elemsize) return NULL;   /* explicit */
p = calloc(n, elemsize);                                /* calloc checks internally */
if (ckd_mul(&total, n, elemsize)) return NULL;          /* C23 <stdckdint.h> */
if (__builtin_mul_overflow(n, elemsize, &total)) return NULL;
```

Signed/unsigned confusion — the length-check bypass

```c
static int copy_bad(const char *src, int len)   /* len from untrusted input */
{
    if (len > (int)sizeof dest) return -1;      /* looks like a bounds check... */
```

```
    memcpy(dest, src, (size_t)len);             /* ...but len can be NEGATIVE */
}
```

```
dest is 64 bytes. Attacker supplies len = -1.
copy_bad: 'len > (int)sizeof dest' -> -1 > 64 is FALSE -> check PASSES
          then memcpy(..., (size_t)-1) = 18446744073709551615 bytes
copy_good: rejects len < 0 first -> returns -1
```

The check compares two signed `int`s, so −1 is correctly judged smaller than 64 — and the check passes. Then the length is converted to `size_t`, and −1 becomes the largest possible size.

And the mirror-image trap from Module 2:

```
int len = -1; size_t cap = 64;
actual result of (len < cap): FALSE   ← -1 became 18446744073709551615
-> a 'len < capacity' check silently accepts every negative value
```

Here the comparison mixes signed and unsigned, so the usual arithmetic conversions turn −1 into a huge unsigned value *before* comparing. One version of the check is fooled in one direction, the other in the other.

> **Rule:** check for negative first, then convert to `size_t`.
>
> ```
> if (len < 0 || (size_t)len > cap) return -1;
> ```

Better still, use `size_t` for lengths throughout, so that no negative value can ever arise.

Truncation

```
size_t 0x100000001 -> unsigned short 1, int 1
```

A length of just over 4 GB, truncated into a 16-bit or 32-bit field, becomes 1. Allocate 1 byte, copy 4 GB. Watch every place a length crosses a type boundary: `size_t` → `int`, `off_t` → `long`, a network `uint32_t` → `int`.

Defences: `-Wconversion -Wsign-conversion`; UBSan (Clang's `-fsanitize=integer` even catches *defined* unsigned wraparound, which is often unintended); and `<stdckdint.h>`.

23.4 Lifetime Bugs — CWE-416/415/843

From Module 10's taxonomy, with the security framing:

Bug	Why it's exploitable
Use-after-free (CWE-416)	The allocator reuses the block. An attacker who can trigger a well-timed allocation of the same size controls its contents — including any function pointers in the freed object, which the program then calls.

Bug	Why it's exploitable
Double free (CWE-415)	Corrupts the free list, letting a later `malloc` return an attacker-chosen address
Type confusion (CWE-843)	Reading a union member that wasn't written, or casting between incompatible structure types, reinterprets attacker data as a pointer. Module 11's tagged-union discipline prevents this.
Dangling after `realloc`	Module 9, §9.8 — every alias becomes a use-after-free
Uninitialized memory (CWE-457/908)	Leaks stack or heap contents. The related Heartbleed bug (CVE-2014-0160) was an over-read that returned up to 64 KB of adjacent server memory — keys, passwords, session data — per request.

Use-after-free is currently one of the most exploited bug classes in large C and C++ programs, such as browsers and kernels, precisely because heap layouts can be groomed: the attacker allocates and frees objects in a pattern that makes their own data land exactly where the dangling pointer points.

Defences: ASan + LSan in CI; `free(p); p = NULL;`; one owner per allocation, with a documented lifetime (Module 10, §10.6); opaque handles with a magic field (Module 11); and, where available, hardened allocators (glibc's tcache double-free detection, `MALLOC_CHECK_`, Scudo, hardened_malloc).

23.5 TOCTOU, Temporary Files, Command Injection

TOCTOU — CWE-367

```c
/* BAD */
if (access(path, W_OK) == 0) {   /* CHECK */
    fd = open(path, O_WRONLY);   /* USE — attacker swaps the file in between */
}
```

Time-of-check to time-of-use (TOCTOU) bugs (Module 13) arise whenever a program checks a property of a *name* and then acts on that name. In the window between the two system calls, an attacker can replace `path` with a symbolic link to something else — `/etc/passwd`, say. `access()` is especially dangerous in *setuid* programs (which run with another user's privileges): it checks permissions using the *real* user ID, while `open()` uses the *effective* one — which is exactly the gap the attacker wants.

```c
/* GOOD — operate on the object, not the name */
fd = open(path, O_WRONLY | O_NOFOLLOW | O_CLOEXEC);
if (fd < 0) return -1;
struct stat st;
fstat(fd, &st);   /* same object, no race */
```

Open first; the file descriptor then refers to one specific file, whatever happens to the name afterwards. Check that object with `fstat`. Also use `openat`/`fstatat` with a directory file descriptor, `O_NOFOLLOW` to refuse symbolic links, and never `stat` a name and then `open` it.

Unsafe temporary files — CWE-377

```
BAD : tmpnam()/mktemp()/tempnam() return a NAME -> race window
GOOD: mkstemp() creates AND opens atomically with mode 0600
  mkstemp -> /tmp/securesBr1rN mode=0600 (0600 = owner only)
```

The old functions produce an unused *name*; by the time the program creates the file, an attacker may have created a file or symbolic link with that name in the shared `/tmp` directory. `mkstemp` creates and opens the file in one atomic step, failing if the name exists.

```c
char tmpl[] = "/tmp/secureXXXXXX";
mode_t old = umask(0077);   /* restrict permissions BEFORE creating */
int fd = mkstemp(tmpl);
umask(old);
unlink(tmpl);               /* unlink immediately; the fd stays valid */
```

The `XXXXXX` is replaced with random characters. Unlinking the file immediately removes its name while the open descriptor keeps the file itself alive, so nothing else can open it, and it disappears automatically when the program closes it or exits.

Also: `O_CREAT | O_EXCL` for a known filename, `mkdtemp` for directories, and `O_TMPFILE` (Linux) for a file that never has a name at all.

Command injection — CWE-78

```c
/* BAD */
snprintf(cmd, sizeof cmd, "ls %s", user);
system(cmd);   /* user = "; rm -rf ~" runs BOTH commands */
```

`system()` and `popen()` pass their command to `/bin/sh`. Every shell metacharacter — `;` `|` `&` `$` `` ` `` `\` `(` `)` `<` `>` `*` `?` and newline — is live. Escaping them correctly is a losing game.

```c
/* GOOD — argument vector, no shell */
char *args[] = { "/bin/ls", (char*)user_path, NULL };
execve("/bin/ls", args, envp);
```

```
execve with a malicious 'filename':
; rm -rf /tmp/nothing; echo PWNED
  ^ passed through as a literal argument. No shell, no injection.
```

With `execve`, each argument is passed directly to the program as a separate string; no shell ever interprets it, so the “malicious” text is just an odd filename. (If the value could begin with `-`, it could still be mistaken for an option; pass `--` before it, for programs that support that convention.)

Also avoid `execlp` and `execvp`, which search the `PATH` environment variable — which may be attacker-controlled — and any construction of a `sh -c` string. Use `posix_spawn`, or `fork` + `execve`, with an absolute path and a sanitised environment (Module 24).

> ⚠ In setuid programs, sanitise the *entire* environment — `PATH`, `LD_PRELOAD`, `LD_LIBRARY_PATH`, `IFS`, and locale variables. glibc strips some of these for setuid binaries, but not all, and not on every platform.

23.6 Input Validation and Canonicalisation

> Validate at the trust boundary, once, using an allow-list. Canonicalise before you validate.

The path-traversal trap — CWE-22

```
snprintf(path, sizeof path, "/var/data/%s", user_file);
/* user_file = "../../etc/passwd" */
```

Rejecting `..` textually is not enough. Attackers use `....//`, URL encoding (`%2e%2e`), overlong UTF-8 (Module 8), Unicode normalisation tricks, symbolic links, and absolute paths.

The correct order:

1. **Canonicalise** with `realpath()` — which resolves `.`, `..`, and symbolic links into a single absolute path — or open relative to a directory file descriptor with `openat`.
2. **Then verify** that the result is under your base directory.
3. **Then open it,** with `O_NOFOLLOW`.

Note that steps 1–3 are themselves a check-then-use sequence: a symbolic link could be swapped in after `realpath` returns. On Linux, `openat2()` with `RESOLVE_BENEATH` performs the containment check atomically as part of opening the file, and is the most robust option.

Better still: **don’t use user input as a filename at all.** Map an opaque identifier through a table you control.

Allow-list, not deny-list

```
/* BAD: deny-list — you will miss something */
if (strstr(input, "..") || strchr(input, '/')) reject();

/* GOOD: allow-list — enumerate what IS permitted */
for (const char *p = input; *p; p++)
    if (!isalnum((unsigned char)*p) && *p != '_' && *p != '-') return -1;
if (strlen(input) > MAX_NAME) return -1;
```

A deny-list must anticipate every bad input, and attackers only need one you missed. An allow-list only needs to describe the good inputs, and rejects everything else by default. Note the `(unsigned char)` cast: Module 14’s `ctype` trap is also a security issue on non-ASCII input.

The checklist for every untrusted input

- **Length** — bounded, before any copy.
- **Type and range** — parsed with `strtol` and all four checks (Module 8).
- **Character set** — an allow-list.
- **Encoding** — validate UTF-8; reject overlong forms and surrogates (Module 8, §8.6).
- **Structure** — for binary formats, validate every length field against the input that actually remains.
- **Canonical form** — resolve before deciding.

23.7 Standards: CERT C, MISRA, CWE

Standard	Scope	Use it when
CERT C (SEI)	~120 rules plus recommendations, each with non-compliant and compliant examples and a risk assessment	The best general secure-coding reference for C. Free online.
MISRA C:2012 / 2023	~175 guidelines for safety-critical embedded code. Forbids or restricts recursion, dynamic allocation, `goto`, much of the preprocessor, unions, and VLAs	Automotive (ISO 26262), medical (IEC 62304), industrial (IEC 61508). Often contractually required.
CWE Top 25 (MITRE)	The 25 most dangerous weaknesses, ranked by real-world prevalence and impact	A prioritisation list for what to test and review
AUTOSAR C++ / JSF AV C++	Adjacent standards you'll hear referenced	Aerospace and automotive C++
Annex K (`_s` functions)	Optional C11 bounds-checked functions	⚠ Effectively Microsoft-only; glibc declined to implement it, and the C committee has discussed removing it. Skip it.

MISRA's rules are divided into *mandatory*, *required*, and *advisory* categories, and projects may formally *deviate* from required and advisory rules with a documented justification — so "MISRA bans X" usually means "X needs a written deviation." Its restrictions exist to make programs analysable and predictable: no recursion means stack depth can be bounded statically, and no dynamic allocation means memory can never run out at run time.

The CWEs that are C-specific

CWE	Name	Module
CWE-787	Out-of-bounds Write	7, 10
CWE-125	Out-of-bounds Read	7, 10
CWE-416	Use After Free	10
CWE-476	NULL Pointer Dereference	9, 16

CWE	Name	Module
CWE-190	Integer Overflow or Wraparound	2
CWE-119/120/121/122	Buffer overflow family	8
CWE-134	Format String	13
CWE-415	Double Free	10
CWE-401	Memory Leak	10
CWE-457/908	Use of Uninitialized Memory	3, 21
CWE-367	TOCTOU	23
CWE-78	OS Command Injection	23
CWE-22	Path Traversal	23
CWE-362	Race Condition	17

Out-of-bounds write and out-of-bounds read have been at or near the top of the CWE Top 25 for years, and they are overwhelmingly C and C++ bugs. Notice that the “Module” column covers almost the whole of Part 1: secure C *is* correct C.

Tooling: clang-tidy has `cert-*` checks; PC-lint, Helix QAC, Coverity, LDRA, and Parasoft all check MISRA compliance.

23.8 Hardening

Hardening flags do not fix bugs. They make bugs that slip through much harder to exploit — turning code execution into a crash, or making addresses unpredictable.

Measured on the same source

```
BINARY   PROTECTIONS
h_plain  PIE=no  RELRO=none NX=NO  canary=no  CET=yes
h_hard   PIE=yes RELRO=full NX=yes canary=yes CET=yes
```

The same program, built with and without the flags below, verified by inspecting the binaries with `readelf` and `nm` (Module 19).

The flags and what they do

Flag	Protection
`-fstack-protector-strong`	A canary between the locals and the return address; abort on corruption. `-strong` covers far more functions than plain `-fstack-protector`, at negligible cost.
`-D_FORTIFY_SOURCE=3`	Compile-time and run-time bounds checks on `memcpy`, `strcpy`, `sprintf`, and friends. Requires `-O1` or higher. Level 3 (GCC 12+) also handles sizes known only at run time.

Flag	Protection
`-fPIE -pie`	Position-independent executable → ASLR applies to the main binary too
`-Wl,-z,relro,-z,now`	Full RELRO: the GOT is made read-only after relocation, blocking GOT-overwrite attacks
`-Wl,-z,noexecstack`	NX/DEP — the stack is not executable
`-fstack-clash-protection`	Probes each stack page so that a huge frame cannot skip over the guard page (the VLA/`alloca` attack, Modules 7 and 10)
`-fcf-protection=full`	Intel CET — the `endbr64` instructions seen in every assembly dump. Indirect branches must land on a valid target, and returns are checked against a shadow stack where the OS supports it.
Clang `-fsanitize=cfi`	Software control-flow integrity: indirect calls must match the called function's type (requires LTO)
`-ftrivial-auto-var-init=zero`	Zero-initialises automatic variables — eliminating the uninitialized-read class (Module 3) at around 1% cost (GCC 12+, Clang)
`-Wl,-z,separate-code`	Keeps code and data on separate pages
`-fno-common`	The default since GCC 10; prevents accidental merging of tentative definitions (Module 3)

RELRO ("relocation read-only") needs a word of explanation. Calls to shared-library functions go through the Global Offset Table (Module 20), which the dynamic loader fills in at run time. If the GOT stays writable, an attacker with a memory-write primitive (such as a format-string `%n`) can overwrite an entry and redirect every future call to `printf`, say, to their own code. Full RELRO resolves every entry at start-up (`-z now`) and then marks the table read-only.

(The topic list for this module in the original roadmap also mentioned `-D_GLIBCXX_ASSERTIONS`. That macro enables bounds checks in the C++ standard library, libstdc++, and has no effect on C code; it belongs in the hardening flags of a C++ project.)

The production build

```
CFLAGS="-std=c17 -O2 -g \
  -Wall -Wextra -Wformat=2 -Werror=format-security \
  -fstack-protector-strong -fstack-clash-protection -fcf-protection=full \
  -D_FORTIFY_SOURCE=3 -ftrivial-auto-var-init=zero \
  -fPIE"
LDFLAGS="-pie -Wl,-z,relro,-z,now,-z,noexecstack,-z,separate-code"
```

Verify the result with `checksec --file=./prog`, `hardening-check`, or the `readelf`/`nm` checks above. Add a CI job that fails if any protection is missing — flags have a way of disappearing when someone rewrites a build script.

Beyond compiler flags

- **Sandboxing:** seccomp-bpf (restricting which system calls a process may make), Linux capabilities, namespaces, `pledge`/`unveil` (OpenBSD), AppArmor and SELinux.
- **Privilege separation:** drop privileges early and permanently, and split risky parsing into an unprivileged process — OpenSSH's model, in which the code that handles network input runs with almost no rights.
- **Hardened allocators:** Scudo, hardened_malloc, `GLIBC_TUNABLES=glibc.malloc.check=3`.
- **Testing aids:** `MALLOC_PERTURB_` fills freed memory with garbage in testing, so that use-after-free bugs produce obviously wrong values instead of stale correct ones.

23.9 Constant-Time Code

The timing side channel, measured

```c
static int ct_equal(const void *a, const void *b, size_t n)
{
    const unsigned char *x = a, *y = b;
    unsigned char diff = 0;
    for (size_t i = 0; i < n; i++) diff |= (unsigned char)(x[i] ^ y[i]);
    return diff == 0;   /* time depends only on n */
}
```

With 65,536-byte buffers and 20,000 comparisons:

```
memcmp,   differs at byte 0     :  0.1 ms
memcmp,   differs at byte 65535 : 23.0 ms   <-- 328x SLOWER
ct_equal, differs at byte 0     : 39.9 ms
ct_equal, differs at byte 65535 : 40.0 ms   <-- 1.00x (constant)
```

A 328× timing difference that directly reveals how many bytes matched (Module 8). An attacker submits candidate tokens one byte at a time, keeping whichever prefix takes longest to reject, and recovers the secret in about $256 \times n$ attempts instead of 256^n. For a 16-byte token, that is roughly 4,000 guesses instead of more than 10^{38}.

(With 32-byte buffers, glibc's vectorised `memcmp` masked the effect in the first test — which is exactly why you must never rely on incidental timing behaviour. It varies with the C library, the buffer size, and the CPU. Over a network, attackers average many measurements to remove noise.)

The rules for secret-handling code

1. **No branches on secret data.** No `if (secret_byte == x)`.
2. **No array indexing by secret data** — the access pattern leaks through the CPU cache. This is how table-lookup implementations of AES were broken.
3. **No division or modulo by secret values** — their latency varies with the operands on many CPUs.
4. **No early returns from comparison loops.**
5. **Use branchless masking:** `result = (mask & a) | (~mask & b)`, where `mask` is all ones or all zeros.

6. **Zero secrets with `explicit_bzero` / `memset_explicit`** — Module 10 showed the compiler deleting a plain `memset` entirely, leaving the secret on the stack.
7. **`mlock` secret pages** so they are never swapped to disk, and disable core dumps in processes that hold keys.
8. **Use a vetted library.** `CRYPTO_memcmp` (OpenSSL), `sodium_memcmp` (libsodium), and similar functions exist because getting this right is hard — the compiler can even reintroduce branches into code that looks constant-time.

> **Do not write your own crypto.** Use libsodium, BoringSSL, or your platform's cryptographic provider. The constant-time discipline above is for the code *around* the crypto — token comparison, HMAC verification, padding checks.

23.10 Secure API Design

Bounds-carrying interfaces

The root cause of C's memory bugs is that a pointer does not know how much memory it points to. Fix that at the type level:

```
typedef struct { const uint8_t *p; size_t n; } Span;   /* read-only view */
typedef struct { uint8_t *p; size_t n; } MutSpan;      /* writable view */

static bool span_take(Span *s, size_t n, Span *out)
{
    if (n > s->n) return false;   /* THE bounds check */
    if (out) *out = (Span){ s->p, n };
    s->p += n; s->n -= n;
    return true;
}
static bool span_u16(Span *s, uint16_t *v)
{ Span t; if (!span_take(s,2,&t)) return false; *v = (uint16_t)((t.p[0]<<8)|t.p[1]); return true; }

static bool span_copy(MutSpan dst, Span src)
{ if (src.n > dst.n) return false; memcpy(dst.p, src.p, src.n); return true; }
```

A *span* is a pointer and a length that travel together. `span_take` is the only function that advances through the data, and it contains the only bounds check the parser needs: it refuses to take more bytes than remain. Every other reader (`span_u8`, `span_u16`, …) is built on top of it, so none of them can read out of bounds.

A parser written entirely against spans:

```
static bool parse_record(Span in, Record *r)
{
    if (!span_u8 (&in, &r->type)) return false;
    if (!span_u16(&in, &r->len)) return false;
    Span body;
    if (!span_take(&in, r->len, &body)) return false;                          /* input bound */
    if (!span_copy((MutSpan){ r->value, sizeof r->value }, body)) return false;   /* output bound */
    r->vlen = body.n;
    return true;
}
```

```
BOUNDS-CARRYING API - the same parser, three inputs:
  valid record          : OK type=1 value="hello"
```

```
  truncated (len>input) : rejected
  oversized (len>dest)  : rejected

  Every read and write is bounds-checked by construction.
  There is no raw pointer arithmetic anywhere in the parser.
ASan exit=0
```

This is the same TLV format as Module 22's fuzzing example — and the two bugs the fuzzer found there (no input bound, no output bound) are simply impossible to write here, because every read goes through `span_take` and every write through `span_copy`.

This is the single highest-leverage secure-coding technique in C. It is what Rust's slices give you for free, and you can have most of it with 30 lines of C.

The API design checklist

Principle	Implementation
Length travels with the pointer	Spans, or (pointer, length) pairs — never a bare `char *`
Fail closed	Default to rejecting; success must be explicit
No silent truncation	Return an error; don't quietly shorten (Module 8's `snprintf` check)
Ownership is documented	Module 10, §10.6
Opaque handles	Module 11, §11.8 — invariants can't be bypassed
`[[nodiscard]]`	Module 18 — ignoring an error becomes a compile-time warning
Validate at the boundary	Then `assert` internally (Module 15)
Minimise the attack surface	`-fvisibility=hidden` (Module 20); expose only what's needed
No global mutable state	Pass a context structure
Make the safe thing the easy thing	If your API is easy to misuse, it will be misused

Threat modelling

Before writing security-relevant code, answer four questions:

1. **What are you protecting?** Data, availability, integrity, keys.
2. **Who is the adversary, and what can they control?** Network input, file contents, the environment, a local user, another process.
3. **What are the trust boundaries?** Every place untrusted data enters — that is where validation goes.
4. **What happens if this component is compromised?** Its "blast radius" — does it need privilege separation?

STRIDE — Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege — is the standard checklist of threat categories. Draw a data-flow diagram, mark the trust boundaries, and enumerate the threats at each crossing.

Module 23 Rules

#	Rule
1	Banned: `gets`, `strcpy`, `strcat`, `sprintf`, `scanf("%s")`, `alloca`, VLAs, `system`, `popen`, `mktemp`, `atoi`
2	Never pass non-literal data as a format string; `-Werror=format-security`
3	Every `n * size` computation gets an overflow check — or use `calloc`/`ckd_mul`
4	Check for negative first, then convert to `size_t`
5	Watch every place a length crosses a type boundary
6	`free(p); p = NULL;` — one owner, documented lifetime
7	Open first, then `fstat` — never `access`/`stat` then `open`
8	`mkstemp` + `umask(0077)` + immediate `unlink`
9	`execve` with an argument vector and an absolute path — never a shell
10	Canonicalise, then validate, with an allow-list
11	Ship with the full hardening flag set; verify it in CI with `checksec`
12	Constant-time comparison for every secret; `explicit_bzero` to wipe
13	Don’t write your own crypto
14	Bounds-carrying spans instead of bare pointers
15	Threat-model before you write; validate at the boundary, assert inside
16	Fuzz everything that parses untrusted input, with ASan + UBSan (Module 22)

Practice

1. **Stack layout.** Reproduce the §23.1 frame dump on your machine and compute the exact distance from `buf[0]` to the return address. Then rebuild with `-fstack-protector-strong` and find the canary in the frame with GDB.

2. **Defence comparison.** Take one `strcpy` overflow and build it five ways: no protection, canary only, FORTIFY only, both, and ASan. Record the exact diagnostic and exit code for each. Which tells you the most? Which costs the least?

3. **Format string.** Write a logging function that takes user input as a format string. Demonstrate the information leak with `%p`. Then add `-Wformat=2 -Werror=format-security` and confirm that the build fails. Fix it, and confirm that `_FORTIFY_SOURCE` also blocks the `%n` case.

4. **Integer-overflow chain.** Write an `array_new(size_t n, size_t elemsize)` with no overflow check, followed by a loop that fills the array. Find an `n` that produces a tiny allocation. Confirm that ASan catches the heap overflow. Fix it three ways (explicit check, `calloc`, `__builtin_mul_overflow`) and verify each.

5. **Signed length bypass.** Write a "safe-looking" length check using a signed `int` that a negative value defeats. Prove it under ASan. Then fix it, and add `-Wconversion -Wsign-conversion` to catch the whole class.

6. **TOCTOU.** Write the access-then-open pattern. Write a shell loop that races it by swapping the file for a symbolic link. Then rewrite it with `open` + `fstat` + `O_NOFOLLOW`, and confirm that the race is closed.

7. **Command injection.** Write a "file lister" using `system()`. Break it with a crafted filename. Rewrite it with `fork` + `execve` and an argument vector, and prove that the same input is now harmless.

8. **Timing attack.** Reproduce the §23.9 measurement. Then write a program that recovers a 16-byte secret using only timing, one byte at a time, against a `memcmp`-based check. Repeat against `ct_equal` and show that the attack fails.

9. **Hardening audit.** Build a project with and without the full flag set. Verify the results with `checksec` (or the `readelf`/`nm` checks). Measure the performance and binary-size cost. Then write a CI check that fails if any protection is missing.

10. **Span library.** Implement a complete `Span`/`MutSpan` API: `span_take`, `span_u8/16/32/64` (in both byte orders), `span_bytes`, `span_find`, `span_split`, `span_copy`, and `span_eq`. Rewrite a parser you have already written to use it exclusively. Fuzz both versions and compare the crash counts.

11. **Threat model.** Take a program you have written. Draw its data-flow diagram, mark the trust boundaries, enumerate the STRIDE threats at each crossing, and list the mitigation for each. Then check: does every untrusted input actually get validated at the boundary you drew?

Module 23 in Summary

In C, security bugs are mostly ordinary correctness bugs on attacker-chosen paths. Stack overflows reach the return address and heap overflows reach allocator metadata, so the unbounded string functions are banned, off-by-one errors are treated as exploitable, and stack canaries, `_FORTIFY_SOURCE`, and ASan convert silent corruption into an immediate abort. Format strings must always be literals, because `%p` leaks memory and `%n` writes it.

Integer bugs chain into memory bugs: a wrapped `n * size` produces a tiny allocation, a negative length slips past a signed check and becomes enormous, and truncation shrinks a length across type boundaries — so check every multiplication, check for negatives before converting, and enable the conversion warnings. Use-after-free, double free, and type confusion give attackers control of reused memory, and are prevented by clear ownership and tagged types. Operate on file descriptors rather than names to avoid TOCTOU races, create temporary files with `mkstemp`, run programs with `execve` and an argument vector rather than a shell, and canonicalise and allow-list every untrusted input.

CERT C, MISRA, and the CWE Top 25 codify these lessons. Ship with the full hardening flag set — PIE, full RELRO, NX, canaries, stack-clash protection, CET, and zero-initialised locals — and verify it in CI. Compare secrets in constant time and wipe them with functions the compiler cannot remove; never write your own cryptography. Above all, design APIs whose lengths travel with their pointers — spans make whole classes of overflow impossible by construction — and threat-model before writing security-relevant code.

Next: Module 24 — Systems / POSIX Programming, the largest module in Part 2: processes and `fork`/`exec`, signals done correctly, IPC, the file-system API, sockets and networking, I/O multiplexing (`select`/`poll`/`epoll`/`io_uring`), and pthreads in practice.

Module 24 — Systems / POSIX Programming

In this module:

- System calls vs library calls
- Processes: `fork`, `exec*`, `wait`, zombies, orphans, process groups
- Signals: handlers, async-signal-safety, `sigaction`, masks, `signalfd`
- IPC: pipes, FIFOs, UNIX sockets, shared memory, semaphores, message queues, `mmap`
- The file system: paths, permissions, `stat`, directories, symbolic links, `inotify`
- Networking: TCP/UDP sockets, `bind`/`listen`/`accept`/`connect`, byte order, non-blocking I/O
- Multiplexing: `select`, `poll`, `epoll`, `kqueue`, `io_uring`
- pthreads: create/join/detach, mutexes, condition variables, rwlocks, barriers, TLS, thread pools
- Lock-free programming, the ABA problem, memory barriers
- Daemons, resource limits, `getopt`, environment variables
- Dynamic loading: `dlopen`/`dlsym`, plugin architectures
- Windows counterparts, if you target it: the Win32 API, the `HANDLE` model, IOCP

This is the largest module in Part 2. All output is real, from Linux 6.18 and glibc 2.39.

ISO C knows nothing about processes, networks, or directories; its world ends at `FILE *` streams and `<threads.h>`. Almost every real C program, however, runs on an operating system and needs those things — and on Linux, macOS, the BSDs, and most embedded Linux devices, the interface to them is **POSIX**, the Portable Operating System Interface standard. This module is a working tour of that interface: how programs create and manage other programs, handle asynchronous events, talk to each other and across networks, wait on many things at once, and load code at run time.

24.1 System Calls vs Library Calls

```
libc write() wrapper
direct syscall(SYS_write)
printf() -> buffered, may issue ZERO syscalls until flush
clock_gettime -> served from the vDSO, no context switch
```

A *system call* is a request to the operating-system kernel: the processor switches from user mode to kernel mode, the kernel does the work, and control returns. A *library call* is an ordinary function call within your process. The C library sits between the two, and many of its functions are thin wrappers around system calls — while others avoid system calls entirely. There are three layers:

```
your code →   libc function  →  syscall wrapper  →  kernel
printf()      buffers           write(2)             sys_write
```

```
malloc()      free lists        brk/mmap            sys_brk
clock_gettime() ──────────────► vDSO (no kernel entry at all)
```

(The "(2)" in `write(2)` is the Unix manual convention: section 2 of the manual documents system calls, section 3 library functions.)

The measured cost

```
200000 iterations:
  empty loop            :  0.00 ms
  clock_gettime (vDSO)  :  6.04 ms (30 ns/call)
  getpid (real syscall) : 18.73 ms (94 ns/call)
```

A real system call costs about 94 ns here; a vDSO call about 30 ns; an ordinary function call about 1 ns. That roughly 100× gap is why buffering exists (Module 13), why `epoll` beats `select`, and why `io_uring` was invented.

The *vDSO* ("virtual dynamic shared object") is a small shared library that the kernel maps into every process. It provides `clock_gettime`, `gettimeofday`, `getcpu`, and `time` by reading data the kernel keeps in shared memory — without any switch into kernel mode. The mitigations for the Spectre and Meltdown CPU vulnerabilities substantially increased the cost of real system calls on many machines, so measure on your target.

```
$ strace -c -e trace=write,clock_gettime,getpid ./sc
 85.71  0.000012  4  3  write
 14.29  0.000002  1  2  getpid
```

Note that `clock_gettime` does not appear at all: it never reached the kernel, so `strace` — which intercepts system calls — never saw it.

Practical implications: batch system calls (`writev`, `readv`, `sendmmsg`), buffer aggressively, prefer `pread`/`pwrite` to `lseek` + `read` (one call instead of two, and thread-safe because the file offset is not shared), and remember that almost any system call can fail with `EINTR` when a signal arrives — always retry (Module 13, §13.9).

24.2 Processes

`fork`

```
FORK: returns TWICE - 0 in the child, the child's pid in the parent
  reaped 747: exited with 1
  reaped 748: exited with 2
  reaped 749: exited with 3
```

```c
fflush(stdout);            /* ← CRITICAL: flush BEFORE fork */
pid_t p = fork();
if (p < 0) { perror("fork"); return -1; }
if (p == 0) { /* child */ _exit(code); }
/* parent */
```

`fork` creates a new process that is a near-exact copy of the caller. It is called once and returns twice: in the new *child* process it returns 0, and in the original *parent* it returns the child's process ID. Both continue executing from the same point, distinguishing themselves only by that return value.

Why the flush: `fork` copies the entire address space, *including stdio's buffers*. Any output sitting unflushed in a buffer is duplicated in both processes, and both eventually print it. This produces the classic "my `printf` appeared twice" bug — often only when output is redirected to a file, because on a terminal the line buffering (Module 13) had already flushed it. The child uses `_exit` rather than `exit` for the same reason: `exit` would flush the child's copy of any buffered output again.

The child gets a copy of the parent's memory (implemented with *copy-on-write*: pages are shared until one process writes to them, so `fork` is cheap), copies of all open file descriptors (sharing the same file offsets), and the same signal handlers. It does *not* inherit pending signals, timers, or any other threads.

> ⚠ **`fork` in a multithreaded program is dangerous.** Only the calling thread survives in the child; a mutex held by any other thread stays locked forever. Between `fork` and `exec` in a threaded program, only async-signal-safe functions (§24.3) may be called — so no `malloc` and no `printf`. Use `posix_spawn`, or `vfork` + `exec`.

exec

```
EXEC: replaces the process image (same pid, new program)
  before exec: pid=750
  after exec: this is /bin/echo
```

```c
char *args[] = { "/bin/echo", "text", NULL };
execve("/bin/echo", args, envp);
perror("execve");   /* only reached if exec FAILED */
_exit(127);
```

`exec` replaces the current program with a new one, in the same process: same process ID, same open file descriptors, but new code, data, and stack. It never returns on success, because the code that called it no longer exists. If it does return, it failed — so always `_exit` straight after it (127 is the conventional status for "command not found," Module 1).

`fork` followed by `exec` is how Unix runs a new program: `fork` makes a copy, and the copy `exec`s the new program, while the parent carries on (and usually waits for it).

Variant	Arguments	`PATH` search	Environment
`execl` / `execlp` / `execle`	list	`p` = yes	`e` = explicit
`execv` / `execvp` / `execve`	vector	`p` = yes	`e` = explicit

The letters in the names encode the differences: `l` takes the arguments as a list of parameters, `v` as an array ("vector"); `p` searches the `PATH` environment variable for the program; `e` takes an explicit environment.

> Use `execve` with an absolute path in security-sensitive code (Module 23). The `p` variants search a `PATH` that an attacker may be able to influence.

File descriptors survive `exec` unless they are marked close-on-exec (`FD_CLOEXEC`). Always open with `O_CLOEXEC` to prevent descriptors — to log files, sockets, secrets — from leaking into every program you run.

Zombies and orphans

```
ZOMBIE: a child that exited but was not reaped
  765 Z zb
^ 'Z' = zombie: holds a pid slot until the parent wait()s
after waitpid: reaped, slot released
```

When a process exits, the kernel keeps a small record of it — its exit status — until its parent collects that status with `wait` or `waitpid`. Until then, the dead child is a *zombie*: it holds a process-ID slot, shown as state `Z` by `ps`. Enough zombies exhaust the process table, and no new processes can be created.

An *orphan* is a *live* child whose parent has exited; it is re-parented to `init` (process 1), or to a designated "subreaper," which reaps it when it eventually exits. That is how daemons detach from the process that started them.

Three ways to avoid zombies:

```c
/* 1. Blocking wait */
waitpid(pid, &status, 0);

/* 2. SIGCHLD handler — the standard pattern */
static void reaper(int sig) {
    (void)sig;
    int saved = errno;                          /* preserve errno (Module 15) */
    while (waitpid(-1, NULL, WNOHANG) > 0) { }   /* reap ALL — signals aren't queued */
    errno = saved;
}
struct sigaction sa = {0};
sa.sa_handler = reaper;
sa.sa_flags = SA_RESTART | SA_NOCLDSTOP;
sigaction(SIGCHLD, &sa, NULL);

/* 3. Ignore SIGCHLD — auto-reap, but you lose the exit status */
signal(SIGCHLD, SIG_IGN);
```

The kernel sends `SIGCHLD` to a parent whenever a child exits. The `while` loop in the handler is essential: standard signals are not queued (§24.3), so three children exiting at almost the same moment may produce only one `SIGCHLD`. `WNOHANG` makes `waitpid` return 0 instead of blocking when no more children have exited, which ends the loop. The handler saves and restores `errno` because `waitpid` may change it, and the code the signal interrupted may be about to examine it.

Status decoding

```c
if      (WIFEXITED(status))   code   = WEXITSTATUS(status);
else if (WIFSIGNALED(status)) signal = WTERMSIG(status);
else if (WIFSTOPPED(status))  signal = WSTOPSIG(status);
```

The status from `waitpid` packs several facts into one `int`; these macros decode it: whether the child exited normally (and with what code), was killed by a signal (and which), or was stopped.

Process groups and sessions

```
parent pid=746 pgid=739 sid=0
```

Processes are organised hierarchically: a *session* contains *process groups*, which contain processes. A terminal-generated signal (Ctrl-C sends `SIGINT`) goes to the *entire* foreground process group — which is why Ctrl-C kills every process in a pipeline such as `a | b | c`, not just one.

`setsid()` creates a new session with no controlling terminal — the core of daemonising (§24.9). `setpgid()` creates a process group, which is useful for managing a whole tree of processes: `kill(-pgid, SIGTERM)` (note the negative ID) signals every process in the group.

24.3 Signals

A *signal* is an asynchronous notification delivered to a process: the user pressed Ctrl-C (`SIGINT`), a child exited (`SIGCHLD`), a write went to a closed pipe (`SIGPIPE`), the program dereferenced an invalid pointer (`SIGSEGV`), or another process sent one with `kill`. Module 14 introduced ISO C's minimal `signal`; this is the POSIX version.

`sigaction`, not `signal`

```
signal()   : reset behaviour, restart behaviour, and masking are
             all IMPLEMENTATION-DEFINED. Do not use it.
sigaction(): defined semantics, a mask during the handler, SA_RESTART,
             SA_SIGINFO for siginfo_t, SA_NOCLDWAIT, SA_ONSTACK
```

```c
struct sigaction sa = {0};
sa.sa_handler = on_term;
sigemptyset(&sa.sa_mask);
sigaddset(&sa.sa_mask, SIGUSR2);   /* additionally blocked during the handler */
sa.sa_flags = SA_RESTART;          /* auto-restart interrupted syscalls */
sigaction(SIGUSR1, &sa, NULL);
```

With `signal()`, different systems historically disagreed about whether a handler stayed installed after it ran, whether interrupted system calls were restarted, and whether the signal was blocked during its own handler. `sigaction` specifies all of it. `sa_mask` lists additional signals to block while the handler runs.

Flags worth knowing:

- **`SA_RESTART`** — automatically restart system calls interrupted by this signal, rather than failing them with `EINTR`.
- **`SA_SIGINFO`** — the handler receives a `siginfo_t` with details such as the sender's process ID or, for `SIGSEGV`, the faulting address.
- **`SA_NOCLDSTOP`, `SA_NOCLDWAIT`** — control `SIGCHLD` behaviour.

- **`SA_ONSTACK`** — run the handler on an alternate stack (set up with `sigaltstack`). This is the only way to handle `SIGSEGV` caused by a stack overflow, since the normal stack is exhausted.
- **`SA_RESETHAND`** — restore the default action after one delivery.

⚠ Async-signal-safety

```
SAFE in a handler: write, _exit, signal, sigaction, kill, read,
  sem_post, and assignment to volatile sig_atomic_t
NOT SAFE: printf, malloc, free, exit, any locking function
  -> calling malloc from a handler can DEADLOCK (the heap lock)
```

A signal can interrupt your program at *any* instruction — including in the middle of `malloc`, while it holds the heap's internal lock. If the handler then calls `malloc`, it waits for a lock that its own interrupted code holds, and will never release: deadlock. The same applies to `printf` (stdio locks), `exit` (which runs `atexit` handlers and flushes stdio), and anything that changes `errno` without saving it.

POSIX lists roughly 180 *async-signal-safe* functions — those guaranteed safe to call from a handler. In practice, the useful ones are `write`, `read`, `_exit`, `kill`, `sigaction`, `sem_post`, and lock-free atomics.

The three correct patterns

1. A flag and the main loop

```c
static volatile sig_atomic_t g_stop = 0;
static void on_term(int sig) { (void)sig; g_stop = 1; }   /* that's ALL */

while (!g_stop) { do_work(); }
```

```
after raise(SIGUSR1): g_stop = 1
```

The handler only records that the signal happened; the main loop notices the flag at a safe point and does the real work.

> ⚠ `volatile sig_atomic_t` is the only type the C standard guarantees for this — and it is for signals, not threads (Module 17). In a threaded program, where the signal may be handled on a different thread from the one checking the flag, use a lock-free `_Atomic` variable.

2. The self-pipe trick — turn signals into file-descriptor events

```c
static void on_pipe(int sig) { char b = (char)sig; write(self_pipe[1], &b, 1); }

pipe2(self_pipe, O_CLOEXEC | O_NONBLOCK);
/* now poll/select/epoll on self_pipe[0] alongside your sockets */
```

```
poll() woke on the pipe: signal 12 delivered as an fd event
-> turns an async signal into an ordinary readable fd
```

The handler writes one byte — an async-signal-safe operation — into a pipe that the program owns. The main event loop (§24.7) watches the pipe's read end along with its sockets. This solves the fundamental event-loop problem: signals and I/O become the same kind of event, handled in one place, with no races. (The pipe is non-blocking so that the handler can never block if the pipe fills up.)

3. `signalfd` (Linux) — no handler at all

```
sigset_t mask; sigemptyset(&mask); sigaddset(&mask, SIGUSR1);
sigprocmask(SIG_BLOCK, &mask, NULL);   /* MUST block it first */
int sfd = signalfd(-1, &mask, SFD_CLOEXEC);

struct signalfd_siginfo si;
read(sfd, &si, sizeof si);             /* si.ssi_signo, si.ssi_pid, ... */
```

```
signalfd read: signo=10 from pid=794
-> no handler at all; signals become readable data
```

The signal is blocked, so it is never delivered asynchronously; instead it waits as a readable record on the `signalfd` descriptor. This is the cleanest option of all: there are no async-signal-safety constraints, because there is no handler. The BSDs and macOS offer the same idea through `kqueue` with `EVFILT_SIGNAL`.

Masks and pending signals

```
sigprocmask(SIG_BLOCK): SIGINT is now PENDING, not lost
sigpending: SIGINT pending = 1
standard signals are NOT queued - N deliveries while blocked = 1 pending
(SIGRTMIN..SIGRTMAX real-time signals ARE queued)
```

A *blocked* signal is not discarded; it becomes *pending*, and is delivered as soon as it is unblocked. But for standard signals, "pending" is a single bit: if the same signal arrives five times while blocked, it is delivered once. (Real-time signals, numbered `SIGRTMIN` to `SIGRTMAX`, are queued individually.)

`sigsuspend` atomically unblocks signals and waits for one, avoiding the classic race in which a signal arrives between checking a flag and going to sleep. In threaded programs, use `pthread_sigmask`: the signal mask is per thread, but handlers are per process.

Signals every server must handle:

- **`SIGPIPE`** — ignore it, or a peer closing its connection kills your process (§24.6).
- **`SIGTERM`/`SIGINT`** — graceful shutdown.
- **`SIGCHLD`** — reaping children.
- **`SIGHUP`** — conventionally, reload the configuration.

24.4 Inter-Process Communication (IPC)

Separate processes have separate address spaces (Module 10), so they cannot simply share variables. IPC mechanisms let them exchange data anyway.

```
=== PIPE (unidirectional, related processes) ===
  parent read 16 bytes: "through the pipe"
  pipe capacity: 65536 bytes (write blocks when full)
  writing to a pipe with no reader -> SIGPIPE (or EPIPE if ignored)

=== UNIX SOCKETPAIR (bidirectional, can pass FDs) ===
  ping -> "pong" (full duplex on one pair)
  SCM_RIGHTS over a UNIX socket passes open FILE DESCRIPTORS between processes

=== SHARED MEMORY (mmap MAP_SHARED|MAP_ANONYMOUS) ===
  4 processes x 10000 increments -> counter = 40000 (expected 40000)
  the memory is the SAME PHYSICAL PAGES in every process
```

A *pipe* is a one-way byte channel created before `fork`, so that parent and child each hold one end — exactly how the shell connects `a | b`. A *socketpair* is a two-way equivalent. And shared memory maps the same physical pages into several processes, so that they really do share variables — which then need synchronisation, exactly as threads do.

The mechanisms

Mechanism	Direction	Relationship	Notes
Pipe	one-way	related (via `fork`)	The simplest. Writes of at most `PIPE_BUF` bytes (4,096 on Linux) are atomic.
FIFO (`mkfifo`)	one-way	unrelated	A pipe with a name in the file system
UNIX socket	two-way	unrelated	Passes file descriptors via `SCM_RIGHTS`; supports datagrams with message boundaries
`socketpair`	two-way	related	The best replacement for pipes
Shared memory	n/a	either	The fastest — no copying. Needs explicit synchronisation.
POSIX semaphore	n/a	either	`sem_init(pshared=1)` in shared memory, or `sem_open("/name")`
POSIX message queue	either	unrelated	`mq_open`/`mq_send` — prioritised, with message boundaries
System V IPC	—	—	`shmget`/`semget`/`msgget`. Legacy: an awkward API, and objects leak on a crash. Avoid.
`eventfd` / `pidfd`	—	—	Linux; lightweight event counters and process handles

The “atomic up to `PIPE_BUF`” guarantee matters when several writers share a pipe: a write of at most that size is never interleaved with another writer’s data. (POSIX requires `PIPE_BUF` to be at least 512 bytes.)

Shared memory with a semaphore

```
struct Shared { sem_t sem; long counter; };
struct Shared *sh = mmap(NULL, sizeof *sh, PROT_READ|PROT_WRITE,
                         MAP_SHARED | MAP_ANONYMOUS, -1, 0);
sem_init(&sh->sem, 1, 1);   /* pshared=1 → usable across processes */
```

A `MAP_SHARED` mapping created before `fork` is shared with every child — shared memory for free. For unrelated processes, use `shm_open("/name")` + `ftruncate` + `mmap`, so that each process maps the same named object. The semaphore, initialised to 1, acts as a mutex: each process waits on it before incrementing the counter and posts it afterwards — which is why the four processes’ 40,000 increments all survived, unlike Module 17’s unsynchronised counters.

> ⚠ **Shared memory has no built-in synchronisation.** All of Module 17 applies: you need semaphores, process-shared mutexes (`PTHREAD_PROCESS_SHARED`), or atomics. And **never put pointers in shared memory** — the same region can be mapped at different addresses in different processes, so a pointer valid in one is garbage in another. Store offsets from the start of the region instead.

`SCM_RIGHTS` is worth knowing about: it passes a live, open file descriptor from one process to another, even an unrelated one, over a UNIX socket. The receiving process gets its own descriptor for the same open file. It is the foundation of privilege separation (a privileged helper opens a protected file and hands the descriptor to an unprivileged worker, which could never have opened it itself) and of systemd’s socket activation.

24.5 The File System

```
/etc/passwd: size=1111 mode=0644 uid=0 nlink=1 ino=26
stat follows symlinks; lstat does NOT; fstat takes an fd (TOCTOU-safe)
```

`stat` and friends

Call	Operates on
`stat(path, &st)`	The file, following symbolic links
`lstat(path, &st)`	The symbolic link itself
`fstat(fd, &st)`	An open file descriptor — race-free (Module 23)
`fstatat(dirfd, path, &st, flags)`	A path relative to a directory descriptor

Key fields: `st_mode` (file type plus permissions), `st_size`, `st_uid`/`st_gid`, `st_nlink` (number of hard links), `st_ino` + `st_dev` (together they identify a file uniquely), and `st_mtim`/`st_atim`/`st_ctim` (modification, access, and status-change times).

The file-type macros: `S_ISREG`, `S_ISDIR`, `S_ISLNK`, `S_ISFIFO`, `S_ISSOCK`, `S_ISCHR`, and `S_ISBLK`.

Directories

```c
DIR *d = opendir("/etc");
struct dirent *e;
while ((e = readdir(d)) != NULL) {
    if (!strcmp(e->d_name,".") || !strcmp(e->d_name,"..")) continue;
    /* e->d_type may be DT_UNKNOWN — fall back to fstatat */
}
closedir(d);
```

```
rc2.d (type=4)       ← DT_DIR
papersize (type=8)   ← DT_REG
d_type may be DT_UNKNOWN on some filesystems -> fall back to fstatat
```

Every directory contains the entries `.` (itself) and `..` (its parent), which a recursive walk must skip or it will loop forever. `d_type` saves a `stat` call per entry, but not every file system fills it in.

`readdir` is not thread-safe for concurrent use of the *same* `DIR *`; `readdir_r` has been deprecated in glibc, so use one `DIR *` per thread instead. For recursive walks, `nftw` or `fts` handle symbolic-link loops and depth limits for you.

The `*at` family — race-free path operations

`openat`, `fstatat`, `unlinkat`, `renameat`, `mkdirat`, `linkat`, `symlinkat`, and `faccessat` each take a *directory file descriptor* as the base for a relative path. Once you have opened a directory, its descriptor keeps referring to that directory even if someone renames or replaces the path to it — so the rest of the path cannot be swapped out from under you. Use these for anything security-sensitive (Module 23, §23.5).

Permissions and ownership

`chmod`/`fchmod`, `chown`/`fchown`, `umask` (the permission bits *removed* from every requested mode), and `access` (⚠ a TOCTOU trap — use `faccessat` with `AT_EACCESS`, or simply attempt the operation and handle the error).

Also: *hard links* (`link`) are additional names for the same inode — the same file; *symbolic links* (`symlink`) are separate small files containing a path; `realpath` canonicalises a path; and `readlink` reads a symbolic link's target without following it.

`inotify` (Linux) — file-system events

```c
int fd = inotify_init1(IN_CLOEXEC | IN_NONBLOCK);
int wd = inotify_add_watch(fd, "/etc", IN_CREATE | IN_MODIFY | IN_DELETE | IN_MOVED_TO);
/* then read() struct inotify_event records; add fd to your epoll set */
```

`inotify` delivers file-system events — file created, modified, deleted, renamed — as records read from a file descriptor, which can join an event loop like any other. (`struct inotify_event` is Module 7's flexible-array-member example.) Watches are *not* recursive: you must add one per directory, including new ones as they are created. Events can be coalesced, or lost if you read too slowly, in which case you receive `IN_Q_OVERFLOW` and must rescan. On the BSDs and macOS, use `kqueue` with `EVFILT_VNODE`, or FSEvents.

24.6 Networking

The TCP server sequence

```c
int lfd = socket(AF_INET, SOCK_STREAM | SOCK_CLOEXEC, 0);
int one = 1;
setsockopt(lfd, SOL_SOCKET, SO_REUSEADDR, &one, sizeof one);   /* avoid TIME_WAIT bind failure */

struct sockaddr_in a = { .sin_family = AF_INET,
                         .sin_port   = htons(port),                /* NETWORK byte order */
                         .sin_addr   = { htonl(INADDR_ANY) } };
bind(lfd, (struct sockaddr*)&a, sizeof a);
listen(lfd, 128);                                                  /* backlog */
int cfd = accept4(lfd, NULL, NULL, SOCK_NONBLOCK | SOCK_CLOEXEC);
```

A *socket* is a file descriptor for network communication. A TCP server creates one, *binds* it to an address and port, marks it as *listening* (the *backlog* is how many completed connections may queue waiting for `accept`), and then *accepts* connections — each `accept` returning a new descriptor for one client connection, while the listening socket carries on accepting more. `INADDR_ANY` means "every local network interface." (`accept4` is the Linux variant that sets flags on the new descriptor atomically.)

Client: `socket` → (optionally `bind`) → `connect` → `read`/`write`.

Byte order

```c
htons/htonl   /* host → network (big-endian) */
ntohs/ntohl   /* network → host */
```

Every multi-byte value on the wire in the Internet protocols is big-endian ("network byte order"). x86 is little-endian, so these are real conversions there (and no-ops on big-endian machines). The names read as "host to network, short" and so on.

Forgetting `htons` on a port number is the single most common socket bug — and it happens to "work" for ports whose two bytes are identical, such as 257 (0x0101) or 7967 (0x1F1F), which makes it worse: the bug appears only when someone changes the port.

The options that matter

Option	Why
`SO_REUSEADDR`	Allows rebinding while old connections are in `TIME_WAIT`. Always set it on servers — otherwise a restarted server fails with "Address already in use" for a minute or more.
`SO_REUSEPORT`	Multiple processes accept on the same port, with the kernel load-balancing between them
`TCP_NODELAY`	Disables Nagle's algorithm (which delays small writes to combine them). Essential for request/response protocols.
`SO_KEEPALIVE`	Detects dead peers
`SO_RCVTIMEO` / `SO_SNDTIMEO`	Timeouts on blocking sockets
`SO_LINGER`	Controls what `close()` does with unsent data

The rules that bite people

1. **TCP is a byte stream, not a message stream.** One `write` may arrive as three `read`s, or three `write`s as one `read`. You must frame your own messages — with a length prefix or a delimiter.
2. **`read` and `write` return short** — always loop (Module 13, §13.9).
3. **`read` returning 0 means the peer shut down the connection in an orderly way,** not an error.
4. **Ignore `SIGPIPE`** (`signal(SIGPIPE, SIG_IGN)`) or use `send(..., MSG_NOSIGNAL)` — otherwise writing to a connection the peer has closed kills your process.
5. **Handle `EINTR`** on every blocking call.
6. **Use `getaddrinfo`, not `gethostbyname`** — it is thread-safe and handles IPv6.
7. **Write IPv6-capable code:** `AF_INET6` with `struct sockaddr_storage`, or dual-stack sockets with `IPV6_V6ONLY` set to 0.

UDP

UDP uses `sendto`/`recvfrom` (or `connect` a UDP socket to set a default peer, and then use `send`/`recv`). Datagrams preserve message boundaries, but they are unreliable, unordered, and may be duplicated. Keep them under the path MTU — about 1,400 bytes is safe — to avoid IP fragmentation, which greatly increases the chance of loss.

24.7 I/O Multiplexing

A server with thousands of connections cannot dedicate a blocking thread to each one cheaply. *Multiplexing* lets a single thread wait on many descriptors at once and handle whichever become ready.

A working `epoll` echo server

```c
int ep = epoll_create1(EPOLL_CLOEXEC);
struct epoll_event ev = { .events = EPOLLIN, .data.fd = lfd };
epoll_ctl(ep, EPOLL_CTL_ADD, lfd, &ev);

for (;;) {
    int n = epoll_wait(ep, evs, 16, timeout_ms);
    if (n < 0) { if (errno == EINTR) continue; break; }
    for (int i = 0; i < n; i++) {
        int fd = evs[i].data.fd;
        if (fd == lfd) {
            for (;;) {                                          /* accept ALL pending */
                int c = accept4(lfd, NULL, NULL, SOCK_NONBLOCK|SOCK_CLOEXEC);
                if (c < 0) { if (errno==EAGAIN) break; else break; }
                struct epoll_event ce = { .events = EPOLLIN|EPOLLET, .data.fd = c };
                epoll_ctl(ep, EPOLL_CTL_ADD, c, &ce);
            }
        } else {
            ssize_t r;
            while ((r = read(fd, b, sizeof b)) > 0) {           /* EDGE-TRIGGERED: drain fully */
                ssize_t off = 0;
                while (off < r) { ssize_t w = write(fd, b+off, r-off); if (w<=0) break; off += w; }
            }
            if (r == 0 || (r < 0 && errno != EAGAIN)) { epoll_ctl(ep,EPOLL_CTL_DEL,fd,NULL);
close(fd); }
        }
    }
}
```

```
echo server listening on 127.0.0.1:33123
  server echoing 19 bytes
  server echoing 19 bytes
  server echoing 19 bytes
server done
```

The *epoll* instance holds the set of descriptors of interest inside the kernel. `epoll_wait` sleeps until at least one is ready and returns just the ready ones. When the listening socket is ready, the server accepts every pending connection and registers each one; when a client socket is ready, it reads everything available and echoes it back, closing the connection when the client disconnects or an error occurs.

Two simplifications keep the example short. The `accept` error branch breaks in both cases — a real server would log errors other than `EAGAIN`. And the echo loop gives up on a partial write: with non-blocking sockets, `write` can fail with `EAGAIN` when the client reads slowly, and a real server must keep the unwritten data in a per-connection buffer and register for `EPOLLOUT` to finish later (practice exercise 7).

Comparison

API	Complexity	Portability	Notes
`select`	O(n) per call	POSIX, everywhere	⚠ `FD_SETSIZE` is 1024 — descriptors numbered above that write out of bounds (typically corrupting the stack). Rebuilds the set on every call.

API	Complexity	Portability	Notes
`poll`	O(n) per call	POSIX	No descriptor limit. Still rescans everything on each call.
`epoll`	O(1)	Linux	The kernel keeps the set. The Linux standard.
`kqueue`	O(1)	BSD, macOS	More general — also files, signals, timers, and processes
IOCP	O(1)	Windows	Completion-based rather than readiness-based (§24.11)
`io_uring`	O(1), batched	Linux 5.1+	Shared submission/completion ring queues; nearly no system calls. Truly asynchronous — works for regular file I/O, which `epoll` cannot.

`select` and `poll` hand the whole list of descriptors to the kernel on every call and get the whole list back, so their cost grows with the number of connections even when only one is active. `epoll` and `kqueue` register descriptors once and report only the ready ones.

Level-triggered vs edge-triggered

	Level-triggered (default)	Edge-triggered (`EPOLLET`)
Reports	Whenever data is available	Only when the state changes
You must	Nothing special	Drain until `EAGAIN`
Risk	More wakeups	Missed data if you don't drain

With `EPOLLET`, `epoll` reports a descriptor only when *new* data arrives. If you read just part of what is available and then wait again, no new event comes for the data still sitting there — and the connection hangs forever. So you must loop reading until `EAGAIN`, and loop accepting until `EAGAIN`. Forgetting to do so is the single most common `epoll` bug.

`EPOLLONESHOT` is essential for thread pools: the descriptor is disabled after one event, so that two worker threads cannot process the same connection concurrently; the worker re-arms it when done.

Portable abstractions: libuv (which underlies Node.js), libevent, and libev. Use one of them unless you need the last few percent of performance.

24.8 pthreads

POSIX threads (pthreads) are the thread API used by most real C code on Unix; Module 17's C11 threads map almost one-to-one onto them.

A production thread pool

```c
typedef struct Task { void (*fn)(void*); void *arg; struct Task *next; } Task;
typedef struct {
    pthread_t *threads; size_t nthreads;
    Task *head, *tail;
    pthread_mutex_t mtx;
    pthread_cond_t work_ready, all_idle;
    size_t working; bool stop;
} Pool;

static void *worker(void *arg)
{
    Pool *p = arg;
    for (;;) {
        pthread_mutex_lock(&p->mtx);
        while (!p->head && !p->stop)
            pthread_cond_wait(&p->work_ready, &p->mtx);   /* WHILE, not if */
        if (p->stop && !p->head) { pthread_mutex_unlock(&p->mtx); break; }
        Task *t = p->head; p->head = t->next; if (!p->head) p->tail = NULL;
        p->working++;
        pthread_mutex_unlock(&p->mtx);

        t->fn(t->arg); free(t);                          /* run OUTSIDE the lock */

        pthread_mutex_lock(&p->mtx);
        if (--p->working == 0 && !p->head) pthread_cond_broadcast(&p->all_idle);
        pthread_mutex_unlock(&p->mtx);
    }
    return NULL;
}
```

```
thread pool: 4 workers, 200 tasks
  all tasks complete, total = 59999000
  clean shutdown, all threads joined
--- TSan ---
  warnings: 0
```

A *thread pool* keeps a fixed set of worker threads alive and hands them tasks through a shared queue, avoiding the cost of creating a thread per task. Three design points: the predicate is checked in a `while` loop (Module 17); the task runs *outside* the lock (otherwise only one task could ever run at a time, and you would have serialised everything); and shutdown is a flag + broadcast + join — set `stop`, wake every worker, and wait for each to finish the remaining work and exit. The `all_idle` condition lets the owner wait until every queued task has completed.

The API

Category	Functions
Threads	`pthread_create`, `pthread_join`, `pthread_detach`, `pthread_exit`, `pthread_self`, `pthread_equal`
Mutexes	`pthread_mutex_init/lock/trylock/timedlock/unlock/destroy`; types `NORMAL`, `ERRORCHECK` (catches double-locking), `RECURSIVE`, and (GNU) `ADAPTIVE`
Condition variables	`pthread_cond_init/wait/timedwait/signal/broadcast/destroy`
RW locks	`pthread_rwlock_rdlock/wrlock/unlock` — many readers, or one writer

Category	Functions
Barriers	`pthread_barrier_wait` — all *N* threads wait until every one has arrived
Spinlocks	`pthread_spin_lock` — only for very short critical sections on multicore machines
Once	`pthread_once` — thread-safe one-time initialisation
TLS	`pthread_key_create` + `pthread_setspecific` (with a destructor), or simply `_Thread_local`
Attributes	`pthread_attr_setstacksize`, `setdetachstate`, `setschedpolicy`

> ⚠ The default stack size of non-main threads varies from about 512 KB to 8 MB depending on the platform (Module 10). Set it explicitly with `pthread_attr_setstacksize` if your threads recurse deeply or use large local arrays.

> ⚠ `pthread_cancel` is nearly impossible to use correctly: a thread can be cancelled at any *cancellation point* (many system calls), and every resource it holds must be released by cleanup handlers. Use a flag or a pipe to tell threads to stop instead.

Deadlock avoidance

1. **A global lock ordering** — always lock A before B, everywhere. Document it.
2. **Never call user callbacks while holding a lock** — they might call back into you and try to take the same lock.
3. **`pthread_mutex_trylock` plus back-off** where no fixed ordering is possible: if the second lock is unavailable, release the first and retry.
4. **`PTHREAD_MUTEX_ERRORCHECK` in debug builds** catches a thread locking a mutex it already holds.
5. **TSan detects lock-order inversions,** not just data races. Run it.

Lock-free programming and the ABA problem

Module 17, §17.5 covers this. The summary: compare-and-swap loops are the foundation, using `atomic_compare_exchange_weak` inside a loop and `_strong` outside one, and the ABA problem (a value changing from A to B and back to A between your read and your CAS) requires tagged pointers, hazard pointers, or epoch-based reclamation.

Memory barriers in the POSIX world: `__atomic_thread_fence` (GCC), `atomic_thread_fence` (C11), or the implicit barriers in every mutex lock and unlock. Prefer C11 atomics with explicit memory orders to hand-written barriers.

> Use a mutex. An uncontended lock/unlock costs about 20 ns. Lock-free algorithms are for measured hot paths, written by people who have read the papers.

24.9 Daemons, Limits, Options, Environment

Daemonising

A *daemon* is a long-running background service with no controlling terminal. The traditional recipe:

```c
/* classic double-fork */
if (fork() > 0) _exit(0);   /* 1. parent exits → child is orphaned */
setsid();                   /* 2. new session, no controlling terminal */
if (fork() > 0) _exit(0);   /* 3. second fork → can never reacquire a TTY */
chdir("/");                 /* 4. don't hold a mount busy */
umask(0);                   /* 5. explicit file modes */
/* 6. close/redirect fds 0,1,2 to /dev/null */
```

Step by step: the first `fork` lets the launching shell regain control and puts the daemon in the background; `setsid` detaches it from the terminal; the second `fork` ensures that the process is not a *session leader*, and so can never acquire a controlling terminal again by accidentally opening one; changing to `/` ensures the daemon doesn't keep some file system busy and un-unmountable; resetting `umask` makes file permissions predictable; and redirecting the standard descriptors avoids writing to a terminal that no longer exists.

Under systemd, don't do any of this. Run in the foreground (`Type=simple` or `Type=notify`), log to `stdout`/`stderr`, and let systemd handle supervision, restarts, socket activation, and log capture. The double-fork dance is legacy.

Related: `sd_notify` to report readiness, `LISTEN_FDS` for socket activation (systemd opens the listening socket and passes it to your process — `SCM_RIGHTS`-style inheritance), and a PID file only if something genuinely requires one.

Resource limits

```
NOFILE (open fds)   soft=1024      hard=finite
STACK               soft=8388608   hard=unlimited
AS (address space)  soft=unlimited hard=unlimited
NPROC               soft=15955     hard=finite
CORE                soft=0         hard=unlimited
```

Each limit has a *soft* value, which is enforced, and a *hard* ceiling, up to which an unprivileged process may raise its own soft limit (with `setrlimit`).

`RLIMIT_NOFILE` defaults to 1,024 — so a server handling more than about a thousand connections must raise it, with `setrlimit` or `LimitNOFILE=` in a systemd unit. This is the classic "my server stops accepting new connections at around 1,000 clients" bug: `accept` starts failing with `EMFILE`.

`RLIMIT_CORE = 0` means no core dumps (Module 21) — raise it in development.

Also useful for testing: `RLIMIT_AS` to force allocation failures (Modules 10 and 15), and `RLIMIT_CPU` and `RLIMIT_NPROC` as sandboxing measures.

`getopt_long`

```
static struct option opts[] = {
    {"verbose", no_argument,       0, 'v'},
    {"output",  required_argument, 0, 'o'},
    {"level",   optional_argument, 0, 'l'},
    {0,0,0,0}
};
while ((c = getopt_long(argc, argv, "vo:l::", opts, &idx)) != -1) { ... }
/* remaining positional args start at argv[optind] */
```

```
$ ./misc -vv --output=/tmp/x --level=3 extra1 extra2
  --level=3
  verbose=2 output=/tmp/x remaining args from argv[4]
```

`getopt_long` parses command-line options, both short (`-v`, with repeats and grouping like `-vv`) and long (`--output=/tmp/x`). In the short-option string, a letter followed by `:` requires an argument, and `::` makes the argument optional (a GNU extension). A leading `+` stops parsing at the first non-option argument; a leading `:` makes a missing argument return `':'` instead of printing an error. After the loop, `optind` indexes the first remaining positional argument.

Environment

```
getenv("HOME");
setenv("VAR", "value", /*overwrite=*/1);
unsetenv("VAR");
extern char **environ;   /* enumerate everything */
```

> ⚠ `getenv` returns data an attacker may control. In setuid programs, sanitise or clear `PATH`, `LD_PRELOAD`, `LD_LIBRARY_PATH`, `IFS`, and the locale variables (Module 23). `setenv` is not thread-safe against concurrent `getenv`: set the environment up at start-up, before creating threads.

24.10 Dynamic Loading and Plugins

Programs can load shared libraries at run time, by name, and look up functions in them — the mechanism behind plugins, optional features, and language runtimes.

```
void *h = dlopen("./plugin.so", RTLD_NOW | RTLD_LOCAL);
if (!h) { fprintf(stderr, "dlopen: %s\n", dlerror()); return -1; }

dlerror();                          /* clear stale errors first */
union { void *o; ApplyFn f; } u;    /* ISO C doesn't bless void*↔fnptr */
u.o = dlsym(h, "plugin_apply");
const char *err = dlerror();
if (err) { ... }
```

```
loaded plugin "uppercase" v3
  before: hello plugin world
  after : HELLO PLUGIN WORLD
  dladdr: plugin_version from ./plugin.so
```

`dlopen` loads the library and returns a handle; `dlsym` looks up a symbol by name and returns its address; `dlerror` returns a description of the most recent error (and clears it); and `dlclose` unloads the library.

Flag	Meaning
`RTLD_NOW`	Resolve every symbol at load time — fail fast
`RTLD_LAZY`	Resolve each function on its first call — faster loading, later failures
`RTLD_LOCAL`	The library's symbols are not visible to later `dlopen`s (the default, and safer)
`RTLD_GLOBAL`	The library's symbols are available to subsequently loaded libraries

Design rules for a plugin ABI

1. **Export one structure of function pointers, not many symbols.** It is versionable, and it takes a single `dlsym` — the vtable pattern from Module 6.
2. **Include a version field, and check it on load.** A plugin built against an older interface must be rejected, not called with the wrong structure layout (Module 20's ABI break).
3. **Build plugins with `-fvisibility=hidden`** (Module 20), so that only the entry point is exported.
4. **Clear `dlerror()` before `dlsym`** — a valid symbol can legitimately have the value `NULL`, so a `NULL` return alone does not prove an error.
5. **Never `dlclose` while a callback from the library may still run,** or while any of its data is still reachable — the code and data vanish on unload.
6. **`dladdr` maps an address back to a symbol and library** — useful for backtraces and diagnostics.
7. **The `void *` ↔ function-pointer conversion is not ISO C** (Module 9). POSIX requires it to work, and the union shown above is the conventional workaround that keeps the compiler quiet.

24.11 Windows Counterparts

POSIX	Win32
fd (`int`)	`HANDLE` (opaque) — not an `int`, and not a small integer
`open/read/write/close`	`CreateFile/ReadFile/WriteFile/CloseHandle`
`fork` + `exec`	`CreateProcess` (there is no `fork` equivalent)
`pthread_create`	`CreateThread` / `_beginthreadex`
`pthread_mutex_t`	`CRITICAL_SECTION` (in-process) / Mutex (cross-process)
`pthread_cond_t`	`CONDITION_VARIABLE`

POSIX	Win32
`epoll/kqueue`	IOCP (I/O Completion Ports) — completion-based, not readiness-based
`mmap`	`CreateFileMapping + MapViewOfFile`
signals	Structured Exception Handling, `SetConsoleCtrlHandler`
`dlopen/dlsym`	`LoadLibrary/GetProcAddress`
`errno`	`GetLastError()`
pipes	Named pipes (much richer — bidirectional and network-capable)

The deep difference: POSIX I/O multiplexing is *readiness-based* ("tell me when I can read, and then I'll read"), while Windows IOCP is *completion-based* ("perform this read, and tell me when it is done"). `io_uring` brings the completion model to Linux.

For portability: use libuv (the best cross-platform abstraction), or write an internal platform layer with `#ifdef _WIN32`. Don't scatter `#ifdef`s through your program's logic — isolate them in one file per subsystem (Module 12's advice on conditional compilation).

Module 24 Rules

#	Rule
1	A system call costs ~100× a function call — buffer and batch
2	`fflush` before `fork`, or your output duplicates
3	`exec` returning means it failed — always `_exit` after it
4	Reap children with `while (waitpid(-1, NULL, WNOHANG) > 0)` — signals aren't queued
5	`sigaction`, never `signal`
6	In a handler: set a `volatile sig_atomic_t`, `write()`, `_exit()`. Nothing else.
7	Self-pipe or `signalfd` to unify signals with your event loop
8	Never put pointers in shared memory — use offsets
9	Use the `*at` family for race-free path operations
10	Every multi-byte network value goes through `htons/htonl`
11	TCP is a byte stream — frame your own messages
12	`SO_REUSEADDR` on every server; ignore `SIGPIPE`; loop on short reads/writes and `EINTR`

#	Rule
13	`EPOLLET` requires draining until `EAGAIN` — for both `read` and `accept`
14	`RLIMIT_NOFILE` defaults to 1024 — raise it for servers
15	Thread pools: `while` on the predicate, run tasks outside the lock, join on shutdown
16	Global lock ordering; never call callbacks while holding a lock; run TSan
17	Under systemd, don't daemonise — run in the foreground
18	Plugins export one versioned structure of function pointers
19	`O_CLOEXEC`/`SOCK_CLOEXEC` on everything

Practice

1. **System-call cost.** Measure `getpid`, `clock_gettime`, a 1-byte `read`, and a 1-byte `write` in a loop. Then measure a 4,096-byte `write`. Compute the per-call and per-byte costs, and explain why buffering exists.

2. **Process supervisor.** Write a program that forks *N* children running a given command, restarts any that exit, reaps them with a `SIGCHLD` handler, and shuts them all down cleanly on `SIGTERM` using a process group. Verify with `ps` that no zombies remain.

3. **Signal-safe logging.** Write a signal handler that logs the signal number, the faulting address (via `SA_SIGINFO`), and a backtrace — using only async-signal-safe functions. Test it on `SIGSEGV` with `SA_ONSTACK` and an alternate stack, so that it survives a stack overflow.

4. **Three signal patterns.** Implement graceful shutdown three ways: flag + loop, self-pipe + `poll`, and `signalfd` + `epoll`. Compare the code and the race characteristics of each.

5. **IPC benchmark.** Send 1 million 64-byte messages between two processes using a pipe, a UNIX socketpair, a POSIX message queue, and shared memory with a semaphore. Measure the throughput and latency of each.

6. **Descriptor passing.** Write a privileged parent that opens files and passes the descriptors to an unprivileged child over a UNIX socket with `SCM_RIGHTS`. This is the OpenSSH privilege-separation model.

7. **Complete TCP server.** Build an `epoll`-based server with: length-prefixed message framing, per-connection state, edge-triggered I/O with proper draining, write buffering for slow clients, idle timeouts, graceful shutdown via `signalfd`, and `SO_REUSEADDR`. Load-test it with 10,000 concurrent connections (raise `RLIMIT_NOFILE` first).

8. **`select` vs `poll` vs `epoll`.** Implement the same server three ways. Benchmark it with 10, 100, 1,000, and 10,000 idle connections plus 10 active ones. Plot the per-event cost and explain the O(n)-versus-O(1) difference.

9. **The edge-triggered bug.** Write an `EPOLLET` handler that reads only once per event. Send a large message and observe the hang. Fix it with the drain loop. Do the same for `accept`.

10. **Thread-pool extension.** Take the §24.8 pool and add: a bounded queue with backpressure, per-task futures (a condition variable plus a result slot), work stealing, and graceful drain-then-stop. Verify with TSan and run 1 million tasks.

11. **Deadlock and detection.** Write two threads that lock A→B and B→A. Confirm that it deadlocks. Confirm that TSan reports the lock-order inversion. Fix it with a global ordering, then with `trylock` plus back-off.

12. **Plugin system.** Define a plugin ABI as a versioned structure of function pointers. Write a loader that scans a directory, `dlopen`s each `.so`, validates the version, and registers it. Write three plugins. Handle load failure, version mismatch, and safe unloading.

13. **File watcher.** Build a recursive directory watcher with `inotify` integrated into an `epoll` loop, handling `IN_CREATE` on directories by adding new watches, and `IN_Q_OVERFLOW` by rescanning.

Module 24 in Summary

A system call costs about a hundred function calls, which is why C libraries buffer, the vDSO answers clock queries without entering the kernel, and good systems code batches its I/O. Processes are created with `fork` — flushing stdio first — and replaced with `exec`, which returns only on failure; parents must reap their children, in a loop inside a `SIGCHLD` handler, or they accumulate zombies. Process groups and sessions determine who receives terminal signals.

Install handlers with `sigaction`, and do almost nothing inside them: set a `volatile sig_atomic_t` flag, write to a self-pipe, or avoid handlers entirely with `signalfd`. Standard signals are not queued, blocked signals are merely pending, and every server must deal with `SIGPIPE`, `SIGTERM`, `SIGCHLD`, and `SIGHUP`. Processes communicate through pipes, socketpairs, UNIX sockets (which can even pass open descriptors), message queues, and shared memory — which needs its own synchronisation and must hold offsets, never pointers. The file-system API offers `stat`, directory iteration, the race-free `*at` family, and `inotify`.

TCP servers bind, listen, and accept; every multi-byte value is converted to network byte order, messages must be framed on top of the byte stream, short reads and writes and `EINTR` must be handled, and `SIGPIPE` ignored. `epoll`, `kqueue`, and `io_uring` let one thread serve thousands of connections — with edge-triggered mode demanding a drain until `EAGAIN`. pthreads provide the thread pools, locks, and condition variables of real servers, used with a consistent lock order and verified by TSan. Modern daemons run in the foreground under systemd, raise `RLIMIT_NOFILE`, parse options with `getopt_long`, and treat the environment as untrusted. Plugins are loaded with `dlopen` and expose a single versioned structure of function pointers — and Windows provides counterparts to all of this, built around handles and completion-based I/O.

Next: Module 25 — Performance Engineering, where the cache-layout effects you measured in Module 11 get a systematic treatment: the memory hierarchy, branch prediction, profiling methodology, SIMD, and — most importantly — how to know whether an optimisation actually helped.

Module 25 — Performance Engineering

In this module:

- Big-O in practice vs constants and memory
- CPU caches, cache lines, locality, prefetching, false sharing
- Branch prediction, branchless techniques
- Data-oriented design, SoA vs AoS
- Profiling: `perf`, gprof, callgrind, VTune, flame graphs
- Benchmarking methodology (warm-up, variance, avoiding compiler elision)
- SIMD: SSE/AVX/NEON intrinsics, auto-vectorisation
- Inline assembly, compiler barriers
- `restrict`, `likely`/`unlikely`, alignment, memory pools
- Understanding what the optimiser does and doesn't do

Every number in this module was measured on one machine: an Intel Xeon at 2.10 GHz, 1 vCPU, with a 48 KB L1 data cache, 2 MB L2, 260 MB L3, and 64-byte cache lines. Your numbers will differ; the *shapes* won't.

C is chosen for performance-critical work because it lets you control exactly what the machine does. But control is not the same as speed. A C program can easily be slower than the same algorithm in a managed language, if it walks memory in the wrong order or branches unpredictably. This module is about the factors that actually determine speed on modern hardware — memory, branches, and parallel arithmetic — and, just as importantly, about measuring correctly, so that you know whether a change helped.

25.1 Big-O vs Constants and Memory

```
BIG-O: linear O(n) vs binary O(log n) search
   n      linear      binary
   4      0.56 ms     0.47 ms
  64      5.05 ms     1.01 ms
4096    197.16 ms    10.16 ms
```

Asymptotic complexity won here — but that is not the general lesson. The general lesson is that asymptotic complexity is a statement about the *limit* as n grows without bound, and real code often lives well below that limit. Big-O deliberately ignores constant factors, and in C those constant factors are dominated by memory access patterns.

Cases where the "worse" algorithm wins in practice:

Situation	Why
Small n (below ~64)	A linear scan of a contiguous array beats a tree: no pointer chasing, perfect prefetching, no unpredictable branches
Sorted-array binary search vs a hash table	The array has better locality; hashing costs a computation plus a random access

Situation	Why
Insertion sort inside `qsort`	Practically every production sort switches to insertion sort below about 16 elements
Array vs linked list	An O(n) array scan often beats an O(1) list insertion, because *finding* the insertion point in the list is a pointer chase with unpredictable latency

A cache miss costs about 100 times as much as an L1 cache hit (§25.2). For small n, that dwarfs the difference between n and $\log n$.

> **Know your *n*.** Optimising for $n = 10^9$ when production n is 20 is a common and expensive mistake.

25.2 The Memory Hierarchy ⚠

Modern processors are far faster than main memory, so they keep recently used data in a hierarchy of small, fast *caches*: L1 (tiny and very fast, private to each core), L2 (larger and slower), and L3 (larger still, often shared between cores), with main memory (RAM) behind them all.

Measured latencies (random pointer chase — pure latency, prefetcher defeated)

```
WORKING SET   ns/access
   8 KB         1.61   ← L1d (48 KB)
  32 KB         1.63   ← L1d
 128 KB         3.94   ← L2
 512 KB         5.54   ← L2 (2 MB)
   2 MB        22.13   ← L2/L3 boundary
   8 MB        57.03   ← L3
  32 MB        66.46   ← L3 (260 MB)
 128 MB       148.81   ← RAM
```

The benchmark follows a chain of pointers arranged in random order through a buffer of the given size, so each load depends on the previous one and cannot be predicted — it measures pure latency. From about 1.6 ns to about 149 ns is a 92× range. Every plateau is a cache level; every jump is where the working set stopped fitting in one.

At 2.1 GHz, one cycle is 0.48 ns. So:

Level	Latency	Cycles
Register	—	0
L1	1.6 ns	~3
L2	5.5 ns	~12
L3	60 ns	~125
RAM	149 ns	~310

A main-memory access costs about 310 cycles. In that time, a modern core could have executed on the order of a thousand instructions. That is why data layout beats clever algorithms in most real optimisation work.

Cache lines

```
CACHE LINE EFFECT: touch one byte every STRIDE bytes (same access count)
stride   time
   1     0.003 ms (multiple accesses share a line)
  32     0.003 ms
  64     0.003 ms <- one access per 64-byte cache LINE
 128     0.005 ms
 256     0.027 ms
 512     0.084 ms ← 28× slower than stride 1
```

Memory moves between RAM and the caches in 64-byte *lines*, not individual bytes. Below a stride of 64, several accesses share each line, so most of them are nearly free. At a stride of 64 or more, every access needs a line of its own. The cost keeps climbing beyond 64 because larger strides also defeat the hardware prefetcher and exhaust the *TLB* (the translation lookaside buffer, which caches the mappings from virtual to physical pages).

Practical consequences:

- **Iterate in memory order.** Row-major for 2D arrays (Module 7, §7.2) — a 5–10× effect.
- **Pack hot data together.** Module 11's field reordering halved a structure's size, and gave a 7.17× speedup on a key-scanning loop.
- **A 64-byte structure means one object per cache line. A 16-byte structure means four.**

Prefetching

The hardware prefetcher detects sequential and constant-stride access patterns and fetches data before it is requested. It *cannot* follow pointer chains or random indices — which is exactly why the pointer-chase benchmark above measures true latency.

```
__builtin_prefetch(&data[i + 16], 0, 3);   /* 0=read, 3=high temporal locality */
```

Software prefetching rarely helps, unless your access pattern is irregular but known some steps in advance — hash-table probing, or tree traversal where you know the next node early. Measure: it usually does nothing, or hurts.

False sharing

```
addresses (shared): 0x...180 0x...188 0x...190 0x...198   ← all within 64 bytes
addresses (padded): 0x...080 0x...0c0 0x...100 0x...140   ← 64 bytes apart
```

Caches keep each line consistent across cores (*cache coherence*): before a core can write to a line, every other core's copy must be invalidated. So when two threads write to *different* variables that happen to share a cache line, the line bounces between their cores on every write. No data is logically shared — but the cache line is.

> ⚠ The measurement machine has 1 vCPU, so this effect could not be measured honestly here. On a real multicore machine, the difference is typically 3–10×.

The fix is Module 11's:

```
struct Counter { alignas(64) long v; };   /* each counter gets its own cache line */
struct Counter counters[NTHREADS];
```

(As noted in Module 11, the C++ spelling `struct alignas(64) Counter` is not valid C; in C the alignment goes on the member.) Detect false sharing with `perf c2c` ("cache-to-cache") on Linux.

25.3 Branch Prediction

A modern CPU does not wait to find out which way a conditional branch goes: it *predicts* the outcome and speculatively executes ahead. A correct prediction is almost free. A wrong one is expensive.

The classic experiment — and a lesson about modern compilers

```
for (int i = 0; i < N; i++) if (data[i] >= 128) s += data[i];
```

```
branchy, RANDOM data  : 28.8 ms
branchy, SORTED data  : 28.9 ms  (1.00x — no difference?!)
BRANCHLESS, random    : 18.7 ms  (1.54x faster)
```

The famous result is that sorting the data first makes this loop much faster, because the branch becomes predictable (all "no" for a while, then all "yes"). But here, sorting made no difference. Why? Look at the assembly:

```
sum_ge:
    ...
    cmovg rdx, rax   ← conditional MOVE, not a branch
```

GCC had already converted the branch into a `cmov` — a conditional-move instruction, which selects between two values without branching — at `-O2`. There was no branch left to mispredict. (Module 19's lesson: read the assembly before drawing conclusions.)

Force a real branch, with a `noinline` function call that the compiler cannot flatten:

```
branchy w/ real call, RANDOM : 281.5 ms
branchy w/ real call, SORTED :  30.9 ms  (9.12x faster)
```

9.12× — from sorting the input. Identical code, identical arithmetic, identical memory access pattern. The only difference is whether the branch is predictable.

Why it costs so much

A modern CPU is a deep pipeline of roughly 15–20 stages, executing speculatively far ahead of the instruction that is actually completing. A misprediction means everything executed down the wrong path must be discarded — roughly 15–20 cycles wasted, every time. With random data, a 50% misprediction rate on a hot loop dominates everything else.

Modern predictors (such as TAGE-style predictors) are extremely good at repeating patterns, loop counts, and branches correlated with earlier branches. They are helpless against genuinely random data.

Branchless techniques

```c
/* conditional add via a mask */
int m = -(v >= 128);   /* 0 or -1 (all bits set) */
s += v & m;

/* branchless min/max/abs */
int min = b ^ ((a ^ b) & -(a < b));
int mask = v >> 31; int abs = (v + mask) ^ mask;

/* branchless select */
result = (mask & a) | (~mask & b);
```

These all rely on the same idea: a comparison yields 0 or 1, and negating it yields 0 or −1 (all bits set, Module 2). ANDing with such a mask either keeps a value or zeroes it, with no branch. (The `abs` trick relies on `v >> 31` being an arithmetic shift of a negative `int` — implementation-defined, Module 4 — and fails for `INT_MIN`, like every `abs`.)

> ⚠ **Branchless is not automatically faster.** It always does the work of both paths. It wins when the branch is unpredictable, and loses when it is predictable — a well-predicted branch is nearly free, and skipping expensive work is a real saving. Measure.

Also: a `cmov` creates a *data dependency* where a correctly predicted branch does not, which can slow down a loop whose speed is limited by a chain of dependent operations. And branchless code is what you *must* write for constant-time cryptography (Module 23, §23.9), regardless of speed.

Branch hints

```c
#define likely(x)   __builtin_expect(!!(x), 1)
#define unlikely(x) __builtin_expect(!!(x), 0)
```

```
BRANCH HINTS (0.1% of elements take the rare path)
  plain            : 31.02 ms
  __builtin_expect : 21.00 ms (1.48x)
```

Hints work mainly through *code layout*, not through the hardware predictor: the hot path is laid out so that it falls straight through, the cold path is moved out of line, and so the instruction cache holds more useful instructions. (C++20 added `[[likely]]` and `[[unlikely]]` statement attributes for this; C23 did not adopt them, so in C `__builtin_expect` remains the way to express it.)

Profile-guided optimisation (PGO) does this automatically, and far better:

```
gcc -O2 -fprofile-generate prog.c -o prog && ./prog <typical workload>
gcc -O2 -fprofile-use prog.c -o prog
```

The first build is instrumented to record which branches are taken and which functions are hot while it runs a representative workload; the second build uses that profile to lay out code, inline, and unroll accordingly. PGO typically yields 5–20% with zero source changes. Use it before hand-annotating.

25.4 Data-Oriented Design

Module 11, §11.3 measured 7.17× from layout alone. Here is the systematic version:

```
/* AoS — natural OOP-ish layout */
struct Particle { float x,y,z, vx,vy,vz; float mass; int id; };
struct Particle p[N];

/* SoA — every field is a dense contiguous array */
struct Particles {
    float x[N], y[N], z[N];
    float vx[N], vy[N], vz[N];
    float mass[N]; int id[N];
};
```

	AoS wins	SoA wins
Access pattern	You touch most fields of one object	You touch one or two fields across many objects
Examples	Per-entity game logic, tree nodes, linked structures	Physics integration, filters, aggregations, column stores
Vectorisation	Hard (strided access)	Trivial — contiguous and aligned
Cache efficiency	Wastes most of each line if you read one field	Every byte loaded is used

Hybrid: AoSoA (array of structures of small arrays) — group 8 or 16 objects' worth of each field into SIMD-width blocks. You get SoA's contiguous, vectorisable fields and much of AoS's locality per object. It is common in physics and rendering engines.

The other layout techniques

- **Hot/cold splitting** — frequently accessed fields in one structure, the rest behind a pointer (Module 11).
- **Field reordering by decreasing alignment** — often halves the structure's size.
- **Indices instead of pointers** — 4-byte `uint32_t` indices instead of 8-byte pointers halve a linked node's size, and survive `realloc` (Module 9).
- **Arena and pool allocation** (Module 10) — sequential allocation means sequential memory. Measured there: 14× for an arena and 5× for a pool, compared with `malloc`.
- **Struct-of-arrays for hash tables** — separate arrays of keys, hashes, and values; probe the dense, cache-friendly hash array before touching any keys.

25.5 Benchmarking Methodology ⚠

Most published C benchmarks are wrong. Here are the three classic failures, measured.

Mistake 1 — dead-code elimination

```
result discarded :  0.000 ms <- the loop was DELETED
result consumed  : 14.525 ms <- real work
```

If nothing uses a computation's result, the optimiser deletes the computation (Module 16's "as-if" rule). The benchmark then measures an empty loop — or nothing at all. Fixes:

```c
volatile double sink; sink = result;              /* forces a store */
__asm__ volatile("" :: "r"(result) : "memory");   /* compiler barrier */
if (result == 0.0) printf("");                    /* opaque consumer */
```

The first forces the result to be written to a `volatile` variable, which the compiler must do (Module 2). The second tells the compiler that an (empty) assembly statement uses the value, so it must be computed. Google Benchmark's `benchmark::DoNotOptimize` is the packaged version of the same trick.

Mistake 2 — no warm-up

```
run 1 : 1.347 ms <- cold
run 10: 1.405 ms <- warm
```

(The difference is small here, because the array is only 16 MB and had just been written. On larger data, or with a cold page cache, the first run can be 10–100× slower.) The first run pays for page faults, cold caches, lazy binding of library functions (Module 20's PLT), and the CPU ramping up its clock frequency. Discard it.

Mistake 3 — reporting the mean

```
min=1.368 median=1.421 mean=1.426 max=1.592 ms
spread: 16.3% -> report MIN or MEDIAN, never the mean
```

The maximum is contamination — a scheduling decision, an interrupt, another process. It is not information about your code. Noise only ever makes a run *slower*, never faster, so it is one-sided: the mean is dragged upward by outliers. Report the minimum (the run least disturbed by noise) or the median.

The checklist

Do	Why
Consume every result	Defeat dead-code elimination
Warm up, then measure	Page faults, cold caches, frequency scaling
Repeat at least 9 times; report the min or median	Noise is one-sided

Do	Why
Report the spread too	If min and max differ by 3×, your number is meaningless
Use `CLOCK_MONOTONIC`	Module 14 — never `time()` or `CLOCK_REALTIME`
Pin the CPU (`taskset`) and disable turbo/ frequency scaling	Removes the largest source of noise
Benchmark the whole realistic workload	Microbenchmarks lie about cache and branch behaviour
Compare against a baseline in the same run	Absolute numbers aren't comparable across machines or days
Randomise the order of the variants	Removes systematic drift (thermal throttling, background load)

> ⚠ **Microbenchmarks are systematically optimistic.** In a microbenchmark, your data is in cache, the branch predictor is trained on your exact pattern, and the instruction cache holds only your loop. In production, none of that holds. Always validate with an end-to-end measurement.

25.6 Profiling

> **Never optimise without profiling.** Programmer intuition about where time goes is famously bad.

`perf` — the Linux profiler

```
perf stat ./prog                              # cycles, IPC, cache misses, branch misses
perf record -g ./prog && perf report          # sampling profile with call graphs
perf top                                      # live, system-wide
perf record -e cache-misses,LLC-load-misses ./prog
perf annotate                                 # per-instruction attribution
perf c2c record ./prog                        # false-sharing detection
perf stat -e task-clock,cycles,instructions,branch-misses,cache-misses ./prog
```

`perf stat` alone tells you the *shape* of your problem:

Signal	Diagnosis
IPC < 1.0	Memory-bound — fix the layout
IPC > 2.5	Compute-bound — consider SIMD
High cache misses	A layout or working-set problem
High branch misses	Unpredictable branches (§25.3)
High task-clock vs wall time	Multithreaded, or spinning

IPC is instructions per cycle. A modern core can retire four or more instructions per cycle when it has the data it needs; a low IPC means it spends most cycles waiting — usually for memory.

Flame graphs are the standard visualisation (Module 21):

```
perf record -F 99 -g ./prog
perf script | stackcollapse-perf.pl | flamegraph.pl > out.svg
```

Width is time spent; stacking shows call depth. One glance tells you where the time is.

(`perf` requires suitable `perf_event_paranoid` permissions and access to hardware counters — often unavailable in containers, as on the machine used for this book.)

callgrind — deterministic and reproducible

```
Ir                  file:function
5,601,209 (97.74%)  prof.c:main
```

callgrind (Module 21) counts every instruction executed rather than sampling, so its results are exact and reproducible — which means it can serve as a performance regression test in CI, where noise would make timing measurements useless. It costs roughly 50× the run time. Visualise its output with KCachegrind; cachegrind adds simulation of the caches and branch predictor.

The rest

Tool	Notes
gprof	`-pg`, instrumentation-based. Legacy: high overhead, distorts inlining, misses shared libraries.
Intel VTune	Best-in-class on Intel CPUs: microarchitecture analysis, memory-access analysis, threading. Free.
AMD μProf	The AMD equivalent
bpftrace	Production tracing with near-zero overhead (Module 21)
massif / heaptrack	Memory profiling over time
Tracy, Perfetto	Frame-based tracing for games and real-time systems

Method

1. **Measure end-to-end first.** Know the total before you optimise a part.
2. **Profile to find the hot spots.** Usually about three functions account for 80% of the time.
3. **Understand *why* they are hot** — `perf stat` for IPC and miss rates, then `perf annotate`.
4. **Fix the biggest one.** Algorithm → layout → micro-optimisation, in that order.

5. **Re-measure end-to-end.** *Amdahl's law*: the overall speedup is limited by the fraction of time the improved part accounts for. Making a function that takes 5% of the run time infinitely fast gains 5%.
6. **Keep the benchmark in CI,** so that regressions are caught.

25.7 SIMD

SIMD — "single instruction, multiple data" — instructions operate on several values at once, packed into wide registers: one AVX2 instruction adds eight `float`s.

```
DOT PRODUCT, 4194304 floats (32 MB touched)
  scalar (-O2)          : 3.06 ms
  restrict (-O2)        : 3.03 ms (1.01x)
  AVX2 intrinsics + FMA : 1.34 ms (2.28x)
```

And auto-vectorisation — the compiler generating SIMD code by itself:

```
-O2                          3.15 ms
-O3                          3.09 ms
-O3 -march=native            3.13 ms
-O3 -march=native -ffast-math 1.36 ms   ← 2.3×, matching hand-written
```

Why `-ffast-math` was required

Floating-point addition is not associative (Module 2): `(a + b) + c` can differ from `a + (b + c)` in the last bits. A vectorised reduction computes eight separate partial sums — one per lane — and combines them at the end. That changes the order of the additions, and therefore the result:

```
values agree: scalar=75122080 avx=75497432
```

They do not agree exactly. That 0.5% difference is real reassociation error — accumulated rounding in different orders. (Neither answer is "the right one"; each has its own rounding error, and for long sums the multi-accumulator version is often *more* accurate.) The compiler therefore refuses to vectorise a floating-point reduction unless you explicitly permit reassociation — with `-ffast-math`, with the narrower `-fassociative-math` (combined with `-fno-signed-zeros -fno-trapping-math`), or per loop with `#pragma omp simd reduction(+:s)` (enabled by `-fopenmp-simd`).

For integer reductions, no such permission is needed: integer addition is associative (with wrapping unsigned arithmetic, or in the absence of overflow).

The intrinsics

```c
#include <immintrin.h>

__m256 acc = _mm256_setzero_ps();
int i = 0;
for (; i + 8 <= n; i += 8) {
    __m256 vx = _mm256_loadu_ps(x + i);
    __m256 vy = _mm256_loadu_ps(y + i);
    acc = _mm256_fmadd_ps(vx, vy, acc);   /* fused multiply-add */
```

```
}
float tmp[8]; _mm256_storeu_ps(tmp, acc);
float s = tmp[0]+tmp[1]+...+tmp[7];       /* horizontal reduction */
for (; i < n; i++) s += x[i] * y[i];      /* scalar tail */
```

Intrinsics are functions that map directly onto individual SIMD instructions. `__m256` is a 256-bit register holding eight `float`s; `_mm256_loadu_ps` loads eight consecutive floats (the `u` means the address need not be aligned); `_mm256_fmadd_ps` multiplies and adds in one instruction, with a single rounding. After the main loop, the eight lanes are added together (the "horizontal reduction"), and a scalar loop handles the last `n % 8` elements. (`i` is declared before the vector loop so that the tail loop can continue from where it stopped. Compile with `-mavx2 -mfma` or a matching `-march`.)

ISA	Width	Header
SSE2	128-bit (4 floats)	`<emmintrin.h>` — the baseline on x86-64
AVX/AVX2	256-bit (8 floats)	`<immintrin.h>`
AVX-512	512-bit (16 floats)	`<immintrin.h>` — ⚠ can cause frequency throttling on some CPUs
NEON	128-bit	`<arm_neon.h>` — the baseline on ARM64
SVE	scalable	ARM; vector-length agnostic

Practical guidance

1. **Try auto-vectorisation first:** `-O3 -march=...` (plus `-ffast-math` where acceptable), checking with `-fopt-info-vec` and `-fopt-info-vec-missed` (Module 19).
2. **Help the vectoriser:** `restrict` (Module 2), aligned data, simple counted loops, no early exits, no function calls in the body, and contiguous access — SoA!
3. **Hand-write intrinsics only where it measurably matters.** They are unreadable and non-portable.
4. **Always keep a scalar fallback, and handle the tail.**
5. **Use run-time dispatch for distributed binaries:** `__builtin_cpu_supports("avx2")` plus `__attribute__((target("avx2")))` function multiversioning. Never ship `-march=native` (Module 19).
6. **Alignment matters** — use `aligned_alloc(64, ...)` and the aligned load/store variants where you can guarantee alignment.
7. **Consider a library first:** OpenBLAS, Eigen, simdjson, xsimd, Highway — they have already done the work.

25.8 The Optimiser: What It Does and Doesn't Do

What it does well

Constant folding and propagation, dead-code elimination, common-subexpression elimination, inlining, loop-invariant code motion (hoisting computations out of loops), strength reduction (`x*8` → `x<<3`), register allocation, instruction scheduling, tail-call optimisation (Module 6), auto-vectorisation, branch-to-`cmov` conversion (§25.3), and exploiting undefined behaviour (Module 16).

What it cannot do

Can't	Because
Change your algorithm	$O(n^2)$ stays $O(n^2)$
Change your data layout	AoS → SoA is a source change
Reorder across possible aliases	Hence `restrict`
Reassociate floats	Not associative — hence `-ffast-math`
Optimise across translation units	Unless you enable LTO (Module 19)
Optimise through indirect calls	Function pointers, `dlsym`, virtual dispatch
Remove `volatile` accesses	By definition (Module 2)
Know your run-time data	Hence PGO
Fix cache misses	It can't move your memory

The hints that actually help

```c
/* restrict — no aliasing (Module 2). Measured: enables vectorization. */
void axpy(float *restrict y, const float *restrict x, float a, int n);

/* alignment */
float *p = aligned_alloc(64, n * sizeof *p);
p = __builtin_assume_aligned(p, 64);

/* branch layout */
if (unlikely(error)) { ... }

/* purity — enables CSE and reordering */
__attribute__((const)) int f(int x);        /* no memory access at all */
__attribute__((pure))  int g(const T *t);   /* reads memory, no side effects */

/* force/prevent inlining after measuring */
__attribute__((always_inline)) / __attribute__((noinline))

/* compiler barrier — prevents reordering across this point, emits nothing */
__asm__ __volatile__("" ::: "memory");
```

Note that `__builtin_assume_aligned` *returns* a pointer carrying the alignment promise; its result must be used (hence the assignment back to `p`). `const` and `pure` let the compiler call a function once instead of repeatedly with the same arguments — a promise which, if false, produces wrong code.

Inline assembly

```
uint64_t rdtsc(void) {
    uint32_t lo, hi;
    __asm__ __volatile__("rdtsc" : "=a"(lo), "=d"(hi));
    return ((uint64_t)hi << 32) | lo;
}
```

This reads the processor's time-stamp counter: the `rdtsc` instruction puts the counter's low 32 bits in `eax` and high 32 bits in `edx`, and the *output constraints* `"=a"` and `"=d"` tell the compiler to take `lo` and `hi` from those registers.

Inline assembly is rarely justified. Intrinsics cover almost everything, are readable, and let the optimiser schedule around them — whereas an inline-assembly block is opaque to it. Reserve inline assembly for instructions with no intrinsic (`rdtsc`, `cpuid`, privileged instructions), atomic sequences the compiler will not generate, and bare-metal start-up code (Module 27). (For `rdtsc` specifically, `__rdtsc()` from `<x86intrin.h>` is an intrinsic.)

If you must use it: get the constraints exactly right, list every clobbered register, including `"memory"` and `"cc"` (the condition flags), and use `__asm__ __volatile__` when the instruction has side effects, so the compiler does not delete or move it.

The order of operations

> **1. Better algorithm → 2. Better data layout → 3. Better memory access → 4. Better branches → 5. SIMD → 6. Inline assembly.**
>
> Most real wins come from steps 1–3. Most wasted effort goes into steps 5–6.

Module 25 Rules

#	Rule
1	Know your actual n — asymptotics may not apply
2	RAM is ~92× L1 on this machine (1.6 ns → 149 ns). Layout is the dominant constant.
3	Memory moves in 64-byte lines — iterate in memory order
4	A mispredicted branch costs ~15–20 cycles; sorting the data gave 9.12×
5	Check the assembly before concluding a branch exists — a `cmov` may already be there
6	Branchless wins on unpredictable branches, loses on predictable ones
7	`alignas(64)` on per-thread data to kill false sharing

#	Rule
8	SoA when you touch few fields across many objects; AoS when you touch many fields of one
9	Consume every benchmark result, or the compiler deletes your loop
10	Warm up; report the min or median, never the mean; report the spread
11	Microbenchmarks are systematically optimistic — validate end-to-end
12	`perf stat` first: IPC < 1 means memory-bound
13	Try auto-vectorisation before intrinsics; `restrict` and SoA are what enable it
14	Float reductions need `-ffast-math` (or equivalent) to vectorise — and the result changes
15	Never ship `-march=native`; use run-time dispatch
16	PGO before hand-annotating branches
17	Algorithm → layout → memory → branches → SIMD → asm, in that order

Practice

1. **Map your own hierarchy.** Run the pointer-chase benchmark across working sets from 4 KB to 1 GB and plot the result. Compare the plateaus with the cache sizes reported by `lscpu`. Then repeat with sequential access and explain why the curve flattens.

2. **Stride experiment.** Measure the cost of touching one byte every 1, 2, 4 … 8,192 bytes. Find the knee. Confirm that it matches `getconf LEVEL1_DCACHE_LINESIZE`.

3. **Branch prediction.** Reproduce the sorted/unsorted experiment. First confirm that GCC flattens it to a `cmov` at `-O2`. Then force a real branch three ways (a `noinline` call, `-O0`, a `volatile` condition) and measure the speedup from sorting each time.

4. **Branchless suite.** Implement branchless `min`, `max`, `abs`, `clamp`, and conditional swap. Benchmark them against branchy versions on (a) random data, (b) sorted data, and (c) all-identical data. Build a table of when each wins.

5. **False sharing.** On a real multicore machine, compare 4 threads incrementing adjacent `long`s with `alignas(64)`-padded ones. Measure. Then use `perf c2c` to see the cache-line contention directly.

6. **AoS vs SoA.** Build a particle system with 8 fields and 1 million particles. Implement position integration (which touches 6 fields) and a “count particles above y = 0” query (which touches 1 field) in both layouts. Measure all four combinations and explain the pattern.

7. **Benchmark harness.** Write a reusable harness with warm-up, *N* repetitions, min/median/max/standard-deviation reporting, dead-code-elimination prevention, and CPU pinning. Use it to re-measure something you benchmarked earlier in this course, and see whether your original number was honest.

8. **Profile something real.** Take a program of more than 500 lines. Profile it with `perf record -g` and generate a flame graph. Identify the top 3 hot functions. Run `perf stat` and diagnose, from IPC and miss rates, whether it is memory- or compute-bound. Optimise the top function. Re-measure end-to-end, and compare the actual speedup with the theoretical Amdahl limit.

9. **Vectorisation.** Write a saxpy (`y[i] += a*x[i]`). Measure it at `-O2`, `-O3`, and `-O3 -march=native`, with and without `restrict`, and with hand-written AVX2. Use `-fopt-info-vec-missed` to find out why each version did or did not vectorise.

10. **Float reassociation.** Sum 10 million floats with scalar and vectorised code. Compare the results bit for bit, and quantify the error. Then implement Kahan summation and compare its accuracy and speed with both.

11. **PGO.** Take any non-trivial program. Build it with `-O2`, then with `-fprofile-generate` / a representative run / `-fprofile-use`. Measure the difference. Then check whether hand-adding `likely`/`unlikely` on top of PGO helps or hurts.

12. **Optimiser limits.** For each of the 9 “can’t do” items in §25.8, write a small program demonstrating it, dump the assembly, and show the source change that fixes it.

Module 25 in Summary

Asymptotic complexity describes the limit, but real C programs often run at small *n*, where constant factors — dominated by memory — decide the winner. Memory latency spans nearly two orders of magnitude between L1 and RAM, data moves in 64-byte lines, the prefetcher rewards sequential and strided access but cannot follow pointers, and threads writing to the same cache line fight over it even when their data is separate. So layout — row-major traversal, packed hot fields, SoA where few fields are touched, hot/cold splitting, indices, and arenas — is the dominant performance lever.

Mispredicted branches waste a pipeline’s worth of work: a real branch on random data was 9× slower than on sorted data, although the compiler had already removed the naive version’s branch with a `cmov`. Branchless code helps only on unpredictable branches; hints mainly improve code layout, and PGO does it better. Benchmarks must consume their results, warm up, repeat, report the minimum or median with the spread, and be validated end-to-end, because microbenchmarks flatter. Profile before optimising — `perf stat` diagnoses memory- versus compute-bound code, and flame graphs show where the time goes — and remember Amdahl’s law.

SIMD processes many values per instruction; try auto-vectorisation (enabled by `restrict`, SoA, and, for floating-point reductions, explicit permission to reassociate) before intrinsics, keep scalar fallbacks, and dispatch at run time. The optimiser is powerful but cannot

change your algorithm or layout, see through aliases or indirect calls, or know your data. Optimise in order: algorithm, layout, memory access, branches, SIMD, and only then assembly.

Next: Module 26 — Portability, covering the endianness, word-size, and alignment assumptions that break when your code leaves your machine; feature-test macros and the `_POSIX_C_SOURCE` maze you have hit repeatedly in this course; freestanding versus hosted implementations; and how to build the abstraction layer that keeps `#ifdef` out of your logic.

Module 26 — Portability

In this module:

- Endianness, word size, LP64 vs LLP64
- Alignment and unaligned access on strict architectures
- Feature-test macros (`_POSIX_C_SOURCE`, `_GNU_SOURCE`, `_DEFAULT_SOURCE`)
- Platform-detection macros, compiler-specific extensions
- Freestanding vs hosted implementations
- Toolchain differences and abstraction layers

All output in this module is real, from GCC 13.3 and glibc 2.39 on x86-64 Linux (the LP64 data model).

C's portability is one of its great selling points: there is a C compiler for virtually every processor ever made. But portable *language* does not mean portable *programs*. C deliberately leaves many details to each implementation (Module 16) — the size of `long`, the order of bytes in memory, whether misaligned accesses work, which library functions exist — and code quietly comes to depend on whatever the author's own machine happened to do. This module is about the assumptions that break when your code leaves your machine, and the disciplines that keep them from being made in the first place.

26.1 Endianness

```
=== DETECTION ===
  __BYTE_ORDER__             : little
  runtime union check        : little

=== HOW IT LOOKS IN MEMORY ===
  uint32_t 0x12345678 in memory: 78 56 34 12
  little-endian: 78 56 34 12   big-endian: 12 34 56 78
```

Endianness is the order in which a multi-byte value's bytes are stored in memory. **Little-endian** machines store the least significant byte first, at the lowest address: x86, ARM (in practice), and RISC-V. **Big-endian** machines store the most significant byte first: SPARC, older PowerPC and MIPS systems, many network processors — and every Internet protocol, which is why big-endian is also called *network byte order* (Module 24).

The names come from *Gulliver's Travels*, in which two nations went to war over which end of a boiled egg should be cracked first — a fitting metaphor for a choice that is arbitrary but must be agreed upon.

Detection, best to worst

```c
/* 1. C23 — the standard way */
#if __STDC_ENDIAN_NATIVE__ == __STDC_ENDIAN_LITTLE__

/* 2. GCC/Clang — universally available in practice */
#if __BYTE_ORDER__ == __ORDER_LITTLE_ENDIAN__

/* 3. Runtime union — always works, no #ifdef, optimizes to a constant */
```

```
union { uint32_t u; uint8_t b[4]; } e = { .u = 1 };
bool little = (e.b[0] == 1);
```

The run-time check stores the integer 1 and looks at which byte it ended up in (Module 11). The compiler knows the answer at compile time and folds the whole check to a constant.

Byte swapping

```
__builtin_bswap16(0x1234)     = 0x3412
__builtin_bswap32(0x12345678) = 0x78563412
__builtin_bswap64             = 0xEFCDAB8967452301
```

Portable names: `htons`/`htonl`/`ntohs`/`ntohl` (POSIX), or glibc's `<endian.h>` family (`htobe32`, `le64toh`, ...).

⚠ The right answer: don't detect at all

```
static void put_u32be(uint8_t *p, uint32_t v)
{ p[0]=(uint8_t)(v>>24); p[1]=(uint8_t)(v>>16); p[2]=(uint8_t)(v>>8); p[3]=(uint8_t)v; }

static uint32_t get_u32be(const uint8_t *p)
{ return ((uint32_t)p[0]<<24)|((uint32_t)p[1]<<16)|((uint32_t)p[2]<<8)|(uint32_t)p[3]; }
```

```
put_u32be(0xDEADBEEF) -> DE AD BE EF
get_u32be()           -> 0xDEADBEEF round-trip OK: 1
```

> This code produces identical bytes on every machine. No `#ifdef`, no detection, no byte-swap builtin, no alignment requirement. And GCC recognises the pattern and compiles it to a single `bswap` instruction on little-endian machines.

The key insight is that shifts and masks operate on *values*, not on memory: `v >> 24` is the most significant byte of `v` on every machine, whatever order the bytes happen to be stored in. So code that builds and takes apart integers with shifts is automatically independent of byte order. (The `(uint32_t)` casts in the getter matter: without them, `p[0] << 24` would be computed in `int`, and a byte of 0x80 or more would shift into the sign bit — Module 4.)

Never `fwrite` a structure or `memcpy` an integer into a wire buffer (Modules 11 and 13). Serialise field by field with explicit shifts. That single discipline eliminates endianness, padding, and alignment portability problems all at once.

Also note that bitfield order is implementation-defined (Module 11), so bitfields are never portable across the wire either.

26.2 Word Size and Data Models

```
=== THIS PLATFORM: x86_64 / LP64 ===
  char 1  short 2  int 4  long 8  long long 8
  void* 8  size_t 8  ptrdiff_t 8  time_t 8  wchar_t 4  long double 16
  CHAR_BIT = 8
```

The models

Module 2 introduced these; here they are with `short` included:

Model	`short`	`int`	`long`	pointer	Where
ILP32	16	32	32	32	32-bit Linux, Win32
LP64	16	32	64	64	Linux, macOS, BSD (64-bit)
LLP64	16	32	32	64	64-bit Windows ← the odd one
ILP64	16	64	64	64	Rare (some Cray systems)

Microsoft chose LLP64 so that the enormous body of existing Windows code — and Windows' binary file formats and API structures, full of `LONG` and `DWORD` fields — would keep the same sizes on 64-bit Windows.

The bugs this causes

```
long p = (long)ptr;            -> TRUNCATES on Win64 (long is 32-bit)
printf("%ld", (long)size)      -> wrong width on Win64
sizeof(long)==sizeof(void*)    -> FALSE on Win64
int used as an array index     -> breaks past 2 GB on all 64-bit models
32-bit time_t                  -> Y2038
```

`long` is the least portable type in C. It is the only integer type whose size differs between the two mainstream 64-bit platforms. (The `printf` case is subtler than it looks: casting a `size_t` to `long` and printing with `%ld` is *type-correct* everywhere, but on Win64 it silently truncates sizes above 4 GB.)

The fixes

Need	Use
Pointer ↔ integer	`intptr_t`/`uintptr_t` — never `long`
Sizes, counts, indices	`size_t`, printed with `%zu`
Pointer differences	`ptrdiff_t`, printed with `%td`
Exact widths (protocols, files)	`int32_t`/`uint64_t` + `PRId32`/`PRIu64`
File offsets	`off_t` + `-D_FILE_OFFSET_BITS=64`
Time on 32-bit systems	`-D_TIME_BITS=64` (Y2038)

Assert your assumptions — Module 15's cheapest tool:

```c
_Static_assert(sizeof(void*) == sizeof(uintptr_t), "uintptr_t must hold a pointer");
_Static_assert(CHAR_BIT == 8, "octet-addressable platform required");
_Static_assert(sizeof(uint32_t) == 4, "exact-width type required");
```

An assumption checked at compile time costs nothing, documents itself, and turns a subtle run-time misbehaviour on some future platform into an immediate, clearly worded build error.

Other size assumptions that bite

Assumption	Reality
`CHAR_BIT == 8`	True everywhere you're likely to meet — but some DSPs use 16 or 32
`sizeof(int) == 4`	2 on some 16-bit embedded targets
`char` is signed	Unsigned on ARM, PowerPC, and s390 (Module 2 — measured)
A plain `int` bitfield is signed	Implementation-defined (Module 11 — measured −1)
`long double` is 80-bit	16 bytes (80 bits used) on x86-64 Linux, 8 bytes on MSVC, a true 128-bit quad on ARM64 Linux
`wchar_t` is 4 bytes	2 on Windows (Module 8)
`time_t` is 64-bit	32-bit on older 32-bit systems
Two's complement	Guaranteed only since C23 (universal in practice)

26.3 Alignment on Strict Architectures

Module 9, §9.9 measured this. The portability framing:

Architecture	Unaligned access
x86/x86-64	Works, slower; faults for some SSE/AVX instructions
ARM64	Works for normal loads; faults for exclusive loads/stores (`LDXR`/`STXR`) and some SIMD
ARMv5 and earlier	Silently returns *rotated* garbage — the worst possible failure mode
SPARC, older MIPS, some DSPs	`SIGBUS`
RISC-V	Implementation-defined; may trap and be emulated by the kernel (extremely slow)

```c
/* NOT portable — misalignment AND strict aliasing (Module 2) */
uint32_t v = *(const uint32_t *)(buf + 1);

/* Portable, correct everywhere, and free at -O2 */
uint32_t v;
memcpy(&v, buf + 1, sizeof v);
```

```
unaligned store/load at offset 1: 0xDEADBEEF   ← via memcpy, ASan/UBSan clean
```

`memcpy` is the universal answer: it handles alignment and aliasing at the same time, and compiles to a single instruction wherever the hardware allows unaligned access — and to the correct byte-by-byte sequence where it does not. The worst case in the table deserves emphasis: on an old ARM, the pointer-cast version does not crash; it returns a *different number*, with no indication that anything went wrong.

Related: `alignas` for stronger alignment; `aligned_alloc` for SIMD and DMA buffers; `__attribute__((packed))` creates misaligned members (Module 11 — `-Waddress-of-packed-member`); and structure layout differs between ABIs, so never `fwrite` a structure.

26.4 Feature-Test Macros ⚠

You have hit this repeatedly throughout the course. Here is the systematic explanation.

glibc's headers *hide* many symbols by default. Which ones you get depends entirely on *feature-test macros* — macros you define to tell the headers which standards you want:

```
### -std=c17 (STRICT: no POSIX at all) ###
error: 'strdup' undeclared
error: 'strtok_r' undeclared
error: 'gmtime_r' undeclared

### -std=gnu17 (the DEFAULT) ###
  _POSIX_C_SOURCE = 200809L, _DEFAULT_SOURCE defined
  strdup, strtok_r, gmtime_r : all visible

### -std=c17 -D_POSIX_C_SOURCE=200809L ###
  all visible

### -std=c17 -D_GNU_SOURCE ###
  all visible
```

The strict ISO mode (`-std=c17`) promises a program that uses only ISO C, so the headers may not declare anything that could clash with a name the program itself uses — which rules out every POSIX function. GCC's default, `-std=gnu17` (Module 1), defines `_DEFAULT_SOURCE`, which exposes POSIX and more.

The macros

Macro	Exposes
(nothing) with `-std=cXX`	ISO C only. No POSIX.
`_DEFAULT_SOURCE`	POSIX 2008 + BSD + SVID extensions. The default with `-std=gnuXX`.
`_POSIX_C_SOURCE=200809L`	POSIX.1-2008. The portable choice.
`_POSIX_C_SOURCE=199309L`	POSIX.1b — real-time functions (`clock_gettime`, `nanosleep`)
`_XOPEN_SOURCE=700`	XSI = POSIX 2008 plus the X/Open extensions (`crypt`, `sync`, ...)
`_GNU_SOURCE`	Everything, including GNU-only functions (`asprintf`, `memmem`, `pipe2`, `qsort_r`, `strchrnul`)
`_FILE_OFFSET_BITS=64`	64-bit `off_t` on 32-bit systems
`_TIME_BITS=64`	64-bit `time_t` (Y2038)
`_LARGEFILE64_SOURCE`	The explicit `*64` variants (`lseek64`)

⚠ The ordering rule

```
/* WRONG */
#include <string.h>   /* string.h is processed FIRST */
#define _GNU_SOURCE   /* ...too late */
```

```
warning: implicit declaration of function 'strdup'
```

```
/* RIGHT */
#define _GNU_SOURCE   /* FIRST LINE, before ANY include */
#include <string.h>
```

```
correct order works: x
```

glibc's headers all include a common internal header (`<features.h>`) the first time any of them is included, and it decides then, once and for all, which symbols to expose. Defining a feature-test macro afterwards has no effect.

> **Feature-test macros must appear before every `#include`, including indirect ones.** In practice: define them on the command line (`-D_POSIX_C_SOURCE=200809L`) or in your build system, never in source files. That way you cannot get the order wrong.

Why this whole system exists

Standards evolve. `strdup` was a BSD function, then part of POSIX 2001, then POSIX 2008 — and is now ISO C23 (Module 18). Feature-test macros let one set of headers serve strict-conformance builds, POSIX builds, and GNU builds, with the right symbols visible for each.

The practical policy

```
# Portable POSIX code — recommended default
CPPFLAGS += -D_POSIX_C_SOURCE=200809L -D_FILE_OFFSET_BITS=64

# Or, for Linux-only code with GNU extensions
CPPFLAGS += -D_GNU_SOURCE

# Or just use -std=gnu17 (GCC's default), which sets _DEFAULT_SOURCE
```

Pick one, set it globally, and document why. Mixing them file by file is how you get "it compiles in one directory and not another."

> ⚠ Non-glibc systems differ. musl is more permissive; macOS uses `_DARWIN_C_SOURCE`; the BSDs have their own conventions; MSVC has none of this. Test on your actual targets.

26.5 Platform and Capability Detection

```
=== COMPILER ===
  gcc 13.3.0
  NOTE: clang defines __GNUC__ too -> check __clang__ FIRST
```

```
=== OS ===           linux, __unix__ also defined
=== ARCHITECTURE === x86-64
```

The macros

Category	Macros
Compiler	`__GNUC__`, `__clang__`, `_MSC_VER`, `__INTEL_LLVM_COMPILER`, `__TINYC__`
OS	`_WIN32` (also on Win64), `_WIN64`, `__linux__`, `__APPLE__`, `__FreeBSD__`, `__unix__`
Architecture	`__x86_64__`/`_M_X64`, `__i386__`, `__aarch64__`/`_M_ARM64`, `__arm__`, `__riscv` (+ `__riscv_xlen`)
Data model	`__LP64__`, `_LP64`, `__ILP32__`

Note the pairs in the architecture row: GCC and Clang use names like `__x86_64__`, while MSVC uses `_M_X64` for the same thing.

Three traps:

- **Clang defines `__GNUC__`** — check `__clang__` first.
- **`_WIN32` is defined on 64-bit Windows too** — use it to mean "Windows," and `_WIN64` to mean "64-bit Windows."
- **`__APPLE__` covers macOS and iOS** — use `<TargetConditionals.h>` to tell them apart.

✓ Prefer capability probes to version checks

```
__has_include(<threads.h>)    = 1
__has_include(<stdbit.h>)     = 1
__has_include(<sys/epoll.h>)  = 1
__has_builtin(add_overflow)   = 1
__has_attribute(nonnull)      = 1
```

```
#if defined(__has_include) && __has_include(<stdckdint.h>)
#  include <stdckdint.h>
#  define mul_ovf(r,a,b) ckd_mul((r),(a),(b))
#elif defined(__has_builtin) && __has_builtin(__builtin_mul_overflow)
#  define mul_ovf(r,a,b) __builtin_mul_overflow((a),(b),(r))
#else
#  define mul_ovf(r,a,b) mul_ovf_fallback((r),(a),(b))
#endif
```

`__has_include`, `__has_builtin`, `__has_attribute`, and `__has_c_attribute` ask "*can* you do this?" rather than "*which version* are you?" That is far more robust: backported features, vendor forks, and partial implementations all work correctly, where a version check would have guessed. The `defined(__has_include)` test first guards against compilers too old to have the probe at all. (Note the argument order: `ckd_mul` takes the result pointer first, `__builtin_mul_overflow` last — the macro hides the difference.)

C11 optional-feature markers:

```
threads.h supported / stdatomic.h supported / VLAs supported
```

`__STDC_NO_THREADS__`, `__STDC_NO_ATOMICS__`, `__STDC_NO_VLA__`, and `__STDC_NO_COMPLEX__` are defined when the corresponding optional feature is *absent*.

Build-mode detection:

```
ASan instrumentation detected    ← __SANITIZE_ADDRESS__
debug build (asserts active)     ← !NDEBUG
```

This is useful for skipping timing-sensitive tests under sanitizers, or lengthening their timeouts.

Reference: the "predef" wiki project (sourceforge.net/p/predef/wiki) has the exhaustive list of predefined macros for every compiler, OS, and architecture.

26.6 Freestanding vs Hosted

The C standard defines two kinds of conforming implementation:

	Hosted	Freestanding
Entry point	`main`	Implementation-defined (`_start`, a reset vector, ...)
Standard library	All of it	Only `<float.h>`, `<iso646.h>`, `<limits.h>`, `<stdalign.h>`, `<stdarg.h>`, `<stdbool.h>`, `<stddef.h>`, `<stdint.h>`, `<stdnoreturn.h>` (C11/C17)
Operating system	Required	None
Examples	Applications	Kernels, bootloaders, bare-metal firmware

A *hosted* implementation is what you have been using throughout this book: a full C library, running on an operating system, starting at `main`. A *freestanding* implementation assumes nothing but the compiler: only the headers that define types, limits, and macros — none of which need any run-time code. (C23 enlarges the freestanding set somewhat, adding for example `<stdbit.h>`, `<stdckdint.h>`, and a handful of `<string.h>` functions such as `memcpy` and `memset`, but toolchain support for the enlarged set varies.)

A real freestanding program

```c
#include <stdint.h>
#include <stddef.h>

/* No libc — provide our own; the compiler may still CALL these */
void *memset(void *d, int c, size_t n){ unsigned char *p=d; while(n--) *p++=(unsigned char)c; return
d; }
void *memcpy(void *d, const void *s, size_t n){ ... }

static long sys_write(int fd, const void *buf, size_t n)
{   long r;
    __asm__ volatile ("syscall" : "=a"(r) : "a"(1L),"D"((long)fd),"S"(buf),"d"(n)
                      : "rcx","r11","memory");
    return r; }
```

```
/* _start, not main: there is no C runtime to call main for us */
__attribute__((noreturn)) void _start(void) { ...; sys_exit(0); }
```

```
gcc -ffreestanding -nostdlib -nostartfiles -static -O2 free.c -o freestanding
```

```
freestanding: no libc, no crt, direct syscalls
value = 42

freestanding    :   9368 bytes
hosted (dyn)    :  15960 bytes
hosted (static) : 785360 bytes   ← 84× larger

freestanding: not a dynamic executable
```

With no C library, there is no `write` function, so the program makes the system call itself (Module 24): on x86-64 Linux, the system-call number goes in `rax` (1 is `write`), the arguments in `rdi`, `rsi`, and `rdx`, and the `syscall` instruction enters the kernel, which overwrites `rcx` and `r11` — hence the clobber list (Module 25). And with no C run-time start-up code, nothing calls `main`: execution begins at `_start`, the symbol the linker uses as the entry point (Module 1), and `_start` must never return, because there is nothing to return to — it exits with a system call instead.

The statically linked hosted version is 84 times larger because it carries the C library's start-up code, `stdio`, locale support, and everything they depend on.

What you lose, and must provide

- **`memset`, `memcpy`, `memmove`, `memcmp`** — GCC emits calls to these even with `-ffreestanding` (for structure copies and zero-initialisation, for example), so you must supply them.
- **Start-up code:** zeroing `.bss`, copying `.data` from flash to RAM, and setting up the stack (Module 27).
- **`malloc`, `printf`, all I/O, all math functions, and `errno`.**
- **Static constructors and `atexit`.**

The flags

Flag	Effect
`-ffreestanding`	The implementation is freestanding: don't assume standard-library semantics for `main`, `printf`, and so on
`-nostdlib`	Don't link the C library or the start-up files
`-nostartfiles`	Don't link `crt1.o`/`crti.o` (the start-up code)
`-nodefaultlibs`	Don't link the default libraries, but keep the start-up files
`-fno-builtin`	Don't treat standard function names as builtins
`-static`	No dynamic loader

`-ffreestanding` implies `-fno-builtin`: the compiler stops assuming that a function called `strlen` behaves like the standard one — which matters, because in a kernel it might not.

This is exactly how the Linux kernel, U-Boot, FreeRTOS applications, and every bare-metal firmware image are built (Module 27).

26.7 Abstraction Layers

> **The single most important portability practice: put every `#ifdef` in one file.**

Application code should be plain, portable C. Platform differences live in a `plat.h` header plus one implementation file per platform — `plat_posix.c`, `plat_win32.c`, and so on.

```c
/* plat.h — the ONE header containing every #ifdef in the project */

/* 1. compiler attributes */
#if defined(__clang__) || defined(__GNUC__)
#  define PLAT_PRINTF(m,n) __attribute__((format(printf,m,n)))
#  define PLAT_UNLIKELY(x) __builtin_expect(!!(x),0)
#  define PLAT_EXPORT      __attribute__((visibility("default")))
#elif defined(_MSC_VER)
#  define PLAT_PRINTF(m,n)
#  define PLAT_UNLIKELY(x) (x)
#  define PLAT_EXPORT      __declspec(dllexport)
#else
#  define PLAT_PRINTF(m,n)
#  define PLAT_UNLIKELY(x) (x)
#  define PLAT_EXPORT
#endif

/* prefer C23 attributes where they exist */
#if defined(__has_c_attribute) && __has_c_attribute(nodiscard)
#  define PLAT_NODISCARD [[nodiscard]]
#elif defined(__GNUC__)
#  define PLAT_NODISCARD __attribute__((warn_unused_result))
#else
#  define PLAT_NODISCARD
#endif

/* 2. checked arithmetic: C23 → GNU → fallback */
#if defined(__has_include) && __has_include(<stdckdint.h>)
#  include <stdckdint.h>
#  define plat_mul_ovf(r,a,b) ckd_mul((r),(a),(b))
#elif defined(__GNUC__)
#  define plat_mul_ovf(r,a,b) __builtin_mul_overflow((a),(b),(r))
#else
#  define plat_mul_ovf(r,a,b) plat_mul_ovf_fn((r),(a),(b))
#endif

/* 3. byte order: EXPLICIT, no detection */
static inline void     plat_put_u32be(uint8_t *p, uint32_t v) { ... }
static inline uint32_t plat_get_u32be(const uint8_t *p)       { ... }

/* 4. alignment-safe access */
static inline uint32_t plat_load_u32(const void *p)
{ uint32_t v; memcpy(&v,p,sizeof v); return v; }
/* (plat_store_u32 is the mirror image) */

/* 5. platform services — implemented per-OS */
PLAT_NODISCARD int      plat_mkdir(const char *path);
PLAT_NODISCARD int      plat_sleep_ms(unsigned ms);
PLAT_NODISCARD uint64_t plat_mono_ns(void);

/* 6. assert your assumptions */
_Static_assert(sizeof(void*) == sizeof(uintptr_t), "uintptr_t must hold a pointer");
_Static_assert(CHAR_BIT == 8, "octet-addressable platform required");
```

The header has three kinds of content: macros that paper over compiler differences (section 1), portable helpers that need no `#ifdef` at all because they are written to work everywhere (sections 3 and 4), and declarations of *services* — time, sleeping, directories — whose implementations live in the per-platform `.c` files (section 5). Only the right one of those files is compiled for each platform, selected by the build system.

The application then contains no `#ifdef` at all:

```c
#include "plat.h"
int main(void)
{
    uint64_t t0 = plat_mono_ns();
    if (plat_sleep_ms(20) != 0) return 1;
    printf("slept %.1f ms\n", (double)(plat_mono_ns()-t0)/1e6);

    uint8_t wire[8];
    plat_put_u32be(wire, 0xCAFEBABE);
    plat_store_u32(buf + 1, 0xDEADBEEF);   /* misaligned — still correct */

    size_t total;
    if (plat_mul_ovf(&total, (size_t)1<<40, (size_t)1<<40)) { /* overflow */ }
}
```

(`buf` is a byte buffer declared in the part of the demo omitted here.)

```
platform layer: posix
  slept 20.1 ms
  serialized: CA FE BA BE -> 0xCAFEBABE
  unaligned store/load at offset 1: 0xDEADBEEF
  overflow correctly detected
ASan/UBSan exit=0
```

Why this matters

Scattered `#ifdef`	One abstraction layer
2^n programs; you test one	One program, *n* small back ends
Every configuration rots independently	Only the back ends can rot
Unreadable business logic	Logic reads as plain C
Adding a platform touches every file	Adding a platform means one new `.c`

Prefer a run-time `if` to `#ifdef` wherever possible (Module 12), so that all configurations stay compiled and type-checked.

Existing abstraction layers worth using

libuv (event loop, files, processes, threads — the best general-purpose one), SDL (windowing, input, audio), APR (the Apache Portable Runtime), GLib (data structures plus a platform layer), PDCurses/ncurses (terminal UIs), and mimalloc (a portable allocator).

Don’t write your own if one of these fits.

Module 26 Rules

#	Rule
1	Serialise with explicit shifts — no endianness detection needed
2	Never `fwrite/memcpy` a structure into a file or socket
3	`long` is the least portable type — it is 32-bit on Win64
4	`intptr_t` for pointers, `size_t/%zu` for sizes, `intN_t/PRId32` for wire formats
5	`_Static_assert` every size and alignment assumption
6	`memcpy` for all unaligned and type-punned access
7	Feature-test macros go on the command line, before every include
8	Pick one feature-test-macro policy per project and document it
9	Check `__clang__` before `__GNUC__`; `_WIN32` is defined on Win64 too
10	Prefer `__has_include/__has_builtin/__has_attribute` to version checks
11	Freestanding gives you 9 headers and no `main` — you supply `memcpy` and the start-up code
12	One file holds every `#ifdef`; application code is plain C
13	Prefer run-time `if` to `#ifdef` so all configurations stay compiled
14	Test on real targets — cross-compile + QEMU in CI (Module 19)

Practice

1. **Endian round-trip.** Write `put_u{16,32,64}{be,le}` and the matching getters. Serialise a structure with 6 fields, dump the bytes with `xxd`, and confirm that they are identical when cross-compiled and run under `qemu-aarch64` (or a big-endian target such as `qemu-mips`).

2. **Find the `long` bugs.** Take any C project and search for `(long)`, `%ld`, and `sizeof(long)`. For each, decide whether it would break on Win64, and fix the ones that would.

3. **Size-assertion header.** Write `assumptions.h`, containing `_Static_assert`s for every size, alignment, and signedness assumption your code makes. Include it everywhere. Then compile with `-m32` and see which ones fire.

4. **Alignment fault.** Cross-compile a program doing `*(uint32_t*)(buf+1)` for a strict-alignment target (`qemu-mips` or `qemu-sparc`) and observe the `SIGBUS`. Then fix it with `memcpy` and confirm that it works. Compare the generated assembly on x86-64 and on the strict target.

5. **Feature-test-macro archaeology.** Take one source file that uses `strdup`, `strtok_r`, `nanosleep`, `asprintf`, and `pipe2`. Find the minimum feature-test macro that exposes each. Then find the single macro that exposes all of them, and determine which of them are not portable beyond Linux.

6. **The ordering bug.** Reproduce the "define `_GNU_SOURCE` after the first include" failure. Then find a header in a real project that defines a feature-test macro in the wrong place.

7. **Capability-probe header.** Write `compat.h` providing `MY_NODISCARD`, `MY_FALLTHROUGH`, `MY_LIKELY`, `ckd_add`, `secure_zero`, and `mono_ns`, using `__has_include`/`__has_builtin`/`__has_c_attribute` throughout — with no compiler-version checks at all. Verify that it builds under `-std=c17` and `-std=c2x`, with GCC and Clang.

8. **Freestanding hello.** Build the §26.6 program. Then extend it: implement `memcmp` and a minimal `printf` supporting `%d` and `%s`. Measure the final binary size. Then try it with `-Os -ffunction-sections -Wl,--gc-sections` (Module 19).

9. **Build the platform layer.** Implement `plat.h` + `plat_posix.c`, covering monotonic time, sleep, `mkdir`, file existence, the path separator, the temporary directory, thread creation, and mutexes. Then write `plat_win32.c` (compile-test it with `x86_64-w64-mingw32-gcc` if available). Confirm that the application file contains zero `#ifdef`s.

10. **Cross-compile CI.** Add a CI job that cross-compiles your project for `aarch64-linux-gnu` and runs the test suite under `qemu-aarch64-static`. Then add a big-endian target. Record every bug it finds.

Module 26 in Summary

Portable C avoids depending on details that implementations are free to choose. Byte order differs between machines, but code that serialises values with explicit shifts produces identical bytes everywhere without ever detecting it — and structures must never be written raw to a file or socket. Integer sizes follow data models, and `long` differs between Linux and Windows, so pointers belong in `intptr_t`, sizes in `size_t`, and wire fields in exact-width types, with every remaining assumption pinned down by `_Static_assert`. Unaligned access ranges from merely slow to silently wrong, and `memcpy` handles it correctly everywhere.

glibc exposes POSIX and GNU functions only when feature-test macros ask for them, and those macros must precede every include — so set one documented policy on the command line. Detect platforms with the standard predefined macros, remembering that Clang also defines `__GNUC__` and that `_WIN32` includes 64-bit Windows, but prefer capability probes such as `__has_include` and `__has_builtin` to version checks. Freestanding implementations provide only a handful of headers and no `main`; programs supply their own `memcpy`, start-up code, and system calls, as kernels and firmware do. Finally, gather every platform difference into one abstraction layer — or adopt an existing one such as libuv — so that application code stays plain, portable C, and test on real targets under emulation in CI.

Next: Module 27 — Embedded / Bare-Metal C, the largest employer of C developers: memory-mapped I/O and correct `volatile` usage, interrupts and reentrancy, linker scripts and start-up code, strategies for working without dynamic allocation, fixed-point arithmetic, the peripheral protocols (UART, SPI, I²C, DMA), RTOS concepts, and the safety standards that govern automotive, medical, and aerospace C.

Module 27 — Embedded / Bare-Metal C

In this module:

- MCU architecture, memory maps, registers
- Memory-mapped I/O and correct `volatile` usage
- Interrupts, ISRs, vector tables, latency, reentrancy
- Linker scripts, start-up code, `.data`/`.bss` initialisation, stack setup
- No dynamic allocation / static allocation strategies
- Fixed-point arithmetic, environments without an FPU
- Peripherals and protocols: GPIO, UART, SPI, I²C, CAN, ADC, PWM, DMA, timers, watchdog
- RTOS: FreeRTOS, Zephyr, ThreadX — tasks, queues, priorities, priority inversion
- Low-power modes, bootloaders, firmware update / OTA
- Debug hardware: JTAG/SWD, OpenOCD, logic analysers, oscilloscopes
- MISRA C compliance, safety standards (ISO 26262, DO-178C, IEC 61508)
- CMSIS, HAL vs register-level programming

Embedded systems are the largest employer of C developers. Cars, washing machines, pacemakers, routers, drones, keyboards, and industrial controllers all run C on small processors with no operating system, or a very small one. Everything in this module was built and inspected with a real `arm-none-eabi-gcc` 13.2 toolchain targeting an ARM Cortex-M4 microcontroller.

This module is where many earlier ideas stop being abstract. `volatile` (Module 2) becomes the difference between a blinking LED and a dead one. The `.data` and `.bss` sections (Modules 1 and 3) become loops you write yourself. Freestanding C (Module 26) becomes the normal way of working, and "no `malloc`" (Module 10) becomes a rule rather than a suggestion.

27.1 MCU Architecture, Memory Maps, Registers

A *microcontroller* (MCU) is a CPU core, memory, and peripherals — timers, serial ports, analogue converters — on a single chip. A typical Cortex-M4 memory map:

```
0x00000000 – Code (aliased flash)
0x08000000 – FLASH     512 KB   ← program + constants + .data initial values
0x20000000 – SRAM      128 KB   ← .data, .bss, heap, stack
0x40000000 – Peripherals        ← GPIO, UART, SPI, timers... memory-mapped
0xE0000000 – Cortex-M private   ← NVIC, SysTick, SCB, debug
0xFFFFFFFF
```

Flash memory is non-volatile (it keeps its contents without power) but slow to write, so it holds the program. SRAM is fast and writable but loses its contents at power-off, so it holds the variables. The "Cortex-M private" region contains the core's own control hardware: the *NVIC* (Nested Vectored Interrupt Controller), the *SysTick* timer, and the *SCB* (System Control Block).

Everything is memory. Configuring a UART means writing to an address. Reading a pin means loading from an address. There are no special I/O instructions, no driver model, and no operating system between your code and the hardware — this is called *memory-mapped I/O*.

Hardware registers come in flavours that C's type system cannot express:

Kind	Behaviour
Read-write	Ordinary
Read-only	Status registers — the hardware changes them under you
Write-only	Reading returns garbage
Write-1-to-clear	Writing 1 to a bit clears that flag (writing 0 does nothing)
Read-to-clear	The act of reading has a side effect
Set/reset pairs (BSRR)	Atomic bit manipulation without read-modify-write

That last one matters. `GPIOA->ODR |= (1<<5)` is a *read-modify-write*: load the register, set the bit, store it back. An interrupt that changes another bit of the same register between the load and the store has its change silently overwritten. `GPIOA->BSRR = (1<<5)`, by contrast, is a single store to a register whose hardware sets exactly the bits written as 1 and leaves the others alone — atomic by design. Use set/reset registers where the hardware provides them.

27.2 Memory-Mapped I/O and `volatile` ⚠

```
#define GPIOA_BASE 0x40020000u
typedef struct {
    volatile uint32_t MODER, OTYPER, OSPEEDR, PUPDR;
    volatile uint32_t IDR;    /* input data — READ ONLY, changes by hardware */
    volatile uint32_t ODR;    /* output data */
    volatile uint32_t BSRR;   /* bit set/reset — WRITE ONLY, atomic */
} GPIO_TypeDef;
#define GPIOA ((GPIO_TypeDef *)GPIOA_BASE)
```

A structure laid over a hardware address, with one member per register in the order and at the offsets given by the chip's reference manual, is the standard idiom: `GPIOA->ODR` then reads or writes the output register at `0x40020014`. The CMSIS device headers supplied by chip vendors (§27.12) are thousands of lines of exactly this.

Why `volatile` is not optional

```
void blink(void) {
    GPIOA->ODR |=  (1u << 5);   /* on */
    GPIOA->ODR &= ~(1u << 5);   /* off */
    GPIOA->ODR |=  (1u << 5);   /* on */
}
```

With `volatile` — three real read-modify-writes:

```
ldr r2, [r3, #20]   orr r2, r2, #32   str r2, [r3, #20]
ldr r2, [r3, #20]   bic r2, r2, #32   str r2, [r3, #20]
ldr r2, [r3, #20]   orr r2, r2, #32   str r2, [r3, #20]
```

Without `volatile` — the compiler collapses it to the net effect:

```
ldr r3, [r2, #20]   orr r3, r3, #32   str r3, [r2, #20]
bx  lr
```

One store. The LED would never blink. From the compiler's point of view, turning a bit on, off, and on again is the same as simply turning it on — which is true of ordinary memory, and completely wrong for a register wired to a pin.

And a status-polling loop:

```
while ((GPIOA->IDR & 1u) == 0) { }
```

With `volatile` — re-reads the register every iteration:

```
.L9: ldr r3, [r2, #16]   lsls r3, r3, #31   bpl .L9
```

Without `volatile` — reads once and spins forever:

```
     ldr r0, [r3, #16]   lsls r3, r0, #31   bmi .L14
.L16: b .L16   ← infinite loop, hardware never re-read
```

This is exactly Module 2's `wait_plain` example, on real hardware. (The `lsls ..., #31` shifts bit 0 into the sign position, so that the branch instructions `bpl`/`bmi` test it.)

> This is `volatile`'s actual purpose (Module 2). It is not thread safety (Module 17). It means: "this memory changes outside the program's control, so emit every access exactly as written."

What `volatile` does not give you

- **No ordering with non-`volatile` accesses.** The compiler may still move ordinary memory accesses across a `volatile` one. Use a compiler barrier: `__asm__ __volatile__("" ::: "memory")`.
- **No CPU memory ordering.** On a Cortex-M with a write buffer, or on systems with several bus masters, you need the barrier instructions `__DSB()` (data synchronisation barrier), `__DMB()` (data memory barrier), and `__ISB()` (instruction synchronisation barrier). A classic bug: write to a peripheral register, then immediately go to sleep — while the write is still sitting in the write buffer. Put `__DSB()` before `__WFI()`.
- **No atomicity** for anything wider than one machine word.
- **No cache coherence** — on Cortex-A processors and cached MCUs, DMA buffers need explicit cache clean and invalidate operations.

27.3 Interrupts, ISRs, Reentrancy

An *interrupt* is a hardware signal — a byte arrived, a timer expired, a pin changed — that makes the CPU suspend what it is doing and run an *interrupt service routine* (ISR), then resume. It is the embedded counterpart of a signal handler (Module 24), with many of the same hazards.

The vector table

On reset, a Cortex-M CPU loads two words from address 0 (which aliases the start of flash):

```
Contents of section .isr_vector:
 8000000 00000220 55000008 51000008 51000008 ...

word 0 = 0x20020000 = initial STACK POINTER (RAM top: 0x20000000 + 128K)
word 1 = 0x08000055 = Reset_Handler (Thumb bit set → odd address)
```

(The bytes are shown in memory order, so `00000220` read as a little-endian word is `0x20020000` — Module 26.) The first word becomes the initial stack pointer, and the second is the address of the first function to run. Cortex-M processors execute only the compact *Thumb* instruction set, and ARM marks Thumb code addresses by setting the lowest bit, which is why the reset handler's address is odd.

The rest of the table is an array of function pointers — one per exception and interrupt source:

```c
__attribute__((section(".isr_vector"), used))
void (* const g_vectors[])(void) = {
    (void (*)(void))&_estack,   /* [0] stack pointer */
    Reset_Handler,              /* [1] reset */
    NMI_Handler, HardFault_Handler, MemManage_Handler,
    BusFault_Handler, UsageFault_Handler,
    0, 0, 0, 0,                 /* reserved */
    SVC_Handler, 0, 0, PendSV_Handler, SysTick_Handler,
    /* device-specific IRQs from here */
    WWDG_IRQHandler, EXTI0_IRQHandler, TIM2_IRQHandler, USART1_IRQHandler,
};
```

When interrupt number *n* fires, the hardware itself looks up the function pointer at the corresponding position and calls it — no software dispatch is involved. `section(".isr_vector")` places the table where the linker script (§27.4) puts it at the start of flash, and `used` stops the compiler from discarding a variable that no code references.

Weak aliases let a driver override only the handlers it actually implements:

```c
#define WEAK_ALIAS __attribute__((weak, alias("Default_Handler")))
void TIM2_IRQHandler(void) WEAK_ALIAS;
void Default_Handler(void){ for(;;){} }   /* trap unhandled interrupts */
```

Every handler name is declared as a weak alias for `Default_Handler`. A driver that defines a real, strong `TIM2_IRQHandler` replaces it; every interrupt nobody handles lands in the infinite loop, where a debugger can find it. This is Module 22's weak-symbol technique, applied to hardware.

⚠ Sharing data with an ISR

A 32-bit read is atomic on ARM (a single instruction):

```
read_tick_ok: ldr r3, .L7   ldr r0, [r3]   bx lr
```

A 64-bit read is not:

```
read_micros_bad: ldrd r0, [r3, #8]   ← two words; an ISR between them tears the value
```

`ldrd` loads two registers from consecutive words — two separate memory accesses. If the timer ISR increments the counter between them, and the low word happens to wrap from `0xFFFFFFFF` to 0, the main code gets the old high word combined with the new low word: a *torn* value, off by about 4 billion.

Two correct fixes:

```c
/* 1. Lock-free double-read (works when the ISR only increments) */
uint64_t read_micros_ok(void)
{
    uint64_t a, b;
    do { a = g_micros; b = g_micros; } while (a != b);
    return b;
}

/* 2. Critical section */
static inline uint32_t irq_save(void) {
    uint32_t p;
    __asm__ volatile ("mrs %0, primask\n cpsid i" : "=r"(p) :: "memory");
    return p;
}
static inline void irq_restore(uint32_t p) {
    __asm__ volatile ("msr primask, %0" :: "r"(p) : "memory");
}
```

```
read_micros_critical:
    mrs   r3, primask      ← save the interrupt-enable state
    cpsid i                ← disable interrupts
    ldrd  r0, [r2, #8]
    msr   primask, r3      ← RESTORE (not blindly re-enable)
    bx    lr
```

The first reads the value twice and retries until two consecutive reads agree, so a read interrupted part-way is simply discarded. The second disables interrupts around the read. `PRIMASK` is the Cortex-M register that masks interrupts: `cpsid i` sets it, and the saved copy restores whatever state it had before.

Save and restore; never blindly `cpsie i` (enable interrupts). The caller might already have had interrupts disabled — for its own critical section — and blindly re-enabling them would break it.

The ISR rules

1. **Keep it short.** Set a flag, push data into a queue, return. Do the real work in the main loop.
2. **`volatile` on every variable shared with an ISR** (Module 2).

3. **No `malloc`, no `printf`, no blocking, no floating point** (unless FPU context saving is enabled — §27.6).
4. **Never wait for anything** — no polling loops, no semaphore-take-with-timeout.
5. **Clear the interrupt flag,** or the interrupt fires again immediately. On some MCUs the clearing write must complete (`__DSB()`) before returning, or you get a spurious re-entry.
6. **Nesting:** the Cortex-M NVIC supports priority-based preemption — a higher-priority ISR can interrupt a lower-priority one. So shared data needs protection between ISRs too, not only between an ISR and the main loop.
7. **Latency** is the time from the interrupt being asserted to the first instruction of its handler. On a Cortex-M3/M4 it is typically 12 cycles, because the hardware saves the caller-saved registers automatically. Disabling interrupts adds directly to the worst-case latency — keep critical sections tiny.
8. **Stack:** ISRs run on the main stack (or on the process stack under an RTOS). Nested interrupts each need their own frame space. Budget for the deepest possible nesting.

27.4 Linker Scripts and Start-Up Code ⚠

This is where “freestanding” (Module 26) becomes concrete: **you write the C run-time yourself.**

The linker script

On a hosted system, the linker uses a built-in default script to decide where each section goes. On a microcontroller, you must describe the chip’s memory and place every section explicitly:

```
ENTRY(Reset_Handler)

MEMORY
{
  FLASH (rx)  : ORIGIN = 0x08000000, LENGTH = 512K
  RAM   (rwx) : ORIGIN = 0x20000000, LENGTH = 128K
}

_estack = ORIGIN(RAM) + LENGTH(RAM);   /* stack grows DOWN from the top */

SECTIONS
{
  .isr_vector : { KEEP(*(.isr_vector)) } > FLASH   /* KEEP: --gc-sections must not remove it */

  .text : { *(.text*) *(.rodata*) . = ALIGN(4); _etext = .; } > FLASH

  /* .data lives in RAM but its INITIAL VALUES are stored in FLASH */
  _sidata = LOADADDR(.data);
  .data : { . = ALIGN(4); _sdata = .; *(.data*) . = ALIGN(4); _edata = .; } > RAM AT> FLASH

  .bss : { . = ALIGN(4); _sbss = .; *(.bss*) *(COMMON) . = ALIGN(4); _ebss = .; } > RAM

  /* fail the LINK if there is no room left for heap + stack */
  ._check : { . = ALIGN(8); . = . + _min_heap + _min_stack; . = ALIGN(8); } > RAM
}
```

The `MEMORY` block names the regions and their permissions. Each output section collects input sections from every object file (`*(.text*)` means “all sections whose names start with `.text`”, which, with `-ffunction-sections`, is every function) and assigns them to a region. `.

is the *location counter* — the current address — and assignments like `_sdata = .;` define symbols that C code can refer to. The final `._check` section reserves the minimum heap and stack sizes; if they do not fit, the link fails with an error, rather than the device failing mysteriously in the field. (`_min_heap` and `_min_stack` are defined elsewhere in the script.)

The measured result:

```
Idx Name        Size      VMA       LMA       File off
  0 .isr_vector 00000050  08000000  08000000
  1 .text       000000c8  08000050  08000050
  2 .data       00000004  20000000  08000118  ← VMA=RAM, LMA=FLASH
  3 .bss        00000008  20000004  0800011c
```

> **`> RAM AT> FLASH` is the key line.** The *VMA* (virtual memory address) is where a section lives while the program runs; the *LMA* (load memory address) is where it is stored in the image. `.data` runs in RAM but is stored in flash — because RAM loses its contents at power-off, the initial values must be kept in flash and copied to RAM at start-up. That copy is the start-up code's job. This is the mechanism behind Module 3's `.data`/`.bss` distinction, made explicit. (`.bss` also shows an LMA, but it contains no data — only zeros to be created.)

The start-up code

```c
extern uint32_t _sidata, _sdata, _edata, _sbss, _ebss;   /* linker-defined */

void Reset_Handler(void)
{
    uint32_t *src = &_sidata, *dst = &_sdata;
    while (dst < &_edata) *dst++ = *src++;           /* 1. copy .data FLASH → RAM */
    for (dst = &_sbss; dst < &_ebss; ) *dst++ = 0;   /* 2. zero .bss (Module 3!) */
    /* 3. __libc_init_array() — C++ ctors, __attribute__((constructor)) */
    main();                                          /* 4. finally, main() */
    for (;;) { }                                     /* main must never return */
}
```

The linker-defined symbols are not variables; they are *addresses*. Declaring them as `extern uint32_t` and taking their addresses (`&_sdata`) gives C code a way to refer to the boundaries the linker script computed.

The real disassembly:

```
08000054 <Reset_Handler>:
 8000054: push {r3, lr}
 8000056: ldr r2, [pc,#40]        ← _sidata (0x080000f0, in FLASH)
 8000058: ldr r3, [pc,#40]        ← _sdata (0x20000000, in RAM)
 800005c: cmp r3, r1
 800005e: bcc 8000070
 8000070: ldr.w r0, [r2], #4      ← the .data copy loop
 8000074: str.w r0, [r3], #4
 8000066: cmp r3, r2
 800007a: str.w r1, [r3], #4      ← the .bss zeroing loop
 800006a: bl 80000a4 <main>
 800006e: b.n 800006e             ← trap if main returns
```

(The listing is an excerpt, so the addresses are not all consecutive.) Module 3's guarantee that "statics are zero-initialised" is these few instructions. On a hosted system, the C run-time's start-up object (`crt0.o`/`crt1.o`) does this for you; here, you do it.

The build and the flashable image

```
arm-none-eabi-gcc -mcpu=cortex-m4 -mthumb -Os -g \
  -ffreestanding -nostdlib -nostartfiles \
  -ffunction-sections -fdata-sections -Wl,--gc-sections \
  -T link.ld startup.c main.c -o fw.elf -Wl,-Map=fw.map
```

```
arm-none-eabi-size fw.elf
 text  data  bss   dec   hex
  280     4  3084  3368  d28
```

```
arm-none-eabi-objcopy -O binary fw.elf fw.bin   # 284 bytes — raw flash image
arm-none-eabi-objcopy -O ihex fw.elf fw.hex     # Intel HEX
```

```
:020000040800F2
:10000000000002205500000851000008510000008BF
```

The ELF file is for the debugger; the raw binary (and the Intel HEX text format, which records addresses and checksums line by line) is what gets programmed into flash. The raw image is `text` plus `data` — the initial values — and nothing for `.bss`, which the start-up code creates.

A complete, flashable firmware image in 284 bytes — compare Module 26's 785 KB statically linked hosted "hello world."

Note `-Wl,-Map=fw.map`: **always generate a map file.** When you are 200 bytes over your flash budget, it tells you exactly what is consuming the space.

27.5 Static Allocation

Most embedded codebases ban `malloc` entirely. MISRA C forbids it (Rule 21.3), and DO-178C and IEC 61508 effectively do too.

Why:

- **Fragmentation** — a device running continuously for years will eventually fail an allocation, even with plenty of free memory in total.
- **Non-deterministic timing** — unacceptable in hard real-time code, which must meet deadlines every single time.
- **No recovery** — what does a thermostat do when `malloc` returns `NULL`?
- **Unbounded memory** — you cannot prove the worst case.
- **Heap corruption is unrecoverable** — and with no memory-management unit, nothing stops it spreading.

The strategies

```
/* 1. Static pools — allocate everything at compile time */
#define MAX_CONNECTIONS 16
static Connection g_conns[MAX_CONNECTIONS];
static uint32_t g_conn_used;   /* a bitmask, or a free list */

/* 2. Fixed-size block allocator (Module 10's pool, statically backed) */
```

```c
static uint8_t g_pool[MAX_BLOCKS][BLOCK_SIZE] __attribute__((aligned(8)));

/* 3. Arena reset per cycle (Module 10) */
static uint8_t g_arena[8192];
/* ... allocate during one control cycle, arena_reset() at the end ... */

/* 4. Caller-provides-storage APIs */
int parser_init(Parser *p, uint8_t *buf, size_t buflen);   /* no allocation at all */

/* 5. Ring buffers for streaming */
static uint8_t g_uart_rx[256];   /* power of 2 → masking, not modulo */
```

Every one of these makes the program's total memory use a number the linker knows — and checks — at build time. A ring buffer whose size is a power of two can wrap its indices with `& (SIZE - 1)` instead of `% SIZE` (Module 4) — a single AND instead of a division, which on many small MCUs has no hardware support.

The discipline this forces

- **Size everything at compile time,** then `_Static_assert` that the totals fit.
- **Bound every input.** No VLAs, no `alloca` (Modules 7 and 10).
- **Compute the worst-case stack usage.** `-fstack-usage` makes GCC emit a `.su` file per source file, listing each function's frame size; tools like puncover and StackAnalyzer combine them along the call graph. Add the ISR nesting depth.
- **Fill unused RAM with a pattern at start-up, and check the high-water mark at run time** — the standard way to measure real stack usage: the lowest address whose pattern has been overwritten shows how deep the stack ever grew.

Recursion is banned too (MISRA Rule 17.2), for the same reason: unbounded stack use.

27.6 Fixed-Point Arithmetic

Many MCUs have no floating-point unit (FPU), or only a single-precision one. *Fixed-point* arithmetic represents fractional values as ordinary integers with an implied binary point. Measured on a Cortex-M4:

```c
typedef int32_t q16_t;                              /* Q16.16: 16 integer, 16 fractional bits */
#define Q16_ONE  (1 << 16)
#define Q16(x)   ((q16_t)((x) * (double)Q16_ONE))    /* compile-time only */

static inline q16_t q16_mul(q16_t a, q16_t b){ return (q16_t)(((int64_t)a * b) >> 16); }
static inline q16_t q16_div(q16_t a, q16_t b){ return (q16_t)((((int64_t)a) << 16) / b); }
```

In Q16.16 format, the real number x is stored as the integer $x \times 65{,}536$. Adding two such numbers is ordinary integer addition. Multiplying them gives a result scaled by 65,536 twice, so the product (computed in 64 bits, since it needs up to 64) is shifted right by 16 to restore the scale; division scales the dividend up first. `Q16(x)` converts a constant, and should only be used with compile-time constants, so that the floating-point arithmetic happens in the compiler rather than on the device.

(Strictly, left-shifting a negative value is undefined behaviour in C — Module 4 — so a fully portable `q16_div` would write `(int64_t)a * Q16_ONE` instead of `<< 16`; the compiler generates the same shift. Right-shifting a negative value is implementation-defined, and is an arithmetic shift on every compiler for these targets.)

The same computation, `x * 3.5 + 1.25`, three ways:

Fixed-point — six inline integer instructions, no library:

```
calc_fixed:
    mov.w r3, #229376         @ 3.5 in Q16.16
    smull r0, r3, r0, r3      @ 64-bit multiply
    lsrs  r0, r0, #16
    orr.w r0, r0, r3, lsl #16
    add.w r0, r0, #81920      @ 1.25 in Q16.16
    bx    lr
```

(3.5 × 65,536 = 229,376 and 1.25 × 65,536 = 81,920; `smull` produces the full 64-bit product in two registers, and the next two instructions extract the middle 32 bits — the shift by 16.)

Float with soft-float — two library calls:

```
calc_float:
    push {r3, lr}
    bl   __aeabi_fmul   ← software floating point
    bl   __aeabi_fadd
    pop  {r3, pc}

soft-float library pulled in:
  __aeabi_dadd __aeabi_dmul __aeabi_fadd __aeabi_fmul
```

Each floating-point operation is a call into a software library that emulates the arithmetic with integer instructions — tens to hundreds of cycles each. (The `__aeabi_d*` functions were pulled in by `double` arithmetic elsewhere in the program.)

Float with a hardware FPU — four VFP instructions, including one fused multiply-add:

```
calc_float:
    vmov.f32 s14, #12   @ 3.5
    vmov.f32 s15, #116  @ 1.25
    vfma.f32 s15, s0, s14   ← fused multiply-add, single instruction
    bx lr
```

(The `#12` and `#116` are the instruction's compact encodings of the constants 3.5 and 1.25.) With a hardware FPU, single-precision floating point is as cheap as fixed point, and much easier to write correctly. Fixed point earns its keep on parts without an FPU, and where deterministic, bit-exact results matter.

The rules

Rule	Why
Never use `double` on a 32-bit MCU	`-mfloat-abi=hard -mfpu=fpv4-sp-d16` is single-precision only — every `double` silently falls back to software. `3.14` is a `double`; write `3.14f`.

Rule	Why
`-Wdouble-promotion`	Catches accidental promotion to `double` (Module 19)
Use the `float` variants	`sinf`, `sqrtf`, `fabsf` — not `sin`, `sqrt`, `fabs`
Q-format for sensor maths	Deterministic timing, no library, exact over the range you chose
64-bit intermediates	`((int64_t)a * b) >> 16` — otherwise `q16_mul` overflows immediately
Watch saturation	Fixed point overflows silently (and signed overflow is UB — Module 2); add saturating helpers where inputs are unbounded
FPU + ISRs	If an ISR uses the FPU, the FPU registers must be saved (Cortex-M "lazy stacking"). Getting this wrong corrupts floating-point state non-deterministically.

27.7 Peripherals and Protocols

Peripheral	Key points
GPIO	General-purpose I/O pins. Mode (input/output/alternate function/analogue), pull-up/pull-down, speed, open-drain. Use BSRR for atomic set/clear.
UART	Asynchronous serial, no shared clock. Both ends must agree on baud rate, parity, and stop bits. Errors: framing, parity, and *overrun* (you read too slowly). Use DMA plus idle-line detection to find message boundaries.
SPI	Synchronous, master/slave, full-duplex, with a chip-select line per device. Fast (tens of MHz). The clock polarity/phase mode (CPOL/CPHA, modes 0–3) must match the device datasheet — a mismatch gives silent garbage.
I²C	Two wires, addressed, multi-drop, open-drain (needs pull-up resistors). Slow (100 kHz / 400 kHz / 1 MHz). Prone to bus lock-up — a slave holding SDA low needs a recovery sequence (clock out 9 pulses). Always implement timeouts.
CAN	Multi-master, differential signalling, arbitration by message ID, built-in error handling. The automotive standard; CAN-FD for higher data rates.
ADC	Analogue-to-digital converter. Resolution (12-bit is typical), sampling time, reference voltage. Needs calibration, oversampling to reduce noise, and often DMA + timer triggering.

Peripheral	Key points
PWM	A timer with a compare register: duty cycle = compare / period. Motor control, LED dimming, cheap DAC emulation.
DMA	Moves data without the CPU. ⚠ Buffers must be in DMA-accessible memory, correctly aligned, and cache-maintained on cached parts. The buffer must be `volatile`, and must not be a stack local that goes out of scope while the transfer is running.
Timers	Free-running counters, input capture (measuring pulse widths), output compare, encoder mode.
Watchdog	Resets the MCU if not "kicked" in time. ⚠ Kick it from the main loop only after verifying that all tasks are alive — a watchdog kicked from a timer ISR protects nothing, because the ISR keeps running while the main loop is hung.

The DMA warning deserves emphasis: the DMA controller writes to memory independently of the CPU, possibly long after the function that started it has returned. If the buffer was a local variable, the controller is now writing into whatever stack frame occupies that space — a dangling-pointer bug (Module 9) committed by hardware.

The universal peripheral patterns

- **Polling** — the simplest; wastes CPU time; fine for initialisation and low data rates.
- **Interrupt-driven** — one interrupt per byte or word. Good up to moderate rates.
- **DMA** — zero CPU work per byte. Essential above about 1 Mbit/s. Combine it with *double-buffering* (using half-transfer and transfer-complete interrupts to process one half of a buffer while DMA fills the other) for continuous streams.

Always implement timeouts. A peripheral that never sets its "ready" bit — because of a wiring fault, a misconfiguration, or a hung device — will otherwise hang your firmware forever:

```
uint32_t deadline = now_ms() + 100;
while (!(SPI1->SR & SPI_SR_TXE)) {
    if (time_after(now_ms(), deadline)) return -ETIMEDOUT;
}
```

`time_after` compares the times in a way that stays correct when the millisecond counter wraps around — typically `(int32_t)(a - b) > 0`, relying on unsigned wraparound (Module 2) — so the timeout keeps working after 49.7 days of uptime.

27.8 RTOS

A *real-time operating system* is a small kernel that provides tasks, scheduling, and synchronisation on a microcontroller. "Real-time" means *predictable*, not fast: the guarantee is that work completes before its deadline.

RTOS	Notes
FreeRTOS	The most widely deployed. Small, MIT-licensed, maintained by AWS. Tasks, queues, semaphores, mutexes, timers, event groups, stream buffers.
Zephyr	A Linux Foundation project. Much larger scope: drivers, networking, Bluetooth, device tree, Kconfig. The modern choice for connected devices.
ThreadX	Formerly Azure RTOS (Microsoft), now Eclipse ThreadX under the Eclipse Foundation. Safety-certified (IEC 61508 SIL 4, ISO 26262 ASIL D, DO-178C).
RT-Thread, NuttX, ChibiOS, µC/OS	Regional and niche strengths

The concepts

Task — a function with its own stack, run by the scheduler. Statically allocate task stacks, and size them from `-fstack-usage` analysis.

Preemptive priority scheduling — the highest-priority task that is ready to run always runs. Tasks of equal priority may round-robin.

Queues — the primary inter-task mechanism. Items are copied by value (safer than passing pointers), operations block with a timeout, and ISR-safe variants exist (`xQueueSendFromISR`).

Semaphores vs mutexes — a semaphore *signals* (typically from an ISR to a task: "data is ready"); a mutex provides mutual exclusion, *with priority inheritance*.

⚠ Priority inversion

```
Low-priority task L takes mutex M.
High-priority task H blocks waiting on M.
Medium-priority task Mp preempts L (it's higher priority).
→ H is now effectively blocked by Mp, which it outranks. Unbounded delay.
```

This is not hypothetical. In 1997, NASA's Mars Pathfinder lander began resetting itself repeatedly after landing. A low-priority meteorological task held a mutex needed by a high-priority bus-management task, while medium-priority communications tasks kept running; a watchdog noticed that the high-priority task had missed its deadline and reset the system. Engineers diagnosed the problem on an identical system on Earth, and fixed it by uploading a change that enabled priority inheritance on that mutex.

Solutions: *priority inheritance* — while L holds a mutex that H is waiting for, L temporarily runs at H's priority, so Mp cannot preempt it (FreeRTOS mutexes do this; its semaphores do not); a *priority ceiling* — the mutex raises any holder to a fixed high priority; or, best of all, don't share resources between tasks of different priorities.

RTOS rules

1. **Use the `...FromISR` variants in ISRs, always**, plus the `xHigherPriorityTaskWoken` yield idiom (which switches immediately to a higher-priority task that the ISR woke).

2. **Never block in an ISR.**
3. **Size every task stack, and enable stack-overflow checking in development.**
4. **Watch for deadlock** — the same global lock-ordering rule as Module 24.
5. **Prefer message passing to shared state.** Queues eliminate most synchronisation bugs.
6. **Measure worst-case response time, not the average.** Real-time means meeting deadlines, not raw speed.
7. **You may not need an RTOS.** A *superloop* — a main loop driving a state machine, with interrupt-driven I/O — is simpler, smaller, and easier to verify. Add an RTOS when you genuinely have concurrent activities with different deadlines.

27.9 Low Power, Bootloaders, OTA

Low power

```
__DSB();   /* flush pending writes BEFORE sleeping */
__WFI();   /* Wait For Interrupt — core clock stops until an IRQ */
```

Power modes range from *sleep* (the core is off, peripherals stay on, and waking takes microseconds) through *stop* (most clocks are off, RAM is retained) to *standby* (RAM is lost, and waking resets the MCU).

The technique: be asleep by default. Wake on an interrupt, do the minimum necessary, and go back to sleep. A well-written sensor node spends more than 99% of its time asleep, and runs for years on a coin cell. Measure with a current probe or a power profiler such as Nordic's PPK: *average* current is what matters, and it is dominated by how long you are awake, not by how fast you compute — which is why finishing quickly at a high clock speed and then sleeping often uses less energy than running slowly.

Also: gate the clocks of unused peripherals, scale the clock frequency dynamically, and configure unused GPIO pins (floating inputs leak current).

Bootloaders and OTA

A minimal bootloader lives at the start of flash, validates the application image, and jumps to it:

```
uint32_t app_sp    = *(volatile uint32_t*)APP_BASE;
uint32_t app_reset = *(volatile uint32_t*)(APP_BASE + 4);
__set_MSP(app_sp);
SCB->VTOR = APP_BASE;   /* relocate the vector table */
((void(*)(void))app_reset)();
```

This performs, in software, exactly what the hardware does at reset (§27.3): read the application's own vector table, load its initial stack pointer, point the *VTOR* (vector table offset register) at the application's table so that its interrupt handlers are used, and call its reset handler. (A production bootloader also disables interrupts and resets the peripherals it used before jumping, so that the application starts from a clean state.)

A production over-the-air (OTA) update must be:

Requirement	Mechanism
Atomic	A/B dual-bank flash — write to the inactive bank, and swap only on success
Verified	A cryptographic signature checked before jumping (a secure boot chain)
Rollback-capable	Boot the new image with a “trial” flag; a watchdog reset reverts to the known-good bank
Interruptible	A power loss mid-update must leave a bootable device
Anti-rollback	A monotonic version counter in one-time-programmable fuses, so that an attacker cannot install an old, vulnerable version

Never write a bootloader from scratch for production. Use MCUboot (the de facto standard, used by Zephyr), or your vendor’s certified equivalent.

27.10 Debug Hardware

Tool	Purpose
SWD / JTAG	The debug port. SWD is 2 wires (SWDIO/SWCLK) plus ground; JTAG is 4–5. Gives full memory access, breakpoints, and single-stepping — even with the core halted.
Probes	ST-Link, J-Link (the best software, with SWO and RTT), CMSIS-DAP/DAPLink (open), Black Magic Probe (with a built-in GDB server)
OpenOCD / pyOCD	Open-source GDB servers: `openocd -f board.cfg`, then `arm-none-eabi-gdb -ex 'target remote :3333'`
SWO / ITM	Single-wire trace output — `printf`-style debugging at MHz rates, without a UART
SEGGER RTT	A ring buffer in RAM read by the probe while the program runs. Very fast; no dedicated pin needed.
ETM trace	Full instruction trace. Needed for hard timing bugs and for certification coverage.
Logic analyser	Saleae, or a $10 clone with sigrok. Indispensable for SPI/I²C/UART protocol bugs — it decodes the bus and shows you what actually went over the wire.
Oscilloscope	Analogue reality: signal integrity, ringing, rise times, glitches, power rails
Current probe	Power profiling (Nordic PPK2, Joulescope)

The GDB skills from Module 21 carry over directly: the probe and GDB server make the microcontroller look like a remote process to `arm-none-eabi-gdb`.

> **The debugging skill that distinguishes embedded engineers: when the code looks right, look at the signal.** Half of all "firmware bugs" are a floating pin, a missing pull-up resistor, the wrong SPI mode, a brown-out, or a noisy power rail.

Also: write a proper **hard-fault handler.** Capture the registers the hardware stacked, and decode the fault-status registers `SCB->CFSR`, `HFSR`, and `BFAR`. That turns "it hangs" into "unaligned access at `0x0800_1234`."

27.11 MISRA C and Safety Standards

MISRA C:2012 / 2023

MISRA C is a set of about 175 guidelines for safety-critical embedded C, in three categories: **mandatory** (never violated), **required** (violated only with a documented deviation), and **advisory**.

Representative rules:

Rule	Content
8.13	Pointers should point to `const` wherever possible
17.2	No recursion
21.3	No `malloc/free`
15.5	A single point of exit from a function
15.6	Compound statements (braces) required for every `if` and loop body
14.4	The controlling expression of an `if` shall be essentially Boolean
10.x	The "essential type" model — heavy restrictions on implicit conversions (Module 2's traps, systematised)
11.x	Restrictions on pointer casts
20.x	Heavy restrictions on the preprocessor
1.3	No undefined or critical unspecified behaviour (Module 16)

Deviations are permitted, but must be documented, justified, and approved. MISRA does not mean "no exceptions"; it means "no *undocumented* exceptions."

> ⚠ **MISRA is a coding standard, not a proof of correctness.** Compliant code can still be wrong. And some rules — single exit, no `goto` — arguably make code *worse* (compare Module 5's `goto` cleanup chain), which is exactly what the deviation process is for.

Tools: PC-lint Plus, Helix QAC, LDRA, Polyspace, Parasoft, Coverity, and — for partial coverage — clang-tidy's `cert-*` checks.

The certification standards

Standard	Domain	Levels
ISO 26262	Automotive	ASIL A–D
DO-178C	Avionics	DAL A–E; Level A requires MC/DC coverage
IEC 61508	Industrial functional safety	SIL 1–4
IEC 62304	Medical devices	Class A/B/C
EN 50128	Railway	SIL 0–4

The levels scale with the consequences of failure: ASIL D, DAL A, and SIL 4 cover systems whose failure could kill people.

What certification demands: full *requirements traceability* (every line of code traces to a requirement, and every requirement to a test), structural coverage (statement → branch → MC/DC, Module 22), a *qualified* toolchain, documented deviations, independent verification, and a configuration-management trail for everything.

This is why safety-critical C moves slowly and costs what it does — and why the roles pay accordingly.

27.12 CMSIS, HAL, and Register-Level Code

The layers

```
Application
    ↕
Vendor HAL / LL drivers         (STM32 HAL, nRF SDK, ESP-IDF, Zephyr drivers)
    ↕
CMSIS-Core + device header      (register definitions, __DSB, NVIC_*, __WFI)
    ↕
Hardware registers
```

CMSIS (ARM's *Cortex Microcontroller Software Interface Standard*) provides CMSIS-Core (register definitions, intrinsics such as `__DSB()` and `__WFI()`, and NVIC access — vendor-neutral), CMSIS-DSP (optimised filters, transforms, and matrix maths), CMSIS-NN (neural-network kernels), CMSIS-Driver, and CMSIS-Pack. A *HAL* (hardware abstraction layer) is the chip vendor's higher-level driver library on top.

HAL vs registers

	Vendor HAL	Register-level
Time to first blink	Minutes	Hours
Code size	Large (often 10–50× for the same function)	Minimal
Determinism	Hidden state machines, hidden timeouts	Exactly what you wrote
Portability	Within one vendor's family	Rewrite per part
Debuggability	Layers of abstraction to step through	Direct
Correctness	Vendor HALs have a mixed reputation for bugs	Yours to get right

The pragmatic position

Use the HAL for initialisation boilerplate (clock trees are genuinely hard), prototyping, and complex stacks (USB, TCP/IP, Bluetooth) that you would never write yourself.

Drop to registers for hot paths, ISRs, anything with tight timing, anything you need to certify, and anything where the HAL's behaviour is unclear.

Regardless: always use the CMSIS device header for register definitions, rather than hand-written `#define`s. It is generated from the vendor's SVD file (a machine-readable description of every register) and it is correct.

> **Read the reference manual.** The single most important embedded skill is being able to open a 2,000-page reference manual, find the register description, and understand what each bit does. No abstraction layer removes that need — it only delays it until something breaks.

Module 27 Rules

#	Rule
1	Every hardware register is `volatile` — without it the compiler deletes your I/O
2	Use atomic set/reset registers (BSRR) instead of read-modify-write on ODR
3	`volatile` gives no ordering, atomicity, or cache coherence — add `__DSB`/`__DMB` and barriers
4	ISRs: short, `volatile` shared data, no `malloc`/`printf`/blocking, clear the flag

#	Rule
5	32-bit accesses are atomic on ARM; wider ones need a critical section or a double read
6	Save and restore PRIMASK — never blindly re-enable interrupts
7	`KEEP()` the vector table so `--gc-sections` can't remove it
8	`> RAM AT> FLASH` — `.data` runs in RAM, is stored in flash, and is copied by `Reset_Handler`
9	You write the C run-time: copy `.data`, zero `.bss`, then call `main` — which must never return
10	Always generate a map file and a `-fstack-usage` report
11	Ban `malloc`, VLAs, `alloca`, and recursion — static allocation with compile-time sizing
12	Never `double` on a 32-bit MCU; `-Wdouble-promotion`, and `f`-suffixed literals and functions
13	Fixed point needs 64-bit intermediates; watch saturation
14	Every peripheral wait needs a timeout
15	Kick the watchdog from the main loop, after verifying all tasks are alive
16	RTOS mutexes have priority inheritance; semaphores do not — remember Pathfinder
17	Prefer message passing to shared state; you may not need an RTOS at all
18	OTA must be atomic, signed, rollback-capable, and power-loss safe — use MCUboot
19	When the code looks right, look at the signal
20	MISRA deviations are allowed, but must be documented and approved
21	Read the reference manual

Practice

1. **Prove `volatile` matters.** Install `gcc-arm-none-eabi`. Write a GPIO toggle with and without `volatile`. Dump both at `-O2` and count the memory accesses. Then do the same for a status-polling loop, and find the infinite loop.

2. **Build the firmware.** Reproduce the complete §27.4 build: linker script, start-up code, vector table, and `main`. Verify that word 0 of the vector table is the top of the stack, word 1 is `Reset_Handler | 1`, and `.data`'s VMA is in RAM while its LMA is in flash. Read the `Reset_Handler` disassembly and identify both loops.

3. **Break the linker script.** (a) Remove `KEEP()` from `.isr_vector` and build with `--gc-sections` — what happens? (b) Set the RAM `LENGTH` to 4K and confirm that the `._check` section fails the link. (c) Forget `AT> FLASH` on `.data`, and explain what the image would then do.

4. **Run it.** Install `qemu-system-arm` and run your firmware on a `-machine lm3s6965evb` or `netduinoplus2` target. Add a semihosting or UART `putchar` and get "hello" out. Then attach GDB (`-s -S`) and single-step `Reset_Handler`.

5. **Torn read.** Write a 64-bit tick counter, incremented by a simulated ISR and read from the main loop. Prove the tearing with a test that detects a non-monotonic sequence. Fix it three ways: a double-read loop, a PRIMASK critical section, and splitting the counter into two 32-bit halves with a sequence counter.

6. **Stack budget.** Build a firmware with `-fstack-usage`. Sum the worst-case call chain by hand. Then fill RAM with `0xAA` at start-up and add a run-time high-water-mark reporter. Compare the static estimate with the measured value.

7. **Fixed-point library.** Implement Q16.16 with add, sub, mul, div, sqrt, sin (via a lookup table plus interpolation), and saturating variants. Test them against `double` for accuracy across the range. Compare code size and cycle counts with soft-float and hard-float on a Cortex-M4.

8. **The `double` trap.** Write `float x = y * 3.14;` (with no `f` suffix) and build with `-mfloat-abi=hard -mfpu=fpv4-sp-d16`. Find the `__aeabi_d*` calls in the disassembly. Then enable `-Wdouble-promotion` and fix it.

9. **Ring buffer + ISR.** Implement a lock-free single-producer/single-consumer ring buffer for UART reception, with the ISR as producer and the main loop as consumer. Show why it needs no critical section on a single-core MCU when the size is a power of two and each index is word-sized and written by only one side. Verify with a stress test.

10. **Priority inversion.** In FreeRTOS (or a simulation), reproduce the Pathfinder scenario with three tasks and a shared resource. Measure the high-priority task's delay. Then switch from a binary semaphore to a mutex and measure again.

11. **MISRA audit.** Take a driver you have written and check it against 10 MISRA rules (17.2, 21.3, 15.5, 15.6, 14.4, 8.13, 11.3, 20.7, 10.3, 1.3). Document each violation with either a fix or a written deviation justification.

12. **Hard-fault decoder.** Write a `HardFault_Handler` in assembly that captures the stacked frame and passes it to a C function that prints PC, LR, PSR, CFSR, HFSR, and BFAR. Trigger it with an unaligned access, a null dereference, and a divide by zero, and confirm that you can identify each. (On Cortex-M, integer division by zero only faults if the `DIV_0_TRP` bit in `SCB->CCR` is set — otherwise it silently returns 0 — so enable that bit first.)

Module 27 in Summary

A microcontroller is a CPU, flash, RAM, and peripherals, all reached through a single memory map: hardware registers are just addresses, overlaid with `volatile` structures, and without `volatile` the compiler collapses your I/O into nothing — though `volatile` still gives no ordering, atomicity, or cache coherence, for which barriers and set/reset registers exist. Interrupts dispatch through a vector table of function pointers; ISRs must be short and non-blocking, clear their flags, and share data only through `volatile` variables protected against torn reads by double-reads or PRIMASK critical sections that save and restore the previous state.

On bare metal you supply the C run-time: a linker script places code and constants in flash and variables in RAM — with `.data` stored in flash and copied at start-up — and a reset handler copies `.data`, zeroes `.bss`, and calls a `main` that never returns, producing images of a few hundred bytes. Dynamic allocation, recursion, VLAs, and `alloca` are banned in favour of statically sized pools, arenas, and ring buffers, with worst-case stack use computed and measured. Without an FPU, fixed-point arithmetic replaces slow software floating point, and `double` must be avoided even with one.

Peripherals — GPIO, UART, SPI, I²C, CAN, ADC, PWM, DMA, timers, watchdogs — are driven by polling, interrupts, or DMA, always with timeouts. An RTOS adds tasks, queues, and mutexes with priority inheritance (the lesson of Mars Pathfinder), but a superloop is often enough. Devices sleep by default, boot through verified, rollback-capable bootloaders such as MCUboot, and are debugged through SWD, trace, logic analysers, and oscilloscopes. Safety-critical work adds MISRA C and certification standards with full traceability and MC/DC coverage. Use vendor HALs where they save effort, registers where precision matters, the CMSIS headers always — and read the reference manual.

Next: Module 28 — Software Design in C, which pulls the opaque handles, function-pointer vtables, callbacks, and intrusive structures you have met into a coherent architectural toolkit: how to build modules, plugin systems, state machines, and generic containers in a language with no classes, no templates, and no namespaces.

Module 28 — Software Design in C

In this module:

- Modularity: one module = header + implementation, with clear boundaries
- Opaque-handle APIs (`typedef struct Db Db; Db *db_open(...)`)
- Polymorphism via function-pointer vtables
- Callbacks with context/user-data pointers
- Object lifecycle: init/deinit, create/destroy, ownership rules
- State machines (table-driven and switch-based)
- Plugin systems, dependency injection
- Generic containers: `void *`, macro templates, intrusive lists
- Layering, avoiding global state, testability
- API design: naming, versioning, deprecation, error reporting, thread-safety documentation

C has no classes, no templates, no namespaces, no destructors, and no module system. Every architectural technique in this course is built from just four primitives: **structures, function pointers, opaque types, and the linker.** This module gathers the patterns you have met one at a time — opaque handles (Module 11), vtables and callbacks (Module 6), intrusive lists (Module 11), X-Macros (Module 12), dependency injection (Module 22) — into a coherent toolkit for designing real C programs. All output below comes from real, ASan-clean builds.

Large, long-lived C codebases — the Linux kernel, SQLite, Redis, curl, PostgreSQL — are not maintainable *despite* C's lack of these features. They are maintainable because their authors used these patterns consistently.

28.1 Modularity

> **One module = one header + one implementation.** The header is the contract; the `.c` file is the secret.

```
widget.h   ← the ONLY thing consumers see
widget.c   ← everything else: static helpers, private structs, globals
```

Rule	Why
`static` everything by default	Remove it only when a symbol must be shared (Module 3)
Prefix every public identifier	C has no namespaces: `widget_create`, `WIDGET_MAX`, `WidgetKind`
Include your own header first in the `.c`	Proves the header is self-contained (Module 20)
Forward-declare rather than include in headers	`struct Db;` instead of `#include "db.h"`
One responsibility per module	If the header has 40 unrelated functions, split it

Rule	Why
No circular dependencies	If A depends on B and B depends on A, they are really one module
`-fvisibility=hidden` for shipped libraries	Only the marked API is exported (Module 20)

The test: can you describe the module in one sentence, and does its header fit on one screen? If not, split it.

28.2 Opaque Handles

Module 11, §11.8 established the mechanism. Here is the design guidance.

```c
/* db.h — the consumer sees only this */
typedef struct Db Db;   /* incomplete type */

Db  *db_open(const char *path);
void db_close(Db *db);   /* accepts NULL */
int  db_query(Db *db, const char *sql, Row **out);

/* db.c — the layout is private */
struct Db { unsigned magic; int fd; Cache cache; Stmt *stmts; size_t nstmts; };
```

What you get:

Benefit	Consequence
ABI stability	Add, remove, or reorder fields without recompiling consumers (Module 20 measured what happens without this)
Real invariants	State changes only through your functions
Fewer header dependencies	The header need not include the types your fields use
Faster builds	Layout changes don’t rebuild the world

The cost: one allocation per object, and a function call per field access. When that is unacceptable — in embedded code (Module 27), or in hot inner loops — expose the structure, but document that only the API may touch its members. That is the weaker, convention-only guarantee that `pthread_mutex_t` and (in glibc) `FILE` provide: the layout is visible, so objects can be embedded and statically initialised, but it is not part of the contract.

Every serious C library works this way: `FILE *`, `sqlite3 *`, `SSL_CTX *`, `png_structp`, and curl’s `CURL *` handles.

28.3 Polymorphism via Vtables

Polymorphism means code written against an interface works with any implementation of it. A *vtable* ("virtual function table") is a structure of function pointers, one per operation; every object carries a pointer to its type's vtable, and calling an operation means calling through that pointer. This is exactly what a C++ compiler generates for virtual functions. Doing it by hand costs about 30 lines and buys you full control.

```
/* stream.h — the interface */
typedef struct Stream Stream;

typedef struct StreamVTable {
    const char *name;
    ptrdiff_t (*read) (Stream *s, void *buf, size_t n);
    ptrdiff_t (*write)(Stream *s, const void *buf, size_t n);
    int       (*flush)(Stream *s);
    void      (*close)(Stream *s);   /* frees the object itself */
} StreamVTable;

struct Stream { const StreamVTable *vt; };   /* every implementation embeds this FIRST */

/* dispatch */
static inline ptrdiff_t stream_write(Stream *s, const void *b, size_t n)
{ return s->vt->write(s, b, n); }
static inline int stream_flush(Stream *s)
{ return s->vt->flush ? s->vt->flush(s) : 0; }   /* optional methods → NULL check */
```

An implementation embeds `Stream` as its *first* member, so that converting between `Stream *` and `MemStream *` is valid:

```
typedef struct {
    Stream base;   /* MUST be first */
    char *buf; size_t len, cap, pos;
} MemStream;

static ptrdiff_t mem_write(Stream *s, const void *b, size_t n) {
    MemStream *m = (MemStream*)s;   /* downcast */
    ...
}

static const StreamVTable MEM_VT = { "memory", mem_read, mem_write, NULL, mem_close };

Stream *memstream_create(size_t cap) {
    MemStream *m = calloc(1, sizeof *m);
    ...
    m->base.vt = &MEM_VT;
    return &m->base;
}
```

The C standard guarantees that a pointer to a structure, suitably converted, points to its first member, and vice versa — there is never padding before the first member. So `&m->base` and `m` are the same address, and each implementation can convert the `Stream *` it receives back to its own type. (For the general case, where the base is not first, Module 11's `container_of` does the same job.)

The client knows nothing about the implementations:

```
static void write_report(Stream *out, const char *title)
{
    stream_write(out, "=== ", 4);
    stream_write(out, title, strlen(title));
    ...
```

```
    stream_flush(out);
}
```

And *decorators* — streams that wrap other streams — compose for free:

```
Stream *tee = teestream_create(file_stream, mem_stream);   /* one write → two sinks */
write_report(tee, "to file AND memory");
stream_close(tee);                                        /* closes both children */
```

```
memstream (memory) captured 45 bytes:
=== to memory ===
  item 1 / item 2 / item 3

tee (tee) -> mirrored copy is 54 bytes

file contents:
  === to file AND memory ===
    item 1 / item 2 / item 3

ASan exit=0
```

The tee stream is itself a `Stream` whose `write` operation writes to both of its children. `write_report` has no idea it is writing to two places at once. That is the payoff of polymorphism: new behaviour without changing existing code.

The rules

1. **The base structure must be the first member.** That is what makes the cast valid (offset 0 is guaranteed).
2. **The vtable is `static const`** — one instance per type, in `.rodata`, shared by every object of that type.
3. **`NULL` entries mean "not supported"** — check before calling, or provide a default.
4. **`close`/`destroy` lives in the vtable,** so the caller does not need to know the concrete type to free an object.
5. **The cost:** one pointer per object, and one indirect call per operation. The indirect call defeats inlining (Module 19) — measure before using this pattern on a hot path.
6. **The alternative for a fixed, small set of types:** a tagged union (Module 11) with a `switch`. Faster, but not extensible — adding a type means editing every `switch`.

Real examples: the Linux kernel's `struct file_operations` and `struct net_device_ops`; SQLite's VFS layer (its file-system abstraction); GObject; and libcurl's protocol handlers.

28.4 Callbacks with Context

Module 6, §6.6 established this. The design rule:

> **Every callback signature takes a `void *userdata`. No exceptions.**

```
typedef void (*VisitFn)(const Item *item, void *ctx);
void tree_walk(Tree *t, VisitFn fn, void *ctx);
```

C has no closures. The context pointer is the only way a caller can attach state to a callback — omit it, and callers must use global variables, which destroys thread safety and testability.

`qsort` gets this wrong (Module 14) — which is why `qsort_r` and `qsort_s` exist as mutually incompatible platform extensions. Learn from the mistake.

Additional guidance:

- **Put the context last,** by convention, so that it reads like a trailing parameter.
- **Document reentrancy:** may the callback modify the container it is iterating? Call back into the API?
- **Never invoke a callback while holding a lock** (Module 24) — the callback might try to take the same lock.
- **Return a value from the callback if the caller might want to stop early:** `bool visit(...)`, where `false` means "stop."
- **A structure of function pointers plus one context is better than five separate callbacks** — that is an *interface* (§28.7).

28.5 Object Lifecycle

Two patterns, both useful

Pattern A — init/deinit: the caller owns the storage.

```
typedef struct { char *buf; size_t len, cap; } Buffer;

bool buffer_init(Buffer *b, size_t cap);   /* does NOT allocate the struct */
void buffer_deinit(Buffer *b);             /* does NOT free the struct */
```

```
Buffer on the stack, cap=32. Embeddable in another struct,
usable with no allocator at all (Module 27).
```

Use it when the object may live on the stack, be embedded in another structure, or be used where there is no heap (embedded systems). The type must be visible in the header, since the caller allocates it.

Pattern B — create/destroy: the module owns everything.

```
Session *session_create(const char *user);
void session_destroy(Session *s);             /* MUST accept NULL */
Session *session_retain(Session *s);          /* shared ownership */
const char *session_user(const Session *s);   /* BORROWED - do not free */
```

```
session_create("alice") -> user=alice refcount=1
after retain: refcount=2
after 1st destroy: refcount=1 (object still alive)
after 2nd destroy: freed
session_destroy(NULL) is a safe no-op
ASan exit=0
```

Use it for opaque types. Here `Session` also supports *reference counting*: each `retain` increments a count, each `destroy` decrements it, and the object is actually freed only when the count reaches zero — so several owners can share one object safely.

The two patterns compose: `Session` embeds a `Buffer` by value, and calls `buffer_init`/`buffer_deinit` within its own create and destroy.

The six rules

```c
static Session *session_create(const char *user)
{
    Session *s = calloc(1, sizeof *s);
    if (!s) return NULL;
    s->user = strdup(user);
    if (!s->user) { free(s); return NULL; }                                  /* unwind (Module 5) */
    if (!buffer_init(&s->io, 64)) { free(s->user); free(s); return NULL; }
    s->magic = SESSION_MAGIC; s->refcount = 1;
    return s;
}

static void session_destroy(Session *s)
{
    if (!s) return;                                                  /* 1. accept NULL */
    if (s->magic != SESSION_MAGIC) { /* UAF or type confusion */ return; }   /* 3. */
    if (--s->refcount > 0) return;                                   /* shared ownership */
    buffer_deinit(&s->io);
    free(s->user);
    s->magic = 0;                                                    /* 4. poison BEFORE free */
    free(s);
}
```

1. **`destroy()` always accepts `NULL`** — it makes cleanup chains trivial.
2. **Every failure path in `create()` unwinds what it acquired** — here written inline; the `goto` chain from Module 5 scales better as the number of resources grows.
3. **A magic field catches use-after-free and type confusion in debug builds.** (Reading the magic of a freed object is itself undefined behaviour — this is a best-effort debugging aid, not a guarantee; ASan is the real defence.)
4. **Poison the magic before `free`, not after** — after `free`, the memory is no longer yours to write.
5. **Getters return borrowed pointers** — document their lifetime in the header.
6. **Use reference counting only when shared ownership is genuinely required** — and the count must be atomic if threads are involved (Module 17).

28.6 State Machines

A *finite state machine* (FSM) is a system that is always in one of a fixed set of *states*, and moves between them in response to *events* according to fixed rules. Protocol handlers, parsers, user interfaces, and device controllers are all naturally state machines, and writing them explicitly as one makes their behaviour visible and checkable.

Table-driven

```c
typedef struct { State next; void (*action)(void*); } Transition;
#define KEEP { S_COUNT, NULL }   /* S_COUNT = "no transition" */
```

```
static const Transition TABLE[S_COUNT][E_COUNT] = {
    [S_CLOSED]      = {{S_LISTEN,NULL}, KEEP, KEEP, KEEP, KEEP, KEEP},
    [S_LISTEN]      = {KEEP, {S_SYN_RCVD,act_send_syn_ack}, KEEP, KEEP, KEEP, {S_CLOSED,act_cleanup}},
    [S_ESTABLISHED] = {KEEP, KEEP, {S_ESTABLISHED,act_deliver}, {S_FIN_WAIT,NULL}, KEEP, {S_CLOSED,act_
cleanup}},
    ...
};

static bool fsm_table(Conn *c, Event e)
{
    const Transition *t = &TABLE[c->st][e];
    if (t->next == S_COUNT) return false;   /* not accepted in this state */
    if (t->action) t->action(c);
    c->st = t->next;
    return true;
}
```

The whole machine is a two-dimensional table indexed by current state and event, and each cell says which state to move to and which action (if any) to run. Designated initializers (Module 7) tie each row to its state. The engine is six lines. (The example is a simplified TCP-style connection; `KEEP` marks an event that is not accepted in that state.)

States and events come from X-Macros (Module 12), with `_Static_assert`ed name tables:

```
STATE MACHINE: 6 states x 6 events

CLOSED      + OPEN    -> LISTEN
LISTEN      + SYN     -> SYN_RCVD
SYN_RCVD    + ACK     -> ESTABLISHED
ESTABLISHED + FIN     -> FIN_WAIT
FIN_WAIT    + ACK     -> TIME_WAIT
TIME_WAIT   + TIMEOUT -> CLOSED
```

Switch-based

```
switch (c->st) {
case S_LISTEN:
    if (e == E_SYN)   { act_send_syn_ack(c); c->st = S_SYN_RCVD; break; }
    if (e == E_RESET) { act_cleanup(c);      c->st = S_CLOSED;   break; }
    return false;
...
}
```

Measured comparison

```
differential test (200k random events): IDENTICAL
table-driven : 73.1 ms
switch-based : 73.6 ms
```

Performance is identical over 20 million transitions — a dense `switch` compiles to a jump table anyway (Module 5). The choice is entirely about maintainability.

Table-driven	Switch-based
Adding a state or event edits *data*	Adding a state or event edits *code*
The whole machine is visible at a glance	The logic is spread across cases
Verifiable and generatable — you can check completeness, draw a diagram, or emit the table from a specification	Easier to add per-case logic and guards

Table-driven	Switch-based
`-Werror=switch` doesn't help	`-Werror=switch` enforces exhaustiveness (Module 11)
Memory: states × events × `sizeof(Transition)`	No table

Use a table for protocol machines, parsers, and anything with many states and a specification to match. Use a `switch` for small machines with complex per-transition logic.

Note the *differential test* — Module 22's technique, used here to verify that the two implementations agree on 200,000 random events. That is how you refactor a state machine safely: keep the old one, write the new one, and let random inputs prove they behave identically.

Hierarchical state machines — in which a state can have substates that inherit its transitions — are worth knowing about for complex user interfaces and protocols; see the UML statechart model, and libraries such as QP/C.

28.7 Dependency Injection and Plugins

> Every external dependency becomes an explicit interface passed in. No globals, no direct system calls, no hidden state.

```
typedef struct { uint64_t (*now_ms)(void *ctx); void *ctx; } Clock;
typedef struct { void (*log)(void *ctx, const char *lvl, const char *msg); void *ctx; } Logger;
typedef struct { int (*send)(void *ctx, const char *dst, const char *body); void *ctx; } Mailer;

typedef struct { Clock clock; Logger logger; Mailer mailer; } Deps;

static int alerter_fire(Alerter *a, const Deps *d, const char *who, const char *msg)
{
    uint64_t t = d->clock.now_ms(d->clock.ctx);
    if (a->last_sent_ms && t - a->last_sent_ms < COOLDOWN_MS) {
        d->logger.log(d->logger.ctx, "DEBUG", "suppressed: cooldown");
        return 1;
    }
    if (d->mailer.send(d->mailer.ctx, who, msg) != 0) { ...; return -1; }
    ...
}
```

Each dependency is a small interface — a function pointer plus a context, exactly as §28.4 recommends — and `alerter_fire` reaches the outside world *only* through them. Its logic sends an alert, unless one was sent within the cooldown period.

Production wiring:

```
Deps prod = { {real_now,NULL}, {real_log,NULL}, {real_send,NULL} };
```

Tests wire in fakes — no mocking framework, no linker tricks, no `#ifdef TEST`:

```
FakeClock clk = {1000};
SpyMail ml = {0};
Deps test = { {fake_now,&clk}, {spy_log,&lg}, {spy_send,&ml} };
```

```
=== TESTS - no globals, no mocking framework, no time.h ===
  test 1: first alert is sent
  test 2: a second alert within the cooldown is suppressed
  test 3: after the cooldown it sends again
  test 4: a mailer failure is reported

  ALL PASS (0 failures)

  Note: test 2 required advancing time by 30 SECONDS.
  With a hard-coded clock() call the test would have to SLEEP.
```

> That last line is the whole argument. With a hard-coded `clock_gettime`, testing the cooldown means putting a 60-second `sleep()` in your test suite — or not testing it at all. With injection, `clk.t += 30000;` and the test is instant and deterministic.

Plugin systems

A plugin is the same idea, loaded at run time (Module 24, §24.10):

```c
/* the ABI: ONE versioned struct of function pointers */
#define PLUGIN_ABI_VERSION 3
typedef struct {
    uint32_t abi_version;
    const char *name;
    int  (*init)(void **state, const Config *cfg);
    int  (*process)(void *state, const Frame *in, Frame *out);
    void (*shutdown)(void *state);
} PluginAPI;

/* the ONE exported symbol */
PLUGIN_EXPORT const PluginAPI *plugin_entry(void);
```

Design rules:

1. **One exported symbol returning a versioned structure** — one `dlsym`, and one thing to version.
2. **Check `abi_version` before using anything else** in the structure.
3. **Keep per-instance state behind an opaque `void *`** returned by `init` — so one plugin can be instantiated several times.
4. **Build plugins with `-fvisibility=hidden`,** so that only the entry point is exported.
5. **Never `dlclose` while any plugin state or callback is still reachable.**

28.8 Generic Containers

Four approaches, all demonstrated working:

```
=== 1. void* + size (runtime generic, NO type safety) ===
  0 1 2 3 4
  cost: a cast at every access; wrong esz = silent corruption

=== 2. macro template (COMPILE-TIME generic, full type safety) ===
  IntVec: 0 1 4 9 16
  RecVec: 1:alpha 2:beta
  IntVec_push(&iv, "str") -> COMPILE ERROR (type-checked)

=== 3. intrusive list (ZERO allocations for the nodes) ===
  disk(10) net(20) cpu(30)
  offsetof(Item,link)=24 -> no node allocation, one object in N lists
```

```
=== 4. _Generic dispatch over the templates ===
  vec_push worked on both types: iv.len=6 rv.len=3
```

1. **`void *` + element size** is Module 9's approach, used by `qsort`: one implementation for every type, and no type checking at all.
2. **Macro templates** generate a separate, fully typed container for each element type.
3. **Intrusive lists** (Module 11) embed the links inside the objects themselves.
4. **`_Generic`** (Module 12) puts a single friendly name, `vec_push`, in front of the per-type functions.

The macro template

```c
#define VEC_DECLARE(NAME, T) \
    typedef struct { T *data; size_t len, cap; } NAME; \
    static inline void NAME##_init(NAME *v){ *v = (NAME){0}; } \
    static inline bool NAME##_push(NAME *v, T e){ \
        if (v->len == v->cap) { \
            size_t nc = v->cap ? v->cap*2 : 8; \
            T *p = realloc(v->data, nc*sizeof *p); if (!p) return false; \
            v->data = p; v->cap = nc; } \
        v->data[v->len++] = e; return true; } \
    static inline T NAME##_at(const NAME *v, size_t i){ return v->data[i]; } \
    static inline void NAME##_free(NAME *v){ free(v->data); *v=(NAME){0}; }

VEC_DECLARE(IntVec, int)
VEC_DECLARE(RecVec, Rec)
```

Token pasting (Module 12) generates a complete, type-safe, fully inlined container for each type: `VEC_DECLARE(IntVec, int)` produces the type `IntVec` and the functions `IntVec_init`, `IntVec_push`, `IntVec_at`, and `IntVec_free`. It uses every technique from earlier modules: doubling growth (Module 8), `realloc` into a temporary (Module 10), and `sizeof *p` so that the element type is never repeated. (For production use, the `nc * sizeof *p` multiplication also needs an overflow check — Module 23.)

Trade-offs

Approach	Type safety	Speed	Code size	Errors
`void *` + size	✗ none	an indirection + `memcpy` per access	one copy	run-time corruption
Macro template	✓ full	inlined, zero overhead	one copy per type	awful macro-expansion messages
Intrusive	partial	the best (no allocation, the best locality)	tiny	—
`_Generic` facade	✓ full	dispatches to any of the above	—	good

> ⚠ **A real trap hit while building this:** `vec_push(&rv, (Rec){3,"gamma"})` fails to compile. The comma inside the compound literal is taken by the preprocessor as an argument separator (Module 12, §12.4), so the macro receives three arguments instead of two. Protect such commas with an extra pair of parentheses, or use a named temporary.

Recommendation: intrusive structures for internal data structures where you own the element type; macro templates for reusable containers; `void *` only for genuinely dynamic cases (like `qsort`). Or use an existing library: stb_ds, klib, sc, or uthash.

28.9 Layering, Global State, Testability

Layering

```
application   ← business logic, knows nothing about I/O
     ↕
services      ← domain operations, depends on interfaces
     ↕
platform      ← the ONE place with #ifdef (Module 26)
     ↕
OS / hardware
```

Dependencies point in one direction only. An upper layer may call down; a lower layer talks back up only through callbacks or interfaces injected from above.

The test: can you compile and test the application layer with the platform layer replaced by fakes? If not, a dependency has leaked through the layers.

Global state

Global variables destroy thread safety (shared mutable state), testability (tests interfere with one another), reentrancy, and predictability of initialisation order (Module 3).

```c
/* BAD */
static Config g_config;
static Logger g_logger;
int process(const char *input);   /* hidden dependencies */

/* GOOD */
typedef struct { const Config *cfg; Logger log; Stats *stats; } Context;
int process(Context *ctx, const char *input);
```

In the good version, everything `process` depends on is visible in its signature — and a test can construct a `Context` with a test configuration, a capturing logger, and fresh statistics.

Where globals are defensible: genuinely process-wide *immutable* data (a lookup table, or configuration read once at start-up and never changed), and a single logging sink — and even then, thread-safe, and initialised through `call_once` (Module 17).

> ⚠ A function-local `static` has the same problems, plus non-reentrancy. It is why `strtok`, `gmtime`, and `strerror` need `_r` variants (Module 14).

Testability is a design property

Hard to test	Why	Fix
Reads the clock directly	Can't control time	Inject a `Clock`
Calls `malloc` directly	Can't test out-of-memory handling	Inject an allocator, or use fault injection
Reads a global	Tests interfere	Pass a context
Does I/O in the middle of its logic	Needs real files and sockets	Separate the pure logic from the I/O
One big function with many branches	Combinatorial test count	Split it
No error return	Can't verify failure handling	Return `err_t` (Module 15)

> **Code that is hard to test has hidden dependencies. Making it testable makes it better.** The dependency-injection example above is the proof: the same change that made time controllable also made the cooldown logic explicit and the mailer swappable.

28.10 API Design

Naming

```c
/* module_noun_verb, consistently */
widget_create / widget_destroy
widget_get_name / widget_set_name
widget_list_append / widget_list_remove

/* types: PascalCase or module_t — pick one */
typedef struct Widget Widget;

/* constants: MODULE_SCREAMING */
#define WIDGET_MAX_NAME 64
enum { WIDGET_FLAG_HIDDEN = 1u << 0 };
```

Prefix everything. C has no namespaces, and an unprefixed `min`, `max`, or `list_add` in a public header will eventually collide with someone else's (Module 12). (If you choose the `module_t` style for types, remember that POSIX reserves names ending in `_t` — Module 11.)

Parameter conventions

```c
/* object first, output last, context last */
int widget_render(Widget *w, const Canvas *c, Rect *out_bounds, void *userdata);

/* const on everything you don't modify */
size_t widget_serialize(const Widget *w, uint8_t *buf, size_t buflen);
```

```c
/* lengths travel with pointers (Module 23) */
int widget_set_data(Widget *w, const uint8_t *data, size_t len);
```

Error reporting

Use Module 15's design: an error code as the return value, out-parameters for results, `[[nodiscard]]` (Module 18) on everything that can fail, and every error code a function can return documented in the header.

Documentation that must be in the header

```c
/**
 * Render the widget onto `canvas`.
 *
 * @param w      Must be non-NULL and initialized.
 * @param canvas Not modified. Must outlive this call.
 * @param out    On success, receives the bounds. Untouched on failure.
 * @return ERR_OK, ERR_INVAL (null args), ERR_NOMEM, ERR_IO.
 *
 * Thread safety: NOT thread-safe. Callers must serialize access to `w`.
 * Reentrancy:    Must not be called from a widget callback.
 * Allocation:    Does not allocate.
 */
[[nodiscard]] err_t widget_render(Widget *w, const Canvas *canvas, Rect *out);
```

Four things every public function's documentation must state:

1. **Ownership** — who frees what (Module 10).
2. **Lifetime** — how long returned pointers stay valid.
3. **Thread safety** — safe, not safe, or safe on different objects.
4. **Every error return.**

Versioning and deprecation

Concern	Approach
Semantic versioning	MAJOR = ABI break, MINOR = additions, PATCH = fixes (Module 20)
ABI stability	Opaque handles; bump the soname on any break
Deprecation	`[[deprecated("use widget_render2()")]]` — warn for at least one major release cycle before removing
Feature detection	`MYLIB_VERSION` macros, a run-time `mylib_version()`, and `MYLIB_HAS_FEATURE_X`
Extensibility	Options structures with a size field, so new fields can be added compatibly
Compile-time checks	`_Static_assert(sizeof(PublicStruct) == 32, "ABI changed")`

The "options structure with a size field" pattern deserves a word: the caller sets `opts.size = sizeof opts` before passing it in, and the library reads only as many fields as that size covers. A program compiled against an older, smaller version of the structure

keeps working with a newer library, which simply uses defaults for the fields the old program did not know about. The Windows API uses this pattern extensively (the `cbSize` fields).

The design test

Before shipping an API, ask:

- **Can it be misused?** If misuse is easy, redesign it (Module 23).
- **Is the safe path the easy path?**
- **Can a caller forget to clean up?** (Make `destroy` accept `NULL`; support cleanup chains.)
- **Is every error distinguishable?**
- **Can it be tested without the real world?**
- **Can you change the implementation without breaking callers?**
- **Does it compose with itself?** (The tee stream is the test.)

Module 28 Rules

#	Rule
1	One module = header + implementation; `static` by default; prefix everything
2	Opaque handles for every non-trivial type — ABI stability and real invariants
3	Vtable = a `static const` structure of function pointers; base structure first
4	Every callback signature takes a `void *userdata`
5	`destroy()` always accepts `NULL`; poison the magic before `free`
6	init/deinit when the caller owns the storage; create/destroy when the module does
7	Every `create` failure path unwinds what it acquired
8	Table-driven FSMs for protocols; `switch` for small machines with complex guards
9	Differential-test two FSM implementations against each other
10	Inject every external dependency — clock, allocator, I/O, logger
11	Plugins export one versioned structure; check the version before use
12	Intrusive for internal use, macro templates for reusable containers, `void *` only when genuinely dynamic

#	Rule
13	Protect commas inside macro arguments
14	Dependencies point one way; the application layer must be testable with fakes
15	No global mutable state — pass a context
16	Document ownership, lifetime, thread safety, and every error return in the header
17	If it’s hard to test, the design is wrong

Practice

1. **Refactor into modules.** Take a single-file program of more than 500 lines and split it into 4 or more modules with clean headers. Make everything `static` that can be, prefix all public names, and verify with `nm` that each `.o` exports only what its header declares.

2. **Opaque conversion.** Take an API that exposes its structure and convert it to an opaque handle. Prove with a test file that `sizeof` and member access are now compile errors. Then change the internal layout, and confirm that consumers need only relinking, not recompiling.

3. **Build the stream hierarchy.** Implement the §28.3 `Stream` interface with memory, file, and null back ends. Then add a gzip decorator, a counting decorator (which tracks bytes written), and a buffering decorator. Verify with ASan that closing the outermost stream closes the whole chain exactly once.

4. **Lifecycle audit.** For every type in a project you have written, classify it as init/deinit or create/destroy. Add `NULL`-safety to every `destroy`, magic fields to every opaque type, and verify every `create` failure path with fault injection plus LSan (Module 22).

5. **Reference counting.** Add atomic reference counting to an opaque type. Write a test with 8 threads randomly retaining and releasing objects. Verify with TSan (Module 17) that there are no races, and with LSan that nothing leaks.

6. **State machine, both ways.** Implement a real protocol FSM (HTTP request parsing, or a vending machine with coins and change) as both a table and a `switch`. Differential-test them on 1 million random event sequences. Then add a new state to each, and compare the size of the diffs.

7. **FSM generator.** Write a Python script that reads a state-transition specification (CSV or YAML) and emits the C table, the enums, the name arrays, and the `_Static_assert`s. Verify that the generated code compiles and matches your hand-written version.

8. **Full dependency-injection refactor.** Take a program that calls `time()`, `malloc()`, `fopen()`, and a network function directly. Convert every one into an injected interface. Then write a test suite that runs with zero real I/O and zero sleeps, covering timeouts, allocation failure, partial reads, and network errors. Measure the coverage improvement.

9. **Plugin system.** Define a versioned plugin ABI. Write a loader that scans a directory, validates versions, and manages per-plugin state. Write three plugins with different behaviours. Handle load failure, version mismatch, `init` failure, and safe unloading while state is live.

10. **Container comparison.** Implement a dynamic array four ways (`void *`, macro template, intrusive-adjacent, and a `_Generic` facade). For each, measure the lines of code, the binary size for 3 element types, the per-operation time, and the quality of the error message when you pass the wrong type.

11. **API review.** Take a public header you have written. For every function, apply the §28.10 test: can it be misused? Is cleanup forgettable? Are errors distinguishable? Is thread safety documented? Rewrite the three worst functions, and write their documentation comments in full.

Module 28 in Summary

C programs are designed from structures, function pointers, opaque types, and the linker. A module is a header contract plus a private implementation, with everything `static` by default and every public name prefixed. Opaque handles give real encapsulation and ABI stability; vtables — `static const` structures of function pointers, with the base structure placed first — give polymorphism and composable decorators; and every callback carries a `void *` context in place of the closures C lacks.

Objects follow one of two lifecycles: init/deinit when the caller owns the storage, create/destroy when the module does, with `destroy` accepting `NULL`, every `create` failure path unwinding, magic fields catching misuse, and reference counting only where ownership is genuinely shared. State machines can be tables or `switch`es with identical performance, so choose by maintainability — and prove refactors with differential tests.

Injecting every external dependency — clock, logger, allocator, I/O — through small interfaces makes code testable without sleeps or mocking frameworks, and plugins apply the same idea at run time through one versioned structure. Generic containers trade type safety and code size between `void *`, macro templates, intrusive links, and `_Generic` facades. Layers depend in one direction, global mutable state gives way to explicit contexts, and every public API documents ownership, lifetime, thread safety, and errors, and evolves through semantic versioning and deprecation. If code is hard to test, the design is wrong.

Next: Module 29 — Data Structures & Algorithms in C, where you'll implement the working set — dynamic arrays, intrusive and non-intrusive lists, ring buffers, hash tables with both collision strategies, balanced trees, heaps, tries, and bitsets — using every design technique from this module, with sanitizer-verified correctness and measured performance.

Module 29 — Data Structures & Algorithms in C

In this module:

- Choosing a data structure: complexity, memory, and cache behaviour
- Dynamic arrays (a type-safe macro template with overflow-checked growth)
- Linked lists: intrusive vs non-intrusive, and when a list is the right answer
- Ring buffers (power-of-two, free-running indices)
- Hash tables: hash functions, open addressing with tombstones, separate chaining
- Binary heaps and priority queues
- Balanced binary search trees (AVL)
- Tries
- Bitsets (and the Sieve of Eratosthenes)
- Algorithms: overflow-safe binary search, `lower_bound`, stable merge sort vs `qsort`
- A measured comparison of every lookup structure on the same data

> **A note on this module.** The original roadmap listed this module, and Module 28 promised it, but its text was never written. This chapter was written to complete the book, in the same style as the others. Every data structure below lives in a single program (`Module29_ds.c`, supplied alongside this chapter), which was compiled with GCC 13.3 and run under AddressSanitizer and UndefinedBehaviorSanitizer (all checks passed, with no errors or leaks). The timings come from a separate `-O2` build with N = 1,000,000, on the same 2.1 GHz Xeon used in Module 25. As always: your numbers will differ; the shapes won't.

C's standard library offers exactly two algorithms — `qsort` and `bsearch` (Module 14) — and no containers at all. Every C programmer therefore ends up writing, or choosing a library for, the handful of data structures that almost every program needs. This module implements that working set, using the design techniques of Module 28 (macro templates, opaque state, overflow-checked sizes), the memory disciplines of Module 10, and the performance lessons of Module 25 — and verifies each one with sanitizers and invariant checks.

29.1 Choosing a Data Structure

Structure	Lookup	Insert	Delete	Ordered?	Memory / locality
Dynamic array	O(n) (O(log n) if sorted)	O(1) amortised at the end	O(n)	if kept sorted	Contiguous — the best locality

Structure	Lookup	Insert	Delete	Ordered?	Memory / locality
Linked list	O(n)	O(1) given a position	O(1) given the node	insertion order	One allocation per node (unless intrusive); pointer chasing
Ring buffer	— (FIFO only)	O(1)	O(1)	FIFO	Fixed, contiguous
Hash table	O(1) average	O(1) average	O(1) average	✗	Open addressing: contiguous; chaining: a node per entry
Binary heap	O(1) for the minimum	O(log n)	O(log n) for the minimum	partially	Contiguous array
Balanced BST	O(log n)	O(log n)	O(log n)	✓ in-order	A node per entry; pointer chasing
Trie	O(key length)	O(key length)	O(key length)	✓ lexicographic	Many small nodes
Bitset	O(1)	O(1)	O(1)	by index	1 bit per element

Big-O is only the start (Module 25, §25.1). The deciding questions are usually:

1. **What operations dominate?** Lookup by key, iteration in order, finding the minimum, first-in-first-out?
2. **How big is *n*, really?** For small *n*, a plain array beats everything.
3. **Do you need order?** Hash tables give none; trees and sorted arrays do.
4. **How much allocation can you afford?** Embedded code (Module 27) often forbids per-element `malloc`, which rules out naive lists, chained hash tables, and node-based trees — or requires pools.
5. **Does it change after it is built?** A structure built once and then only searched can be a sorted array, which is simple and compact.

> **The default in C is the dynamic array.** Reach for something else only when an operation you actually perform is too slow with one.

29.2 Dynamic Arrays

```
#define VEC_DECLARE(NAME, T)                                                \
    typedef struct { T *data; size_t len, cap; size_t grows; } NAME;        \
    static inline bool NAME##_reserve(NAME *v, size_t need) {               \
        if (need <= v->cap) return true;                                    \
        size_t nc = v->cap ? v->cap : 8;                                    \
        while (nc < need) { if (nc > SIZE_MAX / 2 / sizeof(T)) return false; nc *= 2; } \
        T *p = realloc(v->data, nc * sizeof *p);                            \
```

```
        if (!p) return false;                                                   \
        v->data = p; v->cap = nc; v->grows++; return true; }                    \
    static inline bool NAME##_push(NAME *v, T e) {                              \
        if (!NAME##_reserve(v, v->len + 1)) return false;                       \
        v->data[v->len++] = e; return true; }                                   \
    static inline void NAME##_free(NAME *v) { free(v->data); *v = (NAME){0}; }

VEC_DECLARE(IntVec, int)
```

```
=== 1. DYNAMIC ARRAY ===
  1000000 pushes in 1.52 ms, cap=1048576, reallocs=18 (doubling => O(log n))
```

This is Module 28's macro template, hardened in two ways. First, growth is separated into a `reserve` function, which callers can also use to pre-size the array when they know how many elements are coming — turning 18 reallocations into one. Second, the doubling loop refuses to grow past the point where `nc * sizeof(T)` would overflow (Module 23), instead of allocating a tiny buffer and writing past it.

A million pushes took 1.5 ms — about 1.5 ns each — with only 18 calls to `realloc`, because each doubling makes the next one twice as far away (the amortised O(1) growth of Module 8). The counter `grows` exists only to make that visible.

Practical notes:

- **Pointers into a dynamic array are invalidated by growth** (Module 9's `realloc` rule). Hand out indices, not pointers, if elements must be referred to across insertions.
- **Removing from the middle** is O(n) — `memmove` the tail down. If order does not matter, *swap-remove* instead: copy the last element into the hole and decrement `len`, which is O(1).
- **Keep it sorted,** and the array becomes a set or map with O(log n) lookup (§29.10) and the best locality of anything in this module.

29.3 Linked Lists

Module 11, §11.9 built an intrusive doubly linked list in the Linux kernel's style, with `container_of`, and compared it with the non-intrusive kind. The design points, gathered:

	Non-intrusive (`node → data`)	**Intrusive (links inside the object)**
Allocations per element	2	1 (0 extra)
Remove a known object	O(n) search unless you kept the node	O(1)
One object in several lists	several node allocations	one link field per list
Generic	needs `void *`	fully generic via `offsetof`

When is a list the right answer? Much less often than textbooks suggest. Walking a list is a chain of dependent loads to scattered addresses — the pattern Module 25's pointer-chase benchmark used precisely because it defeats the cache and the prefetcher. An array scan is usually faster even when it does "more work." Lists earn their place when:

- objects must be removed from the middle in O(1), given only the object (timers, LRU caches, scheduler run queues);
- objects must sit in several collections at once (intrusive lists);
- elements cannot move, because other code holds pointers to them;
- allocation must never fail at insertion time (the link is already inside the object).

The kernel's heavy use of intrusive lists satisfies all four. A general-purpose "list of things" usually satisfies none, and should be an array.

29.4 Ring Buffers

```c
#define RB_CAP 8u   /* must be a power of two */
_Static_assert((RB_CAP & (RB_CAP - 1)) == 0, "RB_CAP must be a power of two");
typedef struct { int buf[RB_CAP]; unsigned head, tail; } Ring;   /* free-running indices */

static bool rb_push(Ring *r, int v) {
    if (r->head - r->tail == RB_CAP) return false;   /* full */
    r->buf[r->head++ & (RB_CAP - 1)] = v; return true;
}
static bool rb_pop(Ring *r, int *out) {
    if (r->head == r->tail) return false;            /* empty */
    *out = r->buf[r->tail++ & (RB_CAP - 1)]; return true;
}
```

```
=== 2. RING BUFFER (cap 8) ===
  pushed 15, popped 15, rejected when full 5, order preserved
  unsigned index wraparound at UINT32_MAX: OK
```

A *ring buffer* (circular buffer) is a fixed-size FIFO queue: data is written at the head and read from the tail, and both wrap around the end of the array. It is the standard structure for streaming data between a producer and a consumer — a UART interrupt and the main loop (Module 27), a network thread and a worker, an audio callback and a decoder.

The implementation uses a trick worth understanding. `head` and `tail` are **free-running**: they are never reduced modulo the capacity, only masked when used as an index (`& (RB_CAP - 1)`, which equals `% RB_CAP` for a power of two — Module 4). The number of stored items is then simply `head - tail`, and:

- **empty** is `head == tail`;
- **full** is `head - tail == RB_CAP`.

The classic alternative — wrapping the indices themselves — cannot distinguish "full" from "empty" (both have `head == tail`), and must waste one slot or keep a separate count. With free-running indices, all `RB_CAP` slots are usable.

The second line of output tests the subtle part: the indices were started just below `UINT32_MAX`, so that they overflowed during the test. Because they are `unsigned`, overflow wraps around modulo 2^{32} — defined behaviour (Module 2) — and `head - tail` is still the

correct count. That is why the indices must be unsigned, and why the capacity must divide 2^{32} exactly, which any power of two does. (With signed indices, the overflow would be undefined behaviour.)

For a single producer and single consumer on one core, this buffer needs no lock: only the producer writes `head`, only the consumer writes `tail`, and each reads the other's index (Module 27, exercise 9). Across threads on a multicore machine, `head` and `tail` must be `_Atomic` with release/acquire ordering (Module 17), so that the data written into a slot is visible before the index that publishes it.

29.5 Hash Tables

A *hash table* stores key/value pairs in an array, choosing each pair's position by computing a *hash* of its key. Two keys can hash to the same position — a *collision* — and the two classic designs differ in how they handle it.

The hash function

```c
static uint64_t hash64(uint64_t x) {
    x ^= x >> 33; x *= 0xff51afd7ed558ccdULL;
    x ^= x >> 33; x *= 0xc4ceb9fe1a85ec53ULL;
    x ^= x >> 33; return x;
}
```

This is the 64-bit "finaliser" from MurmurHash3: alternating shifts, XORs, and multiplications by large odd constants, so that every input bit affects every output bit. A good hash function makes nearby keys (1, 2, 3, …) land far apart, which is what keeps collisions rare. The tables here have power-of-two capacities, so the position is `hash & (cap - 1)` — keeping the *low* bits of the hash, which is only safe because the mixer spreads the high bits' information down into them. (With a weak hash such as the identity function, power-of-two tables perform terribly on keys that share low bits.)

For string keys, use a proven function — FNV-1a for simplicity, or xxHash or wyhash for speed. And if keys come from untrusted input (Module 23), an attacker who knows your hash function can choose thousands of keys that all collide, degrading every operation to O(n) — a *hash-flooding* denial-of-service attack. Defend against it with a *keyed* hash such as SipHash, seeded with a random secret at start-up; this is what Python, Rust, and Redis do.

Open addressing with linear probing

```c
enum { SLOT_EMPTY = 0, SLOT_FULL = 1, SLOT_TOMB = 2 };
typedef struct { uint64_t *keys, *vals; uint8_t *state; size_t cap, len, tombs; } OHash;

static bool oh_put(OHash *h, uint64_t k, uint64_t v) {
    if ((h->len + h->tombs + 1) * 10 > h->cap * 7 && !oh_grow(h)) return false;   /* load <= 0.7 */
    size_t mask = h->cap - 1, i = hash64(k) & mask, tomb = SIZE_MAX;
    for (;;) {
        if (h->state[i] == SLOT_EMPTY) {
            if (tomb != SIZE_MAX) { i = tomb; h->tombs--; }   /* reuse the first tombstone */
            h->state[i] = SLOT_FULL; h->keys[i] = k; h->vals[i] = v; h->len++; return true;
        }
        if (h->state[i] == SLOT_TOMB) { if (tomb == SIZE_MAX) tomb = i; }
        else if (h->keys[i] == k) { h->vals[i] = v; return true; }   /* update */
```

```
        i = (i + 1) & mask;
    }
}

static bool oh_get(const OHash *h, uint64_t k, uint64_t *out) {
    size_t mask = h->cap - 1, i = hash64(k) & mask;
    while (h->state[i] != SLOT_EMPTY) {
        if (h->state[i] == SLOT_FULL && h->keys[i] == k) { if (out) *out = h->vals[i]; return true; }
        i = (i + 1) & mask;
    }
    return false;
}

static bool oh_del(OHash *h, uint64_t k) {
    size_t mask = h->cap - 1, i = hash64(k) & mask;
    while (h->state[i] != SLOT_EMPTY) {
        if (h->state[i] == SLOT_FULL && h->keys[i] == k) { h->state[i] = SLOT_TOMB; h->len--; h->tombs+
+; return true; }
        i = (i + 1) & mask;
    }
    return false;
}
```

(`oh_init`, `oh_grow` — which allocates a table twice the size and reinserts every live entry — and `oh_free` are in the full source.)

In *open addressing*, every entry lives directly in the table's arrays. A key's home is `hash & mask`; if that slot is taken by another key, the key goes in the next slot, and the next (*linear probing*), wrapping around at the end. A lookup follows the same path until it finds the key or reaches an empty slot, which proves the key is absent.

The keys, values, and slot states are stored in three separate arrays — Module 25's struct-of-arrays layout — so a probe scans a dense array of one-byte states and eight-byte keys, touching as few cache lines as possible. Linear probing's great strength is exactly this locality: the next slot to check is almost always in the same cache line.

Deletion needs tombstones. Simply emptying a deleted slot would break lookups for any key that had probed *past* it: the search would stop early at the new hole and wrongly report the later key as missing. So a deleted slot is marked as a *tombstone* — "something was here; keep searching" — which lookups skip over and insertions may reuse. Tombstones count toward the load factor, and a real table periodically rebuilds itself to clear them. The test exercises exactly this: delete every other key, verify that all the survivors are still found and the deleted ones are not, then reinsert them into the tombstones.

```
=== 3. HASH TABLE, open addressing ===
  1000000 inserts 69.3 ms, delete/reinsert with tombstones OK, len=1000000 cap=2097152 load=0.48
```

The **load factor** (entries ÷ slots) is kept at or below 0.7. As a linear-probing table fills up, entries form long contiguous runs (*primary clustering*), and the expected probe length grows sharply; around 0.7 is the conventional limit. (Faster designs such as Robin Hood hashing and Swiss tables — used by Rust's and Google's C++ hash maps — refine this layout further, but the core idea is the same.) The load reads 0.48 here because the table last doubled when it crossed 0.7.

Separate chaining

```
typedef struct CNode { uint64_t key, val; struct CNode *next; } CNode;
typedef struct { CNode **b; size_t cap, len; } CHash;
```

```
static bool ch_put(CHash *h, uint64_t k, uint64_t v) {
    if (h->len >= h->cap) { /* ...double the bucket array and relink every node... */ }
    size_t i = hash64(k) & (h->cap - 1);
    for (CNode *x = h->b[i]; x; x = x->next) if (x->key == k) { x->val = v; return true; }
    CNode *x = malloc(sizeof *x); if (!x) return false;
    *x = (CNode){ k, v, h->b[i] }; h->b[i] = x; h->len++; return true;
}
```

```
=== 4. HASH TABLE, separate chaining ===
  1000000 inserts 83.0 ms (one malloc per entry), len=1000000 buckets=1048576
```

In *separate chaining*, each slot (“bucket”) holds a linked list of the entries that hash there. It is simpler: deletion is ordinary list removal (no tombstones), the load factor can exceed 1 without disaster, and entries never move, so pointers to them remain valid as the table grows. But it costs one allocation per entry and a pointer chase per lookup.

Which to choose

	Open addressing	Separate chaining
Measured lookups (1M)	**24.2 ms**	36.3 ms
Measured inserts (1M)	**69.3 ms**	83.0 ms
Allocations	a few large arrays	one per entry
Deletion	tombstones	simple
Pointer stability	✗ entries move on growth	✓
Degradation at high load	sharp	gradual

Open addressing was 1.5× faster at lookup, entirely because of locality. Choose chaining when entries are large, must not move, or when simpler deletion matters more than speed.

29.6 Binary Heaps and Priority Queues

```
typedef struct { int *a; size_t len, cap; } Heap;

static bool heap_push(Heap *h, int v) {
    /* ...grow a->cap by doubling if full... */
    size_t i = h->len++;
    while (i > 0) { size_t p = (i - 1) / 2; if (h->a[p] <= v) break; h->a[i] = h->a[p]; i = p; }   /*
sift up */
    h->a[i] = v; return true;
}

static bool heap_pop(Heap *h, int *out) {
    if (!h->len) return false;
    *out = h->a[0]; int v = h->a[--h->len]; size_t i = 0;
    for (;;) {                                                  /* sift down */
        size_t c = 2 * i + 1; if (c >= h->len) break;
        if (c + 1 < h->len && h->a[c + 1] < h->a[c]) c++;
        if (v <= h->a[c]) break;
        h->a[i] = h->a[c]; i = c;
    }
    if (h->len) h->a[i] = v;
    return true;
}
```

```
=== 5. BINARY HEAP ===
  1000000 pushes then pops come out in non-decreasing order: OK (heapsort)
```

A *priority queue* always gives you the smallest (or largest) element next — the core of schedulers, timers, event simulations, Dijkstra's shortest-path algorithm, and "top k" queries. The *binary heap* implements one in a plain array, with no pointers at all.

The array is viewed as a complete binary tree: the children of element i are at $2i + 1$ and $2i + 2$, and its parent is at $(i - 1) / 2$. The only invariant is that every parent is less than or equal to its children — so the minimum is always at index 0, although the array as a whole is not sorted.

- **Push** appends the value at the end, then *sifts it up*: while it is smaller than its parent, the parent moves down into its place.
- **Pop** removes the root, takes the last element, and *sifts it down* from the top: while it is larger than its smaller child, that child moves up.

Both touch one element per level, so both are O(log n). Note that the code *moves* elements into the hole rather than repeatedly swapping — half the memory writes. Popping every element in turn yields them in sorted order, which is *heapsort*; the test checks exactly that for a million random values.

For timers, a heap is often better than a sorted list or tree: you only ever need the *earliest* deadline, and a heap gives you that with the locality of an array.

29.7 Balanced Binary Search Trees (AVL)

A *binary search tree* (BST) keeps every key in the left subtree of a node smaller than the node, and every key in the right subtree larger — so a search can discard half of the remaining tree at each step, and an in-order walk visits the keys in sorted order. But a plain BST has a fatal weakness: insert keys in sorted order, and each new key becomes the right child of the last, producing a "tree" that is really a linked list, with O(n) searches. A *balanced* tree performs small local restructurings to prevent that.

An **AVL tree** (named after Adelson-Velsky and Landis, 1962) keeps, at every node, the heights of the two subtrees within 1 of each other.

```c
typedef struct Avl { uint64_t key; struct Avl *l, *r; int h; } Avl;

static int  avl_h(const Avl *n) { return n ? n->h : 0; }
static void avl_fix(Avl *n) { int a = avl_h(n->l), b = avl_h(n->r); n->h = (a > b ? a : b) + 1; }
static Avl *rot_r(Avl *y) { Avl *x = y->l; y->l = x->r; x->r = y; avl_fix(y); avl_fix(x); return x; }
static Avl *rot_l(Avl *x) { Avl *y = x->r; x->r = y->l; y->l = x; avl_fix(x); avl_fix(y); return y; }

static Avl *avl_balance(Avl *n) {
    avl_fix(n);
    int bf = avl_h(n->l) - avl_h(n->r);
    if (bf > 1)  { if (avl_h(n->l->l) < avl_h(n->l->r)) n->l = rot_l(n->l); return rot_r(n); }
    if (bf < -1) { if (avl_h(n->r->r) < avl_h(n->r->l)) n->r = rot_r(n->r); return rot_l(n); }
    return n;
}

static Avl *avl_insert(Avl *n, uint64_t k, bool *oom) {   /* recursion depth is O(log n) — bounded */
    if (!n) { Avl *x = malloc(sizeof *x); if (!x) { *oom = true; return NULL; } *x = (Avl){ k, NULL,
```

```
NULL, 1 }; return x; }
    if (k < n->key)      { Avl *c = avl_insert(n->l, k, oom); if (*oom) return n; n->l = c; }
    else if (k > n->key) { Avl *c = avl_insert(n->r, k, oom); if (*oom) return n; n->r = c; }
    else return n;   /* duplicate: ignore */
    return avl_balance(n);
}

static bool avl_find(const Avl *n, uint64_t k) {
    while (n) { if (k == n->key) return true; n = k < n->key ? n->l : n->r; }   /* iterative */
    return false;
}
```

Each node stores the height of its subtree. After an insertion, every node on the way back up recomputes its height and checks its *balance factor* (left height minus right height). If it has become ±2, a *rotation* restores balance: `rot_r` lifts the left child into the node's place and makes the old node its right child, preserving the key ordering while shortening the heavy side by one level. When the imbalance zig-zags (the heavy child leans the other way), the child is rotated first — a "double rotation."

A few details reflect earlier modules. The insertion is recursive, which Module 6 warned about — but the depth is the tree's height, which the AVL invariant bounds at about 1.44 $\log_2 n$: about 29 for a million keys, not a million. Lookup, the hot path, is iterative anyway. An allocation failure is reported through `oom` without corrupting the tree (Module 10: check every allocation).

The test verifies the invariants rather than trusting them:

```
static int avl_check(const Avl *n, size_t *count) {   /* verifies AVL + BST invariants */
    if (!n) return 0;
    if (n->l) CHECK(n->l->key < n->key);
    if (n->r) CHECK(n->r->key > n->key);
    int a = avl_check(n->l, count), b = avl_check(n->r, count);
    CHECK(a - b <= 1 && b - a <= 1);
    CHECK(n->h == (a > b ? a : b) + 1);
    (*count)++;
    return n->h;
}
```

```
=== 6. AVL TREE ===
  1000000 random inserts 695.2 ms, invariants verified, height=24
  1000000 SORTED inserts: height=20 (an unbalanced BST would have height 1000000)
```

Inserting the keys 0 to 999,999 in order — the worst case for a plain BST — produced a tree of height 20, the minimum possible for a million nodes. The random insertions give height 24, well inside the AVL bound. (The invariant checker checks each child against its parent; a full BST check would also pass down the allowed key range, which this test's construction makes unnecessary but a general validator should do.)

Deletion follows the same pattern — remove the node (replacing a node with two children by its in-order successor), then rebalance on the way up — and is left as an exercise.

The alternatives:

Tree	Notes
Red-black tree	Looser balance (height ≤ 2 $\log_2$ n), so fewer rotations on insert/delete. The Linux kernel's `rbtree`, C++'s `std::map`.

Tree	Notes
B-tree / B+ tree	Many keys per node, sized to a cache line or disk page. Far better locality; the structure behind every database and file system.
Skip list	Probabilistic, simpler to implement, easy to make concurrent. Used by Redis sorted sets and LevelDB.
Treap	A BST with random priorities — simple, and balanced in expectation.

Trees are the structure to choose when you need the keys *in order* — range queries ("all keys between 100 and 200"), finding the next larger key, or iterating sorted — while also inserting and deleting. If you only need exact lookups, a hash table is several times faster (§29.11).

29.8 Tries

```c
typedef struct Trie { struct Trie *kid[26]; bool end; unsigned words_below; } Trie;

static bool trie_insert(Trie *t, const char *w) {
    for (const char *p = w; *p; p++) { if (*p < 'a' || *p > 'z') return false; }   /* validate first */
    Trie *n = t;
    for (; *w; w++) {
        int c = *w - 'a';
        if (!n->kid[c]) { n->kid[c] = calloc(1, sizeof *n); if (!n->kid[c]) return false; }
        n->words_below++; n = n->kid[c];
    }
    if (!n->end) { n->end = true; n->words_below++; }
    return true;
}

static const Trie *trie_find(const Trie *t, const char *s) {
    for (; t && *s; s++) { if (*s < 'a' || *s > 'z') return NULL; t = t->kid[*s - 'a']; }
    return t;
}
```

After inserting *car, card, care, careful, cat, dog, do,* and *door*:

```
=== 7. TRIE ===
  contains(car)=1 contains(ca)=0  words with prefix 'car'=4 'ca'=5 'do'=3 'cow'=0
```

A *trie* (from re*trie*val) stores strings as paths from the root: each node has one child per possible next character, and a flag marks the nodes where a complete word ends. Looking up a word takes one step per character, however many words are stored, and — the real strength — every word sharing a prefix sits below the same node. That makes tries the natural structure for autocompletion, spell-checking, IP routing tables (tries over address bits), and dictionary compression.

Here, each node also counts the words below it, so "how many words start with *ca*?" is answered by finding the node for *ca* and reading one counter. Note the input validation (Module 23): the insert function checks the whole word *before* modifying anything, so a rejected word such as `"Bad!"` leaves no partial path behind — and the `'a'` to `'z'` range check guards the array index.

The cost is memory: each node here holds 26 pointers — 208 bytes on a 64-bit machine — mostly `NULL`. Production tries compress this: *radix trees* (also called Patricia tries) merge chains of single-child nodes, and nodes can store their children in small sorted arrays instead of a full table.

29.9 Bitsets

```
typedef struct { uint64_t *w; size_t nbits; } Bits;
static inline void bits_set(Bits *b, size_t i)        { b->w[i >> 6] |= (UINT64_C(1) << (i & 63)); }
static inline bool bits_test(const Bits *b, size_t i) { return (b->w[i >> 6] >> (i & 63)) & 1u; }
static size_t bits_count(const Bits *b) {
    size_t c = 0; for (size_t i = 0; i < (b->nbits + 63) / 64; i++) c +=
(size_t)__builtin_popcountll(b->w[i]);
    return c;
}
```

The Sieve of Eratosthenes, with one bit per number:

```
for (size_t i = 2; i * i < LIM; i++)
    if (!bits_test(&comp, i))
        for (size_t j = i * i; j < LIM; j += i) bits_set(&comp, j);
```

```
=== 8. BITSET ===
  sieve below 10000000: 664579 primes in 19.0 ms using 1220 KB (1 bit per number)
```

A *bitset* represents a set of small integers — or an array of Booleans — as individual bits packed into 64-bit words: bit i lives in word `i >> 6` (that is, i / 64) at position `i & 63` (i mod 64), exactly Module 4's bit manipulation. Membership tests and updates are O(1), counting uses the hardware population-count instruction 64 bits at a time (Module 4, or C23's `stdc_count_ones`, Module 18), and set union and intersection are word-wide `|` and `&`.

The sieve marks every multiple of each prime as composite, starting from the prime's square (smaller multiples have already been marked by smaller primes). It found all 664,579 primes below ten million in 19 ms — and the whole table fits in 1.2 MB, small enough to stay largely in the L2 and L3 caches. The same table as an array of `bool` would take 10 MB, eight times as much memory traffic.

Bitsets also appear as allocation maps (which pool slots are free — Module 27's bitmask), Bloom filters, graph adjacency matrices, and permission and feature flags.

29.10 Algorithms: Searching and Sorting

Binary search, done right

```
static size_t lower_bound(const uint64_t *a, size_t n, uint64_t key) {   /* first i with a[i] >= key */
    size_t lo = 0, hi = n;                                                /* half-open [lo, hi) */
    while (lo < hi) { size_t mid = lo + (hi - lo) / 2; if (a[mid] < key) lo = mid + 1; else hi = mid; }
    return lo;
}
```

Binary search is famously easy to get wrong. This version gets three things right:

1. **It returns a position, not just "found."** `lower_bound` returns the first index whose element is ≥ `key` — which is where the key is if it is present, where it should be inserted if it is not, and `n` if every element is smaller. The caller checks `j < n && a[j] == key`. The same function answers "find," "insert position," and "count of elements below *x*." (C's `bsearch` only answers "find," and returns an arbitrary match among duplicates — Module 14.)
2. **It uses a half-open range** `[lo, hi)` (Module 5), which makes the loop condition and updates fall out naturally, with no `±1` adjustments to get wrong.
3. **It computes the midpoint as `lo + (hi - lo) / 2`, not `(lo + hi) / 2`.** The naive form overflows when `lo + hi` exceeds the type's range. This exact bug sat in the Java standard library's binary search for nearly a decade before it was found in 2006 — on arrays of more than a billion elements. With unsigned `size_t`, the overflow wraps rather than being undefined, but it still produces the wrong index.

Sorting: `qsort` vs a hand-written merge sort

```
static void insertion_sort(int *a, size_t n) {
    for (size_t i = 1; i < n; i++) { int v = a[i]; size_t j = i; while (j > 0 && a[j-1] > v) { a[j] =
a[j-1]; j--; } a[j] = v; }
}

static void merge_sort_rec(int *a, int *tmp, size_t n) {
    if (n <= 16) { insertion_sort(a, n); return; }   /* small-n cutoff (Module 25) */
    size_t m = n / 2;
    merge_sort_rec(a, tmp, m); merge_sort_rec(a + m, tmp, n - m);
    if (a[m - 1] <= a[m]) return;                    /* already ordered */
    size_t i = 0, j = m, k = 0;
    while (i < m && j < n) tmp[k++] = (a[j] < a[i]) ? a[j++] : a[i++];   /* '<' keeps it STABLE */
    while (i < m) tmp[k++] = a[i++];
    while (j < n) tmp[k++] = a[j++];
    memcpy(a, tmp, n * sizeof *a);
}
```

```
=== 10. SORTING 1000000 ints ===
  qsort (comparator via fn pointer): 132.2 ms
  merge sort (inlined compare, stable): 79.2 ms
```

Merge sort splits the array in half, sorts each half recursively, and merges the two sorted halves into a temporary buffer. It is O(n log n) in every case, and it is *stable* — equal elements keep their original order — which `qsort` does not promise (Module 14). Stability comes from one character: when the two candidates are equal, the merge takes from the left half (`a[j] < a[i]`, not `<=`).

Three refinements from earlier modules make it fast:

- **The small-*n* cutoff.** Below 16 elements, insertion sort — O(n^2), but with tiny constants and perfect locality — beats recursion (Module 25, §25.1). Every production sort does this.
- **The already-ordered check.** If the last element of the left half is no greater than the first of the right, the halves are already in order and the merge is skipped, making sorted input nearly free.

- **The comparison is inlined.** `qsort` must call its comparator through a function pointer for every comparison — about 20 million indirect calls here, which the compiler cannot inline (Module 25, §25.8). The hand-written sort compares `int`s directly.

The result was 1.7× faster than `qsort`, and stable. That is the general lesson about C's generic `void *` algorithms: they are convenient and correct, but a type-specific version — hand-written, or generated with a macro template (Module 28) — is usually markedly faster. The price is O(n) extra memory for the temporary buffer, which the function allocates once and reuses for every merge.

29.11 The Measured Comparison

The same million random 64-bit keys, and the same million random successful lookups, in each structure (`-O2`):

```
=== 9. LOOKUP: 1000000 random successful lookups among 1000000 keys ===
  open addressing              :  24.2 ms
  separate chaining            :  36.3 ms
  AVL tree                     : 183.3 ms
  sorted array + binary search : 201.9 ms
```

Structure	Per lookup	Cache misses per lookup (roughly)
Open addressing	24 ns	~1: the slot, usually with its neighbours in the same line
Separate chaining	36 ns	~2: the bucket, then the node
AVL tree	183 ns	~20: one node per level, scattered through the heap
Sorted array + binary search	202 ns	~20: one probe per halving, each far from the last

With a million keys, the data is far larger than the caches (Module 25), so each structure's speed is governed by how many *dependent cache misses* a lookup incurs. The hash tables jump almost straight to the answer. The tree and the binary search both take about 20 steps, each probably a miss to main memory, so they are 7–8 times slower — and similar to each other. (The sorted array has no pointers, but at this size its probes are just as scattered; its advantages are compactness, zero per-element allocation, and far better performance when the array is small enough to stay in cache.)

The lessons:

1. **For exact-match lookup on large data, a hash table wins by a wide margin** — and open addressing beats chaining through locality.
2. **Trees earn their cost only when you need order** — range queries, sorted iteration, nearest-key search — while also modifying the set.

3. **A sorted array is the right choice for data built once and searched often,** especially when it is small or fits in cache — and it needs no allocation per element at all.
4. **Measure with realistic sizes.** At a few thousand keys, all of the data fits in the caches, and the ranking changes (practice exercise 8).

Module 29 Rules

#	Rule
1	Default to a dynamic array; choose something else only for an operation you actually need
2	Grow by doubling, check the size multiplication for overflow, and `reserve` when the count is known
3	Never hold pointers into a growable array across insertions — use indices
4	Lists only for O(1) removal of known objects, multiple membership, or pinned objects — prefer intrusive
5	Ring buffers: power-of-two capacity, free-running unsigned indices, `head - tail` is the count
6	Hash tables need a well-mixing hash; use a keyed hash (SipHash) for untrusted keys
7	Open addressing: keep the load ≤ ~0.7 and delete with tombstones
8	Heaps for “always the smallest next” — array-based, O(log n), no pointers
9	Balanced trees only when you need order; verify the invariants in tests
10	Tries for prefix queries; validate input before modifying the structure
11	Bitsets: 1 bit per element, word-wide operations, hardware popcount
12	Binary search: half-open ranges, `lo + (hi - lo) / 2`, return a position (`lower_bound`)
13	Type-specific sorts beat `qsort`’s function-pointer comparator; cut over to insertion sort for small *n*
14	Verify every structure under ASan + UBSan with invariant checks and randomised tests
15	Benchmark at realistic sizes — cache misses, not Big-O, decide the ranking

---

Practice

1. **Vector API.** Extend `VEC_DECLARE` with `insert_at`, `remove_at` (order-preserving), `swap_remove`, `pop`, `shrink_to_fit`, and a bounds-checked `at`. Test each under ASan, including a forced allocation failure (Module 22's fault injection).

2. **Ring buffer across threads.** Convert the ring buffer to a lock-free single-producer/single-consumer queue between two threads, using `_Atomic` indices with release/acquire ordering (Module 17). Stress-test it with 100 million items under TSan. Then deliberately use `relaxed` ordering and see whether TSan objects.

3. **Generic hash map.** Turn the open-addressing table into a macro template `HASHMAP_DECLARE(Name, K, V, hash_fn, eq_fn)` that supports string keys. Add automatic rebuilding when tombstones exceed 20% of the table. Fuzz it (Module 22) against a simple linear-scan reference map, differential-testing every operation.

4. **Hash flooding.** Write a program that generates keys that all collide in the §29.5 table (hint: since the hash is invertible, you can compute keys that map to chosen slots). Measure how much insert time degrades for 50,000 such keys. Then switch to a keyed hash with a random seed and show that the attack no longer works.

5. **Robin Hood hashing.** Modify the open-addressing table so that, on insertion, an entry that has probed further than the entry it collides with takes that slot (and the displaced entry continues probing). Measure the maximum and average probe lengths at load factors 0.5, 0.7, and 0.9, before and after.

6. **Heap-based timers.** Build a timer wheel replacement: a min-heap of `{deadline, callback, ctx}` entries, supporting `timer_add`, `timer_cancel` (hint: store each entry's heap index so it can be removed from the middle), and `timer_run_expired(now)`. Drive it with an injected clock (Module 28) and test it without sleeping.

7. **AVL deletion and ranges.** Add `avl_delete` and `avl_range(lo, hi, callback, ctx)`. Extend `avl_check` to verify the full BST property by passing down the allowed key range. Run a million random inserts and deletes, checking the invariants after every thousand operations.

8. **The size crossover.** Rerun the §29.11 lookup benchmark for N = 100, 1,000, 10,000, 100,000, and 10,000,000. Plot the time per lookup for each structure. Find the sizes at which the ranking changes, and relate them to your machine's cache sizes (Module 25).

9. **Radix tree.** Convert the trie into a compressed radix tree, where each edge carries a string rather than a single character. Load a 100,000-word dictionary into both, and compare the node count, memory use, and lookup time.

10. **Sorting study.** Implement an in-place quicksort with median-of-three pivot selection and an insertion-sort cutoff, and a heapsort. Benchmark them against `qsort` and the merge sort from §29.10 on random, sorted, reverse-sorted, and all-equal inputs. Explain each result — in particular, what happens to a naive quicksort on sorted input.

Module 29 in Summary

C provides no containers, so every C programmer needs a small working set of data structures and the judgement to choose between them. The dynamic array is the default: doubling growth gives amortised O(1) appends, a `reserve` operation and an overflow check make it robust, and sorted arrays double as compact search structures. Linked lists are for O(1) removal of known objects and multiple membership — best intrusive — and are otherwise slower than arrays. Ring buffers with power-of-two capacities and free-running unsigned indices give lock-free FIFO queues that use every slot and survive index overflow.

Hash tables give O(1) average lookup: open addressing with linear probing keeps entries in dense arrays, stays below a 0.7 load factor, and deletes with tombstones, while separate chaining trades locality for simplicity and pointer stability; for untrusted keys, a keyed hash defeats hash flooding. Binary heaps keep a priority queue in a plain array. AVL trees keep a binary search tree balanced with rotations, guaranteeing logarithmic height even for sorted input, and belong wherever order matters. Tries answer prefix queries, and bitsets store one bit per element with word-wide operations — enough to sieve ten million numbers in 19 ms and 1.2 MB.

Binary search should return a position over a half-open range with an overflow-safe midpoint, and a type-specific, stable merge sort with an insertion-sort cutoff beat `qsort` by 1.7×. Measured on a million keys, the hash tables outran the tree and binary search by a factor of seven or eight, because at that size cache misses, not Big-O, decide performance. Every structure here was verified with invariant checks and randomised tests under the sanitizers — the discipline that makes hand-written data structures in C trustworthy.

Next: Module 30 — Interoperability, covering how C talks to the rest of the world: `extern "C"` and C++ interop; foreign-function interfaces for Python (ctypes, cffi, C extensions), Rust (bindgen, cbindgen), Go (cgo), Java (JNI), Lua, and Node (N-API); WebAssembly via Emscripten and WASI; and designing C APIs meant to be bound from other languages.

Module 30 — Interoperability

In this module:

- `extern "C"` and C++ interop
- FFI: Python (ctypes, cffi, C extensions), Rust (bindgen, cbindgen), Go (cgo), Java (JNI), Lua, Node (N-API)
- WebAssembly via Emscripten and WASI
- Designing C APIs meant to be bound from other languages

C is the lingua franca of computing. Almost every language can call C, and almost none can call each other directly — so C's ABI (Module 6) is the universal interoperability layer. Python's scientific stack, Node's native modules, database drivers for every language, operating-system APIs, graphics drivers, and cryptographic libraries all meet at a boundary that looks like a C function call.

A *foreign function interface* (FFI) is the mechanism by which one language calls code written in another. This module shows what crosses that boundary, what it costs, and how to design C libraries that other languages can bind safely and efficiently. All output below comes from real builds: GCC 13.3, G++ 13.3, and Python 3.12 with ctypes and cffi 2.0.

30.1 The Library Everyone Will Bind

One library, bound from C++, from Python with ctypes, and from Python with cffi in both its modes. Every design choice in its header exists because of FFI.

```c
/* mathlib.h */

/* ---- 1. C++ GUARD ---- */
#ifdef __cplusplus
extern "C" {
#endif

/* ---- 2. EXPORT MACRO (Module 20) ---- */
#if defined(_WIN32)
#  ifdef MATHLIB_BUILD
#    define ML_API __declspec(dllexport)
#  else
#    define ML_API __declspec(dllimport)
#  endif
#elif defined(__GNUC__)
#  define ML_API __attribute__((visibility("default")))
#endif

/* ---- 3. FFI-FRIENDLY SIGNATURES ---- */
typedef enum { ML_OK=0, ML_EINVAL=1, ML_ERANGE=2, ML_ENOMEM=3 } ml_status;

ML_API const char *ml_version(void);
ML_API const char *ml_strerror(int32_t status);

ML_API int32_t ml_add(int32_t a, int32_t b);                    /* scalar in/out */
ML_API int64_t ml_sum(const int32_t *data, int32_t n);          /* array + LENGTH */
ML_API int32_t ml_divide(int32_t a, int32_t b, int32_t *out);   /* status + out-param */

typedef struct ml_stats ml_stats;                               /* OPAQUE HANDLE */
ML_API ml_stats *ml_stats_create(void);
ML_API void      ml_stats_destroy(ml_stats *s);                 /* accepts NULL */
ML_API int32_t   ml_stats_add(ml_stats *s, double v);
```

```
ML_API double    ml_stats_mean(const ml_stats *s);

ML_API int32_t ml_format(const ml_stats *s, char *buf, int32_t buflen);   /* CALLER's buffer */

typedef int32_t (*ml_visit_fn)(double value, void *userdata);            /* callback + ctx */
ML_API int32_t ml_stats_foreach(const ml_stats *s, ml_visit_fn fn, void *userdata);

#ifdef __cplusplus
}
#endif
```

(The header also declares a few more functions — `ml_hypot`, `ml_stats_count`, `ml_stats_stddev`, and `ml_version_major` — omitted here for brevity. A complete version would also provide an empty `#else` branch defining `ML_API` for other compilers, and include `<stdint.h>`.)

Every function uses one of a handful of shapes that every language can express: scalars in and out; an array with its length; a status code with an out-parameter; an opaque handle; a caller-supplied buffer; and a callback with a user-data pointer. §30.6 explains why each rule exists.

Built with `-fvisibility=hidden`, the library exports exactly 15 symbols:

```
$ nm -D --defined-only libmathlib.so | grep " T "
ml_add ml_divide ml_format ml_hypot ml_stats_add ml_stats_count
ml_stats_create ml_stats_destroy ml_stats_foreach ml_stats_mean
ml_stats_stddev ml_strerror ml_sum ml_version ml_version_major
```

Every one of them is a deliberate part of the API; nothing internal leaks (Module 20).

30.2 `extern "C"` and C++

Why it’s needed — name mangling, measured

```
int mangled_function(int a, double b);
```

```
symbol the C code looks for   : U mangled_function
symbol the C++ code looks for : U _Z16mangled_functionid
demangled                     : mangled_function(int, double)
```

C++ supports *overloading* — several functions with the same name but different parameter types — so it must give each one a distinct symbol for the linker. It does so by *mangling*: encoding the parameter types into the symbol name. `_Z16mangled_functionid` means “a function whose name is 16 characters long, `mangled_function`, taking an `i`nt and a `d`ouble.” A C compiler does no mangling, so the C library exports plain `mangled_function`. Without `extern "C"`, the C++ program asks the linker for `_Z16mangled_functionid`, the library provides `mangled_function`, and the link fails.

`extern "C"` tells the C++ compiler: use C *linkage* — unmangled names — for these declarations. The `#ifdef __cplusplus` guard (Module 20) means one header serves both languages: C++ compilers see the `extern "C" {`, and C compilers never do.

The idiomatic C++ wrapper — RAII over an opaque handle

```cpp
#include "mathlib.h"   // the extern "C" guard makes this work

class Stats {
    struct Deleter { void operator()(ml_stats *s) const noexcept { ml_stats_destroy(s); } };
    std::unique_ptr<ml_stats, Deleter> h_;
public:
    Stats() : h_(ml_stats_create()) { if (!h_) throw std::bad_alloc(); }

    void add(double v) {
        if (int32_t rc = ml_stats_add(h_.get(), v); rc != ML_OK)
            throw std::runtime_error(ml_strerror(rc));   // C error code -> C++ exception
    }
    std::string format() const {
        char buf[128];
        if (ml_format(h_.get(), buf, sizeof buf) != ML_OK) throw std::runtime_error("format");
        return buf;
    }
    template <class F> void for_each(F f) const {        // C++ lambda -> C callback
        auto trampoline = [](double v, void *ud) -> int32_t {
            (*static_cast<F*>(ud))(v); return 0;
        };
        ml_stats_foreach(h_.get(), trampoline, &f);
    }
};
```

```
C library version 1.4.2 called from C++
  ml_add(20,22)  = 42
  ml_sum         = 15
  ml_divide(7,0) = out of range
  RAII wrapper   : n=8 mean=5.0000 sd=2.1381
  lambda callback: total = 40
  (destructor calls ml_stats_destroy automatically)
```

RAII (“resource acquisition is initialisation”) is the C++ idiom of tying a resource’s lifetime to an object: the `unique_ptr` with a custom deleter calls `ml_stats_destroy` automatically when the `Stats` object goes out of scope — including when an exception is thrown. This is exactly the cleanup that C’s `goto` chains (Module 5) do by hand. The wrapper also converts the C library’s error codes into C++ exceptions, and C strings into `std::string`.

The trampoline pattern is the key technique. A C callback must be a plain function pointer, but a C++ lambda that captures variables is an object, not a function. A lambda that captures *nothing*, however, does convert to a function pointer. So the wrapper passes the capturing lambda’s *address* through the `void *userdata` slot, and gives C a captureless “trampoline” that casts the user data back and calls the real lambda. This is Module 6’s context-pointer idea put to work — and it is how closures cross into C from every language.

The C++ interop rules

Rule	Why
Never let an exception propagate into C	C has no unwinding: the C frames’ cleanup never runs. Wrap every callback body in `try { ... } catch (...) { return ERROR; }`.
`extern "C"` applies to linkage, not to the body	An `extern "C"` function can be written in C++ internally
Only one overload can be `extern "C"`	No mangling means no overloading

Rule	Why
No C++ types in the C header	No `std::string`, templates, references, or default arguments
`extern "C"` on function-pointer types too	`extern "C" typedef void (*Cb)(int);` — strictly, C and C++ language linkage are different function types, although mainstream compilers treat them identically
Union punning is legal in C, UB in C++ (Module 11)	Use `memcpy` in shared headers
`static_cast`, not C casts, on the C++ side	Better diagnostics
C++ doesn't have C's `restrict`	It isn't a C++ keyword; `__restrict` is a common extension. Hide it behind a macro in shared headers.

30.3 Python

ctypes — no compiler, run-time binding

```
import ctypes
lib = ctypes.CDLL("./libmathlib.so")

# YOU MUST DECLARE argtypes AND restype.
# Without them ctypes assumes int arguments and an int return -> silent corruption.
lib.ml_version.restype = ctypes.c_char_p
lib.ml_add.argtypes = [ctypes.c_int32, ctypes.c_int32]
lib.ml_add.restype = ctypes.c_int32
lib.ml_sum.argtypes = [ctypes.POINTER(ctypes.c_int32), ctypes.c_int32]
lib.ml_sum.restype = ctypes.c_int64
lib.ml_stats_create.restype = ctypes.c_void_p   # opaque handle
VISIT = ctypes.CFUNCTYPE(ctypes.c_int32, ctypes.c_double, ctypes.c_void_p)
```

```
=== PYTHON ctypes ===
  ml_version()   = 1.4.2
  ml_add(20,22)  = 42
  ml_hypot(3,4)  = 5.0
  ml_sum([1..5]) = 15
  ml_divide(7,0) = out of range
  opaque handle  : n=8 mean=5.0000 sd=2.1381
  callback total : 40.0
  destroyed cleanly
```

`ctypes`, part of Python's standard library, loads a shared library with `dlopen` (Module 24) and calls its functions directly, converting Python values to C types according to the declarations you supply. It needs no compiler at all. `CFUNCTYPE` creates a real C function pointer that calls back into Python, for use with `ml_stats_foreach`.

> ⚠ **Forgetting `restype` is the number-one ctypes bug.** ctypes cannot read your header; it knows only what you tell it. A function returning a `double` read as an `int` gives garbage (the value is in a floating-point register, and ctypes reads an integer one

— Module 6's missing-prototype bug, in Python); a function returning a 64-bit pointer read as a 32-bit `int` gives a truncated pointer that crashes later, somewhere else. Declare both `argtypes` and `restype`, always.

⚠ **Keep the `CFUNCTYPE` object alive for as long as C might call it.** If Python garbage-collects the wrapper while C still holds the function pointer, the next callback jumps into freed memory — a use-after-free that crashes.

cffi ABI mode — paste the real declarations

```
from cffi import FFI
ffi = FFI()
ffi.cdef("""
    const char *ml_version(void);
    int64_t ml_sum(const int32_t *data, int32_t n);
    typedef struct ml_stats ml_stats;   /* opaque - cffi handles it */
    ml_stats *ml_stats_create(void);
    void ml_stats_destroy(ml_stats *s);
    typedef int32_t (*ml_visit_fn)(double value, void *userdata);
    int32_t ml_stats_foreach(const ml_stats *s, ml_visit_fn fn, void *userdata);
""")
C = ffi.dlopen("./libmathlib.so")

h = ffi.gc(C.ml_stats_create(), C.ml_stats_destroy)   # ties C lifetime to Python GC

@ffi.callback("int32_t(double, void*)")               # a real C function pointer
def visit(v, ud): acc.append(v); return 0
```

```
=== PYTHON cffi (ABI mode) ===
  ml_version()    = 1.4.2
  ml_sum          = 15
  ml_divide(7,0)  = out of range
  opaque + ffi.gc : n=8 mean=5.0000 sd=2.1381
  callback total  : 40.0
```

cffi parses real C declarations, so you copy and paste from the header instead of hand-writing type tables — far less room for the `restype` class of mistake. `ffi.gc()` attaches the destructor to the Python object, so the C object is destroyed automatically when Python no longer references it: RAII by garbage collection.

cffi API mode — compiles a real extension

```
ffibuilder.set_source("_mathlib_api",
    '#include "mathlib.h"',
    include_dirs=[here], libraries=["mathlib", "m"])
ffibuilder.compile()
```

```
=== cffi API MODE (compiled extension) ===
  version : 1.4.2
  ml_add  : 42
  mean    : 5.0
```

API mode generates and compiles a small C extension module against the real header. Type mismatches are therefore caught at build time by the C compiler, macros and inline functions work, and calls are much faster. The cost is needing a C compiler at install time — or shipping pre-built "wheels" for each platform.

⚠ The measured cost of crossing the boundary

```
=== FFI CALL OVERHEAD (2,000,000 calls of ml_add) ===
  pure python add   167.5 ms (  84 ns/call)
  ctypes           1518.4 ms ( 759 ns/call)
  cffi ABI mode     767.7 ms ( 384 ns/call)
  cffi API mode     423.7 ms ( 212 ns/call)
```

The C function itself takes about 1 ns. Everything above is boundary cost: converting Python objects to C values and back, checking types, releasing and reacquiring Python's global interpreter lock, and the call machinery itself. With ctypes, the overhead is roughly 750 times the cost of the work — and calling C is nine times *slower* than simply adding the numbers in Python.

The consequence: batch across the boundary

```
=== BATCHING (2,000,000 elements) ===
  pure python sum()          17.5 ms   result=1999999000000
  ONE C call over an array    1.3 ms   result=1999999000000   <- 13x faster
  ONE C call PER element    813.6 ms   result=1999999000000   <- 47x SLOWER
```

> Same C code. The only difference is how many times you cross the boundary.
>
> Calling C once per element made it 47× slower than pure Python. Calling it once over the whole array made it 13× faster.

This is the single most important FFI design principle: **expose bulk operations, not scalar ones.** `process_array(ptr, n)`, not `process_one(x)` in a loop. It is why NumPy is fast: each Python-level operation crosses into C once and then processes millions of elements.

Python binding options, ranked

Approach	Speed	Type safety	Needs a compiler
ctypes	slowest	none (run-time declarations)	✗
cffi ABI	~2× ctypes	parses C declarations	✗
cffi API	~4× ctypes	build-time	✓
Cython	fastest	build-time	✓
pybind11 / nanobind	fast	build-time	✓ (C++)
Hand-written CPython C API	fastest	manual	✓
NumPy buffer protocol	zero-copy for arrays	—	—

For numeric work, support the *buffer protocol* / NumPy arrays, which give C a raw pointer to Python's contiguous data with no copying at all.

> ⚠ **Release the GIL around long C calls.** Python's global interpreter lock allows only one thread to run Python code at a time. ctypes (with `CDLL`) and cffi release it automatically during each foreign call, but hand-written C extensions and Cython code

must do so explicitly — with `Py_BEGIN_ALLOW_THREADS`/`Py_END_ALLOW_THREADS`, or `with nogil:` in Cython — or a long-running C function freezes every other Python thread. (C code running without the GIL must not touch Python objects.)

30.4 The Other Languages

Rust

```rust
// bindgen generates Rust declarations FROM your C header
// build.rs:
bindgen::Builder::default().header("mathlib.h").generate()?;

// Then wrap it safely:
pub struct Stats(*mut ml_stats);
impl Stats {
    pub fn new() -> Option<Self> {
        let p = unsafe { ml_stats_create() };
        if p.is_null() { None } else { Some(Stats(p)) }
    }
    pub fn add(&mut self, v: f64) -> Result<(), Error> {
        match unsafe { ml_stats_add(self.0, v) } { 0 => Ok(()), e => Err(Error(e)) }
    }
}
impl Drop for Stats { fn drop(&mut self) { unsafe { ml_stats_destroy(self.0) } } }
```

- **bindgen:** C header → Rust FFI declarations, generated at build time.
- **cbindgen:** Rust `#[no_mangle] extern "C"` functions → a C header (the other direction, for exposing Rust code to C).
- **Everything that crosses the boundary is `unsafe`** in Rust, because the compiler cannot verify C's promises. The idiom is a thin `unsafe` layer wrapped in a safe API, with `Drop` (Rust's destructor) calling the C destroy function — the same shape as the C++ RAII wrapper.
- ⚠ **Rust panics must not unwind into C.** Use `std::panic::catch_unwind` in every `extern "C"` function, or build with `panic = "abort"` — the Rust counterpart of the C++ exception rule.

Go (cgo)

```go
/*
#cgo LDFLAGS: -L. -lmathlib -lm
#include "mathlib.h"
*/
import "C"
import "unsafe"

func Sum(data []int32) int64 {
    return int64(C.ml_sum((*C.int32_t)(unsafe.Pointer(&data[0])), C.int32_t(len(data))))
}
```

The C code in the comment immediately above `import "C"` is compiled with the Go package, and its declarations become available as `C.name`.

- ⚠ **cgo calls are expensive** (roughly 50–200 ns): Go's lightweight goroutines run on a small pool of system threads, and a C call must be moved onto a real system thread with a normal C stack.

- ⚠ **Go pointers may not be stored in C memory beyond the call.** Go's garbage collector tracks — and may move — Go memory, so C must not hold on to it. Pass C-allocated memory, or explicitly pin the Go memory (`runtime.Pinner`).
- **Go callbacks into C** need an `//export` comment and a separate file.
- **purego** is a newer alternative that calls C libraries without cgo, avoiding the need for a C compiler at build time.

Java (JNI / Panama)

```
public class MathLib {
    static { System.loadLibrary("mathlibjni"); }
    public static native int add(int a, int b);
}
```

```
JNIEXPORT jint JNICALL Java_MathLib_add(JNIEnv *env, jclass cls, jint a, jint b)
{ return ml_add(a, b); }
```

- **JNI** (the Java Native Interface) is verbose and error-prone, and requires a hand-written C shim for every function, with a name derived from the Java class and method.
- ⚠ **Every `Get*ArrayElements` needs a matching `Release*`,** or you leak memory or pin the Java heap.
- **Project Panama** (`java.lang.foreign`, the Foreign Function & Memory API, final in JDK 22) replaces JNI: there is no C shim at all, and the `jextract` tool generates bindings from headers. Use it for new work.

Node.js

- **N-API / node-addon-api** — ABI-stable across Node.js versions (unlike the older V8 API, which broke native modules with every release). The right choice for native add-ons.
- **ffi-napi** — run-time binding in the style of ctypes; convenient and slow.
- **WebAssembly** — often the best option of all: no native build per platform, and it runs in browsers too (§30.5).

Lua

Lua has the cleanest C API of any scripting language — a stack-based interface designed for embedding Lua in C programs and extending it with C:

```
static int l_add(lua_State *L) {
    lua_pushinteger(L, ml_add((int32_t)luaL_checkinteger(L,1), (int32_t)luaL_checkinteger(L,2)));
    return 1;   /* number of return values */
}
```

Arguments arrive on a virtual stack (`luaL_checkinteger(L, 1)` reads and type-checks the first), results are pushed back onto it, and the return value says how many were pushed. LuaJIT's FFI is remarkable: it parses C declarations at run time and JIT-compiles the calls, often down to zero overhead.

30.5 WebAssembly

WebAssembly (WASM) is a portable, sandboxed bytecode format that runs in every modern browser and in standalone runtimes, at near-native speed. C compiles to it directly.

Emscripten — C in the browser

```
emcc mathlib.c -O3 -o mathlib.js \
  -sEXPORTED_FUNCTIONS='["_ml_add","_ml_sum","_ml_stats_create","_malloc","_free"]' \
  -sEXPORTED_RUNTIME_METHODS='["ccall","cwrap","getValue","setValue","HEAP32"]' \
  -sMODULARIZE=1
```

```
const Module = await createModule();
const add = Module.cwrap('ml_add', 'number', ['number','number']);
console.log(add(20, 22));   // 42

// arrays: allocate in the WASM heap, get a pointer
const n = 5;
const ptr = Module._malloc(n * 4);
Module.HEAP32.set([1,2,3,4,5], ptr >> 2);
const sum = Module._ml_sum(ptr, n);
Module._free(ptr);
```

Key points:

- **Exported symbols get a leading underscore** (`_ml_add`).
- **Pointers are offsets into a single `ArrayBuffer`** — the module's *linear memory*. JavaScript reads and writes it through typed-array views (`HEAP8`, `HEAP32`, `HEAPF64`). That is why the array is copied into the WASM heap first, and why the index is `ptr >> 2`: `HEAP32` counts 4-byte elements, not bytes.
- **Emscripten emulates a POSIX-like environment:** a virtual file system, `emscripten_set_main_loop` instead of a blocking main loop, and pthreads implemented with Web Workers and `SharedArrayBuffer`.
- ⚠ **No blocking and no infinite loops** — the browser's event loop must keep running, or the page freezes. Use `ASYNCIFY` if you genuinely must block.

WASI — WebAssembly outside the browser

```
clang --target=wasm32-wasi -O2 --sysroot=$WASI_SYSROOT prog.c -o prog.wasm
wasmtime prog.wasm
```

WASI (the WebAssembly System Interface) is a standardised system-call interface for WASM — files, clocks, random numbers, arguments, and environment variables. Its security is *capability-based*: a module can access only what you explicitly grant it (`--dir=.` gives access to the current directory, and nothing else). Runtimes include wasmtime, Wasmer, WasmEdge, and wazero. (Newer toolchains name the target `wasm32-wasip1`, for WASI "preview 1".)

Why WASM matters for C: one binary runs everywhere; it is sandboxed by construction — a buffer overflow can corrupt only the module's own linear memory, never escape it; there are no per-platform native builds; and it runs at near-native speed. It is increasingly how C libraries (SQLite, image codecs, compression libraries) reach the web.

> ⚠ **WASM is 32-bit by default** (`wasm32`): `sizeof(void *) == 4`, and `size_t` is 32-bit. Code that assumes 64-bit pointers breaks. And `long` is 32-bit — Module 26's data-model problem, once again.

30.6 Designing C APIs for FFI

Every design decision in §30.1 exists for a reason.

The rules

Rule	Why bindings need it
Fixed-width types only (`int32_t`, `int64_t`, `double`)	`long` is 32-bit on Win64 and WASM (Module 26); binding generators guess wrong
Avoid `size_t` in the ABI where you can	Its width varies by platform; `int32_t`/`int64_t` is unambiguous
No unions, bitfields, or packed structs	Their layout is implementation-defined (Module 11); no binding generator handles them reliably
No varargs	`printf`-style functions are nearly unbindable — every language needs a shim
No function-like macros in the API	The preprocessor does not exist in the binding language. Provide real functions.
Opaque handles, not exposed structs	The layout can change without breaking bindings, and no structure marshalling is needed
Always pass array lengths	The binding must know how much to marshal — and it is a security requirement (Module 23)
Caller-supplied buffers	Avoids allocator mismatch — the caller's `free` may not match your `malloc`
If you must return allocated memory, provide the matching free	`ml_free_string()`, never "call `free()`"
Status return + out-parameters	Every language can express "integer return, pointer out-parameter"
Callbacks always take `void *userdata`	The only way to attach a closure (Modules 6 and 28)
No exceptions, panics, or `longjmp` across the boundary	C frames don't unwind
Thread safety documented per function	The GIL, goroutine scheduling, and JVM threading all interact with it
`extern "C"` guard + export macro	For C++ consumers and Windows DLLs
`ml_version()` plus a version macro	Run-time compatibility checking

The allocator-mismatch rule deserves a word. Memory allocated by `malloc` in your library must be freed by the *same* allocator. On Windows, each DLL may be linked against a different C run-time, each with its own heap, so memory freed by the "wrong" `free` corrupts the heap. And a Python or Java caller cannot call C's `free` at all without a binding for it. Providing your own `ml_free_*` function — or, better, filling the caller's buffer — avoids the problem completely.

The shape that binds cleanly from every language

```
/* Every language can express all four of these */
int32_t fn(int32_t a, double b);                      /* scalars            */
int32_t fn(const uint8_t *data, int32_t len);         /* array + length     */
int32_t fn(Handle *h, int32_t *out_value);            /* status + out-param */
int32_t fn(Handle *h, Callback cb, void *userdata);   /* callback + ctx     */
```

Bulk operations, not scalar ones

The §30.3 measurement is the argument: design for one call over N items, not N calls over one item.

```
/* BAD for FFI: N boundary crossings */
double transform_one(double x);

/* GOOD: one crossing */
int32_t transform_array(const double *in, double *out, int32_t n);
```

A binding-friendly checklist

```
/* Ship all of these: */
mathlib.h                       /* extern "C" guarded, fixed-width, no macros in the API */
mathlib.pc                      /* pkg-config (Module 20) */
libmathlib.so.1.4.2             /* versioned soname (Module 20) */
ml_version() / ML_VERSION_MAJOR  /* run-time + compile-time version */
```

Plus: document ownership, lifetime, thread safety, and every error code (Module 28) — binding authors cannot read your implementation, and often cannot read C at all.

Module 30 Rules

#	Rule
1	`#ifdef __cplusplus extern "C" {` in every public header
2	Never let an exception, panic, or `longjmp` cross into C
3	Fixed-width types only in the ABI — no `long`, no bitfields, no unions, no varargs
4	Opaque handles; expose no structure layouts
5	Always pass array lengths

#	Rule
6	Caller-supplied buffers, or ship the matching free function
7	Status return + out-parameters is the universal FFI shape
8	Every callback takes a `void *userdata` — that is how closures cross
9	Use the trampoline pattern for capturing closures
10	ctypes: declare `argtypes` AND `restype`, always
11	Keep callback objects alive for as long as C may call them
12	Batch across the boundary — measured 47× slower per element, 13× faster batched
13	Design bulk operations, not scalar ones
14	Release the GIL / yield the scheduler around long C calls
15	Go pointers may not be stored in C memory; Rust `extern "C"` needs `catch_unwind`
16	WASM is 32-bit — check your pointer and `long` assumptions
17	Ship a `.pc` file, a versioned soname, and a run-time version function

Practice

1. **Build the library and bind it four ways.** Reproduce §30.1's `mathlib`, then bind it from C++ (with a RAII wrapper), ctypes, cffi ABI mode, and cffi API mode. Verify that all four produce identical results under ASan.

2. **Break ctypes deliberately.** Omit `restype` on `ml_hypot` (which returns a `double`) and on `ml_stats_create` (which returns a pointer). Observe the garbage and the eventual crash. Then add the declarations and confirm the fix.

3. **Measure your own boundary cost.** Reproduce the §30.3 benchmark on your machine. Add Cython and a hand-written CPython extension. Then find the array size at which the batched C call starts to beat pure Python.

4. **Fix a chatty API.** Take an API with a scalar operation that is called in a loop. Add a bulk variant. Measure the speedup from Python, and report the number of boundary crossings before and after.

5. **Callbacks in both directions.** Extend the library with a function that takes a callback and calls it a million times. Measure the cost from ctypes, cffi, and C++. Then invert it: have the library store a callback and invoke it later — and work out how each language keeps the callback object alive.

6. **C++ exception safety.** Write a C API that takes a callback, and call it from C++ with a callback that throws. Observe the resource leak (or `std::terminate`). Then wrap the callback body in `try/catch (...)`, returning an error code, and verify with LSan that nothing leaks.

7. **Name mangling.** Compile the same header from C and from C++ without `extern "C"`. Compare the symbols with `nm` and `c++filt`. Then add the guard, and confirm that they match.

8. **Rust binding** (if you can install Rust). Use bindgen to generate FFI declarations for `mathlib.h`, write a safe wrapper with `Drop`, and add `catch_unwind` to an `extern "C"` callback. Test that a panic in the callback does not unwind into C.

9. **WASM** (if you can install Emscripten or a WASI SDK). Compile `mathlib.c` to WASM, call `ml_sum` from JavaScript with a typed array, and verify the result. Then check `sizeof(void*)` and `sizeof(long)` inside WASM, and find any code that breaks.

10. **API audit for bindability.** Take a public C header (yours, or `zlib.h`, `sqlite3.h`, or libcurl's). Score every function against the §30.6 table. Identify which ones would be painful to bind, and redesign the three worst.

Module 30 in Summary

C's ABI is the meeting point of every language, so a C library designed for FFI can be used from nearly anywhere. C++ needs an `extern "C"` guard to disable name mangling, a RAII wrapper to tie C objects to C++ lifetimes, and the trampoline pattern to pass capturing lambdas through `void *` user data — and exceptions must never propagate into C. Python can bind C through ctypes (no compiler, but every `argtypes` and `restype` must be declared), cffi (which parses real C declarations and can compile a fast extension), and faster options up to Cython and the CPython API. Crossing the boundary costs hundreds of nanoseconds per call from Python, which made per-element calls 47× slower than pure Python and a single batched call 13× faster — so C APIs should expose bulk operations.

Rust wraps C in thin `unsafe` layers with safe `Drop`-based APIs and must stop panics at the boundary; Go's cgo is expensive per call and forbids C from keeping Go pointers; Java is moving from JNI to Panama; Node uses the ABI-stable N-API; and Lua offers a clean stack-based embedding API. WebAssembly compiles C for browsers (Emscripten) and sandboxed runtimes (WASI), with 32-bit pointers and `long`.

Libraries meant to be bound should use fixed-width types, opaque handles, explicit lengths, caller-supplied buffers or matching free functions, status codes with out-parameters, callbacks with user data, and no varargs, macros, unions, or bitfields in the API — and ship a C++ guard, an export macro, version information, a `.pc` file, and documentation of ownership, lifetime, thread safety, and errors.

Next: Module 31 — Professional Practice: Git workflows for C (including `git bisect`, the single most underused debugging tool), coding standards and clang-format, Doxygen and architecture documentation, release engineering and semantic versioning, the licensing implications of C libraries, and how to read and contribute to a large, unfamiliar C codebase.

Module 31 — Professional Practice

In this module:

- Git: branching, rebasing, bisect (`git bisect` is gold for C bugs), code review
- Coding standards: Linux kernel style, GNU, Google, MISRA; clang-format, `.editorconfig`
- Documentation: Doxygen, README, man pages, architecture docs
- Semantic versioning, changelogs, release engineering
- Licensing: GPL/LGPL/MIT/BSD/Apache and the implications for C libraries
- Reading and contributing to large C codebases
- Debugging production issues, logging, telemetry, crash reporting

These are the skills that make you employable rather than merely capable. Knowing C is necessary; working effectively on a team, in a shared codebase, with releases, users, and production incidents, is what the job actually consists of. All output in this module is real, from git 2.43, clang-format 22, and GCC 13.3.

31.1 Git — and `git bisect` Especially

The workflow

Concept	Practice
Branching	Short-lived feature branches off `main`. Trunk-based development beats long-lived branches for C — merge conflicts in widely included headers are painful.
Rebase vs merge	Rebase your *local* work to keep history linear; never rebase a branch others have pulled. Make `git pull --rebase` the default.
Commit hygiene	One logical change per commit. It must build and pass tests. This is what makes `bisect` work.
Commit messages	A summary line of about 50 characters, a blank line, then a body explaining *why* (the diff already shows *what*). Reference the issue. Conventional Commits (`feat:`, `fix:`, `perf:`) enable automated changelogs.
Code review	Small pull requests (under about 400 lines). Review for correctness, not style — clang-format handles style.

Rebasing replays your commits on top of the latest upstream work, as though you had started from there, producing a straight-line history instead of a web of merge commits. Because it rewrites commits, it is only safe on work nobody else has based anything on.

⚠ `git bisect` — the highest-leverage debugging tool in C

A real experiment: a ring buffer with a regression buried four commits deep among eight.

```
$ git log --oneline
bebf5bf feat: add ring_space()
b3b76a0 feat: add ring_clear()
6d744a8 docs: expand README
2a36088 feat: add ring_cap()
a62be7e perf: replace modulo with a mask in the hot path
cdca2e1 feat: add ring_full()
16f9040 feat: add ring_empty()
a59ada0 docs: add README
a00c6c2 feat: initial ring buffer

$ make -s check
FAIL test.c:11 v == (uint8_t)(cycle*n+i)
```

The test script — the only thing you write:

```
#!/bin/sh
# Exit 0 = good, 1 = bad, 125 = SKIP (this commit cannot build)
make -s clean >/dev/null 2>&1
make -s test >/dev/null 2>&1 || exit 125
./test >/dev/null 2>&1
```

```
$ git bisect start HEAD a00c6c2
$ git bisect run ./bisect.sh

Bisecting: 3 revisions left to test after this (roughly 2 steps)
[a62be7e] perf: replace modulo with a mask in the hot path
running './bisect.sh'
Bisecting: 1 revision left to test after this (roughly 1 step)
[16f9040] feat: add ring_empty()
running './bisect.sh'
Bisecting: 0 revisions left to test after this (roughly 0 steps)
[cdca2e1] feat: add ring_full()
running './bisect.sh'

a62be7e9320820d1ada0eb954d1a8838bdbb9ed4 is the first bad commit
    perf: replace modulo with a mask in the hot path
```

You tell `git bisect` one commit that is known to be bad (`HEAD`) and one known to be good (the first commit). It checks out the commit halfway between them and runs your script; the script's exit status marks that commit good or bad; and bisect halves the remaining range again, until exactly one commit is left — the first bad one.

The culprit:

```
-    r->tail = (r->tail + 1) % r->cap;
+    r->tail = (r->tail + 1) & (r->cap - 1);
```

A "performance optimisation" that is valid only when `cap` is a power of two (Module 4 — and exactly the condition Module 29's ring buffer enforced with a `_Static_assert`). Three automated test runs found it precisely, with no debugger and no reading of code.

Why it scales

```
    8 commits ->  3 tests
  100 commits ->  7 tests
 1000 commits -> 10 tests
```

```
 10000 commits -> 14 tests
100000 commits -> 17 tests
```

It is binary search (Module 29): O(log n). A regression introduced 10,000 commits ago is found in 14 automated builds.

Bisect in practice

```
git bisect start
git bisect bad                  # current is broken
git bisect good v1.2.0          # this release worked
git bisect run ./test.sh        # fully automatic
git bisect reset

git bisect skip                 # this commit doesn't build (or use exit 125)
git bisect log > bisect.log     # save the session
git bisect replay bisect.log    # resume it
git bisect start -- src/parser.c # restrict to commits that touched one file
```

For C specifically: make the test script a sanitizer build (`-fsanitize=address,undefined`), and it will bisect memory bugs that do not even crash the plain build. That combination — bisect plus sanitizers — finds regressions that no amount of reading will.

Requirement: every commit must build and be testable. That is the practical argument for commit hygiene: a history full of half-finished “WIP” commits that don’t compile turns bisect into a manual slog of skips.

The other tools

```
git log -S"ring_push" --oneline   # commits that ADDED or REMOVED this string ("pickaxe")
git log -L :ring_push:ring.c      # the full history of ONE function
git blame -w -C -C ring.c         # ignore whitespace, follow code moved between files
git diff --word-diff              # better for prose and dense C
git worktree add ../bugfix        # a second checkout — no stashing
git rerere                        # remember conflict resolutions across rebases
```

`git blame` answers “who last touched this line?”; `git log -S` answers “when did this code appear or disappear?” — usually the more useful question, because the commit that introduced a line normally explains why it is there.

31.2 Coding Standards

The major C styles

Style	Indent	Braces	Notes
Linux kernel	8-wide tabs	K&R (function braces on their own line)	The 8 columns are deliberate — in the kernel documentation’s words, “if you need more than 3 levels of indentation, you’re screwed anyway”

Style	Indent	Braces	Notes
GNU	2 spaces, braces indented	On their own line, indented	`func (a, b)` with a space. Distinctive, and widely disliked outside GNU projects.
Google	2 spaces	K&R	Commonly `int* p` pointer binding
BSD/Allman	4/8	Braces on their own lines	
MISRA	—	Mandates braces, single exit, no recursion	Module 27; a safety standard, not a formatting one

The same file under four styles:

```
--- LLVM ---                          --- GNU ---
static int cmp(const void *a) {       static int
  const Rec *x = a;                   cmp (const void *a)
  ...                                 {
}                                       const Rec *x = a;

--- Google ---                        --- Mozilla ---
static int cmp(const void* a) {       static int
  ...                                 cmp(const void* a)
}                                     {
```

Note `int *p` versus `int* p` — Module 9 explained why the C convention is `int *p` (the `*` binds to the declarator, not the type). Google's C++-derived style says otherwise; both exist in the wild.

`.clang-format` — kernel style

```
BasedOnStyle: LLVM
IndentWidth: 8
UseTab: Always
TabWidth: 8
BreakBeforeBraces: Linux     # functions on a new line, everything else K&R
AllowShortIfStatementsOnASingleLine: false
AllowShortFunctionsOnASingleLine: None
IndentCaseLabels: false
ColumnLimit: 100
PointerAlignment: Right      # `int *p` (Module 9)
SortIncludes: false          # C include ORDER often matters (Module 26 FTMs!)
AlignConsecutiveMacros: true
```

```
static int cmp(const void *a, const void *b)
{
    const Rec *x = a, *y = b;
    return (x->score > y->score) - (x->score < y->score);
}
int process(Rec *recs, size_t n, int flags)
{
    if (!recs || n == 0)
        return -1;
```

> ⚠ **`SortIncludes: false` matters in C.** Feature-test macros must precede every include (Module 26), and some headers must come in a specific order. Automatic sorting silently breaks builds.

Enforce it in CI

```
$ clang-format --dry-run --Werror messy.c
messy.c:2:9: error: code should be clang-formatted [-Wclang-format-violations]
#include<stdlib.h>
        ^
(exit code 1 = would fail the build)
```

Add a pre-commit hook as well, and use `git-clang-format` to format only the lines you changed — so that adopting a formatter doesn't require touching every file at once.

> **The point of a formatter is to end style discussions.** Any consistent style beats an inconsistent one. Pick one, commit `.clang-format`, and never review whitespace again.
>
> ⚠ If you do reformat the whole codebase, do it in one dedicated "reformat everything" commit, and add that commit's hash to `.git-blame-ignore-revs`, so that `git blame` skips it and still shows the real authors.

`.editorconfig` — what every editor understands

```
root = true

[*]
end_of_line = lf
insert_final_newline = true
trim_trailing_whitespace = true

[*.{c,h}]
indent_style = tab
indent_size = 8
max_line_length = 100

[{Makefile,*.mk}]
indent_style = tab   # Make REQUIRES real tabs (Module 20)
```

It is universally supported, needs no plugin in most editors, and prevents the "my editor converted all your tabs to spaces" pull request.

31.3 Documentation

The four kinds, and who reads them

Document	Audience	Answers
README	Someone deciding whether to use it	What is it, how do I build it, show me 10 lines that work
API reference (Doxygen)	Someone writing code against it	What does this function do, and what are the error returns?

Document	Audience	Answers
Architecture doc	Someone changing it	Why is it structured this way, and what were the alternatives?
man page	Someone running the tool	Synopsis, options, exit codes, files, examples

A README structure that works

```
# projectname
One sentence: what it does and for whom.

## Quick start
    git clone ... && cd ... && make && ./example
[10 lines of code that actually work]

## Installation    # package managers, then from source
## Documentation   # link to the full reference
## Requirements    # C standard, compilers, platforms, dependencies
## Contributing    # link to CONTRIBUTING.md
## License         # SPDX identifier + link
```

The single most common README failure is not showing working code on the first screen.

Doxygen

```
/**
 * @brief Render the widget onto a canvas.
 *
 * @param[in]  w       Widget. Must be non-NULL and initialized.
 * @param[in]  canvas  Not modified. Must outlive this call.
 * @param[out] bounds  On success, receives the bounds. Untouched on failure.
 *
 * @return ::ERR_OK on success.
 * @retval ERR_INVAL A required argument was NULL.
 * @retval ERR_NOMEM Allocation failed.
 *
 * @note Thread safety: NOT thread-safe. Callers must serialize access to @p w.
 * @warning Must not be called from a widget callback.
 * @see widget_measure()
 *
 * @code
 * Rect r;
 * if (widget_render(w, canvas, &r) == ERR_OK) { ... }
 * @endcode
 */
[[nodiscard]] err_t widget_render(Widget *w, const Canvas *canvas, Rect *bounds);
```

Doxygen reads specially formatted comments like this one and generates browsable reference documentation. The four things every public function must document (Module 28): ownership, lifetime, thread safety, and every error return. Binding authors and callers cannot read your implementation.

`Doxyfile` essentials: `EXTRACT_ALL=NO` (so that undocumented functions show up as gaps rather than being silently included), `WARN_IF_UNDOCUMENTED=YES`, and `WARN_AS_ERROR=YES` in CI. Alternatives: Sphinx with Breathe, Natural Docs, or simply clean headers with good comments.

Architecture documentation

ADRs (Architecture Decision Records) — one short Markdown file per significant decision:

```
# ADR-0007: Use an arena allocator for per-request memory

## Status
Accepted (2026-03-14)

## Context
Profiling showed malloc/free at 18% of request-handling time...

## Decision
Per-request arena, reset at the end of each request.

## Consequences
+ 14x faster allocation (measured)
+ Zero leak risk within a request
- Cannot free individual objects
- ASan cannot instrument arena allocations without manual poisoning
```

This is the documentation people actually miss. Six months later nobody remembers *why* something was done — and someone "fixes" it back. A `docs/adr/` directory costs 20 minutes per decision and saves days. (The consequences listed here are Module 10's measured arena trade-offs.)

Also worth having: a `docs/ARCHITECTURE.md` with a module diagram and the dependency direction (Module 28), and a `CONTRIBUTING.md` with build, test, and review expectations.

man pages

```
.TH MYTOOL 1 "2026-07-29" "mytool 1.4.2"
.SH NAME
mytool \- process widget files
.SH SYNOPSIS
.B mytool
[\fB\-v\fR] [\fB\-o\fR \fIFILE\fR] \fIINPUT\fR...
.SH EXIT STATUS
.TP
0 Success
.TP
1 Invalid arguments
```

Write them in Markdown and convert with pandoc or scdoc — raw *roff*, the typesetting language man pages use, is miserable to write by hand. And document the exit codes (Module 1): scripts depend on them.

31.4 Versioning and Release Engineering

Semantic versioning for C

`MAJOR.MINOR.PATCH`:

Bump	When
MAJOR	An ABI break or API removal (Module 20 measured what an ABI break does)
MINOR	New API, backward compatible
PATCH	Bug fixes only

For C libraries, the ABI is the contract, not just the API. Bump the soname on every MAJOR release (Module 20):

```
libfoo.so    -> libfoo.so.2        (dev symlink)
libfoo.so.2  -> libfoo.so.2.1.0    (SONAME — what binaries record)
```

Verify with `abidiff` (from libabigail) in CI — it compares two builds and reports ABI differences automatically, catching the "harmless" structure change that would silently corrupt every existing program.

Expose the version three ways

```c
#define FOO_VERSION_MAJOR 1
#define FOO_VERSION_MINOR 4
#define FOO_VERSION_PATCH 2
#define FOO_VERSION_NUM ((1*10000) + (4*100) + 2)   /* for #if comparisons */
#define FOO_VERSION_STR "1.4.2"

const char *foo_version(void);   /* RUNTIME — may differ from compile time! */
int foo_version_num(void);
```

The macros tell code which version it was *compiled* against, and support `#if FOO_VERSION_NUM >= 10400` checks; the functions report the version actually *loaded* at run time. The two can differ — that is the whole point of shared libraries, whose files can be upgraded without recompiling their users. Check both: a program can refuse to start, or disable a feature, when the loaded library is older than the one it was built for.

CHANGELOG

Follow keepachangelog.com: sections for Added / Changed / Deprecated / Removed / Fixed / Security, newest release first, with an `[Unreleased]` section at the top.

```
## [1.4.2] - 2026-07-29
### Fixed
- Ring buffer corruption with non-power-of-two capacities (#142)
### Security
- Integer overflow in `foo_alloc_array()` could undersize the allocation (CVE-2026-XXXX)
```

Write it for users, not by pasting `git log`. "Fixed ring buffer corruption" is useful to a user; "perf: replace modulo with a mask" is not. (Note that both entries here are bugs from earlier in this book: §31.1's bisected regression, and Module 23's overflow chain.)

Conventional Commits plus tools such as git-cliff or semantic-release can generate the skeleton, but a human must edit it.

Release checklist

```
[ ] CHANGELOG updated, version bumped everywhere (header, build files, .pc, spec)
[ ] Full CI green: all compilers, all platforms, all sanitizers, Valgrind, static analysis
[ ] Fuzz corpus replayed clean (Module 22)
[ ] abidiff clean, or the soname bumped
[ ] Documentation regenerated; examples still compile
[ ] Reproducible build verified (no __DATE__/__TIME__ — Module 12)
[ ] Signed tag: git tag -s v1.4.2 -m "Release 1.4.2"
```

```
[ ] Release tarball + signature + SHA256SUMS
[ ] Debug symbols archived (Module 19's split debug info)
```

Archive the unstripped binary or `.debug` file with every release — without it, production crash addresses cannot be decoded (§31.7). A *signed* tag and published checksums let users verify that what they downloaded is what you released, and a reproducible build lets them verify that the binary really came from the tagged source.

31.5 Licensing

> **This is not legal advice.** Licence choice has real legal consequences — consult counsel for anything that matters commercially. What follows is the factual landscape as it is generally understood.

Licence	Type	Effect on your code
MIT	Permissive	Use freely; keep the copyright notice
BSD-2/3-Clause	Permissive	The same; 3-clause adds a no-endorsement clause
Apache-2.0	Permissive	The same, plus an explicit patent grant and a patent-retaliation clause
ISC	Permissive	MIT-equivalent, shorter
LGPL-2.1/3.0	Weak copyleft	Changes to the library itself must be shared; dynamic linking generally lets your application stay proprietary
GPL-2.0/3.0	Strong copyleft	Linking generally makes the whole work subject to the GPL when distributed
AGPL-3.0	Network copyleft	The GPL, plus obligations triggered by offering the software over a network, not just by distribution
MPL-2.0	File-level copyleft	Modified files stay MPL; the rest of your project doesn’t
Public domain / CC0 / Unlicense	None	⚠ Public-domain dedication is not recognised in every jurisdiction; some companies ban it

Copyleft licences require that works built on the licensed code be distributed under the same licence; *permissive* licences only require attribution.

What actually matters for C

Static linking is the sharp edge. With the GPL or LGPL, statically linking a library generally incorporates it into your binary in a way that triggers the licence's conditions. The LGPL specifically contemplates dynamic linking as the way to keep an application separate; statically linking an LGPL library imposes additional obligations — typically providing object files, so that users can relink your program against a modified version of the library.

This is why:

- **glibc is LGPL,** so proprietary Linux programs can link against it dynamically.
- **The Linux kernel is GPL-2.0 with an explicit system-call exception,** so ordinary user-space programs are not derived works of the kernel.
- **musl is MIT-licensed,** which is one reason it is popular for fully static binaries and minimal containers.

Many C libraries are *dual-licensed* — GPL for open-source use, and a commercial licence for proprietary use; this was Qt's historical model.

Practical guidance:

- **A library you want widely adopted:** MIT, BSD, or Apache-2.0 — Apache-2.0 if patents are a concern.
- **A library you want to stay open:** the LGPL (allowing dynamic linking) or the GPL.
- **An application:** whatever you like.
- **Corporate use:** most companies maintain a list of approved licences. Check it *before* adding a dependency, not after.

Compliance hygiene

```
// SPDX-License-Identifier: Apache-2.0
// Copyright (c) 2026 Example Corp
```

SPDX identifiers are short, machine-readable licence tags, and are now standard practice — the Linux kernel uses them in every file. Add them to every file you write.

Also: ship the full `LICENSE` file; maintain a `NOTICE` or `THIRD_PARTY_LICENSES` file listing your dependencies; scan with tools such as FOSSA, ScanCode, licensecheck, or GitHub's dependency graph; and ⚠ **check transitive dependencies** — a permissively licensed library that vendors a GPL component passes the GPL obligation on to you.

31.6 Reading a Large C Codebase

You will spend far more time reading C than writing it. Here is a repeatable method.

1. Orient before reading

```
cloc .                  # size and language breakdown
ls -la                  # README, CONTRIBUTING, docs/, ARCHITECTURE.md
cat README.md CONTRIBUTING.md docs/*.md
git log --oneline | wc -l          # project age and activity
git shortlog -sn | head            # who to ask
find . -name '*.h' | head -30      # the API surface lives in headers
```

Read the headers first. They are the contract (Module 28). A 200,000-line project might have 3,000 lines of public headers — and that is the part you must understand first.

2. Follow the entry point

```
grep -rn "int main" --include=*.c
```

Trace `main` → initialisation → the main loop. Don't read breadth-first, file by file; follow one execution path all the way through, depth-first.

3. Build it, and instrument it

```
make   # or: cmake -B build && cmake --build build
gdb -q -batch -ex 'break main' -ex run -ex bt ./prog
strace -c ./prog   # what does it ACTUALLY do? (Module 21)
ltrace ./prog
```

Running it under a debugger and a tracer teaches you more in 10 minutes than an hour of reading. A breakpoint on a function you are curious about, followed by `bt`, shows you exactly how execution reached it.

4. Navigate mechanically

```
ctags -R .        # then Ctrl-] in vim
cscope -Rbq       # "who calls this?" — the killer feature
bear -- make      # generate compile_commands.json
# then clangd in your editor: real go-to-definition, find-references, rename
```

Set up `compile_commands.json` and clangd first. Grep-based navigation in C is unreliable because of macros and the preprocessor (Module 12) — the same name can mean different things under different `#ifdef`s. clangd understands the actual translation units, with the actual flags (Module 20).

5. Read the history, not just the code

```
git log --oneline -20 -- src/parser.c   # what's been changing here?
git log -S"suspicious_function"         # when did this appear?
git log -L :parse_header:parser.c       # this function's whole history
git blame -w -C src/parser.c
```

The commit that introduced a strange line usually explains it — often it is a bug fix for a case you would never have thought of.

6. Change something small

Fix a typo. Add a test. Improve an error message. Submitting one trivial pull request teaches you the build system, the test suite, the review process, and the maintainers' expectations — all the things documentation does not tell you.

Codebases worth reading

Project	What it teaches
SQLite	Testing discipline (100% MC/DC coverage), a single-file "amalgamation" build
Redis	Clean modern C, excellent data structures, a readable event loop
curl	Portability across everything, protocol implementation
musl libc	A minimal, standards-exact, extremely readable standard library
Lua	Small, elegant, embeddable; one of the best C API designs in existence
nginx	Event-driven architecture, memory pools
Linux kernel	Subsystem design, intrusive data structures, the `goto` cleanup pattern at scale
FFmpeg	Performance work, SIMD
stb single-header libraries	API design under extreme constraints
zstd / lz4	Optimisation craft

31.7 Production Debugging, Logging, Telemetry, Crash Reporting

In production you cannot attach a debugger, add a `printf`, or reproduce the problem at will. Everything you will need must be built in beforehand.

Structured logging

```c
void log_emit(LogLevel lvl, const char *file, int line, const char *func,
              const char *fmt, ...) __attribute__((format(printf, 5, 6)));

#define LOG_AT(lvl, ...) log_emit((lvl), __FILE__, __LINE__, __func__, __VA_ARGS__)
#define LOGI(...) LOG_AT(LOG_INFO, __VA_ARGS__)
```

Human-readable:

```
2026-07-29T16:16:56.349Z INFO  [-]          app.c:52 main(): service starting version=1.4.2 pid=937
2026-07-29T16:16:56.349Z DEBUG [req-a1b2c3] app.c:36 handle_request(): request received size=512
2026-07-29T16:16:56.349Z WARN  [req-d4e5f6] app.c:38 handle_request(): negative size, rejecting
2026-07-29T16:16:56.349Z ERROR [req-g7h8i9] app.c:39 handle_request(): size 99999 exceeds limit 1024
```

JSON for log aggregation systems (Loki, ELK, Datadog):

```
{"ts":"2026-07-29T16:16:56.352Z","level":"DEBUG","trace":"req-a1b2c3",
 "src":"app.c:36","fn":"handle_request","msg":"request received size=512"}
```

Every technique from earlier modules is in there:

Element	Module
`__FILE__`, `__LINE__`, `__func__`	12
`__attribute__((format(printf,...)))`	6 — catches format bugs at compile time
`vsnprintf` into a bounded buffer	8
`_Thread_local` trace ID	3, 15 — like `errno`, per thread
`gmtime_r`, not `gmtime`	14 — thread safety
`setvbuf(stderr, NULL, _IOLBF, 0)`	13 — so logs survive a crash
Early return when a level is filtered out	Near-zero cost for disabled levels

Logging rules:

1. **Use levels, and honour them.** A `LOGD` in a hot loop must cost about one comparison when debug logging is off.
2. **Structured, not prose.** `size=99999 limit=1024`, not "the size was too big" — machines can search and aggregate key=value pairs.
3. **Correlation IDs.** A trace ID threaded through every log line of one request is what makes debugging across threads and services possible.
4. **Never log secrets.** Passwords, tokens, keys, and personal data — logs are among the most commonly leaked artefacts.
5. **`stderr` for logs, `stdout` for data** (Module 13).
6. **Bounded.** Rate-limit, rotate, and cap. A log-flooding bug can fill the disk and take the whole machine down.
7. **Line-buffered or unbuffered,** so that the last lines before a crash survive.

Metrics

```
--- metrics ---
  requests_total      3
  bytes_processed   512
  requests_rejected   1
  requests_failed     1
```

Counters are the cheapest telemetry there is — an increment — and they answer "how often does this actually happen in production?" Export them over an HTTP endpoint (in Prometheus format) or push them periodically. (In a multithreaded program, they must be atomic — Module 17 — and ideally padded against false sharing — Module 25.)

The standard types: **counters** (monotonically increasing), **gauges** (a current value, such as queue depth), and **histograms** (latency distributions — always report percentiles, such as p50, p95, and p99, never averages, for the same reason Module 25 reported medians rather than means).

Crash reporting

```c
static void crash_handler(int sig)
{
    void *fr[32];
    int n = backtrace(fr, 32);
    /* async-signal-safe only: write(), _exit() (Module 24) */
    write(STDERR_FILENO, "\n*** FATAL SIGNAL - backtrace ***\n", 34);
    backtrace_symbols_fd(fr, n, STDERR_FILENO);
    _exit(128 + sig);
}

static void install_crash_handlers(void)
{
    static char altstack[64*1024];
    stack_t ss = { .ss_sp = altstack, .ss_size = sizeof altstack };
    sigaltstack(&ss, NULL);   /* survive a STACK OVERFLOW */
    struct sigaction sa = {0};
    sa.sa_handler = crash_handler;
    sa.sa_flags = SA_ONSTACK | SA_RESETHAND;
    sigaction(SIGSEGV,&sa,NULL); sigaction(SIGBUS,&sa,NULL);
    sigaction(SIGFPE,&sa,NULL);  sigaction(SIGILL,&sa,NULL);
    sigaction(SIGABRT,&sa,NULL);
}
```

```
*** FATAL SIGNAL - backtrace ***
./app(+0x195d)[0x5555b7f8095d]
/lib/x86_64-linux-gnu/libc.so.6(+0x45330)[0x7fdf3e445330]
./app(main+0x11c)[0x5555b7f80d62]
```

And decoding it (Module 19):

```
$ addr2line -e app -f -C -i 0x195d
crash_handler
/home/claude/c_prof/obs/app.c:14
```

(The first frame is the handler itself; the frame after the `libc` entry — the signal trampoline — is where the crash actually happened.)

Three details make this work:

- **`SA_ONSTACK` with an alternate stack** — otherwise a stack overflow leaves no stack on which to run the handler (Module 24).
- **`backtrace_symbols_fd`, not `backtrace_symbols`** — the latter calls `malloc`, which is not async-signal-safe. (As Module 21 noted, `backtrace` itself is not formally async-signal-safe either; calling it once at start-up avoids the lazy library load that makes it risky. This is a pragmatic last-gasp handler.)
- **`-rdynamic` at build time,** so that function names appear.

`SA_RESETHAND` restores the default action after the handler has run once, so that a crash *inside* the handler terminates the process rather than looping.

> ⚠ With PIE, subtract the load base (from `/proc/self/maps`) before calling `addr2line`.

For real production use: Google's Breakpad or Crashpad, Sentry, or `systemd-coredump` with `coredumpctl`. Plus: enable core dumps (`RLIMIT_CORE`), archive the split debug file with every release (Module 19), and run a symbol server so that crash reports can be symbolised automatically.

The production debugging kit

Situation	Tool
The process is hung	`gdb -p PID -batch -ex 'thread apply all bt'`
What is it doing?	`strace -p PID -c`, `perf top -p PID`
Is memory growing?	`/proc/PID/status`, `pmap`, heaptrack, massif
You need data from a live process	bpftrace — attach with near-zero overhead, no restart
A crash, after the fact	Core dump + `addr2line` + the archived debug file
Non-deterministic	`rr` record/replay (Module 21)

> **Build in observability before you need it.** A production incident is not the time to discover that you have no logs, no metrics, and no symbols.

Module 31 Rules

#	Rule
1	Every commit builds and passes its tests — that is what makes bisect work
2	`git bisect run` with a sanitizer build finds regressions in O(log n)
3	`git log -S` answers “when did this change?” better than `blame`
4	One `.clang-format`, enforced in CI; `SortIncludes: false` in C
5	Ignore the reformatting commit via `.git-blame-ignore-revs`
6	The README shows working code on its first screen
7	Document ownership, lifetime, thread safety, and every error return
8	Write ADRs — the *why* is what gets lost
9	For C libraries the ABI is the contract; bump the soname, verify with `abidiff`
10	Expose the version at compile time and at run time
11	Static linking is the sharp edge of the GPL/LGPL — check licences before adding a dependency
12	Add SPDX identifiers to every file
13	Read the headers first, follow one path, set up clangd, then change something small

#	Rule
14	Logs: levelled, structured, correlated, bounded, on `stderr`, never containing secrets
15	`backtrace_symbols_fd` + `SA_ONSTACK` + `-rdynamic` for crash handlers
16	Archive the debug file with every release — without it, crashes are undecodable
17	Build in observability before you need it

Practice

1. **Bisect a real regression.** Reproduce §31.1: create a repository with 10 commits, one of which contains a subtle bug, and find it with `git bisect run`. Then make the test script a sanitizer build, plant a memory bug that does not crash — and confirm that bisect still finds it.

2. **Bisect an actual project.** Clone a large open-source C project, pick a behaviour, and bisect to find when it was introduced. Time yourself. Then try to find the same commit by reading, and compare.

3. **Formatting roll-out.** Take an inconsistently formatted project. Write a `.clang-format`, apply it in one commit, add that commit's hash to `.git-blame-ignore-revs`, add a CI check, and add a pre-commit hook. Verify that `git blame` still shows the real authors.

4. **Document a module.** Take a header you have written. Add complete Doxygen comments for every public function, covering the four required elements. Generate the HTML. Then hand the header — only the header — to someone else, and see whether they can use the API correctly.

5. **Write three ADRs.** For three design decisions in a project you have built, write ADRs with Status, Context, Decision, and Consequences sections. Include the alternatives you rejected, and why.

6. **Release a library.** Take something you have built and do a full release: version macros plus a run-time version function, a soname, a `.pc` file, a CHANGELOG, a signed tag, a tarball with `SHA256SUMS`, and a split debug file. Then make an ABI-breaking change, verify that `abidiff` catches it, and do a MAJOR release.

7. **Licence audit.** For a project with 5 or more dependencies, list every direct and transitive dependency with its licence. Identify anything that would prevent proprietary distribution. Add SPDX headers and a `THIRD_PARTY_LICENSES` file.

8. **Read an unfamiliar codebase.** Pick one from the §31.6 table. Spend two hours: run `cloc`, read the headers, find `main`, trace one execution path in GDB, run it under `strace`, and set up `compile_commands.json` and clangd. Then write a one-page summary of its architecture. Finally, submit one small pull request.

9. **Build the observability layer.** Implement the §31.7 logger, with levels, JSON output, thread-local trace IDs, and the format attribute. Add counters, gauges, and a latency histogram with p50, p95, and p99. Verify with TSan that it is thread-safe.

10. **Crash-to-line pipeline.** Build a program with a crash handler, split debug information (Module 19), and PIE. Crash it, capture the backtrace, and write a script that computes the load base from `/proc/PID/maps` and pipes the adjusted addresses to `addr2line`. Confirm that you get exact file and line numbers from a stripped binary.

11. **Production drill.** Take a running program and, using only `gdb -p`, `strace -p`, `perf top -p`, and `/proc`, determine — without stopping it — which functions are hot, what system calls it is making, how much memory it is using, and where each thread is blocked.

Module 31 in Summary

Professional C work runs on disciplined version control. Short-lived branches, rebased local history, and small, reviewed commits that each build and pass tests make `git bisect run` possible — a binary search through history that found a regression among eight commits in three automated runs, would need only 14 among ten thousand, and, combined with a sanitizer build, finds memory bugs that never even crash. `git log -S` and `git log -L` explain *when* and *why* code changed.

A single `.clang-format` (with include sorting off, because order matters in C), enforced in CI and complemented by `.editorconfig`, ends style debates. Documentation serves four audiences — a README with working code up front, Doxygen reference covering ownership, lifetime, thread safety, and errors, ADRs recording the reasons behind decisions, and man pages with exit codes. Releases follow semantic versioning in which the ABI is the contract, expose compile-time and run-time versions, keep a user-facing changelog, and ship signed, checksummed, reproducible artefacts with archived debug symbols. Licences matter most at static-linking boundaries; permissive licences maximise adoption, copyleft keeps code open, and SPDX tags and dependency audits keep projects compliant.

To learn an unfamiliar codebase, read the headers, follow one execution path under a debugger and tracer, navigate with clangd, read the history, and submit something small. And in production, where no debugger can be attached, rely on what was built in beforehand: levelled, structured, correlated logs that never contain secrets; counters and latency percentiles; crash handlers that print backtraces from an alternate stack; core dumps; and archived symbols that turn raw addresses back into file and line numbers.

That completes Part 2 — the industry toolchain (Modules 19–31).

One module remains from the original roadmap: **Module 32 — Study These Real Codebases**, a guided reading plan covering what to look at in each project, in what order, and which specific technique each one teaches.